

NumPy, Pandas, —— and —— **Scikit-learn** **Masterclass**

Applied data wrangling and machine learning projects with Python



Gary Hutson



NumPy, Pandas, — and — **Scikit-learn** **Masterclass**

Applied data wrangling and machine learning projects with Python



Gary Hutson



NumPy, Pandas, and Scikit-learn Masterclass

*Applied data wrangling and machine learning projects
with Python*

Gary Hutson



www.bpbonline.com

First Edition 2026

Copyright © BPB Publications, India

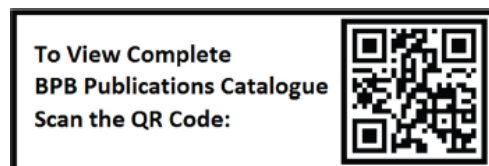
ISBN: 978-93-65898-293

All Rights Reserved. No part of this publication may be reproduced, distributed or transmitted in any form or by any means or stored in a database or retrieval system, without the prior written permission of the publisher with the exception to the program listings which may be entered, stored and executed in a computer system, but they can not be reproduced by the means of publication, photocopy, recording, or by any electronic and mechanical means.

LIMITS OF LIABILITY AND DISCLAIMER OF WARRANTY

The information contained in this book is true to correct and the best of author's and publisher's knowledge. The author has made every effort to ensure the accuracy of these publications, but publisher cannot be held responsible for any loss or damage arising from any information in this book.

All trademarks referred to in the book are acknowledged as properties of their respective owners but BPB Publications cannot guarantee the accuracy of this information.



www.bpbonline.com

Dedicated to

*To **Kerry** and **Charlie**, for always being my rocks.*

*To the colleagues, past and present, who spurred me
on to complete these pages.*

*And to my parents, on both sides, for everything that
made this possible.*

About the Author

Gary Hutson is a machine learning specialist and author whose work bridges research, engineering and applied innovation. He began his career as an intelligence analyst, applying statistics and modeling to solve complex crime and justice problems, and went on to become head of machine learning, leading teams and developing AI systems across healthcare, public services, housing insights, private consulting and defense. More recently, he has worked as a senior generative AI engineer, building retrieval augmented generation (RAG) systems for defense and now applies his expertise as a data scientist, using automation to optimize the pharmaceutical supply industry. His technical interests span natural language processing, computer vision, predictive modeling and generative AI, with a focus on making AI system scalable, and impactful. He has co-authored a book and has created a wide range of R and Python packages – including OddsPlotty, ConfusionTableR, MLDataR, modelviz and Combined Alpha-weighted Random Forest Layered Inference Ensemble (CHARLIE) a model built in PyTorch, that are used by ML engineers and data scientists worldwide. With professional certification as a Google certified machine learning engineer and TensorFlow developer, along with a degree in applied business statistics and a master's in business and finance. Additionally, he is an active blogger on Hutsons-hacks and likes to compete on Kaggle. In his spare time, he is a father and husband, coach for ATFA Brinsley Falcons football team, fan of Nottingham Forest football club and active member in data science communities.

About the Reviewers

- ❖ **Rabia Günaydı is a passionate professional working in the fields of data analysis and software. She holds a master's degree in mathematics and has specialized in developing and analyzing data-driven solutions. Currently, she works as a software assistant specialist at LC Waikiki. She actively works with SQL Server, Python, and reporting tools, and has experience in process improvement, data visualization, and optimization. In the future, she aims to deepen her expertise in data science and artificial intelligence, and to lead impactful projects in these fields.**
- ❖ **Tomislav Dukez is a full stack software developer with a diploma in full stack software development, specializing in predictive analytics from Code Institute, Dublin, and data science training from the London School of Informatics. Drawing on 10+ years across university law lecturing, diplomacy, and public administration, alongside a Ph.D. in Canon Law, he brings clear communication and practical problem solving to technical challenges. He works primarily in Python and JavaScript (Django, Flask, FastAPI, Node.js/Express, React, Next.js), applying data analytics and machine learning to web applications. Currently freelancing as a full stack engineer and coding tutor, Tomislav teaches Python and front end/back end fundamentals. His focus centers on back end engineering and developer education, helping teams and individuals build reliable, well architected solutions. He is a passionate Pythonista and Linux enthusiast.**

- ❖ **Maicon Melo Alves is a senior system analyst working with high performance computing (HPC) systems at Petrobras, the Brazilian energy state company. He has over 25 years of experience in IT infrastructure, four published books, and over ten papers published in journals and conferences of HPC area. He obtained his DSc. degree in computer science from the Fluminense Federal University (UFF) and received his MSc. degree in computer science from Rio de Janeiro Federal University (UFRJ). He has earned the best paper award in 2015 and received in 2019 an honorable mention for his DSc. thesis in WSCAD, the foremost important HPC conference in Brazil. With the advent of AI workloads, he got interested in understanding how HPC systems have been used to leverage the training process of these applications. To better understand this topic, he completed in 2021 the MBA in data science of Pontifícia Universidade Católica of Rio de Janeiro (PUC-RIO). Although working as a system analyst is his primary occupation, he also dedicates part of his time to coordinate and teach on graduate and undergraduate courses related to HPC, computer architecture, and IT infrastructure. His research interests include high performance computing, machine learning, and artificial intelligence.**

Acknowledgement

I would like to express my sincere gratitude to all those who contributed to the completion of this book.

To my family and friends, thank you for your patience, understanding, and constant encouragement. Your belief in me has been grounding and motivating, and this book would not exist without your love and support. A special shout out to Charlie for keeping me smiling through the long writing sessions, to Kerry for your endless encouragement, and to my friends and colleagues, who gave me the drive to finish this book. To my football team ATFA Brinsley Falcons, thank you for giving me another outlet, as well as reminding me that teamwork and persistence matter both on and off the pitch.

I am especially grateful to the editorial team, whose thoughtful feedback and guidance helped shape this manuscript into its best form, and whose vision and initial belief in this project set everything in motion.

My thanks also go to BPB Publications for their guidance and expertise in bringing this book to fruition. Their support was invaluable in navigating the complexities of the publishing process.

Finally, to the readers - thank you for your interest, curiosity, and trust. It is for you that this book was written, and it is your engagement that makes this journey meaningful.

Preface

In today's data-driven world, mastering the ability to clean, analyze and model data is an increasingly critical skill. *NumPy, Pandas, and Scikit-learn Masterclass* equip you with practical knowledge to build, optimize and deploy production-ready machine learning solutions using Python's most powerful, and current, data science libraries.

From data wrangling with Pandas and NumPy to building machine learning pipelines with scikit-learn, this book provides the reader a complete journey – starting from working with raw and unpolished datasets, to various modeling strategies to then putting the model to work, by using production grade ML models and concepts. This is achieved through real-world case studies in health, housing and finance to step-by-step tutorials and hands on projects. With this book, you will learn not only how to apply the concepts but also how to integrate them into practical business and research workflows.

Whether your use case is predicting disease, segmenting customers, forecasting house prices, or deploying ML applications with Docker and FastAPI, this book serves as your comprehensive guide to applied machine learning.

Chapter 1: Overview of NumPy and Pandas - Introduces the aspiring data scientist or machine learning developer to the essential Python data-handling libraries. In this chapter, we cover how to create arrays, how to index items in an array, how to perform operations over arrays, reshaping arrays to work with different dimensions and a dive into linear algebra with NumPy. We then move on to working with Pandas to perform common data engineering patterns, such as working with DataFrames and Series, filtering DataFrames to return a subset, grouping and aggregating for statistical

breakdowns and table views and then we move into working with time series data, looking at time series imputation.

Chapter 2: Introduction to Scikit-learn for Machine Learning - This chapter provides a practical introduction to scikit-learn, one of the most widely used libraries for machine learning in Python. Readers will begin with its structure and key features, learning how to load built-in datasets and preprocess data through encoding, scaling and splitting. The concepts of train-test-split and cross-validation are explained as the foundation of reliable model evaluation. Next, the chapter looks at core families of algorithms supported by scikit-learn, including popular algorithms across machine learning tasks regression, clustering and classification, giving readers a roadmap of what is to come in the later chapters. Model evaluation metrics, such as accuracy, precision, recall and F1 score are introduced, along with examples demonstrating how to interpret them. The chapter concludes with a practical example of building a breast cancer binary prediction model pipeline, tying together the key stages in the machine learning process of data preprocessing, model training and model evaluation, into a reproducible workflow.

Chapter 3: Supervised Binary Classification - This chapter dives into the first major machine learning application: binary classification. Using a medical dataset where the aim is to predict if a patient has thyroid disease, the chapter walks through preprocessing steps, such as handling missing values and conducting exploratory data analysis (EDA) to understand patterns in the data. Readers will experiment with fitting multiple models to the same task, comparing their performance and look at improving the models further with hyperparameter tuning. Special attention is given to the real-world problem of imbalanced datasets and awareness of approaches like SMOTE and ROSE, ensuring that minority classes are not overlooked. Evaluation methods such as the confusion matrix, ROC curves, and AUC scores are applied to measure model effectiveness. This paves the way for the reader to apply these skills to real world classification problems, and

will definitely aid in many industries, especially healthcare, finance and fraud detection, to name a few.

Chapter 4: Supervised Multi-class Classification - This chapter builds on the foundations of binary classification and extends them to multi-class scenarios. It introduces strategies such as One-vs-Rest and One-vs-One, as well as classifiers capable of handling multiple classes directly, including logistic regression, support vector machines, and random forests. Readers will learn how to prepare datasets that contain both categorical and numerical features, explore how to undertake one-hot encoding and how to scale your data effectively, depending on the scaling challenge. Evaluation metrics for multi-class classification are covered in detail, including the differences between macro, micro and weighted averages. The chapter finishes on a practical use case of predicting customer churn across multiple churn risk categories, where readers (ML developers at this point) apply learned techniques to analyze customer behavior and identify actionable business insights.

Chapter 5: Customer Segmentation with Unsupervised Methods - In this chapter the reader is introduced to unsupervised machine learning, focusing on clustering techniques that discover hidden patterns in unlabeled data. This chapter explores algorithms such as K-means and hierarchical clustering, and high dimensionality clustering methods, such as DBSCAN, while learning when and why to use each approach. Practical techniques and heuristics are introduced to help the reader determine the optimal number of clusters, which includes the elbow method and silhouette score. Moving on from the theory and toy examples, we move to a practical use case aiming to segment retail customers based on their purchase history, showing how clustering results can be interpreted and applied in real businesses.

Chapter 6: House Price Estimation with Regression Methods - The core focus of this chapter is regression analysis in depth, beginning with simple and multiple linear regression. Readers will learn preprocessing steps specific to regression problems, including normalization, feature

engineering and outlier handling. Advanced techniques such as feature selection and recursive feature elimination are introduced, followed by gaining an understanding of regularization methods like Lasso, Ridge and Elastic Net regression. The chapter also explores non-linear regression models, including random forests and gradient boosting, providing a toolkit for complex prediction problems. Model evaluation metrics such as mean absolute error (MAE) and root mean squared error (RMSE), alongside others, are introduced. This chapter culminates in a case study of predicting house prices, which serves to tie everything you have learned in this chapter together, and to demonstrate how regression can be used in the wild to solve real world business challenges.

Chapter 7: Handwritten Digits Dimensionality Reduction - This chapter focuses on dimensionality reduction, a vital step when working with high-dimensional data. Readers are introduced to the motivations for reducing dimensions, such as visualization (in 2 dimensions), noise reduction, and computational efficiency. Techniques included in this chapter range from linear dimensionality reduction – principal components analysis (PCA) and non-linear methods, such as t-SNE and UMAP for uncovering hidden structures in non-linear data. The chapter also highlights how dimensionality reductions aid in feature extraction and visualization of complex datasets. Using the MNIST handwritten digits dataset, readers apply these techniques to reduce dimensions and visualize the data in two and three dimensions, making patterns more interpretable. By the end of this chapter, the reader is well equipped to find hidden patterns in high-dimensional data and will have the tools and understanding to apply the methods learned to their use cases.

Chapter 8: Time Series with Scikit-learn - Time series data presents unique challenges, and this chapter addresses them head-on. In this chapter you will learn the essentials of handling temporal data structures, including methods like resampling, time series imputation and decomposition of a time series. In addition, this chapter focuses on methods for splitting time series datasets, such as rolling and expanding time series windows, and

introduces time series cross validation strategies. Once the data is in the shape required, the challenge then becomes modeling the time series and making accurate forecasts. This chapter explores many methods and approaches to achieve this, such as ARIMA. Further, we look at practical preprocessing steps such as lag feature creations and seasonality adjustments. This chapter will introduce the reader to specialized tools, such as statsmodels and prophet, to equip the reader with the skills to make accurate forecasts for any one of their chosen temporal datasets.

Chapter 9: Model Improvement Strategies - Once we have a model, we need to know strategies for how to improve the model further. This chapter begins with hyperparameter tuning techniques such as GridSearchCV and RandomizedSearchCV, guiding readers on how to systematically search for optimal hyperparameter configurations. Feature selection and elimination methods are revisited, including recursive feature elimination (RFE), to ensure models are both efficient and interpretable. Dimensionality reduction techniques like PCA and Linear Discriminant Analysis (LDA) are introduced as complementary tools for feature optimization. The chapter also provides an in-depth discussion of ensemble learning methods, including ensemble methods like bagging, boosting and stacking, showing how combining models can improve accuracy and generalization. The selected case study demonstrates these concepts in action by improving the performance of a classification model through careful tuning and ensemble methods.

Chapter 10: Building Multi-step Pipelines - This chapter emphasizes the importance of reproducibility and scalability in machine learning workflows. Readers are introduced to scikit-learn pipelines, which aid in streamlining preprocessing, feature engineering and model training into unified workflows. We take the techniques you have learned, up to this point, relating to preprocessing, model training and hyperparameter tuning, and wrap these into a simple reproducible pipeline.

Chapter 11: Getting Deep with Deep Neural Networks - Step into the world of deep learning! We will begin with the foundations of neural

networks and multi-layer perceptrons (MLPs), revisiting the preprocessing steps you have mastered so far. From there, you will develop and train your first deep neural network - starting with scikit-learn wrappers before moving on to advanced frameworks like PyTorch and TensorFlow. To bring these ideas to life, we will tackle a real-world problem: predicting a person's income from a set of features. Along the way, you will build a regression network from the ground up, exploring key components such as layers, activation functions, loss functions, weights, and biases. you will visualize how your model learns, refine it, and compare implementations across scikit-learn, PyTorch, and TensorFlow. By the end of this chapter, you won't just understand deep neural networks, you will have the tools to create and train them yourself.

Chapter 12: Deploying Your Machine Learning Application - This chapter takes machine learning beyond the notebook and into the real world, showing you how to turn models into powerful, production ready, services. You will explore Flask and FastAPI to build RESTful APIs that expose your models to the web, then dive into Docker to master containerization through writing Dockerfiles, creating container images and running containers that make your ML applications portable and reliable anywhere. Along the way, you will bring Swagger into the mix to generate interactive documentation that makes your APIs easy to test and maintain. To tie it all together, we will walk the reader through a hands-on project deploying a customer churn model as a web API, while unpacking best practices for versioning, scaling and keeping machine learning services robust in production.

Chapter 13: Machine Learning Future Trends and Ethical Considerations - The final chapter steps back to consider the societal impact of machine learning while equipping readers with the tools to build responsible AI systems. We begin by surveying, and getting hands-on, with transformative technologies such as large language models (LLMs), automated machine learning (AutoML), federated learning and edge computing. From there, we construct an ethical machine learning pipeline

under the TRUSTED ethical framework: triaging risk by classifying high stakes cases in domains like hiring or healthcare and inspecting models with scikit-learn's tools; securing the right data using stratified splits and reproducible pipelines; enabling uninterrupted monitoring with drift detection, MLFlow logging and retraining triggers; and embedding human supervision through human-in-the-loop (HITL) review of flagged edge cases. This framework emphasizes rigorous technical documentation and model serialization for reproducibility, while explainability is reinforced through SHAP analysis, interpretable models, and post-hoc explanations for borderline predictions. Finally, domain ethics oversight ensures that fairness, transparency and accountability are contextualized within sector-specific governance frameworks. The chapter closes with a future-facing discussion of AI trends, encouraging readers to stay adaptable while deploying a fully documented, monitored and ethically guided prediction pipeline built on the adult census dataset, demonstrating how technical precision and ethical responsibility must go hand in hand.

Code Bundle and Coloured Images

Please follow the link to download the *Code Bundle* and the *Coloured Images* of the book:

<https://rebrand.ly/547357>

The code bundle for the book is also hosted on GitHub at <https://github.com/bpbpublications/NumPy-Pandas-and-Scikit-learn-Masterclass>.

In case there's an update to the code, it will be updated on the existing GitHub repository.

We have code bundles from our rich catalogue of books and videos available at <https://github.com/bpbpublications>. Check them out!

Errata

We take immense pride in our work at BPB Publications and follow best practices to ensure the accuracy of our content to provide with an indulging reading experience to our subscribers. Our readers are our mirrors, and we use their inputs to reflect and improve upon human errors, if any, that may have occurred during the publishing processes involved. To let us maintain the quality and help us reach out to any readers who might be having difficulties due to any unforeseen errors, please write to us at :

errata@bpbonline.com

Your support, suggestions and feedbacks are highly appreciated by the BPB Publications' Family.

At www.bpbonline.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on BPB books and eBooks. You can check our social media handles below:



Instagram

Facebook

Linkedin

YouTube

Get in touch with us at: business@bpbonline.com for more details.

Piracy

If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at business@bpbonline.com with a link to the material.

If you are interested in becoming an author

If there is a topic that you have expertise in, and you are interested in either writing or contributing to a book, please visit www.bpbonline.com. We have worked with thousands of developers and tech professionals, just like you, to help them share their insights with the global tech community.

You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Reviews

Please leave a review. Once you have read and used this book, why not leave a review on the site that you purchased it from? Potential readers can then see and use your unbiased opinion to make purchase decisions. We at BPB can understand what you think about our products, and our authors can see your feedback on their book. Thank you!

For more information about BPB, please visit www.bpbonline.com.

Join our Discord space

Join our Discord workspace for latest updates, offers, tech happenings around the world, new releases, and sessions with the authors:

<https://discord.bpbonline.com>



Table of Contents

1. Overview of NumPy and Pandas

Introduction

Structure

Objectives

Introduction to NumPy

Installing and importing NumPy

Creating arrays

One-dimensional arrays

Two-dimensional arrays

Indexing and slicing arrays

Indexing one-dimensional arrays

Slicing one dimensional arrays

Indexing two-dimensional arrays

Slicing two-dimensional arrays

Condition based indexing

Fancy indexing

Basic operations on NumPy arrays

Statistical functions on arrays

Calculate the mean

Calculate the standard deviation

Calculate the variance

Calculate the median average

Calculate the minimum and maximum

Advanced NumPy concepts

Setting up our simple housing dataset

Broadcasting

Reshaping arrays

Joining arrays

Stacking arrays

Matrix multiplication

Normalization of house predictions

Singular value decomposition

Covariance and correlation matrices

Covariance

Correlation matrix

Introduction to Pandas

Creating DataFrames

Creating Series

Data manipulation using Pandas

Importing our Palmer Penguins dataset

Data inspection

Checking data types

Information on your DataFrame

Summary statistics

Selecting by a single column

Selecting by multiple columns

Filtering rows on a condition

Filtering rows based on multiple condition with loc

Filtering rows with AND conditions

Filtering rows with OR conditions

Filtering rows with NOT conditions

Filtering rows with iloc

Filtering rows with numpy.where()

Sorting by a single column

Sorting by multiple columns

Handling missing data

Understanding missing data

Dropping all rows with missing values

Imputing missing values

Merging and applying functions over DataFrames

Joining DataFrames

Applying a lambda function

Saving and loading data

Working with time series in Pandas

Creating a time series DataFrame

Resampling our time series data

Handling missing data in time series

Shifting time series data

Rolling moving average

Dealing with large datasets in Pandas

Generating our Faker dataset

Using chunk_size to read large datasets efficiently

Specify data types

Subsetting with nrows

Downcasting numerical columns

Converting categorical columns

Using sparse data types

Working with chunks to perform operations

Parallel processing with Dask

Use Parquet format with PyArrow

Vectorized operations over looping

Using SQL-like queries

Conclusion

Points to remember

Multiple choice questions

Answers

Key terms

2. Introduction to Scikit-learn for Machine Learning

Introduction

Structure

Objectives

Understanding scikit-learn

Primary use cases

Role of scikit-learn in ML pipeline

Data collection and preprocessing

Modeling and training

Regression algorithms

Classification algorithms

Unsupervised algorithms

Evaluation and model selection

Deployment

Setting up and installing scikit-learn

Key concepts and usage in scikit-learn

Estimators

Transformers

Pipelines

Evaluation metrics

Creating your data to practice machine learning

Creating a regression dataset

Creating a classification dataset

Creating an unsupervised dataset for clustering

Breast cancer classification model development

Getting our data

Using scikit-learn datasets

Export dataset to CSV

Read CSV file

Checking class imbalance

Handling missing values

Scaling numerical features

Split features and labels

Create our train and test partitions

Modelling

Fitting one classifier

Fitting multiple candidate classifiers

Evaluation metrics and visualization of results

Recall and precision

F1 score

Accuracy

Negative predictive value

Visualizing model results

Model selection

K-fold cross validation

Getting started

Fitting the K-fold cross validation

Visualizing the K-fold evaluation metrics

K-fold evaluation the simple way

Conclusion

Points to remember

Multiple choice questions

Answers

Key terms

3. Supervised Binary Classification

Introduction

Structure

Objectives

Introducing binary classification

Role of the logit function

Sigmoid, turning log-odds into probabilities

Thyroid disease prediction case study

Getting the data

Exploratory data analysis

Histogram

Missing values

Feature correlation

Class distribution

Missing values summary

Feature encoding, selection and cleaning

Dropping columns

Splitting data

One-hot encoding

Missing value imputation with MICE

Scaling and normalizing our data

Feature selection

Model training

Modeling setup

Stratified K-fold

Evaluation metrics

Understanding the confusion matrix

Over and under sampling methods

Building the training and evaluation loop

Building the training loop

Building the evaluation loop

Receiver operator characteristics curve

Conclusion

Points to remember

Multiple choice questions

Answers

Key terms

4. Supervised Multi-class Classification

Introduction

Structure

Objectives

Introduction to multi-class classification

Decision boundaries for random forests and logistic regression

Importance of log loss in model training

Log loss in binary classification

Log loss in multi-class classification

Predicting customer churn use case

Data importing

Feature distribution visualization

Missing values analysis

Class distribution

Partitioning the data

Handling categorical and numerical features for classification

Missing value imputation

Modal replacement

Imputation and scaling with ColumnTransformer

One-hot encoding in Pandas

Class rebalancing

Evaluation metrics for multi-class classification

Macro Precision

Macro Recall

Macro F1 Score

Precision per class

Recall per class

Support

Classifier instantiate

One-vs-Rest and One-vs-One strategies

One-vs-Rest

One-vs-One

OutputCodeClassifier

Custom threshold class

Extended strategies applied

Modeling, evaluation, and model trade-offs

Model training and evaluation loop

Model trade-offs with chosen models

Visualizing evaluation results

Conclusion

Points to remember

Multiple choice questions

Answers

Key terms

5. Customer Segmentation with Unsupervised Methods

Introduction

Structure

Objectives

Introducing unsupervised ML

Understanding K-means clustering

Understanding the working of K-means

K-means made easy with scikit-learn

Understanding agglomerative clustering

Understanding agglomerative clustering working

Agglomerative clustering with scikit-learn

Understand density-based scanning clustering

How DBSCAN works

DBSCAN with scikit-learn

Clustering evaluation techniques

Elbow method

Silhouette score

Silhouette plot

Customer segmentation

Import and clean dataset

Partitioning data

One-hot encoding

Feature scaling

Cluster performance visualization

Instantiate the K-means model

Finding optimal K clusters

Finding optimal training clusters

PCA to visualize our clusters

Evaluation of performance on testing set

Conclusion

Points to remember

Multiple choice questions

Answers

Key terms

6. House Price Estimation with Regression Methods

Introduction

Structure

Objectives

Introduction to regression analysis and linear regression

Ordinary least squares regression

OLS in action

Multiple linear regression

Feature selection and recursive feature elimination for regression

Recursive feature elimination

Automating feature selection

Regularization in regression

Ridge regression

Lasso regression

Elastic Net regression

Non-linear regression methods

Random forests

Gradient boosting and XGBoost

Regression evaluation metrics

Mean absolute error

Mean squared error

Root mean squared error

Mean squared logarithmic error

R-squared

Adjusted R-squared

Mean absolute percentage error

Metrics function

Predicting house prices project

Importing and inspecting data

Visualizing relationships

Analyzing missing values and dropping features

Preprocessing function

Modeling setup

Regression model training loop

Training our regression model

Using SHAP for global and local feature importance

Global feature importance with SHAP

Local feature importance with SHAP

Joining predictions with test dataset

Conclusion

Points to remember

Multiple choice questions

Answers

Key terms

7. Handwritten Digits Dimensionality Reduction

Introduction

Structure

Objectives

Introducing dimensionality reduction techniques

Principal component analysis

Step 1 as standardizing data

Step 2 as computing the covariance matrix

Step 3 as computing eigenvalues and vectors

Step 4 as choosing the number of principal components

Step 5 as projecting data onto principal components

t-distributed Stochastic Neighbor Embedding

Step 1 as computing similarities in high dimensional space

Step 2 as Student t-distribution and mapping to lower dimensions

Step 3 as Kullback-Leibler divergence

Step 4 as gradient descent on our cost function

Uniform Manifold Approximation and Projection

Step 1 as finding k-nearest neighbors

Step 2 as constructing a weighted graph

Step 3 as performing dimensionality reduction

Visualizing high-dimensional data in 2D and 3D

Applications of dimensionality reduction in machine learning

Applying dimensionality reduction to the MNIST dataset

Loading and preparing the MNIST dataset

Visualizing MNIST digits

Creating our two-dimensional plotting function

Splitting and scaling the MNIST dataset

PCA for visualization in two-dimensions

t-SNE for visualization in two-dimensions

UMAP for visualization in 2-dimensions

Automatically selecting optimal components

Train SVM classifier to predict MNIST digits

Evaluating the classifier

Conclusion

Points to remember

Multiple choice questions

Answers

Key terms

8. Time Series with Scikit-learn

Introduction

Structure

Objectives

Introduction and challenges in time series data

Loading and preparing time series data

Loading the dataset

Wrangling the date

Preprocessing time series data

Load time series dataset post cleaning

Generating missing time series data points

Forward filling missing time series data

Backward filling missing time series data

Kalman filtering for time series imputation

Resampling time series data

Analyzing time series data

Visualizing the time series

Time series decomposition

Stationarity checks

Differencing to ensure stationarity

Detour into log transformations

Confirming stationarity post differencing

Detecting autocorrelation and partial autocorrelation

Autocorrelation function

Partial autocorrelation function

Creating lagged features for machine learning models

Methods of splitting time series data

Hold-out split

Time series cross-validation

Modeling and tuning time series models

Loading and preparing data

Holts-Winters exponential smoothing

Seasonal ARIMA

Machine learning approaches on lagged features

Automated ARIMA with Prophet

Prophet data processing

Modeling with Prophet

Evaluating time series models

Combine forecasts to compare models

Evaluating model results and finding the best model

Extending predictions beyond the forecast horizon

Conclusion

Points to remember

Multiple choice questions

Answers

Key terms

9. Model Improvement Strategies

Introduction

Structure

Objectives

Hyperparameter tuning

Preparing the data and instantiating the model

Grid search

Random search

Bayesian optimization with Optuna

Evaluation of hyperparameter tuning methods

Feature elimination and dimensionality reduction with LDA

Detour into Linear Discriminant Analysis

Mutation of methods to improve model performance

Baseline model and LDA

Recursive feature elimination

Feature selection with chi-square

Evaluation of feature selection and elimination methods

Ensemble learning

Bagging

Boosting

Stacking

Evaluation of ensemble methods

Other considerations for model improvement

Conclusion

Points to remember

Multiple choice questions

Answers

Key terms

10. Building Multi-step Pipelines

Introduction

Structure

Objectives

Introduction to scikit-learn pipelines and transformers

Preprocessing steps

Loading the customer churn dataset

Creating the numerical pipeline

Creating the categorical pipeline

Creating a custom transformer

Using ColumnTransformer to combine pipelines

Creating a union with our features

Saving our preprocessor

Combining multiple models and transformers into a pipeline

Loading our preprocessor

Combine multiple models with a stacking ensemble

Using oversampling in our pipeline

Building and visualizing the pipeline

Creating a parameter grid and saving as JSON

Create reusable base models in a pipeline

Using pipelines with hyperparameter tuning

Tuning the stacked ensemble model

Tuning the model per pipeline strategy

Evaluating the stacked classifier

Boilerplate code templates for your projects

Conclusion

Points to remember

Multiple choice questions

Answers

Key terms

11. Getting Deep with Deep Neural Networks

Introduction

Structure

Objectives

Introducing neural networks and multi-layer perceptrons

Structure of a neural network

Activation functions

The foundations of a multi-layer perceptron

Building deep neural networks using wrappers

Data preparation

Building and training the MLPRegressor

Hyperparameter tuning of MLP

Visualizing regression performance

Actual versus predicted plot

Distribution of residuals plot

Quantile-quantile chart

Transitioning to PyTorch and TensorFlow

Building multi-layer perceptrons in PyTorch

Split and scale the training data

Setting a PyTorch device

Tensor conversion and dataset creation

Understanding DataLoaders

Creating the base neural network in PyTorch

Define optimizer and loss functions

Creating the training loop

Visualizing the training history

Evaluation of trained model on test dataset

Building a multi-layer perceptron with TensorFlow

Define and compile the TensorFlow Keras model

Add checkpointing and fit the model

Evaluation steps for Keras model

Conclusion

Points to remember

Multiple choice questions

Answers

Key terms

12. Deploying Your Machine Learning Application

Introduction

Structure

Objectives

Introducing Flask and FastAPI for building APIs

Flask, the simple and reliable option

FastAPI, high-performance framework for production inference

Creating RESTful endpoints for ML models

Creating a RESTful Flask endpoint

Folder setup

Creating the Flask application

Running and testing the Flask application

Making requests to the endpoint

Creating a RESTful FastAPI endpoint

Creating the FastAPI application

Running the FastAPI application with Uvicorn

Using the FastAPI endpoint for batch prediction

Update the model with a PUT request

Introduction to Docker

Getting familiar with core Docker concepts

Docker image creation, building, and running containers

Dockerfile for Docker image creation

Building the Docker container

Running the Docker container

Best practices for deploying ML applications

Conclusion

Points to remember

Multiple choice questions

Answers

Key terms

13. Machine Learning Future Trends and Ethical Considerations

Introduction

Structure

Objectives

Introduction to large language models

Setting up OpenAI's API

Code generation with OpenAI

Summarization and translation with OpenAI

Summarizing the document

Translating the document

Text generation with HuggingFace

Automated machine learning

AutoML with tree-based optimization tool

Federated learning

Ethical frameworks, bias, privacy, and AI governance

Detecting and mitigating bias with scikit-learn

Gender shades case study

Unsupervised bias detection case study

Privacy preserving techniques in scikit-learn pipelines

AI governance

Building a responsible pipeline

Loading and preparing the data

Building the pipeline

Cross-validation and sub-group performance

Fitting the model

Evaluating the sub-group performance

Flag uncertain predictions for human review

Documentation and audit trail

Conclusion

Points to remember

Multiple choice questions

Answers

Key terms

Index

CHAPTER 1

Overview of NumPy and Pandas

Introduction

This chapter aims to equip you with the fundamental skills necessary for working with data, particularly in shaping it for processing by downstream **machine learning (ML)** models. Central to these skills is learning NumPy, an essential Python library for numerical computing that excels at handling multi-dimensional data and performing a wide array of mathematical operations. Building upon the capabilities of NumPy, Pandas offers user-friendly data structures such as Series and DataFrames, that simplify data manipulation and analysis.

By mastering NumPy and Pandas, you will be able to efficiently load, clean, transform, and analyze datasets of various sizes and complexities. These libraries form the backbone of data processing in Python and are indispensable tools for any data scientist or analyst looking to prepare data for machine learning tasks. In this chapter, we will look into practical examples using real-world datasets to illustrate how these libraries can be leveraged to streamline your data workflow.

Structure

This chapter covers the following topics:

- Introduction to NumPy
- Advanced NumPy
- Introduction to Pandas

- Data manipulation using Pandas
- Handling missing data
- Working with time series in Pandas
- Dealing with large datasets with Pandas

Objectives

By the end of this chapter, you will understand how to use NumPy and Pandas for data processing tasks that will enable you to prepare data for ML problems. This will be reinforced through every subsequent section, with hands-on approaches to applying these tools for feature engineering throughout this book. When you have finished reading this chapter, you will be comfortable with data manipulation in NumPy and why it is important, as well as understanding the different data structures in Pandas, and how you can perform data wrangling in both packages.

Introduction to NumPy

NumPy is a powerful library for numerical computing that provides support for creating and manipulating arrays (normally multidimensional). This makes it essential for handling and processing large datasets in ML and data analysis tasks. Its speed, memory efficiency, and ability to perform vectorized operations (i.e., performing tasks on many items at once instead of one by one) allow it to greatly outperform native Python structures like lists, especially in handling numerical data. Once you grasp NumPy, it serves as the entry point for all other deep learning architectures, such as **PyTorch** and **TensorFlow**.

Installing and importing NumPy

To get started with NumPy, we need to make sure we have it installed in a Python environment of your choice, whether that be **Anaconda**, **Colab**, **Poetry**, **Pyenv**, or **virtualenv**.

The following steps take you through the process of installing and running NumPy:

1. Once you have your Python environment ready to go and working, you will need to use the following command to install it into the environment:

pip install numpy

2. The Python compiler will install the requirements of the package, and then it will tell you it was successful.
3. To use this in your project, you can simply use the following command in your Python script to **import numpy**:

import numpy as np

4. The **as** keyword, in *Step 3*, is used to create an alias; in this case, **np** is just a commonly used shorthand for NumPy. You could technically choose any name you like (e.g., **import numpy as mynumpy**), but **np** is the widely accepted and standard convention in the Python community.

Now that we have NumPy loaded into our environment, let us take a brief detour to explore how NumPy is used in machine learning. In ML, we often work with large datasets containing numerical data in multidimensional formats (e.g., features used for predictions, like height when predicting weight). NumPy allows us to manage this data efficiently through optimized structures like arrays. As we progress into the modeling sections of this book, you will see how essential these flexible data structures are for handling complex features in ML models.

Creating arrays

The first step in any ML pipeline is to load and represent data. ML models typically expect input data in the form of arrays or matrices (especially **scikit-learn**, **PyTorch**, and **TensorFlow**). NumPy arrays are the foundational data structures for this purpose, also known as **ndarrays**.

We will use a known **ordinary least squares (OLS)** prediction problem to predict weight based on height observations, and our arrays will be numerical values that represent a small sample of these observations, as this

will allow us to see how an understanding of NumPy is pivotal to understanding how to process data for ML models.

The next two steps take you through how to create both one and two-dimensional arrays, which are the fundamental cornerstone for array processing in Python.

One-dimensional arrays

Think of one-dimensional arrays like a single column in a spreadsheet. For example, if we want to store data in an array to perform calculations on that column (such as storing the average heights of males to compute descriptive statistics), we can easily create an array specifically for this purpose:

```
1. import numpy as np
2. # Creating a 1D NumPy array
3. heights = np.array([150, 160, 170, 180, 190])
4. print(heights)
```

This will produce a single array of values, much like Python's list structure. While we will explore various calculations we can perform on this array in the statistical functions on arrays section, for now, consider it a simple way to store numerical data in Python for future mathematical operations.

Two-dimensional arrays

These are what we will be working with mostly, as in many ML problems, data is typically stored in tabular formats (i.e., rows and columns), especially in real-world applications where structured data is common. The 2D array is a good way to represent such a dataset, where our rows represent samples and columns represent our features (traditionally denoted with x in mathematics).

To bring this to life, each row in this array could represent a single data point or observation (e.g., measurements of weight), and each column represents a factor that causes weight to vary, for example, height, **body mass index (BMI)**, etc.:

```
1. arr_2d = np.array([[150, 50],
2.                    [160, 60],
3.                    [170, 65],
4.                    [180, 80],
5.                    [190, 90]])
```

Now that we have created our 2D array representing height and weight, we will move on to how you access elements from the array, also known as *indexing* and *slicing*.

Indexing and slicing arrays

Indexing in NumPy arrays allows you to access individual elements, or a subset of elements, in an array. Slicing allows you to slice a subset of the elements. We will explore how to do this for one and two-dimensional arrays; it becomes more complex as you explore more dimensions.

Indexing one-dimensional arrays

To explore indexing, we can observe the following code using the array we created in the 1D array section, where we created the heights array:

```
1. print(heights[0])
2. print(heights[2])
3. print(heights[-1])
```

The following bullets mention the outputs we will get from each command:

- The first argument will bring back the first index (as Python is zero-indexed as a language), so from our **heights** array, this will return 150, which is the first element of the heights array.
- The second line will return the third position (index=2), and that will bring back the value 170.
- In the third argument, the minus 1 (-1), this is shorthand for the last element in the array and will always bring back the last element. In the case of the heights array, this brings back the value of 190.

Slicing one dimensional arrays

To perform this, you use the same bracket operations to grab what you need from the array ([] is equivalent to giving an index or range):

1. `print(heights[1:4])`
2. `print(heights[:3])`
3. `print(heights[2:])`
4. `print(heights[:])`

The following bullets mention the outputs we will get from each command:

- The second to the fourth element in the heights array: **[160,170,180]**
- Everything from the beginning to the 3rd index in the array: **[150,160,170]**
- Start at the third element and get me everything to the end: **[170,180,190]**
- Including just this is the same as saying just return to me all the elements in the array.

Indexing two-dimensional arrays

Our indexing task becomes a little trickier now because we are going to try and pull back rows and columns and refer back to our heights and weights array we constructed in the two-dimensional arrays sub-section. The following code shows you how to achieve this:

1. `print(arr_2d[0, 0])`
2. `print(arr_2d[1, 1])`
2. `print(arr_2d[4, 1])`

Let us break down this code:

- The first line is saying get me the first item in the first row (remember zero indexing) - this is the same as saying get me the

height of the first person. The resultant output would be **150**.

- From this line, the indexing retrieves the second element in the second row, equating to the weight of the second person. Resulting in the value **60**.
- To reinforce this, here is a further example to say, retrieve the fifth row and second column, i.e., grab me the weight of the fifth person; this gives us **90**.

Now, we will move on to examples working with slicing these arrays.

Slicing two-dimensional arrays

Slicing can be done in a similar way to indexing, but here we want to extract rows and columns, or even specific submatrices. The following code will show how to slice the **arr_2d** NumPy array we created in the two-dimensional arrays section of this chapter:

```
1. print(arr_2d[1:4, 0])
2. print(arr_2d[:, 1])
3. print(arr_2d[2:, :])
```

Breaking this down, we will see the following output:

- The first example takes rows (index 1 to 4) and gets the first column (**heights**) of the person (remember zeroth indexing is used in Python). The results would be: **[160 170 180]**.
- In the second code line, we take all the rows (denoted using the **:** operator) and select the second column (**weights**). This is equivalent to saying *return me the weights of all the people in the study*. Output is: **[50 60 65 80 90]**.
- Thirdly, we slice from the third row (second index) all the way to the end of the data (using the **:** operator again) and select me all the columns (again, using the **:** operator). This would give us back a 3 x 3 array: **[[170 65] [180 80] [190 90]]**.

This lays the foundations nicely for how to index and slice in NumPy, but there are times when you want to index based on a condition.

Condition based indexing

We will use the two-dimensional arrays that we created at the beginning of this chapter, using conditional operators to extract a subset of the array, based on the condition we impose. These conditional operators are captured in the following table:

Operator	Meaning
>	Greater than
>=	Greater than or equal to
<	Less than
<=	Less than or equal to
==	Equal to
!=	Not equal to

Table 1.1: NumPy filtering conditional operators

An example of how we would use these in our array is detailed here:

1. `heights = arr_2d[:, 0]`
2. `tall_people = heights[heights > 170]`
3. `print(tall_people)`

Here, we are extracting just the height column from the 2D array, created earlier in this chapter.

Then we are using the subset notation in NumPy to say give me all the heights of a person where the array value is greater than 170 cm, and then print this out to the Python console, returning: **[180,190]**.

Fancy indexing

Fancy indexing is where we specify specific integer locations of the elements, which can be passed to the subset by wrapping it in an internal list:

```
1. selected_rows = arr_2d[[0, 2, 4], :]  
2. print(selected_rows)
```

This would give us the first, third, and fifth rows of the selection and would choose all columns. This would return an array of: `[[150 50][170 65] [190 90]]`.

Basic operations on NumPy arrays

NumPy's primary strength lies in its ability to efficiently handle numerical data and perform mathematical operations, algorithms, and functions with impressive speed.

We will continue to use the height and weight data we created earlier to work with the mathematical operations; however, in this example, we will recreate the arrays at the top of this section, so we can commence working with them. The arrays are going to be entitled **heights** and **weights** corresponding to our height and weight measurements sampled:

```
1. heights = np.array([150, 160, 170, 180, 190])  
2. weights = np.array([50, 60, 65, 80, 90])  
3. print("Heights:", heights)  
4. print("Weights:", weights)
```

To understand the code, we will break down each step in the following list:

- **Adding a constant to all elements in an array:** Suppose you want to add a constant to all elements in my array, say converting the height into a metric measure, then you would do something like this, which is also known as *broadcasting*, and we will be touching upon this concept again in the *Advanced NumPy* section of this book:

```
1. heights_in_meters = heights / 100  
2. print("Heights in meters:", heights_in_meters)
```

This gives us all the heights in meters: **Heights in meters: [1.5 1.6 1.7 1.8 1.9]**.

- **Multiply or scale the entire array:** Scaling values in an array is something you will come to do a lot when you start preparing data for machine learning models. For now, we will do something simple and just multiply the values in an array by two:

```
1. scaled_weights = weights * 2
2. print("Scaled Weights (multiplied by 2):", scaled_weights)
```

The variable **scaled_weights** will output all the values in the **weights** array, multiplied by 2: **[100, 120, 130, 160, 180]** as a Python list.

- **Element-wise multiplication of two arrays:** For our example, this would not be something you really want to do, as height multiplied by weight is a meaningless metric, but for the sake of example, this is how you would achieve it in code:

```
1. combined = heights * weights
2. print("Combined:", combined)
```

Here, we get an element-wise multiplication of the heights and weights matrix; this means that it will take the first index from both arrays and multiply them together.

Then, it will continue to the second element and do the same calculation until the end of the array is reached.

Statistical functions on arrays

There are many more operations that you can perform on arrays, using the relevant Python operators we have used, but the real power of data analysis is the ability to obtain statistical insights on your data. This sub-section will show how to implement these in our height versus weight case study that we have been working with.

Calculate the mean

The arithmetic mean tells us the sum of all the elements in the dataset and is divided by the number of elements. Means are notoriously problematic if you have extreme outliers in your dataset, as they can skew the mean in a positive or negative direction; with highly skewed datasets, one would look to implement another type of average statistic.

To calculate the average of our heights and weights arrays is relatively straightforward and demonstrated hereunder:

```
1. mean_height = np.mean(heights)
2. mean_weight = np.mean(weights)
3. print("Mean Height:", mean_height)
4. print("Mean Weight:", mean_weight)
```

The output of this will be the arithmetic mean of the height and weight arrays, respectively. This will return a printout to the console to indicate the **Mean Height: 170.0** and the **Mean Weight: 69.0**.

Calculate the standard deviation

The standard deviation is a statistical measure that indicates the amount of variation or dispersion in a set of data. It shows how much individual data points differ from the average (mean) of the data. A low standard deviation means the data points are close to the mean, while a high standard deviation indicates that the data points are spread out over a wider range. This is equally very easy to implement on our heights and weights array:

```
1. std_height = np.std(heights)
2. std_weight = np.std(weights)
3. print("Standard Deviation of Heights:", std_height)
4. print("Standard Deviation of Weights:", std_weight)
```

We would receive an output from our height and weight arrays with a print statement to print these to the console. These outputs will be displayed:

- **Standard deviation of Heights: 14.1421**
- **Standard deviation of Weights: 14.2828** (rounded to remove the 15 place precision values after the decimal place).

Calculate the variance

Variance is a statistical measure that indicates how spread out or dispersed the values in a data set are. It calculates the average of the squared differences from the mean. Essentially, variance helps show how far individual data points deviate from the mean of the data set. A higher variance means greater variability, with data points more spread out, while a lower variance suggests the data points are closer to the mean. Variance is often used in fields like finance, quality control, and data analysis to understand the degree of risk or consistency in a set of observations, and once again, it is a breeze with NumPy to implement. The following code shows how to reference the functions' method and to then use these functions to calculate statistics over the arrays:

```
1. var_height = np.var(heights)
2. var_weight = np.var(weights)
3. print("Variance of Heights:", var_height)
4. print("Variance of Weights:", var_weight)
```

This will output: **Variance of Heights: 200** and **Variance of Weights: 204**.

Calculate the median average

The median is a statistical measure that represents the middle value in a data set when the numbers are arranged in order, either from lowest to highest or vice versa. It divides the data into two equal halves: 50% of the values fall below the median, and 50% are above it. If the data set has an odd number of values, the median is the middle number. If the data set has an even number of values, the median is the average of the two middle numbers. The median is especially useful when dealing with skewed data or outliers, as it provides a better central tendency than the mean in such cases.

Implementation of this using our heights and weights use case is highlighted in the following code:

```
1. median_height = np.median(heights)
2. median_weight = np.median(weights)
3. print("Median Height:", median_height)
4. print("Median Weight:", median_weight)
```

The resultant output would print out, to the console, **Median Height: 170.0** and **Median Weight: 65.0**. The following section will look at the **min** and **max** functions and explain why these are important statistical measures to be aware of.

Calculate the minimum and maximum

The **minimum** (**min**) and **maximum** (**max**) values are important because they define the range of a data set, showing its overall spread. The minimum is the smallest value, and the maximum is the largest. Together, they help determine the range, which gives a quick sense of how widely the data points are distributed.

Min and max also help identify extreme values or outliers, offering insight into potential anomalies or trends. They provide context for other statistical measures like the mean and median, helping analysts better understand the overall distribution of the data. To achieve this, we do the following:

```
1. min_height = np.min(heights)
2. max_height = np.max(heights)
3. min_weight = np.min(weights)
4. max_weight = np.max(weights)
5. print("Min Height:", min_height, "Max Height:", max_height)
6. print("Min Weight:", min_weight, "Max Weight:", max_weight)
```

What we would get from running this line of code would output two lines to the console:

- The first line would show **Min Height: 150, Max Height: 190**, focusing specifically on the **heights** array
- The weights array output would show **Min Weight: 50, Max Weight: 90**.

This brings us to the end of the statistical functions section of NumPy. There are many more functions we could look at here, but for the sake of brevity, here we will reference the NumPy functions guide to allow you to explore the depths of what is possible with functions in NumPy; the link is: <https://numpy.org/doc/stable/reference/>.

Advanced NumPy concepts

In this section, we will be exploring some exciting and practical tools that will help us make smarter predictions, like estimating housing prices. Here, we will cover core concepts such as broadcasting, which allows us to apply operations across different-sized arrays easily, and reshaping arrays, a way to organize and manipulate our data in flexible formats will be discussed. Additionally, we will also explore joining and stacking arrays, two methods for combining data to reveal new insights and make more robust predictions.

When it comes to linear algebra, we will unlock the power behind housing data analysis by learning about matrix multiplication, which is crucial for performing complex calculations at scale. Advanced techniques like normalization, ensuring our data is standardized for more accurate comparisons, and **singular value decomposition (SVD)**, which simplifies large data sets to uncover hidden relationships, will also be covered. Additionally, we will look into the correlation and covariance matrices, allowing us to measure how different factors (like square footage and price) relate to each other.

This journey through linear algebra and array operations will make predicting housing prices not only efficient but also fascinating.

Setting up our simple housing dataset

The dataset we will use for this is a very simplified version of a housing prediction problem, where we implement the NumPy array from scratch and then we will perform various operations on this dataset. Later, we will look at more advanced concepts that are crucial for working with ML models, but for now, let us set it up:

```
1. import numpy as np
2. houses = np.array([
3.     [3, 2000, 10],
4.     [2, 1500, 20],
5.     [4, 2500, 5],
6. ])
```

Here is a breakdown of the features and observations of this code:

- The number of bedrooms in the property (integer count of bedrooms)
- The square footage of the property (square feet)
- The age of the property

From here, we are set up and ready to perform functions and morph the data to our will. Let us get started.

Broadcasting

Suppose we had our original data wrong and wanted to broadcast over our array to add one bedroom to each of the housing calculations, we can do this simply in NumPy, as follows:

```
1. updated_bedrooms = houses[:, 0] + 1 # Adding 1 to each bedroom
2. print("Updated bedrooms after broadcasting:\n", updated_bedrooms)
```

This will just increment all bedrooms by one, as we are indexing the first feature and using all the rows (as we learned, this is indicated by :).

The output will be: **Updated bedrooms after broadcasting: [4 3 5]**.

Next, suppose we want to update the age of the houses; we can again do this simply by indexing our NumPy array and grabbing the right index integer values from the array:

1. `updated_age = houses[:, 2] - 5`
2. `print("Updated house ages after broadcasting:\n", updated_age)`

Our output, after running the preceding code, will show the result as a print statement: **Updated house ages after broadcasting [5 15 0]**. This shows that our operation over the array has worked and broadcasted correctly.

Finally, we now realize we need to increase the dimension of the square footage of the house by 10% (1.1). we can simply multiply by this scaling factor to increase the size of the rooms:

1. `adjusted_size = houses[:, 1] * 1.1`
2. `print("Adjusted house sizes (sq ft) by 10% increase:\n", adjusted_size)`

The adjusted size of the properties should be taken into account by the uplift factor, and printing this out to the console, we can confirm indeed that this has taken place: **Adjusted house sizes (sq ft) by 10% increase [2200. 1650. 2750.]**. This confirms that our broadcasting on the arrays has worked, as expected. However, we have not actually changed the underlying array, as we have been reassigning each time to a different variable, so the original array remains intact for our next exploration.

Reshaping arrays

Reshaping arrays in NumPy is like rearranging the way your data is organized without changing the data itself. This is very useful because sometimes data comes in one form, but you need it in a different form to analyze it or use it in calculations. For example, you might need to turn a flat list into a matrix for an ML model. Reshaping helps you quickly switch between these different shapes, making your data more flexible and easier to work with.

To get the initial sizing of the **housing** dataset, we need to use the following command:

```
1. print(houses.shape)
```

Returning a tuple of the size of the dataset, we get printed to our console **(3,3)**, meaning three observations with three features. Now that we know the dimensions of the dataset, we can do some reshaping of this to introduce the idea of reshaping to you.

For example, if we want to collapse my 3 x 3 array down into a flattened 1-D array, we can use the following:

```
1. reshaped_houses = houses.reshape(-1)
2. print("Reshaped houses (flattened):\n", reshaped_houses)
```

This will give you a completely flattened array, with all the data collapsed to one dimension: **Reshaped houses (flattened): [3 2000 10 2 1500 20 4 2500 5]**.

To extend this example, we could even flatten this down so that we get a 1 x 9 (feature array); this would be pretty meaningless from a modelling challenge, but highlights how this could be achieved:

```
1. reshaped_houses_1x9 = houses.reshape(1, 9)
2. print("Reshaped houses into 1x9:\n", reshaped_houses_1x9)
```

This will return: **Reshaped houses into 1x9: [[3 2000 10 2 1500 20 4 2500 5]]**.

Joining arrays

Concatenation of arrays allows us to stitch different arrays together effectively. To extend our example, we will add another house to our dataset and add the set of features for that house. The powerhouse for this is our concatenation functions in NumPy. To achieve this, we will create a NumPy array with a list (needed for multiple value passing):

```
1. more_houses = np.array([
2.     [5, 3000, 15],
```

```
3. ])
```

We could join across the zeroth axis (axis=0). This is how we would use concatenate to achieve this:

```
1. joined_houses = np.concatenate((houses, more_houses), axis=0)
2. print("Joined houses (by rows):\n", joined_houses)
```

The output of this is essentially an extra row in our housing array. If we want to join on the column (axis=1), we need to make sure that the list we pass, in this instance, the price of the house will be length = 3, as we found earlier, that is the current shape of the houses array. The following example shows how to add this extra data and do the join:

```
1. extra_data = np.array([[100000], [120000], [140000]])
2. joined_columns = np.concatenate((houses, extra_data), axis=1)
3. print("Joined houses with extra column (price):\n", joined_columns)
```

Resulting in an extra column (let us call it price here, because NumPy arrays do not have the column name concept, we will talk about Pandas soon):

Bedrooms	Square feet	Property age	Price (aka extra data)
3	2000	10	100000
2	1500	20	120000
4	2500	5	140000

Table 1.2: Concatenation on axis=1

Stacking arrays

We can achieve the same results, as in the previous section, by using the vertical and horizontal stacking methods in NumPy. The two methods are as follows:

- **vstack** (vertical stack) allows you to stack arrays on top of each other, like adding rows to a matrix. This is helpful when you want

to combine multiple sets of data row by row.

- **hstack** (horizontal stack) stacks arrays side by side, like adding columns to a matrix, which is useful when you want to combine data by adding features or variables.

Starting with the **vstack** method, we will look at the additional house use cases, as we did in the joining section. Let us implement the **vstack** method for achieving the same results:

```
1. stacked_houses_vertically = np.vstack((houses, more_houses))
2. print("Vertically stacked houses:\n", stacked_houses_vertically)
```

This stacks the houses row-wise, as we did using the axis parameter in the concatenate function.

To horizontally stack our new column (price) to the DataFrame, we will achieve it in exactly the same way, and it produces the same output as in *Table 1.2*. To achieve this:

```
1. stacked_houses_horizontally = np.hstack((houses, extra_data))
2. print("Horizontally stacked houses with extra column:\n", stacked_houses_horizontally)
```

Matrix multiplication

In matrix multiplication, the number of columns in the first matrix must match the number of rows in the second matrix. The product is calculated by taking the dot product of rows from the first matrix with columns from the second matrix. Each element in the resulting matrix is the sum of the products of corresponding elements from the row and column.

Matrix multiplication is not commutative, meaning that switching the order of the matrices changes the result. It is particularly useful for representing linear transformations, solving systems of linear equations, and in machine learning, where it helps model and compute relationships between features and data.

For our use case, we will be using a multiple linear regression approach (we will explain this more thoroughly in our regression examples later on in this book) to derive a function, as shown:

$$\text{predicted_prices} = \text{houses} \cdot \text{weights} + \text{bias}$$

Expanding this equation for every observation is derived as shown:

$$\text{predicted_price}_i = (\text{bedrooms}_i \times \text{weight}_{\text{bedrooms}}) + (\text{size}_i \times \text{weight}_{\text{size}}) + (\text{age}_i \times \text{weight}_{\text{age}}) + \text{bias}$$

Now we will define this in Python, and each step will be explained:

1. We will use the previously defined houses array that we have been working with throughout the Advanced NumPy section.
2. Next, we will derive a weights array (these are essentially the learned weights from our algorithm of choice; we will train a model in the next section, but for now, we will use some precomputed weights):

```
weights = np.array([50, 100, -20])
```

3. We will select a bias term, otherwise known as the intercept in regression (we will get on to this properly later):

```
bias = 30000
```

4. Those are now all the ingredients we need to perform the dot product (**np.dot**) of the houses by the weights, adding the bias term:

```
predicted_prices = np.dot(houses, weights) + bias  
print("Predicted prices:", predicted_prices)
```

We now have a prediction from a basic model to inform our predicted prices. These will be output as: **Predicted prices: [229950 179700 280100]**.

Normalization of house predictions

In the following code, normalization is applied to the features of the houses to scale them proportionally. Let us break down what the normalization process does and why it is important:

```
1. house_norms = np.linalg.norm(houses, axis=1, keepdims=True)
```

This line computes the norm of each row (house) in the houses array. A norm is a measure of the magnitude (or length) of a vector. For each house (represented by a row), it calculates the Euclidean norm (also called the L2 norm) by squaring the values of its features, summing them up, and then taking the square root. The following steps will outline the equation for normalization and use the dot product (matrix multiplication) in NumPy:

1. For a house with features [Bedrooms, Size (sq ft), Age], the norm is calculated as:

$$\text{norm} = \sqrt{(\text{Bedrooms}^2 + \text{Size}^2 + \text{Age}^2)}$$

2. Next, we actually apply the normalization to the features; this is implemented in the following code:

```
1. normalized_houses = houses / house_norms
```

3. This simply scales each of the features in the house data by the division of the normalization discussed earlier.
4. The final step is to take the dot product of the normalized houses by the weights, and including the bias term, as we did in the *Matrix multiplication* section:

```
1. predicted_prices_normalized = np.dot(normalized_houses, weights) + bias
```

```
2. print("Predicted prices (normalized):", predicted_prices_normalized)
```

The results of this print out to the console as: **Predicted prices (normalized): [30099.97, 30099.79, 30100.04]**. The results are more

normalized, and because we are only using three observations, they would be much more accurate over larger volumes of data.

Singular value decomposition

SVD is a mathematical technique used to decompose a matrix into three other matrices: $\mathbf{U}$, $\mathbf{S}$, and $\mathbf{V}^t$. In this decomposition, $\mathbf{U}$ contains the left singular vectors, $\mathbf{S}$ holds the singular values (which represent the importance or magnitude of each component), and $\mathbf{V}^t$ contains the right singular vectors. This allows us to break down complex data into simpler components, capturing its most important patterns or relationships. SVD is particularly useful for analyzing data in ways that reduce noise, reveal hidden structures, and simplify how we represent complex datasets.

In ML, SVD is valuable for tasks like dimensionality reduction and feature extraction. By retaining only the most important singular values and their associated components, SVD helps reduce the number of features while preserving the critical information. This makes the data easier to work with and can improve the performance of models by reducing overfitting and computational complexity. SVD is also used in techniques like **principal component analysis (PCA)**, which is widely applied in ML to identify the key factors driving patterns in data and improve prediction accuracy in models.

To implement this, we can simply utilize the following code. We will revisit this section in greater depth when we get to the *PCA and dimensionality reduction* sections, but for now, on our housing example, it would be implemented as in the following code:

```
1. U, S, Vt = np.linalg.svd(houses, full_matrices=False)
2. print("U matrix:", U)
3. print("Singular values:", S)
4. print("V transpose matrix:", Vt)
```

The resultant matrices are explained as follows:

- **U matrix:** This matrix captures the patterns and relationships between the individual houses (rows of the original matrix). It gives you an understanding of how different houses are similar or different based on their features.
- **S (singular values):** The diagonal matrix of singular values tells you how much each dimension or component contributes to the structure of the data. Larger singular values indicate more significant components, while smaller values represent less important ones. You can use these values to reduce dimensionality by keeping only the most important components.
- **Vt matrix:** This matrix captures the relationships between the features (columns of the original matrix, like bedrooms, size, and age). It shows how the features are combined to form the different components of the data.

Covariance and correlation matrices

Covariance and correlation matrices are essential tools in data analysis because they help measure the relationships between variables, offering valuable insights into how different features interact with each other.

Covariance

The covariance matrix can be implemented in NumPy using the following:

```
cov_matrix = np.cov(houses, rowvar=False)
print("Covariance matrix:\n", cov_matrix)
```

In the supporting code repository for this chapter, we have outlined a way to visualize this covariance matrix. The resultant matrix is found in the following figure:

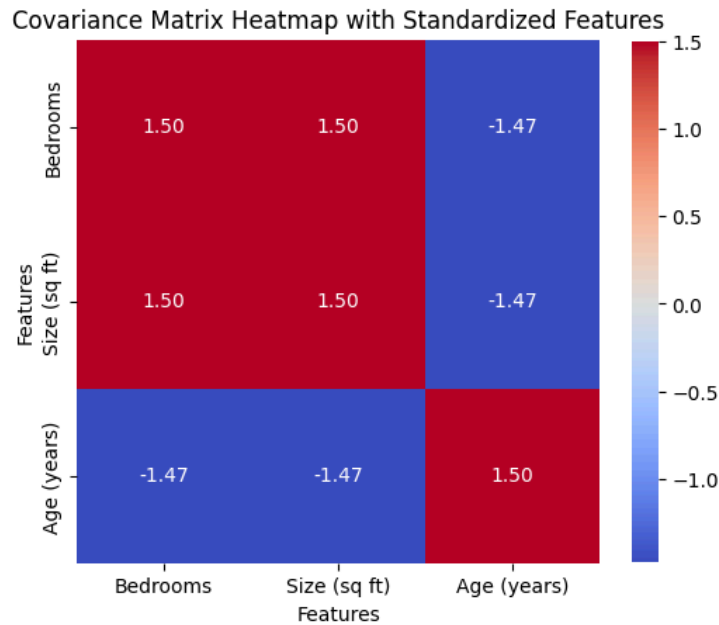


Figure 1.1: Covariance matrix

This matrix shows the extent to which two variables change together. A positive covariance indicates that the variables tend to increase or decrease together, while a negative covariance means that as one variable increases, the other tends to decrease. The covariance matrix helps identify relationships between variables, but since it is sensitive to the scale of the data, the values can be difficult to interpret directly. However, it is useful in understanding the direction of relationships, such as identifying whether larger houses tend to have higher prices or older houses tend to have lower prices.

Correlation matrix

This matrix goes a step further by standardizing the covariance values, giving a measure between -1 and 1 that indicates both the strength and direction of relationships between variables, regardless of their scale. A correlation value close to 1 means a strong positive relationship, -1 means a strong negative relationship, and 0 means no linear relationship. The correlation matrix is particularly useful for understanding the strength of associations between features, such as determining how strongly the number of bedrooms correlates with house prices, which can help in feature selection

for models. To get the matrix here, we have implemented a visualization method, but to get it in an array, simply run the following code snippet:

1. `corr_matrix = np.corrcoef(houses, rowvar=False)`
2. `print("Pearson correlation matrix:\n", corr_matrix)`

The visualization of this is presented in the following figure:

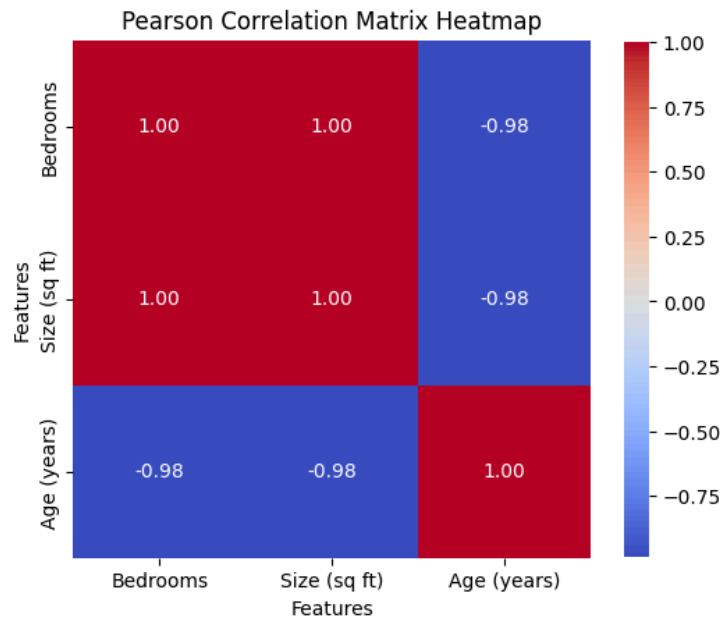


Figure 1.2: Correlation matrix

The next section will move away from height versus weight OLS type equations, housing prices estimation with linear algebra, and move into the world of Pandas working with the famous **Palmer Penguins** dataset. To break up the pure business examples, we will look at the world of zoology.

Introduction to Pandas

Pandas is a powerful data manipulation and analysis library for Python, offering versatile structures like DataFrames and Series to easily manage structured data such as tables or time series. These data structures are designed to efficiently handle large datasets, enabling users to organize, filter, and analyze data with just a few lines of code. Pandas is widely used in fields like data science, finance, and research because it simplifies

working with complex or messy datasets, making tasks such as sorting, filtering, and transforming data much easier.

One of the key strengths of pandas is its ability to handle missing data, which is common in real-world datasets. It provides simple ways to fill in missing values, remove unwanted rows or columns, and perform calculations even when the data is not complete. Pandas also allows for powerful data manipulation, such as filtering and grouping data based on specific conditions, merging multiple datasets, and reshaping data for different analyses. These features make it an essential tool for data exploration, preparing data for machine learning models, or generating reports and insights from various data sources.

Creating DataFrames

Before we enter the world of Palmer Penguins, we will stick with the example of our housing arrays, as we covered before, but this time we will make a DataFrame from scratch for our features (the things that inform the target prediction), and our target will be a Series value.

To create a DataFrame to store features, we will pass in a dictionary of key-value pairs to enable the DataFrame to be constructed simplistically. This is how you can achieve this:

```
1. import pandas as pd
2. data = {
3.     'Bedrooms': [3, 2, 4, 3],
4.     'Size_sqft': [2000, 1500, 2500, 1800],
5.     'Age_years': [10, 20, 5, 15]
6. }
7. df = pd.DataFrame(data)
8. print(df)
9. print(type(df))
```

This will give us our DataFrame with four rows and three columns (**Bedrooms**, **Size_sqft**, and **Age_years**).

The next step is to create the Series in the upcoming sub-section.

Creating Series

Series are a way to create single columns and are normally used in machine learning to create the target (the variable that you are trying to predict). In our case, this would be based on our sample dataset (this dataset is way too small for ML; we will be dealing with much larger datasets in the hands-on modeling stages). This would be the house price, as this is the prediction task, or the y variable (dependent). To implement a new Series, we can do the following:

1. `prices = [500000, 400000, 650000, 550000]`
2. `price_series = pd.Series(prices, name='Price')`
3. `print(price_series)`

Alone, these are the two main data structures you need to know when getting started with Pandas. For a full list of functions with Pandas, see their online resource: <https://pandas.pydata.org/>.

Data manipulation using Pandas

This is where Pandas truly excels, offering a wide range of pre-built functions for data preprocessing, feature engineering, and more. While this section will not cover every possible Pandas feature, doing so could fill an entire book; you will finish with a solid understanding of key concepts like filtering, grouping, sorting, loading, and summarizing data.

Importing our Palmer Penguins dataset

The Palmer Penguins dataset contains physical measurements of three penguin species, *Adélie*, *Chinstrap*, and *Gentoo*, collected from three islands in the *Palmer* archipelago, *Antarctica*.

The dataset includes features such as bill length, bill depth, flipper length, body mass, and the sex of the penguins, along with the island and year of observation. It is a simple, easy-to-understand dataset that offers a great introduction to data exploration, visualization, and machine learning tasks.

To load it into our Python environment, we simply need to reference the Seaborn (<https://seaborn.pydata.org/>) package; simply make sure this library is installed using **pip install seaborn**, and use it, as shown in the following code:

```
1. import pandas as pd
2. import seaborn as sns
3. penguins = sns.load_dataset('penguins')
```

Data inspection

Pandas provides two simple methods for inspecting the head of the dataset and the tail of the dataset. All you must do is pass the number of examples you want to pull out of the head of the dataset, if you do not pass this then you will get the top five examples from the head and the bottom five from the tail.

To get the head of the dataset refer to the following command:

```
1. print(penguins.head())
```

This will display the first five records:

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	Male
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0	Female
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	Female
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	NaN
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0	Female

Figure 1.3: Penguins head of DataFrame

To get the last ten records, otherwise known as the tail, use the following command:

```
1. print(penguins.tail(10))
```

As we passed a parameter into the tail method of ten, we will get the last ten observations:

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
334	Gentoo	Biscoe	46.2	14.1	217.0	4375.0	Female
335	Gentoo	Biscoe	55.1	16.0	230.0	5850.0	Male
336	Gentoo	Biscoe	44.5	15.7	217.0	4875.0	NaN
337	Gentoo	Biscoe	48.8	16.2	222.0	6000.0	Male
338	Gentoo	Biscoe	47.2	13.7	214.0	4925.0	Female
339	Gentoo	Biscoe	NaN	NaN	NaN	NaN	NaN
340	Gentoo	Biscoe	46.8	14.3	215.0	4850.0	Female
341	Gentoo	Biscoe	50.4	15.7	222.0	5750.0	Male
342	Gentoo	Biscoe	45.2	14.8	212.0	5200.0	Female
343	Gentoo	Biscoe	49.9	16.1	213.0	5400.0	Male

Figure 1.4: Penguins last ten observations of DataFrame

Checking data types

It is crucially important to know what data you are working with when you take on a modeling challenge. There is a simple command that allows you to quickly grab the data types for your DataFrame:

```
1. print(penguins.dtypes)
```

This will show you the following:

species	object
island	object
bill_length_mm	float64
bill_depth_mm	float64
flipper_length_mm	float64
body_mass_g	float64
sex	object

Figure 1.5: Penguins data types of features

Information on your DataFrame

To do a quick analysis of your DataFrame, it is easy with the **.info()** method:

```
1. print(penguins.info())
```

This provides you with information on the column name, integer position, how many non-nulls are present in the table and the **Dtype** of all the objects, the memory usage of the table, and the inheriting class.

Summary statistics

Previously, in the NumPy section, we created summary statistics for our data, using **np.mean**, **np.min**, etc. In Pandas, there is just no need to do this; all you need is the **.describe()** method:

```
1. print(penguins.describe())
```

This outputs the summary statistics, including count, mean, std (standard deviation), min, max, and percentile values (25%, 50% and 75%). We did not implement these in Python, but percentiles are the same as the spread of data in the distribution, with 50% being equivalent to the median. The output will be displayed, as shown in the following figure:

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
count	342.000000	342.000000	342.000000	342.000000
mean	43.921930	17.151170	200.915205	4201.754386
std	5.459584	1.974793	14.061714	801.954536
min	32.100000	13.100000	172.000000	2700.000000
25%	39.225000	15.600000	190.000000	3550.000000
50%	44.450000	17.300000	197.000000	4050.000000
75%	48.500000	18.700000	213.000000	4750.000000
max	59.600000	21.500000	231.000000	6300.000000

Figure 1.6: Penguins summary statistics

Selecting by a single column

To select a single column is as easy as just grabbing the index of that column, as we learned in the NumPy section:

```
1. penguins['species']
```

This returns the name of the species of the Penguin only, as we only want one column, which is why we passed it by name.

Selecting by multiple columns

To select by multiple columns, we need to pass a list of column names to be subset:

```
1. penguins[['species', 'island', 'bill_length_mm']]
```

Here, we are selecting only the **species**, **island**, and **bill_length_mm** columns from the dataset. We are returning the same number of rows, but focusing on specific columns by passing their names as a list.

Filtering rows on a condition

The difference between this operation and selecting is that we are filtering rows, and unless we specify, all columns will be returned. To implement a simple filter to pick out all my penguin species that have a bill length that is greater than 45 millimeters, we can do this as shown in the following code:

```
1. filtered_penguins = penguins[penguins['bill_length_mm'] > 45]
2. filtered_penguins
```

This returns 165 observations (instead of the full sample of 344 rows).

Filtering rows based on multiple condition with loc

The **loc** function in Pandas is used to access or modify rows and columns in a DataFrame based on label or Boolean conditions, enabling flexible data selection using index labels.

We can use the **&** operator for logical and (to apply multiple conditions), the **|** operator (pipe) for logical OR (to apply either condition), and the **~** operator for logical NOT (to exclude certain conditions). Examples of each in practice will be demonstrated in the upcoming sections.

Filtering rows with AND conditions

To filter our penguin dataset on multiple conditions, we can perform this as follows:

```
1. filtered_penguins = penguins.loc[
2.     (penguins['species'] == 'Adelie') &
3.     (penguins['island'] == 'Dream') &
4.     (penguins['bill_length_mm'] > 40)
5. ]
6. filtered_penguins
```

This gives us 17 records from our original and states:

- Return me only the **Adelie** penguin **species** (using the equality operator **==**) AND
- Where they are from the **island** equal (**==**) to **Dream** AND
- Those penguins that have a bill length that is greater than (**>**) 40 mm

Filtering rows with OR conditions

We will now try an example with an OR condition:

```
1. filtered_penguins_or = penguins.loc[
2.     (penguins['species'] == 'Adelie') |
3.     (penguins['bill_length_mm'] > 45)
4. ]
5. filtered_penguins_or
```

Here, we are selecting rows where the species is either Adelie or the bill length in mm is greater than 45mm. This results in 314 rows being returned. In contrast, using the AND condition would give fewer rows, as it requires all conditions to be true.

Filtering rows with NOT conditions

Let us try the command NOT:

```
1. filtered_penguins_not = penguins.loc[
2.     ~(penguins['species'] == 'Gentoo')
3. ]
4. filtered_penguins_not
```

Running this code, we get 220 records, as we have just excluded the **Gentoo** species of Penguin.

Filtering rows with iloc

The **iloc** function in **pandas** is used to access or modify rows and columns in a DataFrame by their integer position, allowing for selection based on row and column indices. We can do this by passing integer indices of the columns we wish to return, as well as a slice of the rows:

```
1. subset_penguins = penguins.iloc[20:31, [0, 1, 2]]
2. subset_penguins
```

This brings back rows 20:30 and selects columns **species**, **island**, and **bill_length_mm**.

Filtering rows with `numpy.where()`

Another efficient way to filter columns and rows is by using **`numpy.where()`**, which acts as a search function across arrays and can be faster than the default Pandas filtering methods. Here, we will use the power of NumPy to filter where certain conditions are met, in this example where the species is equal to Adelie and the **`bill_length_mm`** is greater than 45:

```
1. import numpy as np
2. indices = np.where(
3.     (penguins['species'] == 'Adelie') |
4.     (penguins['bill_length_mm'] > 45)
5. )
6. filtered_penguins_or_numpy = penguins.iloc[indices]
7. filtered_penguins_or_numpy
```

This function must always be used in conjunction with **`iloc`**, as **`numpy.where()`** returns integer indices of the matching results. It also generates what is known as a filter mask, where *Boolean* logic is applied to mask out non-matching entities from the query. In this example, the filter query yields 314 rows. Filter masks make it easier to manage large datasets by focusing on relevant rows based on specified conditions.

Sorting by a single column

To put things into ascending and descending order is common in most applications. This code shows how to order our data by the **`bill_length_mm`** variable and return the ascending results back:

```
1. sorted_penguins = penguins.sort_values(by='bill_length_mm', ascending=False)
2. sorted_penguins
```

Sorting by multiple columns

We follow the same process as before, but as we know, when we pass multiple items, these need to be passed as a list into the function:

1. `sorted_penguins = penguins.sort_values(by=['species', 'bill_length_mm'],
ascending=[True, False])`
2. `sorted_penguins`

Handling missing data

In this section, we explore methods for handling missing data, a crucial step in real-world data analysis. Unlike the clean datasets typically used in learning environments, real-world data is often messy and incomplete. As a result, significant time and effort must be spent on data cleaning and feature engineering before you can even begin building ML models. Handling missing values effectively is key to ensuring the quality and accuracy of your analysis.

Understanding missing data

The first step in addressing missing data is to identify which values are missing in your DataFrame and decide on the best approach for handling them. Imputation, or the replacement of missing values, can be done in various ways depending on the situation. In this chapter, we will explore several methods, some of which will be further applied in later machine learning tasks.

To check for missing data, we can look to see where Pandas returns a NULL output. This can be done in one line of operations:

1. `penguins.isnull().sum()`

These returns use a Series for the missing values:

species	0
island	0
bill_length_mm	2
bill_depth_mm	2
flipper_length_mm	2
body_mass_g	2
sex	11

Figure 1.7: Missing data summary

Dropping all rows with missing values

We will begin with the most drastic approach, removing rows with missing data, though it is usually better to try filling in missing values through methods like modal or mean replacement. However, if you have a large enough dataset and can afford to remove incomplete rows, you can do so using the following method:

1. `penguins_dropped = penguins.dropna()`
2. `penguins_dropped`

This reduces our dataset slightly, but for massive numbers of null values, it is recommended to obtain the values through other means, as you may be missing key observational data.

Imputing missing values

Missing value imputation involves filling in missing data with appropriate values, such as using the mean of nearby observations (e.g., nearest neighbour imputation) or simply applying the column mean. There are various methods for imputation, each suited to different scenarios, and we will explore some of these techniques later on. For a straightforward mean replacement strategy, you can implement it as follows:

1. `penguins['bill_length_mm'] = penguins['bill_length_mm'].fillna(penguins['bill_length_mm'].mean())`
2. `penguins`

Merging and applying functions over DataFrames

Merging and applying functions over DataFrames are crucial operations in pandas because they allow for the efficient combination and transformation of data, which is essential in real-world data analysis. Merging enables you to bring together multiple datasets based on common keys or indices, making it possible to consolidate information from different sources for a more comprehensive analysis. This is particularly important when datasets are scattered across different files or databases. Applying functions allows you to transform or manipulate data efficiently across columns or rows, making it easier to clean, standardize, or derive new insights. These operations streamline data preparation, enabling more accurate modeling, reporting, and decision-making in fields like finance, research, and machine learning.

Joining DataFrames

Joining DataFrames is an important concept to know when combining multiple datasets on the same key. A short example is provided in the following code. We will be looking at how to perform a left join on our penguins' data to add an endangered status to the existing data:

1. `extra_data = pd.DataFrame({`
2. `'species': ['Adelie', 'Chinstrap', 'Gentoo'],`
3. `'endangered_status': ['Least Concern', 'Near Threatened', 'Least Concern']`
4. `})`
5. `merged_penguins = pd.merge(penguins, extra_data, on='species', how='left')`
6. `merged_penguins`

This now has a column where the species have been matched, and an **endangered_status** has been applied to each species.

Applying a lambda function

An anonymous function, also known as a lambda function, is a small, unnamed function in Python that is defined using the `lambda` keyword. Unlike regular functions created with **def**, lambda functions are typically used for short, simple operations that are needed temporarily. They can take any number of arguments but contain only a single expression, which is evaluated and returned. Lambda functions are often used in cases where defining a full function would be overkill, such as within **map()**, **filter()**, or **apply()** functions for quick, one-off computations. We can implement a simple lambda function to indicate if the Penguin's bill is **Long** or **Short**:

```
1. penguins['bill_category'] = penguins['bill_length_mm'].apply(lambda
    a
    2.                                x: 'Long' if x > 45 else 'Short')
    3. penguins
```

You would use the same `apply` syntax here for a function you defined **user defined function (UDF)**, but instead of `lambda`, you would just pass the name of the function (this is detailed as a supplementary example in the code).

Saving and loading data

To save our penguins example to a comma-separated value list, we can use the in-built **to_csv** method, which allows us to simply save to a desired output path:

```
1. penguins.to_csv('penguins_data.csv', index=False)
```

To load this back in, we would simply reference the file path we saved it to and use the **read_csv** function:

```
1. penguins_from_csv = pd.read_csv('penguins_data.csv')
2. print(penguins_from_csv.head())
```


Working with time series in Pandas

Working with time series data is a common requirement in data analysis, especially in fields like finance, economics, and environmental science. Pandas provides a comprehensive set of tools to handle time-indexed data efficiently. In this tutorial, we covered some of the key time series operations in pandas, including the following:

- **Creating time series data:** Generating time-based DataFrames using `pd.date_range()`.
- **Resampling data:** Changing the frequency of data, such as converting daily data to weekly or monthly summaries.
- **Handling missing dates:** Filling gaps in time-indexed data and dealing with missing values.
- **Shifting data:** Moving data forward or backward in time, which is useful for calculating lags or making predictions.
- **Rolling window calculations:** Applying moving averages or other statistical operations over a fixed window of time.

These techniques provide a solid foundation for working with time series data, but some areas may require further exploration, such as:

- **Time series decomposition:** Breaking down a time series into trend, seasonality, and residuals.
- **Advanced resampling and frequency adjustments:** Handling non-standard time intervals or working with business day frequencies.
- **Rolling and expanding windows:** More in-depth usage for smoothing, forecasting, and analyzing trends.
- **Time series forecasting models:** Using ARIMA, SARIMA, and other machine learning models for future predictions.

These further exploration areas will be covered in *Chapter 8, Time Series with Scikit-learn*.

Creating a time series DataFrame

In this section, we will generate a data range starting from 1st January 2024 and set it to generate 120 daily periods. We will then use our trusty friend NumPy to produce random numbers, drawn from a random distribution, between 100 and 200. The implementation looks as follows:

```
1. import pandas as pd
2. import numpy as np
3. dates = pd.date_range(start='2024-01-01', periods=120, freq='D')
4. data = pd.DataFrame({
5.     'Sales': np.random.randint(100, 200, size=120)
6. }, index=dates)
7. print(data)
```

This gives us a random sales dataset and can be viewed by this line plot, as shown in *Figure 1.8*:

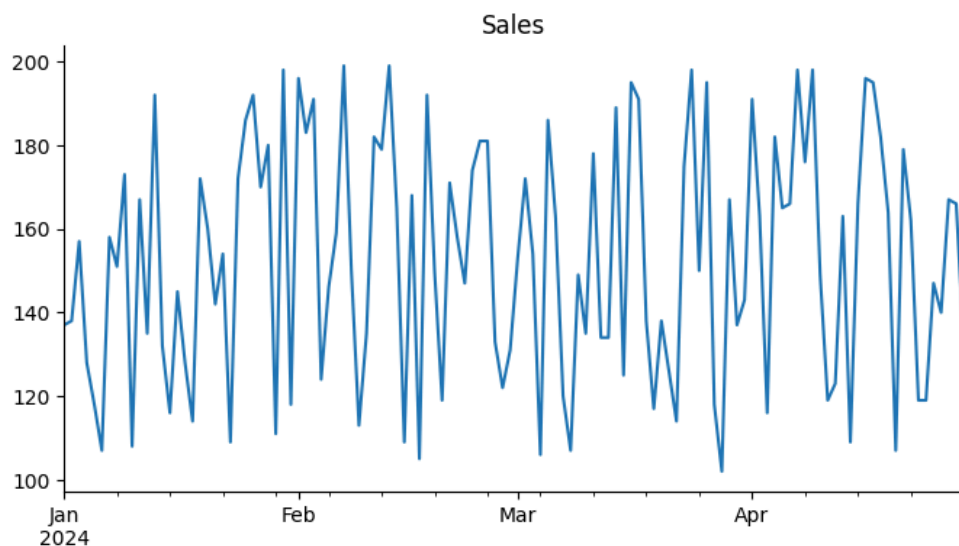


Figure 1.8: Randomly generated Sales data

Resampling our time series data

This is very useful when you want to quickly roll up the daily breakdowns to provide monthly, quarterly, and yearly summaries:

```
1. monthly_data = data.resample('ME').sum()
```

This generated a roll up of our sales volumes, as shown in the following figure:

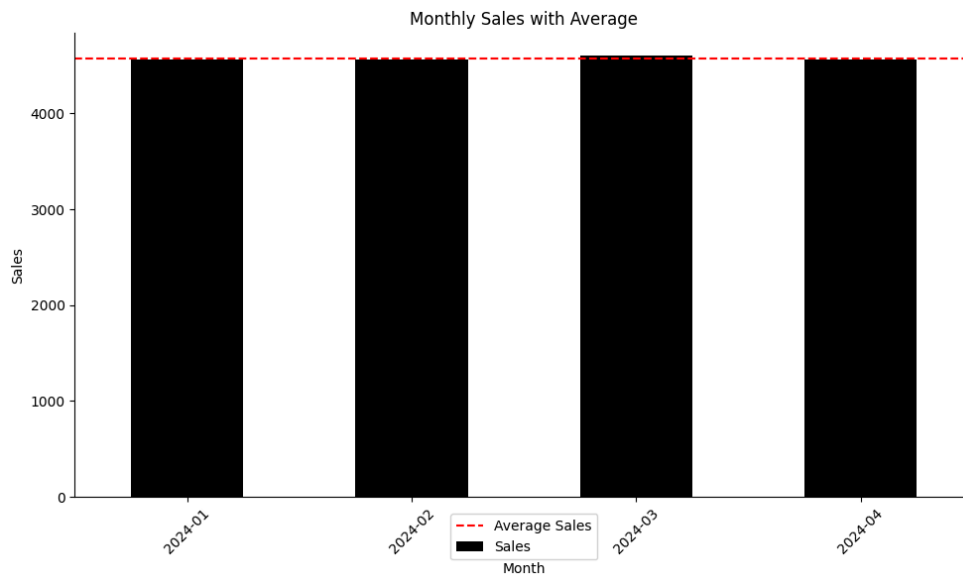


Figure 1.9: Resampled time series data

Handling missing data in time series

Missing dates in time series data can pose challenges when analyzing trends, seasonality, or performing calculations over time. Incomplete date ranges may occur due to holidays, weekends, or irregular data collection. Addressing missing dates is important to ensure the integrity of time series analysis. Pandas provides several methods to handle this, such as reindexing to include the full date range and filling missing data using techniques like forward fill (**ffill**), backward fill (**bfill**), or interpolation. Filling in missing dates ensures that calculations such as moving averages or resampling operate smoothly, producing more accurate and reliable results. We use forward fill in our examples to fill in the dates in a new time series DataFrame:

```
1. dates = pd.to_datetime(['2023-01-01', '2023-01-03', '2023-01-05'])
```

```

2. data_with_gaps = pd.DataFrame({
3.     'Sales': [150, 200, 180]
4. }, index=dates)
5. full_dates = pd.date_range(start='2023-01-01', end='2023-01-07')
6. data_filled = data_with_gaps.reindex(full_dates)
7. data_filled_ffill = data_filled.ffill()
8. print(data_filled_ffill)

```

The resultant output shows that the missing dates are now filled, using the forward fill method.

Shifting time series data

Shifting the time series can be done to examine the change between one day and the next:

```

1. data_shifted = data.shift(1)
2. data['Daily Change'] = data['Sales'] - data_shifted['Sales']
3. print(data)

```

This results in a time series frame with the original sales and the daily change. This can be visualized in the following figure:

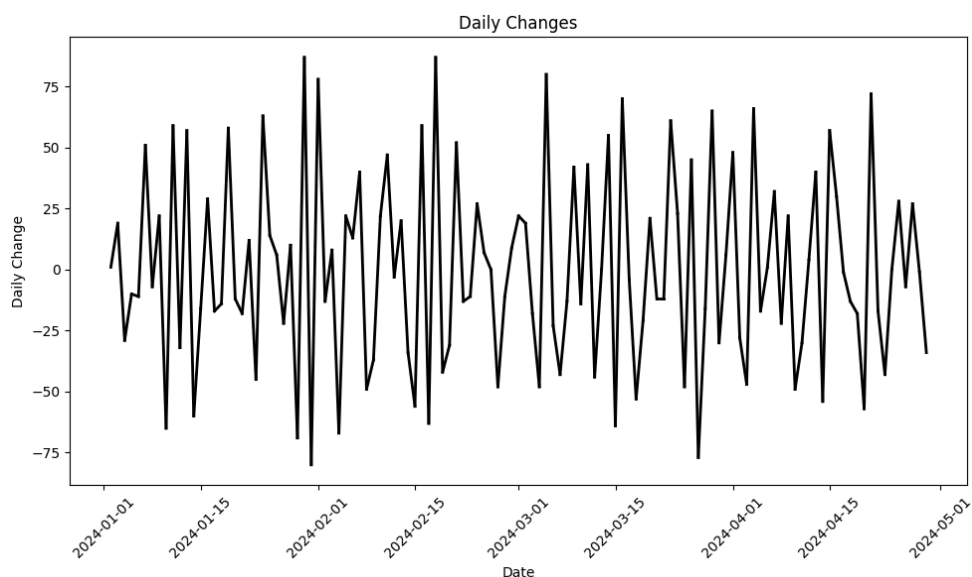


Figure 1.10: Shifted time series

Rolling moving average

A rolling moving average smooths out short-term fluctuations in time series data, making it easier to identify long-term trends and patterns. By reducing noise, it helps provide a clearer understanding of the general direction of the data. This is particularly useful in fields like finance and forecasting, where identifying trends over time is crucial for making informed decisions. This is relatively simple to add to our Sales DataFrame, which we defined at the start of this section. The following figure shows the Sales over each respective date, and the 3-day rolling average is plotted to smooth out the data:

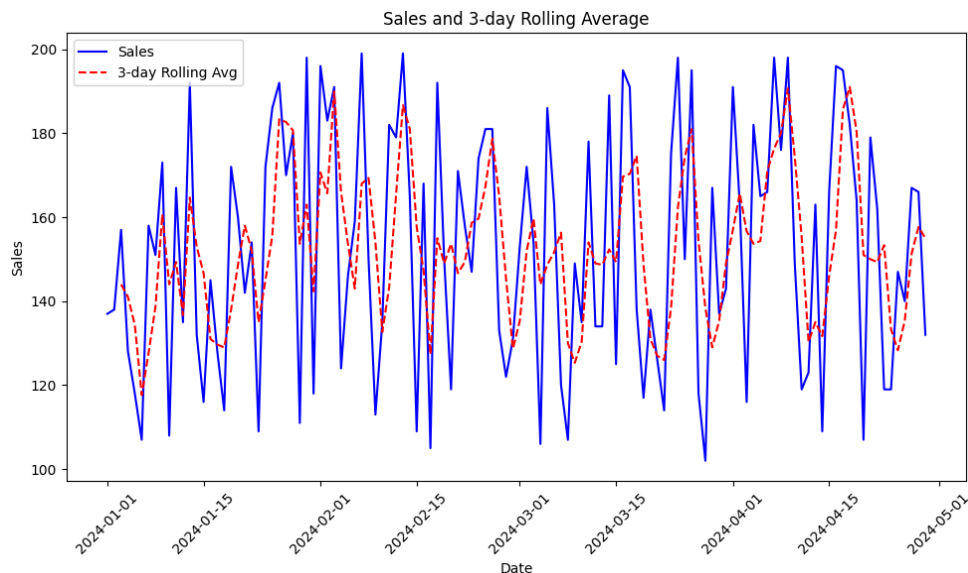


Figure 1.11: 3 day moving average sales

That concludes our look into time series methods in DataFrames. For now, we will revisit this in further depth in *Chapter 8, Time Series with Scikit-learn*.

Dealing with large datasets in Pandas

To work with this section, we are going to have to generate a large dataset. We will use the package Faker (<https://faker.readthedocs.io/en/master/>)

for this purpose, as this package has packages for generating augmented datasets. To install the package, we need to use **pip install faker**.

Once Faker has been installed, we need to import the following packages into our environment:

```
1. import pandas as pd
2. import numpy as np
3. import random
4. from faker import Faker
5. import datetime
```

Generating our Faker dataset

The following code will generate a fake dataset of 5 million records and will create the following fake fields: **id**, **name**, **age**, **email**, **address**, **city**, **country**, **salary**, **date_of_joining**, and **credit_score**. Once finished, the augmentation software then creates a CSV file and saves it. This part of the book takes over one hour to generate, so be wary before hitting the play button. This is how to initialize the script and get started:

```
1. fake = Faker()
2. n_rows = 5_000_000
3. data = {
4.     "id": np.arange(1, n_rows + 1),
5.     "name": [fake.name() for _ in range(n_rows)],
6.     "age": np.random.randint(18, 80, size=n_rows),
7.     "email": [fake.email() for _ in range(n_rows)],
8.     "address": [fake.address().replace("\n", ", ") for _ in range(n_rows)],
9.     "city": [fake.city() for _ in range(n_rows)],
10.    "country": [fake.country() for _ in range(n_rows)],
```

```

11.     "salary": np.round(np.random.uniform(30000, 150000, size=n_r
      ows), 2),
12.     "date_of_joining": [fake.date_between(start_date='-10y', end_da
      te='today') for _ in range(n_rows)],
13.     "credit_score": np.random.randint(300, 850, size=n_rows),
14. }
15. df = pd.DataFrame(data)
16. df.to_csv("large_dataset.csv", index=False)
17. print("Dataset created and saved to 'large_dataset.csv'")

```

Once you receive the print message to your console, you are good to go for working with the file: **large_dataset.csv**, as this will now be stored in your folder root, unless you change the **df.to_csv** command to output to a new save path.

Using chunk_size to read large datasets efficiently

In this section, we will explore how to efficiently handle large datasets using chunking, a technique that allows us to break down massive files into smaller, manageable pieces. When working with extremely large datasets, loading the entire file into memory at once can overwhelm your system, causing performance issues or memory errors. By using chunk size, we can read the data in smaller portions, process each chunk independently, and only keep what we need in memory. This method not only improves performance but also ensures that even the largest datasets can be analyzed effectively, making it an essential strategy when dealing with big data. This can be achieved by using the following code. We will show later on how you perform functions using the following:

```

1. chunk_size = 50000
2. for chunk in pd.read_csv("large_dataset.csv", chunksize=chunk_siz
   e):
3.     print(chunk.head(1))

```

Specify data types

Specifying data types when loading large datasets can significantly reduce memory usage and improve performance. By default, pandas may assign larger data types than necessary, like using **int64** or **float64** for numerical data. However, many datasets contain values that can be stored in smaller data types (e.g., **int8**, **float32**), which take up less memory. Using our Faker data, this is achieved using the following code:

```
1. dtypes = {  
2.     'id': 'int32',  
3.     'age': 'int8',  
4.     'salary': 'float32',  
5.     'credit_score': 'int16'  
6. }  
7. df = pd.read_csv('large_dataset.csv', dtype=dtypes)
```

Subsetting with nrows

Using the **nrows** parameter in **pd.read_csv()** to load only a specific number of rows from a large dataset offers significant benefits in terms of memory efficiency and performance. By loading just the first 100,000 rows, for example, you reduce the amount of data being stored in memory, which is critical when working with massive datasets that could otherwise overwhelm your system.

This approach allows you to quickly access and analyze a manageable portion of the data without risking memory errors or slowdowns, making it especially useful during exploratory data analysis or when testing models.

In addition to reducing memory usage, limiting the number of rows also enhances performance by speeding up data loading times. Since you are processing a smaller amount of data, your system can work more efficiently, allowing for faster experimentation and iteration. You can simply use the

nrows parameter in the **read_csv** method, as demonstrated in the following code:

```
1. df = pd.read_csv('large_dataset.csv', nrows=100000)
2. print(len(df))
```

Inspecting the print method, you will see that the length of the DataFrame is as specified in the **nrows** parameter.

Downcasting numerical columns

Downcasting refers to the process of converting a data type to a more memory-efficient type. This usually means converting from a larger, more general type to a smaller, more specific type, which takes up less memory and can still store the same data. The most common cases involve converting numeric types such as int64 or float64 to smaller ones like int32, int16, float32, etc. This has the effect of altering the precision of the numbers for floating point numbers, thus reducing their memory footprint. Here we have included two examples in Python of how we can downcast the numerical values from our Faker dataset:

```
1. df['age'] = pd.to_numeric(df['age'], downcast='integer')
2. df['salary'] = pd.to_numeric(df['salary'], downcast='float')
```

It really is as simple as indexing the column you want to downcast, and then using the **to_numeric** function.

Converting categorical columns

This process can save significant memory by reducing the amount of space required to store repetitive strings of object data. This is particularly useful when the number of unique values is small compared to the total number of categorical values. To achieve this, simply:

```
1. df['country'] = df['country'].astype('category')
```

Under the hood, this creates a lookup index for each of the unique values and stores the index of the country. Say we have ['England', 'Scotland', 'Wales'], then this would encode the categorical counties to 0,1,2 index

values, for example. Storing data as integers is much more efficient than lengthy strings.

Using sparse data types

Sparse data types are effective when your column contains: 1) lots of zeros, 2) lots of missing values (**NaN**), and 3) repeated values in large numbers. To implement this on our **credit_score** field, we can apply the following function:

```
1. df['credit_score'] = pd.Series(pd.arrays.SparseArray(df['credit_score']))
```

Put simply, this is much more memory efficient, as the **SparseArray** function will store only the non-zero (or non-repetitive) values and a smaller number of positions of the **Not a Number (NaN)** or repeated values.

Working with chunks to perform operations

We have already learned in this section how chunks are useful for processing large data samples, but they are also exceedingly useful for performing operations on your data:

```
1. chunk_size = 100000
2. total_salary = 0
3. row_count = 0
4. for chunk in pd.read_csv('large_dataset.csv', chunksize=chunk_size):
5.     total_salary += chunk['salary'].sum()
6.     row_count += len(chunk)
7. mean_salary = total_salary / row_count
8. print(f'Mean salary: {mean_salary}')
```

Since this is a larger code block, it is explained in detail in the following points:

- We initialize the variables to store **chunk_size**, **total_salary** and **row_count**.
- We loop through each chunk in our read DataFrame, setting each iteration to the size of the chunk we will determine in **chunk_size**.
- The **total_salary** variable is used to add the salary of each chunk.
- **row_count** is set to the length of each chunk.
- Outside of the loop, we set **mean_salary** to divide the **total_salary** by the **row_count** variables.
- Finally, we print out a string to the console to say: **Mean salary: 90008.25**.

Parallel processing with Dask

Dask is a flexible, parallel computing library designed to scale computations across multiple cores or clusters, enabling efficient handling of large datasets that would otherwise exceed the memory capacity of a single machine. This will be useful, and would certainly speed up the loading of our **large_dataset** we have been working with throughout this section. Firstly, it must be installed:

```
1. pip install dask[dataframe] --quiet
```

From Dask you then have the option of importing the **dask.dataframe** module, to work with DataFrames. This is effective, as it breaks the dataset into chunks and processes them in parallel, this is where you see the speed up in computational processing time. To implement this on our dataset, we use:

```
1. import dask.dataframe as dd
2. df = dd.read_csv('large_dataset.csv')
3. mean_salary = df['salary'].mean().compute()
4. print(f'Mean salary: {mean_salary}')
```

This achieves the same function as the previous example, for using chunks to calculate the average salary.

Use Parquet format with PyArrow

PyArrow is a library that enables fast, in-memory data processing and efficient data transfer between different environments. This framework is part of the Apache Arrow framework, with a focus on columnar data storage. Parquet is an efficient columnar storage format designed for big data processing. It is optimized for performance through compression and the ability to retrieve columns at lightning speed.

Let us just call this combination the Big Data professionals' dream, and it will aid faster loading and processing of your larger files.

It can be installed through **pip install pyarrow** and then can be used to save and read data into a Pandas DataFrame:

1. `df.to_parquet('large_dataset.parquet')`
2. `df = pd.read_parquet('large_dataset.parquet')`

The first line saves to parquet format, and the second line is how you can read it in parquet format.

Vectorized operations over looping

Vectorization refers to the process of applying an operation to an entire array or column of data at once, without explicitly writing a loop. NumPy is the king of this, as we saw earlier, and is more performance-efficient than looping over a DataFrame to perform operations. This loop would take forever to execute on a larger dataset:

1. `df['new_col'] = [age * 2 for age in df['age']]`

A vectorized form of this would simply be:

1. `df['new_col'] = df['age'] * 2`

The difference between the two operations is that the vectorized form does the computations in bulk, whereas the list comprehension above loops

through each row of the DataFrame to do the computation. Bulk wins over line-by-line computation here. Therefore, all operations over DataFrames need to be vectorized when dealing with larger to big datasets. In saying that, both would result in the same result at the end, but the loop would be running ad infinitum on larger sets.

Using SQL-like queries

There is a package, namely PandaSQL: <https://pypi.org/project/pandasql/>, that enables SQL-like queries to be performed over Pandas DataFrames. This package is CPU hungry, however, so you need to make sure you have enough RAM on your system to effectively run this package.

To install the package:

```
1. pip install pandasql
```

Now we have implemented a script to look for the **Random Access Memory (RAM)** available on your PC or virtual machine. This function will literally print out the Total RAM available on the machine:

```
1. import psutil
2.
3. total_memory = psutil.virtual_memory().total
4. total_memory_gb = total_memory / (1024 ** 3)
5. print(f"Total RAM: {total_memory_gb:.2f} GB")
```

Once you have these variables, you can run the SQL statement, as shown in the following code:

```
1. import pandasql as psql
2. sql_like_query = """
3. SELECT TOP 100
4. name,
5. salary
6. FROM
```

```
7. df
8. WHERE
9. age > 30
10. ORDER BY
11. salary DESC
12. """
13. if total_memory_gb < 15:
14.     print('Not enough RAM to run this function. Please use High RA
    M environment')
15. else:
16.     result = psql.sqlldf(sql_like_query, globals())
```

This will check that you have more RAM than 15 GB. If you do not, then sadly, it will not run (better than your Jupyter Kernel crashes), but if it passes, then your SQL statement will execute and will return the results in a DataFrame.

Conclusion

This chapter covered the essential tools for data manipulation in Python, focusing on NumPy and Pandas. We began with NumPy, which is ideal for handling large, multidimensional datasets and performing fast mathematical operations, including indexing, slicing, and broadcasting. These skills are foundational for machine learning and scientific computing. Next, we explored Pandas, which builds on NumPy by providing more user-friendly data structures like DataFrames and Series. We focused on data cleaning, filtering, sorting, and handling missing values, which are crucial for preparing real-world datasets for analysis. Time series data preparation and big data strategies were also introduced. By the end of the chapter, you gained a strong understanding of working with NumPy and Pandas, setting the stage for more advanced topics like machine learning model building using real-world datasets. The next chapter will introduce you to scikit-learn.

Points to remember

Here are some key points summed up from this chapter:

- **NumPy** is a powerful Python library for numerical computing, specializing in handling large, multidimensional arrays and performing efficient mathematical operations, essential for managing data in machine learning workflows.
- **Pandas** offers user-friendly data structures, such as DataFrames and Series, designed to simplify tasks like data cleaning, filtering, sorting, and handling missing values, which are common in real-world datasets.
- **Time series** data in Pandas can be managed efficiently using functionalities such as resampling, filling in missing dates, and applying rolling window calculations, making it ideal for financial, forecasting, and temporal data analysis.
- **Handling large datasets** efficiently in Pandas can be achieved by using techniques like chunking, down casting data types, and saving data in Parquet format, significantly improving performance and memory usage.
- **Parquet format** offers an efficient way to store large datasets in a compressed, columnar format, enhancing read/write performance and reducing file sizes, which is ideal for big data tasks.
- **Dask** allows for parallel processing of large datasets, enabling computations across multiple cores or clusters, speeding up the data analysis process when dealing with big data.

Multiple choice questions

1. **What library in Python is primarily used for handling multidimensional arrays and is optimized for numerical computing?**
 - a. Pandas

- b. NumPy
 - c. Matplotlib
 - d. Scikit-learn
2. **What are the main data structures in Pandas for efficient data wangling?**
- a. List and tuples
 - b. Arrays and matrices
 - c. DataFrames and Series
 - d. Dictionaries and sets
3. **What is the key advantage of using vectorization?**
- a. It reduces memory usage by compressing data
 - b. It allows for faster computation by applying operations to entire arrays or columns at once
 - c. It enables reading and writing large datasets efficiently
 - d. It ensures data security in machine learning models
4. **What is the purpose of using chunking when reading large datasets in Pandas?**
- a. To improve file compression
 - b. To split data for parallel processing
 - c. To load and process large datasets in smaller, manageable pieces, without being a memory hog
 - d. To combine multiple datasets into one DataFrame
5. **Which format is recommended for efficiently storing big or large datasets in a compressed, columnar structure?**
- a. CSV
 - b. JSON
 - c. Excel

- d. Parquet
6. **What is the primary benefit of using Dask with large datasets?**
- It enables parallel processing across multiple cores or clusters for faster data handling
 - It reduces memory usage
 - It makes data more secure
 - It reduces data loading times
7. **In Pandas, what does the describe() function provide?**
- Row level detailed information
 - Column names and data types
 - Visual representation of the data
 - Summary statistics, such as mean, max, std, etc.

Answers

1	b
2	c
3	b
4	c
5	d
6	a
7	d

Key terms

Some key terms used in this chapter are:

- **Broadcasting:** A technique in NumPy that allows for operations on arrays of different shapes by stretching smaller arrays to match the larger ones.
- **Chunking:** A technique used in Pandas to process large datasets in smaller, manageable pieces, preventing memory overload.

- **Correlation matrix:** A matrix that standardizes covariance values, showing the strength and direction of relationships between variables, with values ranging from -1 to 1.
- **Covariance matrix:** A matrix that shows the covariance (degree of change) between different variables, useful for identifying relationships between features.
- **DataFrame:** A two-dimensional, tabular data structure in Pandas that allows for easy manipulation of structured data with rows and columns.
- **Dask:** A parallel computing library in Python used to scale computations across multiple cores or clusters, making it efficient for handling large datasets.
- **Downcasting:** The process of converting larger data types (e.g., int64) to smaller, more memory-efficient types (e.g., int8) in Pandas.
- **Indexing:** A method used in NumPy and Pandas to access specific elements, rows, or columns in arrays and DataFrames based on their position or label.
- **Matrix multiplication:** A mathematical operation where two matrices are multiplied to produce a new matrix, used heavily in linear algebra and machine learning.
- **Normalization:** The process of scaling data so that features are comparable, often done by dividing each feature by its magnitude in NumPy.
- **NumPy:** A Python library used for numerical computing, particularly efficient at handling multidimensional arrays and performing fast mathematical operations.
- **Pandas:** A powerful data manipulation library in Python that provides structures like DataFrames and Series to simplify data handling and analysis.

- **Parquet:** A columnar storage file format that allows for efficient reading and writing of large datasets, commonly used in big data tasks.
- **Resampling:** A technique in Pandas to change the frequency of time series data, such as aggregating daily data into monthly or yearly summaries.
- **Rolling window:** A method in Pandas used to apply statistical calculations like moving averages over a fixed window of time in time series data.
- **Series:** A 1-dimensional labeled array in Pandas, often used to represent a single column of data.
- **SVD:** A mathematical technique in NumPy used to decompose matrices into simpler components, often applied in dimensionality reduction.
- **Slicing:** A method in NumPy and Pandas used to extract portions of an array or DataFrame based on specified ranges of indices.
- **Time series:** A sequence of data points indexed by time, often used in finance and forecasting, which can be efficiently handled in Pandas.
- **Vectorization:** A method in NumPy and Pandas where operations are applied to entire arrays or columns at once, increasing computational efficiency.

Join our Discord space

Join our Discord workspace for latest updates, offers, tech happenings around the world, new releases, and sessions with the authors:

<https://discord.bpbonline.com>



CHAPTER 2

Introduction to Scikit-learn for Machine Learning

Introduction

This chapter serves as your gateway to scikit-learn, the definitive Python library for **machine learning (ML)**. With its robust tools for both supervised and unsupervised learning, scikit-learn simplifies the process of building and deploying ML models. The chapter begins with an overview of the library's structure and highlights its key features. From there, it gets into practical steps like loading datasets, preprocessing data, and splitting it for training and testing. You will explore essential ML techniques, including cross-validation and key algorithms for classification, regression, and clustering. To solidify your understanding, the chapter introduces evaluation metrics such as accuracy, precision, recall, and F1 score, culminating in a hands-on example of constructing a complete ML pipeline. By the end, you will be equipped with the foundational skills needed to harness scikit-learn for a variety of ML tasks.

Structure

This chapter covers the following topics:

- Understanding scikit-learn
- Primary use cases
- Role of scikit-learn in ML pipeline
- Setting up and installing scikit-learn

- Key concepts and usage in scikit-learn
- Creating your data to practice machine learning
- Breast cancer classification model development

Objectives

By the end of this chapter, readers will be able to confidently use scikit-learn to tackle fundamental ML tasks. They will understand how to load and preprocess datasets, ensuring data is prepared for analysis. Readers will gain hands-on experience in splitting data into training and testing sets, applying cross-validation techniques, and selecting appropriate ML algorithms for classification, regression, or clustering tasks. Additionally, they will be able to evaluate model performance using key metrics such as accuracy, precision, recall, and F1 score. Equipped with these skills, readers will be capable of building and executing a complete ML pipeline, applying powerful tools of scikit-learn to real-world problems with confidence and efficiency.

Understanding scikit-learn

Scikit-learn is an open-source library aimed at and designed to provide a suite of supervised and unsupervised learning algorithms in a Pythonic-style interface. The key features are explained in the following bullets:

- **Ease of use:** Scikit-learn is known for its straightforward, consistent **application programming interface (API)**, making it an excellent choice for experts and beginners starting on their tabular ML journey.
- **Versatile ML algorithms:** It includes a broad range of algorithms, such as linear regression, decision trees, clustering methods, support vector machines, and more. We will come back to this in the next section.
- **Preprocessing tools:** The package offers utilities for preprocessing data, such as normalization, data partitioning, and more. These will

be covered in the building of a binary breast cancer prediction model in this section, and we will expand on these as we go through each of the chapters of this book.

- **Model selection and evaluation:** Includes modules for model selection, hyperparameter tuning, and cross-validation, ensuring that the predictions from your models are as robust and accurate as they can be before putting them out *in the wild* or in production.
- **Plays well with other Python libraries:** Seamless integration and use between NumPy, Pandas, and Matplotlib, as well as other deep learning libraries.

Primary use cases

The primary use cases of these models are as follows:

- **Classification:** Predicting categorical labels (e.g., spam detection, sentiment analysis, whether someone will be admitted to the hospital, or any other dichotomous outcome).
- **Regression:** Predicting a continuous value (e.g., predicting house prices, income, etc.).
- **Clustering:** Finding hidden structures in unlabeled data (e.g., segmenting customers based on certain features, finding similar comorbidities among patients, etc.).
- **Dimensionality reduction:** Reducing the complexity of features in an ML model. Customarily used to avoid the curse of dimensionality, as explained in *Chapter 7, Handwritten Digits Dimensionality Reduction*.

Role of scikit-learn in ML pipeline

In a typical ML pipeline, scikit-learn tools are used at nearly every stage. The steps in most modern ML pipelines are shown in *Figure 2.1*:

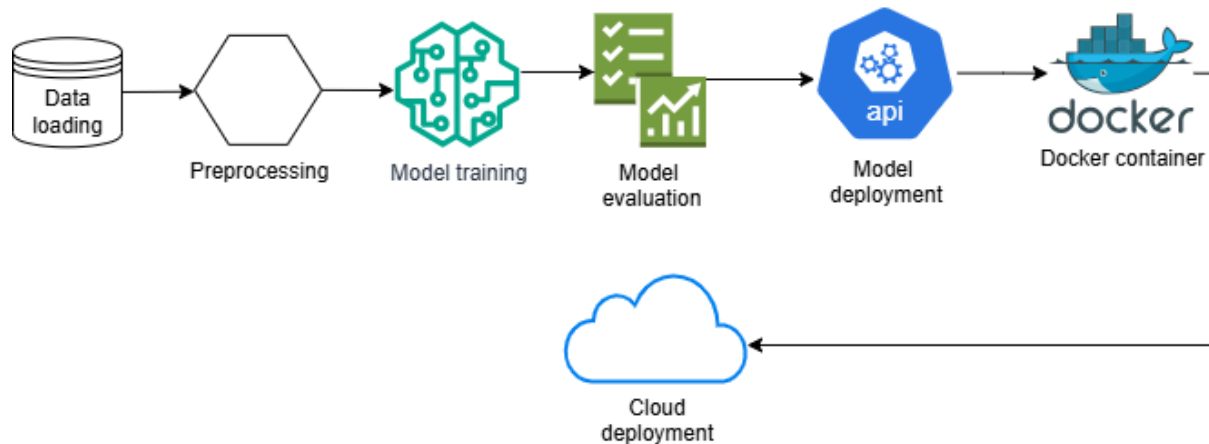


Figure 2.1: ML pipeline

In the next couple of sub-sections, we will explain where this can be especially useful.

Data collection and preprocessing

While data collection is generally outside of the scope of scikit-learn, once data is gathered for your modeling task, the tools provided by this package fall into these main categories:

- **Data cleaning:** Handling missing values and encoding categorical values with one-hot encoding, missing value imputation, etc.
- **Feature scaling:** Normalizing and standardizing data for better performance; we looked at a normalization strategy in *Chapter 1, Overview of NumPy and Pandas*.
- **Feature engineering:** Transforming data into suitable input forms. Remember, ML algorithms can only deal with numerical representations, so this is the art of feature engineering.

Once your data is adequately prepared, it is time to move on to model training. This step often requires substantial time and effort, as real-world datasets are rarely as pristine as the examples in ML tutorials. Throughout these chapters, we will gradually increase the complexity of feature engineering in each tutorial to better reflect the challenges of working with real data.

Modeling and training

The fun part, or perhaps the easy part, is with scikit-learn. The main algorithms available to you for regression, classification, and unsupervised learning are broken down into these subsections.

Regression algorithms

Here are some of the most popular regression algorithms:

- **Linear regression:** Predicts a continuous value (e.g., price) by fitting a straight line to the data. It is useful when predicting things like salary based on years of experience.
- **Ridge regression:** Prevents overfitting by penalizing extensive features with large coefficient values. Customarily used to limit the impact of irrelevant features or have many features that could overfit the model.
- **Lasso regression:** This is similar to Ridge but sets some coefficients to zero; it is used to automatically remove fewer essential features (known as feature elimination).
- **Elastic net:** Combines Ridge and Lasso penalties to handle features with high correlation; an example of a feature with high correlation would be age and weight.
- **Polynomial regression:** This method adds polynomial terms to capture non-linear relationships. It is normally used to predict non-linear sales over time.
- **Support vector regression (SVR):** SVR allows for a margin of error near the support vector (adapted for predicting continuous outcomes instead of classifying data. In SVR, the model tries to find a line, or hyperplane in higher dimensions, that best fits the data while maintaining a *margin of tolerance* around it) and is excellent for dealing with high dimensionality in data.

- **Decision tree regressor:** A decision tree used for predicting continuous outcomes. Simple for minor regression problems, but risks overfitting on small datasets.
- **Random forest regressor:** Many decision trees use an ensemble voting framework and, by doing this, reduce the risk of overfitting.

An exhaustive list of models is available on the scikit-learn page at https://scikit-learn.org/stable/supervised_learning.html.

Classification algorithms

Here are some of the most popular classification algorithms:

- **Logistic regression:** A method to predict dichotomous values like yes or no and estimate the probability of belonging to each category. It works well for linear relationships and helps when you have to explain the outcomes to a higher-up, such as in business or healthcare settings, where interpretability is key.
- **K-nearest neighbours (KNN):** Classifies an item by looking for the closest points (neighbors) and choosing the most common class amongst them. It is suitable for small to medium-type datasets.
- **Support vector machines (SVM):** Find a boundary that best separates different categories in a high-dimensional space. This is effective when you have many features (like image pixel embeddings) and need a clear margin between classes.
- **Decision trees:** It is a flowchart-like structure where each question splits the data until a prediction is made on a leaf. It can memorize and overfit the data if not controlled.
- **Random forest:** It comprises many decision trees where each tree votes on the outcome. The overfitting reduction technique is suitable for datasets of all sizes.
- **Gradient boosting:** It sequentially builds small models (usually trees) and focuses on correcting errors from the previous trees.

Gradient boosting is suitable for complex patterns in data, but it requires more computing. This type of algorithm is a Kaggle-beating monster.

- **Naive Bayes:** It uses probabilities based on Bayes' theorem to classify data, assuming each feature depends on others. Efficient and quick for spam filtering due to its simplicity and speed.
- **Neural network (MLPClassifier):** It is a basic neural network that finds complex patterns in data.

An exhaustive list of models is available on the scikit-learn page at https://scikit-learn.org/stable/supervised_learning.html.

Unsupervised algorithms

Here are some of the most popular unsupervised algorithms:

- **K-means clustering:** This algorithm groups data points into K clusters based on cluster similarity (computed using cluster centroid means).
- **Hierarchical clustering:** This algorithm builds a hierarchy diagram of clusters, which can then be split at different levels. Aids in exploring cluster structure and data with natural hierarchies.
- **Density-based spatial clustering of applications with noise (DBSCAN):** This algorithm groups points in dense areas while labeling sparse areas as noise. Effective for irregular shapes and noisy data, such as geographical points.
- **Principal components analysis (PCA):** This algorithm reduces data dimensions by finding directions with the most variance. Useful for visualization and feature engineering purposes.
- **t-distributed Stochastic Neighbor Embedding (t-SNE):** This algorithm projects high-dimensional data into 2D or 3D space while preserving local relationships between points. Typically used

in **exploratory data analysis (EDA)** and for visualizing complex structures.

- **Independent component analysis (ICA):** Separates mixed signals into independent sources. Used in image and audio processing when you need to isolate components
- **Gaussian mixture models (GMM):** This algorithm models data with a mix of Gaussian (normal) distributions. It is useful when clusters have different shapes and densities.
- **Agglomerative clustering:** This is a bottom-up approach to hierarchical clustering, merging clusters based on geometrical distance. It is effective for understanding nested clusters or cases where hierarchical grouping is needed.

An exhaustive list of models is available on the scikit-learn page at https://scikit-learn.org/stable/unsupervised_learning.html.

Evaluation and model selection

A critical feature of scikit-learn is its suite of options for model evaluation and selection; this includes the following:

- **Cross-validation:** Splitting data into training and test sets multiple times to assess model performance. We will go into this later on in building out our simple binary classifier.
- **Hyperparameter tuning:** Using functions like **GridSearchCV** and **RandomizedSearchCS** for model optimization.

Deployment

While scikit-learn does not offer direct deployment solutions, you can serialize your models with **pickle** (<https://docs.python.org/3/library/pickle.html>) or **jolib** (<https://joblib.readthedocs.io/en/stable/>), and this allows you to store your model artifacts in production to be used on unseen data.

Setting up and installing scikit-learn

To get started with scikit-learn, you will need Python (≥ 3.7) along with other dependencies like NumPy (<https://numpy.org/>), SciPy (<https://scipy.org/>), and Matplotlib (<https://matplotlib.org/>). Refer to the following command to install it:

```
1. pip install scikit-learn
```

Once installed, you can easily import it into your Python script by using the following command:

```
2. import sklearn
```

Now, let us look at some key concepts and usage in scikit-learn before we go on to build out the first classifier.

Key concepts and usage in scikit-learn

The package revolves around a few central concepts that allow users to efficiently build ML workflows. You no longer need to implement these from scratch; now, it is extremely simple.

Estimators

Estimators are the main objects for training models. Each algorithm in scikit-learn is an estimator with a **.fit()** method for training and a **.predict()** method for making predictions. Refer to the following code:

```
1. from sklearn.linear_model import LinearRegression
2. model = LinearRegression()
3. model.fit(X_train, y_train)
4. predictions = model.predict(X_test)
```

Transformers

Transformers, not to be confused with the **natural language processing (NLP)** model of the same name, are used to preprocess data. They also

follow the fit-transform convention with **.fit()** to learn the parameters and **.transform()** to apply the transformations. Refer to the following code:

```
1. from sklearn.preprocessing import StandardScaler
2. scaler = StandardScaler()
3. X_scaled = scaler.fit_transform(X)
```

Pipelines

Pipelines allow chaining multiple steps together (there is a section on this later on in the book) and allow you to dictate the order of execution of the steps. Refer to the following code:

```
1. from sklearn.pipeline import Pipeline
2. pipeline = Pipeline([
3.     ('scaler', StandardScaler()),
4.     ('regressor', LinearRegression())
5. ])
6. pipeline.fit(X_train, y_train)
```

Evaluation metrics

Scikit-learn offers several metrics to evaluate your models' performance, such as accuracy for classification and **mean square error (MSE)** for regression. We will use several of these in the hands-on section later in this chapter. For example:

```
1. from sklearn.metrics import accuracy_score
2. accuracy = accuracy_score(y_true, y_pred)
```

Creating your data to practice machine learning

Scikit-learn has two very handy functions for the creation of datasets for regression, classification, and unsupervised ML. These two functions are:

- For generating regression data, use: **make_regression()**.

- For generating classification and unsupervised data, use: **make_blobs()**.

In the upcoming sub-sections, we will be looking at three simple examples of how you can generate data to start modeling straight away.

Creating a regression dataset

For regression, we will use **make_regression()** from **sklearn.datasets** to simulate a dataset with a continuous output. Refer to the following code:

```
1. from sklearn.datasets import make_regression
2. import matplotlib.pyplot as plt
3. X, y = make_regression(n_samples=100,
4.                        n_features=1,
5.                        noise=10,
6.                        random_state=42)
```

The fields used to generate this are explained as follows:

- **Parameter n_samples:** The number of data points to generate.
- **Parameter n_features:** The number of features (independent variables).
- **Parameter noise:** The standard deviation of the Gaussian noise added to the output.
- **Parameter random_state:** Ensures reproducibility of random values, also called a seed value.

This creates data distributed, as shown in *Figure 2.2*:

- Fixes the randomness for reproducibility **random_state**.

This code will generate a distribution, as illustrated in *Figure 2.3*:

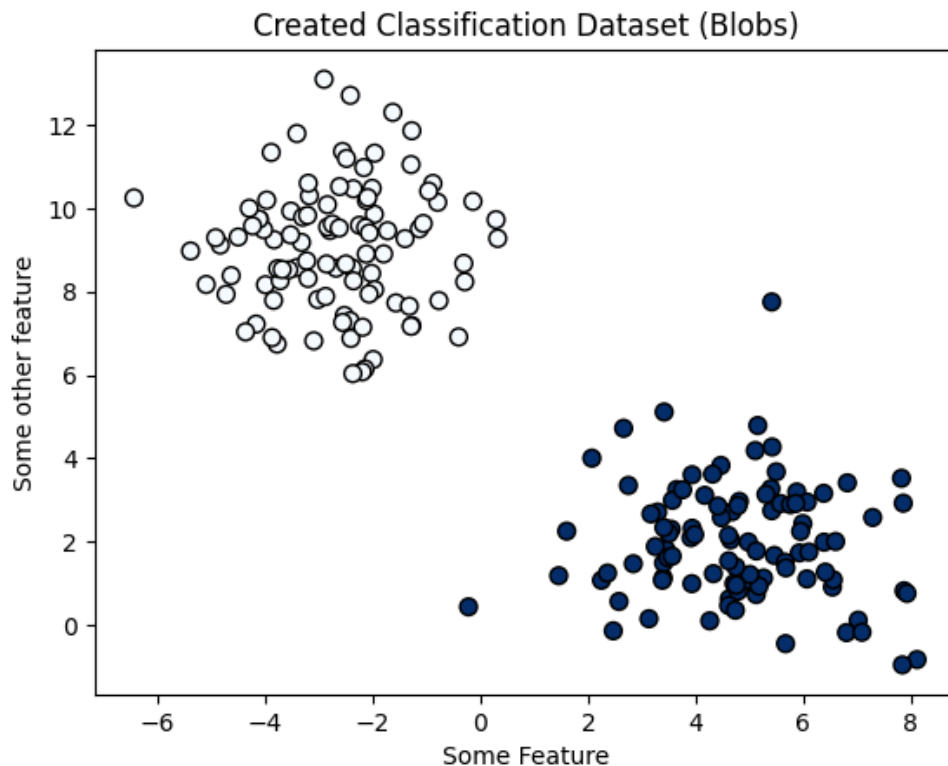


Figure 2.3: Simulated classification data

Creating an unsupervised dataset for clustering

For this task, we will use the trustworthy **make_blobs()** function to generate this, and the process is very similar to that in the prior subsection. Refer to the following code:

```
1. from sklearn.datasets import make_blobs
2. X, _ = make_blobs(n_samples=300,
3.                   centers=4,
4.                   cluster_std=1.0,
5.                   random_state=42)
```

The parameters for the preceding code are the same as those we used in the classification subsection. This creates a distribution, as illustrated in *Figure*

2.4:

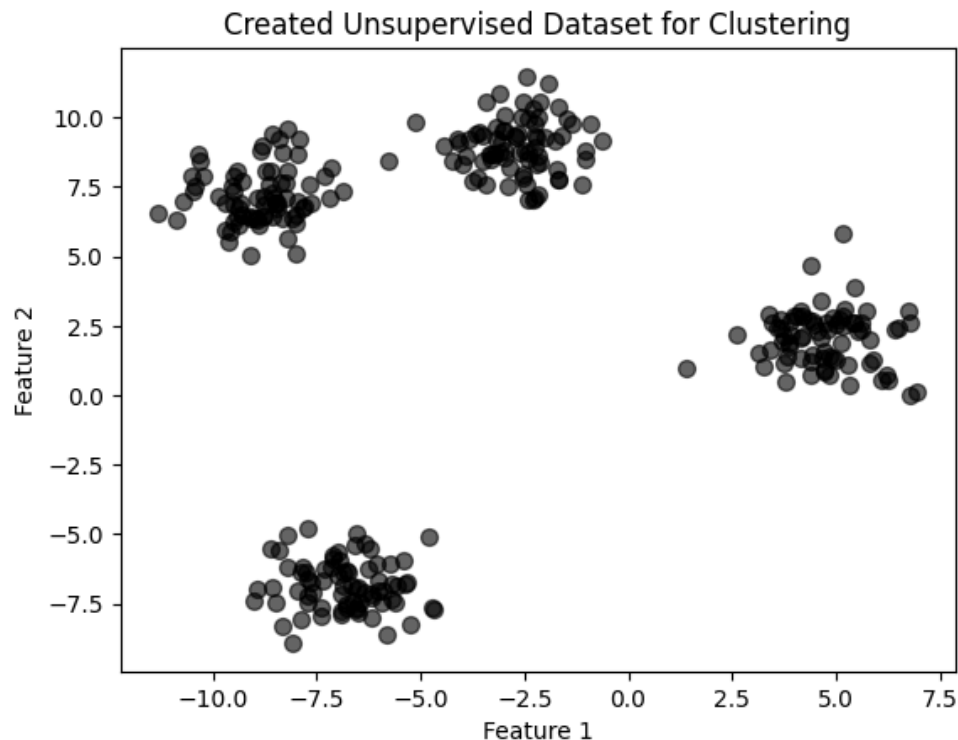


Figure 2.4: Simulated unsupervised clustering dataset

Now that you have learned what scikit-learn has in store for you, we will get on to putting our skills to work in building out a binary classifier for breast cancer detection. However, what has been covered in this section will allow you to hone your skills on regression, classification, and unsupervised ML problems by generating the datasets with the aforementioned functions.

Breast cancer classification model development

The first step in this process is bringing in the imports that we will need to work with on this project. These are defined as follows:

1. `import sklearn`
2. `import pandas as pd`
3. `import seaborn as sns`
4. `import matplotlib.pyplot as plt`
5. `from sklearn.datasets import load_breast_cancer`

The *Wisconsin Breast Cancer* dataset is widely used for binary classification tasks aimed explicitly at distinguishing between benign and malignant breast tumors. It includes 569 samples, each described by 30 features derived from images of breast cell nuclei, such as radius, texture, perimeter, area, and concavity. Each feature is recorded in three forms: mean, standard error, and worst value, providing a detailed profile of each cell sample's characteristics.

Labeled as *M (malignant)* or *B (benign)*, this dataset is well-suited for training and testing classification models due to its balanced classes and minimal missing data. It has become a go-to resource in ML and medical research, enabling researchers to test algorithms and improve model accuracy for breast cancer detection.

The *Wisconsin Breast Cancer* dataset is available within scikit-learn's datasets package, and most ML datasets, including this one, are sourced from the *UCI Machine Learning Repository* (at <https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>), or can alternatively be accessed on Kaggle).

Getting our data

In this subsection, we will look at using scikit-learn datasets to import the *Wisconsin Breast Cancer* data from the scikit-learn library. Then, we will convert it to a DataFrame, export the data to **comma-separated values (CSV)**, and read it back in as a CSV to demonstrate the process of saving and loading ML datasets.

Using scikit-learn datasets

The following steps describe how to load the in-built datasets in scikit-learn into your environment:

1. First, load the breast cancer data into a variable:

```
1. sci_kit_internal_data = load_breast_cancer()
```

2. Next, convert the data to a Pandas DataFrame:

1. `breast_cancer_data = pd.DataFrame(sci_kit_internal_data.data,`
2. `columns=sci_kit_internal_data.feature_names)`
3. These in-built scikit-learn datasets contain the features and the target (outcome variable) in separate properties, so this will only give the features.
4. Further, we will need to add a column called **Diagnosis** to the DataFrame:

```
1. breast_cancer_data['Diagnosis'] = sci_kit_internal_data.target
```

Export dataset to CSV

From here, we will export the file loaded and morphed from the previous steps as follows:

```
1. breast_cancer_data.to_csv('breast_cancer_data.csv', index=False)
```

In the next section, the final step will be to import this back into our environment.

Read CSV file

From here, we will export the file loaded and morphed from the previous steps as follows:

```
1. bc_data = pd.read_csv('breast_cancer_data.csv')
```

We now have a stored CSV file that was converted from an internal dataset file to work with for the rest of my project. We used this as an example to show you how to use internal datasets, modify them, export them, and import them back in again.

Checking class imbalance

Checking for class imbalance is essential in classification tasks because an imbalanced dataset can lead to models that perform poorly on the minority class, often the one with the most critical predictive importance, such as

detecting fraud or diagnosing rare diseases. When the majority class dominates, the model may become biased, predicting the majority class more frequently, and achieving deceptively high accuracy while neglecting the minority class. Techniques like resampling or adjusting class weights help address this issue by giving the model a balanced perspective during training.

Furthermore, using evaluation metrics like precision, recall, and F1 score, rather than relying on accuracy alone, ensures a clearer understanding of the model's performance in each class, making it more reliable and effective in real-world applications.

We will deal with class imbalance using **Synthetic Minority Oversampling Technique (SMOTE)**, **Random Oversampling Examples (ROSE)**, and other techniques in later chapters. Still, it is good practice to check for imbalance when starting all supervised binary classification projects.

To check for class imbalance, we can use the **value_counts()** function in Pandas:

```
1. diagnosis_counts = bc_data['Diagnosis'].value_counts()
2. diagnosis_percentages = diagnosis_counts / len(bc_data) * 100
3. print(diagnosis_counts, diagnosis_percentages)
```

This will print out the result; to visualize, we convert it to a DataFrame and use Seaborn to visualize the class distribution:

```
1. diagnosis_df = diagnosis_counts.reset_index()
2. diagnosis_df.columns = ['Diagnosis', 'Count']
1. sns.set(style="whitegrid")
2. plt.figure(figsize=(8, 6))
3. ax = sns.barplot(data=diagnosis_df,
4.                  x='Diagnosis',
5.                  y='Count',
```

```
6.             hue='Diagnosis',
7.             palette='muted',
8.             legend=False)
9. plt.title('Class Distribution of Breast Cancer Diagnosis', fontsize=14)
10. plt.xlabel('Diagnosis', fontsize=12)
11. plt.ylabel('Count', fontsize=12)
12. plt.show()
```

Upon running this code, we get a bar chart, as shown in *Figure 2.5*:

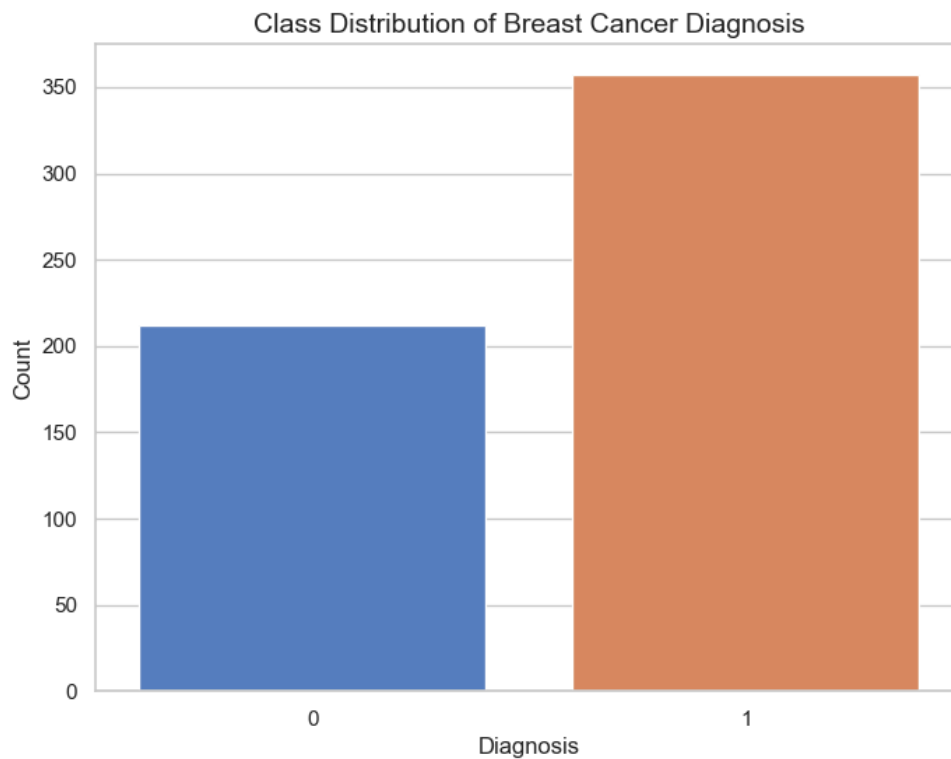


Figure 2.5: *Class imbalance*

Since the dataset has a slight imbalance, with 62.7% malignant and 37.3% benign samples, we must carefully choose evaluation metrics, as accuracy alone can be misleading when classes are not equally represented. In an imbalanced dataset, a model could achieve high accuracy simply by favoring the majority class, in this case, malignant, while failing to

correctly identify the minority class, benign, which could lead to critical misclassifications. While we would not cover rebalancing techniques here, selecting appropriate metrics like precision, recall, and F1 score can provide a more balanced view of the model's performance across both classes.

This approach is particularly relevant in fields with significant class imbalances, like fraud detection, where only a tiny fraction of transactions are fraudulent. In such cases, focusing on accuracy could mask the poor performance of the minority class (fraudulent cases), so metrics that emphasize how well the model detects the minority class become essential for reliable results.

Handling missing values

The first thing to do when handling missing values is to quickly check over all your columns where there are NULL records. Refer to the following code:

```
1. bc_data.isnull().sum()
```

When printing this, you will see each field with a summary of the missing or NULL values in the DataFrame. For this example, as we are working with a relatively clean dataset (do not worry, we will come on to more real-world datasets later on in this book), there are no NULL values in the dataset, as the analysis outputs. However, if you wanted to fill any **NULL values (NAs)**, you could either:

Fill the null values with a median replacement scheme. Refer to the following code:

```
bc_data.fillna(bc_data.median(), inplace=True)
```

You could alternatively drop NULL values if there are any rows that contain NAs in the dataset. Although this could have the effect of reducing your training sample:

```
bc_data.dropna()
```

Scaling numerical features

The benefits of normalizing your data before training your models cannot be overstated. Generally, it aids the following:

- **Speeds up convergence for gradient-based algorithms:** Scaling helps models that use gradient-based optimization (like linear regression) find the best solution faster by balancing the feature sizes.
- **Ensures stability in optimization by balancing feature ranges:** When features have similar ranges, models can train more consistently without getting stuck or bouncing around during training.
- **Improves accuracy for distance-based models (e.g., k-nearest neighbors, SVM):** For models that calculate distances, scaling ensures no single feature unfairly dominates the calculations, leading to more accurate results.
- **Prevents feature dominance in distance calculations:** Scaling keeps all features on equal footing, so large-value features do not overpower smaller ones, especially important in distance-based methods.
- **Balances regularization effects, enhancing interpretability:** In models with regularization (like Ridge or Lasso regression), scaling prevents large-value features from skewing the penalties, making the model's output easier to understand.
- **Reduces computational load, leading to faster model training:** Scaled features often reduce the number of calculations, making the training process quicker and more efficient.
- **Standardizes input ranges across pipeline steps for robustness:** In complex workflows (pipelines), scaling ensures each model step receives inputs in a consistent range, which helps avoid unexpected errors.

- **Allows models to generalize better by normalizing feature influence:** When features are scaled, the model can learn general patterns instead of over-focusing on large-value features, which leads to better generalization.
- **Makes model training more efficient on large or complex datasets:** With scaled features, the model handles large datasets more smoothly, optimizing memory and computational use.
- **Provides consistency in feature impact across various algorithms:** Scaling keeps feature impact consistent across different algorithms, ensuring that no single feature overly influences the model's behavior.

Scikit-learn makes it easy for you to perform various scaling. In this example, we will use the **StandardScaler()** that standardizes features by removing the mean and scaling to unit variance, which means it transforms data to have a mean of 0 and a standard deviation of 1, making features comparable in scale and achieving the benefits that are listed in the bullets. To scale our data, we need to instantiate (make reference to) the class **StandardScaler()** and set it to an instance of the class, namely, scaler. Refer to the following code:

```
1. from sklearn.preprocessing import StandardScaler
2. scaler = StandardScaler()
```

From here, as we still have the **Diagnosis** column in our dataset, we need to drop the reference column and just create a DataFrame of the numerical features. Refer to the following code:

```
1. num_features = bc_data.columns.drop('Diagnosis')
```

As we learned at the start of this chapter, we will use the **.fit()** estimator to fit the scalar to the numerical features in our dataset. Refer to the following code:

```
1. bc_data[num_features] = scaler.fit_transform(bc_data[num_features])
```

This will transform the **num_features** DataFrame by sub-setting the **bc_data** and then passing this through the **fit_transform()** method (essentially, to differentiate between fit and **fit_transform**). These, as we mentioned earlier, will then be transformed into values close to the mean and values that deviate from the mean. Refer to *Figure 2.6*:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension
0	1.097064	-2.073335	1.269934	0.984375	1.568466	3.283515	2.652874	2.532475	2.217515	2.255747
1	1.829821	-0.353632	1.685955	1.908708	-0.826962	-0.487072	-0.023846	0.548144	0.001392	-0.868652
2	1.579888	0.456187	1.566503	1.558884	0.942210	1.052926	1.363478	2.037231	0.939685	-0.398008
3	-0.768909	0.253732	-0.592687	-0.764464	3.283553	3.402909	1.915897	1.451707	2.867383	4.910919
4	1.750297	-1.151816	1.776573	1.826229	0.280372	0.539340	1.371011	1.428493	-0.009560	-0.562450
...	...	...	...	...	...	...	...	...	...	...
564	2.110995	0.721473	2.060786	2.343856	1.041842	0.219060	1.947285	2.320965	-0.312589	-0.931027
565	1.704854	2.085134	1.615931	1.723842	0.102458	-0.017833	0.693043	1.263669	-0.217664	-1.058611
566	0.702284	2.045574	0.672676	0.577953	-0.840484	-0.038680	0.046588	0.105777	-0.809117	-0.895587
567	1.838341	2.336457	1.982524	1.735218	1.525767	3.272144	3.296944	2.658866	2.137194	1.043695
568	-1.808401	1.221792	-1.814389	-1.347789	-3.112085	-1.150752	-1.114873	-1.261820	-0.820070	-0.561032

569 rows × 30 columns

Figure 2.6: Scaled values with StandardScaler()

Split features and labels

It is best practice to keep a copy of the features and my target feature in separate variables, to use as the basis of splitting my data in the next section. This can be done as follows:

1. `X = bc_data.drop('Diagnosis', axis=1)`
2. `y = bc_data['Diagnosis']`

This creates our features (denoted as X) and our target value (Y). We are essentially trying to predict the cancer diagnosis based on features X.

Create our train and test partitions

We are very close to model training, but before we do that, we create a train-testing split to train on the proportion of data using the training set and

then test what this would perform like in production on a testing set. Generally, when considering a proportion to split by, this is where the heuristics of ML come in; we choose three approaches:

- **80% for training and 20% for testing:** This is a balanced tradeoff between having enough data for training the model while giving a large enough sample to test/evaluate our model.
- **90% for training and 10% for testing:** This approach does indeed give more data for training, but with a smaller proportion for testing, we can be less confident about how the model will perform in production or in the ML wilderness.
- **70/30 split:** Though this split helps us out when testing, it limits the training data and can be disadvantageous when your dataset is small.

Later on in this chapter, we will cover what to do with smaller samples and will introduce a concept called cross-validation splitting. However, for now, we will go with an 80/20 split ratio. To implement this in scikit-learn, there is a handy function, namely **train_test_split**. Refer to the following code:

```
1. from sklearn.model_selection import train_test_split
2. split_prop = round(float(1 - 0.8),1)
3. X_train, X_test, y_train, y_test = train_test_split(
4.     X,
5.     y,
6.     test_size=split_prop,
7.     random_state=42)
```

Let us understand this code in detail:

- We import the **train_test_split** method to use for splitting.
- Create a variable called **split_prop** that uses the **round()** function to round down to 1 decimal place, make sure the data type is a

floating-point number, and then take the inverse of the training proportion (0.8) to return the residual 0.2 (20% of data to be left for testing).

- We then use multiple variable assignments (known as multiple assignment) to set the variables **X_train**, **X_test**, **y_train** and **y_test**, in which we pass parameters X (features), y (target), **test_size=split_prop** (our proportion for splitting) and we use a **random_state** to make the results reproducible, in theory, you would remove this in production, as you may not want the same splits every time, but it is kept in when making tutorials.

Finally, we want to find the length of our new splits and print out the sizes of the sets and their stored data types:

```
print(f'The size of the training set is: {len(X_train)} records and is  
now a Numpy array = : {type(X_train)}')
```

```
print(f'The size of the test set is: {len(X_test)} records and is now a  
Numpy array = : {type(X_test)}')
```

The output of the first print statement is: The size of the training set is: 455 records and is now a **Numpy array = : <class 'numpy.ndarray'>**, and the same with the test set, albeit that is 114 records.

Modelling

In this section, we will look at two approaches to modeling with our existing splits and preprocessed data:

- Fitting one classifier on our task as a benchmark classifier, we usually start with logistic regression for this purpose.
- Fitting multiple classifiers on the task and creating a custom benchmarking approach. This is preferable when you want the flexibility of various classifiers for the task.

The following sub-sections will look at both approaches.

Fitting one classifier

In this section, we will focus on fitting one classifier to the task to get you familiar with the steps. In the next section, we will build a custom classification loop and loop through a dictionary of classifiers. For now, follow these steps.

We first need to instantiate our model variable and set the maximum iterations to 1000. For all the parameters to pass to this specific model, see:

https://scikitlearn.org/1.5/modules/generated/sklearn.linear_model.LogisticRegression.html

The **max_iter** parameter essentially says try 1000 iterations to get the best boundaries over 100 iterations and then stop. Take a look at the following steps:

1. To implement this, refer to the following code:

```
model = LogisticRegression(max_iter=1000)
```

2. Fit the model to the training data:

```
model.fit(X_train, y_train)
```

3. Use the fitted model, in step 2, to make class predictions on (please note the default classification threshold in scikit-learn is 0.5, meaning probabilities ≤ 0.5 will be classified as 0 and those > 0.5 will be labelled as 1):

```
class_preds = model.predict(X_test)
```

4. This will fit the class labels as 0 or 1, respectively, and is returned as a NumPy array.
5. Use the fitted model to get the class probabilities (this is where you can create a column with a custom threshold, if required, such as if threshold ≥ 0.7 , then 1 else 0):

```
prob_preds = model.predict_proba(X_test)
```

6. Finally, to get a quick model accuracy score, we use **model.score()** to return these; however, for imbalanced data, this would not be useful, as the base evaluation metric is accuracy:

```
model.score(X_test, y_test)
```

That is how simple it is to fit an ML model in scikit-learn's API. However, in experiments, you rarely test one classifier; in the next section, we will look at fitting multiple candidate classifiers at once and then evaluating them with some matplotlib wizardry.

Fitting multiple candidate classifiers

As seen in the *fitting one classifier* steps, we will approach the model fitting stepwise. We will be using evaluation metrics in this section, but we will explain these measures in the evaluation section. Please note that the higher the value with classification metrics, the better the model fit. Follow these steps:

1. First, let us create a dictionary structure called **my_classifiers** to store the key-value pairs of the classifiers. The key will be the classifier name, and the value will be the actual command to pass to scikit-learn:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

my_classifiers = {
```

```

    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Random Forest": RandomForestClassifier(random_state=42),
    "SVM": SVC(probability=True, random_state=42),
    "KNN": KNeighborsClassifier(),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Naive Bayes": GaussianNB(),
    "LDA": LinearDiscriminantAnalysis()
}

```

2. Now, we will create a second dictionary with empty lists initialized to store the results:

```

model_results = {
    "Model": [],
    "Accuracy": [],
    "Precision": [],
    "Recall": [],
    "F1 Score": [],
    "Training Time (s)": [],
    "Predictions": [],
    "Probabilities": []
}

```

3. Next, let us get the imports we will need for evaluating the models and to track how long each model takes to train for benchmarking the speed of training alongside the respective performance metrics:

```

from time import time

```

```
from sklearn.metrics import (precision_score, recall_score, f1_score,
                             roc_auc_score, accuracy_score)
```

4. Time to get looping. We will explain each step under the code block:

```
for model_name, model in my_classifiers.items():
```

```
    start_timer = time()
```

```
    model.fit(X_train, y_train)
```

```
    end_model_timer = time()
```

```
    y_pred = model.predict(X_test)
```

```
    y_prob = model.predict_proba(X_test)
```

```
    accuracy = accuracy_score(y_test, y_pred)
```

```
    precision = precision_score(y_test, y_pred)
```

```
    recall = recall_score(y_test, y_pred)
```

```
    f1 = f1_score(y_test, y_pred)
```

```
    train_time = end_model_timer - start_timer
```

```
    model_results["Model"].append(model_name)
```

```
    model_results["Accuracy"].append(accuracy)
```

```
    model_results["Precision"].append(precision)
```

```
    model_results["Recall"].append(recall)
```

```
    model_results["F1 Score"].append(f1)
```

```
    model_results["Training Time (s)"].append(train_time)
```

```
    model_results["Predictions"].append(y_pred)
```

```
    model_results["Probabilities"].append(y_prob)
```


The code performs the following:

- Loops through **model_name** and model and selects that from the **my_classifiers** dictionary by using the **.items()** method.
 - Sets a **start_timer** variable to capture the current system time.
 - Fits the model on the **X_train** and **y_train** arrays.
 - Ends the timer for the fitting of that specific model.
 - Predicts the class labels (predict) and predicts the class probabilities (**predict_proba**).
 - Calculates the evaluation metrics, which we will discuss further in a later section, such as accuracy, precision, recall, and the harmonic mean (F1 score).
 - **train_time** is the net time taken to run between the start and end timers.
 - Then we append each of the results to values, matching the relevant keys and empty lists we set up in *Step 2*.
5. Let us create a copy of the model scores obtained and append to each of the lists; that way, we can refer back to them. We will remove the predictions and probabilities keys from the **model_results_summary**, as we only want the associated metrics that have been stored, i.e., recall, precision, etc.:

```
model_scores_summary = model_results.copy()
```

```
model_scores_summary.pop('Predictions', None)
```

```
model_scores_summary.pop('Probabilities', None)
```

6. The very last step is to then save these back as a DataFrame, as they are still in NumPy arrays:

```
results_df = pd.DataFrame(model_scores_summary)
```

This produces the following DataFrame, when we print the output of the **results_df**.

Refer to the following figure:

Model	Accuracy	Precision	Recall	F1 Score	Training Time (s)
Logistic Regression	0.973684	0.972222	0.985915	0.979021	0.022569
Random Forest	0.964912	0.958904	0.985915	0.972222	0.524491
SVM	0.973684	0.972222	0.985915	0.979021	0.035680
KNN	0.947368	0.957746	0.957746	0.957746	0.003830
Decision Tree	0.947368	0.957746	0.957746	0.957746	0.018694
Naive Bayes	0.964912	0.958904	0.985915	0.972222	0.007314
LDA	0.956140	0.945946	0.985915	0.965517	0.017929

Figure 2.7: Model results DataFrame

Evaluation metrics and visualization of results

In the previous section, we used various evaluation metrics, with little explanation of what they actually tell you. In the next section, we will visualize model results using **receiver operating characteristic** (ROC) curves and confusion matrices, but in this section, we will take a statistical approach.

Recall and precision

Recall, also called sensitivity or true positive rate, measures the model's ability to correctly identify all positive samples; for our use case, we have been working with all the malignant cancers. The metric is the proportion of actual positives that were correctly predicted as positive; refer to the following equation:

$$\text{Recall} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}}$$

In *Chapter 3, Supervised Binary Classification*, we will explore **True Positives (TPs)**, **False Negatives (FNs)**, **False Positives (FPs)**, and **True Negatives (TNs)** when evaluating confusion matrices. TP is a case where the model correctly identifies a malignant cancer, matching the ground truth label. FN, on the other hand, occurs when the model incorrectly predicts a benign (no cancer) result while the actual ground truth label indicates cancer.

FN is undoubtedly one of the metrics we need to get as close to perfect as possible, as when the model comes to make new predictions, without knowing the ground truth, it could predict no cancer, and the patient would be discharged with cancerous cells still growing, which could lead to adverse outcomes.

Precision, also known as **positive predictive value (PPV)**, measures the accuracy of optimistic predictions. It is the proportion of predicted positive outcomes that are positive. Refer to the following:

$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

FP are also something you are going to want to optimize in your model for the breast cancer use case, as we are predicting that someone has breast cancer when they do not, meaning this could lead to unnecessary procedures for cancer removal when they just are not needed. Sometimes, if you are not creating models for medical issues that need great accuracy, you might be okay with a few FPs. Still, for this use case, it certainly would not be acceptable from a clinical level.

F1 score

F1 score is the harmonic mean of precision and recall, providing a single metric that balances both. This is the go-to metric when you have a class imbalance, as we have in this use case, as the positive class has more examples. The equation for this is as follows:

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

A high F1 score indicates a good balance between precision and recall, which is ideal in scenarios like ours, where knowing about FP and FNs is important, as explained in the previous section.

Accuracy

Accuracy measures the proportion of correct total predictions (both positive and negative). The equation is shown in

$$\text{Accuracy} = \frac{\text{True Positives (TP)} + \text{True Negatives (TN)}}{\text{Total Predictions (TP + TN + FP + FN)}}$$

Accuracy works well when classes are balanced, but can be misleading in imbalanced datasets. For example, a model that always predicts the majority class (our use case malignant) will have high accuracy but may fail to detect the minority class (benign diagnoses).

Negative predictive value

Negative predictive value (NPV) measures the proportion of true negatives out of all predicted negatives. It shows how often a pessimistic prediction is actually correct. Refer to the following:

$$\text{NPV} = \frac{\text{True Negatives (TN)}}{\text{True Negatives (TN)} + \text{False Negatives (FN)}}$$

A high NPV means that when the model predicts a negative, it is likely to be correct, which is useful in cases where false negatives are particularly

harmful, such as when predicting that someone has cancer when they do not.

Visualizing model results

In the *Modelling* section, we created a **results_df** to store our results from our multiple model training evaluations. The DataFrame is an okay way to present this, but the code ahead uses Matplotlib (<https://matplotlib.org/>) to visualize the results as a paneled bar chart.

The function is implemented in the supporting GitHub repository linked to this chapter under **2.2_Breast_Cancer_Binary_Classification.ipynb**. To run the function, use the following code:

1. `multi_model_visualizer(results_df,`
2. `colors=["#2b68a5", "#1af298", "mediumseagreen", "mediumpurple", "gold"])`

The resultant output is shown in the following figure:

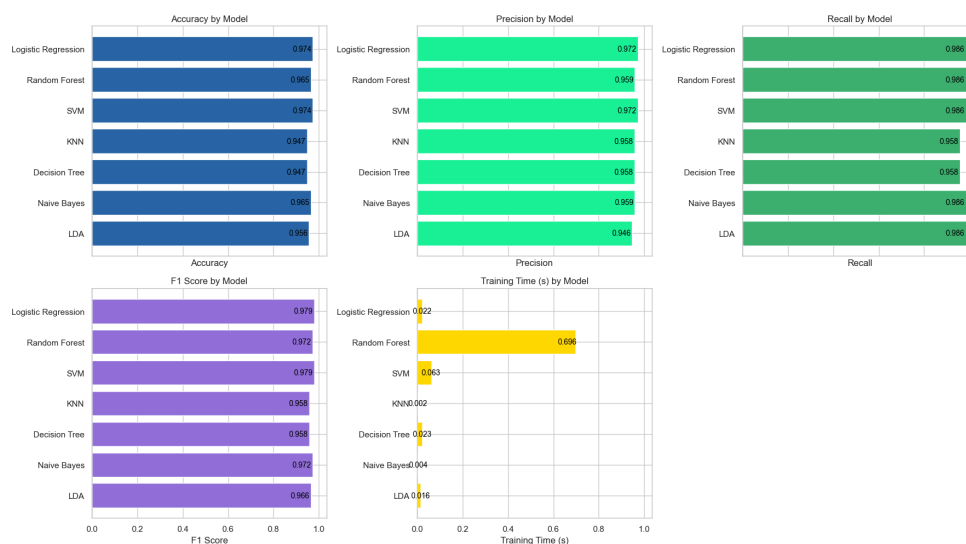


Figure 2.8: Visualized model results

From these results, we observe the following:

- Logistic regression performs well in both precision and recall, achieving the highest F1 score among the models.

- The random forest algorithm requires significantly more training time, which is expected since it is an ensemble method that grows multiple decision trees, as discussed earlier in this chapter.

In the next section, we will analyze these models statistically to select the best one based on performance metrics.

Model selection

The final step is to select the best model from our candidates. You can approach this in various ways, like choosing the model with the highest precision, recall, accuracy, F1 score, or another metric of interest. We will create a Python function to handle this comparison automatically, which you can reuse in your own projects after completing this book.

To define the function, we will perform the following:

- Loop through a list of metrics we want to evaluate.
- Use **idxmax()** to find the index of the model with the best score for each metric.
- Apply **loc** to retrieve the model ID from the **idxmax()** result.
- Print the top model for each metric and return these best models.

Here is how to implement this:

```
1. def get_best_models_by_metric(results_df, metrics):
2.     best_models = {}
3.     for metric in metrics:
4.         best_model_index = results_df[metric].idxmax()
5.         best_model_name = results_df.loc[best_model_index, "Model"]
6.         best_models[metric] = best_model_name
7.         print(f"Best model based on {metric}: {best_model_name}")
```

8. `return best_models`

To run this function against our **results_df** we created earlier, we will pass the metrics of interest as a list and then run the **get_best_models_by_metric** function. Refer to the following code:

```
1. metric = ['Accuracy', 'Precision', 'Recall', 'F1 Score']
2. best_models = get_best_models_by_metric(results_df, metric)
```

After running this, for our use case, you will find that the print model statements seem to indicate that: **Best model based on [metric]: Logistic Regression**

We could automate this further and look for a modal selection of the response, i.e., select the model that outputs the most over those metrics, as here the model selection would be logistic regression because it performs the best over every metric. This approach will fail if you have different models that are better at different use cases, but here, you could implement something like the following code:

```
1. from collections import Counter
2. def find_modal_model(best_models):
3.     model_counts = Counter(best_models.values())
4.     modal_model = model_counts.most_common(1)[0][0]
5.     print(f"Modal (most frequently best) model across metrics: {modal_model}")
6.     return modal_model
```

To run this function, we would then use:

```
1. modal_model = find_modal_model(best_models)
```

This would output a modal (most frequently best) model across metrics: **Logistic Regression.**

These are relatively simple approaches to selecting your model by eyeballing the results. In the next section, we will discuss K-fold cross

validation and how it can be extremely useful from a model selection stance.

K-fold cross validation

K-fold cross validation is a robust technique for assessing model performance, especially useful with smaller datasets where maximizing data usage is crucial. In K-fold cross validation, the dataset is divided into **K** equally sized *folds* or subsets. For each of the **K** iterations, one unique fold is set aside as the test set, while the remaining **K-1** folds serve as the training set. This rotation continues until every fold has served as the test set exactly once.

The model's performance scores from each fold are averaged, providing a more comprehensive and stable estimate of its generalization ability. This approach is particularly advantageous with limited data, as it allows each data point to be used for both training and validation. This leads to less variability in performance metrics and helps mitigate the risk of overfitting, giving a more reliable evaluation than a simple train-test split.

Figure 2.9 shows the process for how K-fold partitions the training dataset from the test dataset, and this does it in more of an iterative sampling fashion:

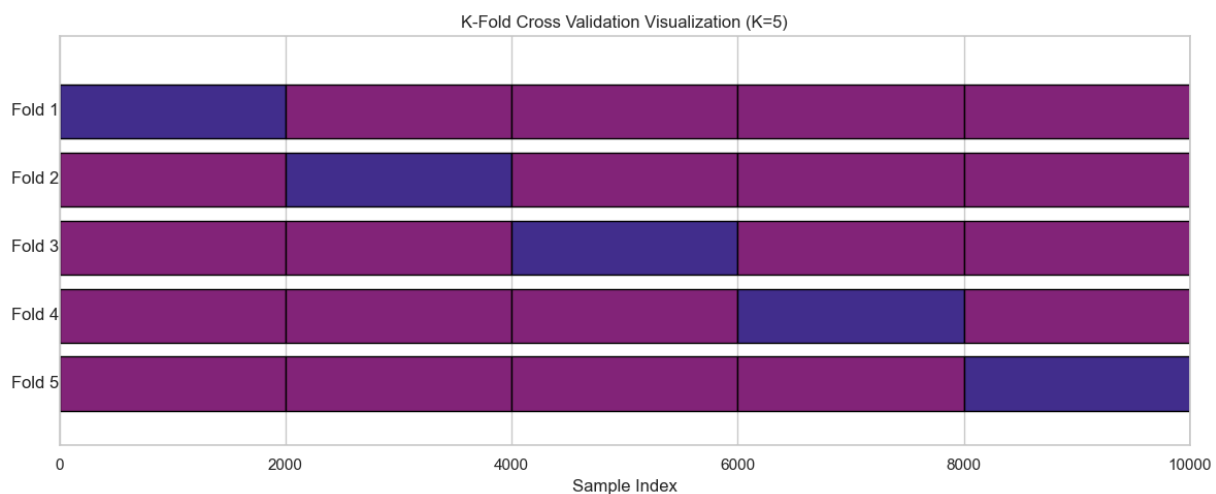


Figure 2.9: K-fold cross validation partitions/splits

In the next section, we will use our best-performing model from the previous step (logistic regression) to perform K-fold cross validation. We will then evaluate the folds based on a custom training loop and using in-built scikit-learn functions.

Getting started

In the following steps, we will set up our data and fit the logistic regression model:

1. Define our imports:

```
from sklearn.model_selection import train_test_split, KFold
```

2. Make sure our X and y labels are of a NumPy type using **isinstance()** and use the **ravel()** command, which flattens an array:

```
X = X.values if isinstance(X, pd.DataFrame) else X
```

```
y = y.values if isinstance(y, pd.Series) else y
```

```
y = y.ravel()
```

3. Fit our model and create a model variable:

```
model = LogisticRegression(max_iter=1000)
```

We are now ready to perform the K-fold cross-validation steps in the next section.

Fitting the K-fold cross validation

We will now perform the steps to split the dataset into the respective folds:

1. Create a variable to store the folds and use the **KFold()** function from scikit-learn to undertake the splits by the relevant folds:

```
K_FOLD = 5
```

```
k_fold = KFold(n_splits=K_FOLD, shuffle=True, random_state=42)
```

2. Initialize an empty list to store the fold metrics:

```
fold_metrics = []
```

3. Loop through the folds using the **train_idx** and **test_idx** produced by the K-fold's **.split()** method, incorporating the **enumerate** function to add a counter to the loop. The **enumerate** function provides an index for each iteration, allowing you to track the fold number. The **.split()** method is used by the **KFold** function to perform the splits, generating the training and testing indices for each fold.
4. The remaining steps are the same as in the multiple-model evaluation process:

```
for fold, (train_idx, test_idx) in enumerate(k_fold.split(X)):
```

```
    X_train, X_test = X[train_idx], X[test_idx]
```

```
    y_train, y_test = y[train_idx], y[test_idx]
```

```
    model.fit(X_train, y_train)
```

```
    y_pred = model.predict(X_test)
```

```
    y_proba = model.predict_proba(X_test)
```

```
    accuracy = accuracy_score(y_test, y_pred)
```

```
    precision = precision_score(y_test, y_pred)
```

```
    recall = recall_score(y_test, y_pred)
```

```
    f1 = f1_score(y_test, y_pred)
```

```
    fold_metrics.append({
```

```
        'fold': fold + 1,
```

```
        'accuracy_fold': accuracy,
```

```
        'precision_fold': precision,
```

```
        'recall_fold': recall,
```

```
        'f1_fold': f1,
```

```
})
```

5. Create an evaluation DataFrame to view the results:

```
fold_eval_df = pd.DataFrame(fold_metrics)
```

The output of our final evaluation DataFrame is shown in *Figure 2.10*:

fold	accuracy_fold	precision_fold	recall_fold	f1_fold
1	0.973684	0.972222	0.985915	0.979021
2	0.982456	1.000000	0.974026	0.986842
3	0.964912	0.958904	0.985915	0.972222
4	0.991228	0.986111	1.000000	0.993007
5	0.973451	0.970588	0.985075	0.977778

Figure 2.10: K-fold evaluation results

Usually, you would calculate an average of these values to determine the overall metric performance. However, examining each fold individually based on this partitioning approach allows us to identify which folds perform best on the task, depending on the metric we focus on.

For example, if the F1 score is our primary metric, we might be particularly interested in analyzing fold 4's performance compared to the others. As before, we will use a panel chart to visualize each fold's performance and gain insights into our metrics.

Visualizing the K-fold evaluation metrics

As we have done throughout, we will break this down into a set of steps to create the visualization:

1. Create a list of metrics we are interested in (these will correspond to columns in *Figure 2.9*):

```
metrics = ['accuracy_fold', 'precision_fold', 'recall_fold', 'f1_fold']
```

2. This step is not necessary, but it shows how to use the random library to create randomized HEX codes for our colors.

Note: Feel free to skip this step if you plan to just pass the chart colors as a list that does not change.

The code is as follows:

```
import random

def generate_random_hex_color():
    return "#{:06x}".format(random.randint(0, 0xFFFFFF))

chart_colors=[generate_random_hex_color() for _ in range(len(metrics))]
```

3. The code creates a 2x2 grid of bar charts to visualize different evaluation metrics across folds in a K-fold cross-validation. Each subplot represents one metric, displaying a bar chart and a line plot for each fold.
4. The bar chart shows the values for that metric, while the line plot connects these points for clearer comparison. Titles, labels, and legends are added to each subplot, and **plt.tight_layout()** ensures the layout is well-spaced.
5. Finally, **plt.show()** displays the completed chart:

```
fig, axes = plt.subplots(2, 2, figsize=(15, 10), sharey=True)
axes = axes.flatten()

for i, metric in enumerate(metrics):
    axes[i].bar(fold_eval_df["fold"], fold_eval_df[metric],
               color=chart_colors[i], alpha=0.6, label=metric)
    axes[i].plot(fold_eval_df["fold"], fold_eval_df[metric],
```

```

marker="o", color='black', linestyle="-",
linewidth=1, markersize=6, label=f"{metric} Points")

axes[i].set_title(f"{metric} across {K_FOLD} folds")
axes[i].set_xlabel("Fold")
axes[i].set_ylabel(metric)
axes[i].legend()

plt.tight_layout()
plt.show()

```

The output of our efforts in these steps is a nice panel chart to visualize the results:

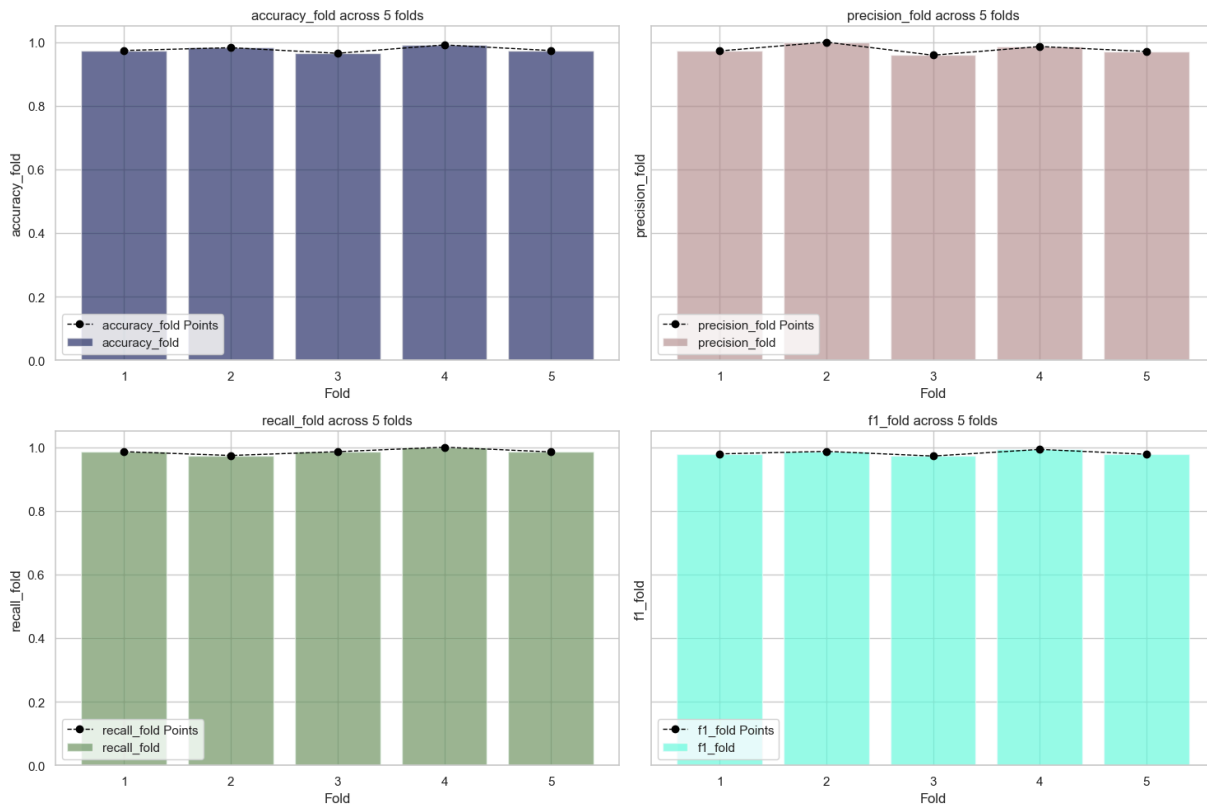


Figure 2.11: K-fold evaluation results visualized

You will see in later examples that the looping structures provide more customization options.

However, there is also a more straightforward way to achieve these results, which we will cover in the next section.

K-fold evaluation the simple way

While the previous approaches to obtaining the results give us more flexibility, if you want to obtain the results quickly, then you can use the `cross_val_score` function in `sklearn.model_selection`:

```
1. from sklearn.model_selection import cross_val_score
2. scores = cross_val_score(model, X, y, cv=k_fold, scoring='recall')
3. print("Individual fold accuracies:", scores)
4. print("Mean recall:", scores.mean())
```

This prints the following:

- The first print statement returns: **Individual fold accuracies: [0.98591549 0.97402597 0.98591549 1. 0.98507463]**
- The second print statement returns: **Mean recall: 0.9861863173614276**

Conclusion

In this chapter, we took our first steps into the fundamentals of scikit-learn, a powerful Python library designed to streamline ML workflows. We explored scikit-learn's core features, including its consistent API, ease of use, and seamless integration with popular Python libraries like NumPy, Pandas, and Matplotlib. Scikit-learn's vast suite of supervised and unsupervised learning algorithms makes it a versatile choice for various ML tasks, including classification, regression, clustering, and dimensionality reduction. This introduction provided an essential foundation, showcasing the ways scikit-learn simplifies complex ML processes, making it accessible to both beginners and seasoned practitioners.

To bring these concepts into a real-world scenario, we applied them to a breast cancer classification problem using the *Wisconsin Breast Cancer*

dataset. This hands-on project involved every step of the ML pipeline, from data preprocessing to model selection and evaluation, emphasizing the importance of metrics like precision, recall, and F1 score in medical contexts. By building and evaluating a binary classifier, we saw firsthand how scikit-learn's tools allow us to tackle practical challenges and make informed, data-driven decisions. This chapter's insights into scikit-learn set the stage for more complex applications, where these foundational skills will support increasingly sophisticated ML solutions.

In the next chapter, we will look into another classification example to solidify your understanding, introducing several new concepts we have not covered yet.

Points to remember

Here are some key points summed up from this chapter:

- **Ease of use and consistent API:** Scikit-learn's straightforward and uniform API makes ML accessible, catering to both beginners and experts.
- **Integration with core Python libraries:** Seamlessly integrates with libraries like NumPy, Pandas, and Matplotlib, simplifying data manipulation, visualization, and model training.
- **Comprehensive algorithm suite:** Scikit-learn offers a range of supervised and unsupervised algorithms, including regression, classification, clustering, and dimensionality reduction, supporting diverse ML needs.
- **Data preprocessing tools:** Equipped with utilities for data cleaning, feature scaling, normalization, and encoding, scikit-learn ensures data is prepared efficiently for model training.
- **Pipeline support for workflow automation:** Pipelines enable chaining of preprocessing, feature engineering, and model training steps, making complex workflows organized and repeatable.

- **Cross-validation for reliable model evaluation:** Cross-validation splits data into multiple training and test sets, providing robust assessments of model performance and reducing overfitting.
- **Hyperparameter tuning options:** Built-in tools like GridSearchCV and RandomizedSearchCV enable automated tuning to optimize model performance.
- **Evaluation metrics for model comparison:** Scikit-learn provides metrics like accuracy, precision, recall, and F1 score, offering deeper insights into model performance beyond simple accuracy, especially for imbalanced datasets.

Multiple choice questions

1. **What is scikit-learn primarily used for in Python?**
 - a. Data cleaning
 - b. Data visualization
 - c. ML and predictive modeling
 - d. Numerical computations
2. **Which scikit-learn function is commonly used to split data into training and test sets?**
 - a. cross_val_score()
 - b. train_test_split()
 - c. StandardScaler()
 - d. GridSearchCV()
3. **What method is used in scikit-learn to train or fit an ML model on data?**
 - a. predict()
 - b. fit()
 - c. transform()

- d. `score()`
- 4. **Which of the following is an evaluation metric that combines precision and recall?**
 - a. ROC curve
 - b. MSE
 - c. F1 Score
 - d. Accuracy
- 5. **What is the purpose of `StandardScaler()` in scikit-learn?**
 - a. To scale features to a standard range (0 to 1)
 - b. To make features have a mean of 0 and a standard deviation of 1
 - c. To convert categorical variables to a numerical format
 - d. To split data into different partitions
- 6. **Which scikit-learn algorithm would be the most appropriate for binary classification problems?**
 - a. Linear regression
 - b. PCA
 - c. K-means
 - d. Logistic regression
- 7. **What does K-fold cross validation do in scikit-learn?**
 - a. Divides data into multiple training and tests sets to provide robust evaluation
 - b. Reduces the dimensionality of the data
 - c. Finds correlations among features
 - d. Performs hyperparameter tuning

8. In a classification model, which metric would be the most relevant for evaluating performance on an imbalanced class distribution?
- a. Accuracy
 - b. Precision
 - c. Recall
 - d. Both Precision and Recall
9. What does the `make_blobs()` function in scikit-learn help with?
- a. Creating data partitions for model evaluation
 - b. Normalizing and standardizing feature values
 - c. Optimizing model parameters through randomized search or in a grid pattern
 - d. Generating sample datasets with specific cluster structures
10. What does the `predict_proba()` method in scikit-learn provide?
- a. The predicted class labels for each sample
 - b. The probability estimates for each class label
 - c. The accuracy of the model training
 - d. Normalized and transformed feature vectors

Answers

1	c
2	b
3	b
4	c
5	b
6	d
7	a
8	d

9	d
10	b

Key terms

Some key terms used in this chapter are:

- **Accuracy:** An evaluation metric that calculates the proportion of correct predictions over the total number of predictions, often less reliable in cases of class imbalance.
- **API:** A structured set of commands and functions in scikit-learn that allows users to interact with ML algorithms in a consistent, user-friendly manner.
- **Confusion matrix:** A table that provides insight into the number of TPs, TNs, FPs, and FNs, useful for understanding model performance in classification tasks.
- **CV:** A technique that splits data into multiple training and testing sets to evaluate model performance more reliably, reducing the risk of overfitting.
- **Estimator:** A scikit-learn object representing a ML model or data transformer with `.fit()` and `.predict()` methods, used for training and making predictions.
- **F1 score:** A harmonic mean of precision and recall, providing a balanced measure of both metrics, particularly valuable in imbalanced datasets.
- **Gradient boosting:** An ensemble technique that builds models sequentially, with each model focusing on correcting the errors of its predecessor, effective for complex patterns.
- **K-fold cross validation:** A type of cross-validation that splits data into K subsets, training on K-1 and testing on the remaining subset, repeated K times for a robust evaluation.

- **KNN:** A classification method that assigns a label based on the majority class among the closest neighbors, ideal for small to medium-sized datasets.
- **Logistic regression:** A classification algorithm that models the probability of a binary outcome, suitable for scenarios requiring interpretable results.
- **make_blobs():** A function that creates synthetic datasets with clusters, helpful for practicing classification and clustering techniques.
- **make_regression():** A scikit-learn function that generates synthetic data for regression tasks, useful for practicing model training and evaluation.
- **Normalization:** The process of scaling data to a standard range (typically 0 to 1) to improve model performance, particularly for algorithms sensitive to feature magnitude, like k-nearest neighbors.
- **Pipeline:** A feature in scikit-learn that enables chaining multiple processing steps (e.g., scaling, transforming, training) into a single sequence, streamlining ML workflows.
- **Precision:** A metric measuring the accuracy of positive predictions, defined as the ratio of true positives to the total predicted positives, helpful in imbalanced datasets.
- **PCA:** A technique that reduces the number of features by finding directions of maximum variance, often used to simplify and visualize data.
- **Random forest:** An ensemble learning method that creates multiple decision trees, combining their predictions to improve accuracy and reduce overfitting.
- **Recall (Sensitivity):** The metric that quantifies the model's ability to identify positive cases correctly, defined as the ratio of true

positives to actual positives, essential for tasks like medical diagnosis.

- **StandardScaler:** A scikit-learn transformer that scales features to have a mean of 0 and a standard deviation of 1, commonly used to normalize numerical data.
- **SVM:** A classification algorithm that finds the optimal hyperplane to separate classes in a high-dimensional space, effective for complex, high-dimensional data.
- **train_test_split:** A scikit-learn function divides data into training and testing subsets, aiding in model evaluation and preventing data leakage.
- **t-SNE:** is a dimensionality reduction technique for visualizing high-dimensional data by projecting it into two or three dimensions.

Join our Discord space

Join our Discord workspace for latest updates, offers, tech happenings around the world, new releases, and sessions with the authors:

<https://discord.bpbonline.com>



CHAPTER 3

Supervised Binary Classification

Introduction

In this chapter, we will be looking at supervised binary classification. This chapter unpacks everything from the fundamentals of decision boundaries and logistic regression to the hands-on techniques to tackle real-world classification problems; think imbalanced datasets, feature selection, and performance tuning. We will refine our models and master techniques like SMOTE and Youden's J statistic to maximize predictive power. Expect a robust blend of theory and practical coding, with advanced evaluation methods, confusion matrices, and ROC curve analysis. By the end, you will not just be building models; you will be optimizing them for precision, recall, and real-world impact.

Structure

This chapter covers the following topics:

- Introducing binary classification
- Thyroid disease prediction case study
- Exploratory data analysis
- Feature encoding, selection and cleaning
- Model training
- Evaluation metrics
- Understanding the confusion matrix

- Over and under sampling methods
- Building the training and evaluation loop
- Receiver operator characteristics curve

Objectives

By the end of this chapter, you will be equipped with the skills to build, evaluate, and optimize binary classification models confidently. Using the sigmoid function, you will understand the core principles behind logistic regression, decision boundaries, and probability mapping. You can preprocess data effectively, handling missing values with techniques like MICE, appropriately scaling features, and encoding categorical variables for machine learning models. You will also master strategies for dealing with imbalanced datasets, from resampling techniques like SMOTE to adjusting decision thresholds for improved classification performance.

Beyond building models, you will gain the ability to assess their effectiveness using key evaluation metrics, confusion matrices, and ROC curves. Most importantly, you will leave with a complete pipeline that integrates feature selection, model evaluation, and class balancing, ensuring your classifiers are not just functional, but optimized for real-world impact.

Introducing binary classification

Binary classification is a fundamental concept in ML and statistics that involves categorizing data points into one of two distinct groups or classes. These classes are typically represented as binary values, such as 0 and 1, or labels like *Yes* and *No*. The primary objective of a binary classification model is to predict the correct class for new (unseen) data based on patterns it learns from existing data.

At the heart of binary classification lies the idea of separating data into two groups using a decision boundary. This boundary, determined during training, represents the dividing line (or curve) that differentiates one class from the other. For example, in a problem predicting thyroid disease, the

decision boundary might separate individuals likely to have the disease (1) from those unlikely to have it (0). Depending on the complexity of the data, this boundary could be linear or more complex, reflecting the subtleties in the relationship between features and outcomes. Scikit-learn comes to the rescue here, as it can create these decision boundaries with linear and non-linear algorithms.

The process begins with a dataset, where each data point consists of a set of features (attributes) and a corresponding label indicating its class. In a thyroid disease prediction task, the features might include hormone levels (e.g., T3, TSH), patient age, and medical history, while the label would indicate whether the patient has thyroid disease (1) or not (0). A machine learning algorithm uses this labeled data to learn the relationship between the features and the labels. The learning process involves minimizing errors, such as the number of patients incorrectly classified as having or not having the disease, to create a model that can generalize well to new cases.

Once trained, the model predicts the class of new data points by analyzing their features and applying the learned decision boundary. For instance, given a new patient's medical data, the model might predict thyroid disease if the patient's features align more closely with patterns observed in patients diagnosed with the disease during training. The prediction might also include a probability score, indicating the model's confidence—for example, a 90% likelihood that the patient has the disease.

The underlying theory of binary classification is deeply rooted in statistical principles. Algorithms like logistic regression use a mathematical function to estimate the probability of a data point belonging to a particular class. Other methods, such as support vector machines or decision trees, rely on geometric or rule-based approaches to define the decision boundary. Regardless of the specific algorithm, the goal is to maximize the model's ability to distinguish between the two classes while minimizing errors. We will look at how Logistic Regression paved the way for this type of classification analysis.

Role of the logit function

Let us understand why this is important before creating a sigmoid function:

- Logistic regression begins with a formula like linear regression where $z = wX + b$, essentially calculating a straight line or (plane) based on the input features X , from our independent variables, the weights (w) and the bias (b). Linear regression would not work for classification, as it gives continuous outputs (like 2.7, -1.5, etc.).
- Output conversion to probabilities with the logit function: this converts the outputs of the linear model to something that the model can use, that is, odds of belonging to a particular class (e.g., Thyroid disease or not). This calculates the odds using the following formula:

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right)$$

Here is an explanation of the logit function terms:

- The parameter p is the probability of having thyroid disease
- $1 - p$ is the inverse probability, i.e., not having the disease
- The division of p by $1-p$ generates the odds e.g. if $p=0.8$, the odds are 0.8 divided by $0.2 = 4$, meaning that the patient is 4 times more likely to have the disease than not have it
- Finally, the *log*, or the *ln (natural logarithm)* translates them into a linear scale that can range from *−infinity* to *+infinity* (∞).
- The last step is to map the odds to probabilities using the Sigmoid function, as described in the next section.

Sigmoid, turning log-odds into probabilities

At its heart, logistic regression is a linear model that predicts probabilities rather than raw output values. For a given input feature X , it calculates the

likelihood $P(y=1|X)$ y belongs to class 1. This probability is computed using the sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

This is explained here:

- **$z=wX+b$:**
 - **w :** Weight (or slope of the line)
 - **X :** Input feature(s)
 - **b :** Bias (shifts the curve left or right)
 - **e :** The base of the natural logarithm
- **$1+e^{-z}$:** Adds 1 to ensure the denominator is never zero, keeping the formula mathematically valid.
- **$1/1+e^{-z}$:** This maps to a value between 0 and 1. The closer z is to a large positive number, the closer the output will be to 1. The closer z is to a large negative number, the closer the output will be to 0.

Let us look at how we apply these to our Thyroid disease prediction problem:

- **Compute $z=wX+b$:** Combine the input features (e.g., TSH, T3, Age) with their weights (w) and bias (b) to calculate a raw score (z). For instance, high TSH might contribute positively to predicting thyroid disease.
- **Apply Sigmoid function:** Use the sigmoid function, to map z to a probability between 0 and 1. Large positive z means a high probability of thyroid disease; large negative z means a low probability, is and generally treated as no risk.
- **Make a decision:** Compare the probability to a threshold (e.g., 0.5). If $\sigma(z) \geq 0.5$, predict the patient has thyroid disease (1);

otherwise, predict they do not (0).

The described process that we have just covered, from converting log odds to a sigmoid function and mapping, can be visualized in *Figure 3.1*:

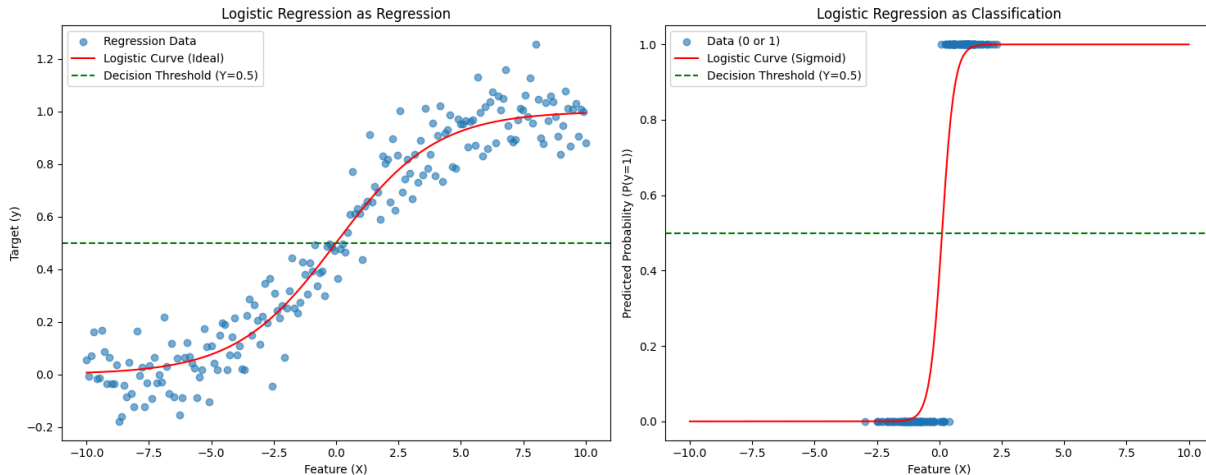


Figure 3.1: Regression continuous variables to discrete

The comparisons below effectively summarize what we have just been talking about and how this conversion led to regression models being transitioned away from predicting continuous values to enabling classification of data.

Left panel: Logistic regression as regression:

- This shows the logistic curve (red line) fit to continuous data points, highlighting how logistic regression computes probabilities using a smooth curve.
- The decision threshold line ($y = 0.5$) emphasizes the boundary where predictions switch between the two classes. This gives a great intuition for how probabilities transition from one class to another.

Right panel: Logistic regression as classification:

- This depicts the binary classification output (0 or 1) from applying the threshold. It visually demonstrates how the sigmoid curve converts continuous probabilities into discrete classes.

- The green decision threshold line visually reinforces the concept that values above 0.5 are classified as 1, and values below are classified as 0.

Logistic regression will be our baseline model for all classification examples in the upcoming chapters. It is a foundational benchmark to determine whether more complex model architectures are necessary. For those interested in the underlying mathematics, it is highly recommended that you visit the excellent documentation available on the scikit-learn website: https://scikit-learn.org/stable/supervised_learning.html.

Thyroid disease prediction case study

This chapter tackles a challenging classification problem that mirrors the complexities of real-world projects you may encounter in your career. This dataset presents several hurdles: it is imbalanced, contains missing data, and has many less predictively essential features. These are common issues in practical machine learning tasks, making this an excellent opportunity to develop and test robust solutions.

To address these challenges, we will leverage the full suite of tools available in scikit-learn and other packages to refine our classifiers and ensure they perform as expected. The goal is to build a working model and explore strategies for handling these real-world complexities effectively.

This dataset has been sourced from the R package **MLDataR**:

<https://cran.r-project.org/web/packages/MLDataR/vignettes/MLDataR.html>.

The data is conveniently consolidated within the package, but it originally comes from a Kaggle challenge where the dataset was less structured, adding another layer of realism to this project. The data is contained in the supporting GitHub repository for this chapter, in the data folder.

Getting the data

First, let us import our data and make sure we have the relevant libraries imported:

```
1. import pandas as pd
2. import matplotlib.pyplot as plt
3. tdf = pd.read_csv("data/MLDataR_thyroid_class.csv")
4. tdf.describe()
```

Here is a breakdown of the code:

- Import **pandas** and **matplotlib** (for visualizations).
- Use the **read_csv()** method to get the data from the subfolder data and with the file name **MLDataR_thyroid_class.csv**.
- Get a description of the data distribution very quickly using **describe()** method to break down the count, mean, **standard deviation (std)**, **minimum value (min)**, 25% (25 percentile: <https://www.mathsisfun.com/data/percentiles.html>), 50% (50th percentile), 75% (75th percentile) and maximum value (max).

Use columns to view the data structure of the columns:

```
1. tdf.columns
```

This provides the list of columns that are contained in the dataset:

```
['ThyroidClass', 'patient_age', 'patient_gender', 'presc_thyroxine', 'queried_why_on_thyroxine', 'presc_anthyroid_meds', 'sick', 'pregnant', 'thyroid_surgery', 'radioactive_iodine_therapyI131', 'query_hypothyroid', 'query_hyperthyroid', 'lithium', 'goitre', 'tumor', 'hypopituitarism', 'psych_condition', 'TSH_measured', 'TSH_reading', 'T3_measured', 'T3_reading', 'T4_measured', 'T4_reading', 'thyrox_util_rate_T4U_measured', 'thyrox_util_rate_T4U_reading', 'FTI_measured', 'FTI_reading', 'ref_src'].
```

Exploratory data analysis

EDA comprises the process of analyzing and visualizing datasets to understand their structure, uncover patterns, detect anomalies, test hypotheses, and summarize key characteristics, often using statistical and graphical techniques.

We will use EDA to determine missing values, show distribution of features, highlight class imbalance on our Y variable (**ThyroidClass**) and look at any correlations that may cause issues with multicollinearity (normally meaning the features contain redundant information and can confuse linear models, specifically).

Histogram

A histogram is a graphical representation that organizes and displays the distribution of a dataset by dividing it into intervals called bins. Each bin groups a range of values, and the height of the bar corresponding to a bin shows how many data points fall within that range. This makes histograms an effective way to visualize numerical data's shape, spread, and central tendencies, revealing patterns such as skewness, modality, and the presence of outliers. We will follow these steps to get up and running quickly:

1. The first step is to install the **modelviz** (<https://pypi.org/project/modelviz/>) package:

```
pip install modelviz --quiet
```

2. We will now import the **plot_feature_histograms()** function from the **modelviz** package:

```
from modelviz.histogram import plot_feature_histograms
```

3. To generate the visual, we will use the following code, making sure to set the parameter **exclude_bin_encode=True** (this excludes any one-hot encoded values that take the values 0 or 1, as we want to just visualize continuous data):

```
plot_feature_histograms(tdf, library='matplotlib', exclude_bin_encode=True)
```

4. This will generate a visualization similar to *Figure 3.2*:

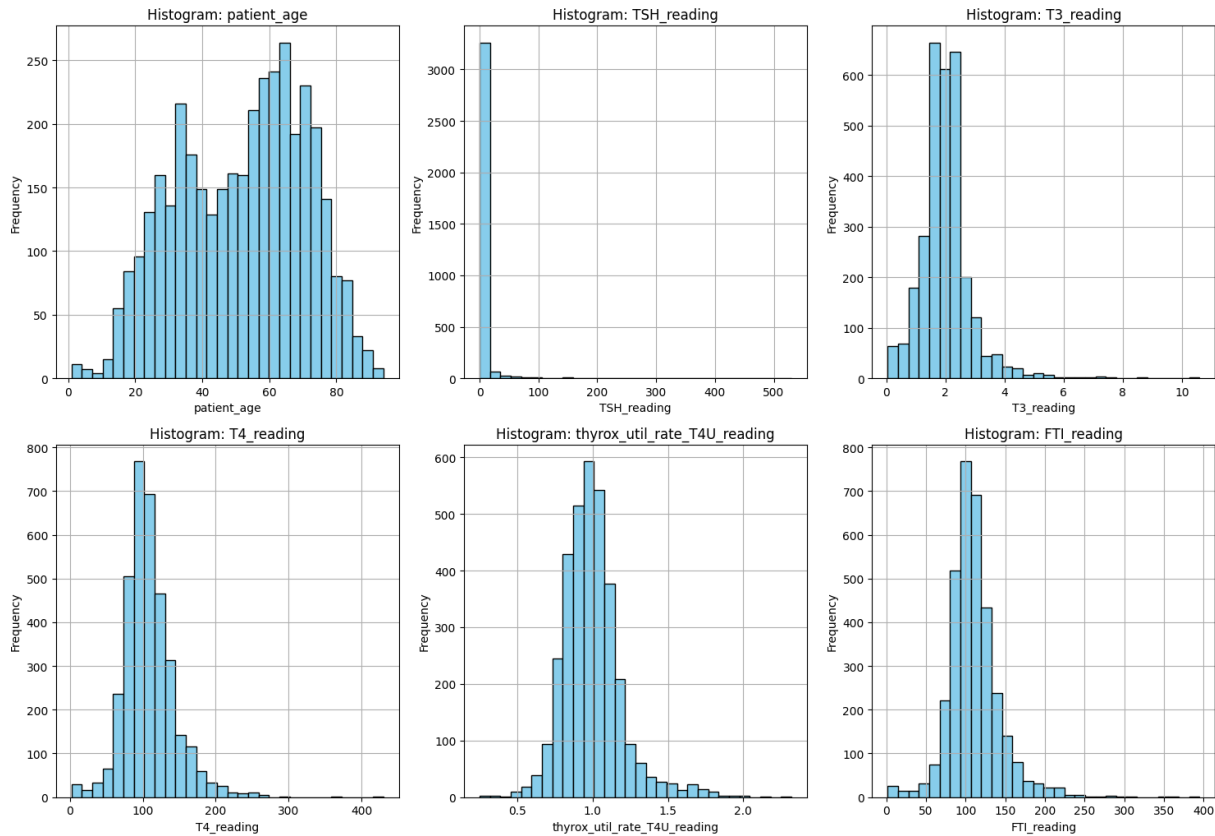


Figure 3.2: *Thyroid disease continuous features*

5. From these distributions, a few patterns are apparent:
- Patient age:** Displays a bimodal distribution, with a primary peak around 37 years and a secondary peak around 70 years.
 - TSH reading:** Indicates a high frequency of values recorded as 0.
 - t3 reading:** Suggests potential missing values.
 - Additional features:** Exhibit various degrees of skewness in their distributions.
6. The next logical step is to confirm whether there are missing values in the dataset.

Missing values

Missing values visualizations are effective, allowing you to easily see which columns in your dataset contain a high proportion of missing items. We can do this in the **modelviz** package, by following these steps:

1. Import the relevant module from **modelviz**:

```
from modelviz.missvals import plot_missing_values_heatmap
```

2. Use the imported function to prepare the visualization:

```
plot_missing_values_heatmap(tdf)
```

3. The visualization as shown in *Figure 3.3* is then produced:

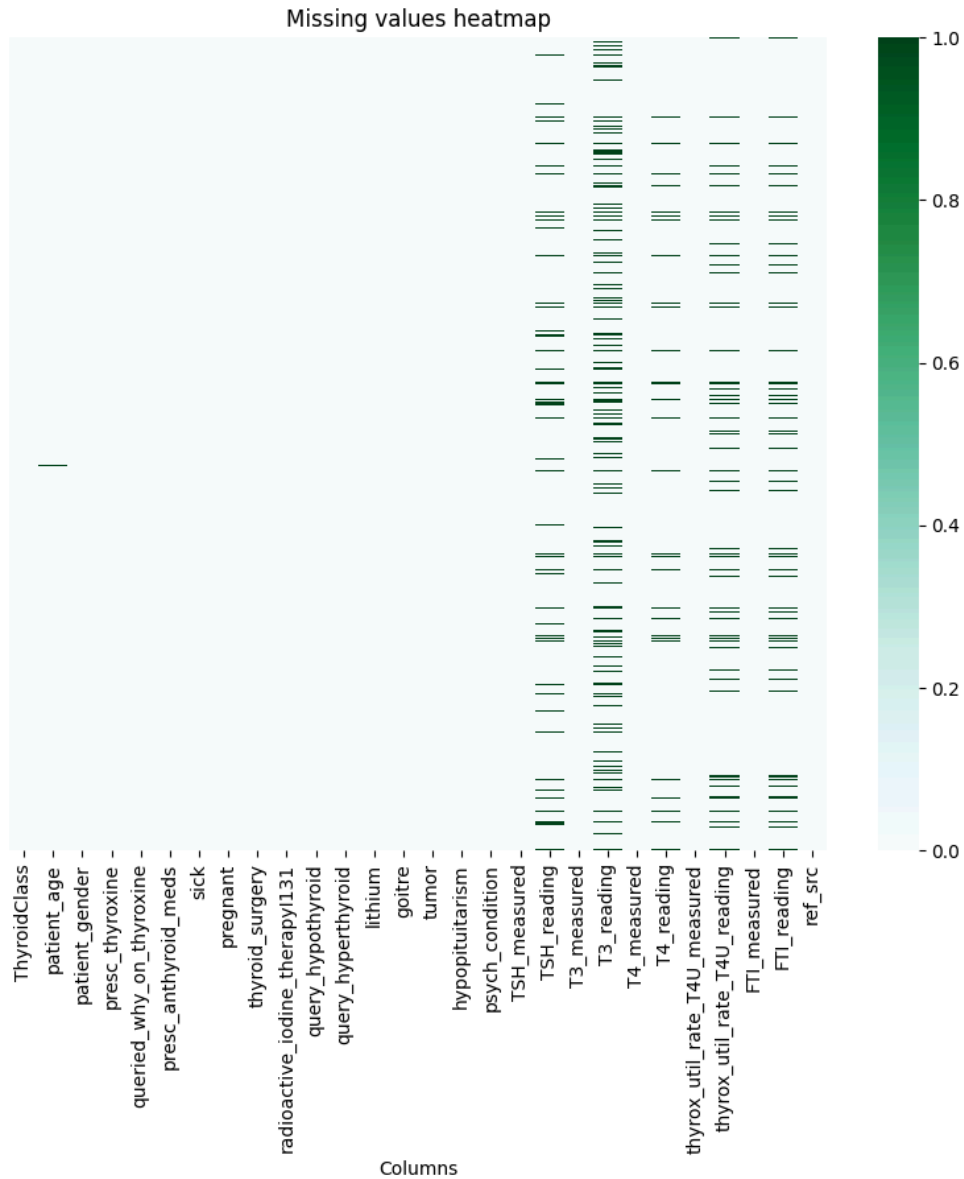


Figure 3.3: Missing values heatmap

4. Missing values are apparent in:
 - a. **T3 reading** shows the highest proportion of misses.
 - b. All the variables where there are readings show missing values. Now we know that this is because if the associated measured **flag = 0**, this means a reading was not taken and, therefore will always present as a NULL value.

We will use our visuals to find correlations amongst our features in the next step.

Feature correlation

Before building effective machine learning models, it is essential to understand the relationships between features in your dataset. Correlation measures the degree to which features are linearly related, providing critical insights into potential redundancies or dependencies in your data.

Highly correlated features can cause problems such as multicollinearity, where redundant information confuses the model, leads to unstable predictions, and reduces interpretability. Assessing correlation helps identify these issues early, allowing you to take corrective actions like removing redundant features, combining related ones, or applying dimensionality reduction techniques. By addressing correlation appropriately, you can enhance model stability, efficiency, and accuracy, ensuring your features contribute meaningful, independent information to the learning process.

We can quickly grab a correlation matrix from the `model viz` package for assessing these relationships, or we could build our own as we did in *Chapter 1, Overview of Numpy and Pandas*:

1. Import the **`plot_correlation_matrix()`** function from the **`modelviz`** package:

```
from modelviz.relationships import plot_correlation_matrix
```

2. Use the function, as mentioned:

```
plot_correlation_matrix(tdf, max_columns=18)
```

3. Here, we use **`max_columns=18`**, as the visual will become unreadable beyond this point to generate that in *Figure 3.4*:

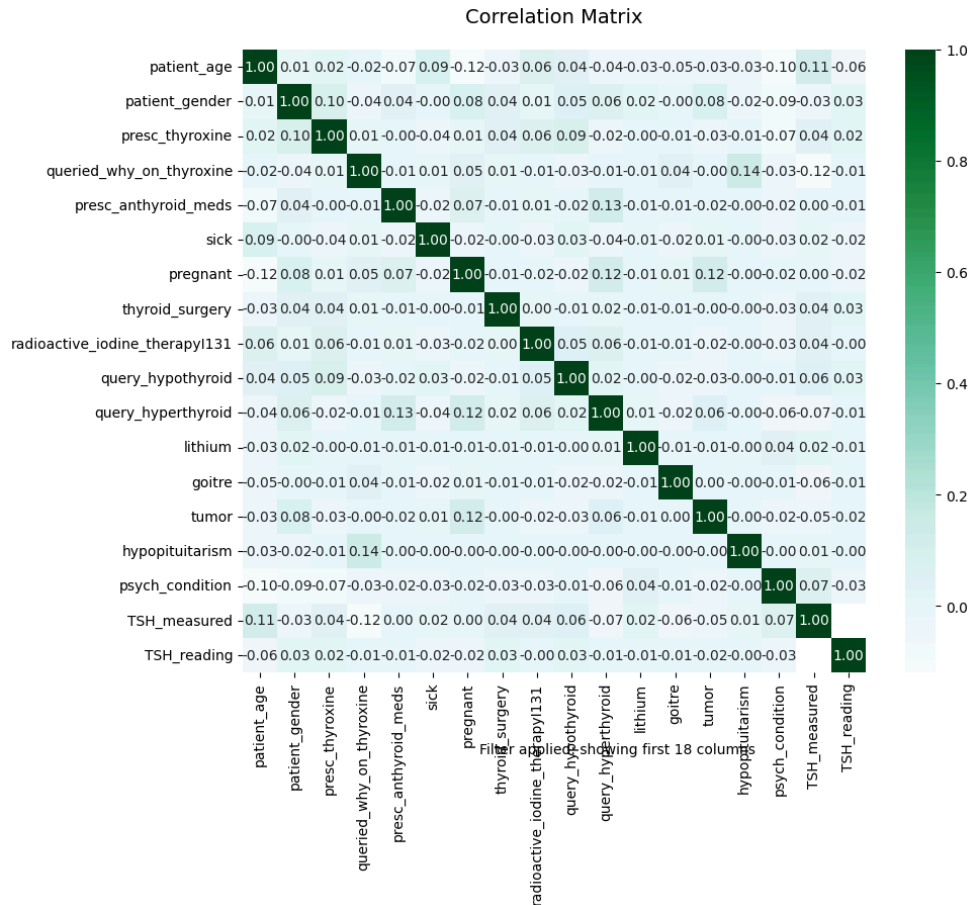


Figure 3.4: Correlation matrix

- No immediate correlations appear from the first 18 columns; however, inspecting the matrix, the columns with the highest correlation are the measured and reading columns that are presented as missing values in the missing values section.

Finally, let us look at the class distribution.

Class distribution

As mentioned earlier, let us examine the class distribution of the **ThyroidClass** variable (our dependent variable) to assess the class balance. To achieve this, you can easily convert the variable to a Series and apply the **value_counts()** function directly:

```
1. pd.Series(tdf['ThyroidClass']).value_counts().plot()
```

2. `kind='bar', title='Class Distribution')`

The bar chart is produced to show the distribution:

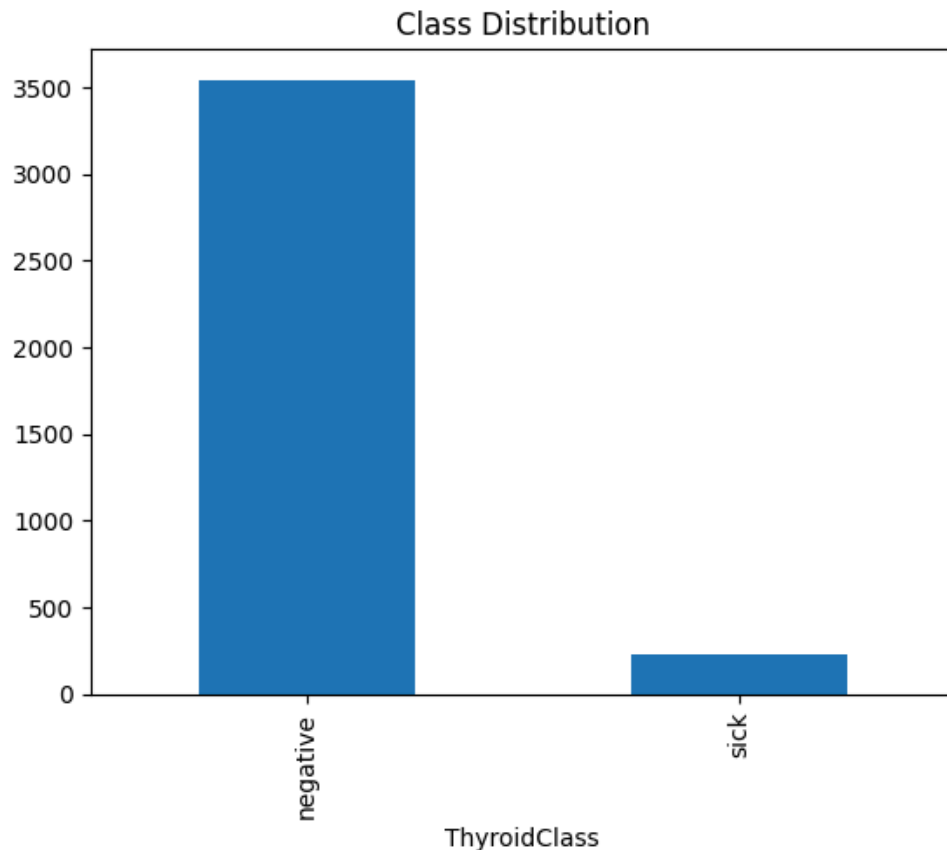


Figure 3.5: Class distribution

The majority of cases in the dataset belong to the negative class, indicating a significant class imbalance. This imbalance means that the sample contains far more instances of one class (negative) compared to the other (positive).

Imbalanced datasets are a common challenge in real-world ML projects and scenarios, particularly in fields like medical diagnostics, fraud detection, and anomaly detection, where one class often dominates the dataset.

When working with imbalanced data, models tend to favor the majority class, as it minimizes overall error during training. However, this can result in poor performance on the minority class, which is often the class of most interest. For example, in the case of thyroid disease prediction, incorrectly

predicting that a patient does not have the disease (a false negative) can have severe consequences, as it may lead to missed diagnoses and untreated conditions. On the other hand, a well-balanced model ensures better detection of these critical cases.

We will incorporate a resampling method as part of our training pipeline to address this imbalance. Resampling can be done in two main ways:

- **Oversampling the minority class:** This involves duplicating or generating synthetic examples of the minority class to make its size comparable to the majority class.
- **Undersampling the majority class:** This involves reducing the number of examples in the majority class to achieve balance, though at the cost of losing potentially helpful information.

By rebalancing the dataset, we improve the model's ability to learn from both classes equally, leading to more accurate and reliable predictions, especially for the minority class. Understanding how to handle class imbalances is crucial for working with real-world datasets, as imbalances are the norm rather than the exception. Addressing these imbalances ensures that your models are robust and can make meaningful predictions even in challenging scenarios.

Missing values summary

Next, we are going to take a statistical approach to looking at the missing values in our dataset, as these statistics will be important for knowing what to do when it comes to imputation methods, such as median, mode, average, etc. In this section, we are going to create a function to allow us to do just that:

1. We will decompose the steps first and then present the function as a whole to aid in your understanding.
2. We will create a **missing_counts** variable to store the missing volumes, which takes the sum of **isnull()** values for that

DataFrame:

```
missing_counts = df.isnull().sum()
```

3. Then, get the non-missing, or those that have values:

```
non_missing_counts = df.notnull().sum()
```

4. Create a variable to grab the total volume for the DataFrame:

```
total_vol = len(df)
```

5. Thinking about proportions, let us create a variable to calculate proportionally how many are missing:

```
missing_proportions = missing_counts / len(df)
```

6. For completeness, let us create a non-missing proportions variable as well:

```
non_missing_proportions = non_missing_counts / len(df)
```

7. We will now create a DataFrame to store all the results in, and this will do the calculations per feature column:

```
props_df = pd.DataFrame({  
    "feature": df.columns,  
    "missing_count": missing_counts,  
    "non_missing_count": non_missing_counts,  
    "total_vol": total_vol,  
    "prop_missing": missing_proportions,  
    "prop_non_missing": non_missing_proportions,  
    'data_type': df.dtypes.values})  
props_df.reset_index(inplace=True, drop=True)
```

8. The main function for this, with a column drop threshold (if the data is below a threshold, there is an additional option to auto-drop the column):

```

def missing_values_summarizer(df:pd.DataFrame,
                              drop_cols:bool=False,
                              drop_threshold:float=0.1,
                              verbose=False):
    missing_counts = df.isnull().sum()
    non_missing_counts = df.notnull().sum()
    total_vol = len(df)
    missing_proportions = missing_counts / len(df)
    non_missing_proportions = non_missing_counts / len(df)
    props_df = pd.DataFrame({
        "feature": df.columns,
        "missing_count": missing_counts,
        "non_missing_count": non_missing_counts,
        "total_vol": total_vol,
        "prop_missing": missing_proportions,
        "prop_non_missing": non_missing_proportions,
        'data_type': df.dtypes.values
    })
    props_df.reset_index(inplace=True, drop=True)
    if drop_cols:
        cols_to_drop=props_df[props_df['prop_missing'] > drop_threshold]['feature'].tolist()
        df_resized = df.drop(columns=cols_to_drop)
        if verbose:
            print(f'Dropping columns: {cols_to_drop}')
    else:
        df_resized = df.copy()

```

```
return props_df, df_resized
```

9. To use this function, you can pass multiple assignments to the results, to get the proportions and the actual DataFrame with any processing applied, like auto dropping columns with a high proportion of NULL columns:

```
props_df, res_df = missing_values_summarizer(tdf,  
                                             drop_cols=True,  
                                             drop_threshold=0.5)
```

10. The resultant output of this function is a DataFrame that contains the feature, **missing_count**, **non_missing_count**, **total_vol**, **prop_missing**, **prop_non_missing** and **data_type**.
11. The reason this is included here is that you can use this for any of your projects from this point in time.

Feature encoding, selection and cleaning

In the following sections, we are going to explore feature encoding, feature selection, and data cleaning. We will perform steps such as dropping columns, splitting data, one-hot encoding, missing value imputation, and feature selection.

Dropping columns

Sometimes, you may encounter columns in your dataset where a single value overwhelmingly dominates, and your only viable option is to remove the column entirely. For instance, in *Figure 3.2*, the **TSH_reading** column contained an overwhelming number of entries recorded as 0. This suggests either incorrect coding or that these measurements were not taken for most patients. When a single value dominates a column, it can skew the model's results by reducing variability and adding little to no predictive value. In such cases, retaining the column does more harm than good.

Since the **TSH_reading** column does not contribute meaningful information to our analysis, we will need to drop it from the dataset. Removing irrelevant or uninformative features ensures that our model focuses on variables that genuinely impact predictions. This step is straightforward and can be accomplished as shown in the following code. By doing so, we maintain the quality of our dataset and avoid introducing noise that could undermine the model's performance.

To drop the column, simply use the Pandas **drop()** functionality:

```
1. res_df.drop(columns=['TSH_reading'],  
2.             axis=1,  
3.             inplace=True)
```

Further, we will also look for columns that have zero variance; these columns are useless because they provide no discriminative power, add redundancy, introduce unnecessary noise, and can mislead interpretations by appearing as though they contribute to the model, when they do not. To drop zero variance columns, use the following:

```
1. variances = res_df.var(numeric_only=True)  
2. zero_variance_columns = variances[variances == 0].index.tolist()  
3. print(zero_variance_columns)  
4. res_df.drop(columns=zero_variance_columns, inplace=True)
```

It looks like all columns are useful, after we have dropped **TSH_reading**, so we will retain the rest and now move on to splitting our data and doing further preprocessing to get the data ready for modeling. To confirm the column has been dropped from the DataFrame, we will use the following code:

```
print(res_df.columns)
```

Splitting data

Follow these steps to split data:

1. The next step, as we did in *Chapter 2, Introduction to Scikit-learn for Machine Learnings* project, we will divide the dataset into an 80/20 split, retaining 80% of the data for training and the 20% for testing the model to enable evaluation.
2. It is good practice, as a first step, to save your features as variable X and your dependent variable as Y, as mentioned in the following code:

```
1. from sklearn.model_selection import train_test_split
2. X = res_df.drop(columns='ThyroidClass', axis=1)
3. y = pd.Series(res_df['ThyroidClass'])
```

3. In the next step, we will create a label mapping dictionary, and use the in-built map function to apply a function across a DataFrame:

```
1. label_mapping = {
2.     'negative': 0,
3.     'sick': 1
4. }
5. y_mapped = y.map(label_mapping)
```

4. Finally, we will use the **train_test_split()** function:

```
1. X_train, X_test, y_train, y_test = train_test_split(X,
2.                                                     y_mapped,
3.                                                     stratify=y_mapped,
4.                                                     test_size=0.2,
5.                                                     random_state=42)
```

5. We use the stratify method here, which ensures that the train and test sets maintain the same proportion of classes as the original dataset, which is especially important for imbalanced datasets.
6. Here, we have derived a function to extract the sample sizes and to make sure that the columns from both DataFrames match the

original DataFrame. Refer to the following code:

```
def get_split_sizes(X_train, X_test):
    X_train_size = len(X_train)
    X_train_cols = len(X_train.columns)
    X_test_size = len(X_test)
    X_test_cols = len(X_test.columns)

    if X_train_cols != X_test_cols:
        assert ValueError(f'Columns in both DataFrames should match, as X_train has: {X_train_cols} columns and X_test has: {X_test_cols} columns')

    print(
        f"Train and testing set statistics\n\n"
        f"X_train size: {X_train_size} | X_train columns: {X_train_cols}\n"
        f"X_test size: {X_test_size} | X_test columns: {X_test_cols}")
```

7. This is a pretty simple function, as it makes extensive use of the **len()** method to get the length of the rows and columns in the arrays, as the **train_test_split()** function saves the data as a NumPy array. To use the function, we will pass in the feature training and testing sets:

```
get_split_sizes(X_train, X_test)
```

8. The resultant output of this function, prints out the sizes of the train and test sets, respectively:

Train and testing set statistics

X_train size: 3017 | X_train columns: 26

X_test size: 755 | X_test columns: 26

One-hot encoding

In machine learning, many algorithms work with numerical data, which means categorical features, variables that represent categories or labels, must be transformed into a numerical format. One-hot encoding is a popular and effective technique for this transformation. It converts categorical data into a binary matrix where each unique category is represented as a separate column.

For every data point, a value of 1 is placed in the column corresponding to its category, while all other columns are set to 0. In this section, we will look at how to convert our one categorical field **ref_src** from a categorical to a numeric representation. We will build a function to describe this process, and we will start taking you through the function step by step, and will include the entire function at the end of this section. Let us get on to the steps needed to encode categorical variables in scikit-learn effectively:

1. We need to first import the **OneHotEncoder** module from **sklearn.preprocessing**:

```
from sklearn.preprocessing import OneHotEncoder
```

2. Next, we define the function in its entirety, and then we will talk through the various steps:

```
def encode_categorical_columns(X_train, X_test,
                              categorical_columns, drop='first'):
    encoder = OneHotEncoder(sparse_output=False,
                            handle_unknown='ignore',
                            drop=drop)
    encoder.fit(X_train[categorical_columns])
    X_train_encoded_array = encoder.transform(X_train[categorical_columns]).astype(float)
    X_test_encoded_array = encoder.transform(X_test[categorical_columns]).astype(float)
```

```

        encoded_feature_names = encoder.get_feature_names_out(
            categorical_columns)

        X_train_encoded_df = pd.DataFrame(X_train_encoded_array,
                                           columns=encoded_feature_names,
                                           index=X_train.index)

        X_test_encoded_df = pd.DataFrame(X_test_encoded_array,
                                           columns=encoded_feature_names,
                                           index=X_test.index)

        print("Encoded Data Example (Train):")
        print(X_train_encoded_df.head())

        X_train = X_train.drop(columns=categorical_columns)
        X_test = X_test.drop(columns=categorical_columns)

        X_train_encoded = pd.concat([X_train.reset_index(drop=True),
                                     X_train_encoded_df.reset_index(drop=True)], axis=1)

        X_test_encoded = pd.concat([X_test.reset_index(drop=True),
                                    X_test_encoded_df.reset_index(drop=True)], axis=1)

        return X_train_encoded, X_test_encoded

```

3. The very first line of the function in *Step 2* instantiates the encoder using the **OneHotEncoder()** function – here, we specify we want to drop the first column created, as there is a statistical reason for this, it is called ‘*leave one out*’, as leaving the original column in can cause issues with multicollinearity.

4. Once the encoder is instantiated, we then use the **fit()** method to fit the encoder to the training dataset, it is very important to make sure any preprocessing is done on the training set, as doing it on the full data set can lead to a concept called data drift and concept drift.
5. Next, we transform the encoded **X_train** and **X_test** arrays to float, and make sure they are cast to an array structure using **encoder.transform**.
6. The step after the conversion to arrays essentially extracts the categorical column names; these are essentially the distinct column names from the referral source (**ref_src**) column, i.e., SVHC, SVHD, SVI, and other. We use the **get_feature_names_out()** function to achieve this.
7. Following this, we create two DataFrames to perform some data operations on downstream, such as dropping the original categorical column from the **X_train** and **X_test** datasets.
8. Finally, we use **pd.concat**, which concatenates two separate DataFrames together; unlike a join, these frames have to have the same number of rows. Otherwise, an error is thrown.
9. Also, we reset the original index to make sure the original indexes are not retained. Then, we return the **X_train_encoded** and **X_test_encoded** frames.
10. This sets up the function to then use. We can call the function and use it on our partitioned arrays, as follows:

```
1. X_train_ohenc, X_test_ohenc = encode_categorical_columns(  
2.     X_train=X_train,  
3.     X_test=X_test,  
4.     categorical_columns=['ref_src'])
```

11. We use multiple assignments to set the training and testing set with a postfix of **ohenc** (my shorthand for one-hot encoded).

12. The print function outputs the binarized values for the features:

ref_src_SVHC	ref_src_SVHD	ref_src_SVI	ref_src_other
0.0	0.0	0.0	1.0
0.0	0.0	0.0	1.0
0.0	0.0	0.0	1.0
0.0	0.0	1.0	0.0
0.0	0.0	1.0	0.0

Table 3.1: One-hot encoded referral source

One-hot encoding transforms categorical variables that a model cannot directly interpret into numerical features, making them machine-readable in just one function. This versatile approach can be applied repeatedly to multiple columns across various projects, both current and future. With our categorical data now properly encoded, the next step is to address missing values using a robust imputation technique called **Multiple Imputation by Chained Equations (MICE)**.

Missing value imputation with MICE

MICE is a sophisticated method for handling missing data by iteratively predicting and filling in missing values using other features in the dataset. The process begins with initializing missing values with simple guesses, such as column means or medians. Then, for each feature with missing data, MICE treats it as the target variable and trains a predictive model using the other features to estimate its missing values. This cyclical imputation process continues across all features, refining the predictions at each iteration until the imputations stabilize or a set number of iterations is reached.

MICE's ability to consider relationships between features when imputing missing values is what sets it apart. Unlike simple imputation methods

disregarding data structure, MICE leverages dependencies between variables for more accurate predictions.

For example, missing medical values can be better estimated using related patient metrics rather than a general average. This ensures that the imputed values align with the dataset's overall patterns, preserving its integrity and improving the reliability of downstream analyses and machine learning models.

A key advantage of MICE is its ability to handle complex patterns of missingness, even when multiple columns have missing values simultaneously. It supports various data types, such as continuous, categorical, or binary, by tailoring predictive models to each feature. Additionally, MICE creates multiple imputed datasets to account for uncertainty in the imputations, enabling robust statistical analyses.

In the following steps, we will implement MICE using scikit-learn's **IterativeImputer**, which provides a convenient and powerful way to apply this method. The following steps get us there with scikit-learn:

1. To use the **IterativeImputer** function, we must use the **enable_iterative_imputer** method to activate it prior to loading in the module:

```
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer, SimpleImputer
```

2. It is very straightforward to use the **IterativeImputer** from this point:

```
use_simple = False
```

```
if use_simple:
```

```
    imputer = SimpleImputer()
```

```
else:
```

```
    imputer = IterativeImputer(random_state=42)
```


3. The lines above use a Boolean switch to go from simple imputation to MICE; here, it is set to False, so the **IterativeImputer()** is activated.
4. Next, as we did in the previous section, we fit the missing values imputation to the training set:

```
imputed_data = imputer.fit_transform(X_train_ohenc)
```

5. Once the **fit_transform** has been fitted, we can use the **transform** (transformer) operation to apply the fitted imputation to our training and testing sets:

```
X_train_imputed = imputer.transform(X_train_ohenc)
```

```
X_test_imputed = imputer.transform(X_test_ohenc)
```

6. That has now applied the imputation to our training and testing sets. To finally check that there are no null values in the arrays, we can turn to NumPy to check this for us:

```
import numpy as np
```

```
if np.isnan(X_train_imputed).any() | np.isnan(X_test_imputed).  
any():
```

```
    print('Null values remain')
```

Next, we need to scale our data so that the data is not sensitive to outliers; we will discuss three methods in scikit-learn that can help you achieve this.

Scaling and normalizing our data

As we learned in *Chapter 2, Introduction to Scikit-learn for Machine Learning*, it is very important to scale your data before modeling, as extreme outliers can affect model results drastically. For data with outlier issues, the following table shows what options are available to you in the preprocessing module for scikit-learn:

Scaler	Use case	Outlier sensitivity
StandardScaler	Data with a normal distribution or algorithms requiring centered data	Sensitive to outliers
MinMaxScaler	Features with varying scales, particularly for bounded ranges e.g. [0,1]	Sensitive to outliers
RobustScaler	Data with outliers, robust scaling using median and interquartile range	Resistant to outliers

Table 3.2: Scaler options in scikit-learn

Let us use the imputed arrays created in the previous step to perform normalization of our data:

1. Grab the imports needed to scale the data; for this example, we will be using the **RobustScaler** function:

```
from sklearn.preprocessing import RobustScaler
```

2. Instantiate the scaler:

```
robust_scale = RobustScaler()
```

3. Use the **fit_transform** transformer method to fit the scaler, first off:

```
X_train_scaled = robust_scale.fit_transform(X_train_imputed)
```

4. Now the transformer has been fitted to our training data, we can then transform our testing set:

```
X_test_scaled = robust_scale.transform(X_test_imputed)
```

5. The last piece of the puzzle is to check once more that there are not any NULL values present in the arrays:

```
assert not np.isnan(X_train_scaled).any(),
```

```
"NaN values found in X_train_pp."  
assert not np.isnan(X_test_scaled).any(),  
"NaN values found in X_test_pp."
```

Feature selection

Feature selection is used to identify and retain the most relevant features in a dataset while removing redundant or irrelevant ones. This process improves model performance by reducing complexity, enhancing interpretability, and minimizing the risk of overfitting, while ensuring that only the features contributing meaningfully to the analysis or prediction are used. To perform this, we will go through these steps:

1. Following this description, this code snippet implements feature selection using the **Analysis of Variance (ANOVA)** statistical method, specifically for classification tasks. ANOVA is employed through the **SelectKBest** function from the **sklearn.feature_selection** module, which ranks features based on their statistical significance in distinguishing between different classes of the target variable.
2. Here, the **f_classif** function is used as the scoring function, which calculates the F-statistic (<https://www.statisticshowto.com/probability-and-statistics/hypothesis-testing/f-test/>) for each feature and its correlation with the target variable.
3. By specifying **k=15**, the method selects the top 15 features with the highest scores, reducing the dataset's dimensionality while retaining the model's most relevant features.
4. The feature selection process is performed in two stages. First, the **fit_transform** method is applied to the scaled training data (**X_train_scaled**) to select the best features based on the training labels (**y_train**).

5. Next, the same features are applied to the scaled test data (**X_test_scaled**) using the transform method to ensure consistency in feature representation across both datasets. If needed, the names of the selected features can be retrieved using **selector.get_feature_names_out()**, providing insight into which features are most significant for the classification task.
6. This process helps improve model performance, reduce overfitting, and streamline computational efficiency by focusing on the most informative variables. The code for this process is indicated in the following code:

```
1. from sklearn.feature_selection import SelectKBest, f_classif
2. k = 15
3. selector = SelectKBest(score_func=f_classif, k=k)
4. X_train_selected = selector.fit_transform(X_train_scaled, y_train)
5. X_test_selected = selector.transform(X_test_scaled)
6. selected_features = selector.get_feature_names_out()
```

Now we have both the training and testing set, with 15 features selected using the ANOVA F (difference in variance) metric. We will use this full preprocessed array to begin modeling.

Model training

We have reached the point where we are going to want to start training our model. The following subsections will explain how to set up the models and then how to perform Stratified K-fold, as opposed to the train-test splitting we have been undertaking up until this point.

Modeling setup

The first thing we are going to do is bring in three external models that are very useful when it comes to severely imbalanced data. To install them into

your project or **venv** you will need to pip install on the command line. We will use the following:

```
1. pip install catboost xgboost lightgbm --quiet
```

Let us have a quick detour on why these models are useful:

Algorithm	Strengths for imbalance data	How does it improve detection
Catboost	Native handling of categorical features Class weights in loss functions (logloss and cross entropy) Strong overfitting control (l2_leaf_reg)	Efficient feature encoding reduces noise Better minority class prediction due to robust regularization
XGBoost	Class weights support in the loss function L1/L2 regularization for overfitting prevention Focuses on hard-to-predict instance	Improved recall for minority classes Strong tree pruning ensures balance learning
LightGBM	Efficient on large datasets with histogram binning is_unbalance and scale_pos_weight for class imbalance Leaf-wise tree growth for focused learning	Faster training enables handling large imbalanced data Highlights minority classes using imbalanced specific parameters

Table 3.3: Algorithms outside of scikit-learn for imbalanced data

Once these external packages have been installed, we need to follow these steps to import the packages, make weight adjustments to scale the positive weight labels, and define the models in a dictionary for downstream modeling:

1. Import all the relevant packages:

```
from sklearn.svm import SVC
```

```
from catboost import CatBoostClassifier
```

```
from sklearn.linear_model import LogisticRegression
```

```
from xgboost import XGBClassifier
```

```
import lightgbm as lgb
```

2. Scale our minority classes; in this case, it is the thyroid disease class with severe imbalance. This creates a scaling factor:

```
scale_pos_weight = len(y_train[y_train == 0]) / len(y_train[y_train == 1])
```

3. Create a dictionary with our initialized models with hard-coded hyperparameters. We will get on to tuning these in *Chapter 9, Model Improvement Strategies*, but for now, we are just going to set some defaults:

```
models = {
```

```
    "logistic_regression": LogisticRegression(max_iter=2000,
```

```
        random_state=42,
```

```
        class_weight='balanced'),
```

```
    "svc": SVC(kernel='rbf', probability=True,
```

```
        random_state=42,
```

```
        class_weight='balanced'),
```

```
    "cat_boost": CatBoostClassifier(iterations=500,
```

```
        depth=6,
```

```
        learning_rate=0.1,
```

```

class_weights=[1, 10],
verbose=0),
"xgboost_scaled": XGBClassifier(n_estimators=500,
max_depth=6,
learning_rate=0.1,
scale_pos_weight=scale_pos_weight,
random_state=42),
"xgboost_unscaled": XGBClassifier(n_estimators=500,
max_depth=6,
learning_rate=0.1,
random_state=42),
"LGBM": lgb.LGBMClassifier(n_estimators=500,
learning_rate=0.1,
class_weight='balanced',
random_state=42,
max_depth=6)
}

```

The hyperparameters in this section can be explained as follows:

- **Iterations/Trees (iterations, n_estimators):** Controls the number of boosting steps (trees). Higher values increase complexity and potential accuracy but take longer to train.
- **Tree depth (max_depth, depth):** Limits the depth of each tree to prevent overfitting while capturing enough complexity.
- **Learning rate (learning_rate):** Defines the step size during optimization; smaller values improve stability but require more iterations.

- **Class weights (class_weight, class_weights, scale_pos_weight):** Adjusts for imbalanced data by giving more importance to minority classes.
- **Kernel (kernel):** Specifies the type of decision boundary (e.g., RBF for non-linear patterns in SVC).
- **Random State (random_state):** Ensures reproducibility by fixing the randomness in algorithms.

Stratified K-fold

Stratified K-fold is crucial for imbalanced data because it ensures that each fold in the cross-validation process maintains the same class distribution as the original dataset. This is particularly important in scenarios with imbalanced classes, as standard K-fold might result in folds with significantly skewed distributions, leading to biased evaluation metrics and unreliable model performance.

By preserving the class ratios, stratified K-fold allows the model to be trained and validated on representative samples, ensuring consistent performance across all folds. This method reduces the risk of overestimating the model's ability to generalize, particularly for the minority class, and helps produce more reliable metrics like precision, recall, and F1 score for imbalanced datasets. We will perform stratified K-fold in the following steps:

1. We will utilize this, as follows:

```
1. from sklearn.model_selection import StratifiedKFold
2. n_splits = 5
3. skf = StratifiedKFold(n_splits=n_splits,
4.                        shuffle=True,
5.                        random_state=42)
```

2. The final two steps will be to define an empty dictionary for our fitted models and to create the final arrays for our analysis:

- a. Create a fitted models dictionary:

```
fitted_models = {}
```

- b. Copy our arrays, for final modelling (optional step):

```
X_train_final = X_train_selected
```

```
X_test_final = X_test_selected
```

```
y_train_final = y_train
```

The next section will look into a function designed to compute a comprehensive set of evaluation metrics by comparing our ground truth labels with the corresponding predictions. This function will provide valuable insights into the model's performance, allowing us to assess its accuracy, precision, recall, and other key metrics for evaluating balanced and imbalanced datasets.

Evaluation metrics

In *Chapter 2, Introduction to Scikit-learn for Machine Learning*, we explored evaluation metrics such as precision, recall, F1 score, and accuracy. This chapter expands on these concepts by introducing confusion matrices, ROC curves, and the ROC AUC score. While we will visualize ROC curves and confusion matrices in the following sections, let us first focus on understanding the **receiver operating characteristic (ROC) area under the curve (AUC)** score.

The ROC AUC score measures a model's ability to distinguish between classes, particularly in binary classification problems. It is derived from the ROC curve plots the **true positive rate (TPR)** against the **false positive rate (FPR)** at various threshold settings. The ROC AUC score represents the area under the ROC curve, summarizing the model's performance across all thresholds. A score of 1.0 indicates perfect discrimination, while 0.5 suggests no better performance than random guessing.

Essentially, the ROC AUC score evaluates how well the model ranks positive instances higher than negative ones. It is particularly valuable in

datasets with class imbalances because it focuses on ranking predictions rather than absolute values, making it less sensitive to skewed class distributions. This makes it an essential metric for assessing classification models.

We are going to want a function we can use in the evaluation loop later on in the following sections, so for now, we will define it:

```
1. from sklearn.metrics import (  
2.     precision_score,  
3.     recall_score,  
4.     f1_score,  
5.     accuracy_score,  
6.     confusion_matrix,  
7.     roc_auc_score  
8. )  
9. def compute_metrics(y_true, y_pred, y_pred_proba):  
10.     precision = precision_score(y_true, y_pred)  
11.     recall = recall_score(y_true, y_pred)  
12.     f1 = f1_score(y_true, y_pred)  
13.     accuracy = accuracy_score(y_true, y_pred)  
14.     cm = confusion_matrix(y_true, y_pred)  
15.     roc_auc = roc_auc_score(y_true, y_pred_proba)  
16.     metrics = {  
17.         'precision': precision,  
18.         'recall': recall,  
19.         'f1_score': f1,  
20.         'accuracy': accuracy,  
21.         'roc_auc': roc_auc,
```

```
22.         'confusion_matrix': cm
23.     }
24.     return metrics
```

This function computes a comprehensive set of evaluation metrics, including precision, recall, F1 score, accuracy, ROC AUC score, and the confusion matrix, to assess the performance of a classification model based on proper labels, predicted labels, and predicted probabilities.

Understanding the confusion matrix

A confusion matrix is a tabular representation used to evaluate the performance of a classification model by comparing its predictions to the true class labels. It provides detailed insights into how well the model distinguishes between different classes. The matrix has rows and columns corresponding to the actual and predicted classes, making it a powerful diagnostic tool for understanding classification errors and successes.

The matrix is typically divided into four components for binary classification:

- **TP:** Instances where the model correctly predicted the positive class.
- **TN:** Instances where the model correctly predicted the negative class.
- **FP:** Instances where the model incorrectly predicted the positive class (Type I error).
- **FN:** Instances where the model incorrectly predicted the negative class (Type II error).

These components allow us to derive important metrics like precision, recall, and F1 score. Visualizing a confusion matrix is often more intuitive than numerical metrics alone. Here is a simple implementation in Python that uses Seaborn and Matplotlib to plot a confusion matrix.

The following function **plot_simple_confusion_matrix**, also supports normalization to show the proportions rather than absolute values, making it easier to compare models with different scales:

```
1. import seaborn as sns
2. import matplotlib.pyplot as plt
3. import numpy as np
4. def plot_simple_confusion_matrix(cm, classes,
5.                                model_name, normalize=False,
6.                                cmap='Blues'):
7.     if normalize:
8.         cm = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
9.         fmt = '.2f'
10.        title = f'Normalized Confusion Matrix for {model_name}'
11.    else:
12.        fmt = 'd'
13.        title = f'Confusion Matrix for {model_name}'
14.    plt.figure(figsize=(6,6))
15.    sns.heatmap(cm, annot=True, fmt=fmt, cmap=cmap, cbar=False,
16.                xticklabels=classes, yticklabels=classes)
17.    plt.title(title)
18.    plt.ylabel('True Label')
19.    plt.xlabel('Predicted Label')
20.    plt.show()
```

Here is an explanation of the components in the code:

- **cm**: The confusion matrix, typically computed using **sklearn.metrics.confusion_matrix**.

- **normalize:** A flag to toggle between absolute values and normalized proportions.
- **classes:** The names of the classes being predicted are used as axis labels.
- **sns.heatmap:** Creates a visual heatmap of the confusion matrix with annotations for easier interpretation.
- **cmap='Blues':** Defines the color scheme for the heatmap, with shades indicating magnitude.

This visualization provides a clear breakdown of model performance, helping identify patterns of misclassification and areas for improvement:

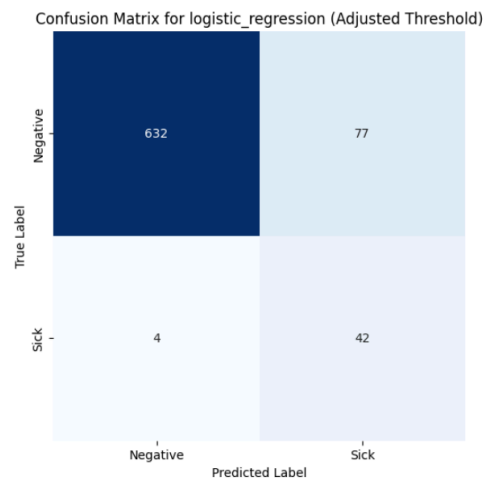


Figure 3.6: Simple correlation matrix

A more detailed confusion matrix is available by the **modelviz** package and can be imported using:

```
1. from modelviz.confusion_matrix import plot_confusion_matrix
```

We will use this later on in our evaluation loop, but for now, it will look like the following figure:

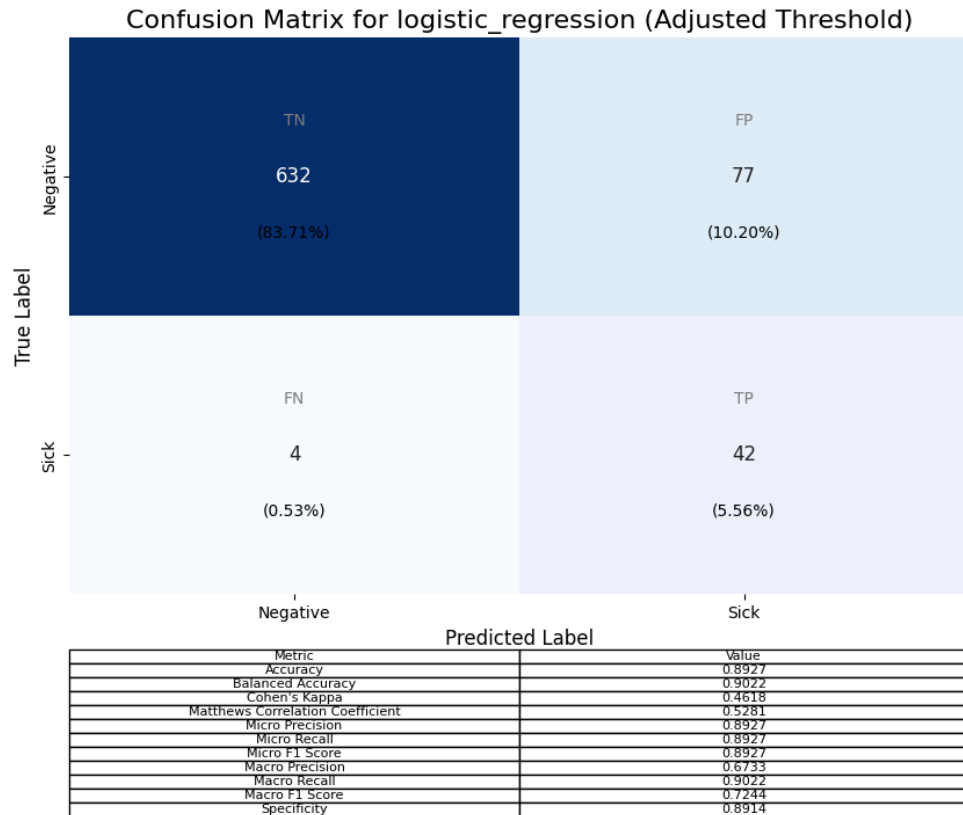


Figure 3.7: Confusion matrix from modelviz PyPi package

We will come back to these later on, as these will be our primary form of assessment, alongside ROC curves, which we have not discussed yet. The next step is to build our training loop.

Over and under sampling methods

When working with imbalanced datasets, where one class significantly outnumbers the other, machine learning models often become biased towards the majority class. This bias can result in poor predictive performance for the minority class, which is frequently the class of interest, such as detecting fraudulent transactions or identifying rare diseases. To mitigate this, two main approaches are commonly used:

- Oversampling
- Undersampling

The following table describes the type of sampling, the associated Python package, and a description of how it is used:

Type	Python package	Description
Synthetic Minority Oversampling Technique (SMOTE)	imbalanced-learn	Synthetic Minority Oversampling Technique. Generates synthetic samples by interpolating existing minority samples. Suitable for numeric datasets.
Random over sampling (ROS)	imbalanced-learn	Replicates samples from the minority class to balance the dataset. Simple but can lead to overfitting.
Adaptive Synthetic Sampling (ADASYN) approach for imbalanced learning	imbalanced-learn	Adaptive Synthetic Sampling. Focuses on generating synthetic samples in regions where the minority class is sparse. Variant of SMOTE.
Synthetic Minority Oversampling Technique Edited Nearest Neighbors (SMOTEENN)	imbalanced-learn	Combination of SMOTE (oversampling) and edited nearest neighbors (under sampling). Balances the dataset and removes noisy samples.

Type	Python package	Description
SMOTE-Tomek	imbalanced-learn	Combines SMOTE with Tomek Links (under sampling technique). Removes borderline samples after oversampling.
Random undersampling	imbalanced-learn	Reduces the dataset by removing samples from the majority class to balance the dataset. This can lead to the loss of important information.
Class weight adjustment	scikit-learn	Adjusts model training weights to penalize misclassification of the minority class more heavily. It does not change the dataset but impacts model training.

Table 3.4: Over and under sampling techniques

As you can see, many of these techniques are outside of scikit-learn, and require the **imbalanced-learn** package (<https://imbalanced-learn.org/stable/>).

To install this, use the following command in your terminal/command line:

```
1. pip install imbalanced-learn --quiet
```

Building the training and evaluation loop

In the next sections, we will build the training and evaluation loop to train the model and then evaluate it afterwards.

Building the training loop

Follow these steps to build the training loop:

1. In order to build our training loop, we will need to import our packages to use:

```
from sklearn.model_selection import cross_val_score
```

```
from imblearn.pipeline import Pipeline
```

```
from sklearn.base import clone
```

```
from imblearn.combine import SMOTEENN
```

2. The next step is to create some empty dictionaries, determine the evaluation metric to optimize (recall in this instance), and set our adjustment probability threshold cut off for the binary classification problem (set to 0.5, meaning anything above this threshold would be deemed a risk, and anything below, would be deemed no risk):

```
fitted_models = {}
```

```
model_scores = {}
```

```
threshold_results = {}
```

```
EVAL_METRIC = 'recall'
```

```
ADJUSTED_THRESHOLD = 0.5
```

3. In the following code, we define the training loop and post the code, we will discuss what each step does:

```
for model_name, model in models.items():
```

```
    print(f"Evaluating {model_name}...")
```

```
    pipeline = Pipeline([
```

```
        ('smoteenn', SMOTEENN(random_state=42, sampling_
strategy=0.5)),
```

```
        ('model', model)
```

```

    ])

    cv_scores = cross_val_score(pipeline, X_train_final,
                                y_train_final, cv=skf, scoring=EVAL_METRIC)

    mean_score = cv_scores.mean()
    std_score = cv_scores.std()

    model_scores[model_name] = {'mean_score': mean_score,
                                'std_score': std_score}

    print(f"{model_name} Mean {EVAL_METRIC} Score: {mean_score:.4f} (+/- {std_score:.4f})\n")

    smoteenn = SMOTEENN(random_state=42,
                        sampling_strategy=0.5)

    X_resampled, y_resampled = smoteenn.fit_resample(X_train_final, y_train_final)

    model_clone = clone(model)
    model_clone.fit(X_resampled, y_resampled)
    fitted_models[model_name] = model_clone

    y_pred_proba = model_clone.predict_proba(X_test_final)[:, 1]

    y_pred_adjusted = (y_pred_proba >= ADJUSTED_THRESHOLD).astype(int)

    metrics = compute_metrics(y_test, y_pred_adjusted, y_pred_proba)

    threshold_results[model_name] = metrics

```

A breakdown of each of these steps is bulleted here:

- **Iterate through models:**
 - Loops through each model in the model's dictionary and evaluates them sequentially.
- **Set up a pipeline:**

- Creates a pipeline object with two components:
 - **SMOTEENN**: Combines SMOTE (oversampling the minority class) and ENN (removing noisy samples) to balance the dataset.
 - **model**: The current classification model to be trained.
- This pipeline ensures that SMOTEENN is applied during cross-validation, preventing data leakage.
- **Cross-validation with SMOTEENN:**
 - Uses **cross_val_score** to perform cross-validation on the training data, applying SMOTEENN and evaluating the model using the specified evaluation metric (**EVAL_METRIC**).
 - Computes mean and standard deviation of the cross-validation scores and stores them for reporting and comparison.
- **Resample the entire training dataset:**
 - Applies SMOTEENN to the entire training dataset to generate a balanced dataset (**X_resampled**, **y_resampled**) for final model training.
- **Clone the model:**
 - Clones the current model to ensure it starts with a fresh instance, avoiding unintended reuse of the model fitted during cross-validation.
- **Train the model on resampled data:**
 - Fits the cloned model on the balanced training dataset produced by SMOTEENN.
- **Store the fitted model:**
 - Saves the trained model to **fitted_models** for future use (e.g., predictions on unseen data).
- **Predict probabilities on test data:**

- Uses the trained model to predict probabilities for the test dataset (**y_pred_proba**) without applying any resampling or SMOTEENN.
- **Adjust predictions using a custom threshold:**
 - Applies the specified threshold (**ADJUSTED_THRESHOLD**) to the predicted probabilities to generate binary predictions (**y_pred_adjusted**).
- **Evaluate metrics:**
 - Computes evaluation metrics such as precision, recall, F1 score, accuracy, and ROC AUC using the **compute_metrics** function, we defined in *Evaluate Metrics* section.
- **Store metrics:**
 - Saves the computed metrics for the current model into the **threshold_results** dictionary for later analysis and comparison.

Triggering this part of the code will evaluate all our candidate's model in one go, and is included for you to use in your own projects. To reference individual model results, use the following command:

```
1. threshold_results['cat_boost']
```

This will output something similar to this dictionary structure:

```
{'precision': 0.7258064516129032, 'recall': 0.9782608695652174,
'f1_score': 0.8333333333333334, 'accuracy': 0.976158940397351,
'roc_auc': 0.9962592751579077, 'confusion_matrix': array([[692, 17], [
1, 45]], dtype=int64)}
```

Building the evaluation loop

Now we have the models fitted, the next step will be to create an evaluation loop, get the model results, calculate the metrics (using the compute metrics function), create confusion matrices and ROC curves, and optimize the curves with an optimal threshold (using Youden's J measure).

We will cover ROC curves in a separate section, as this evaluation loop will output multiple plots. The following code implements the evaluate function:

```
1. from modelviz.roc import plot_roc_curve_with_youdens_thresholds
2. from sklearn.metrics import roc_curve
3.
4. def evaluate_model(model_name, model, X_test, y_test,
5.                    adjusted_threshold=0.5,
6.                    classes=['Negative', 'Sick'],
7.                    calculate_youdens=False, plot_simple_cm=False):
8.     print(f"Evaluating {model_name} on test data...")
9.     y_pred_proba = model.predict_proba(X_test)[:, 1]
10.    y_pred_adjusted = (y_pred_proba >= adjusted_threshold).astype(int)
11.    adjusted_metrics = compute_metrics(y_test, y_pred_adjusted, y_pred_proba)
12.    fpr, tpr, thresholds = roc_curve(y_test,
13.                                     y_pred_proba)
14.    evaluation_results = {
15.        'fitted_model': model,
16.        'adjusted_threshold': adjusted_threshold,
17.        'metrics_at_adjusted_threshold': adjusted_metrics,
18.        'roc_curve': (fpr, tpr, thresholds),
19.    }
20.    if calculate_youdens:
21.        youdens_j = tpr - fpr
22.        optimal_idx = np.argmax(youdens_j)
```

```
23.     optimal_threshold = thresholds[optimal_idx]
24.     max_youdens_j = youdens_j[optimal_idx]
25.     print(f"Maximum Youden's J for {model_name}: {max_youdens_j:.4f} at threshold {optimal_threshold:.4f}\n")
26.     y_pred_youden = (y_pred_proba >= optimal_threshold).astype(int)
27.     youden_metrics = compute_metrics(
28.         y_test,
29.         y_pred_youden,
30.         y_pred_proba)
31.     evaluation_results.update({
32.         'optimal_threshold': optimal_threshold,
33.         'metrics_at_optimal_threshold': youden_metrics,
34.         'max_youdens_j': max_youdens_j
35.     })
36.     plot_confusion_matrix(
37.         youden_metrics['confusion_matrix'],
38.         classes=classes,
39.         model_name=f"{model_name} (Youden's J Threshold)",
40.         table_fontsize=8)
41.     if plot_simple_cm:
42.         plot_simple_confusion_matrix(
43.             adjusted_metrics['confusion_matrix'],
44.             classes=classes,
45.             model_name=f"{model_name} (Adjusted Threshold)")
46.
47.     else:
```

```

48.     plot_confusion_matrix(
49.         adjusted_metrics['confusion_matrix'], classes=classes,
50.         model_name=f"{model_name} (Adjusted Threshold)",
51.         table_fontsize=8)
52. plot_roc_curve_with_youdens_thresholds(
53.     fpr, tpr,
54.     thresholds,
55.     adjusted_metrics['roc_auc'],
56.     model_name,
57.     adjusted_threshold,
58.     optimal_threshold if calculate_youden else None)
59. return evaluation_results

```

Here is an explanation of the function:

1. **Log message:** Prints a message indicating the evaluation process for the given model.
2. **Predict probabilities and adjust threshold:**
 - a. Computes prediction probabilities (**y_pred_proba**) for the test data.
 - b. Converts probabilities to class predictions using the **adjusted_threshold**.
3. **Compute metrics:** Calculates metrics (e.g., accuracy, confusion matrix) for predictions using the adjusted threshold.
4. **ROC curve and AUC:**
 - a. Computes the ROC curve (**fpr, tpr, thresholds**).
 - b. Stores ROC-related information and adjusted metrics in the **evaluation_results** dictionary.
5. **Youden's J calculation (if calculate_youden is True):**

- a. Finds the optimal threshold that maximizes *Youden's J* statistic (**tpr - fpr**).
 - b. Logs the maximum *Youden's J* and its corresponding threshold.
 - c. Computes metrics for predictions made with this optimal threshold.
 - d. Updates **evaluation_results** with optimal threshold details and corresponding metrics.
 - e. Plots the confusion matrix for the *Youden's J* optimal threshold.
6. **Plot confusion matrices:** Plots either a standard or simplified confusion matrix for the adjusted threshold (and for *Youden's J* if applicable).
 7. **Plot ROC curve:** Plots the ROC curve, marking thresholds including the adjusted and optionally the *Youden's J* optimal threshold.
 8. **Return results:** Returns the **evaluation_results** dictionary containing:
 - a. The fitted model.
 - b. Adjusted threshold and its metrics.
 - c. ROC curve details.
 - d. *Youden's J* threshold and metrics (if calculated).

To loop through the evaluation results dictionary, we finally use the following script to generate all the evaluation visuals, as described in the previous steps:

```
1. ADJUSTED_THRESHOLD = 0.5
2. all_evaluation_results = {}
3. for model_name, model in fitted_models.items():
4.     evaluation_results = evaluate_model(
```



```
5.         model_name=model_name,
6.         model=model,
7.         X_test=X_test_final,
8.         y_test=y_test,
9.         adjusted_threshold=ADJUSTED_THRESHOLD,
10.        calculate_youdens=True,
11.        plot_simple_cm=False
12.    )
13.
14.    all_evaluation_results[model_name] = evaluation_results
```

This function loops through the final model's dictionary as part of the original training loop, and it then uses the evaluate model function created in the previous step to make the predictions, compute the metrics, and generate the confusion matrices and the ROC curves.

Receiver operator characteristics curve

A ROC curve is a graphical representation used in statistics and machine learning to assess the performance of a binary classification model. The curve plots the TPR, or sensitivity, against the FPR at various threshold settings. Showing the trade-off between sensitivity and specificity helps visualize how well a model distinguishes between the positive and negative classes across all possible decision thresholds.

The area under the ROC curve (AUC-ROC) provides a single metric summarizing the model's ability to discriminate between the two classes, with a value closer to 1 indicating better performance.

Key elements of the ROC curve include the diagonal line, which represents a random classifier, and points above this line, which indicate better-than-random predictions. Models with curves that hug the top-left corner of the plot are highly effective, demonstrating a high TPR with a low FPR. ROC curves are widely used in medicine, finance, and other domains where

binary classification problems involve imbalanced datasets or critical cost-sensitive decisions. They comprehensively evaluate a model's performance, complementing traditional metrics like accuracy.

Figure 3.8 shows the threshold performance of our chosen **ADJUSTED_THRESHOLD** and an optimal threshold calculated using *Youden's J* statistic (essentially, the ROC curve optimal threshold selection metric). The red point is the threshold we have specified (with 0.02 indicating the FPR and 0.96 indicating the TPR), and the green point is the *Youden's J* optimal threshold (0.05 FPR and 1 TPR).

To analyze these charts, you can find various points on the line, but there is always a tradeoff between FPR and TPR. Refer to the following figure:

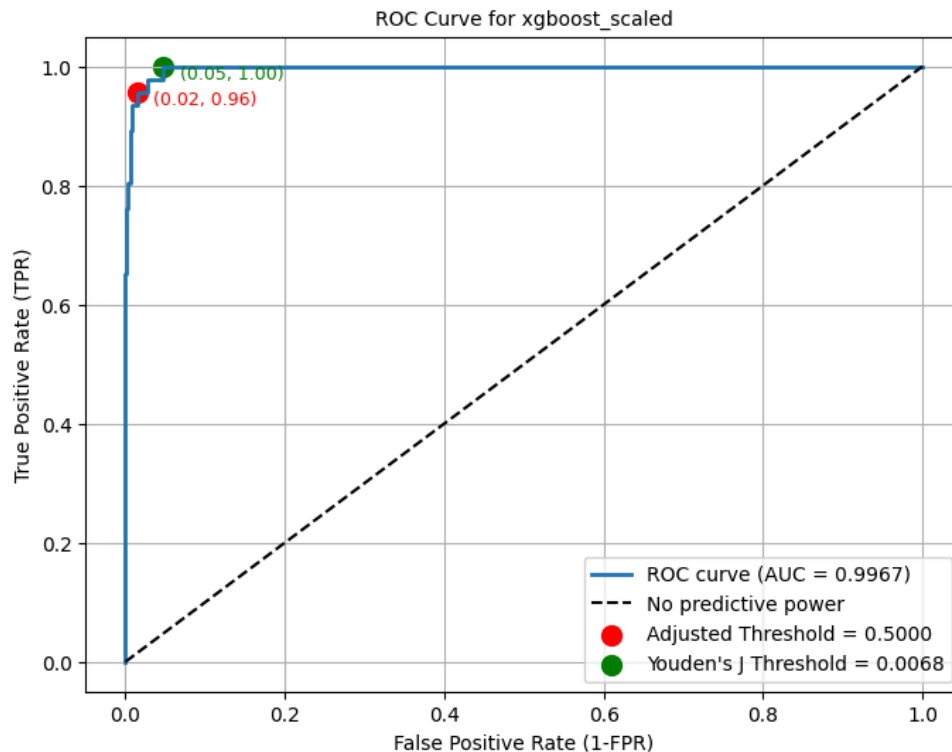


Figure 3.8: Confusion matrix from modelviz PyPi package

In addition to this ROC curve, the evaluate function will output a *Youden's* optimized confusion matrix and a confusion matrix based on the threshold you have chosen, an example of these matrices is captured in the *Confusion matrix* section.

Conclusion

In this chapter, we looked into the intricacies of supervised binary classification, exploring its foundational principles, key algorithms, and practical implementation strategies. Binary classification is a core task in machine learning that involves predicting one of two possible outcomes, often used in domains like medical diagnostics or fraud detection. We learned about the theoretical underpinnings of logistic regression, including the transformation of log odds into probabilities using the sigmoid function, which forms the basis for mapping continuous data to binary outcomes. These concepts provide a solid groundwork for understanding and building binary classifiers.

We examined practical solutions to ensure robust model performance when faced with severely imbalanced datasets, as in the thyroid disease prediction problem. Techniques like resampling (e.g., SMOTE and SMOTEENN), adjusting class weights, and selecting optimal decision thresholds using metrics like Youden's J statistic and our own defined thresholds were employed to tackle the challenges of imbalanced classes.

Additionally, evaluation strategies, including confusion matrices and ROC curves, offered insights into model strengths and weaknesses. By integrating these tools and methodologies, we emphasized building reliable models that effectively detect minority classes, even under challenging conditions.

Finally, this chapter provided reusable recipes, such as preprocessing pipelines, feature selection strategies, and model evaluation loops, enabling readers to apply these techniques to their own projects. Combining scikit-learn's capabilities with advanced tools like CatBoost, XGBoost, and LightGBM gives you a comprehensive framework to address real-world classification problems. These pragmatic approaches empower readers to confidently handle complex datasets and develop models that deliver actionable insights.

In the next chapter we will explore multi-class machine learning problems and highlight the increasing complexity of dealing with multiple classes.

Points to remember

Here are some key points summed up from this chapter:

- **Binary classification:** Predicts one of two possible outcomes, represented as 0 and 1 or *Yes* and *No*.
- **Class imbalance:** A common challenge in real-world datasets; resampling techniques like SMOTE and class weight adjustments help address it.
- **Confusion matrices:** To evaluate classification performance and provide insights into true positives, false negatives, and false negatives.
- **Cross-validation:** Ensures robust evaluation by dividing data into multiple training and test sets.
- **Data preprocessing** includes missing value imputation, feature scaling, and categorical variables encoded to prepare datasets for modeling.
- **Evaluation metrics:** Use precision, recall, F1 score, accuracy, and ROC AUC for comprehensive model assessment.
- **Feature scaling:** StandardScaler, MinMaxScaler, and RobustScaler ensure models are not sensitive to outliers or feature magnitude.
- **Feature selection:** Retains the most relevant features using methods like ANOVA F-statistics to simplify datasets.
- **Logistic regression:** A foundational algorithm for binary classification that transforms input features into probabilities.
- **Missing data imputation:** Methods like MICE and SimpleImputer preserve dataset integrity and patterns by filling missing values effectively.

- **One-hot encoding:** Transforms categorical variables into a binary matrix format, enabling machine-readable input.
- **Pipelines:** Streamline machine learning workflows by chaining preprocessing, feature engineering, and modeling steps.
- **ROC curve:** Visualizes the trade-off between sensitivity and specificity; AUC measures overall performance.
- **Resampling techniques:** Methods like SMOTE and SMOTEENN balance datasets by addressing class imbalances.
- **Stratified K-fold:** Maintains class distribution across folds to ensure reliable evaluation in cross-validation.
- **Threshold optimization:** Youden's J statistic identifies optimal thresholds for balancing true positive and false positive rates.

Multiple choice questions

1. **What does the sigmoid function do in logistic regression?**
 - a. Computes the mean of input features
 - b. Maps log-odds to probabilities between 0 and 1
 - c. Scales data to have a standard deviation of 1
 - d. Selects the best features for a model
2. **What is the purpose of a confusion matrix?**
 - a. To visualize the correlation between features
 - b. To evaluate model predictions by comparing true and predicted labels
 - c. To reduce the number of features in a dataset
 - d. To optimize hyperparameters
3. **What does stratified K-fold ensure during cross-validation?**
 - a. Faster computation of results

- b. Equal distribution of samples across folds
 - c. Same class ratio in all folds as in the original dataset
 - d. Transformation of categorical data to numerical
4. **What is the main advantage of using SMOTE for imbalanced datasets?**
- a. It removes noisy data points
 - b. It reduces the size of the dataset by removing majority samples
 - c. It optimizes hyperparameters automatically
 - d. It generates synthetic samples for the minority class
5. **In a ROC curve, what does the AUC represent?**
- a. The optimal threshold for classification
 - b. The proportion of true positive and false positive rates
 - c. The model's ability to distinguish between classes
 - d. The feature importance scores
6. **What does the class_weight parameter in scikit-learn models do?**
- a. Reduces the number of features
 - b. Adjusts the importance of each class during model training
 - c. Normalizes the dataset to a standard range
 - d. Automatically tunes hyperparameters
7. **What is the main advantage of using RobustScaler over StandardScaler?**
- a. It is faster to compute
 - b. It handles outliers more effectively
 - c. It works with categorical data
 - d. It ensures values are between 0 and 1

8. **Why is cross-validation important in model evaluation?**
- a. It reduces the size of the dataset
 - b. It prevents data leakage
 - c. It ensures the model is evaluated on multiple splits of the data
 - d. It tunes hyperparameters automatically
9. **Which package is commonly used for oversampling techniques like SMOTE?**
- a. Imbalanced-learn
 - b. NumPy
 - c. Scikit-learn
 - d. Pandas
10. **What is the purpose of the StandardScaler in data preprocessing?**
- a. To eliminate outliers in the dataset
 - b. To ensure features have a mean of 0 and a standard deviation of 1
 - c. To reduce the dimensionality of the dataset
 - d. To balance the dataset

Answers

1	b
2	b
3	c
4	d
5	c
6	b
7	b
8	c

9	a
10	b

Key terms

Some key terms used in this chapter are:

- **Accuracy:** An evaluation metric that calculates the proportion of correct predictions over the total number of predictions, often less reliable in cases of class imbalance.
- **ADASYN:** An oversampling technique that generates synthetic samples in regions where the minority class is sparse, improving representation in imbalanced datasets.
- **API:** A structured set of commands and functions in scikit-learn that allows users to interact with machine learning algorithms in a consistent, user-friendly manner.
- **CatBoost:** A gradient boosting library with native handling of categorical features, class weights for imbalance adjustment, and strong overfitting control.
- **Class weight:** A technique for handling imbalanced datasets by assigning higher penalties to misclassified instances of the minority class during model training.
- **Confusion matrix:** A table that provides insight into the number of true positives, true negatives, false positives, and false negatives, useful for understanding model performance in classification tasks.
- **Cross-validation (CV):** A technique that splits data into multiple training and testing sets to evaluate model performance more reliably, reducing the risk of overfitting.
- **F1 score:** A harmonic mean of precision and recall, providing a balanced measure of both metrics, particularly valuable in imbalanced datasets.

- **Gradient boosting:** An ensemble technique that builds models sequentially, with each model focusing on correcting the errors of its predecessor, effective for complex patterns.
- **Iterative imputer:** A method for imputing missing values by iteratively predicting missing data using other features, ensuring more accurate estimations in datasets with missing values.
- **K-fold cross-validation:** A type of cross-validation that splits data into K subsets, training on K-1 and testing on the remaining subset, repeated K times for a robust evaluation.
- **Logistic regression:** A classification algorithm that models the probability of a binary outcome, suitable for scenarios requiring interpretable results.
- **Normalization:** The process of scaling data to a standard range (typically 0 to 1) to improve model performance, particularly for algorithms sensitive to feature magnitude, like k-nearest neighbors.
- **ROC curve:** A graphical representation of a classification model's performance, plotting the trade-off between true positive rate and false positive rate across thresholds.
- **SMOTE:** A technique for generating synthetic samples for the minority class by interpolating between existing samples to address class imbalance.
- **StandardScaler:** A scikit-learn transformer that scales features to have a mean of 0 and a standard deviation of 1, commonly used to normalize numerical data.
- **Stratified K-fold:** A variation of K-fold cross-validation that ensures the same class distribution is maintained across all folds, particularly useful for imbalanced datasets.
- **Threshold adjustment:** The process of modifying the decision threshold for classification probabilities to optimize performance

metrics like precision or recall.

- **Youden's J statistic:** A metric used to identify the optimal threshold on a ROC curve by maximizing the difference between true positive rate and false positive rate.

Join our Discord space

Join our Discord workspace for latest updates, offers, tech happenings around the world, new releases, and sessions with the authors:

<https://discord.bpbonline.com>



CHAPTER 4

Supervised Multi-class Classification

Introduction

In this chapter, we will explore supervised multi-class classification, focusing on predicting one of multiple distinct categories from input data. Unlike binary classification, which differentiates between two outcomes, multi-class classification involves distinguishing among several categories that may have overlapping features. We will examine the formation of decision boundaries and various algorithms' approaches to tackle these tasks. Evaluation metrics such as log loss, precision, recall, and F1 scores will be discussed to assess model performance in multi-class scenarios. These concepts are essential for developing effective predictive models in applications like customer churn prediction, sentiment analysis, and image recognition.

Beyond theory, this chapter offers practical guidance on implementing multi-class classification using scikit-learn. We will cover data preprocessing techniques, including feature scaling, handling missing values, and encoding categorical variables. To ensure fair model performance, we will also explore addressing class imbalance through strategies like SMOTE and under-sampling. We will investigate various classification approaches, such as One-vs-Rest, One-vs-One, and **Error-Correcting Output Codes (ECOC)**, to extend binary classifiers for multi-class tasks.

Structure

This chapter covers the following topics:

- Introduction to multi-class classification
- Decision boundaries for random forests and logistic regression
- Importance of log loss in model training
- Predicting customer churn use case
- Handling categorical and numerical features for classification
- Evaluation metrics for multi-class classification
- One-vs-Rest and One-vs-One strategies
- Modeling, evaluation, and model trade-offs

Objectives

By the end of this chapter, readers will have a comprehensive understanding of supervised multi-class classification, including the formation of decision boundaries and the application of various algorithms to distinguish between multiple categories with overlapping features. They will be proficient in evaluating model performance using metrics such as log loss, precision, recall, and F1 scores, which are essential in real-world applications like customer churn prediction, sentiment analysis, and image recognition.

Additionally, readers will gain hands-on experience implementing multi-class classification using scikit-learn, mastering data preprocessing techniques such as feature scaling, handling missing values, and encoding categorical variables. They will also learn to address class imbalance through strategies like SMOTE and under-sampling, ensuring fair model performance. Finally, readers will explore various classification approaches, including One-vs-Rest, One-vs-One, and ECOC, to effectively extend binary classifiers for multi-class tasks.

Introduction to multi-class classification

Multi-class classification is a supervised learning problem that predicts one of three or more distinct categories for a given input. Unlike binary classification, which focuses on distinguishing between two possible outcomes, multi-class classification requires the model to handle a broader range of possibilities. For instance, a model might categorize data into labels such as low risk, medium risk, or high risk, each representing a distinct class. This problem arises in many applications, including customer churn prediction, sentiment analysis, and image recognition. The model must learn decision boundaries in the feature space to separate the data into multiple categories based on patterns in the training data.

The primary distinction between multi-class and binary classification lies in the complexity of decision-making. Binary classification involves determining whether an input belongs to one of two categories, making the decision boundaries relatively simple. In contrast, multi-class classification requires separating the feature space into regions corresponding to each class, often with overlapping or ambiguous boundaries; we will use some scikit-learn functions in the next section to visualize how this is applied across two features as a way to visualize these decision boundaries.

This complexity also extends to evaluation metrics, which must assess the model's performance for each class individually and in aggregate. Unlike binary problems, where accuracy, precision, and recall are computed for a single decision, multi-class classification requires these metrics to be calculated for each class and summarized using averages such as macro or weighted scores.

Multi-class classification introduces unique challenges that do not typically arise in binary settings. One significant issue is *class imbalance*, where some classes are underrepresented in the dataset compared to others. This imbalance can cause the model to favor majority classes, reducing its ability to predict minority classes correctly. Another challenge is *inter-class similarity*, where features of different classes overlap, making it harder for the model to learn accurate decision boundaries. Furthermore, the computational complexity of some multi-class strategies, such as One-vs-

One classification, increases with the number of classes, requiring more resources and time. These factors make multi-class classification inherently more intricate than binary classification.

Despite these challenges, multi-class classification is essential for addressing problems that require nuanced predictions (meaning multi-class classifiers can assign labels across a spectrum of distinct groups; their predictions, therefore, carry more detail and *nuance* about what is happening in the data). These models provide richer insights than binary classifiers by categorizing data into multiple classes. For example, in a customer churn scenario, a model predicting risk levels (low, medium, high) allows businesses to tailor interventions to each group, prioritizing high-risk cases while maintaining cost-efficiency for lower-risk segments. Beyond churn, the ability to handle multiple categories is fundamental to applications such as multi-class disease diagnosis, document categorization, and fraud detection. Multi-class classification expands the scope of machine learning by enabling models to handle the complexity and diversity of real-world problems effectively.

In the next section, we will implement a way to visualize these decision boundaries across two features to simplify and visualize them. However, the true relationships would be hard for a human to compute in a multidimensional feature space.

Decision boundaries for random forests and logistic regression

As we explained in the previous section, *decision boundaries* are a critical concept in multi-class classification. They represent how a model divides the feature space to assign inputs to specific classes. These boundaries determine the model's predictions for unseen data. The better the separation between classes achieved by the decision boundaries, the more accurate the model will be on new, unseen data. Visualizing decision boundaries provides insights into the model's understanding of the data and highlights

areas where the model might struggle, such as overlapping regions or class imbalances.

It is essential to recognize that different algorithms create decision boundaries in distinct ways. For instance, in multinomial logistic regression, the decision boundaries are linear, meaning they form straight lines (or planes in higher dimensions) to separate classes. This simplicity makes logistic regression effective for linearly separable data but less suitable for more complex, nonlinear relationships.

On the other hand, methods like *tree-based algorithms* (e.g., decision trees, random forests) generate nonlinear decision boundaries, adapting flexibly to the data's structure. These methods excel in capturing intricate patterns but may risk overfitting if not correctly regularized.

Decision boundaries can be highly complex and nonlinear for models like **support vector machines (SVMs)** and neural networks, depending on the kernel used (in SVMs) or the network's architecture. For example, SVMs with a **radial basis function (RBF)** kernel can create intricate, curved boundaries, while neural networks can learn arbitrarily shaped boundaries given sufficient training data and hidden layers.

The type of decision boundary a model learns impacts its performance and interpretability. Linear boundaries, like those in logistic regression, are straightforward to understand and visualize, making them appealing for problems where interpretability is crucial. Nonlinear boundaries, while powerful, can be more challenging to interpret, especially in high-dimensional spaces. Understanding these differences is key to selecting the right model for a given problem and assessing how well it generalizes to new data.

The following code shows how we can generate a decision boundary utilizing NumPy and scikit-learn utilities, and we will take you through the steps needed to get there:

1. Import the packages we will need to produce the decision boundary visualization:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import make_blobs
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.inspection import DecisionBoundaryDisplay
```

```
from sklearn.ensemble import RandomForestClassifier
```

2. Use the **make_blobs** function we previously utilized in *Chapter 2, Introduction to Scikit-learn for Machine Learning*, to generate a synthetic classification dataset:

```
X, y = make_blobs(n_samples=300, centers=3,  
                  random_state=42, cluster_std=1.5)
```

3. Next, instantiate the random forest and logistic regression classifiers and save them with a variable name of your choosing:

```
log_reg = LogisticRegression(multi_class='multinomial', solver=  
                             'lbfgs')
```

```
rf = RandomForestClassifier(random_state=42)
```

4. As we have the classifiers instantiated, we will now fit the classifiers:

```
rf.fit(X, y)
```

```
log_reg.fit(X,y)
```

5. Now we will use the **DecisionBoundaryDisplay** function from scikit-learn and create a panel chart to show the decision boundaries of the logistic regression classifier:

```
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
```

```
DecisionBoundaryDisplay.from_estimator(log_reg, X,
```



```

        response_method="predict",
        alpha=0.5, cmap="Blues",
        ax=axes[0])
scatter1 = axes[0].scatter(X[:, 0], X[:, 1],
        c=y, cmap="Blues",
        edgecolor="k")
axes[0].set_title("Logistic Regression: Decision Boundaries",
        fontsize=14)
axes[0].set_xlabel("Feature 1")
axes[0].set_ylabel("Feature 2")
axes[0].grid(alpha=0.2)

```

6. This line then creates the next visual for the random forest classifier:

```

DecisionBoundaryDisplay.from_estimator(rf, X, response_method="predict",
        alpha=0.5, cmap="Blues",
        ax=axes[1])
scatter2 = axes[1].scatter(X[:, 0], X[:, 1], c=y, cmap="Blues",
        edgecolor="k")
axes[1].set_title("Random Forest: Decision Boundaries", fontsize=14)
axes[1].set_xlabel("Feature 1")
axes[1].set_ylabel("Feature 2")
axes[1].grid(alpha=0.2)

```

7. Add color bars to our chart to show differences in points, i.e., by their class:

```

cbar1 = fig.colorbar(scatter1, ax=axes[0], ticks=[0, 1, 2])
cbar1.set_label("Classes")

```

```
cbar2 = fig.colorbar(scatter2, ax=axes[1], ticks=[0, 1, 2])
cbar2.set_label("Classes")
```

8. Display the plot in the console:

```
plt.tight_layout()
```

```
plt.show()
```

After following these steps, the following illustration is generated, showing how the model fits the decision boundaries to divide the data points:

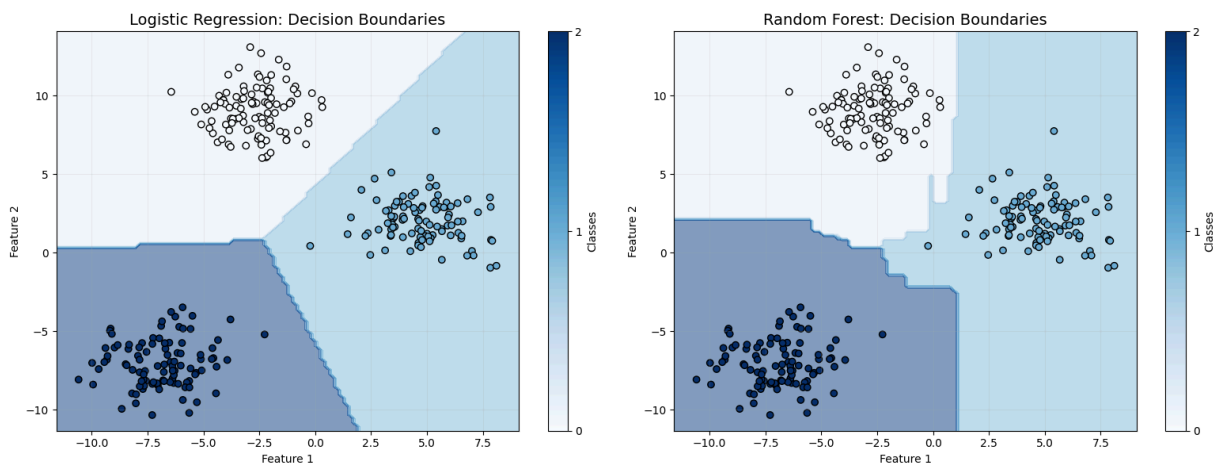


Figure 4.1: *Decision boundary comparison*

The difference in decision boundaries arises from these algorithms' fundamental assumptions and mechanisms. Let us look at them in detail:

- Logistic regression simplifies the decision-making process by fitting a hyperplane for each class in the feature space, which inherently produces linear separations.
- Random forest combines binary splits at each decision node, enabling it to create highly adaptive, nonlinear boundaries. These splits reflect how well the data can be separated at various thresholds.

In the following section, we will look at the importance of log loss in model training.

Importance of log loss in model training

Log loss, or cross-entropy loss, is a widely used metric in machine learning for evaluating and training classification models. It measures the difference between the predicted probabilities and the actual class labels, penalizing confident (A **confident prediction** means the model gives a high probability, close to 1 or 100%, to a particular class) but incorrect predictions more heavily than less confident ones. During model training, algorithms aim to minimize log loss to improve the accuracy and reliability of their predictions.

Log loss in binary classification

In binary classification, the log loss formula evaluates the probability of a single class (e.g., *positive*). Refer to the following equation:

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

The components of the equation are:

- Here, $\hat{y}$ is the predicted probability of the positive class, and $1 - \hat{y}$ is the actual binary label (0 or 1).
- The formula only requires computing probabilities for one class since the probability of the negative class is $1 - \hat{y}$.

Next, we will have a look at the multi-class log loss equation.

Log loss in multi-class classification

For multi-class classification, log loss extends to considering multiple classes' probabilities. The formula becomes the following:

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^k y_{ij} \log(\hat{y}_{ij})$$

Here is a breakdown of the preceding example:

- **k**: Total number of classes.
- **y_{ij}** : Binary indicator (1 if the true class of instance i is j , otherwise 0).
- **$\hat{y}_{ij}$** : Predicted probability that instance i belongs to class j .
- **N**: Is the total number of instances in the dataset.

This formula requires computing and summing probabilities across all k classes for every sample, making it computationally more intensive than the binary case.

In binary classification, log loss focuses on the confidence of predictions for just two outcomes, while in multi-class classification, it evaluates how well the model distributes probabilities across multiple classes. As the number of classes increases, log loss becomes more sensitive to incorrect predictions, especially when the model assigns high probabilities to the wrong class. This makes minimizing log loss more challenging in multi-class problems, requiring the model to balance its predictions across all classes effectively.

Now that we understand how decision boundaries are drawn and the log loss functions have been outlined, we will proceed to our project of predicting customer churn based on three classes (high, medium, and low churn risk).

Predicting customer churn use case

We will now look at our use case, which involves using an imbalanced multi-class sample of customer churn data.

Data importing

The first stage of the process relies on importing the data; as always, we will use the Pandas library to read our **comma-separated value (CSV)** file,

and then we will inspect the data; this will be elaborated in the following steps:

1. Import packages needed for this importing data; as this is a CSV file, we will turn to our trusty friend Pandas:

```
import pandas as pd
```

2. Following, we will create a DataFrame, and we shall name this **cc_df** (short for customer churn; we always follow this parlance when preparing my imports):

```
cc_df = pd.read_csv('data/synth_customer_churn.csv')
```

3. This customer churn dataset is synthetically generated, and the code to generate it is contained in the supporting GitHub repository for this chapter.

Following these steps, you will have imported the customer churn dataset, which will resemble that in the illustration in *Figure 4.2*:

	CustomerID	Age	Gender	Tenure	MonthlyCharges	ServiceUsage	ContractType	PaymentMethod	CustomerSupportCalls	ChurnCategory
0	CUST00001	34.783101	Male	22.460555	86.311178	1.368797	Two-Year	Mailed Check	0.000000	Low Risk
1	CUST00002	45.499645	Male	8.517961	169.618214	19.829722	Month-to-Month	Credit Card	2.000000	Medium Risk
2	CUST00003	31.865032	Male	18.335428	44.411863	5.645398	Month-to-Month	Credit Card	1.000000	Medium Risk
3	CUST00004	57.852490	Male	24.930795	84.472422	21.522832	Two-Year	Electronic Check	5.000000	Low Risk
4	CUST00005	53.849231	Male	7.671134	131.978854	5.631937	Month-to-Month	Credit Card	3.000000	Medium Risk
5	CUST00006	43.477008	nan	14.562302	74.950799	6.408231	Two-Year	Electronic Check	1.000000	Low Risk
6	CUST00007	33.965275	Male	13.690659	75.977816	18.151007	Month-to-Month	Bank Transfer	4.000000	Low Risk
7	CUST00008	34.205514	Male	nan	26.674165	nan	Two-Year	Credit Card	2.000000	Low Risk
8	CUST00009	42.882097	Male	17.795620	14.338559	7.681775	One-Year	Credit Card	0.000000	Low Risk
9	CUST00010	61.678405	Male	24.263394	48.821052	5.711640	Month-to-Month	Credit Card	3.000000	Low Risk
10	CUST00011	26.834346	Female	5.776561	61.782602	3.557274	Month-to-Month	Bank Transfer	3.000000	Low Risk

Figure 4.2: Customer churn DataFrame

Upon inspection, you can see that we have a mixture of categories that will require feature encoding, the two main types that are visible are categorical (Gender, for example, has Male and Female genders) and numerical (either integer or floating-point values). The next step will be to examine the distribution of the features in the dataset.

Feature distribution visualization

This will require a package we used in our previous chapter. When we look at binary classification, we will use **modelviz** (<https://pypi.org/project/modelviz/>) to plot our feature histograms by following these steps:

1. Import the relevant function from the **modelviz** package:

```
from modelviz.histogram import plot_feature_histograms
```

2. Next, we will use the **plot_feature_histograms** function on our imported DataFrame:

```
plot_feature_histograms(cc_df)
```

3. The resultant output of this is a histogram for each of the numeric features in the DataFrame:

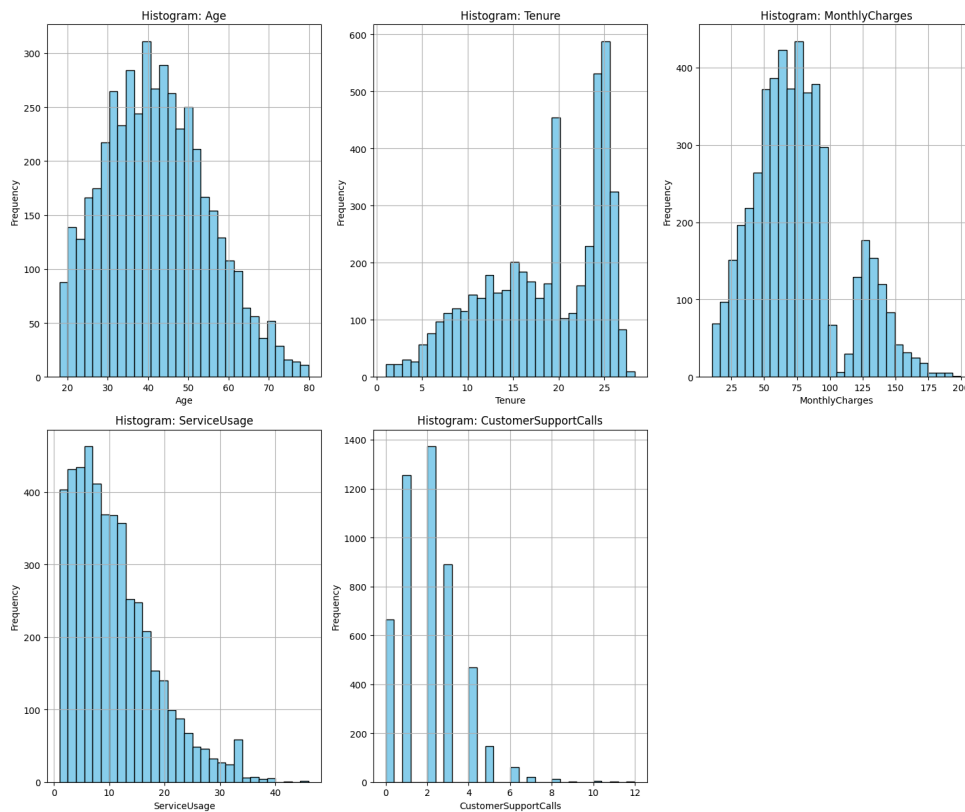


Figure 4.3: Feature distribution

These distributions suggest the following:

- **Age** is unimodal and approximately conforming to a normal distribution. Most individuals are 30 to 50 years old, with peaks at 40.
- **Tenure** shows a bimodal distribution and observes peaks around 0-5 and 20-25 months, possibly indicating two groups:
 1. New customers with short tenures
 2. Long-term customers who have stayed with the company longer
- **Monthly charges** observe a right skew and suggest that the highest volume of customers pay between 50 and 80 for subscriptions.
- **Service usage** also indicates a right skew.
- **Customer support calls** also show a skew, with most customers making 0-4 calls and fewer after 4.

This analysis will aid us in:

- Choosing the best **scaling approach** (standardization for normal, **log/sqrt** for skewed, quantile scaling for multi-modal, etc.).
- Deciding when to transform variables to reduce skewness or highlight important groups.
- Considering binning or creating new features for highly uneven or multi-modal variables.

Missing values analysis

We now move on to understanding the completeness of our data and where and what type of imputation may need to be applied to this dataset.

To do that, we will use a function we previously created in *Chapter 3, Supervised Binary Classification*, to enable this. This function can be found in the support repository for this chapter, and we will import it from our project folder and use it in our analysis:

1. Import the relevant function from our local repository:

```
from multi-class.class_utils.missing_values import missing_values_summarizer
```

2. Use this function with our DataFrame:

```
props_df, _ = missing_values_summarizer(df=cc_df)
```

By following these steps, we will then get a summary of missing values, as a volume and proportion:

feature	missing_count	non_missing_count	total_vol	prop_missing	prop_non_missing	data_type
CustomerID	0	5000	5000	0.000000	1.000000	object
Age	306	4694	5000	0.061200	0.938800	float64
Gender	160	4840	5000	0.032000	0.968000	object
Tenure	112	4888	5000	0.022400	0.977600	float64
MonthlyCharges	74	4926	5000	0.014800	0.985200	float64
ServiceUsage	242	4758	5000	0.048400	0.951600	float64
ContractType	237	4763	5000	0.047400	0.952600	object
PaymentMethod	316	4684	5000	0.063200	0.936800	object
CustomerSupportCalls	100	4900	5000	0.020000	0.980000	float64
ChurnCategory	0	5000	5000	0.000000	1.000000	object

Figure 4.4: Missing values breakdown

Here, we can see that we either have floating-point values (numeric) or object data types; these will need to be handled by one-hot encoding and other methods to convert them to machine-readable values for our ML models to be trained downstream.

Furthermore, as the imputation methods, we will need to look at different methods for each feature, as object data types will require modal replacement schemes.

Let us now move to look at the class imbalance in our various churn categories. We will implement a function for looking at class imbalance and then use this function to display the imbalance in the next section.

Class distribution

To enable this, we will make a visualization, as a function, that you can use in your projects; this will accept multiple class labels and allow you to plot

them side by side as a bar chart.

We will take you through each step of the function in the following steps to explain each line:

```
1. import matplotlib.pyplot as plt
2. import seaborn as sns
3. def visualize_class_imbalance(df, column,
4.                               palette="Set2",
5.                               plt_title='Class imbalance',
6.                               xlabel='Class',
7.                               ylabel='Frequency',
8.                               label_fontsize=12,
9.                               title_fontsize=14,
10.                              tick_fontsize=12,
11.                              text_fontsize=12,
12.                              text_halignment='center',
13.                              figsize=(8,6)):
14.
15.     class_counts = df[column].value_counts()
16.     plt.figure(figsize=figsize)
17.     bar_plot = sns.barplot(
18.         x=class_counts.index,
19.         y=class_counts.values,
20.         hue=class_counts.values,
21.         palette=palette
22.     )
23.     for i, value in enumerate(class_counts.values):
24.         bar_plot.text(i, value + 0.5, str(value),
```

```

25.             ha=text_halignment,
26.             fontsize=text_fontsize)
27.
28. plt.title(plt_title, fontsize=title_fontsize)
29. plt.xlabel(xlabel, fontsize=label_fontsize)
30. plt.ylabel(ylabel, fontsize=label_fontsize)
31. plt.xticks(fontsize=tick_fontsize)
32. plt.yticks(fontsize=tick_fontsize)
33. sns.despine()
34. bar_plot.get_legend().remove() if bar_plot.get_legend() else No
    ne
35. plt.show()

```

The function does the following:

1. Takes the Pandas DataFrame as input (**df**) and a column of interest; this will be the **ChurnCategory** column.
2. Several keyword argument parameters are then passed, such as color palette, title and labels, font sizes, text alignment, and the size of the figure.
3. Then, we count the classes using **value_counts** to get the frequency of labels by class for display.
4. Creates a seaborn (<https://seaborn.pydata.org/>) visual with the **sns.barplot** command.
5. Loops through the class count values with enumerate (counts each enumeration through the **class_count.values**) and adds the labels to the chart.
6. The rest of the steps add labels to the chart, and **sns.despine()** removes the spines from the top and right of the borders for a cleaner appearance.

To use the function, we use the following command:

1. `visualize_class_imbalance(df=cc_df,`
2. `column='ChurnCategory')`

This produces the following chart:

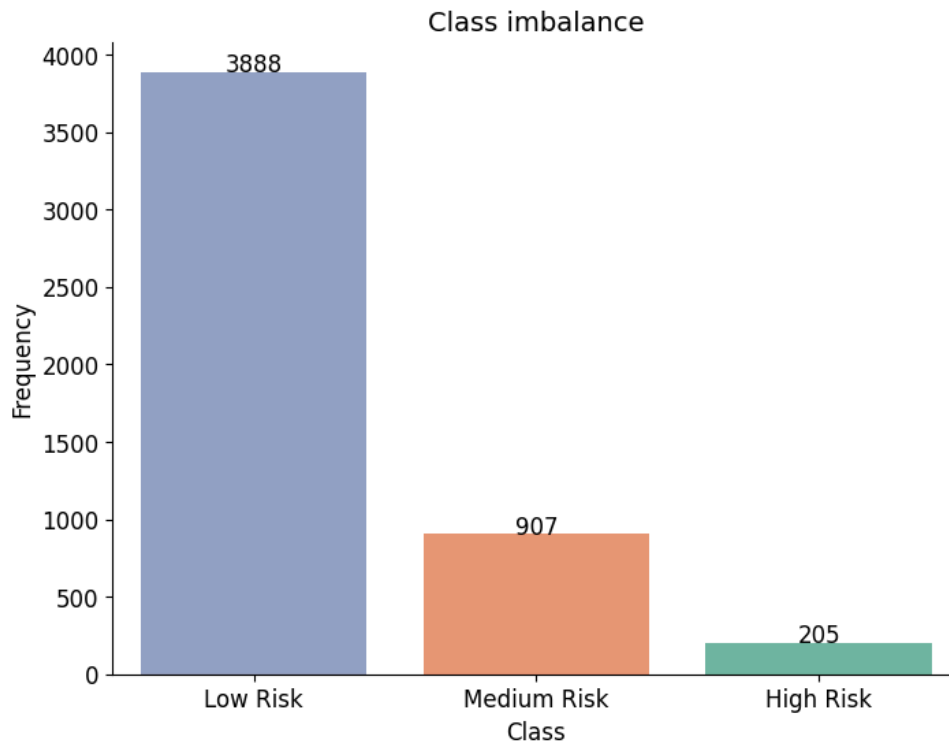


Figure 4.5: Class distribution chart

The chart shows that most cases are low-risk churn and very few are high-risk. This will be challenging, as the class imbalance here may be beyond what you can do with resampling and threshold selection. However, we will endeavor to take you through various approaches we use in practice to address these imbalances.

Partitioning the data

After this point, we will preprocess and feature encode the data. As we have already discussed, preprocessing on the full dataset is not advised. Here, we will partition the data into training and testing sets.

The following steps will get the data into the splits we require:

1. Import the **train_test_split** function from **model_selection**:

```
from sklearn.model_selection import train_test_split
```

2. Drop the **ChurnCategory** and **CustomerID** fields, as **ChurnCategory** is what we are aiming to predict, and **CustomerID** does not have any meaningful information we can use for modeling:

```
X = cc_df.drop(columns=['ChurnCategory', 'CustomerID'])
```

3. Make sure the **CustomerSupportCalls** variable is cast to a floating-point value:

```
X['CustomerSupportCalls'].astype(float)
```

4. Set the **X_cols** variables to keep track of the column names of the features, as we are going to be converting between NumPy arrays and Pandas DataFrames in this chapter:

```
X_cols = X.columns
```

5. Assign the **ChurnCategory** to the Y variable, as this is what we are going to try and estimate:

```
y = cc_df['ChurnCategory']
```

6. The labels are currently text labels representing the various levels of risk, i.e., low, medium, and high. Our model will not be able to accept these as inputs, so we use a label mapping dictionary to map the label to an integer representation:

```
label_mapping = {
```

```
    'Low Risk': 0,
```

```
    'Medium Risk': 1,
```

```
    'High Risk': 2
```

```
}
```

```
y_mapped = y.map(label_mapping)
```

7. Finally, we will pass our **X** (features) and **y** (dependent variable) into the **train_test_split** function, and we will use multiple assignment to assign the output variables of **X_train**, **X_test**, etc.:

```
X_train, X_test, y_train, y_test = train_test_split(X,  
                                                    y_mapped,  
                                                    test_size=0.2,  
                                                    random_state=42,  
                                                    stratify=y_mapped)
```

8. Here we pass in the **test_size** we require, a **random_state** variable, so when you are following along at home, you get the same splits as us, and we will also pass the **stratify** argument (to perform stratified sampling of the target variables).
9. Additionally, as we are going to be switching between Pandas and NumPy in this section, we will create a DataFrame of the **X_train** and **X_test** arrays. Here we will use the **X_cols** variable we stored earlier to keep track of the column names, as NumPy arrays do not have the concept of column names:

```
X_train_df = pd.DataFrame(X_train, columns=X_cols)  
X_test_df = pd.DataFrame(X_test, columns=X_cols)
```

This has now partitioned the data the way we expect. The next section looks at imputation on our dataset, as there are a number of ways we now need to fill in the blanks.

Handling categorical and numerical features for classification

We now move on to understanding the completeness of our data and where and what type of imputation may need to be applied. The following sub-sections will show you how to deal with missing values, normalizing and scaling our features, one-hot encoding variables, and class rebalancing.

Missing value imputation

In the previous analysis, we found missing values in numerical columns and differences in how the data is distributed across features. Similarly, categorical columns, those with non-numeric values like labels or categories, also have missing entries that must be addressed. The plan for these categorical variables is to use modal imputation, meaning the missing values will be filled with the most frequent category in each column. This helps ensure that the data is complete and ready for further analysis. We will go through our imputation strategies in the subheadings under this section.

Modal replacement

Our strategy for modal imputation will be using the **SimpleImputer** class in scikit-learn with a strategy parameter set to *most frequent*. This will enact the modal imputation option, filling missing values by looking at the most frequently occurring value in each column.

For ease, we will create a function that you can then reuse to do modal imputation on your ML projects:

```
1. from sklearn.impute import SimpleImputer
2. import pandas as pd
3. def modal_impute(train_df, test_df, columns):
4.     imputer = SimpleImputer(strategy="most_frequent")
5.     train_df[columns] = imputer.fit_transform(train_df[columns])
6.     test_df[columns] = imputer.transform(test_df[columns])
7.     return train_df, test_df
```

As already stated, this simple function uses the **SimpleImputer** method, and then we use the **fit_transform** to fit the imputation to our training set, and then the transform method to use the fitted imputation from the training data to also transform the testing dataset.

To use the function, we will utilize and carry out the following steps:

1. Extract the **object** columns from the training features, using the **select_dtypes** method:

```
cols_to_impute = list(X_train_df.select_dtypes(include=["object"]).columns)
```

2. Use the function we defined at the start of this section to then get the imputed X train and test DataFrames:

```
modal_X_train, modal_X_test = modal_impute(X_train_df,  
                                           X_test_df,  
                                           cols_to_impute)
```

2. We will once again pass this through the **missing_values_summarizer** to see if the imputation has been effective:

```
props_df, _ = missing_values_summarizer(df=modal_X_train)
```

Outputting this, we can see that the **Gender**, **ContactType**, and **PaymentMethod** variables show a 0 missing count, whereas at the start of the missing values analysis, there were many more. Refer to the following figure:

feature	missing_count	non_missing_count	total_vol	prop_missing	prop_non_missing	data_type
Age	243	3757	4000	0.060750	0.939250	float64
Gender	0	4000	4000	0.000000	1.000000	object
Tenure	91	3909	4000	0.022750	0.977250	float64
MonthlyCharges	56	3944	4000	0.014000	0.986000	float64
ServiceUsage	201	3799	4000	0.050250	0.949750	float64
ContractType	0	4000	4000	0.000000	1.000000	object
PaymentMethod	0	4000	4000	0.000000	1.000000	object
CustomerSupportCalls	78	3922	4000	0.019500	0.980500	float64

Figure 4.6: Modal imputation missing values

The remaining values from the preceding figure are numerical, and we will examine how to handle them in the next subsection.

Imputation and scaling with ColumnTransformer

The **ColumnTransformer** is a powerful addition to your preprocessing arsenal. It allows the user to apply different preprocessing steps to different dataset columns. This is especially useful when working with mixed datasets containing numerical and categorical features, as in our use case. For this example, we have purposefully separated these as separate steps, but in the pipeline section of this book, we will make extensive use of this function.

To define our transformer and use it to preprocess our data, we must follow these steps, which will enable us to define our imports, create our preprocessing pipelines, and spit out the results:

1. Gather the necessary imports, as we will be scaling, as we looked at in *Chapter 3, Supervised Binary Classification*, and we will also be doing various forms of imputation:

```
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import (MinMaxScaler,
                                   StandardScaler,
                                   RobustScaler)
```

2. Now, we will use the **ColumnTransformer** and set it to the variable name `preprocessor`:

```
preprocessor = ColumnTransformer(
    transformers=[
        ('age_pipeline', Pipeline([
            ('imputer', SimpleImputer(strategy='median')),
            ('scaler', MinMaxScaler())
        ]), ['Age']),
        ('tenure_pipeline', Pipeline([
```



```

        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', RobustScaler())
    ]), ['Tenure']),
    ('monthlycharges_pipeline', Pipeline([
        ('imputer', SimpleImputer(strategy='mean')),
        ('scaler', StandardScaler())
    ]), ['MonthlyCharges']),
    ('serviceusage_pipeline', Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', MinMaxScaler())
    ]), ['ServiceUsage']),
    ('customer_service_pipeline', Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler())
    ]), ['CustomerSupportCalls'])
],
remainder='passthrough'
)

```

3. The **transformers** argument specifies different transformations for different columns. Each transformation has a unique name, a pipeline of steps, and the column from the DataFrame it applies to.
4. For the numerical values, either a mean or median imputation strategy is enacted.
5. Based on the distribution of the features we analyzed at the start, a suitable normalizer or scalar is applied to the features.
6. The argument **remainder='passthrough'** states that any columns that are listed will be untouched, and the scale of those features

will stay *as is*.

7. Now that we have created the **preprocessor** variable, we can use it to **fit_transform** based on the training data:

```
X_train_transformed = preprocessor.fit_transform(modal_X_train)
```

8. Time to transform the test data:

```
X_test_transformed = preprocessor.transform(modal_X_test)
```

9. We need to specify the pipeline column order to map them back directly. The downside of the argument **remainder = 'passthrough'** is that it reorders the DataFrame and puts the columns it has processed at the start of the DataFrame, and the remainders get concatenated on the end. We will create a variable and store these as a mutable list:

```
pipeline_cols = ['Age', 'Tenure', 'MonthlyCharges',
                 'ServiceUsage', 'CustomerSupportCalls',
                 'Gender', 'ContractType', 'PaymentMethod']
```

10. The native library that scikit converts everything to is NumPy, so we will need these columns to store the imputed and scaled columns back as a DataFrame; you will see why we are doing this in the next section, but for now, for further encoding, the type must be a Pandas DataFrame:

[illegible][illegible]

Once this process has been completed, your data should look as illustrated in the following figure:

Age	Tenure	MonthlyCharges	ServiceUsage	CustomerSupportCalls	Gender	ContractType	PaymentMethod
0.407662	-0.508479	1.449652	0.367984	-0.702703	Female	One-Year	Bank Transfer
0.592072	-0.655921	3.284177	0.446277	-0.702703	Female	Month-to-Month	Credit Card
0.380605	0.207571	-1.022728	0.023692	-0.017807	Female	Two-Year	Mailed Check
0.298196	0.502495	-0.796529	0.191226	0.667088	Female	One-Year	Credit Card
0.661305	0.088530	0.030502	0.191258	-0.017807	Female	Two-Year	Mailed Check
0.242794	-0.392757	-0.716009	0.467341	-0.702703	Male	One-Year	Bank Transfer
0.380605	0.262192	0.601420	0.510327	-0.017807	Female	Month-to-Month	Electronic Check
0.584174	0.326814	-0.118643	0.053858	0.667088	Male	Two-Year	Electronic Check
0.210886	-0.480832	1.491834	0.270215	-0.017807	Male	One-Year	Bank Transfer
0.147033	-0.167250	-0.829246	0.088425	-0.017807	Male	One-Year	Credit Card

Figure 4.7: Imputed and scaled numerical features

We have nearly fully encoded all of our data so that it can be used in an ML model or multiple models. The remaining step is to consider what to do with the categorical values. The solution is to hot encode them. We have not done this in scikit-learn because there is a way to encode in this fashion using Pandas, and it will be helpful to know both approaches to this method.

One-hot encoding in Pandas

One-hot encoding in Pandas is done using the **get_dummies** command, and it is a handy way to turn categorical variables into numerical ones. This works by creating a new column for each category, with values of 0 or 1 to indicate whether a row belongs to that category. It is beneficial because it lets ML models work with categorical data without assuming any ranking or order between the categories. This makes it a simple and effective tool for preparing data for analysis or modeling.

In the following steps, we will use the **get_dummies** function to convert our categorical variables into their columnar feature:

1. Firstly, we will initialize a list to store the names of the categorical variables (features) we wanted to be used by the one-hot encoding method:

```
cat_cols = ['Gender', 'ContractType', 'PaymentMethod']
```

2. We will now use the **get_dummies** method with the **drop_first** parameter being set to True. This drops the first categorical column, as with one-hot encoding, you should drop at least one category, otherwise, issues with multicollinearity can ensue with specific algorithms:

```
train_dummies = pd.get_dummies(X_train_transformed_df[cat_cols],
```

```
drop_first=True).astype(int)
```

```
test_dummies = pd.get_dummies(X_test_transformed_df[cat_cols],
```

```
drop_first=True).astype(int)
```

3. Printing out the DataFrame from the previous step, we see that the one-hot encoded variables have been created, as the following figure shows:

Gender_Male	ContractType_One-Year	ContractType_Two-Year	PaymentMethod_Credit Card	PaymentMethod_Electronic Check	PaymentMethod_Mailed Check
0	1	0	0	0	0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	1	0	0
0	0	1	0	0	1

Figure 4.8: One-hot encoded features

4. From here, we will use the **concat** command in Pandas to add the encoded features back to the main training DataFrame and make sure that we drop the categorical columns (**cat_cols**) list used to create the encodings.
5. Let us specify **axis=1**, as we want to append new columns, not rows (this would be **axis=0**):

```
X_train_final = pd.concat(
    [
        X_train_transformed_df.drop(columns=cat_cols,
```

```
axis=1), train_dummies
], axis=1)
```

6. We will repeat this process with the test DataFrame additionally:

```
X_test_final = pd.concat(
    [X_test_transformed_df.drop(columns=cat_cols,
                                axis=1),
     test_dummies], axis=1)
```

7. Next, let us check that the length of the columns in both DataFrames match, and if they do not, we will throw a **ValueError** using **assert**, which is a way to stop the code executing any further at this point and break with an error outputted to the console:

```
if len(X_test_final.columns) != len(X_train_final.columns):
    assert ValueError("The number of columns of the train and
test data frame should match")
else:
    print(f'The number of columns in the training set is {len(X_
train_final.columns)} and in the testing set is: {len(X_test_fina
l.columns)}')
```

8. Finally, as the **get_dummies** method needs the data structures as DataFrames, and scikit-learn does not play well with DataFrames, we need to convert the final preprocessed data back into NumPy arrays:

```
import numpy as np
X_train = np.array(X_train_final)
X_test = np.array(X_test_final)
```

Now that our data is in the required shape, we can shift our focus to rebalancing the dataset. Our earlier class distribution analysis identified a significant imbalance in medium and high-risk churn customers.

While we will explore synthetic rebalancing methods to address this issue, the data should also be revisited. By sampling more balanced subsets directly from your customer databases, you can enhance the effectiveness of predicting multiple classification labels.

Class rebalancing

In this section, we will implement a two-step strategy to rebalance the dataset by combining over-sampling and under-sampling techniques in the following steps:

1. We will need imbalanced-learn installed for this, so we will use the following:

```
pip install imbalanced-learn
```

2. Once this has been installed, we will import this, with the **Counter** method from the collections library:

```
from imblearn.combine import SMOTEENN
from collections import Counter
```

3. As our medium and high-risk churn examples are smaller than those of low-risk churners, we will proportionally inflate these estimates using variables entitled **increase_class_1** and **increase_class_2**:

```
increase_class_1 = 1.4
increase_class_2 = 2.5
```

4. Now that we have the proportions to inflate the classes by, we will then apply them using the **Counter** function to count the occurrences of each class and then multiply by our uplift factors, based on the relevant class:

```
original_counts = Counter(y_train)
target_counts = {
    1: int(original_counts[1] * increase_class_1),
```

```
2: int(original_counts[2] * increase_class_2)
}
```

5. Our **target_counts** dictionary now allows us to pass this as our sampling strategy to the SMOTEENN method, which we learned about in *Chapter 3, Supervised Binary Classification*:

```
smote_enn = SMOTEENN(sampling_strategy=target_counts, r
andom_state=42)
```

6. This has initialized the method to use for synthetic generation:

```
X_train_resampled, y_train_resampled = smote_enn.fit_resamp
le(X_train, y_train)
```

7. We have now oversampled the minority classes. The second step of this approach is to then downsample the majority class to achieve a greater balance. Now we will set a proportion to reduce the majority class:

```
from imblearn.under_sampling import RandomUnderSampler
reduction_proportion = 0.5
```

8. Then, we use the same method as we did in *Step 4*:

```
original_counts = Counter(y_train)
target_majority_size = int(original_counts[0] * reduction_prop
ortion)
```

9. Now, we will use the **RandomUnderSampler** method to under-sample our majority class (i.e., low risk):

```
undersampler = RandomUnderSampler(sampling_strategy={0:
target_majority_size},
random_state=42)
```

10. Finally, we will **fit_resample** the **undersampler** to the training set features (X) and the dependent class (risk of churn):

```
X_train_resampled,
```

```
y_train_resampled = undersampler.fit_resample(X_train_resampled,  
                                              y_train_resampled)
```

That completes the full process of getting the data ready for modelling and evaluation downstream. This is what we will cover in the next section.

Evaluation metrics for multi-class classification

In this section, we will define a **compute_metrics** function, similar to those created in earlier sections, to specify how we plan to evaluate our multi-class model. Our evaluation will include micro and macro precision, recall, and F1 scores to compare the performance of different models and strategies, which we will detail later. Additionally, we will analyze precision and recall on a per-class basis.

We will begin by implementing the function, followed by subsections that explain each metric. However, we will not revisit micro precision, recall, and F1 score in detail, as these were already covered in *Chapter 2, Introduction to Scikit-learn for Machine Learning*.

The following steps will implement our function, and then we will explain, in subsections, what each of the metrics means:

1. Import the internal metrics we will need from scikit-learn:

```
from sklearn.metrics import (  
    accuracy_score, precision_score, recall_score, f1_score,  
    classification_report, confusion_matrix  
)
```

2. All these imports should look familiar by now; the only import we have not used hitherto is the **classification_report** (including key metrics such as precision, recall, F1 score, and support (the number of true instances for each class)). These metrics are calculated for each class individually, offering a granular view of how well the model performs on different categories.

Additionally, it provides aggregated scores such as micro, macro, and weighted averages for a comprehensive evaluation across all classes. This report is particularly useful for diagnosing model performance, identifying issues like class imbalance, and understanding trade-offs between precision and recall in multi-class or binary classification tasks.

3. To define the function, we implement the following code:

```
def compute_metrics(y_true, y_pred):
    class_report = classification_report(y_true, y_pred,
                                         output_dict=True)
    per_cls_metrics = {
        f"Precision per (Class {cls})": class_report[cls]["precision"]
        for cls in class_report if cls not in ["accuracy",
                                                "macro avg",
                                                "weighted avg"]
    }
    per_cls_metrics.update({
        f"Recall per (Class {cls})": class_report[cls]["recall"]
        for cls in class_report if cls not in ["accuracy",
                                                "macro avg",
                                                "weighted avg"]
    })
    metrics = {
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision (Micro)": precision_score(y_true, y_pred,
                                             average='micro'),
        "Precision (Macro)": precision_score(y_true, y_pred,
```

```

        average='macro'),
    "Recall (Micro)": recall_score(y_true, y_pred,
        average='micro'),
    "Recall (Macro)": recall_score(y_true, y_pred,
        average='macro'),
    "F1 Score (Micro)": f1_score(y_true, y_pred,
        average='micro'),
    "F1 Score (Macro)": f1_score(y_true, y_pred,
        average='macro'),
    "Classification Report": class_report,
    "Confusion Matrix": confusion_matrix(y_true,
        y_pred),
    **per_cls_metrics
}
return metrics

```

4. The per class metrics essentially create a dictionary with precision and recall per class label and then store these as a key in the dictionary, i.e., **Precision per (Class 0): 0.98**. These are then passed at the end of the metric dictionary as an unpacked dictionary (****per_cls_metrics** is a way to say unpack everything into that dictionary as key-value pairs).

The obvious next question is how macro precision, recall, F1, and precision and recall per class are calculated. We will explain this in the following sub-sections.

Macro Precision

Macro Precision is a performance metric used to evaluate the precision of a classification model by calculating the precision for each class individually and then averaging these values equally, regardless of class size. Unlike

micro precision, which aggregates all true positives and false positives across all classes, macro precision treats each class independently, ensuring that smaller or less represented classes have the same influence on the final score as larger classes. This makes macro precision particularly useful in scenarios with imbalanced datasets or when all classes are equally important. It highlights the model's ability to accurately classify each class without being biased toward the more frequent ones.

The formula for Macro Precision is as follows, with n being the total number of classes and i being the iterator over each class example:

$$\text{Macro Precision} = \frac{1}{n} \sum_{i=1}^n \text{Precision for Class } i$$

Macro Recall

Macro Recall is a performance metric used to evaluate the recall of a classification model by calculating the recall for each class individually and then averaging these values equally, regardless of class size. Similar to before, this is ideal for imbalanced datasets, such as those we are dealing with in this use case.

The formula is identical to the one of Macro Precision and is also given here:

$$\text{Macro Recall} = \frac{1}{n} \sum_{i=1}^n \text{Recall for Class } i$$

Macro F1 Score

Macro F1 Score is a performance metric used in multi-class classification to balance both precision and recall for each class, and then average these scores equally across all classes. The formula for this would look like the following:

$$\text{Macro F1} = \frac{1}{n} \sum_{i=1}^n F1_i$$

Let us break down the preceding example:

- n is the total number of classes.
- $F1_i$ is the score for class i given by:

$$F1_i = 2 \cdot \frac{\text{Precision}_i \cdot \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i},$$

Precision per class

This formula is useful for analyzing how well a model distinguishes class i from all the other classes. By examining precision per class, you can identify if the model is over predicting certain classes, leading to more false positives. Furthermore, it helps to show if the model is accurately pinpointing the true instances for each class. Sometimes, this level of granularity can be valuable in scenarios where certain classes are more critical or when dealing with imbalanced samples.

The formula for this is given here:

$$\text{Precision for Class } i = \frac{\text{True Positives for Class } i}{\text{True Positives for Class } i + \text{False Positives for Class } i}$$

Recall per class

This formula is very similar to the previous formula and is given as follows:

$$\text{Recall for Class } i = \frac{\text{True Positives for Class } i}{\text{True Positives for Class } i + \text{False Negatives for Class } i}$$

It takes the *True Positives for Class i* and divides them by the sum of the TP and FN of Class i .

Support

Support tells you how many samples belong to each class in the ground truth labels. This is important for understanding class balance in your dataset. If one class has significantly more support than another, you have a class imbalance situation, which can affect how the model trains and how

you interpret performance metrics like precision, recall, and F1 score. The support formula is detailed here:

$$\text{Support for Class } i = \text{True Positives for Class } i + \text{False Negatives for Class } i$$

That concludes everything you need to know about the evaluation metrics in our custom function. As always, scikit-learn makes this process straightforward by allowing you to specify micro or macro precision as a parameter, but it is still essential to understand what each metric conveys to evaluate your model outputs effectively.

Classifier instantiate

We will use several multi-class classifiers in our experiments. This list is not exhaustive, but by including logistic regression, SVMs, random forests (which we will explore in more detail in the ensemble methods section), and CatBoost, we have enough variety to effectively address our classification task.

In each model, we will leverage the **class_weight** parameter, which helps mitigate class imbalance by adjusting weights inversely proportional to class frequencies.

Here, we will build a dictionary, as we have done in previous chapters, to list out our models:

1. `from sklearn.linear_model import LogisticRegression`
2. `from sklearn.svm import SVC`
3. `from sklearn.ensemble import RandomForestClassifier`
4. `from catboost import CatBoostClassifier`
5. `classifiers = {`
6. `"LogisticRegression": LogisticRegression(max_iter=1000,`
7. `class_weight='balanced'),`
8. `"SVM Balanced": SVC(probability=True,`
9. `class_weight='balanced'),`

```
10.     "RandomForest": RandomForestClassifier(n_estimators=100,
11.                                           class_weight='balanced',
12.                                           random_state=42),
13.     "Balanced CatBoost": CatBoostClassifier(auto_class_weights
14.                                           ='Balanced',
15.                                           random_state=42)
16. }
```

We have done all the preprocessing, picked our evaluation strategies and created a function to support them, defined our modelling strategy, and are now using multiple multi-class strategies to aid with multi-class classification problems. These will be explained in the next section.

One-vs-Rest and One-vs-One strategies

The following subsections explore various methods for turning binary classifiers into multi-class models. We will examine their theoretical foundations, discuss when each technique is most appropriate, and show practical implementation examples.

One-vs-Rest

One-vs-Rest (also called **One-vs-All**) is a simple yet effective way to adapt binary classification algorithms for multi-class tasks. In predicting high, medium, or low churn risk, this approach involves training three separate binary classifiers. The first classifier distinguishes *High Risk* from the combined *Medium + Low Risk*, the second identifies *Medium Risk versus High + Low Risk*, and the third focuses on *Low Risk versus High + Medium Risk*. When a new customer's data is evaluated, each classifier generates a probability or score indicating how likely that customer is to fall into its specific risk category, and the category with the highest score becomes the final prediction.

This method is beneficial when each churn risk category is relatively distinct or when you want a straightforward setup that is easy to interpret

and tune. For instance, if your historical data shows that *High Risk* has significantly fewer examples than *Medium Risk*, you can adjust the class weight or other hyperparameters specifically in the *High Risk vs. Others* classifier. However, because each classifier works independently, you might see occasional overlap where more than one classifier strongly predicts its label. Despite this, One-vs-Rest remains a popular choice for multi-class problems like churn risk prediction, as it provides a clear, modular way to manage separate models for different categories.

One-vs-One

One-vs-one tackles multi-class problems by building a distinct binary classifier for every pair of classes. In predicting high, medium, or low churn risk, you would train three classifiers: one to separate *High* from *Medium*, another to distinguish *High* from *Low*, and a third to classify *Medium* vs. *Low*. When a new customer's data is evaluated, each pairwise classifier votes for which of its two categories is more likely, and the category with the most votes overall becomes the final prediction.

This approach can excel when each risk category is well-defined, as focusing on just two classes at a time often allows for cleaner decision boundaries. However, it does require training more models (one per class pair) and can become more complex to manage if you have many classes.

OutputCodeClassifier

The OutputCodeClassifier applies an ECOC method, where each churn risk category, high, medium, or low, is assigned a unique binary code. Rather than training a separate model for each category or pair of categories, this approach trains multiple binary classifiers, each responsible for predicting one *bit* of the code. When new customer data is presented, each binary classifier provides an output, and the final prediction is the code that best matches the combined results across all bits.

Although ECOC introduces additional complexity compared to more uncomplicated strategies like One-vs-Rest or One-vs-One, it can improve

robustness by reducing correlations among the classifiers, which can be especially useful when *High Risk* or *Medium Risk* overlap in features with *Low Risk*.

Custom threshold class

All the other strategies we have just mentioned have direct implementation in scikit-learn. Still, another approach is to set different thresholds of risk across each of the classes, so if you have three classes, then you can define three classes. In the following code we implement a **ThresholdClassifier** class:

```
1. class ThresholdClassifier:
2.     def __init__(self, base_classifier, thresholds):
3.         self.base_classifier = base_classifier
4.         self.thresholds = thresholds
5.
6.     def fit(self, X, y):
7.         self.classes_ = np.unique(y)
8.         self.base_classifier.fit(X, y)
9.
10.    def predict(self, X):
11.        probabilities = self.base_classifier.predict_proba(X)
12.        predictions = []
13.        for prob in probabilities:
14.            for i, threshold in enumerate(self.thresholds):
15.                if prob[i] >= threshold:
16.                    predictions.append(self.classes_[i])
17.                    break
18.        else:
```



```
19.         predictions.append(self.classes_[np.argmax(prob)])
20.     return np.array(predictions)
```

To explain what this function does, we will have a look at the following bullets:

- **Uses a base classifier:** Uses any existing model's probability outputs as input.
- **Per class thresholding:** allows you to pass different probability thresholds for each class, i.e., low risk ≥ 0.7 = risk, < 0.7 = no risk.
- **Customized predictions:** Assigns a class when the threshold is met, otherwise falls back to the highest probability.
- **Control:** Enabling the handling of classes with varying importance or costs, i.e., high-risk churners can be handled differently from low-risk.
- Allows you to use the fit and predict interface of scikit-learn.

Extended strategies applied

This gives us four strategies we can implement, alongside various multi-class algorithms, to evaluate the performance of our model on our training data. In the following steps, we will set up the imports needed for the evaluation strategies mentioned in this section, and then store each of the strategies in a list of tuples:

1. Import the relevant strategies from **sklearn.multi-class**:

```
from sklearn.multi-class import (OneVsRestClassifier, OneVs
                                OneClassifier
                                ,OutputCodeClassifier)
```

2. Store the results and confusion matrices in three empty lists:

```
results = []
```

```
cms = []
```

3. Use the inbuilt classifiers with default options for One-vs-Rest and One-vs-One, and use lambda (anonymous functions) to pass parameters to the classifier. For our **CustomThreshold** class, this allows us to pass a list of thresholds through to the class:

```
strategies = [  
    ("OvR", OneVsRestClassifier),  
    ("OvO", OneVsOneClassifier),  
    ("ECOC", lambda clf: OutputCodeClassifier(clf,  
                                                code_size=2,  
                                                random_state=42)),  
    ("CustomThreshold", lambda clf: ThresholdClassifier(clf,  
                                                         thresholds=[0.6, 0.5, 0.2]))  
]
```

This sets the scene for the creation of our training and evaluation loop. When we get to the pipeline section of this book, we will show you how most ML projects are structured in separate files, but for now, we will be utilizing one Jupyter Notebook for the sake of demonstration.

Modeling, evaluation, and model trade-offs

In this section, we will dive into constructing the model training and evaluation loop, ensuring a robust and efficient approach to machine learning development. Following this, we will explore key evaluation metrics, reinforcing our previous discussions to assess model performance effectively. Finally, we will conduct a comparative analysis of model trade-offs, examining the strengths and weaknesses of logistic regression, SVM, random forest classifiers, and CatBoost classifiers, enabling a data-driven selection of the most suitable model for a given task.

Model training and evaluation loop

In this section, we will create a nested loop to go through each classifier and fit it to the training data, grab the evaluation metrics, test on the testing data, and then loop through the strategies we defined in the previous section to get the full model results.

It is probably best to include the loop here for reference and then explain what it is doing in the following steps of the function:

```
1. def train_and_eval_class(classifiers, strategies,
2.                           X_train, y_train,
3.                           X_test,
4.                           y_test):
5.     results = []
6.     cms = []
7.     for classifier_name, clf in classifiers.items():
8.         for strategy_name, strategy in strategies:
9.             print(f'Applying strategy: {strategy_name} to Classifier:
               {classifier_name}')
10.            model = strategy(clf)
11.            model.fit(X_train, y_train)
12.            preds = model.predict(X_test)
13.            # Compute metrics
14.            metrics = compute_metrics(y_test, preds)
15.            result = {
16.                'Classifier': classifier_name,
17.                'Strategy': strategy_name,
18.                "Accuracy": metrics['Accuracy'],
19.                'Precision (micro)': metrics['Precision (Micro)'],
20.                "Precision (macro)": metrics['Precision (Macro)'],
21.                'Recall (micro)': metrics['Recall (Micro)'],
```

```

22.         "Recall (macro)": metrics['Recall (Macro)'],
23.         "F1 Score (micro)": metrics['F1 Score (Micro)'],
24.         "F1 Score (macro)": metrics['F1 Score (Macro)']
25.     }
26.     # Add precision per class
27.     for key in metrics:
28.         if key.startswith("Precision per (Class)"):
29.             result[key] = metrics[key]
30.     # Add recall per class
31.     for key in metrics:
32.         if key.startswith("Recall per (Class)"):
33.             result[key] = metrics[key]
34.     results.append(result)
35.     cms.append({
36.         "Classifier": f'{classifier_name} - {strategy_name}',
37.         "ConfusionMatrix": metrics['Confusion Matrix']
38.     })
39.     print(f"Classification Report for {classifier_name} ({strategy_name}):\\n")
40.     print(pd.DataFrame(metrics["Classification Report"]).transpose())
41.     print("\\n")
42.     results_df = pd.DataFrame(results)
43.     return results_df, cms

```

This loop does the following:

- **Loop through classifiers:** For each classifier in the classifiers dictionary, we pick its name and instance (e.g., Logistic

Regression).

- **Loop through strategies:** For each specified strategy (like **OneVsRest** or **OneVsOne**, etc.), we create a new model by applying that strategy to the current classifier.
- **Train the model:** We fit this newly created model on the training data (**X_train**, **y_train**).
- **Predict on test data:** Once trained, we generate predictions (**preds**) on **X_test**.
- **Compute metrics:** We call **compute_metrics** to evaluate these predictions, calculating accuracy, precision (micro/macro), recall (micro/macro), F1 scores, plus precision/recall per class.
- **Record results:** We build a dictionary of results (including confusion matrices) for each classifier-strategy pair and store it in **results** and **cms**.
- **Display output:** We print a classification report for quick insight into each classifier-strategy performance.
- **Return summary:** Finally, we compile all metrics into a DataFrame (**results_df**) and return it alongside the confusion matrices, providing an organized overview of all the experiments.

To set the training running, we use our function over the training and testing set, as follows:

```
1. results_df, cms = train_and_eval_class(classifiers=classifiers,
2.                                     strategies=strategies,
3.                                     X_train=X_train_resampled,
4.                                     y_train=y_train_resampled,
5.                                     y_test=y_test,
6.                                     X_test=X_test)
```

This will then iterate through every algorithm and train each algorithm for each of the four strategies and move on to the next model, all the while recording the results in the empty lists we defined earlier. When the training is finished, what you should see is a large DataFrame with all the relevant model results, similar to the following figure:

Classifier	Strategy	Accuracy	Precision (micro)	Precision (macro)	Recall (micro)	Recall (macro)	F1 Score (micro)	F1 Score (macro)	Precision per (Class 0)	Precision per (Class 1)	Precision per (Class 2)	Recall per (Class 0)	Recall per (Class 1)	Recall per (Class 2)
LogisticRegression	OvR	0.874000	0.874000	0.665938	0.874000	0.679637	0.874000	0.665367	0.942667	0.917647	0.137500	0.908740	0.861878	0.268293
LogisticRegression	OvO	0.752000	0.752000	0.637413	0.752000	0.628002	0.752000	0.602551	0.939394	0.915152	0.057692	0.757069	0.834254	0.292683
LogisticRegression	ECOC	0.783000	0.783000	0.609710	0.783000	0.638458	0.783000	0.601119	0.942424	0.809783	0.076923	0.799486	0.823204	0.292683
LogisticRegression	CustomThreshold	0.672000	0.672000	0.637338	0.672000	0.612592	0.672000	0.575372	0.942593	0.919255	0.050167	0.654242	0.817680	0.365854
SVM Balanced	OvR	0.916000	0.916000	0.710748	0.916000	0.680816	0.916000	0.694424	0.942356	0.917160	0.272727	0.966581	0.856354	0.219512
SVM Balanced	OvO	0.823000	0.823000	0.639225	0.823000	0.630442	0.823000	0.622257	0.939007	0.916168	0.062500	0.850900	0.845304	0.195122
SVM Balanced	ECOC	0.848000	0.848000	0.643773	0.848000	0.641153	0.848000	0.633162	0.943681	0.910714	0.076923	0.883033	0.845304	0.195122
SVM Balanced	CustomThreshold	0.911000	0.911000	0.680972	0.911000	0.654155	0.911000	0.666248	0.938750	0.916667	0.187500	0.965296	0.850829	0.146341
RandomForest	OvR	0.932000	0.932000	0.895553	0.932000	0.658277	0.932000	0.688067	0.935680	0.917647	0.833333	0.991003	0.861878	0.121951
RandomForest	OvO	0.932000	0.932000	0.895553	0.932000	0.658277	0.932000	0.688067	0.935680	0.917647	0.833333	0.991003	0.861878	0.121951
RandomForest	ECOC	0.930000	0.930000	0.838037	0.930000	0.648306	0.930000	0.671983	0.935680	0.911765	0.666667	0.991003	0.856354	0.097561
RandomForest	CustomThreshold	0.931000	0.931000	0.855845	0.931000	0.657849	0.931000	0.686374	0.935601	0.917647	0.714286	0.989717	0.861878	0.121951
Balanced CatBoost	OvR	0.931000	0.931000	0.840325	0.931000	0.665551	0.931000	0.696913	0.936663	0.917647	0.666667	0.988432	0.861878	0.146341
Balanced CatBoost	OvO	0.929000	0.929000	0.786528	0.929000	0.664694	0.929000	0.692796	0.936508	0.923077	0.500000	0.985861	0.861878	0.146341
Balanced CatBoost	ECOC	0.926000	0.926000	0.760556	0.926000	0.660582	0.926000	0.686868	0.936430	0.916667	0.428571	0.984576	0.850829	0.146341
Balanced CatBoost	CustomThreshold	0.927000	0.927000	0.773775	0.927000	0.670125	0.927000	0.698521	0.937500	0.917160	0.466667	0.983290	0.856354	0.170732

Figure 4.9: Model results by classifier and strategy

Random forest (OvO) demonstrates the best overall performance, making it the choice. In the next section, we will visualize its results. However, it is worth noting that Class 2 (high-risk churners) consistently show low performance across all models.

This highlights a need to revisit the data sampling process and create a more balanced dataset from the customer databases to improve results for this critical class. Alternatively, if you use the model, you could interpret its strengths in predicting low and medium churn risk as a signal to flag any customers not labeled as such for manual review until the class distribution is improved.

Model trade-offs with chosen models

When addressing class imbalance in machine learning, several models offer parameters to adjust class weights, enhancing performance on minority classes. Here is a concise overview of the specified models and their associated trade-offs:

- **Logistic regression:** By setting `class_weight='balanced'`, the model adjusts weights inversely proportional to class frequencies, helping to mitigate bias toward the majority class. However, logistic regression assumes a linear relationship between features and the log odds of the outcome, which may not capture complex patterns in the data.
- **Support vector machine:** Utilizing `class_weight='balanced'` ensures that the SVM accounts for class imbalance by assigning higher weights to minority class samples. While effective, SVMs can be computationally intensive, especially with large datasets, and may require careful tuning of parameters like the kernel and regularization to achieve optimal performance.
- **Random forest classifier:** Setting `class_weight='balanced'` adjusts for class imbalance during training. Random forests are robust to overfitting and can handle a mix of feature types. However, they can be less interpretable than single decision trees and may require more computational resources due to the ensemble nature of the method.
- **CatBoost classifier:** With `auto_class_weights='Balanced'`, CatBoost automatically adjusts weights to counteract class imbalance. It efficiently handles categorical features and often provides high performance with minimal parameter tuning. The trade-off includes increased computational complexity compared to simpler models and the necessity of understanding its specific parameters and data preprocessing requirements.

Visualizing evaluation results

The final sojourn around practical multi-class ML is to analyze our results in a confusion matrix. We will follow these steps to achieve this:

1. Import the `plot_confusion_matrix` function from the `modelviz` package.

2. Create a label mapping list to label the visualization:

```
classes = ['Low risk', 'Medium risk', 'High risk']
```

3. The previous step had us store many confusion matrices in a list. We know the model we need to retrieve from the previous step, so we will create a variable, namely **best_model**:

```
best_model = 'RandomForest - OvO'
```

4. The last step, will be to loop through our confusion matrix list (**cms**) until we find the matching name of the classifier, and we will use `enumerate` to store the index of the model:

```
for idx, cm in enumerate(cms):
    if cm['Classifier'] == best_model:
        print(f"Model: {cm['Classifier']} found at index: {idx}")
        plot_confusion_matrix(cm['ConfusionMatrix'],
                               model_name=cm['Classifier'],
                               classes=classes)
```

This will print: **Model: RandomForest - OvO found at index: 9** and return the following visual:

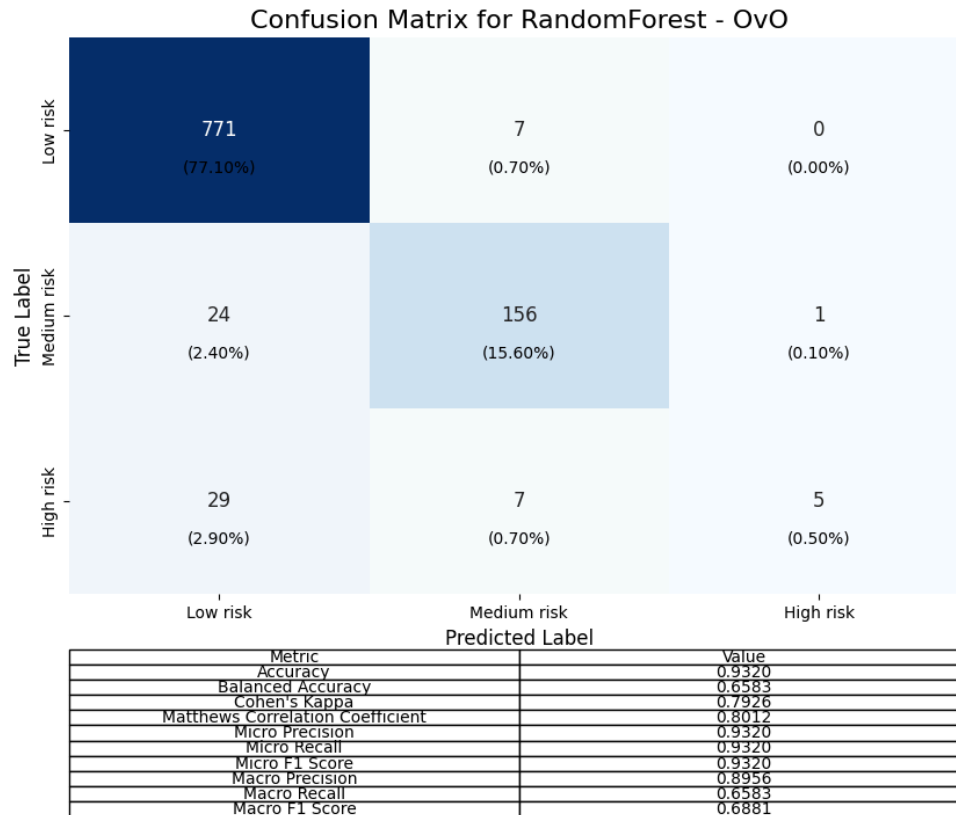


Figure 4.10: Confusion matrix of best performant model/strategy

Conclusion

This chapter explored the complexities of multi-class classification, where models predict one of three or more distinct classes. Compared to binary classification, multi-class problems introduce challenges such as class imbalance, overlapping features, and the need for nuanced evaluation metrics like macro and micro averages of precision, recall, and F1 scores.

Using our customer churn example, we tackled these issues by preprocessing the data with imputation, scaling, and one-hot encoding. We also addressed imbalances in churn risk levels (low, medium, and high) through oversampling and undersampling techniques to improve predictions for underrepresented classes.

We evaluated classifiers like logistic regression, random forest, SVM, and CatBoost with multi-class strategies such as One-vs-Rest, One-vs-One, ECOC, and custom threshold adjustments. random forest, with the One-vs-

One strategy, achieved the best overall performance, though high-risk churners were difficult to classify accurately. This highlighted the need to revisit sampling methods or consider manual interventions for critical cases. Through tools like confusion matrices and decision boundary visualizations, we underscored the importance of understanding how different models and strategies perform, ensuring robust and informed application of machine learning to multi-class problems.

The next chapter will examine unsupervised learning, focusing on clustering techniques. We will explore prominent algorithms such as K-means, hierarchical clustering, and DBSCAN, discussing their methodologies and applications. To assess the effectiveness of these clustering methods, we will examine evaluation metrics like the Silhouette score and the elbow method. A practical example will illustrate segmenting retail customers based on purchase history, highlighting how customer segmentation can inform and enhance business strategies.

Points to remember

Here are some key points summed up from this chapter:

- **Class imbalance:** Addressing imbalance through oversampling (e.g., SMOTE) and undersampling improves the model's ability to predict minority classes.
- **Class weights:** Adjusting weights inversely proportional to class frequencies helps mitigate imbalance during model training.
- **Confusion matrices:** Provide detailed insights into model performance, mainly where class misclassifications occur.
- **Custom threshold adjustment:** Allows fine-tuning class predictions by setting different thresholds, making it suitable for imbalanced datasets.
- **Decision boundaries:** Different models create varying decision boundaries. Linear models like logistic regression produce more

straightforward boundaries, while tree-based and SVM models create complex, nonlinear ones.

- **Evaluation metrics:** Macro metrics treat each class equally, while micro metrics focus on overall performance by aggregating across classes.
- **Feature scaling:** Depending on the feature distributions, choosing the right scaler (e.g., MinMax, StandardScaler) is crucial.
- **Multi-class classification:** It involves predicting one of three or more distinct classes, which adds complexity compared to binary classification.
- **One-vs-One strategy:** Builds binary classifiers for every pair of classes, often producing cleaner boundaries but requiring more computation.
- **One-vs-Rest strategy:** A simple approach where each class is treated as a binary classification problem against all other classes.
- **Output code classifier:** ECOC improves robustness by encoding classes as binary patterns, reducing correlations between classifiers.
- **Preprocessing:** Proper data preparation, including handling missing values, scaling numerical features, and one-hot encoding categorical features, is critical for model performance.
- **Visualization:** Tools like decision boundary plots and feature histograms help diagnose model behavior and identify areas for improvement.

Multiple choice questions

1. **Compared to binary classification, what is the main challenge in multi-class classification?**
 - a. Handling missing values

- b. Predicting one of several distinct classes
 - c. Ensuring features have a mean of 0 and a standard deviation of 1
 - d. Reducing the dimensionality of the dataset
2. **What does the One-vs-Rest strategy involve?**
- a. Building binary classifiers for every pair of classes
 - b. Assigning binary codes to each class
 - c. Treating each class as a separate binary classification problem against all others
 - d. Adjusting decision thresholds for different classes
3. **Which strategy creates binary classifiers for every pair of classes?**
- a. One-vs-Rest
 - b. One-vs-One
 - c. Output code classifier
 - d. Custom threshold adjustment
4. **What is the purpose of the ECOC?**
- a. To reduce the need for preprocessing
 - b. To simplify decision boundaries into linear separations
 - c. To balance class weights during training
 - d. To improve robustness by encoding classes as binary patterns
5. **Which model created the best performance in the case study?**
- a. Random forest with One-vs-One
 - b. Logistic regression with One-vs-One
 - c. SVM with One-vs-Rest
 - d. Balanced CatBoost with output code classifier

6. **Why is setting custom thresholds for classes useful?**
 - a. It reduces computational complexity
 - b. It adjusts class predictions based on varying importance or costs
 - c. It simplifies model interpretability
 - d. It creates binary classifiers for every pair of classes
7. **Which method balances class distributions by generating synthetic samples for minority classes?**
 - a. StandardScaler
 - b. RandomUnderSampler
 - c. SMOTE and variants
 - d. MinMaxScaler
8. **What type of decision boundaries are created by logistic regression?**
 - a. Curved and adaptive boundaries
 - b. Linear boundaries such as lines or planes
 - c. Nonlinear and complex
 - d. Binary codes for each class
9. **What does the class_weight parameter do in scikit-learn classifiers?**
 - a. Adjusts the importance of each class during training
 - b. Adjusts the importance of each class during validation
 - c. Adjusts the importance of each feature in the training set
 - d. Reduces dimensionality of the dataset
10. **What does SMOTEENN combine to handle imbalanced datasets?**

- a. Macro and micro metrics
 - b. Synthetically alters the dependent variable
 - c. Oversampling with SMOTE and undersampling with Edited Nearest Neighbors
 - d. Creates a copy of the dataset
11. **What is the main advantage of using One-vs-One classification?**
- a. It reduces the number of models needed
 - b. It simplifies feature engineering
 - c. It focuses on cleaner decision boundaries by comparing pairs of classes
 - d. It eliminates the need for class balancing
12. **Which visualization tool helps analyze decision boundaries in multi-class classification?**
- a. Confusion matrix
 - b. DecisionBoundaryDisplay
 - c. Feature importance plots
 - d. ROC curve

Answers

1	b
2	c
3	b
4	d
5	a
6	b
7	c
8	b

9	a
10	c
11	c
12	b

Key terms

Some key terms used in this chapter are:

- **Balanced CatBoost:** A gradient boosting algorithm that handles categorical features and class imbalance effectively.
- **Class imbalance:** A dataset issue where certain classes are underrepresented compared to others.
- **Class weights:** Weights assigned to classes to counteract imbalance during training.
- **Classification report:** A summary of precision, recall, F1 score, and support for each class.
- **Confusion matrix:** A table showing true positives, false positives, true negatives, and false negatives.
- **Cross-entropy loss:** Also known as log loss, it evaluates the difference between predicted probabilities and actual class labels, penalizing confident but incorrect predictions.
- **Custom threshold adjustment:** Fine-tuning class predictions by setting different thresholds for each class.
- **Decision boundaries:** Regions in the feature space that define model classification decisions.
- **ECOC:** A multi-class strategy using binary patterns to improve robustness.
- **Evaluation metrics:** Measures such as accuracy, precision, recall, and F1 score to assess model performance.

- **Feature scaling:** Normalizing features to ensure consistency across inputs.
- **F1 score:** The harmonic mean of precision and recall.
- **Gradient boosting:** An ensemble method that builds models sequentially to reduce errors.
- **Log loss:** A metric for penalizing confident but incorrect predictions, closely related to cross-entropy loss.
- **Logistic regression:** A model that predicts probabilities for binary or multi-class outcomes.
- **Macro metrics:** Evaluation metrics treating all classes equally, regardless of size.
- **Micro metrics:** Aggregated metrics calculated across all classes for overall performance.
- **Modal imputation:** A technique for handling missing categorical data by replacing missing values with the most frequently occurring category in each feature, ensuring data completeness while preserving the original distribution.
- **Multi-class classification:** Predicting one of three or more distinct classes.
- **One-hot encoding:** Converting categorical variables into binary columns.
- **One-vs-One strategy:** Building binary classifiers for every pair of classes.
- **One-vs-Rest strategy:** Treating each class as a binary classification problem against all others.
- **Oversampling:** Generating synthetic samples for underrepresented classes.

- **Output code classifier:** A strategy for multi-class classification using binary coding patterns.
- **Precision:** Proportion of true positives out of all positive predictions.
- **Random forest:** An ensemble learning method using multiple decision trees.
- **Recall:** Proportion of true positives identified out of all actual positives.
- **SMOTEENN:** A combined approach to handle class imbalance by oversampling the minority class using SMOTE and cleaning the data through under sampling the majority class with Edited Nearest Neighbors.
- **StandardScaler:** A tool to normalize features to a mean of 0 and a standard deviation of 1.
- **Stratified K-fold:** A cross-validation technique ensuring consistent class distributions across folds.
- **Visualization:** Tools like decision boundary plots and confusion matrices to understand model behavior.

Join our Discord space

Join our Discord workspace for latest updates, offers, tech happenings around the world, new releases, and sessions with the authors:

<https://discord.bpbonline.com>



CHAPTER 5

Customer Segmentation with Unsupervised Methods

Introduction

Clustering is a fundamental technique in unsupervised learning, enabling the discovery of hidden patterns and structures within unlabeled data. This chapter explores key clustering methods, including K-means, agglomerative clustering, and DBSCAN, highlighting their strengths, limitations, and real-world applications. We delve into the mechanics of these algorithms, from how they form clusters to how their performance is evaluated using metrics such as silhouette scores and the elbow method.

Additionally, we introduce **principal component analysis (PCA)** as a powerful tool for visualizing high-dimensional clusters. Through practical implementations in scikit-learn, we demonstrate how clustering techniques can be applied to tasks such as customer segmentation, guiding readers in selecting and optimizing the best clustering approach for their datasets.

Structure

This chapter covers the following topics:

- Introducing unsupervised ML
- Understanding K-means clustering
- Understanding agglomerative clustering
- Understanding density-based scanning clustering

- Clustering evaluation techniques
- Customer segmentation

Objectives

By the end of this chapter, you will have a strong foundation in unsupervised learning, with a particular focus on clustering techniques. You will understand how K-means, agglomerative clustering, and DBSCAN work, their key differences, and when to apply each method. You will be able to preprocess data for clustering, including feature scaling, one-hot encoding, and dimensionality reduction using PCA to improve cluster formation and visualization. You will master strategies for selecting the optimal number of clusters, using techniques such as the elbow method and silhouette scores, and evaluate clustering performance with metrics like the Davies-Bouldin index and Calinski-Harabasz score.

Additionally, you will learn how to visualize cluster formations through scatter plots, dendrograms, and silhouette plots to assess the quality of your model. Finally, you will gain hands-on experience implementing these techniques using scikit-learn, culminating in a practical use case on customer segmentation, ensuring you can apply clustering effectively to real-world datasets.

Introducing unsupervised ML

Unsupervised learning is a branch of ML that deals with data without predefined labels (unlike supervised learning, which requires a label to train the model). This method aims to uncover patterns, structures, or relationships in data without predefined output labels.

In unsupervised learning, algorithms explore data to uncover hidden patterns. The two approaches used with this method are clustering and dimensionality reduction. **Clustering** involves grouping data points based on their similarities, such as grouping customers with similar patterns, based on data, as our use case will cover later. **Dimensionality reduction** simplifies high-dimensional data into fewer dimensions, allowing for the

retention of the most essential features while reducing noise in the data (e.g., PCA).

Understanding K-means clustering

K-means is a clustering algorithm that partitions a dataset into K distinct clusters, where the number of clusters, K, is determined in advance. Each cluster is represented by a central point, known as the cluster centroid, which serves as the average position of all data points in that cluster. The algorithm aims to optimize the clustering process by minimizing within-cluster variance and indirectly maximizing between-cluster separation, forming compact and distinct clusters.

Understanding the working of K-means

The algorithm is exploratory in nature, especially when compared to more statistical approaches, as it does not require predefined statistical assumptions about the data distribution. Instead, K-means focuses on uncovering patterns and structures within the feature space by iteratively minimizing within-cluster variance. This makes it a flexible and versatile tool for exploring unlabeled datasets, as its primary goal is to partition the feature space into compact and distinct clusters without relying on probabilistic models or prior hypotheses.

The steps involved in this process are outlined here:

1. **Initialization:** Randomly select K points in the dataset to serve as the initial centroids. These points can be chosen randomly or using techniques like K-means++ for improved convergence.
2. **Assignment step:** Assign each point to the cluster whose centroid is the closest. The distance is normally measured using the geometric **Euclidean** metric, though other metrics can be used depending on the data's characteristics.
3. **Update step:** Recalculate the centroid of each cluster by taking the mean of the feature values of all the points assigned to the cluster.

This step ensures that the centroid represents the updated *center of gravity* for that cluster.

4. **Iteration until convergence step:** Repeat the assignment and update steps until the centroids no longer move significantly, or a predefined maximum number of iterations is reached. The algorithm converges when the clusters stabilize, resulting in minimized within-cluster variance and well-separated clusters.

The visualization of these steps is captured in *Figure 5.1*:

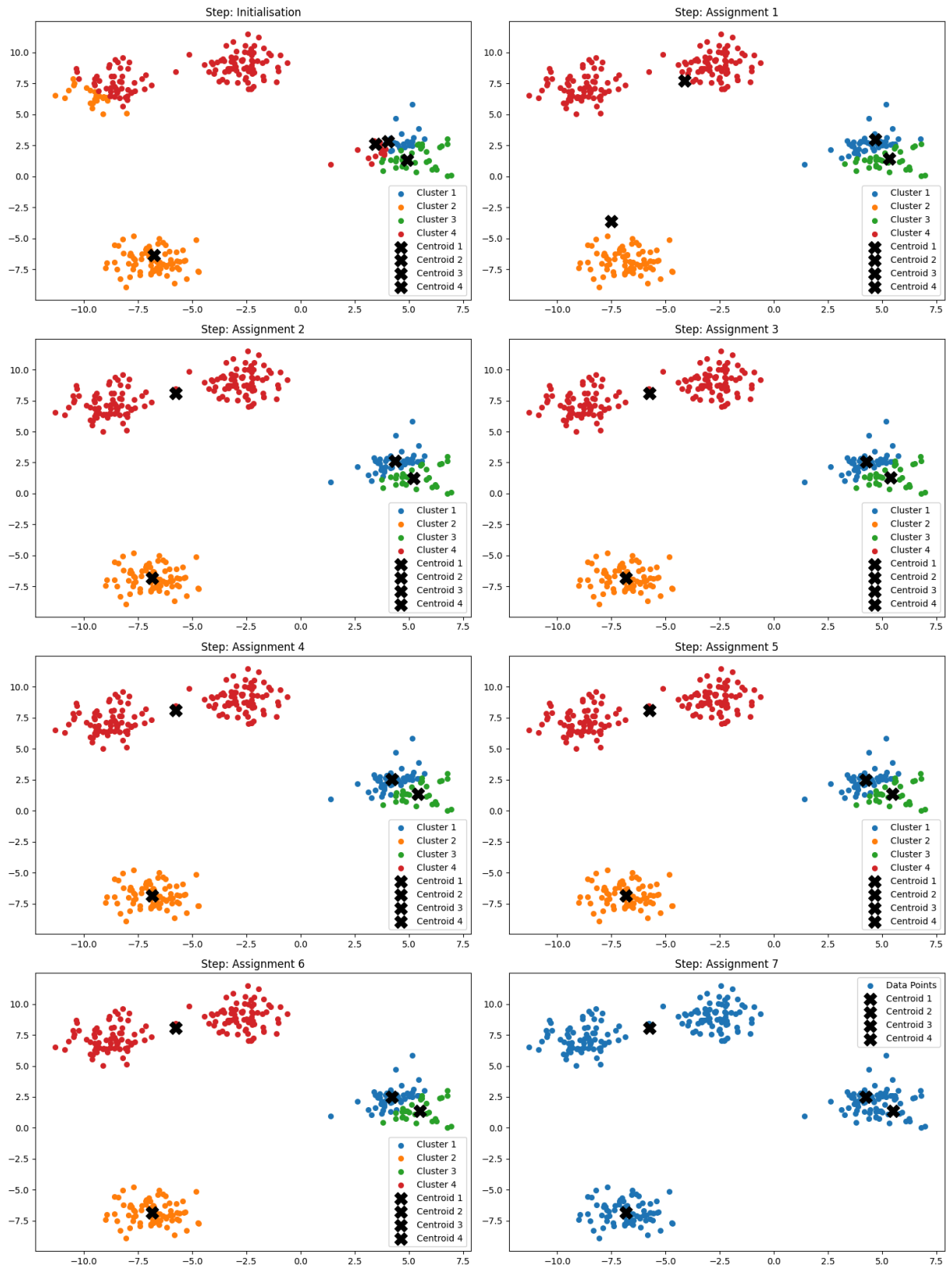


Figure 5.1: K-means stepwise process

K-means made easy with scikit-learn

In this section, we will generate some random clustering data, fit the K-means model, and then visualize our clusters. Let us start with the steps:

1. Import the relevant packages we are going to need for this:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
```

2. Generate synthetic cluster data, using scikit-learn's **make_blobs** function:

```
n_samples = 300
n_features = 5
n_clusters = 4
random_state = 42
X, y = make_blobs(n_samples=n_samples,
                  centers=n_clusters,
                  n_features=n_features,
                  random_state=random_state)
```

3. Initialize the K-means model:

```
kmeans = KMeans(n_clusters=n_clusters,
                random_state=random_state)
```

4. Fit the K-means model:

```
kmeans.fit(X)
```

5. Obtain the cluster labels, i.e., cluster 1, 2, 3, 4 (or with Python 0,1,2,3), as we have 4 clusters in *Step 2*:

```
labels = kmeans.labels_
```


6. Find the geometric position of the centroids:

```
centroids = kmeans.cluster_centers_
```

7. Get the cluster inertia (this is the within-cluster variance we mentioned earlier) to see how varied the variance is in the clusters. High variance would indicate clusters are not well defined, the K value is too small, or the data is complex, with high variance:

```
within_cluster_variance = kmeans.inertia_
```

8. Create custom Matplotlib code to produce a visualization of the clusters and analyze the mean centroid location:

```
plt.figure(figsize=(10, 5))
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis',
            marker='o', edgecolor='k', alpha=0.2)
for i, center in enumerate(centroids):
    circle = plt.Circle(center, 1.5,
                        color='black',
                        fill=False,
                        linestyle='--')
    plt.gca().add_artist(circle)
    plt.text(center[0], center[1], f'Cluster {i+1}',
            fontsize=10,
            color='black',
            ha='center')
plt.scatter(centroids[:, 0],
            centroids[:, 1],
            c='red',
            marker='X',
            s=200,
```

```

label='Centroids',
alpha=0.5)

plt.title("Clusters Identified by K-means")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
Pl. Legend()

plt.show()

```

That is all that is required to fit a model in scikit-learn. In this example, we fit the model on the full dataset. However, in the practical implementation, we will use a train-test split for evaluation. The final step generates the cluster visual, as shown in *Figure 5.2*:

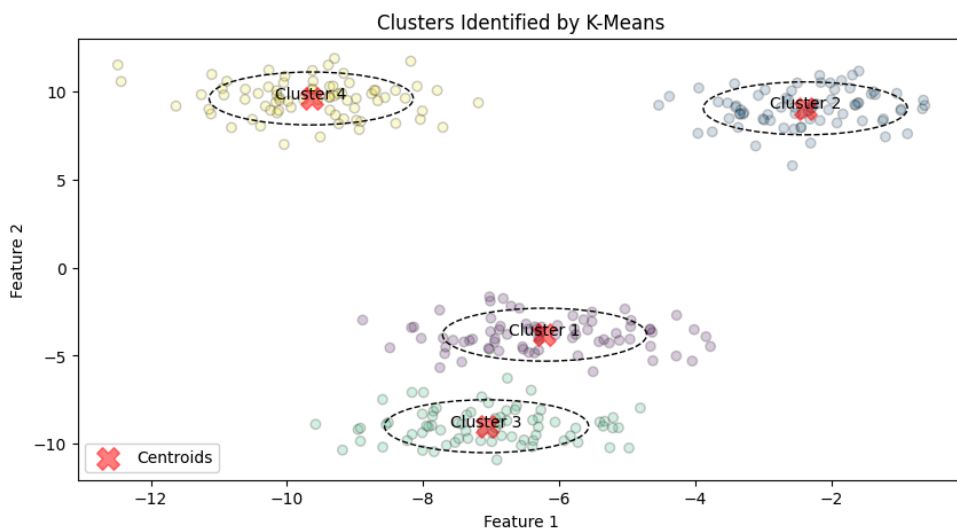


Figure 5.2: K-means clusters

Understanding agglomerative clustering

Agglomerative clustering is a hierarchical clustering algorithm that builds clusters incrementally by merging smaller clusters or individual data points. Unlike partitional methods like K-means, hierarchical clustering does not require the number of clusters to be specified in advance, making it a flexible tool for exploring datasets with unknown structures.

The algorithm produces a hierarchy of clusters, typically visualized as a dendrogram, which allows for an intuitive understanding of how data points relate to each other at different granularities.

Understanding agglomerative clustering working

Agglomerative clustering is known as a **bottom-up approach**, starting with each data point as its own cluster. It iteratively merges the closest clusters based on a distance metric until all data points become part of a single cluster or a specified number of clusters is reached. This procedure ensures that clusters are built hierarchically, capturing the data's global and local underlying structures.

Follow these steps:

1. **Initialization:** Each data point starts as its own cluster, meaning that if there are 500 data points, then there would be 500 clusters at this point.
2. **Computer distances pairwise:** a distance matrix is calculated to measure the similarity between clusters. Common metrics include Euclidean distance, Manhattan distance, cosine similarity, etc.
3. **Merge closest clusters:** the two clusters with the smallest distance are merged into a single cluster. This step reduces the total number of clusters by one.
4. **Update the distance matrix with linkage criterion:** recalculation happens using a linkage criterion, such as:
 - a. **Single linkage:** Minimum distance between points in two clusters.
 - b. **Complete linkage:** Maximum distance between points in two clusters.
 - c. **Average linkage:** Mean distance between all points in two clusters.

- d. **Ward's method:** Minimizes the variance within clusters:
<https://online.stat.psu.edu/stat505/lesson/14/14.7>.

This process is visualized in *Figure 5.3*:

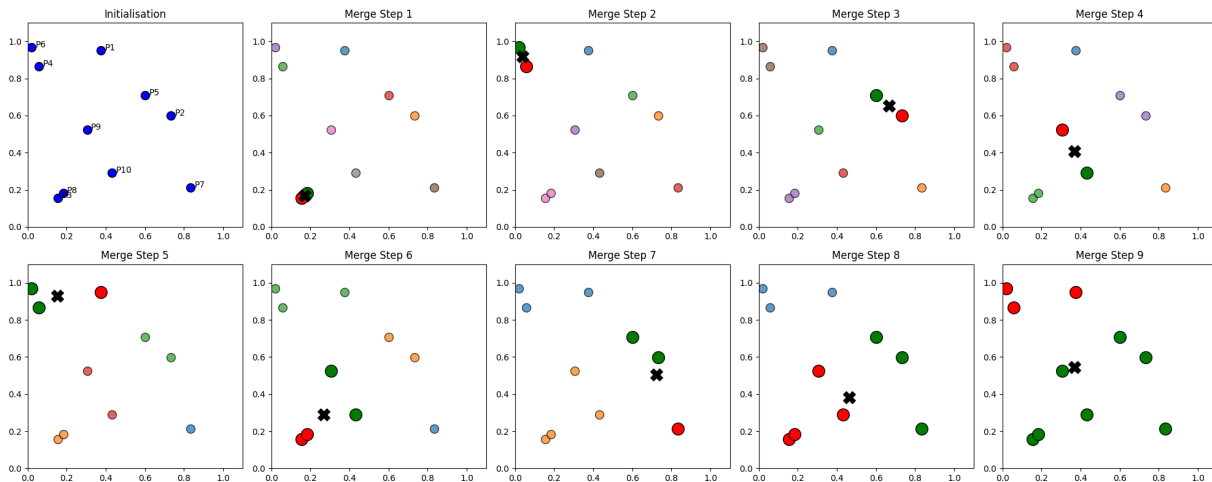


Figure 5.3: Agglomerative clustering steps

In the next section, we will look at how to fit and evaluate the agglomerative algorithm.

Agglomerative clustering with scikit-learn

In this section, we will perform agglomerative clustering, relatively painlessly, utilizing the scikit-learn package, and will learn how to visualize the results:

1. Import the relevant Python packages needed; here, you may need to install SciPy: <https://scipy.org/> using **pip install scipy**:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import make_blobs
```

```
from sklearn.cluster import AgglomerativeClustering
```

```
from scipy.cluster.hierarchy import dendrogram, linkage
```

2. Define variables and use **make_blobs** to generate a dataset for our foray into unsupervised clustering:

```
n_samples = 300
```

```
n_features = 2
```

```
n_clusters = 4
```

```
random_state = 42
```

```
X, _ = make_blobs(n_samples=n_samples,  
                  centers=n_clusters,  
                  n_features=n_features,  
                  random_state=random_state)
```

3. Fit our model using hierarchical clustering:

```
model = AgglomerativeClustering(n_clusters=n_clusters,  
                               linkage='ward')
```

```
labels = model.fit_predict(X)
```

4. Create a scatter plot to visualize the cluster distribution:

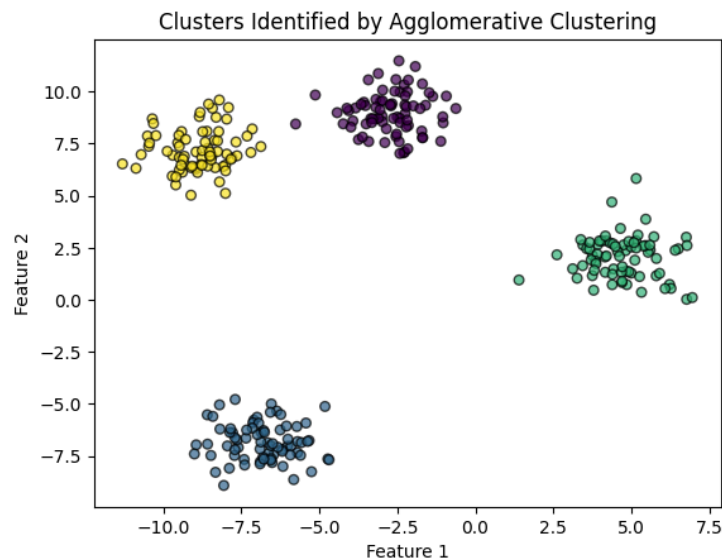


Figure 5.4: Clusters using agglomerative clustering

5. We will finally create a dendrogram to visualize our clusters, and we can then see how the clusters have been split based on the distance metric provided:

```

linked = linkage(X, method='ward')
plt.figure(figsize=(10, 7))
dendrogram(linked, p=4,
            orientation='top',
            show_leaf_counts=True)

plt.title("Hierarchical Clustering Dendrogram")
plt.xlabel("Data Points")
plt.ylabel("Distance")
plt.legend()
plt.show()

```

6. This generates a dendrogram that shows the clusters based on the point distance, by the relevant linkage method:

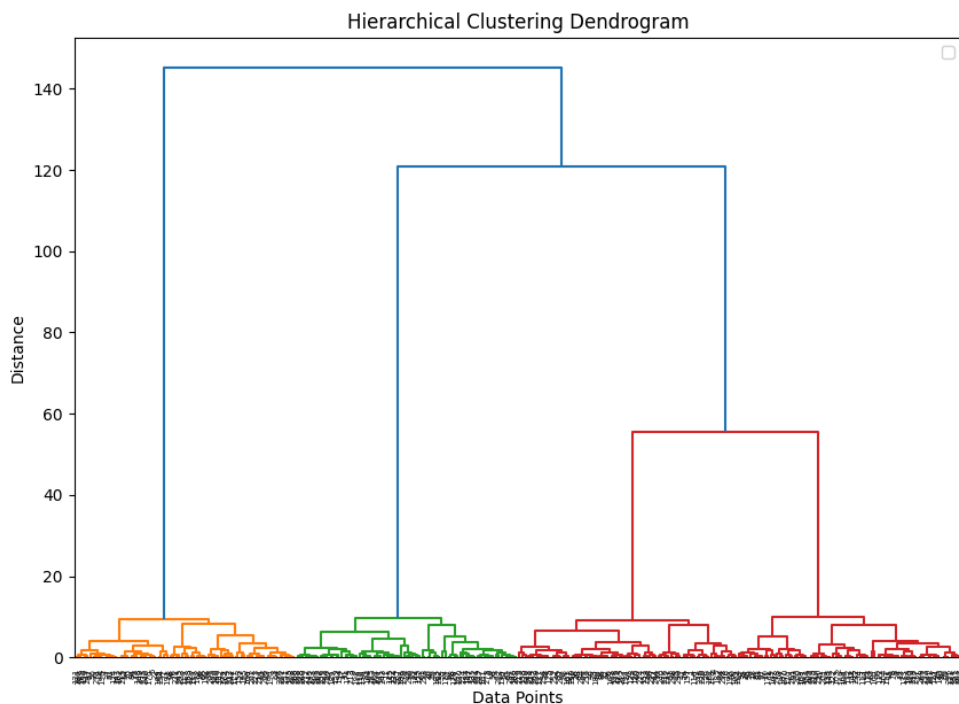


Figure 5.5: Hierarchical clustering dendrogram

In the next section, we will examine density-based scanning and show how it can be used to address more complex clustering challenges than the ones

we have outlined thus far.

Understand density-based scanning clustering

Density-based spatial clustering of applications with noise (DBSCAN) is a powerful clustering algorithm that groups data points based on density. It identifies clusters of varying shapes and sizes while effectively detecting and handling noise (outliers). Unlike partitioning methods like K-means or hierarchical clustering, DBSCAN does not require the number of clusters to be specified beforehand, making it highly adaptable for datasets with complex structures.

DBSCAN's ability to form clusters based on regions of high density, separated by regions of low density, makes it particularly useful for datasets with arbitrary cluster shapes, high noise levels, and even high-dimensional data.

How DBSCAN works

DBSCAN is based on the concepts of density, core points, border points, and noise. It iteratively forms clusters by expanding high-density areas and distinguishing noise points that do not belong to any cluster.

Typically, the process involves these steps:

1. **Parameter definition:**
 - a. **Epsilon (eps)** is the maximum distance between two points for them to be considered neighboring points.
 - b. **min_samples** is the minimum number of data points required to form a dense region (core point).
2. **Identify core and border points:** A core point is a data point with at least the **min_samples** neighbors within the epsilon radius, and a border point lies within the epsilon radius of the core point.
3. **Classify noise:** Points that are neither core points nor border points are classified and defined as noise, or noisy points.

4. **Cluster expansion:** Follow these steps:
 - a. Start from an unvisited core point.
 - b. Expand the cluster by iteratively adding all directly reachable points and their neighbors with the epsilon radius.
 - c. Repeat this process until all reachable points are included in the cluster.
5. **Iterate and repeat for remaining points:** Move to the next unvisited core point and repeat the process until all points are processed.

This iterative approach ensures DBSCAN can effectively handle arbitrary shapes, high-dimensional data, and noise, making it robust for exploratory data analysis and clustering data of various types of complexity.

The steps outlined in this section are brought to life. Refer to *Figure 5.6*:

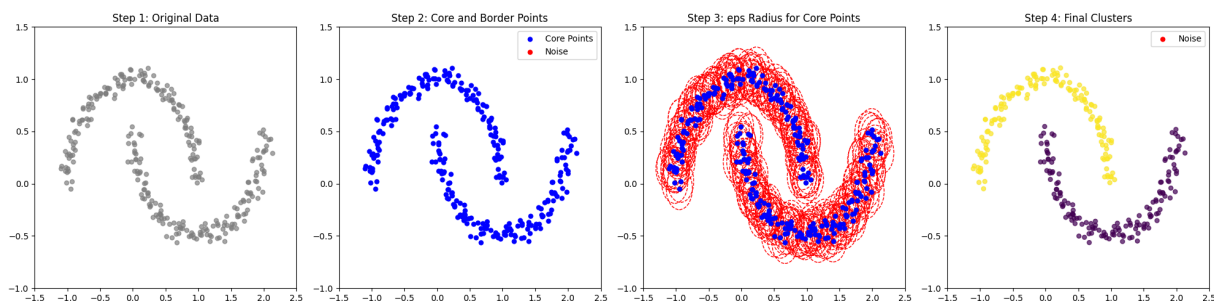


Figure 5.6: DBSCAN steps to determine optimal clusters

DBSCAN with scikit-learn

With a solid understanding of what DBSCAN does, let us apply it to a high-dimensional dataset using the built-in **make_swiss_roll** function. The following steps will guide you through importing the dataset, fitting the model, and creating meaningful visualizations to interpret the results:

1. Gather the imports needed to perform this unsupervised learning technique on high-dimensional data:

```
from sklearn.datasets import make_swiss_roll
```



```
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN
from sklearn.decomposition import PCA
import numpy as np
```

2. Synthetically generate high-dimensional data, using the **make_swiss_roll** method in **sklearn.datasets**:

```
n_samples = 1000
noise = 0.05
X, _ = make_swiss_roll(n_samples=n_samples,
                       noise=noise)
```

3. Next, we will use Matplotlib (<https://matplotlib.org/>) to generate a high-dimensional visualization of the generated data:

```
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(X[:, 0],
           X[:, 1],
           X[:, 2],
           c='blue', s=5)
plt.title("Swiss Roll Dataset")
plt.show()
```

4. This generates a plot, as shown in *Figure 5.7*:

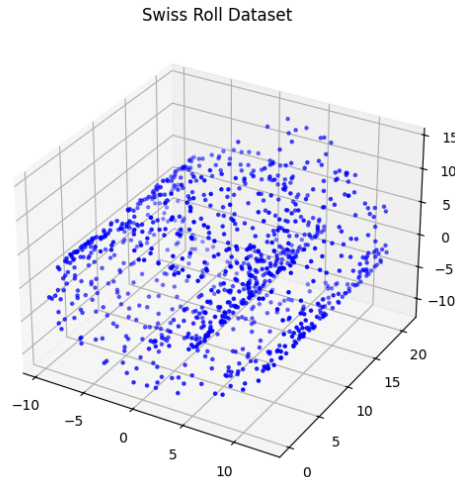


Figure 5.7: Dataset generated by `make_swiss_roll` method

5. As this data is highly dimensional across several features, we are going to need to use PCA to shrink the features down to two components that can be displayed on a scatter chart to enable you to understand how the algorithm performs on the task of grouping this dataset into clusters:

```
pca = PCA(n_components=2)
```

```
X_reduced = pca.fit_transform(X)
```

6. Model fitting is the next step; here, we will use **eps** (the maximum distance between points for them to be considered as neighbors) and **min_samples** (the minimum number of data points to be considered for the formation of a dense region, known as a core point). We will set the epsilon to 1.0 and the **min_samples** to 4, these are parameters you can tweak with hyperparameter tuning in addition, but we will start with some initializations that work well:

```
eps = 1.0
```

```
min_samples = 4
```

```
model = DBSCAN(eps=eps,
```

```
min_samples=min_samples)
```

7. We will now **fit_predict** on our **X_reduced** data (a byproduct of the PCA step in *Step 4*):

```
labels = model.fit_predict(X_reduced)
```

8. Finally, we will visualize the cluster results across the two reduced components as part of the PCA process:

```
plt.figure(figsize=(8, 6))
```

```
plt.scatter(X_reduced[:, 0],
```

```
            X_reduced[:, 1],
```

```
            c=labels,
```

```
            cmap='viridis',
```

```
            s=10)
```

```
plt.title("DBSCAN Clusters - Swiss Roll Dataset")
```

```
plt.xlabel("PCA Component 1")
```

```
plt.ylabel("PCA Component 2")
```

```
plt.show()
```

9. The code produces the visual in *Figure 5.8*:



Figure 5.8: Swiss roll with colored clusters identified by DBSCAN

The chart in *Figure 5.8* illustrates that clusters have been successfully identified based on the chosen hyperparameter values (**eps** and **min_samples**). These parameters play a crucial role in defining the density thresholds that determine which points form clusters and which are classified as noise. The results demonstrate DBSCAN's ability to discover clusters of varying densities and shapes, highlighting its suitability for datasets with non-linear structures or irregular geometries.

In addition to the **make_swiss_roll** dataset, the **sklearn.datasets** module provides other synthetic datasets like **make_circles** and **make_moons**. These datasets are excellent tools for practicing the application of DBSCAN on data with diverse shapes, sizes, and geometric complexities. For instance, **make_circles** produces concentric circles, while **make_moons** generates crescent-shaped clusters. These datasets allow users to explore how hyperparameter tuning influences DBSCAN's performance and better understand its strengths in handling arbitrarily shaped clusters.

Now, we will move on to the ways to assess your cluster performance, which combine heuristics with statistical methods.

Clustering evaluation techniques

Selecting the optimal number of clusters and evaluating the quality of clustering results are critical steps in unsupervised learning. These tasks ensure that the clusters accurately represent the underlying structure of the data without overfitting or underfitting. Several techniques aid in this process, each providing unique insights into cluster validity. The elbow method helps determine the number of clusters by analyzing within-cluster variance, while silhouette scores and plotting assess the cohesion and separation of clusters. Metrics like the Davies-Bouldin Index and Calinski-Harabasz score offer quantitative evaluations of cluster compactness and separability, making them valuable tools for comparing different clustering models. By leveraging these methods, practitioners can make informed decisions and ensure robust clustering outcomes.

Elbow method

The elbow is a popular and flexible method that evaluates how the total **Within-Cluster Sum Of Squares (WCSS)**, also known as **inertia**, changes as the number of clusters increases. The elbow method involves plotting the **WCSS (inertia)** against the number of clusters and identifying the point where the rate of decrease in WCSS sharply diminishes, forming an "elbow" shape. This point typically represents the optimal number of clusters, as adding more clusters beyond this does not significantly reduce WCSS. The rationale behind this method is that, while increasing the number of clusters improves how well the data fits within clusters, doing so beyond a specific point results in diminishing returns. The WCSS is defined as:

$$\text{WCSS} = \sum_{k=1}^K \sum_{i \in C_k} \|x_i - c_k\|^2$$

Where K is the number of clusters, C_k is the set of points in cluster k , x_i is a data point, and c_k is cluster k 's centroid. By plotting WCSS against the number of clusters, the method identifies the elbow point, where the rate of decrease in WCSS sharply diminishes. This point indicates the optimal number of clusters, balancing model complexity and performance, as increasing the number of clusters beyond these yields diminishing returns in reducing WCSS.

In Python, we have created a function to enable plotting the elbow method, which helps determine the optimal number of clusters for a clustering model, such as K-means. It runs the model for different values of K (from 1 to `max_k`), records the inertia (a measure of cluster compactness), and plots the results. The elbow point on the graph shows where increasing K no longer significantly reduces inertia, helping to choose the best number of clusters.

This is implemented in the following code:

```

1. def plot_elbow_method(X, model_class,
2.                        max_k=10, figsize=(8, 6),
3.                        title='Elbow Method For Optimal K',
4.                        xlabel='Number of clusters (K)',
5.                        ylabel='Inertia (SSE)',
6.                        grid=True, **kwargs):
7.     inertia = []
8.     k_val_rng = range(1, max_k + 1)
9.     for k in k_val_rng:
10.         model = model_class(n_clusters=k, **kwargs)
11.         model.fit(X)
12.         if hasattr(model, 'inertia_'):
13.             inertia.append(model.inertia_)
14.         else:
15.             print(f"{model_class.__name__} doesn't have 'inertia_' f
or k={k}.")
16.             break
17.     if inertia:
18.         plt.figure(figsize=figsize)
19.         plt.plot(k_val_rng, inertia, marker='o')
20.         plt.title(title)
21.         plt.xlabel(xlabel)
22.         plt.ylabel(ylabel)
23.         if grid:
24.             plt.grid(True)
25.         plt.show()

```

To use this function, we pass in the data as shown in this example:

```
plot_elbow_method(X=X,  
                  model_class=KMeans,  
                  max_k=10)
```

This outputs the visualization captured in *Figure 5.9*:

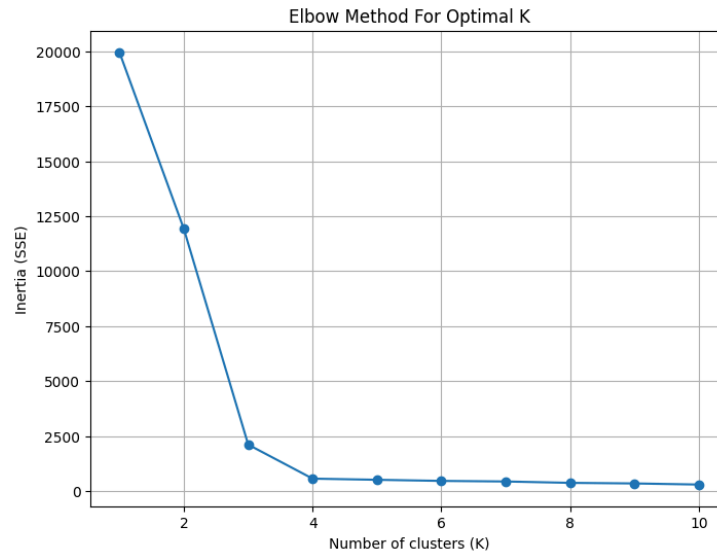


Figure 5.9: Elbow plot

Here, you can see the optimal clusters, for this specific dataset is 4. The next area we will look at is the silhouette score.

Silhouette score

The silhouette score is a widely used metric for evaluating the quality of clustering results by measuring how well-separated and cohesive the clusters are. It calculates how similar a data point is to its cluster (cohesion) compared to the nearest neighboring cluster (separation). The score ranges from -1 to 1, where 1 indicates well-defined clusters, 0 suggests overlapping clusters, and negative values imply that points may be incorrectly clustered.

The formula for this is here:

$$S(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

The formula terms are explained as follows:

- $S(i)$ is the silhouette score for a single data point i .
- $a(i)$ is the average intra-cluster distance, meaning the average distance between i and all other points in the same cluster.
- $b(i)$ is the average nearest-cluster distance, meaning the average distance between i and all points in the nearest neighboring cluster.

In the next section, we will look at a way to visualize this, with the silhouette plot.

Silhouette plot

A silhouette plot visually represents the silhouette scores of individual data points within each cluster, helping to assess cluster cohesion and separation. Each cluster is displayed as a bar, with the width of the bars indicating the number of points in the cluster and their silhouette scores. Ideally, all bars should be broad and extend close to 1, indicating well-separated clusters. Narrow or negative silhouette values suggest overlapping or poorly assigned clusters. By examining the plot, practitioners can quickly identify inconsistencies and determine whether the chosen number of clusters is appropriate.

To implement this in Python, we have provided a function to plot this for you, which will be needed for the use case:

```
1. from sklearn.metrics import silhouette_samples, silhouette_score
2. import matplotlib.pyplot as plt
3. import numpy as np
4. def silhouette_plot(X, labels, n_clusters,
5.                    title="Silhouette Plot for Clustering",
6.                    fig_size=(8,6),
7.                    x_lab='Coefficient Values',
8.                    y_lab='Cluster',
```



```
9.             silh_colors=None,
10.             x_axis_line_col='black'):
11.     sil_vals = silhouette_samples(X, labels)
12.     mean_score = silhouette_score(X, labels)
13.     _, ax = plt.subplots(figsize=fig_size)
14.     y_lower = 10
15.     if silh_colors is None:
16.         silh_colors = plt.cm.tab10.colors
17.     for i in range(n_clusters):
18.         cluster_sil_vals = sil_vals[labels == i]
19.         cluster_sil_vals.sort()
20.         y_upper = y_lower + len(cluster_sil_vals)
21.         colour = silh_colors[i % len(silh_colors)]
22.         ax.fill_betweenx(
23.             np.arange(y_lower, y_upper),
24.             0, cluster_sil_vals, alpha=0.7, color=colour
25.         )
26.         ax.text(-0.05, y_lower + 0.5 * len(cluster_sil_vals), str(i))
27.
28.         y_lower = y_upper + 10
29.     ax.axvline(x=mean_score, color=x_axis_line_col, linestyle="--")
30.     ax.set_title(title)
31.     ax.set_xlabel(x_lab)
32.     ax.set_ylabel(y_lab)
33.     ax.set_yticks([])
34.     grid_lines = np.arange(-0.1, 1.1, 0.1)
```

```
35. plt.show()
```

To use this plot, we use the following code:

```
1. silhouette_plot(X, labels, n_clusters,  
2. silh_colors=['red', 'black', 'navy', 'orange'])
```

This produces the silhouette plot:

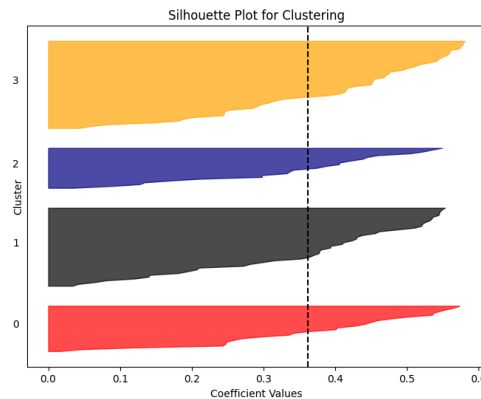


Figure 5.10: Silhouette plot

In the practical section, we will reuse these visualization techniques, so we will store them as functions in the supporting repository and import them into our project to avoid redundant coding.

Customer segmentation

Customer segmentation helps businesses identify distinct groups based on purchasing behavior, enabling personalized marketing and better decision-making. In this use case, we will analyze customer spending data with fields such as **Age**, **AnnualIncome**, **SpendingScore**, **OnlineShoppingFrequency**, **CreditCardUsage**, **CustomerValueScore**, **CustomerID**, and **Gender**.

Our approach involves importing and cleaning the data, removing unnecessary fields, and preparing it by one-hot encoding categorical variables and scaling numerical features. We will then apply clustering models, iterating through different cluster counts using techniques like the elbow method and silhouette scores to find the optimal segmentation.

Finally, we will evaluate the results using clustering metrics and visualizations to ensure meaningful and actionable customer groupings.

Import and clean dataset

The first step is to import and clean the dataset before we do anything on the clustering side. The following steps will take us from data import to understanding missing values to dropping the fields that are not of numeric value or used for the ML model:

1. Let us import the data, as the first step:

```
import pandas as pd  
cs_ds = pd.read_csv('data/customer_data.csv')
```

2. We now want to examine the head of the dataset:

```
cs_ds.head()
```

This outputs the following table. Refer to *Figure 5.11*:

Age	AnnualIncome	SpendingScore	OnlineShoppingFrequency	CreditCardUsage	CustomerValueScore	CustomerID	Gender
74	120013	33.000000	6	13.970000	33.584103	1	Female
24	29997	57.000000	1	9.180000	17.278559	2	Female
58	120005	14.000000	8	32.200000	42.867957	3	Female
20	29991	48.000000	7	3.560000	31.959367	4	Female
59	119983	27.000000	1	6.250000	42.510361	5	Female

Figure 5.11: Customer segmentation dataset head

3. Previously, in *Chapter 3, Supervised Binary Classification*, we defined a function for analyzing missing values; we will use that again here, and the function is stored in a folder that comes with the GitHub repository, namely *unsupervised*.
4. We will import this function into our environment from that folder and use it against our dataset in *Figure 5.13*:

```
from unsupervised.missing_values import missing_values_summarizer
```

```
prop_miss_df, _ = missing_values_summarizer(cs_ds)
```

5. Printing the **prop_miss_df**, we get the summary as follows:

feature	missing_count	non_missing_count	total_vol	prop_missing	prop_non_missing	data_type
Age	0	12000	12000	0.000000	1.000000	int64
AnnualIncome	0	12000	12000	0.000000	1.000000	int64
SpendingScore	0	12000	12000	0.000000	1.000000	float64
OnlineShoppingFrequency	0	12000	12000	0.000000	1.000000	int64
CreditCardUsage	0	12000	12000	0.000000	1.000000	float64
CustomerValueScore	0	12000	12000	0.000000	1.000000	float64
CustomerID	0	12000	12000	0.000000	1.000000	int64
Gender	0	12000	12000	0.000000	1.000000	object

Figure 5.12: Missing values summary

6. The last step in this section will be to drop the **CustomerID** field, as this has no relevance for model training:

```
cs_dataset = cs_ds.drop(['CustomerID'],  
                        axis=1)
```

Partitioning data

Unsupervised methods can be applied to the entire dataset. Still, splitting the dataset into training and testing sets allows us to evaluate how well the clustering model generalizes to unseen data. By fitting the model on the training set and validating it on the testing set, we can assess whether the identified clusters remain consistent and meaningful across different subsets of the data.

This approach provides checks and balances against the optimal cluster sizes, ensuring that the clustering is robust and not overfit to the specific characteristics of the training data. It also helps identify potential issues, such as clusters being too dependent on particular features or data points, ultimately leading to a more reliable model.

To split the data, we will perform the following:

1. Get the **train_test_split** function imported from **sklearn.model_selection**:

```
from sklearn.model_selection import train_test_split
```

2. Split just the features for X, as we do not have a label here, we would not need to pass in the Y (dependent variable) field, as before:

```
X_train, X_test = train_test_split(cs_dataset,  
                                  test_size=0.3,  
                                  random_state=42)
```

3. Now, the data is ready for feature engineering, such as one-hot encoding and scaling.

One-hot encoding

As with supervised techniques, we must convert categorical columns into integer representations of those columns. We will do this specifically on the **Gender** feature:

1. Initialize the **OneHotEncoder** function:

```
from sklearn.preprocessing import OneHotEncoder  
  
one_hot_enc = OneHotEncoder(drop='first',  
                             handle_unknown='ignore')
```

2. Fit the encoder to the **Gender** field:

```
one_hot_enc.fit(X_train[['Gender']])
```

3. Now that we have fitted the one-hot encoder, we will need to **transform** our original **X_train** and **X_test** data:

```
X_train_encoded = one_hot_enc.transform(X_train[['Gender']]).toarray()  
  
X_test_encoded = one_hot_enc.transform(X_test[['Gender']]).toarray()
```

4. The above will create the encodings in arrays. It is also ideal if we save these to a Pandas DataFrame object, and here we will use the **get_feature_names_out** function to get the encoded field name back, i.e., **Gender**:

```

X_train_enc_df = pd.DataFrame(
    X_train_encoded,
    columns=one_hot_enc.get_feature_names_out(['Gender'])
)
X_test_enc_df = pd.DataFrame(
    X_test_encoded,
    columns=one_hot_enc.get_feature_names_out(['Gender'])
)

```

5. It is good practice, before concatenating DataFrames, to reset and drop the original indices from the source **X_train** and **X_test** datasets, to make sure that when the concatenation occurs (stitching two DataFrames together), the package does not get confused with mixing the indices. This can be done as shown in the following example:

```

X_train.reset_index(drop=True, inplace=True)
X_test.reset_index(drop=True, inplace=True)

```

6. The last step to stitch these frames together is to use **concat()** from the Pandas library on the train and testing feature sets. We use **axis=1**, as we want to append columns, not rows, which would be **axis=0**:

```

X_train = pd.concat(
    [X_train.drop(['Gender'], axis=1),
    X_train_enc_df], axis=1)
X_test = pd.concat(
    [X_test.drop(['Gender'], axis=1),
    X_test_enc_df], axis=1)

```

This has completed the one-hot encoding phase; now we will use the concatenated **X_train** and **X_test** arrays to scale the values in

the data, as the remaining data is all numeric.

Feature scaling

Feature scaling is essential in K-means clustering because the algorithm relies on Euclidean distance to assign clusters. Without scaling, features with larger ranges dominate, skewing the clustering process and leading to biased groupings.

We have chosen **StandardScaler()** as it standardizes features to have a mean of 0 and a standard deviation of 1, ensuring all features contribute equally. This prevents distortion in distance calculations, leading to more accurate and reliable clusters.

We will perform the following steps to scale our datasets:

1. Import and initialize the **StandardScaler()**:

```
from sklearn.preprocessing import StandardScaler  
  
scaler = StandardScaler()
```

2. Fit the scaler to the training set:

```
scaler.fit(X_train)
```

3. Transform the splits:

```
X_train_scaled = scaler.transform(X_train)  
  
X_test_scaled = scaler.transform(X_test)
```

We now have all that we need to start fitting our K-means model to this dataset and then do some statistical evaluation of the performance of the clusters and the algorithm itself.

Cluster performance visualization

In this section, we have created a function to allow you to visualize clustering metrics across different values of K, helping you determine the optimal number of clusters in K-means clustering. This function plots

silhouette scores, Davies-Bouldin index, and Calinski-Harabasz scores, providing a comparative assessment of clustering quality.

Each metric provides a distinct view of cluster performance. Let us look at them in detail:

- **Silhouette score** measures how well-separated clusters are, with higher values indicating clearer boundaries.
- **Davies-Bouldin index** evaluates cluster compactness and separation, where lower values indicate better clustering.
- **Calinski-Harabasz score** assesses cluster dispersion, with higher values suggesting well-defined and distinct clusters.

By analyzing these plots, you can make an informed decision on the best K-value, balancing compactness, separation, and overall clustering structure. The function is implemented in the following code, and we will use this when we come to dynamically select different clusters:

```
1. import matplotlib.pyplot as plt
2. def plot_clustering_metrics(k_vals,
3.                             sil_scores,
4.                             davies_scores,
5.                             cal_hab_scores,
6.                             x_axis_lbl='Number of clusters (k)',
7.                             figsize=(10,15)):
8.
9.     _, axes = plt.subplots(3, 1, figsize=figsize)
10.    axes[0].plot(k_vals, sil_scores, marker='o', color='blue')
11.    axes[0].set_title('Silhouette Score vs. Number of Clusters (K-means)')
12.    axes[0].set_xlabel(x_axis_lbl)
13.    axes[0].set_ylabel('Silhouette Score')
```



```
14. axes[0].grid(True)
15. axes[1].plot(k_vals, davies_scores, marker='o', color='red')
16. axes[1].set_title('Davies-Bouldin Index vs. Number of Clusters')
17. axes[1].set_xlabel('Number of clusters (K)')
18. axes[1].set_ylabel('Davies-Bouldin Index')
19. axes[1].grid(True)
20. axes[2].plot(k_vals, cal_hab_scores, marker='o', color='green')
21. axes[2].set_title('Calinski-Harabasz Score vs. Number of Clusters')
22. axes[2].set_xlabel('Number of clusters (K)')
23. axes[2].set_ylabel('Calinski-Harabasz Score')
24. axes[2].grid(True)
25. plt.tight_layout()
26. plt.show()
```

We will not use this function in this section, as we will use it in the dynamic evaluation of cluster sizes, like what we have already learned about with the elbow plot method in this chapter.

Instantiate the K-means model

Next, we will instantiate the K-means model, as it will be a key component in our optimal cluster search loop. This model will be repeatedly fitted to the data for different values of K, allowing us to systematically evaluate cluster quality and identify the most suitable number of clusters.

By iterating over multiple K-values and assessing the resulting clustering metrics, we can determine the configuration that best balances compactness, separation, and overall structure. This step is crucial in ensuring that our clustering solution is both meaningful and effective for our dataset. This is done by using the following code:


```

4. def find_optimal_clusters(X, model,
5.                             n_clusters=10,
6.                             majority_vote=True,
7.                             visualize=True):
8.     if not hasattr(model, "fit"):
9.         raise ValueError("Not a scikit-learn Model")
10.    k_vals = range(2, n_clusters)
11.    sil_scores = []
12.    davies_scores = []
13.    cal_hab_scores = []
14.    for k in k_vals:
15.        if hasattr(model, 'n_clusters'):
16.            model.set_params(n_clusters=k)
17.            model.fit(X)
18.            labels = model.labels_ if hasattr(model, "labels_") else model.fit_predict(X)
19.            sil_scores.append(silhouette_score(X, labels))
20.            davies_scores.append(davies_bouldin_score(X, labels))
21.            cal_hab_scores.append(calinski_harabasz_score(X, labels))
22.    optim_k_silhouette = k_vals[np.argmax(sil_scores)]
23.    optim_k_davies_bouldin = k_vals[np.argmin(davies_scores)]
24.    optim_k_cal_hab = k_vals[np.argmax(cal_hab_scores)]
25.    print(f'Optimal cluster based on Silhouette score is: {optim_k_silhouette}')
26.    print(f'Optimal cluster based on Davies-Bouldin score is: {optim_k_davies_bouldin}')
27.    print(f'Optimal cluster based on Calinski-Harabasz score is: {optim_k_cal_hab}')

```

```

28.     optim_k_comb = None
29.     if majority_vote:
30.         optim_k_comb = max(set([optim_k_silhouette,
31.                                 optim_k_davies_bouldin,
32.                                 optim_k_cal_hab]),
33.                             key=[optim_k_silhouette,
34.                                 optim_k_davies_bouldin,
35.                                 optim_k_cal_hab].count)
36.
37.     if visualize:
38.         plot_clustering_metrics(k_vals, sil_scores=sil_scores,
39.                                 davies_scores=davies_scores,
40.                                 cal_hab_scores=cal_hab_scores)
41.
42.
43.     return (optim_k_comb, optim_k_silhouette,
44.            optim_k_cal_hab, optim_k_davies_bouldin)

```

To explain this function, we will break down the steps:

1. Creates a function that calculates the silhouette, Davies-Bouldin, and Calinski-Harabasz scores (these are additional ways of finding the right clustering values).
2. Takes an unsupervised model as input.
3. For each value K (cluster) in a range of cluster values, iterate through:
 - a. Fits the model to the dataset at each k interval.
 - b. Retrieves the cluster labels using the model's **fit_predict** method.

- c. Compare and store the scores from the statistical measures, i.e., silhouette, etc.
4. Determine optimal clusters:
 - a. Identify the K with the highest silhouette and Calinski-Harabasz score.
 - b. Identify the K with the lowest Davies-Bouldin score.
5. Majority vote decision. If the majority vote flag is set, determine the overall optimal cluster number by selecting the most common value among the three statistical measures with their respective optimal K.
6. Visualize metrics. If the visualization flag is enabled in the function, call the plotting function we implemented in the *Cluster performance visualization* section.
7. Return the results. Return a tuple data type containing the majority vote optimal cluster and the individual cluster labels.

The next stage is to test on the train and testing sets and assess the cluster performance.

Finding optimal training clusters

We will use the function we just created in the previous section to find the optimal clusters on the training set. To do this, we use the following command:

```
1. clusters, __, __, __ = find_optimal_clusters(X=X_train_scaled,  
2.                                         n_clusters=10,  
3.                                         model=model)
```

As the visualize flag is set to true by default, we get a plot, as shown in *Figure 5.13*:

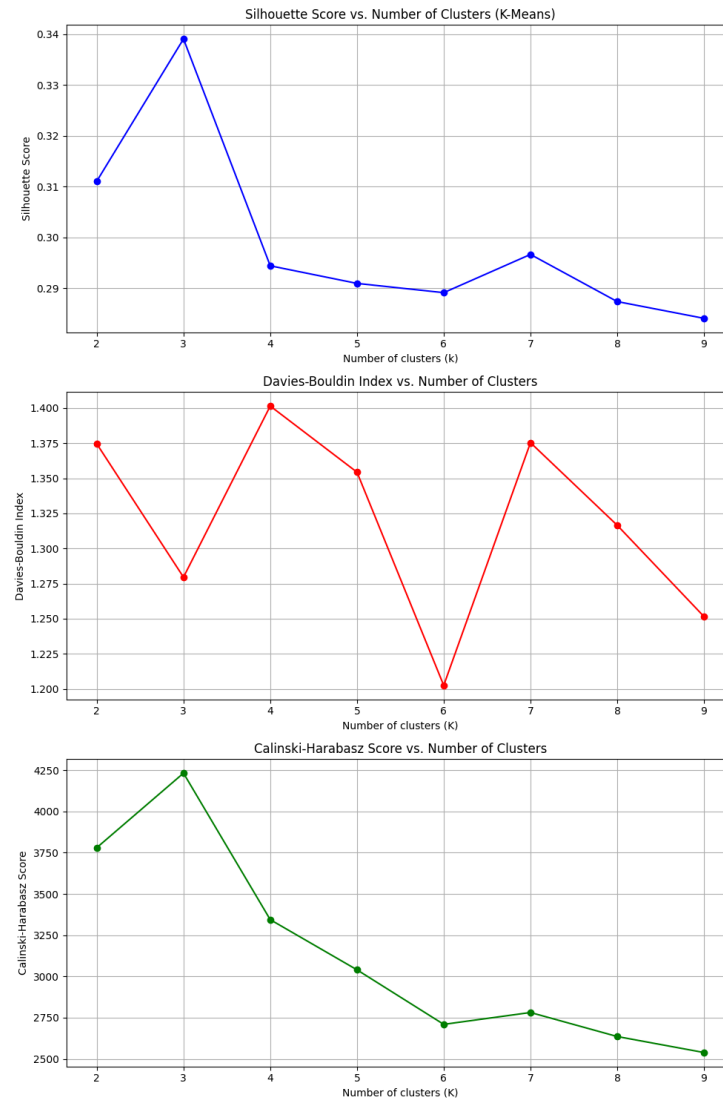


Figure 5.13: Optimal cluster charts

The preceding figure shows that the silhouette score and Calinski-Harabasz score both agree that the optimal cluster arrived at is 3, whereas Davies-Bouldin seems to think something different.

Now that we have this optimal cluster, we will fit our final model:

1. `model = KMeans(n_clusters=clusters)`
2. `kmeans_labels = model.fit_predict(X=X_train_scaled)`

In the next section, we will use the final model and assigned clusters and apply PCA to the dataset to visualize our clusters on a scatter chart.

PCA to visualize our clusters

When working with high-dimensional data, visualizing clusters can be challenging. PCA helps by reducing the data to two or three dimensions while preserving its key structure. This makes it easier to see the patterns and groupings formed by K-means clustering.

PCA highlights the most important variations in the data, often aligning with the natural cluster boundaries. Simplifying complex datasets without losing valuable information provides a clear and intuitive way to interpret K-means results.

To perform PCA on our dataset and then visualize, this can be done in the following simple steps:

1. Import the **PCA** function from **sklearn.decomposition**:

```
from sklearn.decomposition import PCA
```

2. Initialize a PCA model to reduce all our features in the dataset to two principal components – a principal component aims to retain the variability of the features:

```
pca = PCA(n_components=2)
```

3. We will fit the PCA model to the dataset and transform it using our trusty **fit_transform** method:

```
reduced_data = pca.fit_transform(X_train_scaled)
```

4. Next, we use a scatter matplotlib method to plot our scatter chart across the reduced two principal components:

```
import matplotlib.pyplot as plt
```

```
def plot_scatter(X, labels, title,
```

```
                title_fsize=14,
```

```
                lab_font_size=12,
```

```
                cmap='winter',
```

```
                axis1=0,
```

```

        axis2=1,
        scatter_edge_color='k'):
plt.figure(figsize=(8, 6))
scatter = plt.scatter(X[:, axis1], X[:, axis2],
                      c=labels, cmap=cmap, s=40,
                      edgecolor=scatter_edge_color, alpha=0.
7)

plt.colorbar(scatter, label='Cluster/Outlier Labels')
plt.title(title, fontsize=title_fsize)
plt.xlabel(f"Feature {axis1+1}", fontsize=lab_font_size)
plt.ylabel(f"Feature {axis2+1}", fontsize=lab_font_size)
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()

```

5. To run the function, that is a simple scatter chart, we use the **plot_scatter** method to call the function:

```

plot_scatter(X=reduced_data,
             labels=kmeans_labels,
             title='PCA on KMeans')

```

6. On running this line, we get a scatter chart of the cluster separation across those two components, as trying to visualize this across the original seven features would not be possible, as essentially, we would have to find a way to visualize this across greater than several dimensions:

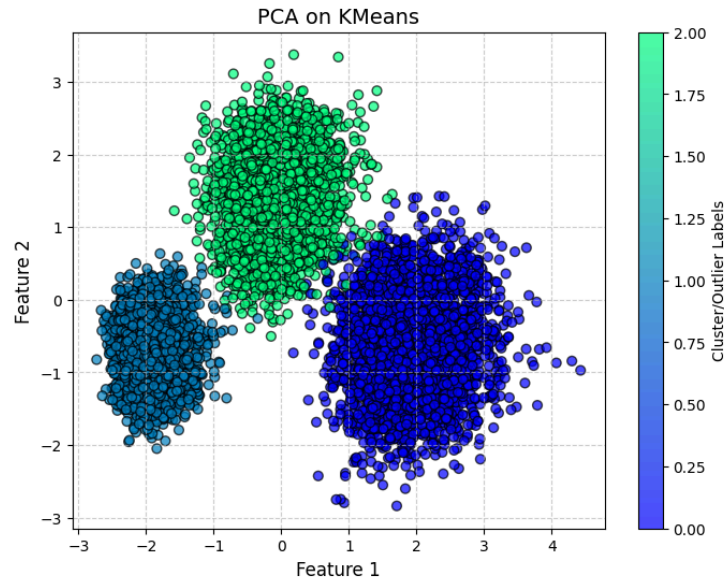


Figure 5.14: Cluster distributions on scatter chart

Evaluation of performance on testing set

To confirm that the training dataset and testing dataset both indicate the optimal clusters for separation, we will run the **find_optimal_clusters** function we implemented previously on the testing dataset as well:

1. `test_cluster_labels = model.predict(X_test_scaled)`
2. `test_clusters, _, _, _ = find_optimal_clusters(X=X_test_scaled,`
3. `n_clusters=10,`
4. `model=model,`
5. `visualize=False)`

The majority cluster label is three in the testing set as well, meaning that both of our models are arriving at similar results.

The next set is to plot silhouette plots for both the training and testing sets to check that the shape of the silhouettes is similar. We can do this in the following steps:

1. Generate a silhouette plot for the training set:

```
silhouette_plot(X_train_scaled, kmeans_labels, clusters,
```

```
silh_colors=['red', 'black', 'navy'],  
title='Training Silhouette Plot')
```

2. This results in the plot shown in *Figure 5.15*:

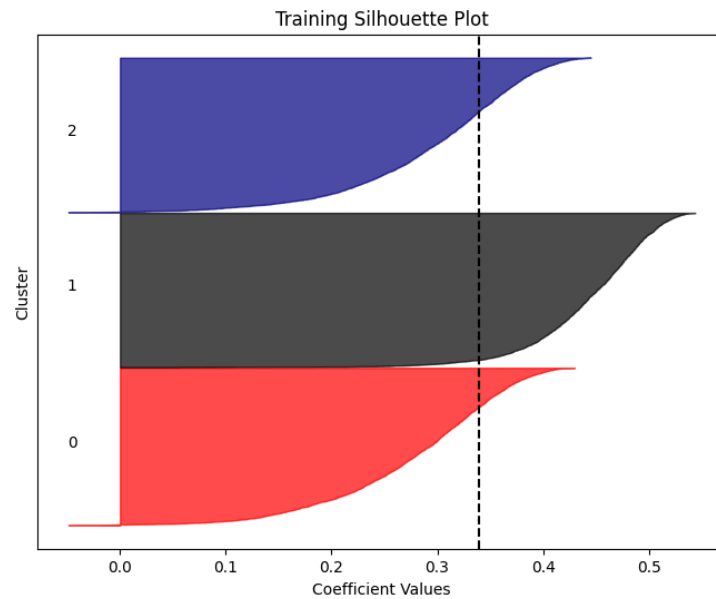


Figure 5.15: Silhouette plot on training dataset

3. Generate a silhouette plot for the testing set:

```
silhouette_plot(X_test_scaled, labels=test_cluster_labels,  
                n_clusters=test_clusters,  
                silh_colors=['red', 'black', 'navy'],  
                title='Testing Silhouette Plot')
```

4. This results in the plot seen in *Figure 5.16*:

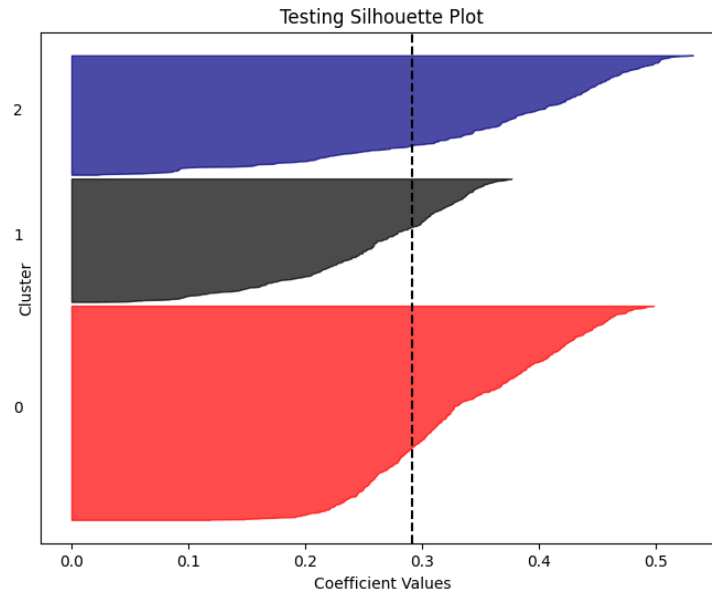


Figure 5.16: Silhouette plot on testing dataset

The silhouette plots for the training and testing data show similar patterns, indicating consistent clustering algorithm performance across both datasets. However, the clustering quality is only moderate, as the average silhouette scores (around 0.3–0.4) suggested. This moderate quality may be due to poor cluster separation and overlapping points, which indicate the algorithm might not be the best fit for the dataset.

We have included a Jupyter Notebook in the supporting GitHub repository that explores other clustering techniques to address this. Some of these methods may provide better results and improve cluster cohesion and separation for this problem.

Conclusion

Unsupervised learning offers a powerful way to uncover patterns and structures in data without relying on predefined labels. By exploring clustering and dimensionality reduction techniques, such as K-means, agglomerative clustering, and DBSCAN, we can segment data into meaningful groups, even in cases where the underlying relationships are not immediately apparent. Each algorithm has unique strengths, from K-means' simplicity and efficiency to DBSCAN's ability to handle noise and non-

linear structures. Throughout this chapter, we have demonstrated the practical implementation of these techniques using Python's robust libraries, showcasing how they can be applied to real-world scenarios like customer segmentation and high-dimensional data visualization.

Clustering, however, is not without challenges. As highlighted in the evaluation of the K-means algorithm, issues like moderate silhouette scores and overlapping clusters can indicate limitations in the chosen method for a given dataset. These results underline the importance of thorough evaluation, including using metrics such as silhouette scores and the Davies-Bouldin index, to assess clustering performance. Furthermore, leveraging advanced techniques or exploring alternative clustering algorithms, as detailed in the accompanying Jupyter Notebook, can often yield better results. By iteratively refining our approach, practitioners can ensure that clustering solutions are robust, interpretable, and aligned with the specific needs of their problem.

In the next chapter, we will explore the fundamental principles and advanced techniques of regression analysis, beginning with the basics of linear regression and progressing through essential topics such as data preprocessing, feature selection, regularization, non-linear models, and evaluation metrics. To consolidate these concepts, we will conclude with a practical case study focused on predicting housing prices.

Points to remember

Here are some key points summed up from this chapter:

- **Addressing cluster quality:** Moderate silhouette scores or overlapping clusters may suggest the need for a different algorithm or better feature engineering.
- **Choosing the right clustering algorithm:** Different algorithms work better for specific datasets—K-means is efficient for spherical clusters, while DBSCAN handles noise and irregular shapes effectively.

- **Cluster evaluation metrics:** Silhouette score, Davies-Bouldin index, and Calinski-Harabasz score help evaluate cluster quality by measuring separation and cohesion.
- **Clustering techniques:** Algorithms like K-means, agglomerative clustering, and DBSCAN each excel in different scenarios, such as compact groups, hierarchical relationships, or non-linear structures.
- **Dimensionality reduction:** PCA reduces high-dimensional data to key components, simplifying visualization and improving clustering efficiency.
- **Feature engineering:** Preprocessing steps, such as one-hot encoding categorical variables and scaling numerical features, are crucial for clustering algorithms like K-means, which rely on distance metrics.
- **Hyperparameter tuning:** Parameters like the number of clusters in K-means (K), epsilon, and min_samples in DBSCAN, and linkage criteria in hierarchical clustering significantly affect model performance.
- **Iterative improvement:** Experimenting with multiple clustering techniques, evaluating performance, and refining hyperparameters ensures robust and meaningful clusters.
- **Train-test split in unsupervised learning:** Splitting data for training and testing allows validation of cluster consistency and prevents overfitting.

Multiple choice questions

1. **What is the primary goal of unsupervised learning?**
 - a. Discovering relationships in labelled data
 - b. Optimizing hyperparameters for supervised models
 - c. Uncovering patterns and structures in unlabeled data

- d. Model learns to reward positive outcomes and penalize negative outcomes

2. Which of the following is a key feature of PCA?

- a. Grouping data points into clusters based on similarity
- b. Reducing high-dimensional data to fewer dimensions while retaining variance
- c. Assigning data points to their nearest cluster centroid
- d. Creating dendrograms for hierarchical clustering

3. In K-means clustering, what is the role of the centroid?

- a. It defines the boundaries of each cluster
- b. It minimizes between-cluster variance
- c. It calculates the distance matrix
- d. It represents the average position of all data points in the cluster

4. What metric is commonly used to determine the quality of clusters in K-means?

- a. Mean squared error
- b. Categorical cross entropy
- c. Silhouette score
- d. Root mean squared error

5. Which clustering algorithm is best suited for datasets with noise and clusters of arbitrary shapes?

- a. DBSCAN
- b. K-means
- c. Agglomerative clustering
- d. Fuzzy sphere partitioning

6. **What is the key advantage of agglomerative clustering compared to K-means?**
 - a. It uses centroids to represent clusters
 - b. It does not require the number of clusters to be specified beforehand
 - c. It minimizes within-cluster variance
 - d. It eliminates outliers during clustering
7. **What does a silhouette plot visually represent?**
 - a. The distance between cluster centroids
 - b. The optimal number of clusters based on within-cluster variance
 - c. The silhouette scores of individual data points, indicating cluster cohesion and separation
 - d. The hierarchical structure of clusters in a dendrogram
8. **What is the primary function of the elbow method?**
 - a. Identifying the optimal number of clusters by analyzing within-cluster variance
 - b. Determining the distance metric for clustering
 - c. Dimensionality reduction
 - d. Assigning points to an elbow near the centroid of the cluster
9. **Which of the following metrics evaluates clustering by balancing within-cluster cohesion and between-cluster separation?**
 - a. Silhouette score
 - b. Calinski-Harabasz score
 - c. Log loss
 - d. Root mean clustering error

10. **Why is feature scaling essential for clustering algorithms like K-means?**

- a. To balance class frequencies during training
- b. To reduce noise in high-dimensional data
- c. To prevent features with larger ranges from dominating the distance calculations
- d. To identify the optimal number of clusters

Answers

1	c
2	b
3	d
4	c
5	a
6	b
7	c
8	a
9	b
10	c

Key terms

Some key terms used in this chapter are:

- **Agglomerative clustering:** A hierarchical clustering algorithm that builds clusters by iteratively merging smaller clusters based on a distance metric.
- **Calinski-Harabasz score:** A clustering evaluation metric that assesses the ratio of between-cluster dispersion to within-cluster dispersion.
- **Centroid:** The central point of a cluster, representing the mean position of all points within the cluster.

- **Cluster cohesion:** The degree to which data points within a cluster are similar to each other.
- **Cluster separation:** The degree to which clusters are distinct from one another.
- **Cluster inertia:** A measure of how tightly data points are grouped within clusters, used to evaluate K-means performance.
- **Cluster variance:** The spread of data points within a cluster, which affects clustering quality.
- **Clustering algorithm:** An unsupervised learning method that groups similar data points based on defined criteria.
- **DBSCAN:** Clustering algorithm that groups data points based on density, effectively handling noise and clusters of arbitrary shapes.
- **Dimensionality reduction:** Techniques like PCA that reduce the number of features while preserving the essential structure of the data.
- **Dendrogram:** A tree-like diagram used to visualize hierarchical clustering and the merging of clusters.
- **Distance metric:** A mathematical function used to measure the similarity between data points in clustering algorithms (e.g., Euclidean, Manhattan, cosine similarity).
- **Elbow method:** A technique for determining the optimal number of clusters by analyzing the rate of decrease in within-cluster variance.
- **Euclidean distance:** A common distance metric used in clustering to measure the straight-line distance between two points.
- **Feature engineering:** The process of transforming raw data into meaningful input features for machine learning models.

- **Feature scaling:** A preprocessing step that standardizes feature values to ensure fair contributions to distance-based clustering methods.
- **Hierarchical clustering:** A clustering method that builds a hierarchy of clusters using either a bottom-up (agglomerative) or top-down (divisive) approach.
- **Hyperparameter tuning:** The process of optimizing clustering algorithm parameters (e.g., K in K-means, epsilon in DBSCAN) for improved performance.
- **K-means clustering:** A partitional clustering algorithm that groups data into K clusters based on distance to cluster centroids.
- **K-means++:** A smarter initialization technique for K-means that selects starting centroids to improve convergence.
- **Linkage criterion:** A method in hierarchical clustering for measuring distances between clusters, such as single, complete, or average linkage.
- **Manhattan distance:** A distance metric that calculates the sum of absolute differences between feature values.
- **Minibatch K-means:** A variation of K-means that improves efficiency by using small random subsets of data to update centroids.
- **Noise points:** Data points in DBSCAN that do not belong to any cluster and are considered outliers.
- **Outlier detection:** Identifying data points that do not fit well within any cluster and may indicate anomalies.
- **PCA:** A dimensionality reduction technique that transforms high-dimensional data into fewer components.
- **Silhouette plot:** A visualization tool that shows the silhouette scores of data points, helping assess cluster cohesion and

separation.

- **Silhouette score:** A metric for evaluating clustering quality by measuring how well a data point fits within its cluster compared to neighboring clusters.
- **Ward's method:** A linkage criterion in hierarchical clustering that minimizes variance within merged clusters.

Join our Discord space

Join our Discord workspace for latest updates, offers, tech happenings around the world, new releases, and sessions with the authors:

<https://discord.bpbonline.com>



CHAPTER 6

House Price Estimation with Regression Methods

Introduction

Estimating house prices is a fundamental problem in real estate, finance, and urban planning. Accurately predicting property values helps homeowners, investors, and policymakers make informed decisions. In this chapter, we explore regression techniques, a powerful set of tools used to model relationships between variables and generate price predictions based on key property attributes. Beginning with linear regression, we will examine how **ordinary least squares (OLS)** estimates the relationship between features such as square footage, number of bedrooms, and location with house prices. We will also introduce multiple linear regression, feature selection strategies, and methods to refine our models for better predictive performance.

Beyond traditional linear models, we will delve into advanced regression techniques, including regularization methods like Ridge and Lasso, which help mitigate overfitting, and non-linear approaches such as random forest and gradient boosting for capturing complex market dynamics. To evaluate model effectiveness, we will discuss key metrics like **mean absolute error (MAE)** and **root mean squared error (RMSE)**. The chapter culminates in a hands-on project using real-world housing data, where we will apply these techniques to predict house prices, assess model accuracy, and gain insights into feature importance.

Structure

This chapter covers the following topics:

- Introduction to regression analysis and linear regression
- Feature selection and recursive feature elimination for regression
- Regularization in regression
- Non-linear regression methods
- Regression evaluation metrics
- Predicting house prices project

Objectives

By the end of this chapter, you will have a solid understanding of regression techniques for estimating house prices, including both linear and non-linear methods. You will learn how to select relevant features, apply regularization to prevent overfitting, and evaluate model performance using key metrics. Additionally, you will gain hands-on experience building and optimizing regression models, culminating in a real-world project where you predict house prices and analyze feature importance.

Introduction to regression analysis and linear regression

Regression analysis is a key technique in machine learning and statistics used to model relationships between variables and make predictions based on continuous data. This chapter begins with OLS regression, the most common method for estimating coefficients in a linear model, followed by multiple linear regression, which incorporates multiple predictor variables for improved accuracy. We will also discuss essential data preprocessing techniques, such as feature scaling, handling missing values, and data transformations, to enhance model performance.

Beyond linear regression, we will explore feature selection methods like **recursive feature elimination (RFE)** and regularization techniques such as

Lasso and Ridge regression to mitigate overfitting. Non-linear regression approaches, including random forest and gradient boosting, will also be introduced for capturing complex patterns. To assess model performance, we will cover key evaluation metrics such as MAE and RMSE. The chapter concludes with a real-world case study on housing price prediction, demonstrating the practical application of regression techniques in data-driven decision-making.

Ordinary least squares regression

OLS is one of the most fundamental techniques in regression analysis, used to estimate the relationship between independent variables (predictors) and a dependent variable (outcome). The goal of OLS is to find the best-fitting line that minimizes the sum of squared differences (which we will discuss in more detail later) between the actual values (x_i) and the predicted values (also known as $\hat{y}$). This method assumes a linear relationship between variables and helps quantify the impact of predictors on the target variable.

The regression formula for OLS is:

$$y = \beta_0 + \beta_1 x + \epsilon$$

For an explanation of this equation and its terms, refer to the following:

- y is the dependent variable (target variable to be predicted), i.e., price of the house.
- x is the independent variable (predictor), i.e., number of bedrooms in house.
- B_0 is the intercept term (the value of y when $x = 0$).
- B_1 is the coefficient of the independent variable, representing its influence on the change in y ; another way would be how much the number of bedrooms in the house influences the change in the actual house price, on average.

- Error term (ϵ) is residual, which represents all the other variance that has not been accounted for by our model. A way to interpret this would be everything else, other than the number of bedrooms, that causes a house price to vary.

The primary objective of OLS is to estimate the values of B_0 and B_1 that minimize the sum of squared residuals. Think of this process as predicting the scores of multiple football matches. After making your predictions, you wait for the actual results and then compare them to your estimates.

The difference between your prediction and the actual result is the residual. To ensure that positive and negative errors do not cancel each other out, you square each residual and then sum them all up, resulting in the sum of squared residuals. A smaller value indicates that your predictions were more accurate, while an enormous value suggests a more significant deviation from the actual outcomes. The equation for this process is as follows:

$$\sum (y_i - \hat{y}_i)^2$$

In the next section, we will implement the OLS process in Python code.

OLS in action

Now that we have the theory, we will implement this in Python, with scikit-learn, in a couple of easy steps:

1. Import the packages needed for making the regression dataset, performing modeling, and then evaluation:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.datasets import make_regression
```

```
from sklearn.metrics import mean_squared_error, r2_score
```

2. We will now synthetically generate 100 data points with one feature using the **make_regression** function:

```
X, y = make_regression(n_samples=100,  
                       n_features=1,  
                       noise=5, random_state=42,  
                       bias=7)
```

3. Then we initialize the model for fitting our OLS equation:

```
model = LinearRegression()
```

4. Next, we fit our initialized model to the regression dataset we generated in *Step 2*:

```
model.fit(X, y)
```

5. Now that we have fitted our OLS model, we want to get our **y** predictions:

```
y_pred = model.predict(X)
```

6. To get our sum of squared residuals, we can take the sum of our **y** minus our **y_pred** squared to achieve this, as we have discussed in the sum of squared residuals equation:

```
ssr = np.sum((y - y_pred) ** 2)
```

7. Next, we will expose the intercept term, as previously explained:

```
print(f"Intercept: {model.intercept_}")
```

8. To get the model coefficients, these can be exposed by using the **model.coef_** function:

```
print(f"Coefficients: {model.coef_}")
```

9. Finally, we will visualize how our model's line of best fit, determined by minimizing the sum of squared residuals, has performed and assess the deviation of our predictions from this optimal regression line:


```
plt.scatter(X, y, label="Data", alpha=0.7, color='black')  
plt.plot(X, y_pred, color="grey",label="Regression (OLS) Line")  
plt.title("Linear Regression with positive correlation")  
plt.xlabel("Independent variables (IVs)")  
plt.ylabel("Dependent variable (DV)")  
plt.legend()  
plt.show()
```

10. The output of the last step is our fitted line of best fit (our regression line) on our data points:

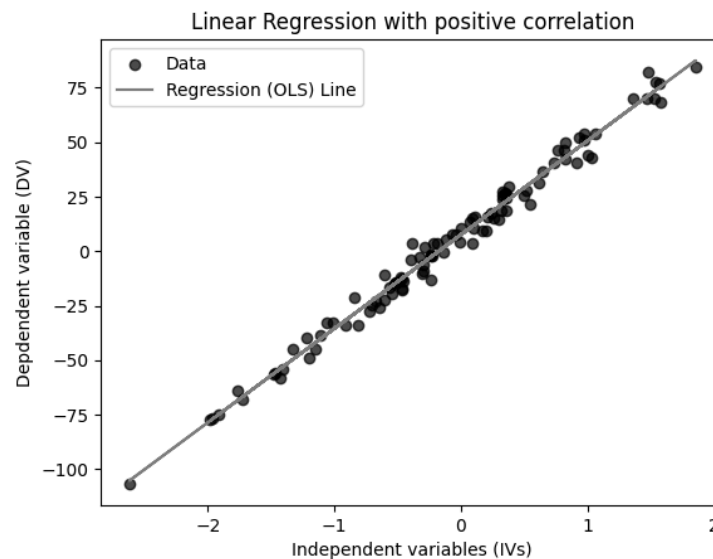


Figure 6.1: Line of best fit

Now that we have generated data, trained our model, calculated relevant metrics, and visualized the line of best fit, the next step is to use this model to make predictions on unseen data. The line of best fit represents the general relationship between the input features and the target variable based on patterns learned from the training data.

When new, unseen data is introduced, the model uses this learned relationship to estimate an outcome. The accuracy of these predictions depends on how similar the new data is to the training set. If the latest data

falls within a similar range, the predictions are likely to be reliable. However, if the new data differs significantly from what the model has seen before, the predictions may be less accurate. The line of best fit allows us to make informed estimates by comparing new inputs to past trends observed in the data.

Multiple linear regression

Multiple linear regression is an extension of OLS (simple linear regression). Instead of using just one feature (independent variable) to predict an outcome, the equation considers multiple features that might influence the dependent variable.

Put simply, when we are trying to predict the price of a house, previously, we just used the feature number of bedrooms. Square footage, age of property, and a whole host of variables will also be responsible for determining the price of a house. Regression analysis aims to find the variables that cause this change and expunge those that do not; we will get on to recursive feature elimination later in this chapter.

The equation for multiple linear regression is:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$$

Let us understand this in detail:

- y is the dependent variable (the thing you are trying to predict, i.e., house price value).
- $x_1, x_2, \dots, x_n$ are the independent (feature) variables, i.e., square footage of house, number of bedrooms, etc.
- B_0 is the intercept (the value of y when all x values are 0).
- $\beta_1, \beta_2, \dots, \beta_n$ are the coefficients for each independent variable.
- ϵ is the error term.

Unlike simple linear regression, multiple regression considers several predictors simultaneously. The coefficients are estimated using matrix algebra.

The model fitting process follows the same steps as in the OLS example; the key difference is that we now use four features instead of one. As a result, the model will generate four coefficients rather than one. A detailed implementation of this process is available in the supporting GitHub repository as a Jupyter Notebook.

We will return to regression and regression analysis when analyzing our model results as part of the primary use case in this chapter.

Feature selection and recursive feature elimination for regression

Feature selection is a crucial step in regression modeling, helping improve accuracy, reduce overfitting, and enhance interpretability. By eliminating irrelevant or redundant variables, we create simpler models that generalize better to new data.

In regression, feature selection is particularly useful when dealing with:

- **Multicollinearity:** When independent variables are highly correlated, making coefficient estimates unstable.
- **High-dimensional data:** Too many features increase the risk of overfitting.
- **Irrelevant predictors:** Features with little influence on the target variable add noise to the model.

We will explore two main techniques for this: **Recursive feature elimination (RFE)** and **recursive feature elimination with cross-validation (RFECV)**.

Recursive feature elimination

RFE is a wrapper method that selects the most relevant features by iteratively removing the least important ones. It trains a regression model, ranks feature importance, eliminates the weakest feature, and repeats this process until a predefined number of features remain.

It follows these steps:

1. Train a regression model on all available features.
2. Rank feature importance based on model coefficients.
3. Remove the least important feature and retain the model.
4. Repeat the process until the optimal number of features is selected, as shown in *Figure 6.2*:

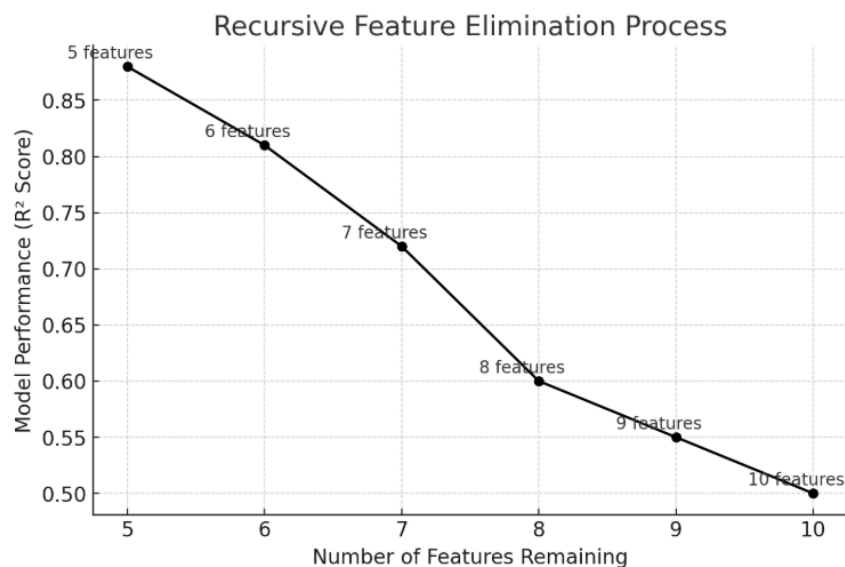


Figure 6.2: Recursive feature elimination process

In the following steps, we will implement RFE on a synthetic dataset; this will prepare you to use this on our house price dataset in the deep dive section:

1. Define our imports:

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression
from sklearn.feature_selection import RFE
```

2. Generate dataset using **make_regression** function:

```
X, y = make_regression(n_samples=100,
                       n_features=10,
                       noise=10,
                       random_state=42)
```

3. Initialize our model:

```
model = LinearRegression()
```

4. Use the **RFE** function to pass in the model and the number of features to select; in this case, retain five features from the original ten:

```
rfe = RFE(estimator=model,
           n_features_to_select=5)
```

5. Fit the recursive feature elimination function to the data:

```
rfe.fit(X, y)
```

6. Use **np.where()** to find the features utilizing the **.support** argument:

```
selected_features = np.where(rfe.support_)[0]
print(f"Selected Features: {selected_features}")
```

This outputs: **Selected Features: [3 4 5 6 9]**

7. Use the **rfe.ranking_** argument to rank all the features:

```
print(f"Feature Rankings: {rfe.ranking_}")
```

This returns the following: **Feature Rankings: [3 2 5 1 1 1 1 4 6 1]**

8. Select the columns by index and fit the model with the reduced set of features:

```
X_selected = X[:, selected_features]
model.fit(X_selected, y)
```

9. To print the model coefficients and intercept, use the following code:

```
print(f"Model Coefficients: {model.coef_}")
print(f"Model Intercept: {model.intercept_}")
```

10. Finally, to visualize the ranking scheme, we will build a function that you can pass in the rank from our RFE process, and it will visualize the retained and eliminated features:

```
def visualize_rfe(rfe_rank,
                  title='Feature Importance Ranking',
                  x_lab='Feature Index',
                  y_lab='Ranking',
                  retained_color='grey',
                  eliminated_color='black'):
    plt.figure(figsize=(8, 5))
    colors = np.array([retained_color if rank == 1 else
                       eliminated_color for rank in rfe_rank])
    plt.bar(range(len(rfe_rank)), rfe_rank, color=colors)
    plt.xlabel(x_lab)
    plt.ylabel(y_lab)
    plt.title(title)
    plt.show()
```

11. To use this function, we use the function:

```
visualize_rfe(rfe_rank=rfe.ranking_)
```

This will produce the visualization as shown in *Figure 6.3*:

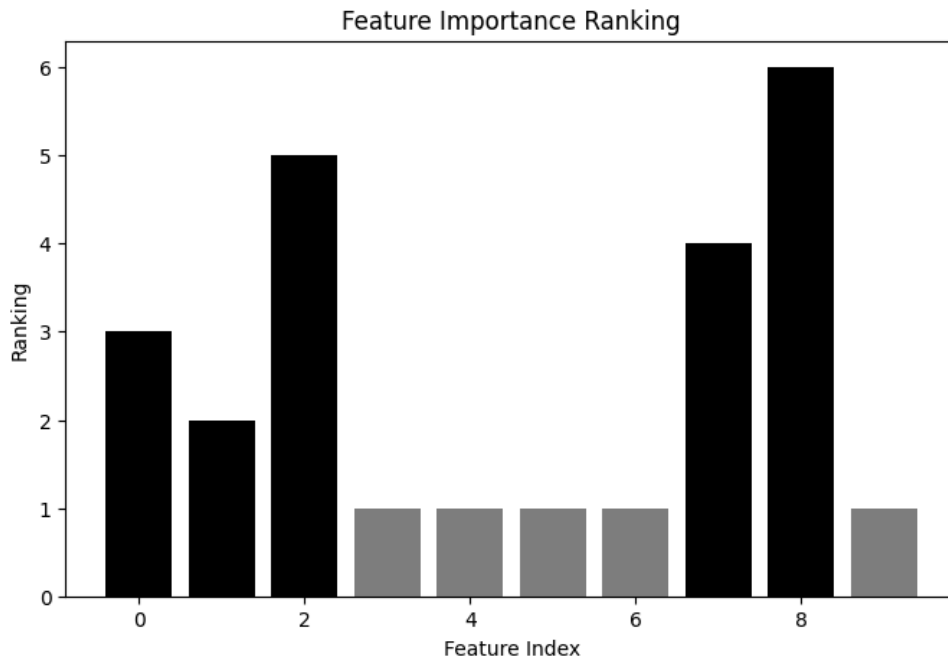


Figure 6.3: Recursive feature elimination rank chart

Automating feature selection

While basic RFE requires us to specify the number of features, RFECV automatically determines the optimal number by evaluating model performance at each step. Here, our model performance is measured using the R2 formula; we will discuss this further in the section on model performance for regression models.

To implement RFECV in Python, we follow these steps:

1. Import the necessary packages:

```
from sklearn.feature_selection import RFECV
```

```
from sklearn.model_selection import cross_val_score
```

2. Initialize the RFECV packages to use 5-fold cross-validation and optimize the regression metric R-squared (we will get on to this in

the evaluation metrics section):

```
rfecv = RFECV(estimator=LinearRegression(),  
              cv=5,  
              scoring="r2")
```

3. Fit the feature elimination method to the data:

```
rfecv.fit(X, y)
```

4. Extract the mean cross-validation scores with the R-squared value:

```
cv_scores = rfecv.cv_results_["mean_test_score"]
```

5. Create a plot to show the optimal feature selection based on the target metrics R-squared using the following code:

```
plt.figure(figsize=(8, 5))  
plt.plot(range(1, len(cv_scores) + 1),  
         cv_scores, marker='o',  
         linestyle='dashed',  
         color='black')  
  
plt.xlabel("Number of Features Selected")  
plt.ylabel("Cross-Validated R2 Score")  
plt.title("Optimal Feature Selection with RFECV")  
plt.grid(True, linestyle="--", alpha=0.6)  
plt.show()
```

6. This creates a plot, as shown in *Figure 6.4*, where the R-squared value plateaus at six selected features, and there is minimal predictive power by adding the additional four features:

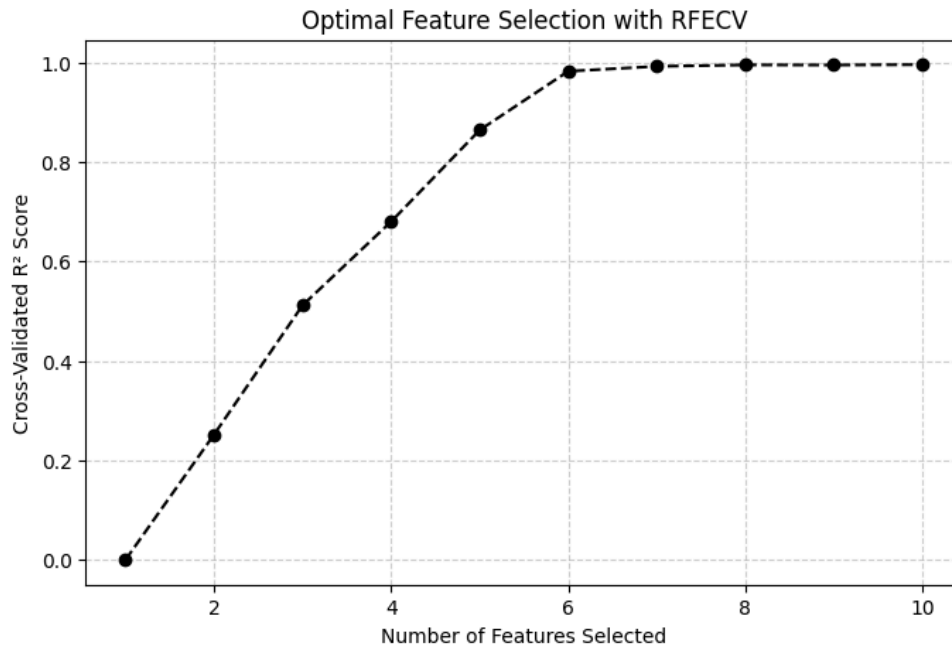


Figure 6.4: RFECV feature elimination

Regularization in regression

Regression models are essential in statistical analysis and machine learning when it comes to predicting continuous outcomes. However, when dealing with complex datasets, overfitting can be a significant issue (where your model learns the training data so well that it struggles to make predictions in the wild).

Regularization techniques such as **Least Absolute Shrinkage and Selection Operator (Lasso)**, Ridge regression, and Elastic Net help prevent overfitting by introducing penalties to the model coefficients (for example, with our use case predicting house prices, the coefficient for square footage would tell you how much one additional square foot would increase the overall price).

Here is a detailed look at why you would want to use these methods:

- **Overfitting prevention:** Regularization techniques help prevent models from fitting noise in the data, improving generalization.

- **Feature selection (Lasso):** Linked to the previous section, we can use Lasso regression for feature selection, as the aim of this model is to shrink some coefficients to exactly zero (in our use case, the feature would have no impact on house price and would therefore be expunged).
- **Collinearity handling (Ridge and Elastic Net):** When predictors are highly correlated, Ridge and Elastic Net help reduce variance by distributing weight among correlated features.
- **Balance between Ridge and Lasso (Elastic Net):** Elastic Net combines both Lasso and Ridge penalties, providing flexibility in handling multicollinearity while performing feature selection.

As we learned earlier, the goal of regression models is to minimize the models' effects on the sum of squared residuals.

This method still aims to do that, but the difference is that these methods introduce additional constraints, as you will learn in the next sections.

Ridge regression

Ridge regression, otherwise known as L2 regularization, adds an L2 penalty (sum of squared coefficients), reducing the magnitude of coefficients (their impact) but keeping all predictors in the model. The Ridge regression cost function is as follows:

$$\text{Minimize } \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p \beta_j^2$$

Let us understand this in detail:

- λ (lambda) is the regularization parameter that controls the strength of the penalty.
- The penalty term helps reduce the large coefficient value, preventing overfitting while maintaining all predictors (no feature

elimination with this method).

These models will be implemented during the practical hands-on session of building a regression model to predict house prices.

Lasso regression

Lasso regression applies a L1 penalty (otherwise known as L1 regression), shrinking some coefficients to zero, which effectively acts as a feature selection technique by only selecting the most important features for the prediction problem and removing the irrelevant features. The Lasso regression cost function (again aimed at minimizing the sum of squared residuals) is given here:

$$\text{Minimize } \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p |\beta_j|$$

Let us understand this in detail:

- λ (lambda) controls the strength of the penalty on the feature, i.e., influencing how many features are retained.
- The absolute value in the penalty term ensures sparsity in the feature set by forcing some of the coefficients to become exactly zero.

Elastic Net regression

Elastic Net regression combines the L1 and L2 penalties, benefiting from Ridge's ability to handle multicollinearity (due to feature weights being made weaker through the lambda regularization term) and Lasso's feature selection capabilities. The cost function for Elastic Net is similar to the previous two equations, but we have two lambdas being added to the squared residuals:

$$\text{Minimize } \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda_1 \sum_{j=1}^p |\beta_j| + \lambda_2 \sum_{j=1}^p \beta_j^2$$

Let us understand this in detail:

- λ_1 controls the L1 penalty (sparsity effect).
- λ_2 controls the L2 penalty (weight shrinkage).
- The balance between Lasso and Ridge is determined by the ratio between the terms.

These techniques are fundamental tools for improving model performance, as we will see later in this chapter when we fit these models for our use case.

Non-linear regression methods

In this section, we will explore various non-linear methods, with a particular focus on two key algorithms, which are random forests and gradient boosting.

While there are many other techniques, such as support vector machines (<https://scikit-learn.org/stable/modules/svm.html>) and Naïve Bayes (https://scikit-learn.org/stable/modules/naive_bayes.html), we will concentrate on these two for brevity.

Random forests

Random forests are a powerful ensemble learning method used for both classification and regression tasks. They create multiple decision trees during training and aggregate their predictions to improve accuracy and reduce overfitting. Follow these steps for the construction of random forest:

1. **Bootstrap sampling:** Random forests generate multiple subsets of data (with replacement) from the original dataset. Each subset is used to train an individual decision tree.

2. **Decision trees:** Each tree learns patterns from its respective bootstrap sample by splitting features at different nodes.
3. **Random feature selection:** At each split in a tree, only a random subset of features is considered, which promotes feature diversity across trees.
4. **Prediction aggregation:**
 - a. For **classification** tasks, the final prediction is made through a majority vote across all trees. For example, this could involve predicting whether a patient has heart disease or not.
 - b. For **regression**, the final prediction is the average of all tree outputs; here, it would be the house price averaged across the trees.

The process described is visualized in *Figure 6.5*:

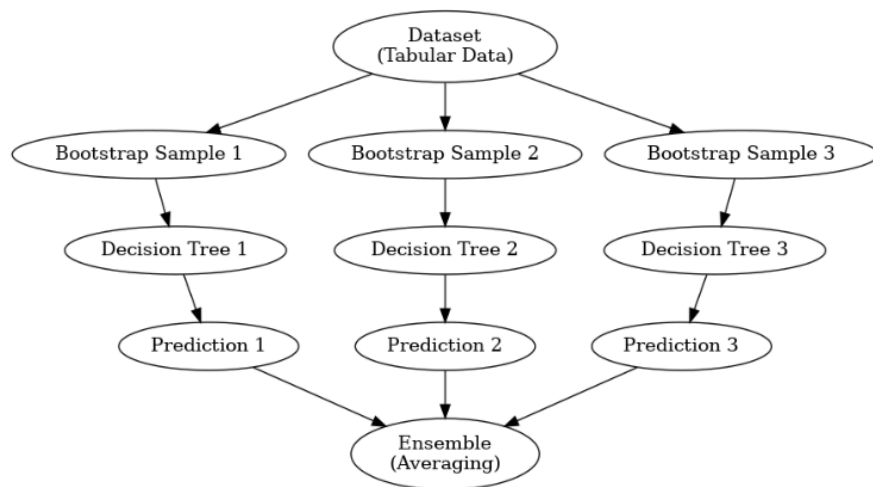


Figure 6.5: Random forest diagram

With classification, the features are split using Gini impurity, but with regression, we use **mean squared error (MSE)** and MAE to determine the best splits. A simple decision tree would look like the example shown in *Figure 6.6*:

Single Decision Tree (Regression)

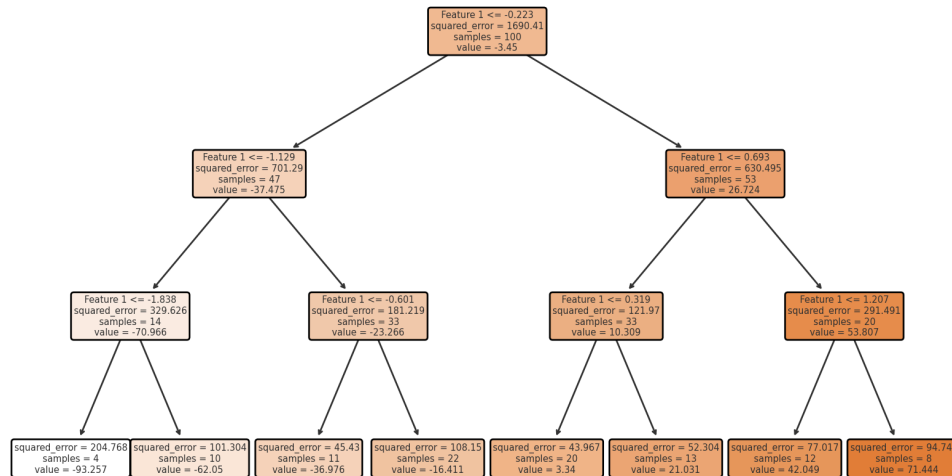


Figure 6.6: Single decision tree

Multiple trees are then grown (in ML, we can produce hundreds of trees and average out their predictions), as shown in *Figure 6.7*:

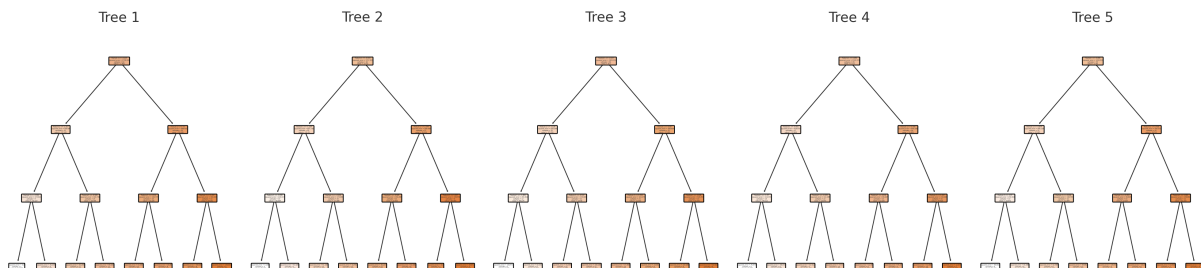


Figure 6.7: Multiple decision trees

As stated previously, the output of each of the trees is a continuous value prediction for regression and a classification for classification. With regression, the predictions are averaged across trees, and that is our estimate per observation. We will fit random forests as part of the Housing Prediction Price Project.

Gradient boosting and XGBoost

Gradient boosting's aim is to build an ensemble of weak learners (normally decision trees) in a sequential manner, where each new tree attempts to

correct the errors made by the previous trees. This iterative approach allows the model to improve and become more accurate over time.

The steps to gradient boosting are detailed here:

1. **Initialize the model:** The first prediction is a simple one, usually the mean (for regression) or log-odds (for classification).
2. **Calculate errors (residuals):** Compute the difference between the true values and the current predictions.
3. **Train a weak learner (decision tree):** A new tree is trained to predict the residuals from *Step 2*.
4. **Update predictions:** The new tree's prediction is added to the existing predictions using a small learning rate (we will cover this in *Chapter 11, Getting Deep with Neural Networks*).
5. **Repeat Steps 2 to 4:** This process continues for a predefined number of iterations or until the error stops improving, as shown in *Figure 6.8*:

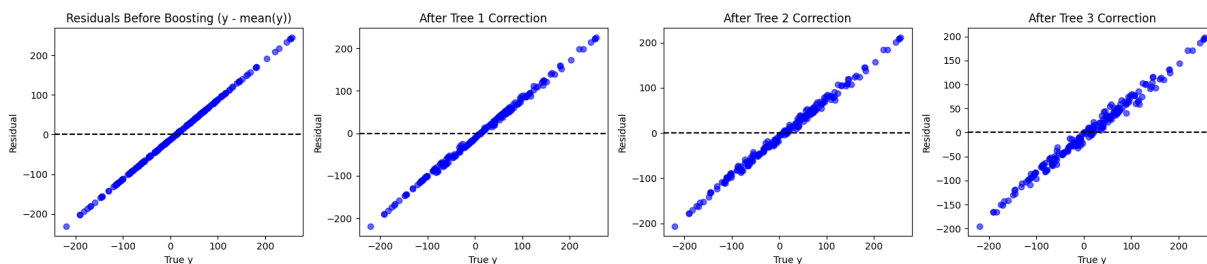


Figure 6.8: Gradient boosting Correction steps

6. **Final prediction:** The final prediction is usually the weighted sum or average of all trees' outputs, depending on the specific implementation and learning rate used. This is shown in *Figure 6.9*:

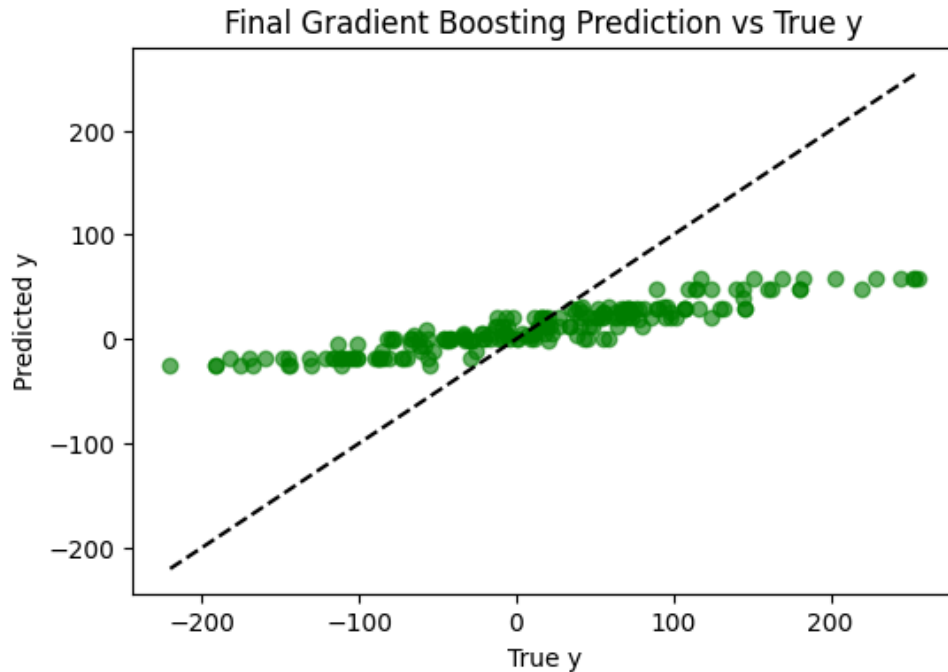


Figure 6.9: Gradient boosting predictions after corrections

The benefits of XGBoost are explained in the following bullets:

- **Extreme gradient boosting (XGBoost)** enhances traditional gradient boosting methods by introducing several key improvements that address performance and efficiency.
- One significant advancement is the inclusion of both L1 (Lasso) and L2 (Ridge) regularization techniques, which help prevent overfitting and improve model generalization, features not present in standard gradient boosting implementations.
- Additionally, XGBoost employs a sophisticated Weighted Quantile Sketch algorithm to identify optimal split points, enabling it to handle large datasets more efficiently.
- Another notable feature of XGBoost is its ability to perform parallel processing. While traditional gradient-boosting methods build trees sequentially, XGBoost constructs trees in parallel, significantly reducing training time on large datasets.

- Furthermore, XGBoost incorporates max-depth pruning, which eliminates splits that do not substantially enhance model performance, thereby reducing unnecessary complexity.
- It also features built-in mechanisms to handle missing data by learning the optimal imputation direction during training, thereby eliminating the need for separate data preprocessing steps.
- Moreover, XGBoost supports GPU acceleration, offering substantial speed improvements when working with extensive datasets.

Regression evaluation metrics

Evaluating regression models is crucial to ensuring that predictions are accurate and reliable. There are several key metrics for this, each providing a distinct perspective on a model's performance. In the following sections, we will review the primary evaluation functions with which we have worked in the industry. A portion of my career was spent at a housing data company, where we predicted house prices using a multitude of features.

Mean absolute error

MAE measures the average magnitude of the errors in a set of predictions without considering their direction. It is calculated as the average of the absolute difference between predicted and actual values:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

The equation says where y_i is the actual value, $\hat{y}_i$ is the predicted value, and n is the number of observations. MAE is relatively easy to interpret and provides a linear score, meaning all individual differences are weighted equally in the average.

Mean squared error

MSE calculates the average of the squared differences between predicted and actual values:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

By squaring the errors, MSE penalizes larger errors more than smaller ones, making it sensitive to outliers. It is useful when large errors are particularly undesirable.

Root mean squared error

This is the square root of the MSE:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

This metric provides the error in the same units as the target variable, say RMSE house price, which makes it more interpretable; however, it is still sensitive to outliers.

Mean squared logarithmic error

Mean squared logarithmic error (MSLE) measures the ratio between the true and predicted values by calculating the squared logarithmic differences:

$$\text{MSLE} = \frac{1}{n} \sum_{i=1}^n (\log(y_i + 1) - \log(\hat{y}_i + 1))^2$$

This is useful when the target variable spans several orders of magnitude, as it penalizes underestimations more than overestimations.

R-squared

R^2 , the coefficient of determination, indicates the proportion of the variance in the dependent variable that is predictable from the independent variables,

i.e., do our features cause our target to vary:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

An R^2 value closer to 1 suggests a better fit, while a value closer to 0 indicates that the model does not explain much of the variance.

Adjusted R-squared

Adjusted R^2 modifies the R^2 value to account for the number of predictors in the model, providing a more accurate measure when multiple variables are involved, as you can improve R^2 by simply adding more features:

$$\text{Adjusted } R^2 = 1 - (1 - R^2) \times \frac{n-1}{n-p-1}$$

Where p is the number of predictors, this metric adjusts for the addition of variables and only increases if the new variable improves the model more than would be expected by chance, thus safeguarding against adding predictors for the sake of gaming the R^2 value.

Mean absolute percentage error

Mean absolute percentage error (MAPE) expresses accuracy as a percentage, measuring the average absolute percentage difference between predicted and actual values:

$$\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

This metric is useful for understanding the relative magnitude of errors. However, it can be sensitive to values close to zero, leading to extremely large percentage errors. We will revisit these metrics when we come to *Chapter 8, Time Series with Scikit-learn*, as they use the same metrics as regression.

Metrics function

The **compute_metrics** function calculates the evaluation metrics we have mentioned in this section and can be reused for your projects in the wild.

To define the function, we will use the following Python code:

```
1. from sklearn.metrics import (  
2.     mean_squared_error, mean_absolute_error,  
3.     r2_score, mean_squared_log_error)  
4. import numpy as np  
5. def compute_metrics(y_true, y_pred):  
6.     metrics = {}  
7.     metrics['MSE'] = mean_squared_error(y_true, y_pred)  
8.     metrics['RMSE'] = np.sqrt(metrics['MSE'])  
9.     metrics['MAE'] = mean_absolute_error(y_true, y_pred)  
10.    metrics['R2'] = r2_score(y_true, y_pred)  
11.    metrics['MSLE'] = (  
12.        mean_squared_log_error(y_true, y_pred)  
13.        if np.all(y_pred >= 0) else None  
14.    )  
15.    metrics['MAPE(%)'] = (  
16.        np.mean(np.abs((y_true - y_pred) / y_true)) * 100  
17.        if np.all(y_true != 0) else None  
18.    )  
19.  
20.    return metrics
```

We will use this function in our hands-on section.

Predicting house prices project

In this project, we will use the famous house price prediction project that has been made famous by Kaggle (home of the best ML competitions). This comes already separated into a training and testing set, so we will use these instead of splitting on this occasion, and a download of this data is already in the supporting GitHub repository for this book and chapter.

Importing and inspecting data

The first step is to do some exploration on the house prices dataset and make sure it is in the relevant format required for modeling.

We will follow these steps to import our data:

1. Gather the required imports needed to work with our data in Python:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

2. Read the data using **read_csv** from Pandas:

```
train = pd.read_csv('data/train.csv')
test = pd.read_csv('data/test.csv')
```

3. Check the length of the features where we have integer or floating-point numbers, essentially the numerical data; this will show us we have 38 features:

```
len(train.describe(include=['int', 'float']).columns)
```

4. Check the length of the features where we have categorical data (object data):

```
len(train.describe(include=['object']).columns)
```

5. We have 43 features of a categorical type.

Now that we have imported our data, which contains 81 features, we will then look to create a function to allow you to visualize different relationships on a scatter chart from a linear perspective; we will look at that in the next section.

Visualizing relationships

We are going to build a scatter chart since it is a great way to visualize relationships; it clearly highlights correlations, trends, and outliers between two numerical variables. However, it only gives a snapshot of how one variable relates to another. We could also use correlation matrices for this, but we covered those in detail earlier.

To produce and visualize our scatter chart, we will do so in the following steps:

1. Create a function, using Seaborn, that creates a **regplot** (regression plot) between our two features. There is logic built into the function to check that the columns passed to the function contain numeric and continuous data types:

```
def scatter_correlation(df, x_col,
                        y_col,
                        title=None,
                        size=8, line_color='red'):
    if x_col not in df.columns or y_col not in df.columns:
        raise ValueError(f"Columns '{x_col}' or '{y_col}' not found in DataFrame")
    if not pd.api.types.is_numeric_dtype(df[x_col]) or not pd.api.types.is_numeric_dtype(df[y_col]):
        raise ValueError("Both x_col and y_col must be numeric")
    plt.figure(figsize=(10, 7))
    sns.regplot(
```

```

        x=df[x_col],
        y=df[y_col],
        scatter_kws={'s': size},
        line_kws={'color': line_color},
        ci=None
    )
    plt.xlabel(x_col)
    plt.ylabel(y_col)
    plt.title(title if title else f"{y_col} vs {x_col}")
    plt.show()

```

2. We will use this on our training dataset and use the **LotFrontage** (how big the front of the property is) and the **SalePrice** (our variable we are going to want to predict):

```

scatter_correlation(df=train,
                    x_col="LotFrontage",
                    y_col="SalePrice")

```

3. This produces the visualization as shown in *Figure 6.10*, showing the relationship between the two variables. There are a few noticeable outliers in this chart, so we will need to handle this later. When it comes to our choice of normalization technique, we use the following:

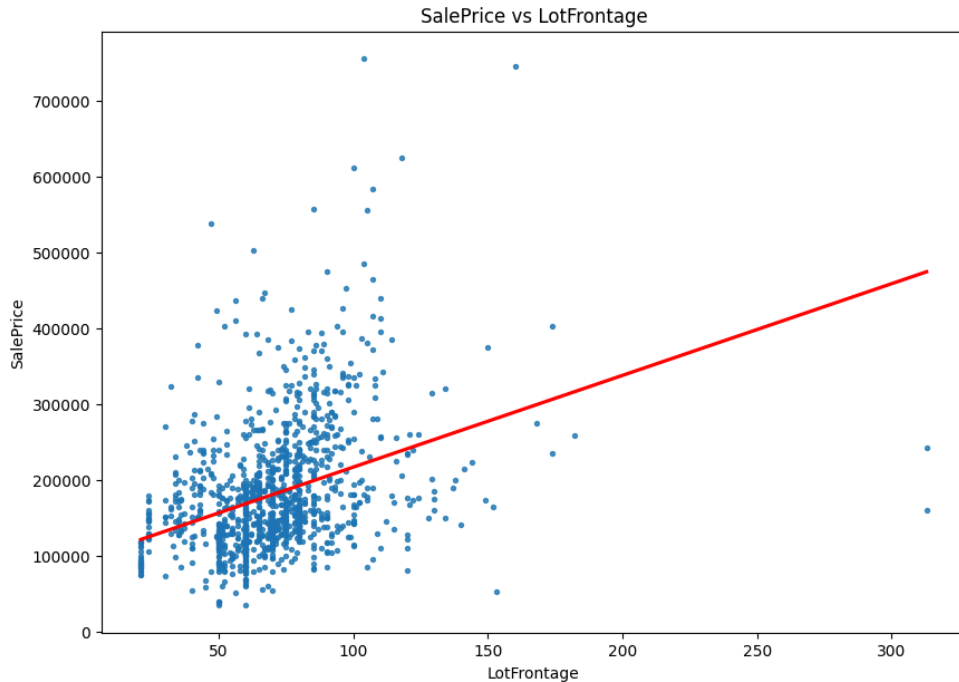


Figure 6.10: Scatter chart showing the relationship

In the next step, we will drop unnecessary features and create a validation sample, which may be used in our training loop that we will create.

Analyzing missing values and dropping features

We will drop the **Id** and **SalePrice** columns from our feature set (**X**), and then we will retain the **SalePrice** for the **y_train** set:

1. `X_train = train.drop(columns=['Id', 'SalePrice'])`
2. `y_train = train['SalePrice'].copy()`

Previously, we defined a function named **missing_values_summarizer** to go over our data and find the missing values; we will use this function in combination with the trusted NumPy to find those features that have missing values:

1. Use the missing values summarizer to expose the **props_df**, i.e., the number of missing values in the data:

```
props_df, _ = missing_values_summarizer(df=X_train)
```


2. We will then use **np.where** in combination with Pandas (**iloc**) to get the integer location of the missing values, based on a condition we supply:

```
props_df.iloc[np.where(props_df['missing_count'] > 0)]
```

3. The command in *Step 2* creates a summary of the missing values, as shown in *Figure 6.11*:

feature	missing_count	non_missing_count	total_vol	prop_missing	prop_non_missing	data_type
LotFrontage	259	1201	1460	0.177397	0.822603	float64
Alley	1369	91	1460	0.937671	0.062329	object
MasVnrType	872	588	1460	0.597260	0.402740	object
MasVnrArea	8	1452	1460	0.005479	0.994521	float64
BsmtQual	37	1423	1460	0.025342	0.974658	object
BsmtCond	37	1423	1460	0.025342	0.974658	object
BsmtExposure	38	1422	1460	0.026027	0.973973	object
BsmtFinType1	37	1423	1460	0.025342	0.974658	object
BsmtFinType2	38	1422	1460	0.026027	0.973973	object
Electrical	1	1459	1460	0.000685	0.999315	object
FireplaceQu	690	770	1460	0.472603	0.527397	object
GarageType	81	1379	1460	0.055479	0.944521	object
GarageYrBlt	81	1379	1460	0.055479	0.944521	float64
GarageFinish	81	1379	1460	0.055479	0.944521	object
GarageQual	81	1379	1460	0.055479	0.944521	object
GarageCond	81	1379	1460	0.055479	0.944521	object
PoolQC	1453	7	1460	0.995205	0.004795	object
Fence	1179	281	1460	0.807534	0.192466	object
MiscFeature	1406	54	1460	0.963014	0.036986	object

Figure 6.11: Missing values summary with filter

4. We have a preprocessing job to undertake, as we have missing values from both categorical and numeric columns.
5. Additionally, as this is a regression problem, there will undoubtedly be outliers that affect the model results, so we will need to implement a **RobustScalar** to deal with this.

Preprocessing function

Preprocessing categorical and numerical data is essential for every regression problem. To streamline this process, we have created a function that you can use in both personal and industry projects. The preprocessing function, namely **preprocess_data**, is available in the supporting GitHub repository. Exploring the function, the chapter will take you through what the function does:

- Identifies numeric and categorical columns using **select_dtypes**.
- Drops columns with too many missing values by:
 - Calculating a threshold as **min_non_na = int(threshold * len(X_train))**.
 - Drops numerical and categorical columns where missing values exceed the threshold value stated.
 - Updates **new_numeric_cols** and **new_categorical_cols** with the remaining features.
- Defines a numerical data processing pipeline.
- Imputes missing values using the median replacement strategy: **SimpleImputer(strategy='median')**.
 - Scales numeric features using **RobustScaler()** - this is due to the presence of outliers in the dataset, and in many regression datasets.
- Defines a categorical pipeline:
 - Uses modal replacement to fill in missing values with **SimpleImputer(strategy='most_frequent')**.
 - Encodes categorical values using **OneHotEncoder**, as we have used throughout.
- Combines the pipelines using **ColumnTransformer**:
 - Applying this to the numeric and categorical pipelines.
 - Dropping all other columns using **remainder='drop'**.

- Disables verbose feature names (**verbose_feature_names_out=False**).
- Fits and transforms the data using the **preprocessing_pipeline**.
- Transforms the validation sample, if provided; otherwise, it is set to **None**.
- Aligns the columns across all the datasets, to make sure that when we are undertaking one-hot encoding, that all columns are present in all data samples.
- Finally, returns the processed DataFrames **X_train_df**, **X_test_df** and **X_val_df** (if **X_val** is provided as an input), otherwise will just return training and testing frames.

The final step is to preprocess our training and test sets:

1. `X_train_pp, X_test_pp = preprocess_data(X_train=X_train,`
2. `X_test=test)`

This function will get your data in the perfect shape for training our models on this regression problem of predicting house prices.

Modeling setup

The modeling steps will focus on importing the packages we need, initializing our models in a dictionary, creating a regression training loop, and finally training this loop on our data.

The first two steps we will do in this section are to do the package imports and to initialize our models in a dictionary:

1. We will start by importing the necessary packages needed to perform regression on our training data:

```
from sklearn.linear_model import LinearRegression, Ridge, ElasticNet
```

```
from sklearn.ensemble import RandomForestRegressor
```

```
from xgboost import XGBRegressor
```

2. In this step, we will create the dictionary to store the initialized models. We will use Linear, Ridge, and Elastic Net regression and non-linear methods, random forests, and XGBoost:

```
models = {  
    "linear_regression": LinearRegression(),  
    "ridge": Ridge(alpha=1.0),  
    "elastic_net": ElasticNet(alpha=0.1,  
                               l1_ratio=0.5),  
    "random_forest": RandomForestRegressor(n_estimators=100,  
                                           random_state=42),  
    "xgboost": XGBRegressor(n_estimators=500,  
                             max_depth=6,  
                             learning_rate=0.1,  
                             random_state=42)}
```

3. The next phase is to then build the training loop; however, this is somewhat more complex than these steps and indeed warrants its own subsection.

Regression model training loop

This function, included in the supporting GitHub repository entitled **regression_training_loop**, automates the training, evaluation, and saving of multiple regression models using **K-fold cross-validation (CV)**. It ensures model selection based on a specified metric, stores trained models, and optionally makes predictions on a validation dataset. The following steps will describe what the training loop does:

1. **Initialize and set up the training environment:** This phase consists of:

- a. Create a directory to save the serialized models, for loading back in when we do the evaluation step.
 - b. Initialize variables: **results** (stores the model performance metrics; **best_model** (keeps track of the best-performing model) and **best_score** (tracks the best metric score).
 - c. Sets up K-fold cross validation using **cv_folds** to determine how many times the data is split into training and validation subsets.
2. **Train each model using K-fold cross validation:** In this step, we iterate through the dictionary of the model, where the name of the model and the model object are present, then we perform the following:
 - a. Initialize a list of **fold_metrics** to store the performance of each cross-validation fold.
 - b. Perform K-fold splitting for each fold, we split **X_train** and **y_train** into training (**X_train_cv** and **y_train_cv**) and validation into (**X_val_cv** and **y_val_cv**) training subsets.
 - c. Train the specific model on the fold-specific training data.
 - d. Make predictions on the fold-specific validation data.
 - e. Compute the evaluation metrics between the training and validation folds and store them.
 - f. Save each trained model from each fold into a **save_path** on disk (we serialize the models to load back in later).
3. **Compute and store model performance:** Here, we perform the following:
 - a. Compute the **fold_metrics** to a DataFrame and compute mean performance across all folds, using our **compute_metrics** function we defined in the *Evaluation Metrics* section.
 - b. Store the average metrics in results.
 - c. Determine the best model based on a scoring metric:

- i. **For R^2** , select the model with the highest score.
 - ii. **For error metrics** (MSE, RMSE, etc.) select the model with the lowest value.
4. **Display and format results:** These steps are:
 - a. Convert results to a Pandas DataFrame and format numerical values for readability.
 - b. Print the formatted cross-validation results.
5. **Retrain the best model on full training data:** This step then uses the best model and performs a full train on the whole training dataset, as K-fold cross validation uses a portion of the training sample to create a validation set. Additionally, keep these in mind:
 - a. If **use_best_model=True**, retrain the best-performing model on the entire X_train dataset.
 - b. Save the retrained model for future use.
6. **Return results:** The final step, in the function, returns the:
 - a. Best trained model
 - b. The formatted results
 - c. The saved file path of the best model

With a clear understanding of the training loop and after reviewing it in the accompanying GitHub repository, we are now ready to use the function to train our model.

Training our regression model

We have dealt with the most complex sections, the preprocessing and model training loop development; now, all that is left to do is run our training loop against our chosen models and data and use our evaluation metrics to pick the best one. To train our model, we use the following command:

```
1. best_model, model_results, df_best_model_fn = regression_trainin  
   g_loop(
```

2. models,
3. X_train=X_train_pp,
4. y_train=y_train,
5. scoring_metric="MAE")

This will run the model training, and when it has been completed, you will be able to inspect the **model_results** DataFrame, as the following figure shows:

	MSE	RMSE	MAE	R2	MSLE	MAPE(%)
linear_regression	173,286,949,065,445,504.00	186,196,899.16	20,177,342.49	-31,366,070.88	0.02	12,390.52
ridge	1,246,950,809.43	33,634.11	18,891.86	0.79	0.03	11.30
elastic_net	1,248,645,039.89	33,537.03	18,001.81	0.79	0.02	10.55
random_forest	963,456,625.17	30,251.21	17,862.67	0.84	0.02	10.44
xgboost	970,729,380.91	30,343.90	16,891.06	0.84	0.02	9.66

Figure 6.12: Regression model results

Inspecting the results, we can see that random forests and XGBoost are the clear winners; using mean absolute error, XGBoost wins and shows that out of all those predictions, it is only off by \$30 343 dollars, and its MAPE (%) is 9.66%. Additionally, you can see how wrong linear regression gets it (due to the non-linearity in the modeling task) without Ridge and Elastic Net regularization methods.

Using SHAP for global and local feature importance

Shapley Additive Explanations (SHAP) and global feature importance both help us understand how a model makes predictions, but they provide insights at different levels. Global feature importance gives an overall view of which features the model relies on the most, showing their average impact across all predictions.

This helps identify the most influential variables, guiding feature selection and model interpretation. For example, in a house price prediction model, features like square footage or location might have the highest importance, meaning they strongly affect price predictions across the dataset.

On the other hand, SHAP values go deeper by explaining the contribution of each feature to an individual prediction. Instead of just knowing which features are generally important, SHAP tells us *why* a model made a specific decision for a given instance.

For example, in the same house price model, SHAP might reveal that a garden increased the predicted price for one house, while being far from the city center decreased it. This level of transparency is especially useful in complex models, making predictions more interpretable and trustworthy. While global importance provides a big-picture view, SHAP values allow for more granular, case-by-case explanations.

We will perform global and local feature importance on our testing data by loading our best model from the previous steps.

Global feature importance with SHAP

To get global feature importance using SHAP, we follow the methods to load our serialized model and then use the package to undertake this analysis:

1. Load our best model from the previous training section:

```
with open(df_best_model_fn, 'rb') as f:
```

```
    best_model = pickle.load(f)
```

2. Import the **shap** package. To install this, use **pip install shap**:

```
import shap
```

3. Create a sub-sample of our testing set to grab 100 records. You could implement this to do random sub-sampling, but we have just chosen the first 100:

```
sample_X_test = X_test_pp[:100]
```

4. Fit a **shap Explainer** object to the testing data, which quantifies each feature's positive or negative impact on the specific prediction. There are variants of the Explainer object, such as

TreeExplainer, **LinearExplainer**, and **DeepExplainer** (for deep learning with neural networks):

```
explainer = shap.Explainer(best_model, sample_X_test)
```

5. Get the **shap** values from the explainer:

```
shap_values = explainer(sample_X_test)
```

6. Finally, generate a summary plot for the global feature importance:

```
shap.summary_plot(shap_values, sample_X_test)
```

Step 6 generates the global feature importance plot defined in Figure 6.13:

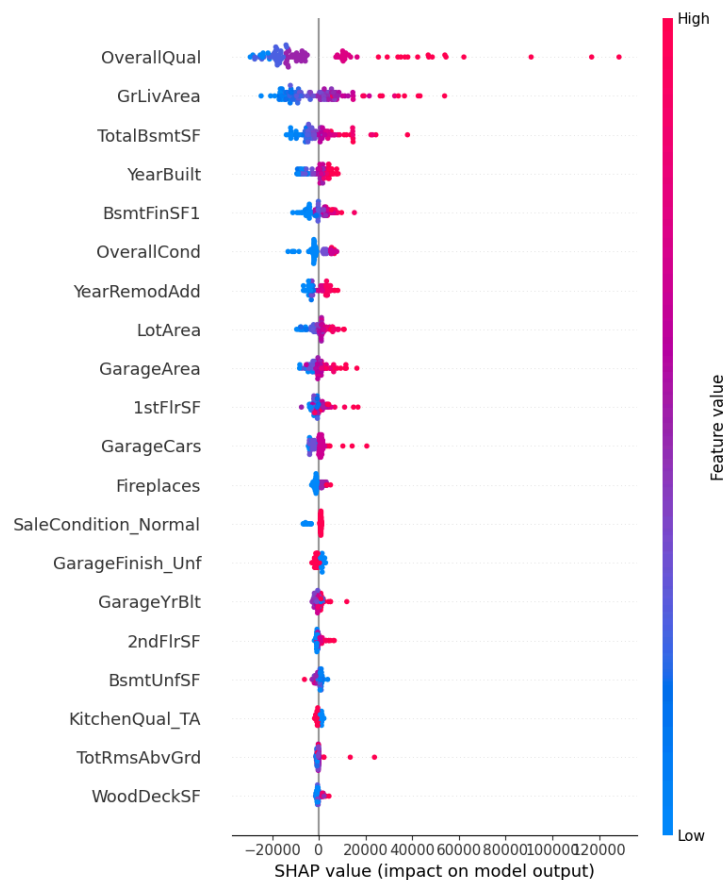


Figure 6.13: Global feature importance SHAP plot

You can interpret this as follows:

- Features are ranked from most important (top) to least important (bottom) based on their average impact on predictions.

- **Overall quality (OverallQual)** is the most influential feature in predicting the price of a house, followed by **above-ground living area (GrLivArea)** and **total basement square feet (TotalBsmtSF)**.
- The x-axis represents the impact of each feature on the model's prediction, where Negative SHAP values decrease the predicted output and positive SHAP values (moving to the right) increase the expected output. Color coding shows feature value contribution.
- Feature variability is interpreted from the spread of the points on the x-axis; large spreads vary significantly across different instances, whereas features with a narrow spread have a more consistent, minor impact on predictions.

Local feature importance with SHAP

Now we have fitted the Shapley models and obtained Shapley Values. In the previous section, we can simply point to one observation in our training data, we want to know more about it, and then use a plot to inspect it:

1. Choose an index from the test data to sample (we will go with the first observation):

```
idx = 0
```

2. Use **shap.waterfall_plot** to generate the plot for that index:

```
shap.waterfall_plot(shap_values[idx])
```

Once you run the line in *Step 2*, *Figure 6.14* will appear:

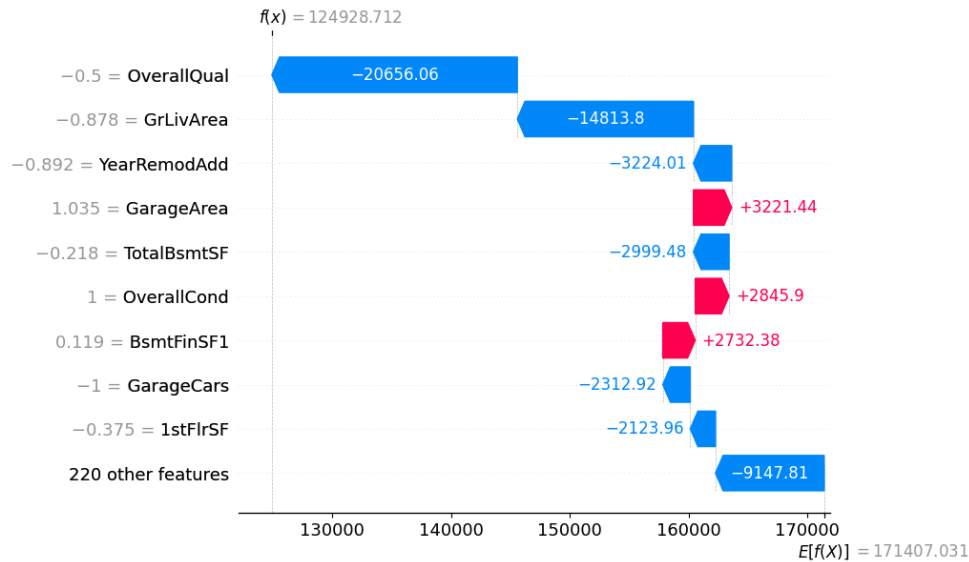


Figure 6.14: Local feature importance waterfall plot

Here is an explanation of the chart:

- The baseline prediction on the bottom right $E[f(x)]$ represents the baseline average model prediction: \$171,407.
- The prediction for this record shows the value predicted is much lower than baseline: \$124,928. The reasons for this are:
 - The most significant decrease is OverallQual (-20,656), meaning that the house has a much lower quality than others.
 - The final negative contribution from 220+ other features suggests that many smaller factors cumulatively reduced the predicted price.
- The great thing about local importance is that you can analyze the direct impact at a record level, meaning you can see which features are having a positive or negative effect on the target variable.

Additionally, using this command, we can get a force plot, which shows you the overall effect of the main contributing features, with blue indicating an impact on lower price and red having a greater impact:

1. `shap.force_plot(explainer.expected_value,`
2. `shap_values[idx].values,`

3. `sample_X_test.iloc[idx], matplotlib=True)`

This produces the force plot as shown in *Figure 6.15*:

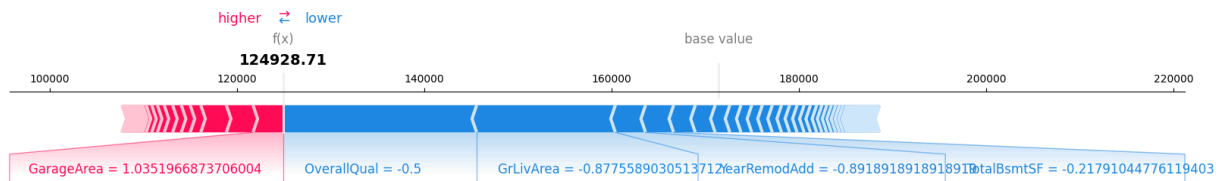


Figure 6.15: Local feature importance force plot

The previous plot demonstrates a similar impact as the previous one, though one could argue that the waterfall plot is easier to interpret. Ultimately, it comes down to personal preference. Before concluding this chapter, we will finalize by concatenating our results with the testing data.

Joining predictions with test dataset

For completeness, the final step is to merge our predictions from the best model with the testing dataset. This approach mirrors how you would handle live, unseen data from a database or another system, storing model predictions alongside the records and later comparing them with actual values once they become available. This allows you to assess the model's real-world performance over time.

Follow these steps to achieve this:

1. Use the previously loaded serialized model to use the predict method on the test data:

```
y_test_preds = best_model.predict(X_test_pp)
```

2. Concatenate the predictions, using a Pandas Series object, with column name of **PredSalePrice** (or whatever you want to name it) onto the preprocessed or unpreprocessed testing dataset and join on the row axis (**axis=1**):

```
x_test_with_preds = pd.concat(
```

```
[
    pd.Series(y_test_preds, name='PredSalePrice'),
    X_test_pp
],
axis=1)
```

3. We can inspect the head of the new concatenated DataFrame to see the predictions:

```
x_test_with_preds.head()
```

Throughout this chapter, we have preprocessed our training data, tackled a real-world Kaggle tournament dataset, applied cross-validation across multiple models to evaluate accuracy, and achieved competitive results. We have also saved and loaded models efficiently, explored SHAP values for interpretability with clear visualizations, and finalized our workflow by integrating predictions back into the testing set.

Conclusion

In this chapter, we explored regression techniques for house price estimation, starting with fundamental linear regression and progressing to feature selection, regularization, and non-linear models, such as random forest and gradient boosting. We examined key evaluation metrics, including MAE and R-squared, to assess the model's accuracy. We learned how to refine models for better predictive performance by implementing these techniques, ensuring they generalize well to unseen data. Additionally, we leveraged feature importance analysis, including SHAP values, to interpret model predictions and understand the impact of different variables on house prices.

The chapter culminated in a hands-on project using a Kaggle tournament dataset, where we applied these concepts to real-world housing data. We preprocessed features, trained multiple regression models, and selected the best-performing approach based on cross-validation results. We gained

insights into the factors influencing house prices by analyzing predictions and visualizing the contributions of key features. With our optimized model, we achieved results competitive enough to place us on the Kaggle leaderboard, demonstrating the real-world effectiveness of our approach.

In the next chapter, we will delve into the key concepts and methods of dimensionality reduction, beginning with foundational techniques like PCA and extending to powerful non-linear approaches such as t-SNE and UMAP. We will also explore how to effectively visualize high-dimensional data in lower dimensions, examine the practical role of dimensionality reduction in machine learning workflows, and conclude with a hands-on case study applying these techniques to the MNIST dataset.

Points to remember

Here are some key points summed up from this chapter:

- **Applying regression to real-world data:** Our final project using a Kaggle tournament dataset demonstrated how these techniques translate into competitive, leaderboard-worthy predictions for house prices.
- **Cross-validation for model reliability:** K-fold cross-validation ensures that model performance is not dependent on a single train-test split, leading to more reliable results.
- **Data preprocessing matters:** Handling missing values, scaling numerical data, and encoding categorical variables properly ensures that regression models perform optimally.
- **Feature importance analysis:** SHAP values provide both global and local interpretability, helping identify which features most influence predictions and explaining individual model decisions.
- **Feature selection for better predictions:** RFE and RFECV help remove irrelevant features, improving model efficiency and accuracy.

- **Model evaluation metrics:** Metrics such as MAE, RMSE, and R-squared assess how well a model predicts house prices and highlight areas for improvement.
- **Non-linear regression methods:** Algorithms like random forest and gradient boosting capture complex relationships that linear models may miss, making them effective for house price prediction.
- **Regularization techniques:** Ridge regression shrinks coefficients but retains all features, Lasso forces some coefficients to zero for automatic feature selection, and Elastic Net balances both approaches.
- **Training and deploying models:** Saving trained models for future use enables real-world deployment and ongoing evaluation with new data.
- **Understanding regression models:** Linear regression is a fundamental technique for predicting continuous variables, while advanced methods like Ridge, Lasso, and Elastic Net improve model generalization by reducing overfitting.

Multiple choice questions

1. **What is the primary advantage of using multiple linear regression over simple OLS regression?**
 - a. It simplifies the model by reducing the number of independent variables
 - b. It can capture relationships between multiple independent variables and the dependent variable
 - c. It ensures that all independent variables are equally weighted
 - d. It is only used when the dataset has categorical variables
2. **Which of the following best describes RFE?**

- a. A method that removes features one by one based on their importance until an optimal subset remains
 - b. A technique that randomly selects features for regression modeling
 - c. A data transformation method that reduces dimensionality using principal components
 - d. A method that increases the number of predictors in a regression model
3. **What is the primary role of regularization techniques like Ridge and Lasso regression?**
- a. To remove categorical variables from the regression equation
 - b. To improve the accuracy of models by maximizing coefficient values
 - c. To increase the variance of the model
 - d. To prevent overfitting by adding a penalty to large coefficients
4. **How does Lasso regression differ from Ridge regression?**
- a. Lasso regression does not use regularization, while Ridge regression does
 - b. Lasso regression applies an L2 penalty, while Ridge regression applies an L1 penalty
 - c. Lasso regression shrinks some coefficients to zero, effectively performing feature selection
 - d. Ridge regression removes variables with low importance from the model
5. **Which of the following non-linear regression methods is known for building multiple decision trees and averaging their results?**
- a. Polynomial regression

- b. Ridge regression
 - c. Lasso regression
 - d. Random forest regression
6. **What is the primary difference between random forest and gradient boosting models?**
- a. Gradient boosting trains trees sequentially to correct errors, while Random Forest trains trees independently and averages results
 - b. Random forest always outperforms gradient boosting on all datasets
 - c. Gradient boosting uses L2 regularization, while random forest does not
 - d. Random forest is only used for classification tasks, while gradient boosting is only used for regression
7. **Which of the following evaluation metrics is most useful when interpreting the percentage error of a regression model?**
- a. RMSE
 - b. MLSE
 - c. MAE
 - d. MAPE
8. **What is the primary advantage of using SHAP values in regression analysis?**
- a. They are only used for categorical target variables
 - b. They replace the need for model evaluation metrics like RMSE
 - c. They identify which features contribute most to individual predictions
 - d. They reduce model complexity by eliminating weak predictors

- 9. Why is cross-validation commonly used in regression modeling?**
- a. It increases the training dataset size by duplicating existing data
 - b. It helps assess model generalization by testing performance on multiple subsets of data
 - c. It replaces the need for a separate test dataset
 - d. It is only useful for classification models
- 10. In the housing price prediction project, why was XGBoost one of the best-performing models?**
- a. It minimizes error by sequentially improving weak learners
 - b. It always provides perfect predictions for any dataset
 - c. It is a simple linear model that outperforms tree-based methods
 - d. It does not require hyperparameter tuning to achieve optimal results
- 11. What is the main advantage of using RFECV over standard RFE?**
- a. RFECV reduces the dataset size by eliminating unnecessary observations
 - b. RFECV automatically selects the optimal number of features based on model performance
 - c. RFECV guarantees a higher R-squared value compared to standard RFE
 - d. RFECV increases model complexity by adding more predictors
- 12. Why was RobustScaler chosen as the scaling method in the house price prediction project?**

- a. It is less sensitive to outliers compared to MinMaxScaler and StandardScaler
- b. It standardizes data by subtracting the mean and dividing by the standard deviation
- c. It ensures all numerical features follow a normal distribution
- d. It is required for categorical encoding to function properly

Answers

1	b
2	a
3	d
4	c
5	d
6	a
7	d
8	c
9	b
10	a
11	b
12	a

Key terms

Some key terms used in this chapter are:

- **Adjusted R-squared:** A modified version of R-squared that accounts for the number of predictors in a regression model, preventing overestimation of model fit.
- **Bias term:** The intercept in a regression equation, representing the expected value of the dependent variable when all independent variables are zero.

- **Bootstrap sampling:** A resampling technique used in statistics and machine learning to improve model estimates and assess variability.
- **Collinearity:** A situation in which two or more independent variables in a regression model are highly correlated, potentially leading to unreliable coefficient estimates.
- **Cross-validation:** A technique for evaluating a model's performance by training and testing on different subsets of the data to reduce overfitting.
- **Elastic Net:** A regression technique that combines both L1 (Lasso) and L2 (Ridge) regularization to improve model performance and feature selection.
- **Error term (residual):** The difference between the actual and predicted values in a regression model, representing unexplained variance.
- **Feature engineering:** The process of transforming raw data into meaningful features that improve model performance.
- **Feature scaling:** A preprocessing step that standardizes or normalizes data to ensure fair contributions of variables in a regression model.
- **Gradient boosting:** An ensemble learning method that sequentially builds weak models, correcting the errors of previous ones to improve performance.
- **Heteroscedasticity:** A condition in regression analysis where the variance of residuals is not constant across different levels of an independent variable.
- **Homoscedasticity:** A desirable property in regression where the variance of residuals remains constant across all levels of the

independent variable, ensuring that the model does not favor certain data points.

- **Intercept (β_0):** The baseline value of the dependent variable when all predictor variables are set to zero in a regression equation.
- **Lasso regression:** A regularization method that applies an L1 penalty, shrinking some coefficients to zero, effectively performing feature selection.
- **Least squares method:** A statistical technique used in regression to find the best-fitting line by minimizing the sum of squared residuals.
- **MAE:** A regression evaluation metric that calculates the average absolute difference between predicted and actual values.
- **MAPE:** Expresses the model's prediction error as a percentage, making it useful for understanding relative errors.
- **MSE:** A regression evaluation metric that penalizes larger errors by squaring the differences between predicted and actual values.
- **MSLE:** A variation of MSE that considers the logarithm of the predictions and actual values, useful for datasets with large value ranges.
- **Multicollinearity:** A scenario in which independent variables are highly correlated, making it difficult to determine their individual effects on the dependent variable.
- **Multiple linear regression:** An extension of simple linear regression that uses multiple independent variables to predict a dependent variable.
- **Non-linear regression:** Regression models that capture more complex relationships between independent and dependent variables, such as decision trees and gradient boosting.

- **OLS:** A standard method for estimating the coefficients of a linear regression model by minimizing the sum of squared residuals.
- **Overfitting:** A situation where a model performs well on training data but fails to generalize to new data due to excessive complexity.
- **Predictive modeling:** The process of using statistical algorithms and machine learning techniques to forecast outcomes based on input data.
- **Random forest regression:** A non-linear ensemble learning method that constructs multiple decision trees and averages their predictions.
- **RFE:** A technique that iteratively removes the least important features to improve model interpretability and accuracy.
- **Regularization:** A technique used to prevent overfitting by adding a penalty to the model's coefficients, as seen in Ridge and Lasso regression.
- **Ridge regression:** A regularization method that applies an L2 penalty, reducing coefficient magnitudes while retaining all features.
- **RMSE:** The square root of MSE, providing predictions in the same units as the dependent variable and penalizing large errors more than MAE.
- **R²:** A metric that measures how well the independent variables explain the variance in the dependent variable, with values closer to 1 indicating a better fit.
- **SHAP:** A method for explaining model predictions by quantifying the impact of each feature on an individual prediction.
- **Train-test split:** The process of dividing a dataset into training and testing subsets to evaluate model performance.

- **Variance Inflation Factor (VIF):** A metric used to detect multicollinearity by measuring how much an independent variable is explained by other predictors.
- **Weighted quantile sketch:** An algorithm used in XGBoost to efficiently determine the best split points in a large dataset.
- **XGBoost:** A highly efficient, scalable, and optimized gradient boosting algorithm that incorporates both L1 (Lasso) and L2 (Ridge) regularization to improve generalization and prevent overfitting. It is designed for speed and performance, leveraging parallel processing, tree pruning, and handling missing values automatically. XGBoost also supports GPU acceleration, early stopping, and hyperparameter tuning, making it one of the most powerful and widely used machine-learning models for regression and classification tasks.

Join our Discord space

Join our Discord workspace for latest updates, offers, tech happenings around the world, new releases, and sessions with the authors:

<https://discord.bpbonline.com>



CHAPTER 7

Handwritten Digits Dimensionality Reduction

Introduction

In this chapter, we address one of the most significant challenges in modern machine learning: high-dimensional data. You will learn how to simplify complex datasets using powerful dimensionality reduction techniques, such as **principal components analysis (PCA)**, **t-SNE**, and **Uniform Manifold Approximation and Projection (UMAP)**, which uncover hidden patterns while reducing noise, overfitting, and computational load. We will keep things hands-on and practical, showing you how to apply these techniques in Python with clear, step-by-step examples.

To bring everything together, we will explore the MNIST handwritten digit dataset, a classic benchmark of 70,000 grayscale images representing digits 0 through 9. Each image is 28x28 pixels, resulting in 784 features per sample, perfect for seeing the impact of dimensionality reduction in action. You will learn how to visualize the data in 2D, improve model training, and build a high-performing digit classifier using a reduced feature set. By the end of this chapter, you will know how to transform overwhelming data into meaningful, visual, and model-ready representations.

Structure

This chapter covers the following topics:

- Introducing dimensionality reduction techniques

- Principal components analysis
- t-distributed Stochastic Neighbor Embedding
- Uniform Manifold Approximation and Projection
- Visualizing high-dimensional data in 2D and 3D
- Applications of dimensionality reduction in machine learning
- Applying dimensionality reduction to the MNIST dataset

Objectives

By the end of this chapter, you will have a clear understanding of how to simplify high-dimensional datasets using dimensionality reduction techniques and the skills to apply them in practice. You will explore the challenges of working with high-dimensional data and learn how reducing dimensions can improve model performance, interpretability, and visualization.

Through hands-on Python examples, you will learn to apply techniques such as PCA, t-SNE, and UMAP to reveal structure in complex datasets and transform them into lower-dimensional representations. Using the MNIST handwritten digit dataset, you will learn how dimensionality reduction enhances data exploration and accelerates training. Ultimately, building and evaluating a classification model using reduced features will equip you with practical tools to confidently apply dimensionality reduction in your own machine learning projects.

Introducing dimensionality reduction techniques

In modern machine learning, dealing with high-dimensional data presents challenges such as increased computational complexity, overfitting, and difficulties in visualizing the data. Dimensionality reduction techniques help mitigate these issues by transforming high-dimensional data into a lower-dimensional representation while preserving essential patterns and

structures. These techniques enhance model performance, improve interpretability, and facilitate data visualization.

In this section, we will explore both linear and non-linear dimensionality reduction techniques. We will begin with PCA, a widely used linear method that projects data onto principal components to maximize variance retention. Next, we will delve into non-linear techniques such as t-SNE and UMAP, which are particularly effective for uncovering complex structures in data that PCA might miss. We will also discuss methods for visualizing high-dimensional data in 2D and 3D to gain intuitive insights into data distributions.

Beyond visualization, dimensionality reduction has numerous significant applications in machine learning, including improving model efficiency, reducing noise, and enhancing clustering and classification tasks. To illustrate these concepts in practice, we will conclude with a case study on the MNIST dataset, demonstrating how dimensionality reduction techniques can be applied to enhance data exploration and model performance.

Principal component analysis

PCA is a dimensionality reduction technique that transforms high-dimensional data into a lower-dimensional space while preserving as much variance as possible. It achieves this by identifying the principal components, orthogonal axes (independent directions in a geometric space) along which the data varies the most.

The key steps in PCA are:

1. **Standardizing the data:** Ensuring all features have the same scale.
2. **Computing the covariance matrix:** Understanding how features relate to each other.
3. **Calculating eigenvalues and eigenvectors:** Identifying principal components.

4. **Projecting data onto principal component:** Transforming data into a lower-dimensional space.

In the following sections, we will examine these steps in more detail, accompanied by Python examples for added clarity. Before we start to understand the steps, we will bring in our imports and generate some random data using NumPy:

1. Load in the packages needed for this section:

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.decomposition import PCA
```

```
from sklearn.preprocessing import StandardScaler
```

2. Use NumPy to select a random seed:

```
np.random.seed(42)
```

```
data = np.random.rand(10,4) * 10
```

3. Create some feature names for our generated features:

```
col_names = ['Feature 1', 'Feature 2', 'Feature 3', 'Feature 4']
```

```
df = pd.DataFrame(data, columns=col_names)
```

4. This will generate a DataFrame, as shown in *Figure 7.1*:

Feature 1	Feature 2	Feature 3	Feature 4
3.745401	9.507143	7.319939	5.986585
1.560186	1.559945	0.580836	8.661761
6.011150	7.080726	0.205845	9.699099
8.324426	2.123391	1.818250	1.834045
3.042422	5.247564	4.319450	2.912291
6.118529	1.394939	2.921446	3.663618
4.560700	7.851760	1.996738	5.142344
5.924146	0.464504	6.075449	1.705241
0.650516	9.488855	9.656320	8.083973
3.046138	0.976721	6.842330	4.401525

Figure 7.1: Generated PCA data

From this step, we go on to the standardization of the data in the next section.

Step 1 as standardizing data

PCA is affected by scale because it relies on variance, which in turn depends on the magnitude of the values. If one feature has much larger values than another, it will dominate the principal components. For this, we will use our **StandardScaler** that we should be aware of by now:

```
scaler = StandardScaler()
data_scaled = scaler.fit_transform(data)
df_scaled = pd.DataFrame(data_scaled, columns=col_names)
```

As this is a scaler, the results will now be within a magnitude of the mean value and will have zero mean and unit variance.

Step 2 as computing the covariance matrix

The covariance between two features X_i and X_j is represented mathematically as:

$$\text{Cov}(X_i, X_j) = \frac{1}{n-1} \sum_{k=1}^n (X_{ki} - \bar{X}_i)(X_{kj} - \bar{X}_j)$$

Let us understand this in detail:

- X_{ki} is the value of feature i for the k^{th} sample
- $\bar{X}_i$ is the mean of feature i
- The number of samples is n

The covariance matrix is what is known as an $m \times m$ matrix (linked to linear algebra) where m is the number of features. For example, if we have four

features in our training data, then a 4×4 matrix is formed. To implement this in Python, we use the following code:

1. Use **np.cov** to take the covariance of the features we defined before and take what is known as the transpose (**T**), which flips the matrix's rows and columns:

```
cov_matrix = np.cov(data_scaled.T)
```

2. Create a DataFrame to store the transposed matrix:

```
df_cov_mat = pd.DataFrame(cov_matrix,  
                           columns=col_names,  
                           index=col_names)
```

3. This displays the matrix as shown in *Figure 7.3*:

	Feature 1	Feature 2	Feature 3	Feature 4
Feature 1	1.111111	-0.372501	-0.520656	-0.571833
Feature 2	-0.372501	1.111111	0.303003	0.562230
Feature 3	-0.520656	0.303003	1.111111	-0.151681
Feature 4	-0.571833	0.562230	-0.151681	1.111111

Figure 7.2: Covariance matrix (normalized values)

Step 3 as computing eigenvalues and vectors

The eigenvectors (v) and eigenvalues (λ) satisfy the following formula:

$$Cv = \lambda v$$

To implement in Python, follow these steps:

1. Use the Linear Algebra functions from NumPy to get the **eigens** of the covariance matrix:

```
eigenvals, eigenvecs = np.linalg.eig(cov_matrix)
```

2. Next, we will sort the eigenvalues to identify the ones that account for the most variance, along with their corresponding eigenvectors. Since covariance reflects how variables move together, it can also be understood as a measure of influence:

```
sort_indices = np.argsort(eigenvals)[::-1]
eigenvals = eigenvals[sort_indices]
eigenvecs = eigenvecs[:, sort_indices]
```

3. Printing the sorted eigenvalues, we get a list returned **[2.25231771 1.2822067 0.72217742 0.18774261]**.

The eigenvalues tell us how much variance each component explains, whereas the eigenvectors define the directions of the new principal components; we will visualize this in *Step 5*.

Step 4 as choosing the number of principal components

The heuristic here is to choose the components that retain most of the variance while reducing dimensionality in the feature set, as these methods are geared to avoid the curse of dimensionality, meaning the model struggles to find patterns in higher-dimensional spaces.

The explained variance of that component ratio is explained as follows:

$$\frac{\lambda_i}{\sum_{j=1}^m \lambda_j}$$

Where λ_i is the eigenvalue of the i^{th} principal component. To determine the number of components to retain, we examine the cumulative variance and select the k value such that 95% of the variance is retained. To undertake this in Python, we will follow these steps:

1. For the sake of the visualization, we will select three components and visualize their projections:

```
top_eigenvecs = eigenvecs[:, :3]
```

2. The final step is to project the data onto the principal components, which transforms the data into this new feature space.

Step 5 as projecting data onto principal components

Now that we have found the main patterns (principal components) in our data, we can transform our original data to align with these patterns. We do this using a special kind of multiplication called a dot product. This means that each row of our original data is multiplied by each column of the principal component's matrix, with corresponding numbers being multiplied and then added together.

The result is a new table of numbers that represents our data in terms of the main patterns we identified. The formula is $Z = XW$ (Z is the transformed dataset, X is the original standardized data, and W is the matrix of eigenvectors, principal components).

To implement this in Python code, we perform the following:

1. Take the **dot** product using **np.dot** (shorthand for matrix multiplication):

```
data_pca = np.dot(data_scaled, top_eigenvecs)
```

2. Once we have carried out *Step 1*, we create a DataFrame:

```
df_pca = pd.DataFrame(data_pca,  
                      columns=['PC1', 'PC2', 'PC3'])
```

3. This will produce an output that is scaled, as per the first step in this process, and will produce an output as shown in *Figure 7.3*:

PC1	PC2	PC3
-1.401688	-0.503125	1.007150
-0.506301	1.348706	-1.788846
-0.334655	2.222431	0.530091
2.334498	0.103169	0.694457
-0.016734	-0.598367	-0.006437
1.396679	0.010608	-0.150722
-0.175443	0.700908	0.722761
1.502394	-1.281104	-0.145452
-2.871185	-0.899741	0.108557
0.072435	-1.103488	-0.971558

Figure 7.3: Principal component projections

- The natural next step would be to visualize these components on a 3D visualization. This can be done using the **modelviz** (make sure the package is installed using **pip install modelviz**) package simply, using this command:

```
from modelviz.pca import plot_pca_projection
```

```
plot_pca_projection(df_original=df_scaled, df_projected=df_pca,
```

```
labelpad=2, projected_color='black')
```

- This will produce a visualization as shown in *Figure 7.4*:

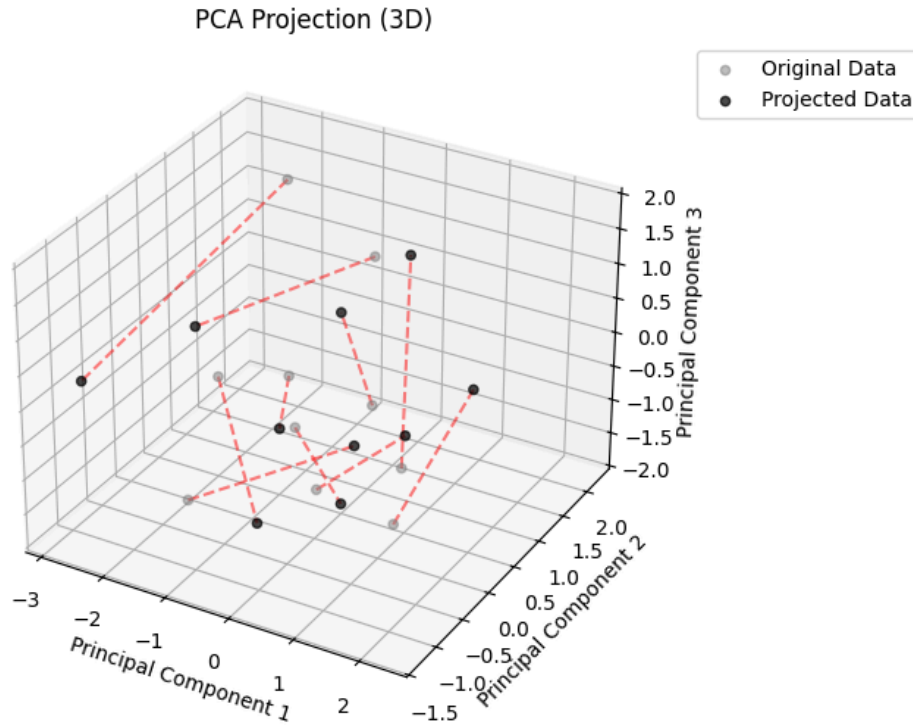


Figure 7.4: Projected principal components

This 3D plot illustrates PCA, a technique used to reduce dimensionality by transforming high-dimensional data (gray points) into a lower-dimensional space (black points), while keeping as much important information (variance) as possible.

By simplifying the data structure, PCA helps remove noise, reduce computation time, and make patterns more recognizable for ML models, improving efficiency and interpretability. However, it is based on linear separation, and this can be its disadvantage at times, as it struggles to work with high-dimensional data effectively.

t-distributed Stochastic Neighbor Embedding

t-SNE helps visualize high-dimensional data by mapping similar data points close together in a lower-dimensional space. It is advantageous when dealing with complex datasets where patterns are not immediately apparent. This will be the first non-linear dimensionality reduction technique we will look at.

In the following subsections, we will outline the logic of this process, including the top-level mathematics of each step, as Python implementations would be verbose. We will then demonstrate how to conduct t-SNE using scikit-learn.

Step 1 as computing similarities in high dimensional space

This step is centered around calculating a normal distribution of how similar points are to others. For example, imagine ranking your ten best friends based on closeness (proximity). The method, behind the madness, is to calculate the probability that a point x_i would pick x_j as its neighbor. The equation for this is included and mentioned here, but we will produce this in Python as well to solidify understanding:

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)}$$

We will create the steps in Python, with some simulated data, to bring this equation to life:

1. Import the necessary packages:

```
import numpy as np
```

```
from sklearn.metrics.pairwise import euclidean_distances
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

2. Generate 100 data points with 50 features:

```
np.random.seed(42)
```

```
X = np.random.rand(100, 50)
```

3. Get Euclidean distances from each of the X points:

```
distances = euclidean_distances(X, X)
```

4. Create a function, that replicates the given equation, to get the conditional probabilities:

```
def compute_p_ij(distances, sigma=1.0):  
    P = np.exp(-distances ** 2 / (2 * sigma ** 2))  
    P /= np.sum(P, axis=1, keepdims=True)  
    return (P + P.T) / (2 * len(P))
```

5. The initial aims here are to compute the distance matrix (similar points are closer and less similar are further away); the **P** /= step normalizes the values between 0 and 1, and the return statement makes sure that the matrices maintain symmetry.
6. We then use the function over the distances we created in *Step 3* to get the relative probabilities, and we then check the shape, as the matrix should be 100 x 100, as there are 100 data points; this will change depending on the number of rows in the array or DataFrame:

```
P = compute_p_ij(distances)  
print("Shape of P:", P.shape)
```

Now on to the next step.

Step 2 as Student t-distribution and mapping to lower dimensions

The **t-distribution** (or **Student's t-distribution**) is a probability distribution that is similar to the normal distribution but has heavier tails. This means it has more probability mass in the tails and is used when dealing with smaller sample sizes or when the population standard deviation is unknown, generally with sample sizes where $n < 30$.

In this step, the aim is to map our friend's proximities, as per the first example, so that their distances reflect the original high-dimensional

similarities. The t-distribution comes into play, as this helps manage crowded areas by allowing distant points to remain far apart.

This is the equation:

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq i} (1 + \|y_k - y_l\|^2)^{-1}}$$

As with the previous step, we will bring this to life with Python, through the following steps:

1. We compute the Euclidean distance matrix between all pairs of points in Y:

```
distances_low = euclidean_distances(Y, Y)
```

2. The t-distribution kernel comes into play next; this ensures that nearby points have higher similarity values, while distant points have lower values:

```
Q = (1 + distances_low ** 2) ** -1
```

3. This step removes self-similarities by setting the diagonal of the matrix to zero, effectively ignoring each point's similarity to itself since we are only interested in how it relates to its neighbors:

```
np.fill_diagonal(Q, 0)
```

4. Next, we normalize Q so that all its values sum to 1, turning it into a valid probability distribution. This allows it to be compared directly with P during the KL divergence calculation:

```
Q /= np.sum(Q)
```

5. We now combine Steps 1 to 4 into a function for later reuse:

```
def compute_q_dist(Y):
```

```
    distances_low = euclidean_distances(Y, Y)
```

```
    Q = (1 + distances_low ** 2) ** -1
```

```
    np.fill_diagonal(Q, 0)
```

```
Q /= np.sum(Q)
```

```
return Q
```

6. Next, we will randomly initialize a starting point to use as our reference:

```
np.random.seed(42)
```

```
Y = np.random.randn(100, 2)
```

7. The last step is to use our function to compute the q-distribution and to check the shape of the matrix:

```
Q = compute_q_dist(Y)
```

```
print("Shape of Q:", Q.shape)
```

8. To help visualize how a point relates to its neighbors in both the original and reduced space, we will use a function from the **modelviz** package:

```
from modelviz.relationships import plot_similarity
```

```
plot_similarity(data=X, point_of_interest=X[0],
```

```
std_range=3, similarity_color='black',
```

```
curve_color='grey', seaborn_style='whitegrid')
```

9. Step 8 produces the visualization as shown in *Figure 7.5*:

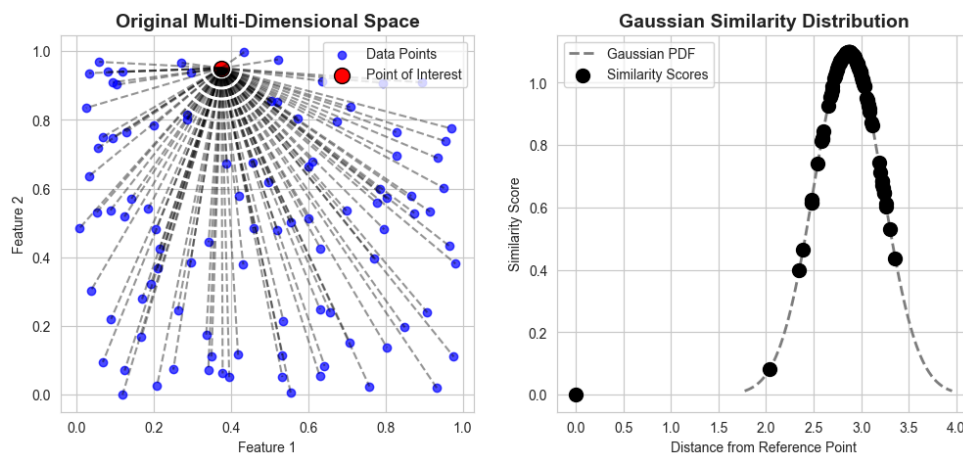


Figure 7.5: Gaussian similarity

Now that we have our pairwise similarities in Q and probabilities P, we will use KL divergence to measure this, and this is the loss function we will use to optimize.

Step 3 as Kullback-Leibler divergence

Kullback-Leibler divergence (KL divergence) is a method for measuring the difference between two probability distributions. It helps us quantify the *distance* between two distributions, but it is not an accurate distance metric because it is not symmetric (i.e., swapping the distributions gives a different result).

Imagine you have two probability distributions:

- P (the true distribution of some data)
- Q (an approximation or alternative distribution)

KL divergence measures how much information is *lost* when we use Q to approximate P. If Q is very close to P, the KL divergence is small. If Q is very different, the KL divergence is large. We will use it here to verify that our mapping from a high-dimensional space (*Step 1*), denoted as P, is like Q (*Step 2*), where we employed the t-distribution.

The loss equation for KL divergence is shown here:

$$C = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

We will continue with bringing these equations to life, using Python to highlight what is happening:

1. We define the KL loss function as:

```
def kl_divergence(P, Q):
```

```
    return np.sum(P * np.log((P + 1e-9) / (Q + 1e-9)))
```

2. We add a small constant to each side of the P and Q distributions to avoid the division by zero error, as some of these probabilities will be close to zero and will indeed be zero on the diagonal. The next step is to use this function against our Q and P distributions.
3. Now that we have defined the function, we will use it to calculate the difference between the two distributions. This loss will be important to track, as our goal in the next section is to reduce it:

`loss = kl_divergence(P=P, Q=Q)`

4. This results in a KL divergence loss of approximately 5, meaning the predictions are off from the true reference of P.

In the next section, we will utilize gradient descent to optimize the cost function, as shown in the loss equation for KL divergence.

Step 4 as gradient descent on our cost function

In this step, we move points one at a time to reduce the loss in the cost function in *Step 3*.

In this process, the following happens:

- We start with randomly placed points in 2-dimensions.
- We used our Q matrix to calculate low-dimension similarities. This was what we did in *Step 2*. This shows us how close the points are currently in 2D.
- This step computes gradients and formula nudges points closer or farther apart depending on their relationship in high dimensions.
- We move points slightly over each iteration to minimize the KL divergence and reduce the error between the Q and P distributions.
- The aim is for the positions to stabilize over time and reduce the error.

The following code shows how we would implement this in Python, step by step:

1. To start, we define a function that takes in parameters Y (the current 2D position of all points); P (the pairwise similarities from the high-dimensional space, computed in *Step 1*); Q (the pairwise similarities in low-dimensional space) and **learning_rate** (this controls how much we adjust the positions in each step, a higher value would make larger jumps, while a lower value makes smaller and more precise adjustments):

```
def update_Y(Y, P, Q, learning_rate=0.1):
```

2. We initialize an empty matrix called gradients, filled with zeros and matching the shape of Y, which represents the current positions of all points:

```
gradients = np.zeros_like(Y)
```

3. Loop over all points:

```
for i in range(Y.shape[0]):
```

4. This step calculates how far the current point is from every other point, giving us a set of vectors that describe its position in relation to the rest:

```
diff = Y[i] - Y
```

5. Next, we compute the gradient to figure out how each point should move, bringing it nearer to the points it is similar to and farther from those it is not:

```
grad = 4 * np.sum((P[i, :, None] - Q[i, :, None]) * diff * (1 + np.sum(diff ** 2, axis=1, keepdims=True)) ** -1, axis=0)
```

- a. **P[i,j]** is the true similarity between points i and j (from the high-dimensional space).
- b. **Q[i,j]** is the current similarity in 2 dimensions.
- c. We use **np.sum(dif ** 2, axis=1, keepdims=True)** to compute the squared Euclidean distance.

- d. We then scale the influence of distant points, which helps prevent larger jumps using $1 + \text{distance}^{**2}$.
 - e. The $** -1$ ensures the closer points have a stronger effect.
 - f. Summation over all the pairwise interactions is done by **np.sum(..., axis=0)**.
 - g. Multiply by 4 (The **factor of 4** comes from the mathematical derivation of t-SNE's gradient update).
6. We save the gradient for the current point in the gradients list. This shows how far and in which direction it should move:

```
gradients[i] = grad
```

7. Now we adjust each point's position in Y, using the gradient and **learning_rate** to control how big the move is:

```
Y -= learning_rate * gradients
```

8. As we keep updating the points over multiple steps, the 2D layout improves and starts to reflect the true relationships in the original data.
9. The full function is defined here:

```
def update_Y(Y, P, Q, learning_rate=0.1):
```

```
    gradients = np.zeros_like(Y)
```

```
    for i in range(Y.shape[0]):
```

```
        diff = Y[i] - Y
```

```
        grad = 4 * np.sum((P[i, :, None] Q[i, :, None]) * diff *
(1 + np.sum(diff ** 2, axis=1, keepdims=True)) ** -1, axis=0)
```

```
        gradients[i] = grad
```

```
    Y -= learning_rate * gradients
```

```
    return Y
```

10. Next, we will use the function to go over 500 iterations and store the loss results in a list:

```

ITERATION = 500
losses = []
iters = []
for idx, _ in enumerate(range(ITERATION)):
    Q = compute_q_dist(Y)
    Y = update_Y(Y, P, Q)
    loss = kl_divergence(P, Q)
    if idx % 50 == 0:
        print(f"\U0000231B ITERATION: {idx + 1} Loss:
{loss:.4f}")
        losses.append(loss)
        iters.append(idx + 1)

```

11. To visualize the outputs of those losses, stored in the empty list, we will define a visualization set of steps:

```

import seaborn as sns
plt.figure(figsize=(8, 5))
sns.lineplot(x=iters, y=losses, marker='o',
             color='blue',
             label="KL Divergence Loss")
plt.xlabel("Iteration")
plt.ylabel("Loss Value")
plt.title("KL (Cost) Function - minimization")
plt.legend()
plt.grid(True)
plt.show()

```

12. The outputs of *Step 10* show the iterations of the gradient descent method trying to find the optimal cost function:

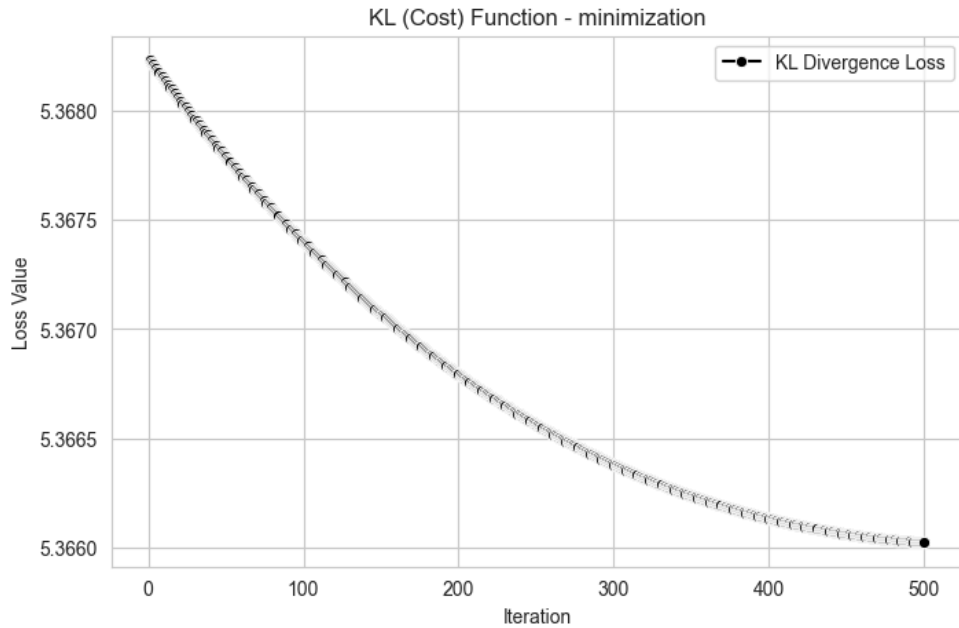


Figure 7.6: KL gradient descent

While we have walked through each step to help you understand how the algorithm works under the hood, it is worth noting that scikit-learn’s built-in methods are far more efficient and reliable for real-world use. These implementations are optimized to handle edge cases, tune parameters automatically, and minimize loss more effectively. In the MNIST case study later in this chapter, we will use scikit-learn’s version of t-SNE to apply what we have learned in a more practical way.

Uniform Manifold Approximation and Projection

UMAP is a powerful, efficient, and scalable algorithm for dimensionality reduction. It maintains both local and global data structures, serving as a faster and more efficient alternative to t-SNE. This efficiency stems from its process of constructing a k-nearest neighbors similarity graph, leveraging graph theory to determine edge weights between points, and performing an optimization step to project high-dimensional data into 2D or 3D spaces.

In the following sections, we will provide a high-level overview of these concepts. Due to the complexity of the graph-based algorithms, we will focus on the mathematical formulas in an abstract sense. We will utilize the

scikit-learn package to run UMAP, keeping the explanation at a higher level compared to t-SNE, as UMAP involves more sophisticated steps.

Step 1 as finding k-nearest neighbors

In a high-dimensional space, each data point is surrounded by others. UMAP connects each point to its closest neighbors using a similarity measure, like distance. These connections form a network, called a k-nearest neighbors' graph, which captures relationships between points.

For example, think of a social network. Imagine each person in a social network is linked to their 15 closest friends. The strength of each link indicates how similar they are, such as shared hobbies or mutual friends. UMAP utilizes this concept to create a map that preserves these connections while reducing the data to just two or three dimensions.

The equation for this similarity computation is illustrated here:

$$p_{ij} = \exp(-d(x_i, x_j)/\rho_i)$$

Let us understand this in detail:

- $d(x_i, x_j)$ is the distance between points x_i and x_j .
- p_i is the normalization factor to ensure neighborhood consistency.

Let us implement this simply with the UMAP package:

1. Generate our data, we will use the same method as with t-SNE here:

```
import numpy as np
```

```
import umap
```

```
from sklearn.metrics.pairwise import euclidean_distances
```

```
np.random.seed(42)
```

```
X = np.random.rand(100, 50)
```

```
distances = euclidean_distances(X, X)
```

2. We will now fit our UMAP model to the generated data, and use the example of 15 nearest neighbors, i.e., the 15 friends in our network example:

```
umap_model = umap.UMAP(n_neighbors=15,  
                        min_dist=0.1,  
                        random_state=42)
```

3. In the next step, we will construct the weighted graph and explain those steps.

Step 2 as constructing a weighted graph

UMAP constructs a graph-based representation of data by treating each data point as a node. It then establishes edges between nodes based on their similarity, with edge weights reflecting the strength of these relationships. The algorithm first creates a high-dimensional fuzzy topological representation of the data, capturing both local and global structures.

It then optimizes a lower-dimensional embedding by preserving the weighted relationships as much as possible, ensuring that points with strong connections remain close while maintaining overall data distribution. This process enables effective dimensionality reduction while preserving meaningful patterns in the data. The equation for this is here:

$$w_{ij} = \exp \left(-\frac{d(x_i, x_j)}{\sigma_i} \right)$$

Let us understand this in detail:

- w_{ij} is the weight of the edge between points x_i and x_j , representing their similarity.

- $d(x_i, x_j)$ denotes the distance between the two points in the original high-dimensional space.
- σ_i is a local scaling factor that adjusts for the density around x_i .

In Python, we will follow a couple of steps to repeat this process:

1. We will use our UMAP model, which we initialized in the first step, to fit the graph to the data:

```
graph = umap_model.fit(X)
```

2. We will store the KNN graph in its own variable, for later utilization:

```
knn_graph = graph.graph_
```

3. To inspect the size of the graph, we can print out the number of nodes (data points) and their connections (edges), which represent the relationships between each point and its nearest neighbors in the network:

```
print(f"Graph has {knn_graph.shape[0]} nodes and {knn_graph.shape[1]} edges")
```

4. This will print that the graph has 100 nodes and 100 edge relationships. The next step is to reduce these 100 edges down to 2 with dimensionality reduction; this will be the next step we cover in the following section.

Step 3 as performing dimensionality reduction

We now *squeeze* the high-dimensional graph into a 2D layout while preserving distances. To do this, we will use the following mathematical equation, to minimize the cross-entropy loss, as we learned about in *Chapter 4, Supervised Multi-class Classification*. Refer to the equation:

$$\mathcal{L} = \sum_{(i,j) \in \mathcal{E}} -\log(q_{ij}) + \gamma \sum_{(i',j') \notin \mathcal{E}} -\log(1 - q_{i'j'})$$

Components of the equation are explained as follows:

- **L**: Represents the total loss that UMAP aims to minimize during the dimensionality reduction process.
- **Σ (sigma) of i, j element ($\in$) of E**: Does a summation over all pairs of points (i, j) that are considered neighbors in the original high-dimensional space.
- **The $-\log(q_{ij})$** : Measures how well the pairs of points i and j are represented as neighbors in the low dimensional embedded space. Higher would indicate closeness, and this results in a reduced (lower) log value.
- **γ is a weighting factor**: It controls the relative contribution of the second term compared to the first, allowing a balance between the two objectives.
- **There is an inverse of Σ (i, j) not an element of E**: Which means sum all the pairs of points that are not neighbors, it is the inverse of the second bullet provided here.

The overall aim is to minimize loss in the lower-dimensional space and maintain meaningful relationships and closeness in the original structure.

To perform this with scikit-learn, we simply enact the **fit_transform** method:

1. Use the **fit_transform** as previously stated:

```
X_embed = umap_model.fit_transform(X)
```

2. We will confirm our dimensionality reduction has worked, by using the following:

```
print(f"New shape: {X_embed.shape}")
```

This will print the shape of **(100, 2)**, which is the nodes and features tuple we had previously, showing that the algorithm has effectively reduced it down to two dimensions.

Those are all the steps required to perform UMAP on our dataset effectively. In the practical section, we will apply this to the handwriting dimensionality reduction task.

Visualizing high-dimensional data in 2D and 3D

After covering dimensionality reduction, it is important to understand how further reducing data from 3D to 2D helps humans visualize patterns more effectively. While 3D projections offer additional spatial information, 2D visualizations are often cleaner, easier to interpret, and more suitable for presentation or analysis. One may ask, why reduce three-dimensional data to two-dimensional data for visualization?

Here are some points we have observed while presenting this type of information:

- **Human perception:** Our ability to understand spatial relationships is more intuitive in 2D than 3D projections.
- **Clarity and simplicity:** 2D plots avoid issues like occlusion, perspective distortion, and depth misinterpretation.
- **Ease of interpretation:** Comparing clusters, trends, and distributions is simpler in a 2D space.
- **Better computational handling:** 2D visualizations are faster to render and analyze in most tools.

We will look at how a very complicated dataset, that we will generate with `make_swiss_roll` from scikit-learn datasets (https://scikit-learn.org/stable/modules/generated/sklearn.datasets.make_swiss_roll.html), can be visualized in the Python steps. Let us look at them in detail:

1. Import the necessary packages:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import make_swiss_roll
```


2. Make a synthetic **swiss_roll** dataset with 1000 examples and a bit of random noise:

```
X_swiss, color = make_swiss_roll(n_samples=1000,  
                                noise=0.1,  
                                random_state=42)
```

3. Create a **matplotlib** chart to quickly visualize this data in three-dimensions:

```
fig = plt.figure(figsize=(8, 6))  
ax = fig.add_subplot(111, projection='3d')  
ax.scatter(X_swiss[:, 0],  
           X_swiss[:, 1],  
           X_swiss[:, 2], c=color,  
           cmap='Greys')  
ax.set_title("3-dimensional `swiss roll` dataset")  
plt.show()
```

4. This will generate a visual, as shown in *Figure 7.7*:

3-dimensional `swiss roll` dataset

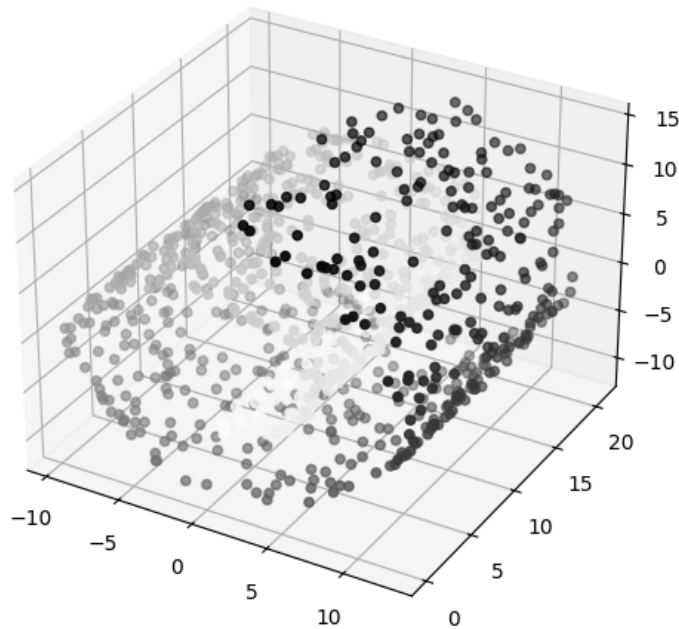


Figure 7.7: Three-dimensional visualization of high dimensional data

5. Create a UMAP model function to reduce the dimensionality, as we have previously delved into in the last section:

```
import umap

def apply_umap(X, **kwargs):
    reducer = umap.UMAP(**kwargs)
    return reducer.fit_transform(X)
```

6. Use our function to undertake dimensionality reduction on our dataset:

```
X_swiss_umap = apply_umap(X=X_swiss,
                            n_components=2,
                            random_state=42)
```

7. Finally, we visualize the results in two dimensions, because we set **n_components=2** earlier. This means the data was reduced to two main features for easier viewing:

```
plt.figure(figsize=(8, 6))
plt.scatter(X_swiss_umap[:, 0], X_swiss_umap[:, 1],
            c=color,
            cmap='Blues', edgecolors='k')
plt.title("Swiss Roll UMAP Projection")
plt.xlabel('Component 1')
plt.ylabel('Component 2')
plt.show()
```

8. The result, as shown in *Figure 7.8*, illustrates how UMAP effectively organizes the data by preserving both local and global relationships, revealing clear groupings within the dataset:

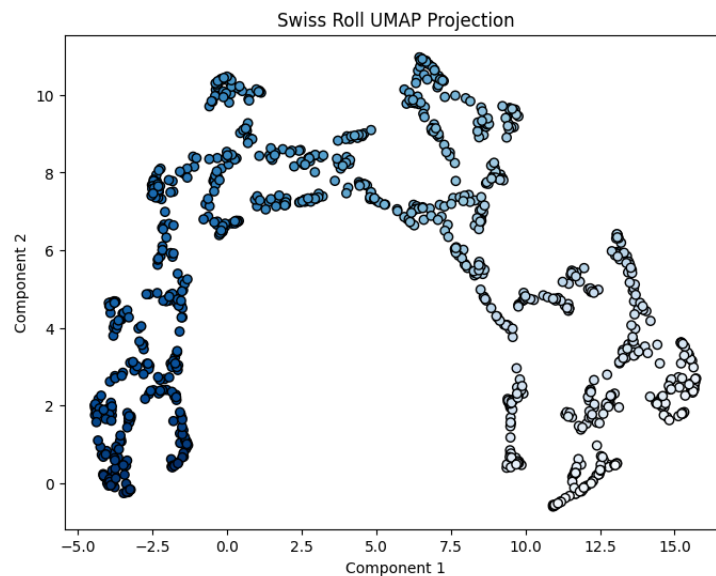


Figure 7.8: Two-dimensional representation of *swiss_roll* dataset

Figure 7.8 encompasses the following:

- **Unfolding the manifold:** The projection successfully unrolls the Swiss Roll, transforming its spiral structure into a flatter, more linear representation.
- **Preserved local structure:** Points that were close together in 3D remain close in 2D, indicating that UMAP effectively retains

relationships within the dataset.

- **Global structure:** Although some distortions may occur, the overall shape still follows a continuous pattern, indicating that UMAP preserves the broader trends within the data.

In the next section, we will examine the application of dimensionality reduction in Machine Learning and industry.

Applications of dimensionality reduction in machine learning

In machine learning, more data is not always better; it can become overwhelming. Dimensionality reduction simplifies complex datasets by removing redundant features while retaining essential information, leading to faster, more accurate, and interpretable models.

This is why it matters:

- **Feature selection:** Filters out irrelevant features, improving performance and clarity.
- **Visualization:** Enables plotting high-dimensional data in 2D or 3D (as shown earlier).
- **Efficiency:** Reduces computational cost, speeding up training and saving memory.
- **Prevents overfitting:** Reduces noise, helping models generalize better.
- **Anomaly detection:** Makes outliers more visible, aiding fraud detection and cybersecurity.
- **Recommendation systems:** Techniques like matrix factorization uncover hidden patterns in user preferences.
- **Bioinformatics and medical diagnosis:** Simplifies large genomic and imaging datasets to highlight critical biomarkers.

Here are some industry applications:

- **Finance:** Detects fraud by spotting unusual transaction patterns.
- **Healthcare:** Identifies disease markers and diagnoses via reduced medical data.
- **Retail:** Powers recommendation engines and customer behavior analysis.
- **Social media:** Analyzes trends and sentiment in vast text datasets.
- **Cybersecurity:** Detects network anomalies and threats more efficiently.

In short, dimensionality reduction helps extract meaningful insights, streamline models, and tackle high-dimensional data challenges across industries.

Applying dimensionality reduction to the MNIST dataset

In this section, we will apply our learning by tackling the Digit Prediction challenge, utilizing the MNIST dataset, a widely used collection of handwritten digits in machine learning. This dataset was instrumental in the development of LeNet-5, one of the pioneering convolutional neural network architectures designed by *Yann LeCun* and his colleagues for handwritten digit recognition.

Loading and preparing the MNIST dataset

In the series of steps that follows, we will get our imports, shuffle our data, and identify the unique class labels we will be working with:

1. Define our imports:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.decomposition import PCA
```

```
from sklearn.manifold import TSNE
```

```
import umap
```

```
from sklearn.datasets import fetch_openml
```

```
from sklearn.utils import shuffle
```

```
from sklearn.metrics import confusion_matrix
```

2. Fetch an OpenML (<https://openml.org/>) dataset for this task:

```
mnist = fetch_openml('mnist_784', version=1)
```

```
len(mnist)
```

3. Then, we will split out the feature set (X) and the target label (y):

```
X, y = mnist.data, mnist.target.astype(int)
```

4. Next, we shuffle the dataset to ensure the data is randomly distributed before splitting, which helps avoid any unintended patterns or ordering in the original data. We also set a **random_state** for reproducibility, so the results can be consistently replicated:

```
X, y = shuffle(X, y, random_state=42)
```

5. It is vital that we inspect our class labels to make sure we know what targets we are dealing with. we will use **np.unique** to get unique values in an array:

```
class_labels = np.unique(y)
```

```
class_labels
```

6. The class labels will print an array: **array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])**.
7. In the next stage, we will create a small function to visualize the MNIST digits using a small sample of our data.

Visualizing MNIST digits

The MNIST dataset consists of 70,000 grayscale images of handwritten digits (0 through 9). Each image is 28x28 pixels in size, resulting in 784 pixels per image. Each pixel represents a feature, with its value indicating the grayscale intensity ranging from 0 (black) to 255 (white). We will create a function to visualize these digits:

```
1. def plot_mnist_samples(X, y, num_samples=10,
2.                        cmap='Greys', figsize=(10,3)):
3.     fig, axes = plt.subplots(1, num_samples, figsize=figsize)
4.     for i, ax in enumerate(axes):
5.         ax.imshow(X[i].reshape(28, 28), cmap=cmap)
6.         ax.axis('off')
7.     plt.show()
```

This function displays a row of handwritten digit images from the MNIST dataset:

- **X** is the feature matrix containing flattened 28x28 pixel images.
- **y** is the label array, though it is not directly used in this visualization.
- **num_samples** determines how many digit images to display (default is 10).
- **cmap** sets the color map for the grayscale images (default is 'Greys').
- **figsize** controls the overall size of the plotted figure.
- The function creates a single row of subplots and loops through the first **num_samples** images, reshaping each one back to its original 28x28 pixel format for display. Axes are hidden for a cleaner look, and the images are shown side by side using **matplotlib**.

To make the plot visible, you need to use the following code:

```
plot_mnist_samples(X.to_numpy(), y)
```

This will output the graphic, as illustrated in *Figure 7.9*:

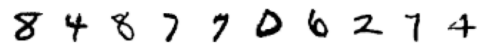


Figure 7.9: MNIST handwritten digits

In the next section, we will create another function for visualization, this time to visualize our dimensionality reductions from various models, i.e., PCA, t-SNE, and UMAP.

Creating our two-dimensional plotting function

The goal of our dimensionality reduction models is to project the data into two dimensions, not only to decrease variability and reduce the number of features, but also to allow us humans to visualize complex relationships on a simple two-dimensional plane.

The function included hereunder will allow us to visualize these relationships:

```
1. def plot_2D(X_proj, y,
2.             title,
3.             marker_size=5,
4.             col_pal='tab10',
5.             alpha=0.7,
6.             marker_scale=2, **kwargs):
7.     plt.figure(figsize=(10, 6))
8.     for key in ['s', 'alpha', 'hue', 'palette']:
9.         kwargs.pop(key, None)
10.    sns.scatterplot(x=X_proj[:, 0],
11.                  y=X_proj[:, 1],
12.                  hue=y, palette=col_pal,
13.                  s=marker_size,
14.                  alpha=alpha,
```



```
15.         **kwargs)
16.
17.     plt.title(title)
18.     plt.legend(loc='best', markerscale=marker_scale)
19.     plt.show()
```

In the following steps, we will split and scale our data, as we have done multiple times throughout this book.

Splitting and scaling the MNIST dataset

We aim to use the dimensionally reduced features from the MNIST dataset to build a multi-class classification model. This will help us evaluate whether the reduced features are effective in accurately predicting the digits in the test data, providing insight into the viability of building a classification model based on these reduced features.

1. Therefore, we will need a training and testing set for this purpose:

```
1. from sklearn.model_selection import train_test_split
2. X_train, X_test, y_train, y_test = train_test_split(X,y,
3.         test_size=0.8,
4.         random_state=42)
```

2. This gives us the necessary train-test splits.
3. Next, we will scale the features using **StandardScaler**, fitting and transforming the training data, then applying the same transformation to the test set:

```
1. from sklearn.preprocessing import StandardScaler
2. scaler = StandardScaler()
3. X_train_scaled = scaler.fit_transform(X_train)
4. X_test_scaled = scaler.transform(X_test)
```

We now have the features preprocessed and ready for initial exploratory reduction and visualization. We will then use the PCA method to select our optimal reduced features and fit an ML classification model.

PCA for visualization in two-dimensions

In the following steps, we will fit a PCA model to extract two principal components, use the **fit_transform** method to transform the training data, and then we will use the `plot_2D` visualization method we coded in the *Creating our 2-dimensional plotting function* section:

1. Use the PCA method from **sklearn.decomposition** module:

```
pca = PCA(n_components=2)
```

2. Use the **fit_transform** method on our scaled training features:

```
X_pca = pca.fit_transform(X_train_scaled)
```

3. The final step is to use the **plot_2D** function we created to generate a visualization:

```
plot_2D(X_pca,  
        y_train,  
        title="PCA on MNIST")
```

4. *Step 3* will generate the visualization, which shows the MNIST dataset projected onto two dimensions using PCA. Each point represents a handwritten digit, colored by its label:
 - a. As is visible, many of the points are tightly clustered and overlap significantly, making it difficult to clearly distinguish between different digit classes.
 - b. This is likely since we are only using two principal components, which may not capture enough of the variance and complexity inherent in the high-dimensional data.
 - c. While PCA helps reduce dimensionality and allows for visual exploration, reducing to just two components may not provide

sufficient separation for effective classification in this space; however, we will use it to show its initial attempts at generating a visualization, as shown in *Figure 7.10*:

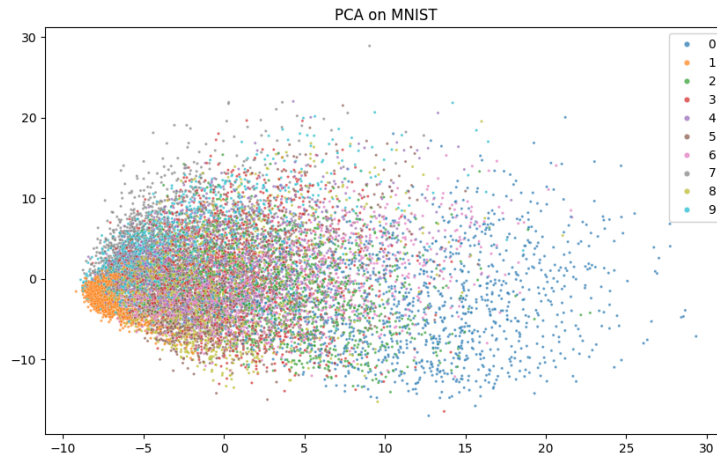


Figure 7.10: PCA on two principal components

t-SNE for visualization in two-dimensions

In this section, we will go through the same steps to generate this visualization as we did with the PCA steps:

1. In this t-SNE setup, we reduce the data to two dimensions (**n_components=2**), allowing for easy visualization of complex, high-dimensional data. The **perplexity** parameter, set to 25, balances how t-SNE considers local versus global aspects of the data; it effectively controls the number of nearest neighbors t-SNE focuses on when positioning points.

A higher perplexity captures broader structures, while a lower one emphasizes local detail.

The **learning rate**, set to 200, influences how quickly the algorithm adjusts point positions during optimization; too low can lead to poor convergence, while too high may cause instability.

Lastly, **random_state=42** ensures reproducibility by fixing the randomness in the algorithm's initialization:

```
tsne = TSNE(n_components=2,  
            perplexity=25,  
            learning_rate=200,  
            random_state=42)
```

2. We **fit_transform** the feature set:

```
X_tsne = tsne.fit_transform(X_train_scaled)
```

3. As before, we use our custom function to generate the plot:

```
plot_2D(X_tsne, y_train, "t-SNE on MNIST")
```

4. This generates the plot shown in *Figure 7.11*, and it clearly demonstrates that the t-SNE algorithm does a significantly better job at separating the digits into distinct, meaningful clusters compared to previous methods:

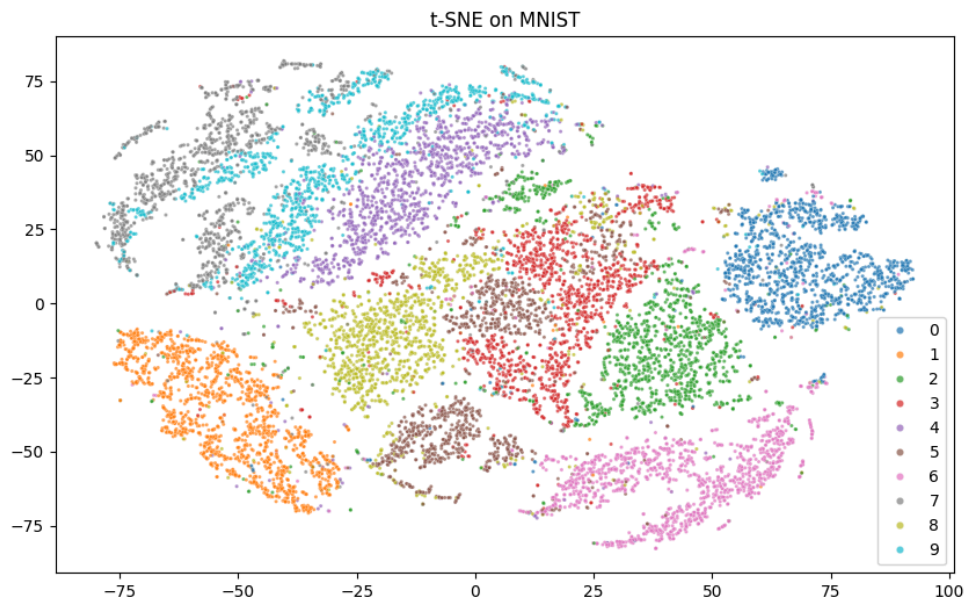


Figure 7.11: t-SNE on two components

For completeness, we will now look at the same visualizations using UMAP.

UMAP for visualization in 2-dimensions

We will use UMAP this time, in a stepwise fashion, as follows:

1. Import the **umap** package, and if you do not have this installed, use **pip install umap**:

```
import umap.umap_ as umap
```

2. Instantiate the **umap** function, as follows:

```
umap_reducer = umap.UMAP(n_components=2,  
                           random_state=42)
```

3. Transform the training feature set using **fit_transform**:

```
X_umap = umap_reducer.fit_transform(X_train_scaled)
```

4. Plot the transformed dataset:

```
plot_2D(X_umap, y_train, "UMAP on MNIST")
```

5. *Figure 7.12* shows how the UMAP maps the digits across two dimensions:

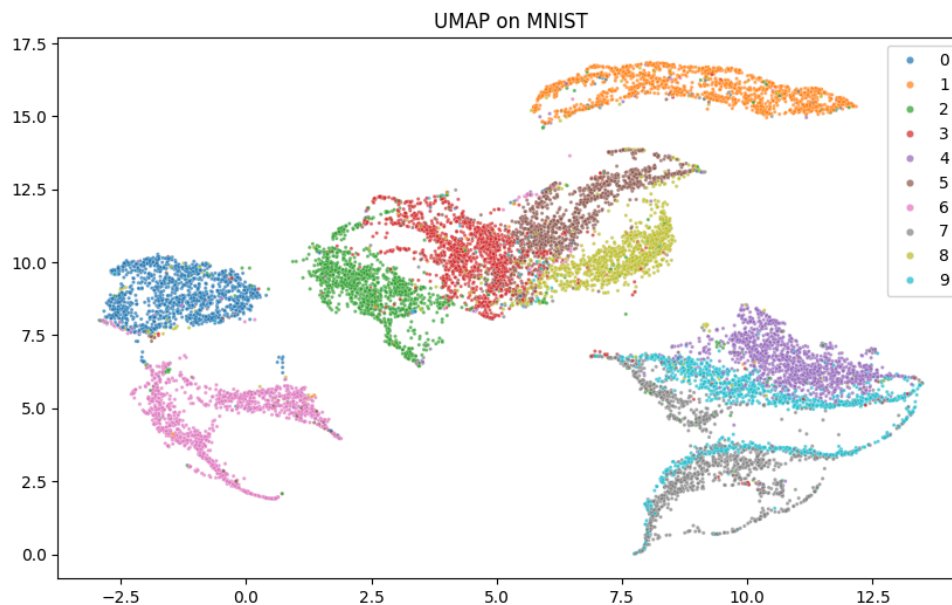


Figure 7.12: UMAP on two components

There are several dimensionality reduction algorithms available, each suited to different tasks. Techniques like t-SNE and UMAP excel at preserving

local structure and visualizing complex, non-linear relationships, but often require careful hyperparameter tuning and may not scale well to very large datasets.

In contrast, PCA performs exceptionally well when the relationships in the data are primarily linear and when interpretability and computational efficiency are priorities. To keep things simple and scalable, we will use PCA to determine the optimal number of components for reducing our feature set.

Automatically selecting optimal components

In this section, we will create a function that gets the optimal components. There are four important lines in this function that we will explain, and then we will include the entire function for clarity:

1. First, we will fit our PCA based on whatever X features are passed to the function:

```
pca = PCA(n_components=X.shape[1], **pca_args)
```

2. Followed by **fit_transform** on the X dataset:

```
pca_transformed = pca.fit_transform(X)
```

3. We retrieve the explained variance from the fitted PCA model to understand how much variance each individual component captures.

Since there are many components, we calculate the cumulative sum to see the total variance explained as more components are included, effectively showing how much information is retained as we add each one:

```
explained_variance_ratio = np.cumsum(pca.explained_variance_ratio_)
```

4. This line checks whether the cumulative explained variance has reached the desired threshold (such as 90% or 95%) for each principal component.

It generates a list of True or False values, where True indicates that the threshold has been met or exceeded at that point. Using **np.argmax**, we find the index of the first True value, which marks the earliest point where enough variance is captured.

Since Python indexing starts at zero, we add 1 to get the actual number of components needed. This value is stored in **optimal_components**, representing the minimum number of components required to retain the desired amount of variance:

```
optimal_components = np.argmax(explained_variance_ratio >=
variance_threshold) + 1
```

5. Here, we include the rest of the function for context:

```
def get_optimal_pca(X, variance_threshold=0.95,
```

```
figsize=(10, 5),
```

```
plot=True,
```

```
x_lab='No. of PCA Components',
```

```
y_lab='Cumulative Explained Variance',
```

```
plot_title='Optimal Explained Variance',
```

```
optimal_thresh_line_color='grey',
```

```
variance_line_color='black',
```

```
linestyle='--',
```

```
***pca_args):
```

```
pca = PCA(n_components=X.shape[1], **pca_args)
```

```
pca_transformed = pca.fit_transform(X)
```

```
explained_variance_ratio = np.cumsum(pca.explained_variance_ratio_)
```

```
optimal_components = np.argmax(explained_variance_ratio_
o >= variance_threshold) + 1
```

```
if plot:
```

```

plt.figure(figsize=figsize)
plt.plot(range(1, len(explained_variance_ratio) + 1),
         explained_variance_ratio,
         marker='o',
         linestyle='--',
         color='b')

plt.axhline(y=variance_threshold,
            color=variance_line_color,
            linestyle=linestyle,
            label=f"{variance_threshold * 100}% Variance")

plt.axvline(x=optimal_components,
            color=optimal_thresh_line_color,
            linestyle='--',
            label=f"Optimal Components: {optimal_components}")

plt.xlabel(x_lab)
plt.ylabel(y_lab)
plt.title(plot_title)
plt.legend()
plt.grid()
plt.show()

return (optimal_components, explained_variance_ratio, pca_transformed)

```

6. To use the function, we will assign the three returned values to separate variables using multiple variable assignment. We can also specify the desired cumulative variance threshold to determine the cutoff for selecting the number of components:


```
(opt_comp, explained_variance_ratio, _) = get_optimal_pca(  
    X_train_scaled,  
    variance_threshold=0.9)
```

7. Upon running *Step 6*, the visualization is generated, as shown in *Figure 7.13*, showing that the optimal components are 208, based on our variance threshold of 0.9:

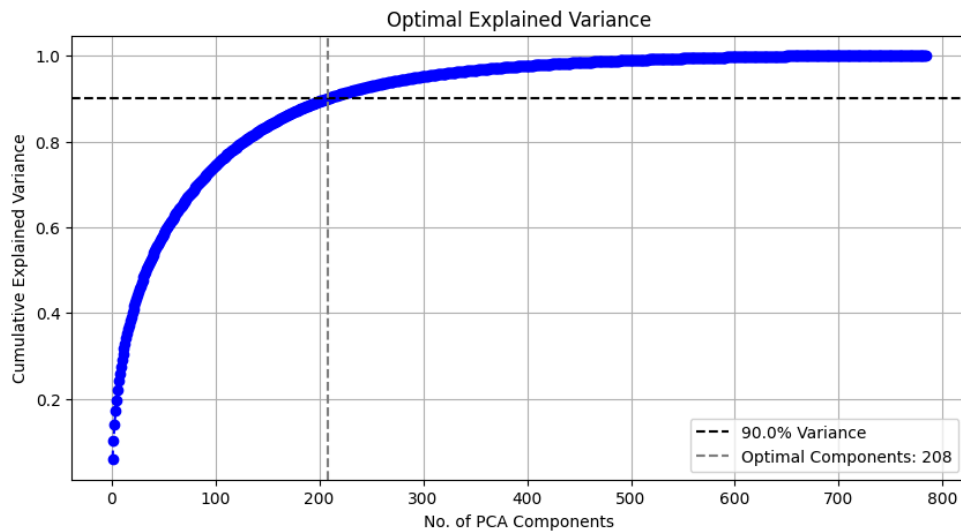


Figure 7.13: Optimal components of PCA

8. We tuple unpacked the variable **pca_optim** in *Step 6*; we will now use this variable to fit the PCA model with the optimal number of principal components:

```
pca_optim = PCA(n_components=opt_comp)
```

9. Then, we will **fit_transform** our scaled training set:

```
X_train_pca = pca_optim.fit_transform(X_train_scaled)
```

10. Finally, we transform our testing dataset:

```
X_test_pca = pca_optim.transform(X_test_scaled)
```

By completing these steps, we have successfully preprocessed the data and reduced it to the optimal number of principal components. Next, we will

use a **support vector machine (SVM)** to classify the digits based on these reduced pixel features.

Train SVM classifier to predict MNIST digits

For this task, we will use an SVM with a **radial basis function (RBF)** kernel, a regularization parameter C set to 10 to balance margin width and classification accuracy, and a gamma value of 0.01 to control the influence of individual training points on the decision boundary. SVMs are particularly effective for classification tasks like this because they focus on finding the optimal boundary (hyperplane) that best separates the classes. The model identifies key data points, known as support vectors, which lie closest to the decision boundary and have the most influence on its position, making SVMs robust even in high-dimensional or reduced feature spaces.

To fit the model, we will follow these steps:

1. Import the necessary package needed to do the SVM:

```
from sklearn.svm import SVC
```

2. Instantiate the SVM model with the parameters outlined in the initial description:

```
svm_model = SVC(kernel="rbf", C=10, gamma=0.01)
```

3. Fit out model to the training data:

```
svm_model.fit(X_train_pca, y_train)
```

To this point, we now have a trained ML classifier that can predict each of the 0-9 handwritten digits. The final step is to check how accurate the classifier is with new unseen data; for that, we will use our testing (hold-out dataset).

Evaluating the classifier

The remaining steps at this point are to evaluate the classifier on the testing dataset so that we will generate label predictions from the model on the test set, and then we will use a confusion matrix, loaded from **modelviz**

package (<https://pypi.org/project/modelviz/>), to produce the visualized confusion matrix. This will be achieved in the remaining steps:

1. We use the **predict** method to generate the model's predicted labels, based on the reduced feature set obtained from the optimal PCA components step, resulting in the final model predictions:

```
y_pred = svm_model.predict(X_test_pca)
```

2. To import the **plot_confusion_matrix** function, we will gather the necessary imports from the **modelviz** package:

```
from modelviz.confusion_matrix import plot_confusion_matrix
```

3. In this step, we will create the confusion matrix using the **sklearn.metrics confusion_matrix** function. Here, we will need to pass the ground truth labels and the predictions obtained from *Step 1*:

```
cm = confusion_matrix(y_true=y_test, y_pred=y_pred)
```

4. The last step is to use the visualization (displayed in *Figure 7.14*) to plot our multi-class confusion matrix. We will pass in the confusion matrix (created in *Step 3*), the class labels we created as a list at the beginning of this section, we will give the model a suitable name, alter the font sizes of all the figures on the plot, set a color palette and set the proportions on the confusion matrix as a color as well:

```
plot_confusion_matrix(cm=cm, classes=class_labels,  
                      model_name='MNIST Digit Accuracy',  
                      label_fontsize=8, annot_fontsize=8,  
                      cell_fontsize=8, table_fontsize=8,  
                      cmap='Greys',  
                      proportions_color='grey')
```

5. *Step 4* generates the plot as shown in the following figure:

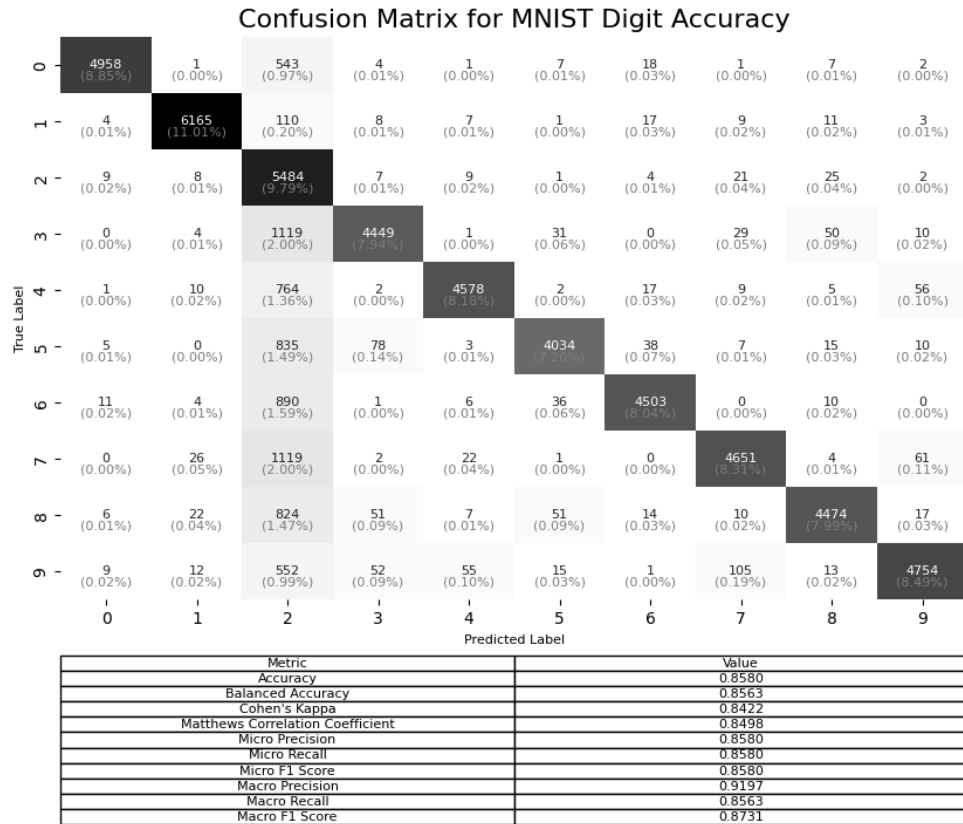


Figure 7.14: MNIST digit prediction confusion matrix

This graphic displays the confusion matrix and evaluation metrics for a digit classification model on the MNIST dataset, highlighting solid overall accuracy at 85.8%, with most digits correctly identified despite some misclassifications. Notably, the model achieved a macro precision of 91.97%, demonstrating strong accuracy across all digit classes, even in the presence of class imbalance.

Further improvements could be achieved by adjusting the optimal component reduction threshold and fine-tuning both the threshold and model hyperparameters.

We will leave you with the challenge of improving the accuracy presented in this chapter. These results are intended as a springboard for further exploration and refinement of the modeling approach.

Conclusion

In this chapter, we explored the fundamentals of dimensionality reduction, covering both linear methods, such as PCA, and non-linear techniques, including t-SNE and UMAP. These methods help simplify high-dimensional data, making it easier to visualize, interpret, and model. We applied each method to actual and simulated datasets, showing how they preserve structure while reducing complexity.

To put theory into practice, we utilized the MNIST handwritten digit dataset, reducing its 784 features to a smaller, optimal subset using PCA. We then trained an SVM on the reduced data, achieving an accuracy of over 85% and substantial precision across all digit classes. This demonstrated how dimensionality reduction can enhance both performance and efficiency in real-world tasks.

In the next chapter, we will shift our focus to time series data, where we will cover preprocessing, modeling, and evaluation strategies tailored to temporal patterns.

Points to remember

Here are some key points summed up from this chapter:

- **Applied case study, MNIST dataset:** We reduced the MNIST dataset's 784 features and trained an SVM, achieving over 85% accuracy with significantly fewer features.
- **Choosing optimal components:** Selecting the number of components based on explained variance ensures a balance between simplicity and information retention.
- **Dimensionality reduction for modeling:** Reducing features can decrease noise, speed up training, and improve model generalization, especially for classification tasks.
- **Dimensionality reduction in practice:** Techniques like PCA, t-SNE, and UMAP help simplify high-dimensional data, making it easier to visualize, interpret, and model.

- **Importance of standardization:** Scaling data before applying PCA is crucial, as unscaled features can dominate variance and distort results.
- **Non-linear methods for complex data:** t-SNE and UMAP uncover non-linear patterns in data, often revealing meaningful clusters that PCA might miss.
- **PCA versus t-SNE versus UMAP:** Use PCA for linear problems and scalability, t-SNE for detailed local structure, and UMAP for a balance between speed, clarity, and global structure.
- **PCA:** A linear method that transforms data to a new coordinate system, retaining maximum variance with fewer components for improved efficiency.
- **Visualizing high-dimensional data:** Projecting data into 2D or 3D using reduction techniques makes complex datasets more interpretable for human analysis.

Multiple choice questions

1. **What is the primary advantage of using t-SNE for dimensionality reduction?**
 - a. It guarantees linear separability in all datasets
 - b. It retains only a global data structure and ignores local patterns
 - c. It excels at preserving local relationships in high-dimensional data
 - d. It is faster and more scalable than PCA
2. **Which statement best describes how UMAP performs dimensionality reduction?**
 - a. It uses linear algebra to find principal components
 - b. It constructs a nearest-neighbor graph and optimizes layout in low dimension

- c. It removes outliers to improve visualization
 - d. It sorts features by correlation and drops the least useful ones
3. **In the MNIST case study, why was dimensionality reduction applied before training the classifier?**
- a. To convert images to grayscale
 - b. To increase the number of training examples
 - c. To balance the dataset
 - d. To reduce computational cost and improve model efficiency
4. **What type of data structure does UMAP use to model relationships between points?**
- a. Graph-based structure using k-nearest neighbors
 - b. Decision tree
 - c. Cluster centroid
 - d. Linear-based structure
5. **Why might 2D visualizations be preferred over 3D for high-dimensional data?**
- a. 3D visualizations always reveal more patterns
 - b. 2D is easier to interpret and avoids visual clutter
 - c. 3D plots require no preprocessing
 - d. 2D projections increase model accuracy
6. **Which of the following best describes the role of KL divergence in t-SNE?**
- a. It projects data into 3D for visualization
 - b. It is used to scale data before applying t-SNE
 - c. It identifies linear relationships between features

- d. It measures the difference between high- and low-dimensional similarity distributions
7. **Which dimensionality reduction method is most suitable when both local and global structures need to be preserved?**
- a. t-SNE
 - b. IMAP
 - c. PCA
 - d. UMAP
8. **Why is t-SNE often not used for very large datasets in production environments?**
- a. It lacks visualization capabilities
 - b. It is computationally intensive and scales poorly
 - c. It requires labeled data
 - d. It always results in poor accuracy
9. **What does the Swiss Roll example demonstrate in dimensionality reduction?**
- a. Non-linear methods like UMAP can "unroll" complex data structures into meaningful 2D spaces
 - b. Non-linear methods like PCA can "flatten" complex data into perfect clusters for modeling
 - c. Non-linear methods like t-SNE can "unroll" high-dimensional noise into strictly linear features
 - d. Non-linear methods like KMeans can "unwrap" tangled data into categorical labels automatically
10. **What is the main goal of gradient descent in machine learning?**
- a. Gradient descent finds the highest point on the error curve to improve variance

- b. Gradient descent cycles through random weights to enhance learning speed
- c. Gradient descent updates model parameters to minimize the loss function over time
- d. Gradient descent adjusts the model to increase error slowly for stability during training

11. What does the learning rate control in gradient descent?

- a. The size of each update step as the algorithm moves toward the optimal solution
- b. The number of iterations required to split the data into clusters
- c. The total number of parameters included in the model's structure
- d. The number of hidden layers in a neural network trained with reduced features

12. What does the perplexity parameter control in t-SNE?

- a. It adjusts the number of output dimensions after data is projected
- b. It balances attention between local and global data relationships during embedding
- c. It changes the angle of rotation for the reduced components in visualization
- d. It scales the input data before eigenvectors are calculated

13. What is the first computational step when performing PCA?

- a. Standardize the dataset to ensure features have zero mean and unit variance
- b. Create a nearest-neighbor graph to initialize the data space
- c. Convert the dataset to integer values for faster projection
- d. Generate a class probability matrix to prepare for reduction

Answers

1	c
2	b
3	d
4	a
5	b
6	d
7	d
8	b
9	a
10	c
11	a
12	b
13	a

Key terms

Some key terms used in this chapter are:

- **Covariance matrix:** Square matrix showing how features vary together, used in PCA to find directions of greatest variance.
- **Cumulative explained variance:** Total variance captured by chosen principal components in PCA; helps decide how many to keep.
- **Dimensionality reduction:** Techniques that reduce the number of features while preserving key data patterns.
- **Eigenvalues:** Values showing how much variance each principal component in PCA explains.
- **Eigenvectors:** Directions (principal components) along which data varies most in PCA.

- **Explained variance ratio:** Shows the proportion of variance each principal component captures in PCA.
- **Feature scaling:** Standardizing features so they contribute equally in models like PCA and SVM.
- **Gradient descent:** Algorithm that iteratively adjusts parameters to minimize loss, used in training models and t-SNE.
- **High-dimensional data:** Data with many features, making it harder to model or visualize.
- **KL divergence:** Measures difference between two probability distributions; used in t-SNE for structure comparison.
- **Local structure preservation:** Keeping nearby points close after dimensionality reduction, important for t-SNE and UMAP.
- **Manifold learning:** Techniques that find low-dimensional patterns within high-dimensional data.
- **MNIST dataset:** Dataset of 70,000 handwritten digit images, commonly used for ML experiments.
- **Nearest neighbors graph:** Connects each data point to its closest neighbors; used in UMAP to maintain structure.
- **Overfitting:** When a model learns noise instead of patterns, it can be reduced by dimensionality reduction or regularization.
- **PCA:** Linear method projecting data onto axes (principal components) that capture the most variance.
- **Projection:** Mapping high-dimensional data onto fewer dimensions, retaining key information.
- **Similarity distribution:** Shows how close data points are in high- or low-dimensional space, used in t-SNE and UMAP.
- **Standardization:** Scaling features to zero mean and unit variance before using PCA or similar models.

- **Swiss roll:** Synthetic 3D dataset used to demonstrate non-linear dimensionality reduction.
- **t-SNE:** Non-linear technique that places similar points close together in a lower-dimensional space.
- **UMAP:** Fast, non-linear dimensionality reduction that preserves both local and global data structure.
- **Variance retention:** Percentage of total variance kept by selected principal components in dimensionality reduction.

Join our Discord space

Join our Discord workspace for latest updates, offers, tech happenings around the world, new releases, and sessions with the authors:

<https://discord.bpbonline.com>



CHAPTER 8

Time Series with Scikit-learn

Introduction

In this chapter, we are getting hands-on with time series forecasting, working with data that unfolds over time, like monthly sales or passenger numbers. It is a bit different from regular machine learning, since the order of the data matters and past values influence future ones. We will cover everything from cleaning and prepping time-based data to building and comparing models like Holt-Winters, **Seasonal Autoregressive Integrated Moving Average (SARIMA)**, XGBoost, and Prophet. By the end of the chapter, you will know how to spot trends, handle seasonality, and make solid predictions you can trust.

Structure

This chapter covers the following topics:

- Introduction and challenges in time series data
- Loading and preparing time series data
- Preprocessing time series data
- Analyzing time series data
- Differencing to ensure stationarity
- Methods of splitting time series data
- Modeling and tuning time series models
- Evaluating time series models

Objectives

By the end of this chapter, you will be able to confidently work with time series data, from loading and cleaning it to spotting patterns like trends and seasonality. You will know how to handle missing values, apply key transformations, and prepare features for modeling. You will build and evaluate a range of forecasting models, from classic approaches like Holt-Winters and SARIMA to machine learning and Prophet, and understand when and why to use each one. We even create our own tuned SARIMA model that goes head-to-head with and beats Prophet in performance. Most importantly, you will be able to make solid future predictions and assess how reliable they really are.

Introduction and challenges in time series data

In an era where forecasting drives everything from business planning to infrastructure and environmental monitoring, time series data plays a central role in helping us understand and predict how things change over time. Whether it is monthly airline passenger volumes, stock prices, or web traffic, time series analysis is all about making sense of data collected at regular intervals and using that insight to forecast what comes next.

In this chapter, we take a practical look at both classical univariate models, like Holt-Winters and SARIMA, and modern machine learning methods such as random forests, XGBoost, and Facebook Prophet. Statistical models are great for capturing seasonality and trend in a clean, interpretable way, while ML brings flexibility and the ability to handle complex, nonlinear relationships. We will explore the strengths of each and see how they stack up against each other on real-world forecasting tasks.

Working with time series comes with its own set of challenges: missing data, seasonality, autocorrelation, and the need for stationarity. Throughout this chapter, we will tackle these head-on with clear examples, reusable code, and practical workflows. By the end, you will have a solid

understanding of how to wrangle, model, and evaluate time series data, and even build a tuned model that can outperform baseline tools like Prophet.

Loading and preparing time series data

Before any meaningful time series analysis can take place, it is essential to begin with a well-structured and clean dataset. In this section, we focus on preparing the classic Airline Passengers dataset, which records monthly totals of international airline passengers from 1949 to 1960. This dataset is a popular example in time series forecasting, known for its clear seasonal patterns and upward trend over time.

The first step involves loading the dataset and ensuring that the date column is properly parsed and set as a datetime index, which is crucial for time-aware operations such as resampling, lagging, and decomposition. We then remove redundant or non-informative columns, streamlining the dataset for univariate analysis.

While we note here that the dataset exhibits characteristics of a multiplicative time series, where seasonal fluctuations increase with the level of the series, we will explore this concept in more depth while analyzing time series in later sections. For now, the goal is to ensure our time series is clean, correctly indexed, and ready for preprocessing and modeling steps to follow.

Loading the dataset

As mentioned at the start of this section, we will be using the flight passenger's dataset, which includes monthly breakdowns of airline passengers between 1949 and 1960. This dataset is native to the seaborn package (<https://seaborn.pydata.org/>), and we will import it using the following steps:

1. Load our imports:

```
import pandas as pd
```

```
import seaborn as sns
```

2. Load the dataset from the seaborn library:

```
flights = sns.load_dataset("flights")
```

3. This loads the data, as displayed in *Figure 8.1*:

year	month	passengers
1949	Jan	112
1949	Feb	118
1949	Mar	132
1949	Apr	129
1949	May	121

Figure 8.1: Flight passengers time series dataset

4. The final step (optional) will be to rename the columns of the DataFrame:

```
flights.rename(columns={"year": "Year",  
                        "month": "Month",  
                        "passengers": "Passengers"},  
               inplace=True)
```

5. As we have used **inplace**, the original DataFrame will be altered.

In the next section we will look at the conversion of dates, as we need a Date Index to be able to work with time series data.

Wrangling the date

In this section, we will make use of Python's built-in datetime (<https://docs.python.org/3/library/datetime.html>) module, which provides a robust and efficient way to work with date and time data. Handling temporal information accurately is crucial in time series analysis, as it enables us to parse, format, and manipulate date-related values with precision. By leveraging the datetime package, we can convert string

representations of dates into proper datetime objects, extract useful components such as month and year, and prepare our data for time-aware operations such as indexing, resampling, and plotting. We will follow these steps to get up and running quickly:

1. Import the datetime package:

```
from datetime import datetime
```

2. Before we can convert our data into proper datetime format, we need to convert the abbreviated month names (e.g., 'Jan', 'Feb') in the **Month** column into their corresponding numeric values. To do this, we use a lambda function, a type of anonymous function in Python, along with **datetime.strptime()**, which allows us to parse the three-letter month abbreviations using the **%b** format specifier. This effectively translates each month name into its numerical equivalent (e.g., 'Jan' as 1, 'Feb' as 2), preparing it for integration into a full date object:

```
flights['Month'] = flights['Month'].apply(lambda x: datetime.strptime(x, '%b').month)
```

3. In the following step, we combine the **Year** and **Month** columns to create a unified **Date** field. Since this is monthly time series data, each date will represent the first day of the respective month, resulting in a format like 1959-01-01. While the month and year will change to reflect the correct time period, the day component will remain fixed at 01 to standardize the format:

```
flights['Date'] = pd.to_datetime(flights['Year'].astype(str) + '-' + flights['Month'].astype(str), format='%Y-%m')
```

4. Finally, we must set the **Date** as an index, for it to be used with time series models. To do this, we use the following code:

```
flights.set_index('Date', inplace=True)
```

5. As a sense check, we need to check the data type of the index of the DataFrame, to make sure it is a

pandas.core.indexes.datetimes.DatetimeIndex class. To do this, use the following code:

```
type(flights.index)
```

6. To tidy up the DataFrame, we will now drop the independent **Month** and **Year** columns, as we have combined these into our unified **Date** column, in the previous *Step 3*:

```
flights.drop(columns=['Year', 'Month'], inplace=True)
```

7. Now that the data has been processed and structured appropriately, we can save it in a format suitable for the next stage, preprocessing. Typically, this involves exporting the dataset as a **comma-separated values (CSV)** file, though other formats can be used depending on the workflow. To do this, we make use of the familiar **to_csv()** method, which has been consistently used throughout this book:

```
flights.to_csv('data/flights_prepared.csv')
```

In the next section, we will move on to preprocessing, focusing on filling missing values and resampling time series data. Since these steps were already introduced in *Chapter 1, Overview of NumPy and Pandas*, this will serve as a brief refresher rather than an in-depth walkthrough, to keep things concise and avoid repetition.

Preprocessing time series data

In this section, we simulate missing data within our time series by randomly setting a portion of the observations to NaN using NumPy. We then explore several methods for handling these gaps, including forward fill, backward fill, and more advanced techniques like Kalman filtering. Finally, we demonstrate how to resample the data to different time frequencies (such as quarterly and yearly) to prepare it for broader trend analysis and modeling.

Load time series dataset post cleaning

In the last section, we did some cleaning of the data; in this section, we will simply load that dataset into Python, using the following steps:

1. Use Pandas to load in the data:

```
flights = pd.read_csv('data/flights_prepared.csv')
```

2. Inspect the **head** of the DataFrame:

```
flights.head()
```

3. We can see the **Date** concatenation we did in the previous section has been successful, by inspecting the **Date** column in *Figure 8.2*:

Date	Passengers
1949-01-01	112
1949-02-01	118
1949-03-01	132
1949-04-01	129
1949-05-01	121

Figure 8.2: Modified Date field

4. The last step is to inspect how many records are in the dataset. We use the following code for this:

```
len(flights)
```

In the next section, we will begin by generating missing values within our dataset, followed by a summary of how much data is missing. We will then explore various techniques for imputing these gaps to ensure the time series remains consistent and ready for analysis.

Generating missing time series data points

In this section, we will use a bit of NumPy magic to generate missing values in our time series dataset. Let us understand, step by step, how this is achieved:

1. Import NumPy, as we are going to be working with this for this section:

```
import numpy as np
```

2. To ensure reproducibility, we will set a random seed (the value you pass can be changed to whatever seed you want to generate, but 42 tends to serve as a heuristic) before generating missing values. This guarantees that the missing entries in your dataset will match mine exactly when you follow along:

```
np.random.seed(42)
```

3. Introduce missing values by taking 15% of the dataset's total length. Here, we are saying add 15% of the data as missing Passenger counts:

```
num_missing = int(len(flights) * 0.15)
```

4. Pick random rows from the dataset where we will insert missing values, making sure each row is only picked once. We will use the **np.random.choice** method for this purpose:

```
miss_idx = np.random.choice(flights.index,  
                             size=num_missing,  
                             replace=False)
```

5. Finally, set the values in the selected rows to NaN, which marks them as missing data points:

```
flights.loc[miss_idx, "Passengers"] = np.nan
```

As the previous step added missing values, we will now look at strategies for missing value imputation in the following section.

Forward filling missing time series data

Forward fill is a simple method used to handle missing data in a time series. It works by carrying the last known (non-missing) value forward to fill any missing entries. This is particularly useful when you assume that the last observed value remains valid until a new value is recorded.

For example, if data is missing for March, but February has a known value, forward fill will use February's value to fill in March. We will perform forward filling in the next steps:

1. Generate a new DataFrame to hold the forward fill results:

```
flights_ffilled = flights.ffill()
```

2. Take the mean value of the data to determine how that has impacted the average (mean):

```
np.mean(flights_ffilled['Passengers'])
```

3. After applying forward fill, the mean number of passengers is 280, which matches the original dataset's mean before we introduced synthetic missing values. This suggests that forward fill does a reasonable job of estimating the missing data, at least in terms of preserving overall averages.

Backward filling missing time series data

Backward fill replaces missing values in a time series by using the next available non-missing value, essentially filling gaps by looking forward in time rather than backward, which can be useful in preprocessing when future values are assumed to already reflect the missing entries. To perform this, we will use the same stepwise process as we did with forward fill:

1. Use the **bfill** method to perform backfill:

```
flights_bfilled = flights.bfill()
```

2. Get the mean of this to see how it has affected the Passenger counts:

```
np.mean(flights_bfilled['Passengers'])
```

3. Backward fill results in two additional filled values compared to the original dataset and yields a mean passenger count of **282.36**, which is close to, but slightly higher than, the **282.32** mean observed after introducing the missing values. This slight deviation

suggests that backward fill may be less accurate than forward fill in approximating the original data distribution.

In the next section, we will look at how Kalman filtering can be applied to missing values replacement.

Kalman filtering for time series imputation

Unlike simple techniques such as forward or backward fill, Kalman filtering is a more advanced, model-based method used to estimate missing values in a time series. It belongs to a class of recursive filters that combine predictions from a model with actual observed data to estimate the true state of a system over time.

Kalman filters are particularly powerful in time series analysis because they account for both **system dynamics** (how the variable is expected to change) and **measurement uncertainty** (how noisy the observations are). When used for imputation, the Kalman filter leverages the structure of the data, such as trends or autocorrelations, to make more informed guesses about missing values. Kalman filtering has two main concepts:

- **Prediction step:** Estimate the current state based on the previous state and a model of the system's dynamics.
- **Update step:** Refine the prediction by incorporating new observations, balancing the model's expectations with actual data.

While the mathematical details of Kalman filtering involve complex equations and matrix operations, many statistical software packages provide built-in functions to perform these calculations. In this section, we will make use of the **pykalman** (<https://pypi.org/project/pykalman/>) package, and we will create a function that will do the replacement for you, but we will break down what the function does in the following steps:

1. Make a copy of the original flights DataFrame, so we do not inadvertently change any of the underlying data:

```
flights_kalman = flights.copy()
```

2. Import the package we will need for this:

```
from pykalman import KalmanFilter
```

3. Create a **kalman_imputer** function to perform Kalman Filtering. This function makes a copy of the data; finds where the missing values are, if there are missing values, it applies a Kalman Filter to learn from the existing (non-missing) data, and then it smooths the data, filling in the gaps (using **ffill**) based on estimated trends, and puts the filled values back into the original column:

```
def kalman_imputer(df,
                    column,
                    initial_state_mean = 0,
                    n_dim_obs=1, **kalmanargs):
    df = df.copy()
    df[column] = df[column].astype(float)
    missing_mask = df[column].isna()
    if not missing_mask.any():
        return df
    kf = KalmanFilter(initial_state_mean=initial_state_mean,
                      n_dim_obs=n_dim_obs, **kalmanargs)
    state_means, _ = kf.em(df[column].dropna()).smooth(df[column].ffill())
    df.loc[missing_mask, column] = state_means[missing_mask.to_numpy()]
    return df
```

4. To use the function, we pass in the DataFrame and the column to use for the imputation:

```
kalman_df = kalman_imputer(df=flights_kalman,
                            column='Passengers')
```

5. Taking the mean of the Kalman-filtered series shows results very similar to the forward fill method on this small dataset, but Kalman filtering tends to perform significantly better on larger or more complex time series where patterns and trends are more pronounced:

```
np.mean(kalman_df['Passengers'])
```

In the next section, we will get on to resampling time series, using a date index.

Resampling time series data

Resampling is the process of changing the frequency of your time series data, either by increasing it (up sampling) or decreasing it (down sampling). For example, you might resample daily data into monthly averages or convert monthly data into quarterly totals. This is particularly useful when you want to analyze trends at different time scales, reduce noise in high-frequency data, or align the dataset with the time resolution required by a specific model or business need.

Resampling can also help smooth out irregularities, reveal seasonal patterns, or simply make the data easier to visualize and interpret. In **pandas**, this is done using the **.resample()** method, often followed by an aggregation function like **.mean()**, **.sum()**, or **.count()** to define how values should be combined within each new time window. In the next steps we will play with our data to resample this in a couple of different ways, to get to grips with the concept by being hands on:

1. The initial step is to make sure the **Date** field is a **DateTimeIndex**, we can do this by using the following two lines:

```
flights["Date"] = pd.to_datetime(flights["Date"])  
flights.set_index("Date", inplace=True)
```

2. To check the index has been created successfully, we can use the **type** method, to check the Python class it belongs to. Here, you

should see the type is `<class 'pandas.core.indexes.datetimes.DatetimeIndex'>`. To do this, use the following code:

```
type(flights.index)
```

3. To perform quarterly resampling, we use the 'QE' frequency string in pandas, which stands for Quarter End. This groups the data into calendar quarters and aligns each period to the end of the quarter: March, June, September, and December, allowing for aggregation such as quarterly averages or totals:

```
flights_quarterly = flights.resample("QE").sum()
```

4. To undertake this with the year-end frequency string, we implement as the following step explains:

```
flights_yearly = flights.resample("YE").sum()
```

5. Execute these in the accompanying notebook, and you will observe the effects; the data is instantly resampled into a different aggregation form, without having to collect this data again.
6. The last step is to save the data again for the analyzing step of time series analysis; we use the forward filled DataFrame, with the relevant missing values replacement strategy applied, as it is the closest match to our original:

```
flights_ffilled.to_csv("data/flights_preprocessed.csv")
```

The next section will get into working with univariate time series analysis and the tools available at your disposal.

Analyzing time series data

In this section, we walk through the practical steps of analyzing a time series dataset using the airline passenger's data. We begin by visualizing the time series, then apply decomposition to separate it into trend, seasonality, and residuals. We check stationarity using statistical tests and apply

transformations like differencing and log scaling to help meet the assumptions of time series models.

We also explore **autocorrelation function (ACF)** and **partial autocorrelation function (PACF)** to understand temporal dependencies and create lagged features to prepare the data for use in machine learning models. Finally, we examine the difference between additive and multiplicative time series structures and save the fully processed dataset for later use.

Visualizing the time series

In this section, we will load the preprocessed dataset from the previous section and ensure that it has a proper datetime index, which is essential for time series analysis. If you would like to review the preprocessing steps in detail, please refer to the accompanying GitHub notebook: **08_03_Analyze_Timeseries.ipynb**. Additionally, included is a plotting function in this section that builds upon the basic line plot introduced in *Chapter 1, Overview of NumPy and Pandas*. Here, we will focus solely on using that function to visualize time series data. To use the visualization, we will import the custom module created with the function **plot_time_series** and pass in the flights data and the Passengers column:

1. Import the **plot_time_series** function from the folder that gets automatically built by the notebook:

```
from timeseries.viz import plot_time_series
```

2. Use the function by passing in the relevant DataFrame and passing the relevant column:

```
plot_time_series(data=flights, columns=["Passengers"],  
                 colors=['grey'], show_legend=True)
```

3. The chart in *Figure 8.3* shows the current Passenger volumes by month:

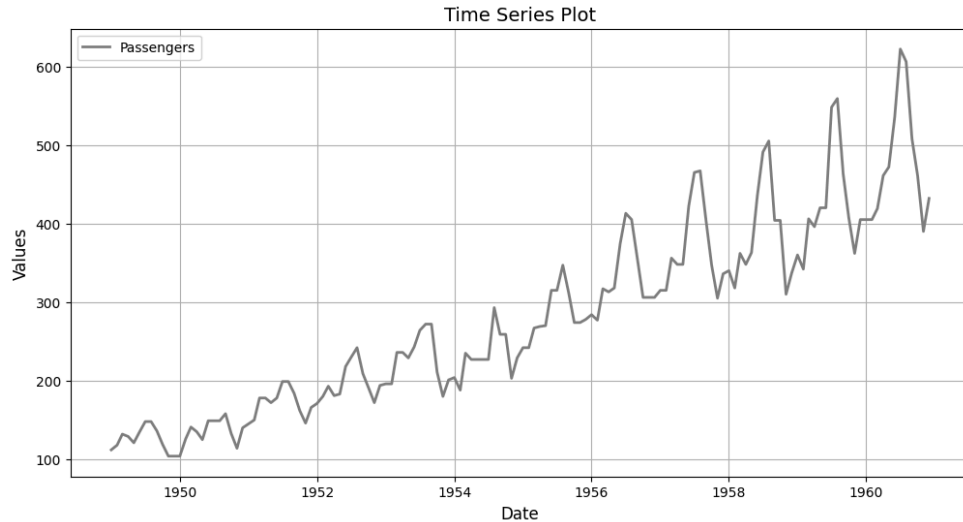


Figure 8.3: *Passengers time series plot*

4. The chart shows a clear upward trend in passenger volumes over time, reflecting a steady increase in air travel. Keep in mind that this data spans from the 1940s to the 1960s, a period when commercial air travel was still gaining popularity. Interestingly, this dataset follows a multiplicative pattern, meaning the seasonal variation grows alongside the overall trend, unlike the more common additive time series.

We will explore how to detect and work with multiplicative versus additive patterns in more detail in the next section on time series decomposition.

Time series decomposition

Before we dive into modeling time series data, it is essential to understand its underlying structure. Time series decomposition is the process of breaking down a time series into its core components, such as trend, seasonality, and residuals (noise). By separating these elements, we gain valuable insight into the patterns driving the data over time. This not only helps us better understand the behavior of the series but also improves the accuracy of forecasting models by allowing us to handle each component individually.

To perform decomposition, we will be using the **statsmodels** (<https://www.statsmodels.org/stable/index.html>) library, which offers specialized tools for time series analysis including classical decomposition, statistical testing, and forecasting methods. While libraries like scikit-learn are powerful for machine learning, they do not natively support time-aware features or temporal structures. statsmodels bridges that gap, giving us the statistical depth needed for interpreting and preparing time series data effectively.

In the following steps, we will perform time series composition on our data:

1. Import the **seasonal_decompose** function from **statsmodels** time series analysis (**tsa**) module:

```
from statsmodels.tsa.seasonal import seasonal_decompose
```

2. For this analysis, we will start by assuming a multiplicative trend, as the data clearly shows an upward trend over time. In a multiplicative model, the size of seasonal fluctuations grows in proportion to the overall trend. This means the seasonal spikes should increase alongside the trend. Let us decompose and find out:

```
decomp = seasonal_decompose(flights['Passengers'],  
                             model='multiplicative',  
                             period=12)
```

3. Running the decomposition will create a variable (e.g., `decomp`) that stores the results. You can access individual components—such as the trend, seasonal, or residual elements, using dot notation (e.g., **`decomp.seasonal`**). For simplicity, we will use **`decomp.plot()`** here to visualize all components at once in a single figure:

```
decomp.plot()
```

4. This creates the chart, as in *Figure 8.4*:

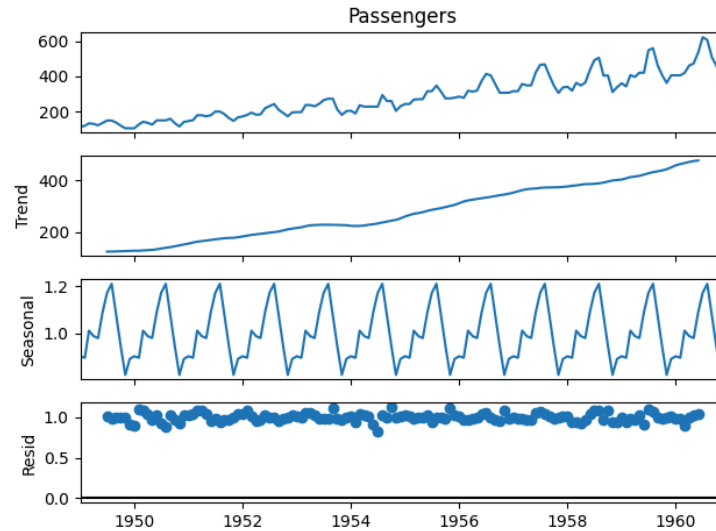


Figure 8.4: Decomposition plot

5. The decomposition plot illustrates a multiplicative time series, where seasonal variation scales with the trend. The original data shows an upward trend and yearly seasonal spikes, while the trend component reflects steady growth in passenger numbers from the late 1940s to 1960. Seasonal fluctuations increase alongside the trend, a key trait of multiplicative models. Residuals appear stable, suggesting the decomposition captures the trend and seasonality effectively.

In the next section, we will examine whether our time series data is stationary, as many univariate forecasting models rely on stationarity to produce accurate and reliable results.

Stationarity checks

A prior step, before applying many time series models such as **Autoregressive Integrated Moving Average (ARIMA)** (we will get into ARIMA in detail in the modeling section), it is important to determine whether the data is stationary. A stationary time series has a constant mean, constant variance, and no long-term trends over time. To check for stationarity, we use the **Augmented Dickey-Fuller (ADF)** test, a statistical test that helps us identify whether a **unit root** (A unit root means a time

series is unstable and does not return to a steady level after changes. If a series has a unit root, past shocks keep affecting future values. To fix this, we often take differences between data points to make the series stable) is present in the series, which would indicate non-stationarity.

Mathematically, the ADF test is based on the following regression equation:

$$\Delta y_t = \alpha + \beta t + \gamma y_{t-1} + \delta_1 \Delta y_{t-1} + \dots + \delta_p \Delta y_{t-p} + \varepsilon_t$$

To explain the equation:

- Δy_t is the first difference (we will get on to differencing shortly, so do not worry if you do not know what this is yet).
- α is a constant (the intercept term).
- βt is the time series trend.
- γ is the coefficient we test to determine stationarity.
- ε_t is the **white noise** (white noise is a random series of values with no pattern, where each point is completely independent of the others and has constant average and variance over time).

This is a statistical test to confirm a *null* and *alternate* hypothesis: the null would be non-stationarity, and the alternate would be a stationary time series. We use a p-value (measures the likelihood that your results happened just by chance) to determine whether there is statistical significance in the data. If the resulting p-value is below a chosen significance level (commonly 0.05), we reject the null hypothesis and conclude that the series is stationary.

We do not have to implement this from scratch, and we can use the **statsmodels** package to do the heavy lifting for us:

1. Import the **adfuller** module:

```
from statsmodels.tsa.stattools import adfuller
```

2. Use the function and pass in the Passengers column:

```
result = adfuller(flights['Passengers'].dropna())
```

3. To obtain the ADF statistic and p-value, we pass these lines of code:

```
print(f'ADF Statistic: {result[0]}')
```

```
print(f'p-value: {result[1]}')
```

4. These combined tell us that the data is non-stationary, a way to check for this programmatically would be to assess if our p-value is less than some significance value, say 5%:

```
if result[1] < 0.05:
```

```
    print("The time series is stationary.")
```

```
else:
```

```
    print("The time series is non-stationary.")
```

5. The printed result from *Step 4* indicates that the time series is non-stationary.

Now that we have confirmed the data is non-stationary, the next step is to transform it into a stationary series. One of the most common techniques for doing this is called differencing, which we will explore in the following section.

Differencing to ensure stationarity

Differencing is a key technique for transforming a non-stationary time series into a stationary one, an essential step for many forecasting models. First-order differencing removes linear trends by subtracting each value from its previous one. If a single pass is not enough (e.g., the trend is nonlinear), second-order differencing can be applied to further stabilize the series. In the next section, we will apply both, first-order and second-order differencing and see how they help prepare the data for modeling.

Later, we will re-run the ADF test to confirm whether the series has become stationary. While many libraries handle differencing automatically behind

the scenes, it is valuable to understand the process manually, especially if you ever need to implement or fine-tune a model from scratch. Here is how to do it:

1. We use the **.diff()** function to calculate the difference between each value and its preceding value in the time series, which helps in removing trends and making the data more stationary:

```
flights['Diff1'] = flights['Passengers'].diff().dropna()
```

2. To apply second-order differencing, we take the difference of the already differenced series. In other words, we are calculating the change in the rate of change, which helps remove more complex trends. This can be done by chaining **.diff()** twice, like so:

```
flights['Diff2'] = flights['Passengers'].diff().diff().dropna()
```

3. We will take the **head()** of the DataFrame to inspect what has happened:

```
flights[['Passengers', 'Diff1', 'Diff2']].head()
```

4. This produces the DataFrame, as in *Figure 8.6*:

	Passengers	Diff1	Diff2
Date			
1949-01-01 00:00:00	112.000000	nan	nan
1949-02-01 00:00:00	118.000000	6.000000	nan
1949-03-01 00:00:00	132.000000	14.000000	8.000000
1949-04-01 00:00:00	129.000000	-3.000000	-17.000000
1949-05-01 00:00:00	121.000000	-8.000000	-5.000000

Figure 8.5: First-order and second-order differencing

5. To visualize how this affects the data, we can use **matplotlib** to produce a chart showing the differences; this will be implemented as follows:

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(10,5))
```



```
plt.plot(flights['Diff1'],
        label="First-order Differenced Data",
        color='black', linestyle='-')

plt.plot(flights['Diff2'],
        label='Second-order Differenced Data',
        color='grey', linestyle='-.', alpha=0.3)

plt.legend()

plt.title('Differencing on Passengers dataset')

plt.show()
```

6. When running this code, the output is like *Figure 8.6*:

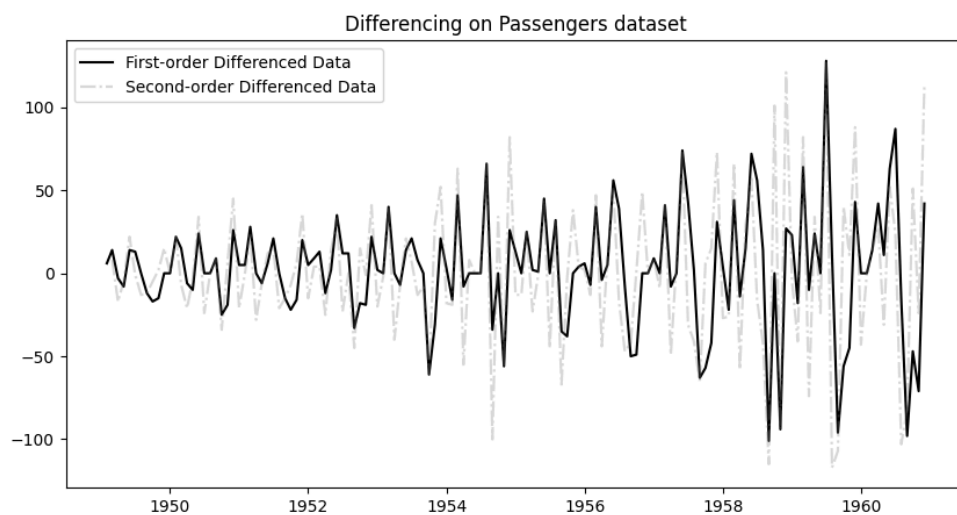


Figure 8.6: First-order and second-order differencing visual

To confirm whether our differencing has successfully made the time series stationary, we will need to run the ADF test again. Before that however, we will take a brief detour to explore log transformations and how they can help stabilize variance in time series data.

Detour into log transformations

In time series analysis, one common issue, especially in datasets with strong upward trends, is non-constant variance, also known as **heteroscedasticity**.

This means the spread of the data increases over time, which can distort forecasting models and reduce their accuracy.

A way to address this is to apply a log transformation; by applying the natural logarithm to the data, we compress large values more than smaller ones, which helps to stabilize variance across the time series; transform a curved (exponential) trend into a straight (linear) line, using the natural logarithm and prepare the data for models like ARIMA, which assume there is a constant variance. To apply a log transformation, we will rely on NumPy and its plethora of mathematical functions, shown as follows:

1. Create a column called log in the original DataFrame, this will display the log transformations alongside the differencing we did earlier:

```
import numpy as np
```

```
flights['log'] = np.log(flights['Passengers'])
```

2. Furthermore, it is possible to apply log transformation and differencing, as follows:

```
flights['log_diff'] = flights['log'].diff().dropna()
```

3. Printing the head (**flights.head()**) of our DataFrame, we can see the log transformation and log differencing have been applied:

	Passengers	Diff1	Diff2	log	log_diff
Date					
1949-01-01 00:00:00	112.000000	nan	nan	4.718499	nan
1949-02-01 00:00:00	118.000000	6.000000	nan	4.770685	0.052186
1949-03-01 00:00:00	132.000000	14.000000	8.000000	4.882802	0.112117
1949-04-01 00:00:00	129.000000	-3.000000	-17.000000	4.859812	-0.022990
1949-05-01 00:00:00	121.000000	-8.000000	-5.000000	4.795791	-0.064022

Figure 8.7: Log and log differencing DataFrame

Confirming stationarity post differencing

In this section, we will create a reusable function to streamline the ADF testing process. We will then apply it to the first-order, second-order, and log-differenced versions of our data to verify whether each transformation has successfully achieved stationarity:

```
1. def stationarity_check(data: pd.DataFrame,
2.                          column:str,
3.                          conf_level:float = 0.05,
4.                          **kwargs):
5.     adf_test_dict = dict()
6.     adf_test = adfuller(data[column].dropna(), **kwargs)
7.     if adf_test[1] < conf_level:
8.         result = "Stationary series"
9.     else:
10.        result = "Non-Stationary series"
11.    adf_test_dict = {
12.        "ADF Stat" : adf_test[0],
13.        "p-value": adf_test[1],
14.        "TS Type": result
15.    }
16.    return adf_test_dict
```

This function:

- Creates an empty dictionary namely **adf_test_dict**.
- Uses the **adfuller** method, on the DataFrame and column supplied.
- Checks the **p-value**, as we did previous, to make the data is stationary, or non-stationary.
- Once the process has been completed, the results are passed as values, alongside the associated keys in the dictionary.

- To use this function, we simply create an instance of the function and pass in the data we want to use, and the column we want to perform the ADF test on:

1. `stationarity_check(data=flights, column="Diff2")["TS Type"]`

By accessing the **TS Type** key from the results dictionary, we get a textual summary of the outcome. In this case, it returns **Stationary series**, confirming that second-order differencing successfully transformed the time series into a stationary one.

Detecting autocorrelation and partial autocorrelation

When working with time series data, understanding the relationship between current values and their past values is essential. This is where ACF and PACF come in, these tools will aid in uncovering patterns and dependencies over time.

Autocorrelation function

The **autocorrelation function (ACF)** measures the correlation between a time series and its own past values (lags). For example, it can tell us how strongly the value today is related to the value one period ago, two periods ago, and so on. A high autocorrelation at a certain lag suggests that past values still have some value (correlation) with the current value. This function is useful in identifying repeating patterns (seasonality) and is pivotal in estimating the **moving average (MA)** values of ARIMA.

Partial autocorrelation function

The **partial autocorrelation function (PACF)** goes a step further by measuring the correlation between a time series and a specific lag, after removing the effects of earlier lags. In simpler terms, it shows the direct relationship between a time point and its lag, without the influence of anything in between. It helps you see how much a past value really matters on its own.

This is useful for understanding which lags have a meaningful influence on the current value and understanding the **autoregressive (AR)** component of ARIMA. The only bit we are missing is the I, which stands for Integrated (which specifies what order differencing needs to be applied).

ACF and PACF both have an important role to play and aid with:

- Diagnosing patterns in time series data
- Choosing the right parameters for ARIMA models
- Understanding the structure of the time series data

The most effective way to analyze these values is through visual plots, which we will create using the passengers data we have been working with throughout this chapter. To generate these visualizations, we will follow a few key steps outlined as follows:

1. Import the **tsaplot** from the **graphics** package of **statsmodels**:

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
```

2. Create a matplotlib method to analyze the juxtaposed ACF and PACF:

```
import matplotlib.pyplot as plt
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
plot_acf(flights['Passengers'].dropna(), lags=50, ax=axes[0])
axes[0].set_title("Autocorrelation (ACF)")
plot_pacf(flights['Passengers'].dropna(), lags=50, ax=axes[1])
axes[1].set_title("Partial Autocorrelation (PACF)")
plt.tight_layout()
plt.show()
```

3. This creates the plot shown in *Figure 8.8*:

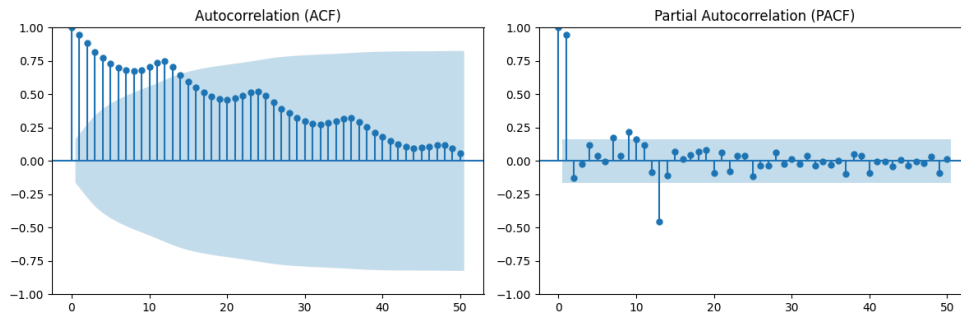


Figure 8.8: ACF and PACF plot

4. We will analyze the left ACF plot first:
 - a. Significant autocorrelation at lag 1, and it slowly declines over many lags.
 - b. The gradual, exponential decay is a classic sign of a non-stationary series.
 - c. Strong positive autocorrelation up to around lag 15-20, indicating persistence in the data (values are highly dependent on previous ones).
 - d. Moving average component may be needed, at lower lags.
 - e. Suggests a seasonal structure repeating at every 12 lags, meaning yearly seasonality.
5. Analyzing the PACF plot, we conclude:
 - a. A strong spike at lag 1, followed by a sharp drop off.
 - b. Lags 1, 9 and 12 are the lags that display statistical significance.
 - c. This is typical of a process with a strong autoregressive (AR) component of order 1 or 2.
 - d. The sharp cutoff after lag 1 suggests an AR(1) or AR(2) model may be a good fit.
 - e. Would be best modeled with SARIMA.

In the next step, we will focus on creating lagged features for use in machine learning models. The insights from the ACF and PACF plots can

guide us in selecting the most relevant lags to include as features, helping the model capture temporal patterns more effectively.

Creating lagged features for machine learning models

Working with the first step, we will focus on creating lagged features for use in machine learning models. The insights from the ACF and PACF plots can guide us in selecting the most relevant lags to include as features, helping the model capture temporal patterns more effectively. To create these features and append them to the time series dataset we have been working with, we will create a function and then use this function, with the specified lags, in the following steps:

1. Create a function that uses `.shift()` to generate lagged features based on a list of integers. Use the **typing** (<https://docs.python.org/3/library/typing.html>) module to enforce that **lag_periods** is a **List[int]** for proper type checking, i.e., a list of integer lag values:

```
from typing import List
```

```
def create_lagged_columns(df: pd.DataFrame,
```

```
                        column:str,
```

```
                        lag_periods:List[int]):
```

```
    for lag in lag_periods:
```

```
        df[f'Lag_{lag}'] = df[column].shift(periods=lag).fillna  
(value=0) return df
```

2. To use this function, pass the DataFrame along with the time series column name and a list of lag periods to generate new columns. The selected lags are commonly beneficial for machine learning modeling, but you can also use ACF and PACF to determine specific lags:

```
create_lagged_columns(df=flights,
```

```
                    column='Passengers',
```

```
lag_periods=[1,2,3,12,24,36])
```

3. You should now have a DataFrame that contains our differenced data, log transformations, log-differenced transformations, and columns with lags, as per *Figure 8.9*:

	Passengers	Diff1	Diff2	log	log_diff	Lag_1	Lag_2	Lag_3	Lag_12	Lag_24	Lag_36
Date											
1949-01-01 00:00:00	112.000000	nan	nan	4.718499	nan	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
1949-02-01 00:00:00	118.000000	6.000000	nan	4.770685	0.052186	112.000000	0.000000	0.000000	0.000000	0.000000	0.000000
1949-03-01 00:00:00	132.000000	14.000000	8.000000	4.882802	0.112117	118.000000	112.000000	0.000000	0.000000	0.000000	0.000000
1949-04-01 00:00:00	129.000000	-3.000000	-17.000000	4.859812	-0.022990	132.000000	118.000000	112.000000	0.000000	0.000000	0.000000
1949-05-01 00:00:00	121.000000	-8.000000	-5.000000	4.795791	-0.064022	129.000000	132.000000	118.000000	0.000000	0.000000	0.000000

Figure 8.9: Final passengers DataFrame

4. The last step is to save our transformed dataset, as we will be using this in the next section:

```
flights.to_csv('data/flights_decomp.csv')
```

In the next section, we will explore how to split time series data into training and testing sets which is a crucial step for validating forecasting models effectively over time.

Methods of splitting time series data

In this section, we will look at two common methods for splitting time series data: hold-out splitting and rolling forecasting origin (using **TimeSeriesSplit**). Unlike random train-test splits used in typical machine learning tasks, time series splits must preserve the chronological order of data to avoid data leakage. The functions we define here will help prepare your time series for proper validation and modeling, whether you are doing a simple hold-out test or rolling evaluations across multiple forecasting windows.

To get started, we will need to load our data back in and make sure it is processed as a time series index, by following these steps:

1. Load in our data, assuming you have already imported pandas:

```
flights = pd.read_csv('data/flights_decomp.csv',
```



```
parse_dates=['Date'],  
index_col="Date")
```

2. Cast the index to a DateTime format:

```
flights.index = pd.to_datetime(flights.index)
```

In the next section we will explore the hold out split, and we will create a function you can use to simply do this in your projects.

Hold-out split

A hold-out split is one of the simplest and most used methods for evaluating model performance in time series models. This involves splitting the data into two sets:

- A training set, containing the earlier portion of the series
- A test set, containing the most recent observations

This approach respects the chronological order of the data, which is critical in time series analysis to avoid future information leaking into the past. This is the technique we will be using to perform the evaluation of our modeling steps later, but we will touch on **time series cross-validation (TSCV)** as well. To enable us to perform this, a function has been included, which performs the split, based on a proportional split:

```
1. def hold_out_split(series, test_size:float=0.2):  
2.     split_idx = int(len(series) * (1 - test_size))  
3.     train, test = series[:split_idx], series[split_idx:]  
4.     return train, test
```

To use this function with our data, we will go through the following steps:

1. Create the variables to store the results and use the function with the desired split proportion:

```
holout_train, holout_test = hold_out_split(flights, test_size=0.2)
```

2. To test that the split has been successful, you can check the length of the data, this should return a tuple of size **(115, 29)**:

```
len(holout_train), len(holout_test)
```

3. It is good practice to check the maximum value in the training set and the minimum value in the test set to ensure the dates follow consecutively. The expected result should look like **Timestamp('1958-07-01 00:00:00'), Timestamp('1958-08-01 00:00:00')**:

```
max(holout_train.index), min(holout_test.index)
```

4. Create a directory to save the splits to, as we will be using this data again in the modeling step:

```
SAVE_DIR = 'data/splits'
```

```
os.makedirs(SAVE_DIR, exist_ok=True)
```

5. Save our files to the newly created directory, in the path specified in **SAVE_DIR**:

```
holout_train.to_csv(f'{SAVE_DIR}/holout_train.csv', index=True)
```

```
holout_test.to_csv(f'{SAVE_DIR}/holout_test.csv', index=True)
```

In the next step, we will use TSCV to create the splits.

Time series cross-validation

When working with time series data, it is essential to evaluate models in a way that respects the chronology and ordering of the time index. Unlike standard cross-validation methods, which randomly shuffle data, TSCV preserves the temporal structure by ensuring that the training set always comes before the test set.

One common approach to TSCV is the **rolling forecasting origin**, also known as **expanding window validation**. In this method, the training data

grows with each split, while the test data slides forward in time. This setup closely mirrors real-world forecasting, where each new prediction is made based on all available historical data.

To visualize how this rolling forecast origin works, see *Figure 8.10*:

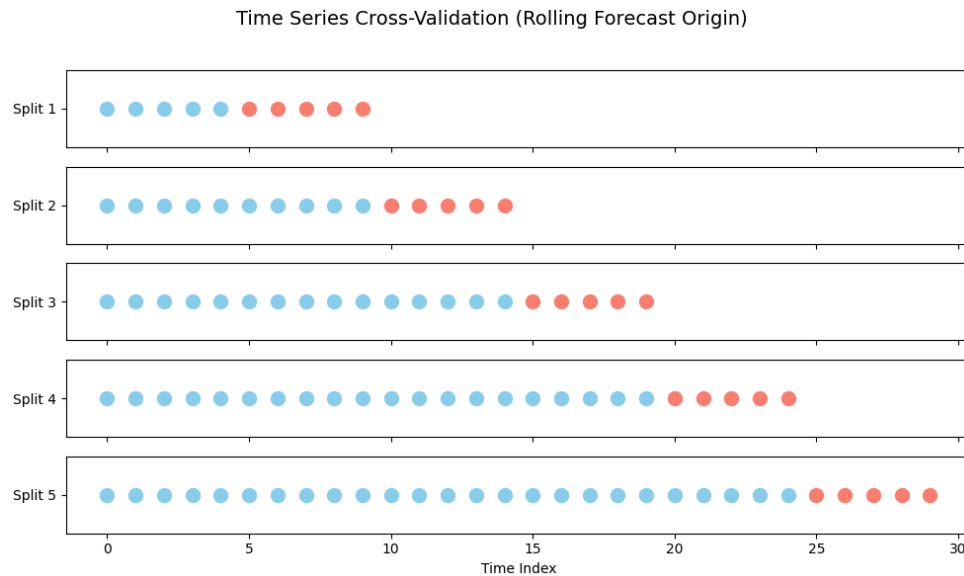


Figure 8.10: Rolling forecast origin

To create a rolling forecast origin, we will define a function to do this for us:

```
1. def rolling_forecasting_origin(series, n_splits:int=5):
2.     tscv = TimeSeriesSplit(n_splits=n_splits)
3.     splits = []
4.     for train_index, test_index in tscv.split(series):
5.         train, test = series.iloc[train_index], series.iloc[test_index]
6.         splits.append((train, test))
7.     return splits
```

The function uses the following:

- Takes the time series and the number of folds you want to create.

- Uses **TimeSeriesSplit** to create rolling (expanding window) train/test splits.
- Creates an empty list, extracts the train and test parts using index positions and then adds each pair to the list.
- Returns a list of all train-test split tuples.

In the following steps, we will apply this function to our dataset to generate and save the relevant data splits for future use in our modeling workflow. While this chapter focuses on using a simple hold-out split, switching to TSCV is just as straightforward—it simply involves looping over your models and recording the evaluation metrics, as we have done throughout this book and as the following steps illustrate:

1. Use the function on our data to create the splits:

```
splits = rolling_forecasting_origin(flights, n_splits=5)
```

2. Unpack all the training and test sets using the **zip()** function, separating them into two tuples, one for training sets and one for test sets:

```
trains, tests = zip(*splits)
```

3. Save the files as CSV files, to be iterated over by a modeling loop:

```
for i, (train, test) in enumerate(splits):
```

```
    train.to_csv(f'{SAVE_DIR}/train_split_{i+1}.csv', index=True)
```

```
    test.to_csv(f'{SAVE_DIR}/test_split_{i+1}.csv', index=True)
```

While this chapter focuses on hold-out splitting and TSCV using a rolling forecasting origin, there are several other techniques available for splitting time series data, such as **sliding window**, **blocked cross-validation**, and **nested time series CV**.

In the next section, we will get into modeling the time series data.

Modeling and tuning time series models

In this section, we will apply a range of forecasting techniques to model our time series data. We will explore both traditional statistical methods and modern machine learning approaches, each suited to different types of patterns and data structures.

The models covered include:

- **Holt-Winters exponential smoothing:** This is for capturing trend and seasonality.
- **SARIMA:** SARIMA is a powerful model for handling seasonal, trend, and autoregressive patterns.
- **XGBoost and random forests:** Machine learning models using lagged features for flexible forecasting.
- **Facebook Prophet:** It is an intuitive, scalable tool for modeling seasonality, holidays, and trend shifts.

Each method offers unique strengths, and we will compare them to understand their performance in real-world forecasting tasks.

Loading and preparing data

In this section, we will get our data in the right shape to be modeled. We will explore the following steps to get the data loaded, converted in the right way and separate our features for training the machine learning models, as well as the univariate models:

1. Load in the train and test data from the splitting section of this chapter:

```
flights_train = pd.read_csv('data/splits/holdout_train.csv',  
                             index_col=0,  
                             parse_dates=True)  
flights_test = pd.read_csv('data/splits/holdout_test.csv',
```

```
index_col=0,  
parse_dates=True)
```

2. Set the date index to DateTime format:

```
flights_train.index = pd.to_datetime(flights_train.index)  
flights_test.index = pd.to_datetime(flights_test.index)
```

3. Create our target variables for the train and test datasets:

```
y_train = flights_train['Passengers']  
y_test = flights_test['Passengers']
```

4. Choose only the features that will be included in the machine learning models, since univariate models rely solely on the **Date** and **Value** features for predictions:

```
X_train = flights_train[['log', 'Lag_1', 'Lag_2', 'Lag_3',  
                        'Lag_12', 'Lag_24', 'Lag_36']]  
X_test = flights_test[['log', 'Lag_1', 'Lag_2', 'Lag_3',  
                      'Lag_12', 'Lag_24', 'Lag_36']]
```

5. We will also define two key variables: the forecast horizon, which specifies how many future time steps we want to predict, and the seasonal period, which we will set to 12 based on our ACF and PACF analysis indicating annual seasonality in the monthly data:

```
FORECAST_HORIZON = 12  
SEAS_PERIODS = 12
```

In the next section, we will introduce exponential smoothing.

Holts-Winters exponential smoothing

Holt-Winters exponential smoothing is a forecasting method that works well when your time series shows both a trend (values increase or decrease over time) and seasonality (regular repeating patterns, like yearly peaks). It

is an extension of basic exponential smoothing, and includes separate components for:

- **Level:** Level is the average value of the series at a given point.
- **Trend:** Trend shows the direction and rate at which the series is moving (upward/downward).
- **Seasonality:** Seasonality shows the repeating pattern based on the time of year, month, etc.

The equations, for both the additive and multiplicative model are:

$$\begin{aligned}l_t &= \alpha \cdot (y_t \oplus s_{t-m}) + (1 - \alpha)(l_{t-1} + b_{t-1}) \\b_t &= \beta \cdot (l_t - l_{t-1}) + (1 - \beta)b_{t-1} \\s_t &= \gamma \cdot (y_t \oplus l_t) + (1 - \gamma)s_{t-m} \\\hat{y}_{t+h} &= (l_t + h \cdot b_t) \oplus s_{t+h-m(k+1)}\end{aligned}$$

Let us understand the components:

- **$\oplus$ (The model dependent operator):** For multiplicative use division, and for additive use, subtraction in the level, trend, and seasonality steps. For the future value prediction (y^{t+h}) use multiplication for multiplicative and addition for additive.
- **y_t (Actual value at time):** The observed data point (e.g., sales, traffic).
- **l_t (Level):** The smoothed estimate of the current average.
- **b_t (Trend):** The estimated rate of change (slope).
- **s_t (Seasonality):** Repeating pattern based on time (e.g., monthly or weekly effect).
- **α (Level smoothing factor):** Controls how fast the model updates the average.

- **β (Trend smoothing factor):** Controls how quickly the trend adjusts.
- **γ (Seasonal smoothing factor):** Controls how fast seasonal patterns update.
- **m (Seasonal period):** Number of time steps in a full season (e.g., 12 for monthly data with yearly seasonality).
- **h (Forecast horizon):** How far forward to predict.
- **y^{t+h} :** The predicted future value.

Since this is just an introduction, we will not build the model from scratch. Instead, we will use **statsmodels** to import, fit, and forecast with Holt-Winters. We will apply it to our test data so we can compare the forecast with actual values and measure the prediction error. These steps will get us started:

1. Import the Holts-Winters exponential smoothing from the **statsmodels** library:

```
from statsmodels.tsa.holtwinters import ExponentialSmoothing
```

2. Initialize the model and specify a multiplicative pattern, as we concluded our time series conforms to this:

```
es_model = ExponentialSmoothing(
    y_train,
    trend='mul',
    seasonal='mul',
    seasonal_periods=SEAS_PERIODS)
```

3. Fit the model and get a summary of the model coefficients smoothing parameters. These will be the values that are plugged into the equation we went through earlier:

```
es_fit = es_model.fit()
```



```
fit_params = es_fit.params  
es_summary_df = pd.DataFrame(fit_params.items(), columns=  
    ['Parameter', 'Value'])
```

4. The summary provides the parameters and values, as in *Figure 8.11*:

Parameter	Value
smoothing_level	0.609357
smoothing_trend	0.000064
smoothing_seasonal	0.000002
damping_trend	nan
initial_level	62.267945

Figure 8.11: *Holts-Winters fit coefficients*

5. We will use these fitted parameters to generate predictions on the test dataset. All models will be evaluated on the same test data to determine which one best captures the patterns for accurate future forecasting:

```
,es_forecast = es_fit.predict(start=y_test.index[0],  
                             end=y_test.index[-1])
```

6. That will give us predictions for every value in the test set, later, we will compare the actual with the prediction to work out how far off the model was.

That wraps up the Holts-Winters step, in the next section, we will look at ARIMA (with seasonal components, as our time series has seasonal components present).

Seasonal ARIMA

SARIMA extends the basic ARIMA model by handling recurring seasonal patterns, which are common in most real-world time series, like monthly

sales or yearly plane passenger volumes. Since seasonality is almost always present, ARIMA alone often is not enough. That is why we are jumping straight into SARIMA: it gives us the flexibility to model both trends and seasonality from the start. The equation to define SARIMA is:

$$\text{SARIMA}(p, d, q)(P, D, Q)_m$$

SARIMA models have two main components: one for capturing general patterns in the data, like trends and noise, and the other for capturing seasonal patterns, such as recurring yearly cycles. The model includes non-seasonal terms (p, d, q) and seasonal terms (P, D, Q) . Each term has a seasonal and non-seasonal counterpart:

- p (and P) control autoregression, which is how much past values influence the forecast.
- q (and Q) refers to moving averages, which use past errors to improve predictions.
- d controls how much differencing is applied to remove trends, while D removes seasonal trends, such as regular spikes in sales every December.

To understand this at a lower level, we include the full SARIMA equation:

$$\Phi_P(L^m) \phi_p(L) (1 - L)^d (1 - L^m)^D y_t = \Theta_Q(L^m) \theta_q(L) \varepsilon_t$$

Let us understand the components:

- y_t is the time series at time t .
- L is the lag operator, where we compare the lag offset to the current time series value minus the lag value ($Ly_t = y_{t-1}$).
- $\phi_p(L)$ is non-seasonal AR terms.
- $\theta_q(L)$ is non-seasonal MA terms.

- $\Phi_p(L^m)$ is seasonal AR terms (with lag = season length).
- $\Theta_Q(L^m)$ is seasonal MA terms.
- $(1-L)^d$ is the non-seasonal differencing to remove trend.
- $(1-L^m)^D$ is the seasonal differencing to remove seasonal trend.
- ε_t is the white noise (random error).

This formula combines both non-seasonal and seasonal elements, using past values, past errors, and differencing, to model trends, cycles, and random fluctuations in a time series, enabling accurate forecasting even when seasonality is present. In the following steps, we will start with some default orders for the seasonal and ARIMA components:

1. Import the **statsmodels** API:

```
import statsmodels.api as sm
```

2. Determine an initial p,d,q order, here we are saying use 1 past lag value for q, use first order differencing for d and 1 past forecast error, for the initialization:

```
order = (1, 1, 1)
```

3. Set an initial seasonal ordering P, D, Q as well:

```
seasonal_order = (1, 1, 1, 12)
```

4. Fit a baseline SARIMA model, with random initializations, to do a comparison against:

```
model = sm.tsa.SARIMAX(y_train,
                        order=order,
                        seasonal_order=seasonal_order)

fit = model.fit()
```

5. Forecast against the test set, as we did with the exponential smoothing model, the difference here is we use the **get_forecast** method:

```
sarima_base_forecast = fit.get_forecast(steps=len(y_test))
```

6. Finally, from the forecast extract the predicted mean, as the prediction:

```
sarima_base_test_preds = sarima_base_forecast.predicted_mean
```

In the supporting notebook, there is a function included that performs an automated grid search (we will learn more about this in the next chapter) to find the best SARIMA model configuration by evaluating combinations of ARIMA and seasonal parameters. It checks multiple combinations of (p,d,q) and (P,D,Q) , values, fits a SARIMA model for each, and ranks them by their **Akaike Information Criterion (AIC)** and **Bayesian Information Criterion (BIC)**. If the dataset is large enough, it includes seasonal components in the search. The function returns a DataFrame of the top models sorted by AIC. Since this function is relatively verbose and performs many combinations behind the scenes, you can refer to the supporting Jupyter Notebook (**08_05_Modeling_time_series.ipynb**) for the full implementation. Here, we will use the function provided in the notebook to suggest the best order and seasonal orders for us, as the following steps show:

1. Execute the **search_arima_orders** function with the target column (**y**), specify the seasonal periods (12 for yearly seasonality), choose whether to print output to the console using **verbose**, and set **top_n** to retrain the top recommended candidate orders:

```
arma_orders_results_df = search_arma_orders(y_train,  
                                             s=12,  
                                             verbose=False,  
                                             top_n=20)
```

2. Once this has run (should take about a minute on this data) you will get the results in a DataFrame, as in *Figure 8.2*:

order	seasonal_order	aic
(1, 1, 2)	(0, 1, 1, 12)	749.751937
(2, 1, 2)	(0, 1, 1, 12)	750.333099
(1, 1, 2)	(1, 1, 1, 12)	751.751679
(2, 1, 2)	(1, 1, 1, 12)	752.332501
(0, 1, 2)	(0, 1, 1, 12)	752.630817

Figure 8.12: Chosen SARIMA orders

3. AIC is a metric used to compare models. It balances model fit and complexity by penalizing models with more parameters. Lower AIC values indicate a better model, one that fits the data well without being overly complicated. Therefore, in the next step, we will choose the orders and seasonal orders by getting the lowest AIC values:

```
chosen_order = arima_orders_results_df.iloc[0]['order']  
chosen_seasonal = arima_orders_results_df.iloc[0]['seasonal_order']
```

4. Now, we have these parameters, we can fit another SARIMA model:

```
SARIMA_tuned_model = sm.tsa.SARIMAX(y_train,  
                                     order=chosen_order,  
                                     seasonal_order=chosen_seasonal)  
fit = SARIMA_tuned_model.fit()
```

5. We will obtain the forecasts, as we did in the baseline SARIMA model:

```
forecast = fit.get_forecast(steps=len(y_test))  
arima_tuned_forecast_mean = forecast.predicted_mean
```

In the next section, we will fit machine learning regression models using our lagged features, allowing us to make predictions that account for the time-based structure of the data.

Machine learning approaches on lagged features

In this section, we are going to take our lagged features and use them with two popular machine learning models, XGBoost and random forests, both applied with default parameters. This approach allows us to repurpose standard regression models into time-aware predictors, without needing to fully restructure them for time series data. To do this, we will go through these steps, to first fit the random forest and then the XGBoost regressor:

1. Initialize the random forest model to use the lagged features we created at the start of this section:

```
rf = RandomForestRegressor(n_estimators=100,  
                           random_state=42)
```

2. Fit the model on the target (y) and lagged features (X):

```
rf.fit(X_train, y_train)
```

3. Make predictions on the testing set:

```
rf_pred = rf.predict(X_test)
```

4. Repeat this process for the XGBoost regressor:

```
xgb = XGBRegressor(n_estimators=100,  
                   learning_rate=0.1,  
                   max_depth=3,  
                   random_state=42)
```

```
xgb.fit(X_train, y_train)
```

```
xgb_pred = xgb.predict(X_test)
```

As we have explored regression in *Chapter 6, House Price Estimation with Regression Methods* let us move on to the final model type, we will be

using, which is a package *Facebook* (now *Meta*) built for time series forecasting, called **Prophet**.

Automated ARIMA with Prophet

Prophet is a powerful, user-friendly forecasting tool developed by Facebook (now Meta), designed for people who need accurate time series forecasts without deep statistical expertise. It handles trend shifts, seasonal cycles, holidays, and missing data right out of the box, making it especially useful for real-world business data.

The benefits of Prophet are that it is flexible, requires minimal preprocessing, and does not rely on strict assumptions like stationarity.

In this section, we will use Prophet to build and evaluate forecasts with just a few lines of code, and we will then use it to compare the performance alongside all the forecasting models we have built so far, to see if the knowledge we have acquired can beat Prophet.

Prophet data processing

Prophet requires your time series data to be in a specific format: a **pandas** DataFrame with two columns named **ds** and **y**. The **ds** column should contain the date or datetime values, and the **y** column should contain the numeric values you want to forecast. We will make sure our data is converted, using the following steps:

1. Import the prophet package:

```
from prophet import Prophet
```

2. Make a copy of the **y_train** series:

```
y_train_prophet = y_train.copy()
```

3. Check whether the index of the DataFrame is a **PeriodIndex** and converts to a Prophet suitable Date format:

```
if isinstance(y_train_prophet.index, pd.PeriodIndex):
```

```
y_train_prophet.index = y_train_prophet.index.to_timestamp()  
p()
```

4. The next step is to move the DateTime from the index and prepare as a column:

```
df_prophet = y_train_prophet.reset_index()
```

5. From there, we specify the column names needed to work with the Prophet package, i.e., **ds** and **y**:

```
df_prophet.columns = ['ds', 'y']
```

Now, we have our data in the right shape to work with the Prophet package.

Modeling with Prophet

Before we begin modeling with Prophet, it is helpful to understand why it is such a practical tool for real-world forecasting. Unlike traditional models that require stationary data and fine-tuned parameters, Prophet is built to handle messy, irregular, and seasonal time series with minimal effort. It supports different types of growth trends (linear, logistic, or flat) and can automatically detect changepoints, which are moments where the trend shifts over time.

Prophet also makes it easy to include yearly, weekly, and daily seasonality, either automatically or with custom settings, using the **seasonality_mode** parameter (additive or multiplicative). If your data includes holidays or special events, the holidays parameter allows you to incorporate their effects directly into the forecast. You can even adjust how flexible the model is using **changepoint_prior_scale**, which controls how sensitively the model responds to changes in trend. To get help on the model parameters, just call on **help(prophet)** to learn more about the class. We will look at a couple of simple steps to show how easy it is to initialize and fit a Prophet model:

1. Pass in the multiplicative **seasonality_mode** and indicate that the series contains **yearly_seasonality**:


```
model = Prophet(  
    seasonality_mode='multiplicative',  
    yearly_seasonality=True,  
    weekly_seasonality=False,  
    daily_seasonality=False)
```

2. Fit the model on our Prophet formatted DataFrame:

```
model.fit(df_prophet)
```

3. We make a future DataFrame, which extends the periods to the length of the test set and says extend by a monthly pattern (**freq='M'**):

```
future = model.make_future_dataframe(periods=len(y_test), freq='M')
```

4. Make predictions with the model into the future; here the length of the test set:

```
prophet_forecast = model.predict(future)
```

5. Obtain the forecast from the predictions in *Step 4*:

```
prophet_fc = prophet_forecast['yhat']
```

We have fit a model that will be a competitor when we do the final evaluation of all the models in the next sections.

Evaluating time series models

Evaluating the performance of time series models is critical to understanding how well they generalize to future data. Fortunately, most of the core error-based metrics used in time series forecasting are the same as those covered in *Chapter 6, House Price Estimation with Regression Methods*, earlier in this book. We will be using the **compute_metrics** function we created in this chapter, and it is available in the supporting notebook. In the next sections, we will combine the forecasts from the models, analyze the results, and pick a model to make future forecasts.

Combine forecasts to compare models

In this section, we will create a function to combine the forecasts together from all the models and use the **compute_metrics** function, which we created previously. We will do this in the following steps:

1. Create a dictionary with key value pairs, the keys are the model names, and the values are the forecasts derived from those models:

```
combined_forecasts = {  
    'exponential_smoothing': es_forecast,  
    'sarima_tuned': arima_tuned_forecast_mean,  
    'sarima_base': sarima_base_test_preds,  
    'random_forest': rf_pred,  
    'xgboost': xgb_pred,  
    'prophet': prophet_fc  
}
```

2. Create a function that aligns all the forecasts together, as certain forecasts have different indices:

```
def combine_and_evaluate_forecasts(y_true, forecast_dict,  
                                   compute_metrics_fn,  
                                   sort_by='RMSE'):  
    combined = {'actual': y_true.values}  
    index = y_true.index  
    for name, preds in forecast_dict.items():  
        preds = pd.Series(preds)  
        if len(preds) != len(y_true):  
            preds = preds[-len(y_true):]  
        preds.index = index  
        combined[name] = preds.values
```

```

combined_df = pd.DataFrame(combined, index=index)
metrics_records = []

for model_name in forecast_dict.keys():
    y_pred = combined_df[model_name]

    metrics = compute_metrics_fn(combined_df['actual'], y
_pred)

    metrics['Model'] = model_name

    metrics_records.append(metrics)

metrics_df = pd.DataFrame(metrics_records).set_index('Model')

best_model = metrics_df[sort_by].idxmin() if sort_by in metrics_df.columns else None

return combined_df, metrics_df, best_model

```

3. To explain the function in *Step 2*, we break it down, as follows:
 - a. Create a dictionary to store the actual values from the predictions called **combined**.
 - b. Loop through each model's predictions, using the **forecast_dict** items that is passed as a parameter, exposing the key (name of the model) and value (the model predictions).
 - c. Ensure predictions match the length of the true values with a conditional statement checking the lengths of the predictions and ground truth values.
 - d. Convert the combined predictions to a DataFrame.
 - e. Compute evaluation metrics for each model and append them to a **metrics_records** list.
 - f. Format the metrics into a summary **metrics_df** DataFrame.
 - g. Sorts by the best model, if the **sort_by** column is passed.
 - h. Returns a tuple of results: 1) the actual vs predictions; 2) performance scores (**metrics_df**) and the name of the best

performing model.

Now we have all the results combined, the next step is to get on to evaluating the results, this is what we will cover in the next section.

Evaluating model results and finding the best model

We will now look at how to choose the best-performing model from our set of candidate forecasts. By comparing each model's predictions against the actual values using evaluation metrics such as **root mean squared error (RMSE)** and **mean absolute error (MAE)**, we can objectively determine which model fits the data best and is most suitable for future forecasting. To do this, we will undertake these steps:

1. Choose our target metric to pick the best model and sort out the model results:

```
EVAL_METRIC = 'RMSE'
```

2. Use the **compute_metrics** function we created in *Chapter 6, House Price Estimation with Regression Methods* and pass in the testing target ground truth, the combined forecasts data, the function we created in the aforementioned chapter and enable the sort by the metric chosen in *Step 1*:

```
combined_df, metrics_df, best_model = combine_and_evaluate  
_forecasts(
```

```
    y_test,
```

```
    combined_forecasts,
```

```
    compute_metrics_fn=compute_metrics,
```

```
    sort_by=EVAL_METRIC)
```

3. You can then use the **best_model** string to return the best model:

```
print(f"    Best model by {EVAL_METRIC}:{best_model}")
```

4. To inspect the **metrics_df** we will use the following, but we will drop the coefficient of determination:

```
metrics_df.drop(columns='R2')
```

5. This produces a DataFrame, as in *Figure 8.3*:

	MSE	RMSE	MAE	MSLE	MAPE(%)
Model					
exponential_smoothing	1258.632225	35.477207	30.778380	0.006863	7.484096
sarima_tuned	792.905068	28.158570	22.801935	0.003906	5.190365
sarima_base	826.588981	28.750461	23.277991	0.004084	5.315516
random_forest	2121.492552	46.059663	28.238621	0.007904	5.581172
xgboost	1550.313360	39.374019	21.462702	0.005499	4.063262
prophet	796.653657	28.225054	23.021129	0.004293	5.502641

Figure 8.13: Forecast model results

6. As RMSE is our target metric, our **sarima_tuned** model outperforms the prophet model, with **sarima_base** coming in third. If we were to use MAE, we would choose **xgboost**. In the next step, before we pick a model, we will look at the forecasts on a panel plot, to make sure we are confident in our choice of model.
7. To do our second sense check, as recommended in *Step 6*, we will make use of a function in **modelviz**, called **plot_forecast_subplots**, which will enable this visual comparison:

```
from modelviz.forecast import plot_forecast_subplots  
plot_forecast_subplots(y_train, y_test, combined_df)
```

8. *Figure 8.4* shows the forecast plot and ground truth compared with the predictions:

Forecasts by Model

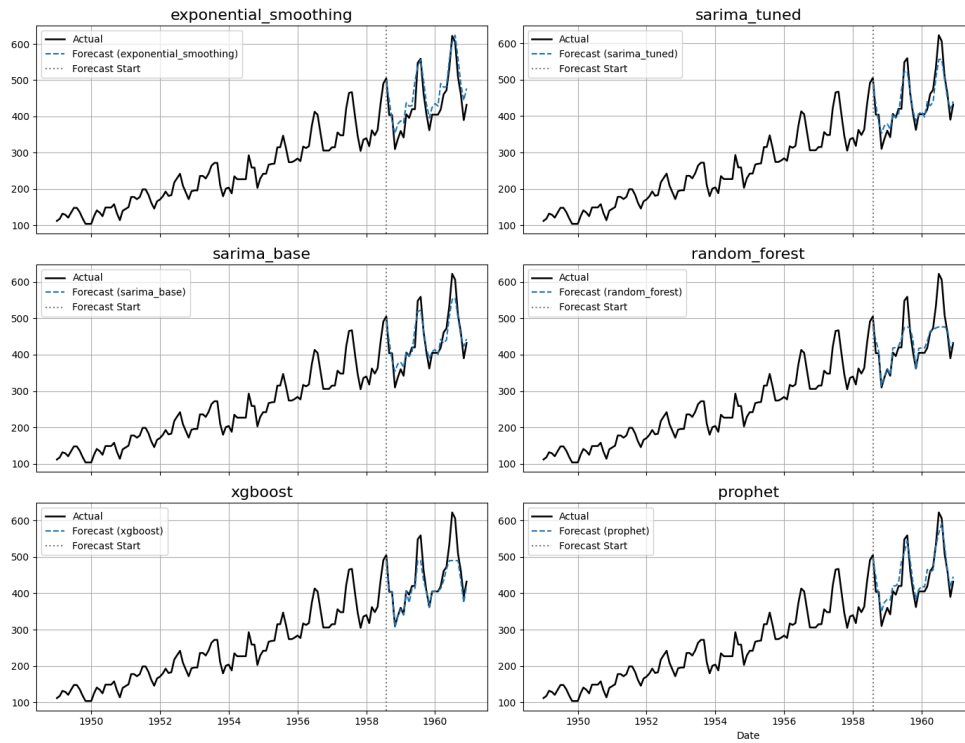


Figure 8.14: Model by forecast and actual plot

From visual inspection of *Figure 8.4*, Prophet, exponential smoothing, and tuned SARIMA appear to be the most balanced in terms of fitting and forecasting. In contrast, random forest and XGBoost show signs of overfitting, likely due to their sensitivity to short-term patterns and the lack of explicit seasonality handling. As tuned ARIMA shows good performance, as is not potentially overfitting on the data, we will use this model to forecast forward past all the data to 6 points in the future, known as a forecast horizon.

Extending predictions beyond the forecast horizon

Now that we have compared our candidate models using evaluation metrics like RMSE and MAE, we can identify which one best captures the underlying structure of the data. With a reliable model selected, the next step is to extend our predictions beyond the test period and generate

forecasts into future time steps, where no observed data exists. This allows us to move from model evaluation to practical forecasting.

However, it is important to note that the further we predict into the future, the greater the uncertainty and risk of compounding error. Forecasts become increasingly sensitive to small changes in trend, seasonality, or external factors not captured by the model. For this reason, long-range forecasts should always be interpreted with caution and ideally supplemented with confidence intervals.

To move forward, we will walk through the following steps:

1. The first step is to readjust the time period index, since we need to merge the training and testing sets back together. This allows us to generate future forecasts, as the model requires a complete view of the data distribution:

```
def ensure_datetime_index(series):  
    if isinstance(series.index, pd.PeriodIndex):  
        series.index = series.index.to_timestamp()  
    elif not isinstance(series.index, pd.DatetimeIndex):  
        series.index = pd.to_datetime(series.index)  
    return series
```

2. This function will come in handy, as it is one of the downsides working with time series, that you are forever running into index mismatch and type errors.
3. Once the function is created, we will check that our train and test series have consistent indexes:

```
y_train = ensure_datetime_index(y_train)  
y_test = ensure_datetime_index(y_test)
```

4. The next step, is to merge our **y_train** and **y_test** series:

```
y_full = pd.concat([y_train, y_test]).sort_index()
```

5. This line sets the time series to a regular frequency by inferring the most likely interval between time points and applying it using **asfreq**, to ensure consistency:

```
y_full = y_full.asfreq(pd.infer_freq(y_full.index))
```

6. An optional step would be to check the length of the combined series to make sure it matches the train and test series lengths, as a check we can simply use the **len(y_full)** command.
7. Since the model was previously trained only on the training set, we need to update it, along with its underlying parameters, to ensure it captures the full data distribution, which we do by passing in the complete **y_full** series:

```
arima_tune_fitted = sm.tsa.SARIMAX(  
    y_full,  
    order=chosen_order,  
    seasonal_order=chosen_seasonal).fit()
```

8. To make predictions into the future, we can use the **get_forecast** method of the fitted model (the model must be fitted before using this), we use our integer value of the number of steps forward we want to predict into the future (**FORECAST_HORIZON**), and we will pull out the **predicted_mean** and the associated confidence intervals:

```
arima_tune_fitted.get_forecast(steps=FORECAST_HORIZO  
N).predicted_mean  
arima_tune_fitted.get_forecast(steps=FORECAST_HORIZO  
N).conf_int()
```

9. To plot the forecast, we will use the same process as we have done in steps 1-8, but we will use the **plot_forecast** method from the **modelviz** package:

```
from modelviz.forecast import plot_forecast
```



```

forecast_df = plot_forecast(
    data=y_full,
    model=arima_tune_fitted,
    horizon=FORECAST_HORIZON,
    model_type='statsmodels',
    alpha=0.05,
    plot=True,
    ci_color='grey',
    forecast_color='black',
    historical_color='grey')

```

10. This will produce the forecast plot, as per *Figure 8.5*:

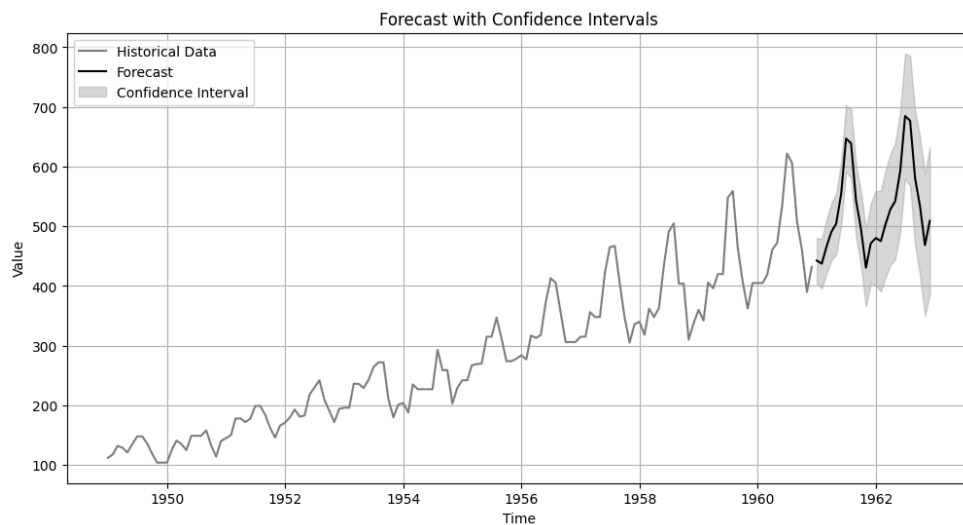


Figure 8.15: Forecast plot with confidence intervals

The confidence intervals (with a forecast horizon of 12 months) show where the true future values are likely to fall, based on the model's predictions, with a given level of confidence (typically 95%). A wider interval indicates more uncertainty, while a narrower interval suggests the model is more certain about its forecast. A way to show how the confidence intervals are useful, and why we should not extend the forecast too far forward, is shown in *Figure 8.6*:

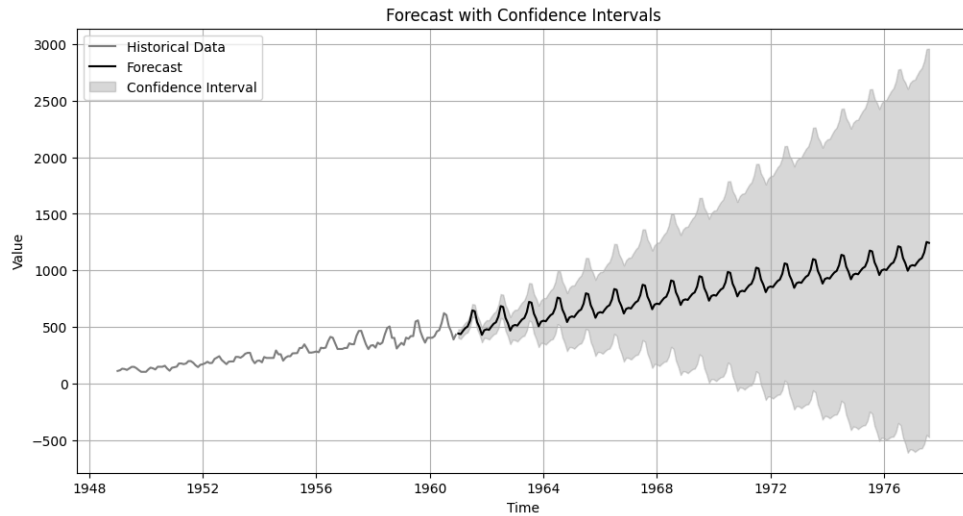


Figure 8.16: *Forecast plot with an extended forecast horizon*

This plot illustrates how uncertainty grows the further we project into the future. The forecast (black line) offers the best estimate of future values, but the widening shaded confidence intervals reveal a broader range of possible outcomes over time. Early predictions are reliable, but uncertainty grows with longer forecasts. It is crucial to treat long-term forecasts cautiously and consider the full range of potential variations, not just the central estimate. Furthermore, without the confidence interval, bad decisions can be made with the predicted values.

Conclusion

In this chapter, we explored the full lifecycle of time series forecasting, from preparing and transforming temporal data to applying and evaluating a range of forecasting models. We covered traditional statistical techniques like Holt-Winters and SARIMA, alongside modern machine learning approaches including random forests, XGBoost, and Facebook Prophet. By combining visual diagnostics, evaluation metrics, and confidence intervals, we learned how to select the most effective model and generate future forecasts with greater confidence and interpretability.

Now that we have built a solid foundation in time-aware modeling, it is time to shift gears and take your machine learning models to the next level.

In the upcoming chapter, we will explore hyperparameter tuning techniques like **GridSearchCV** and **RandomizedSearchCV**, dive into feature selection and dimensionality reduction methods such as **recursive feature elimination (RFE)** and **Linear Discriminant Analysis (LDA)**, and introduce ensemble learning strategies like bagging, boosting, and stacking. We will wrap up with a hands-on case study focused on boosting the performance of a breast cancer detection model.

Points to remember

Here are some key points summed up from this chapter:

- **Autocorrelation and PACF:** Understanding autocorrelation and partial autocorrelation helps reveal how past values influence the present which is crucial for model selection and lag creation.
- **Backfilling and forward filling:** Simple imputation techniques like forward and backward fill can quickly fill in missing values, while maintaining the structure of the series.
- **Confidence intervals matter:** Forecasts should always be paired with confidence intervals to reflect uncertainty, especially when projecting further into the future.
- **Differencing for stationarity:** Many time series models require stationarity. Differencing (first or second order) helps remove trends and stabilize the mean.
- **Evaluating models:** Time-aware train-test splits (like hold-out or rolling forecasting origin) avoid data leakage and give more reliable model evaluations.
- **Feature engineering for machine learning models:** Creating lagged features turns a time series into a supervised regression learning problem, making it suitable for machine learning models like random forests and XGBoost.

- **Grid search for SARIMA:** We tune SARIMA hyperparameters using grid search, comparing configurations with AIC and BIC to find the best-fitting model.
- **Handling seasonality:** Whether additive or multiplicative, understanding seasonal patterns is key, especially when using models like Holt-Winters or Prophet.
- **Kalman filtering for imputation:** A more advanced technique for filling missing values, Kalman filters model both trends and uncertainty in the series.
- **Log transformations:** Applying a log transformation can help stabilize variance in time series with exponential growth or heteroscedasticity.
- **Model comparison:** We evaluate multiple models, statistical and machine learning, and compare them using RMSE, MAE, and visual plots.
- **Resampling data:** Resampling helps change the frequency of the time series, which is useful for zooming out (yearly, quarterly) or aggregating noisy data.
- **Seasonal decomposition:** Breaking a series into trend, seasonality, and residuals helps you understand what is behind the data and helps you to select suitable forecasting techniques for your time series problem.
- **Visualization first:** Plotting your time series upfront reveals trends, cycles, anomalies, and the overall shape of the data; this is a pivotal part of the process, so do not skip it.

Multiple choice questions

1. **What is the purpose of differencing in time series analysis?**
 - a. To increase the frequency of observations in the dataset

- b. To handle missing values by interpolating between them
 - c. To transform a non-stationary series into a stationary one
 - d. To reduce the size of the dataset for faster training
2. **Which of the following best describes a multiplicative time series?**
- a. Seasonality and trend remain constant over time
 - b. Seasonality grows proportionally with the level of the series
 - c. Seasonal effects shrink as the overall level increases
 - d. The data has no observable pattern or variance
3. **What does the seasonal decompose function from statsmodels do?**
- a. It separates a time series into trend, seasonal, and residual components
 - b. It fills in missing values with seasonal averages
 - c. It automatically forecasts the next season's values
 - d. It standardizes the dataset using rolling statistics
4. **How does SARIMA differ from ARIMA?**
- a. SARIMA is based on exponential smoothing, not autoregression
 - b. SARIMA works only with daily frequency data
 - c. SARIMA removes the need for differencing
 - d. SARIMA includes additional terms to model seasonal patterns
5. **Why is forward fill commonly used in time series preprocessing?**
- a. It replaces missing values with zero to avoid skewing averages
 - b. It preserves recent values by carrying them forward to fill gaps

- c. It uses the next known value to fill previous missing ones
 - d. It interpolates between missing and non-missing values
6. **What does the ADF test check for?**
- a. Whether a series has missing values
 - b. Whether a series has seasonal components
 - c. Whether a series is stationary
 - d. Whether a series can be split evenly for cross-validation
7. **Why are lag features useful in machine learning models for time series?**
- a. They eliminate the need for scaling and normalization
 - b. They allow models to account for the time-based dependencies in the data
 - c. They reduce dimensionality by collapsing features into a single column
 - d. They convert categorical data into numerical features
8. **What is the seasonal_periods parameter used for in Holt-Winters?**
- a. To set the number of features used in the model
 - b. To control the depth of trend smoothing
 - c. To limit how far ahead predictions can be made
 - d. To define the number of time steps in one seasonal cycle
9. **In TSCV, what does a rolling forecast origin mean?**
- a. Each train/test split moves forward in time, expanding the training set
 - b. All data points are used to train the model
 - c. A test set that is randomly selected in each fold

- d. The training set shrinks over time to improve speed
10. **Which of the following is a downside of long-term forecasting in time series?**
- a. Decreased memory usage during inference
 - b. Smoother trend lines
 - c. Greater uncertainty and wider confidence intervals
 - d. Higher model accuracy

Answers

1	c
2	b
3	a
4	d
5	b
6	c
7	b
8	d
9	a
10	c

Key terms

Some key terms used in this chapter are:

- **Augmented Dickey-Fuller (ADF) test:** A statistical test used to determine whether a time series is stationary. It tests for the presence of a unit root, with a low *p-value* indicating that the series is likely stationary.
- **Additive time series:** A time series model where the components (trend, seasonality, and residual) are added together. Seasonal variation remains constant regardless of the overall trend level.

- **Autoregressive Integrated Moving Average (ARIMA):** A foundational time series forecasting model that combines three components: **AR**, differencing to ensure stationarity (**Integrated, I**), and **MA**. ARIMA is best suited for univariate time series with trend, but no strong seasonal pattern. It models relationships between an observation and several lagged observations and forecast errors.
- **Autocorrelation:** A measure of how related a time series is with its own past values.
- **Backfilling:** A method for imputing missing values by propagating the next available value backward through the time series.
- **Confidence interval:** A range around forecasted values that expresses uncertainty in the prediction. Wider intervals indicate greater uncertainty, especially in long-term forecasts.
- **Decomposition:** The process of breaking a time series into its component parts, trend, seasonality, and residuals, to better understand the structure of the data.
- **Differencing:** A transformation used to make a time series stationary by removing trends or seasonality.
- **Exponential smoothing:** A forecasting method that applies weighted averages to past observations, giving more weight to recent values. Variants like Holt-Winters also capture trend and seasonality.
- **Forecast horizon:** The number of future time steps you want to predict in a time series model. Longer horizons increase uncertainty in the forecast.
- **Forward fill:** A technique for imputing missing values by carrying the most recent observed value forward until a new value is encountered.

- **Hold-out split:** A time series validation approach that splits the dataset chronologically into training and testing segments to avoid data leakage and preserve temporal structure.
- **Lag features:** New features created by shifting a time series backward by one or more-time steps. These help machine learning models capture time-based dependencies.
- **Log transformation:** A mathematical transformation applied to compress large values and stabilize variance in time series data, often used before modeling exponential growth trends.
- **Multiplicative time series:** A model where trend and seasonal components interact multiplicatively. Seasonal effects increase or decrease in proportion to the trend.
- **Partial autocorrelation function (PACF):** Measures the direct correlation between a time series and its lagged values, excluding the influence of intermediate lags. Useful for identifying autoregressive terms.
- **Resampling:** The process of changing the frequency of time series observations (e.g., from monthly to quarterly) to support different levels of analysis or modeling.
- **Rolling forecast origin:** A TSCVTSC method where the training window expands with each iteration.
- **Seasonal ARIMA (SARIMA):** An extension of ARIMA that models both non-seasonal and seasonal components using parameters for autoregression, differencing, and moving averages.
- **Seasonality:** Repeating patterns or cycles in a time series that occur at regular intervals (e.g., monthly, yearly). It can be additive or multiplicative in nature.
- **Stationarity:** A property of a time series where statistical characteristics (mean, variance) remains constant over time.

Stationary data is often a requirement for traditional forecasting models.

- **Trend:** The long-term movement or direction in a time series, either upward, downward, or flat. Identifying the trend helps in choosing suitable models for forecasting.
- **Visual aids and tools:** Plots such as time series graphs, decomposition plots, ACF, and PACF that help in understanding the structure of time series data before modeling.
- **White noise:** A random, pattern less component of a time series. A good residual from a well-fitted model should resemble white noise, predictable, and with constant variance.

Join our Discord space

Join our Discord workspace for latest updates, offers, tech happenings around the world, new releases, and sessions with the authors:

<https://discord.bpbonline.com>



CHAPTER 9

Model Improvement Strategies

Introduction

In this chapter, we will cover critical techniques for taking your machine learning models from good to exceptional, like hyperparameter tuning, feature elimination, and dimensionality reduction with **Linear Discriminant Analysis (LDA)**, the power of ensemble learning, and other key considerations that influence model performance. These strategies go beyond choosing the right algorithm, and they focus on refining, optimizing, and intelligently combining methods to unlock higher accuracy, robustness, and generalization in your models.

So far, you have built solid machine learning models, but true performance comes from fine-tuning and enhancement. This chapter will show you how to boost results through smarter hyperparameter selection, feature reduction, and ensemble methods, while addressing real-world challenges like data quality and class imbalance. By the end, you will have a practical toolkit to turn baseline models into reliable, production-ready solutions.

Structure

This chapter covers the following topics:

- Hyperparameter tuning
- Feature elimination and dimensionality reduction with LDA
- Ensemble learning
- Other considerations for model improvement

Objectives

By the end of this chapter, you will be able to improve machine learning models through effective hyperparameter tuning with grid search, random search, and Bayesian optimization; apply feature elimination and dimensionality reduction techniques like **recursive feature elimination with cross-validation (RFECV)** and LDA to reduce complexity and overfitting; enhance performance using ensemble methods such as bagging, boosting, and stacking; and confidently evaluate these approaches using metrics like precision, recall, and F1 score.

You will also be equipped to recognize and address critical non-technical factors, such as data quality issues, class imbalance, and feature drift, that impact real-world model reliability and deployment readiness.

Hyperparameter tuning

In the realm of machine learning, achieving the optimal model performance extends far beyond selecting the right algorithm, or feeding it quality data. A critical, yet often challenging, aspect of the process lies in the careful adjustment of hyperparameters; think of these as dials on a radio acting to govern the behavior of the learning algorithm, but these parameters are not learned from the data itself.

Hyperparameter tuning is the art and science of selecting the ideal set of these parameters to enhance the model's accuracy, robustness, and its ability to generalize with new, unseen data. Whether it is determining the learning rate in a neural network, the depth of a decision tree, the optimal configuring of **Autoregressive Integrated Moving Average (ARIMA)** orders, the strength of regularization in a regression model, or any other hyperparameter optimization, getting the right hyperparameters can significantly affect and influence the outcomes, similar to how if you get the wrong frequency on a radio leading to static, the careful choice of these parameters matters.

Several strategies exist to navigate this search for the optimal parameter values and settings. Among the most widely adopted are grid search, random search, and more advanced techniques, such as Bayesian optimization. To give a bit more insight into each of these approaches, we will highlight what each strategy does:

- **Grid search:** It systematically explores predefined (by you) hyperparameter combinations, ensuring that the search is thorough, but can come at the cost of computational complexity.
- **Random search:** By contrast, random search samples hyperparameter values at random within a specified range, offering a more scalable alternative, particularly when only a few parameters critically impact performance.
- **Bayesian optimization:** It introduces a probabilistic model to guide the search, balancing exploration and exploitation to converge (getting closer to the best possible solution) on optimal configurations (the best hyperparameters) with few evaluations.

By leveraging these hyperparameter tuning methods, in this section, you will be able to unlock significant improvements in model performance, transforming a good model (which is all we have had access to up to this point) into a performant and production-ready model. This chapter introduces these techniques, illustrating how a structured approach to tuning can elevate your machine learning workflows and deliver better predictive results.

Preparing the data and instantiating the model

Let us return to the starting point of our ML journey by revisiting the breast cancer classification dataset introduced in *Chapter 2, Introduction to Scikit-learn for Machine Learning*. This time, we can bypass the initial preparation steps, as the dataset is already cleaned and perfectly suited for demonstrating example binary classification use cases.

In the following steps, we will import the data, process it, and split it into a holdout partition:

1. Get the breast cancer dataset from the **sklearn.datasets** module and the imports needed for splitting the data:

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
```

2. Create a variable to hold the data from the breast cancer dataset:

```
data = load_breast_cancer()
```

3. Set the target (y) and feature (X) variables:

```
X, y = data.data, data.target
```

4. Split our dataset into a train and test set partition, retraining 20% of the data for the testing set and setting a random seed value, so the results can be replicated:

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2,
                                                    random_state=42)
```

5. We now have the data in the format we need for working with; there is no need for scaling, as we will be working with a gradient boosted model (which belongs to the family of boosting models):

```
from xgboost import XGBClassifier
```

```
xgb_clf = XGBClassifier(
    use_label_encoder=False,
```

```
    eval_metric='logloss')
```

6. We have chosen a Gradient Boosted model, which combines the strengths of decision trees with gradient-based optimization. This approach not only handles tabular data effectively but also provides several key hyperparameters we can tune to improve performance, such as the maximum depth of each tree, the number

of trees (estimators) to grow, and the learning rate, which controls how quickly the model adapts to the errors during training.

Let us look at the first hyperparameter optimization tool in our toolbox. We will start with grid search in the following section.

Grid search

Grid search can be thought of as a brute-force search through a checklist of possible hyperparameter combinations. You tell the search which hyperparameters you want to tune and what to try, and it will:

- Create a grid of all possible combinations
- Train and evaluate the model using each combination
- Select the best set based on a performance metric (like accuracy, F1 score, precision or log loss)

Let us switch from the tuning a radio analogy; to open a safe, we try each possible code combination on the safe until it opens, except here you are unlocking better model performance and the ability to generalize to new examples. A visualization in *Figure 9.1* has been included to understand how two hyperparameters get tuned:

Grid Search Process

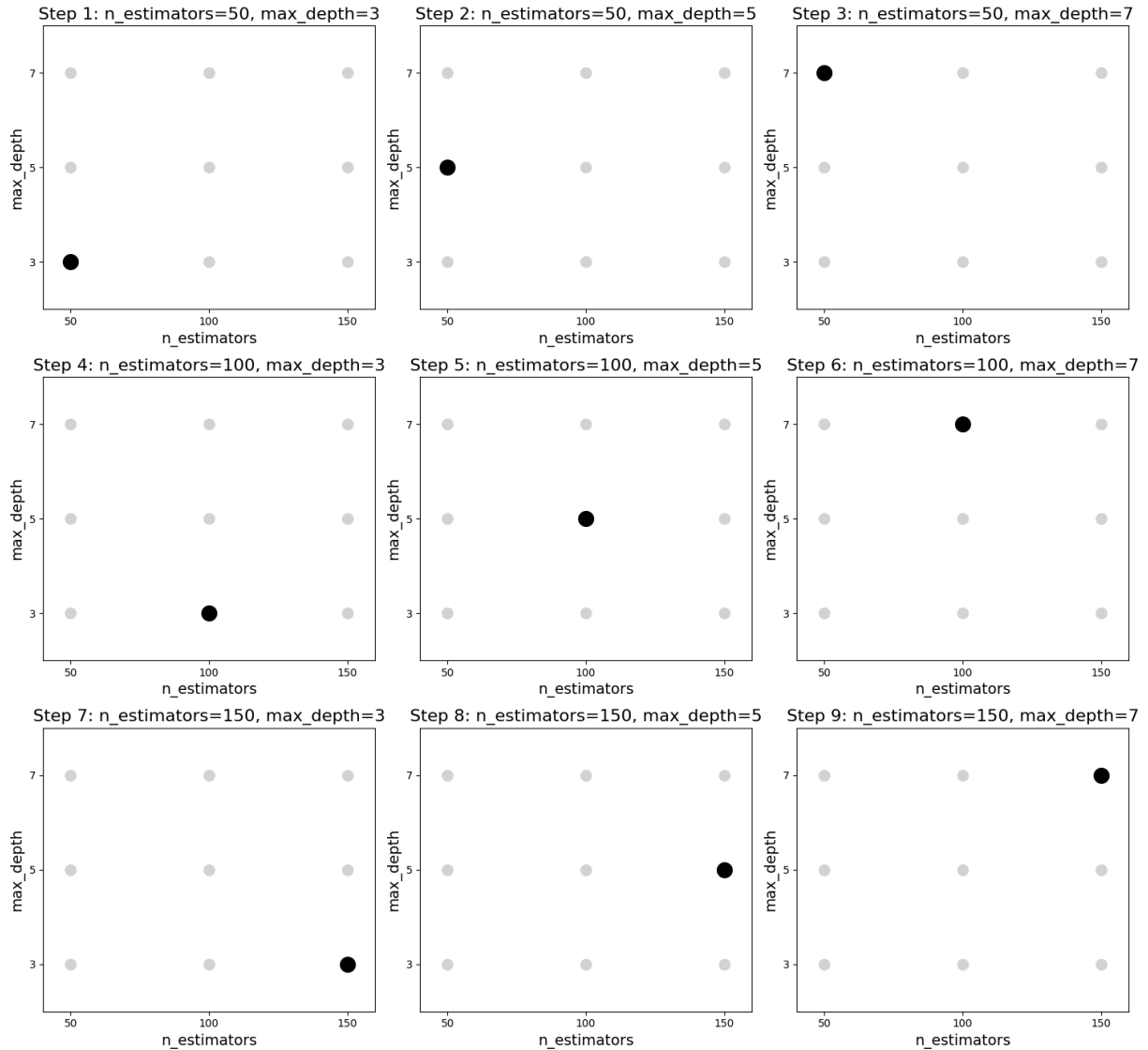


Figure 9.1: Grid search visualized

The grids in *Figure 9.1* can only be created from our parameter grid in 2-dimensions. If we were to add any more hyperparameters to optimize, we as humans would not be able to conceptually include this. To perform a grid search on our XGBoost model we created in the previous section, we can follow these steps:

1. Create a hyperparameter dictionary to hold the parameters we want to optimize. These can be much larger than the example provided,

but come at the cost of computational power:

```
grid_params = {  
    'n_estimators': [50, 100, 150],  
    'max_depth': [3, 5, 7],  
    'learning_rate': [0.001, 0.01, 0.1]  
}
```

2. To check the number of model combinations this will try, we can use the following code:

```
len(grid_params['n_estimators']) * len(grid_params['max_depth']) * len(grid_params['learning_rate'])
```

3. We need to import the **GridSearchCV** function to do searching effectively:

```
from sklearn.model_selection import GridSearchCV
```

4. Next, we instantiate the grid search by providing our classifier as the estimator to be optimized, along with the hyperparameter grid defined as a dictionary. We specify the number of cross-validation folds, select the scoring metric (in this case, the F1 score), and set **n_jobs=-1** to utilise all available CPU cores for parallel processing, which helps improve efficiency by reserving one core for system stability:

```
grid_search = GridSearchCV(  
    estimator=xgb_clf,  
    param_grid=grid_params,  
    cv=5, scoring='f1',  
    verbose=1, n_jobs=-1)
```

5. We have everything in place to commence grid searching over our hyperparameters. The last step is to fit the model:

```
grid_search.fit(X_train, y_train)
```

6. Upon running *Step 5* you will see a message stating: **“Fitting 5 folds for each of 27 candidates, totalling 135 fits”**.

7. To extract the best parameters, we can use the following code, which will use an F-string to print out the best parameters, which are `{'learning_rate': 0.1, 'max_depth': 3, 'n_estimators': 150}`:

```
print("Best Grid Search Params:", grid_search.best_params_)
```

8. To get the F1 score of the best estimator, we can use the following code line:

```
from sklearn.metrics import f1_score
```

```
print("Grid Search F1 Score:",
```

```
      f1_score(y_test,
```

```
      grid_search.best_estimator_.predict(X_test)))
```

9. Executing the code in *Step 8* reveals that the highest F1 score achieved across all models, with the optimal estimator selected, is **97%**. This indicates an excellent balance between precision and recall.

The only modification required for your specific use case pertains to the grid parameters. You may incorporate additional hyperparameters for tuning or extend the parameter values within the dictionary lists. Subsequently, we will delve into the methodology of random search.

Random search

While grid search systematically works through every possible combination of hyperparameters, random search takes a different, more flexible approach. Instead of exhaustively checking each option, random search samples a combination of hyperparameters stochastically (randomly) from the ranges of distributions you define.

You still tell the search which hyperparameters to tune and what ranges to consider, but rather than testing them all, the searching algorithm will:

- Randomly select a set number of combinations to try
- Train and evaluate the model using each chosen combination, at random
- Return the best performing set based on a chosen performance metric, the same way as with grid search

Think of random searching like spinning the dials on a safe, but instead of trying every possible combination, you randomly guess a handful of codes, and surprisingly often, you get close enough to unlock the safe faster than if you had tried every option. This method is particularly useful when the hyperparameter space is large (lots of them). You suspect only a few hyperparameters significantly impact performance, and you want quicker results, without the heavy computational cost of a full grid search.

A visualization in *Figure 9.2* illustrates how random search explores the hyperparameter space. Notice how it scatters its attempts across the grid, rather than following a structured path:

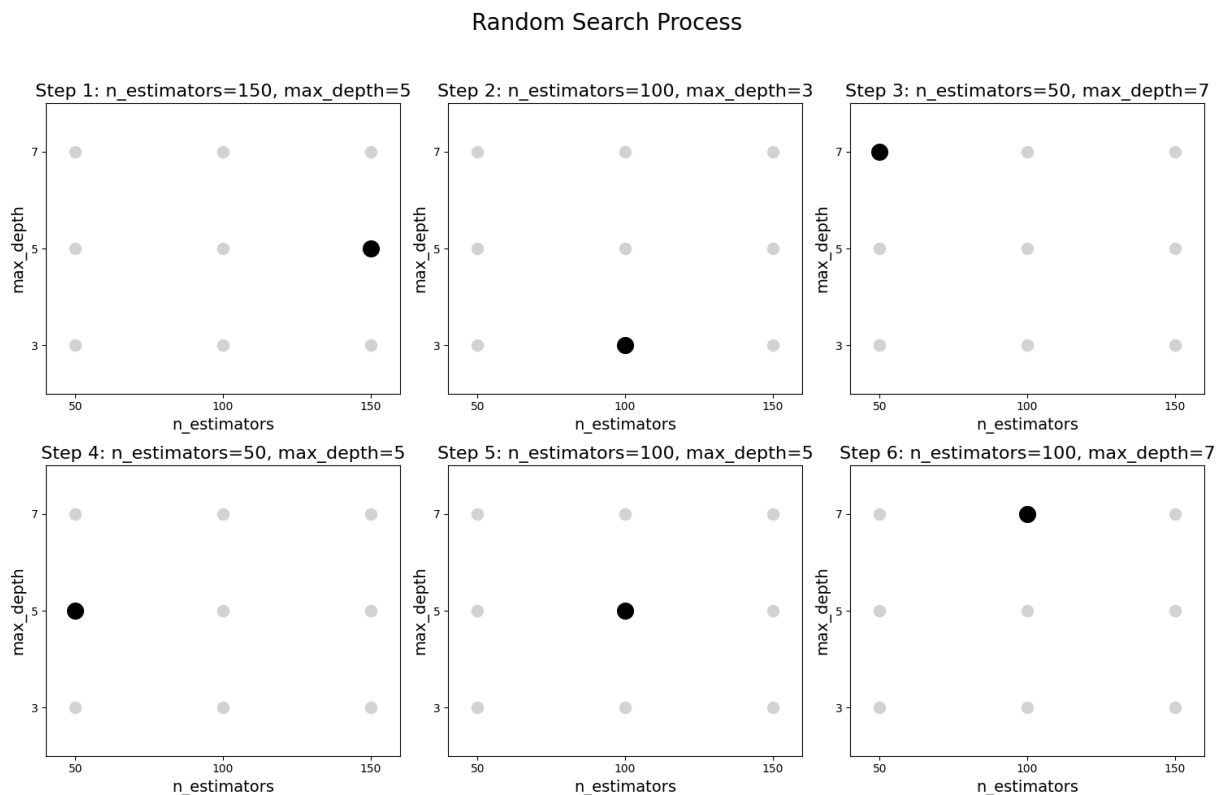


Figure 9.2: Random search visualized

To conduct a random search in Python is relatively simple. All you need to do is follow these steps:

1. Define a hyperparameter search space (**random_params**) to guide randomized tuning, where **n_estimators** controls the number of trees, **max_depth** limits tree complexity, **learning_rate** adjusts the learning speed, and **subsample** introduces randomness to reduce overfitting, using **np.arange** to generate sequences of integers and **np.linspace** to create evenly spaced float values within a range:

```
import numpy as np

random_params = {
    'n_estimators': np.arange(50, 200, 10),
    'max_depth': np.arange(3, 10),
    'learning_rate': np.linspace(0.01, 0.3, 30),
    'subsample': [0.6, 0.8, 1.0]}
```

2. We then create a **RandomizedSearchCV** object, which we name **random_search**, to tune the **XGBoost** model by testing 20 random combinations from the **random_params** dictionary, using 5-fold cross validation, with the aim of optimizing the F1 score, setting progress to be displayed as the tuning processes is in progress (using **verbose**) and ensuring that the results you get when you are following along are reproducible, using **random_state** (with a seed value of 42). Furthermore, we aim to use all but one of the CPU cores available, by setting **n_jobs=-1**:

```
from sklearn.model_selection import RandomizedSearchCV

random_search = RandomizedSearchCV(
    estimator=xgb_clf,
    param_distributions=random_params,
    n_iter=20, cv=5, scoring='f1',
```

```
verbose=1, random_state=42,  
n_jobs=-1)
```

3. All that is left from this point is to fit the model on our training target and features:

```
random_search.fit(X_train, y_train)
```

4. Once this has been run, we can use the same **best_params** attribute of the **random_search**:

```
print("Best Random Search Params:", random_search.best_params_  
_)
```

5. To obtain the F1 score for the best estimator, we used the following:

```
print("Random Search F1 score:", f1_score(y_test,  
random_search.best_estimator_.predict  
(X_test)))
```

6. The results from *Step 5* show that the random search process has been more successful than the grid search, albeit with grid search, we were only using a small search space to optimize the model. The random search shows a F1 score of 97% (or 0.972).

In the next section, we will introduce a package called **Optuna** (<https://optuna.org/>), as we will need this for doing Bayesian optimization, as this is not natively supported in scikit-learn.

Bayesian optimization with Optuna

Optuna (<https://optuna.org/>) is a powerful hyperparameter optimization framework that leverages advanced techniques, like Bayesian optimization and effective parallelization, making it a competing choice over scikit-learn's traditional methods, such as grid and random search. This does not resemble scikit-learn, which does not natively support Bayesian optimization. Optuna intelligently explores the hyperparameter space to achieve faster convergence with fewer evaluations. Additionally, Optuna

excels at its ability to parallelize trials across multiple cores or distributed systems that further accelerate hyperparameter tuning, making it ideal for complex, large-scale machine learning tasks where efficiency and scalability are critical. One final point is that it is agnostic to the modeling framework of choice.

Getting on to understanding what Bayesian optimization is. This optimization algorithm smartly searches for the best hyperparameters by learning from past results, instead of blindly guessing (like grid or random search). The algorithm tries to predict where the best values might be, so it does not waste time testing bad options. To understand this further, let us break this down into a series of steps:

1. **Starts with a few trials:** Testing a few random sets of hyperparameters to get an idea of how they perform; in Bayes' Theorem, this would be the prior belief step.
2. **Builds a model of the performance:** Uses these past results to create a probabilistic model (called a surrogate model) that predicts how different hyperparameters might perform without running them.
3. **Posterior equation:** The *Prior* is what we believed about hyperparameter performance before seeing data; the *Likelihood* is how likely the observed results are, given different hyperparameters, and the *Posterior* equation is defined as follows:

$$\text{Posterior} = \frac{\text{Likelihood} \times \text{Prior}}{\text{Evidence}}$$

4. **Decides where to search next:** Let us call this smart guessing. This uses an acquisition function to balance which new areas to explore (exploration) and provide focus to the known good areas (exploitation) that enables the model to pick the next most promising hyperparameters to try.

5. **Run the next trial:** The chosen hyperparameters are tested on your actual machine learning model. The true performance is observed; this becomes new data to feed back into the Bayesian process.
6. **Update the model with new data:** It improves the surrogate model with the latest result, so it becomes smart about where to search next. The Bayesian explanation would be that with each new trial result, the surrogate model updates its posterior belief about where the best hyperparameters are likely to be. This is an application of what is known as Bayesian updating, i.e., turning yesterday's posterior into today's prior
7. **Repeat steps 3 through 5:** The continuous learning cycle happens in Bayesian thinking, that updates the beliefs as new evidence arrives and always acts on the most current data led understanding of the search space.

The visualization in *Figure 9.3* attempts to illustrate the Bayesian way of navigating the search space:

Bayesian Optimisation Process (Simulated)

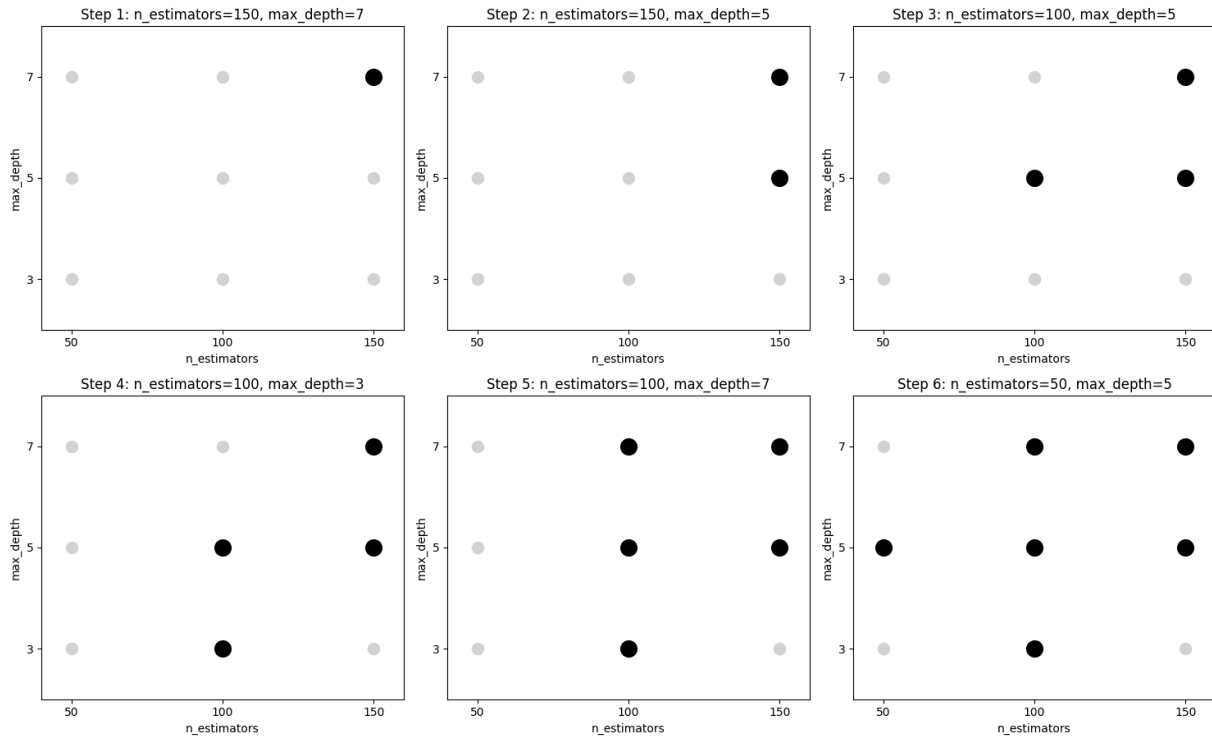


Figure 9.3: Bayesian Optimization (simulated) visualization

Now that the process has been explained, let us put Optuna (please make sure this is installed using **pip install optuna**) and Python to work to optimize our XGBoost model:

1. Import **Optuna** and the **TPESampler** (this is the sampler that uses Bayesian optimization under the hood):

```
import optuna
```

```
from optuna.samplers import TPESampler
```

2. Optuna is built around the concept of an objective function, which defines the metric or outcome you aim to optimize, whether that is maximizing accuracy, F1 score, or minimizing error. Within this function, Optuna introduces the concept of a trial, where each trial represents a single run of the model using a unique combination of hyperparameters. Every time the model is trained and evaluated with different hyperparameter values; it constitutes a new trial in

the optimization process. To explain further, the trial object is used inside the objective function to define how Optuna should explore the hyperparameter space, using methods like **trial.suggest_int**, **trial.suggest_float**, and **trial.suggest_categorical** to guide the search. This allows Optuna to intelligently test different configurations across multiple trials, efficiently honing in on the best-performing hyperparameters using Bayesian optimization techniques. The params then get unpacked into the XGBClassifier model as parameters and then we calculate the F1 score and return it, if you want to optimize by a different metric, just change the cross validation scoring metric, it is as simple as that:

```
import optuna

from sklearn.model_selection import cross_val_score
from xgboost import XGBClassifier

from sklearn.metrics import f1_score

def objective(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 50, 200),
        'max_depth': trial.suggest_int('max_depth', 3, 10),
        'learning_rate': trial.suggest_float('learning_rate',
                                              0.001, 0.3),
        'subsample': trial.suggest_float('subsample', 0.6, 1.0),
        'use_label_encoder': False,
        'eval_metric': 'logloss',
        'n_jobs': -1
    }
    model = XGBClassifier(**params)
    f1 = cross_val_score(model, X_train, y_train,
                        cv=5, scoring='f1').mean()
```

```
return f1
```

3. Next, we instantiate the **TPESampler** for using Bayesian optimization:

```
from optuna.samplers import TPESampler
```

```
tpe_sampler = TPESampler()
```

4. This prepares us to create the Optuna study, where we define how the optimization process should evaluate success. Since our goal is to maximize the model's performance by achieving the highest possible F1 score, we set the direction parameter to "maximize". If, instead, we were working with a metric where lower is better, such as minimizing a regression error (e.g., MSE), we would set the direction to "minimize":

```
study = optuna.create_study(direction='maximize',
```

```
sampler=tpe_sampler)
```

5. We have everything in place to then run our optimization of hyperparameter, with the **n_trials** being how many times we run the optimization:

```
study.optimize(objective, n_trials=30)
```

6. To obtain our best parameters, after *Step 5* has completed, we can access the **best_params** attribute:

```
best_params = study.best_params
```

7. Now that we have our best parameters, we will pass these to the final model we will fit. We use dictionary unpacking to pass the **best_params** to the final model:

```
best_model = XGBClassifier(**best_params,
```

```
use_label_encoder=False,
```

```
eval_metric='logloss', n_jobs=-1)
```

8. From here, we will fit the best model on the training target and features:

```
best_model.fit(X_train, y_train)
```

9. Finally, we will get our metrics from the process, and we will explore how this method stacks up in the hyperparameter comparison section in this section:

```
optuna_f1 = f1_score(y_test, best_model.predict(X_test))
```

```
print("Best Optuna (Bayesian) Params:», best_params)
```

```
print(f"Optuna Bayesian F1 score on Test Set: {optuna_f1:.4f}")
```

10. Optionally, you can import the **optuna.visualization** package to plot the optimization history over the chosen number of trials:

```
import optuna.visualization as vis
```

```
vis.plot_optimization_history(study).show()
```

Step 10 will create the plot, as shown in *Figure 9.4*:

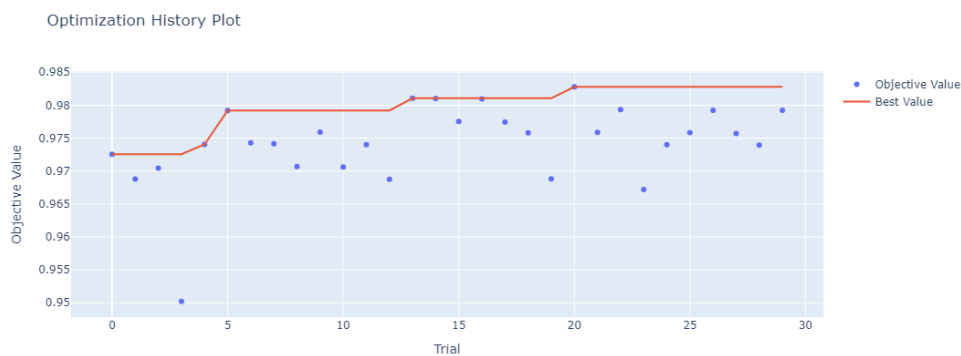


Figure 9.4: Bayesian optimization history plot

11. To understand which hyperparameters had the most significant impact on improving the F1 score, you can use the command below to visualize parameter importance. This insight is particularly valuable when working with complex models that have millions or even billions of tunable parameters, such as large

language models, where identifying the most influential settings can greatly simplify and focus future optimization efforts:

```
vis.plot_param_importances(study).show()
```

Figure 9.5 will be shown on the console, once the line in Step 11 is run:

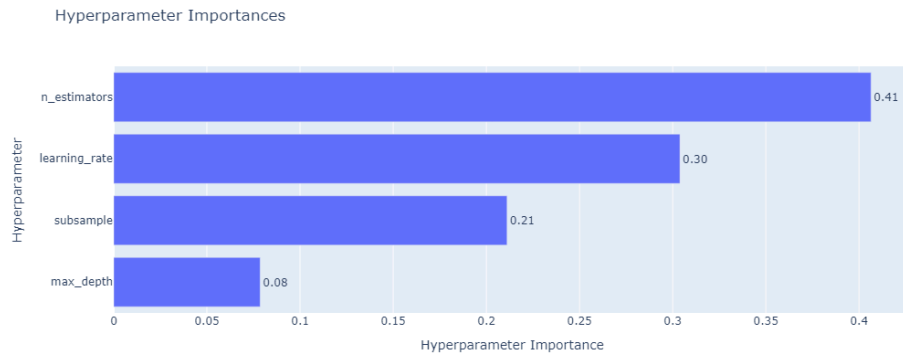


Figure 9.5: Hyperparameter importance plot

In the following section, we will evaluate the outcomes of our hyperparameter tuning efforts.

Evaluation of hyperparameter tuning methods

In this section, we will look at the evaluation of our hyperparameter tuning methods, comparing grid, random, and Bayesian search methods. To get started, we will combine the results into a DataFrame and then create a visualization to compare them:

1. The first step is to ensure that all evaluation metrics are stored in variables and explicitly converted to floating-point numbers for consistency and accuracy in subsequent analysis:

```
import pandas as pd
```

```
grid_f1 = float(f1_score(y_test,  
                        grid_search.best_estimator_.predict(X_test)))
```

```
random_f1 = float(f1_score(y_test,
```

```
random_search.best_estimator_.predict(X_test)))
```

```
optuna_f1 = float(optuna_f1)
```

2. We can now build a DataFrame from a dictionary by creating columns **Method** and **F1_Score**:

```
hp_results_df = pd.DataFrame([  
    {'Method': 'Grid Search', 'F1_Score': grid_f1},  
    {'Method': 'Random Search', 'F1_Score': random_f1},  
    {'Method': 'Optuna Bayesian', 'F1_Score': optuna_f1}])
```

3. In this step, we will use a function (namely **plot_hp_comparison**) in the supporting notebook to this chapter, to plot the metrics for the visualization and comparison step:

```
plot_hp_comparison(hp_results_df, metric_cols=['F1_Score'],  
    x_col_name='Method', palette='Greys',  
    label_position='center')
```

4. Running the line in *Step 3* will generate a comparison, as in *Figure 9.6*:

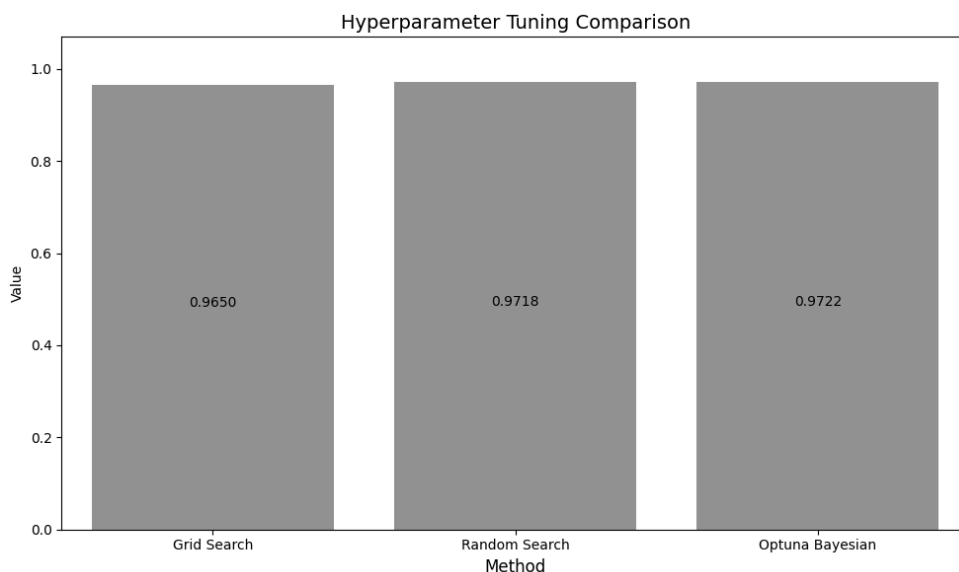


Figure 9.6: Hyperparameter tuning comparison

Figure 9.7 shows that Bayesian and random search yield the best F1 scores. In the next section, we will look at combining recursive feature engineering, dimensionality reduction, and random search to try and improve these results even further.

Feature elimination and dimensionality reduction with LDA

When working with ML models, one of the most effective ways to improve performance, reduce overfitting, and enhance interpretability is by removing redundant or irrelevant features. A popular approach to achieving this is using RFECV, which iteratively prunes away features with the least contribution to model accuracy.

However, beyond feature elimination, another powerful strategy for handling high-dimensional data is dimensionality reduction, and that is where LDA comes in. We will focus on LDA, as we have touched on PCA extensively in *Chapter 7, Handwritten Digits Dimensionality Reduction*.

Detour into Linear Discriminant Analysis

LDA is best known as a classification technique; however, it serves as a powerful dimensionality reduction tool, especially when the goal is to maximize class separability.

Not like PCA, which we have already studied, which is unsupervised and focuses on maximizing variance without regard to class labels, the difference is that LDA is a supervised learning technique, as it explicitly tries to project data onto a lower-dimensional space in a way that best separates the classes. For our breast cancer example, it would aim to separate the classes of benign or malignant breast cancer.

Let us focus on the goal of LDA in separating our two classes from the breast cancer dataset. We have two classes (malignant and benign), and the features in the dataset have measurements such as mean radius, texture, smoothness of the tumor, etc. Some of these features are more helpful in

distinguishing malignant and benign tumors clearly. LDA helps us *find the best direction* in this 30-dimensional space to separate these two classes. This involves:

1. **Compute the mean vectors for each class:** Think of this as finding the average patient for each group (one average for benign and one for malignant), and then for each feature (tumor size, texture), we calculate the average value within each class. This helps us isolate where the class is located in the feature space.
2. **Compute the within-class scatter matrix (SW):** This measures how *spread out the data points are within each class*. For example:
 - a. If malignant tumors have a wide variety of sizes and textures, they will have a *large within-class scatter*.
 - b. If benign tumors are very consistent in their features, they will have a *small within-class scatter*.
 - c. Effectively, we want classes to be as tight and compact, because tightly grouped points are easier to separate.
3. **Compute the between-class scatter matrix (SB):** Measuring how far apart the two class averages are, for example, if the average malignant tumor is very different from the average benign tumor, the between-class scatter will be large. The motivation here is to make the class centers as far apart as possible to make separation easier.
4. **Solve the generalized eigenvalue problem:** Now comes the magic part, where LDA figures out the best direction to project the data, using the formula:

$$S_W^{-1} S_B \mathbf{w} = \lambda \mathbf{w}$$

- a. In the equation, we are telling LDA to find me a direction (called $\mathbf{w}$) where the classes are as far apart as possible

(maximize between-class scatter) and do this while keeping each class tight (minimize within-class scatter).

- b. The solution gives us $\mathbf{w}$ (the direction to project the data, called a **linear discriminant**) and λ (a number that tells us how good that direction is for separating classes).

Once we have this **best direction** ($\mathbf{w}$), we can project all our patient data points onto this line, and the result is that malignant and benign cases are now *spread out along a single axis*, making them much easier to classify. The following figure visualizes this process:

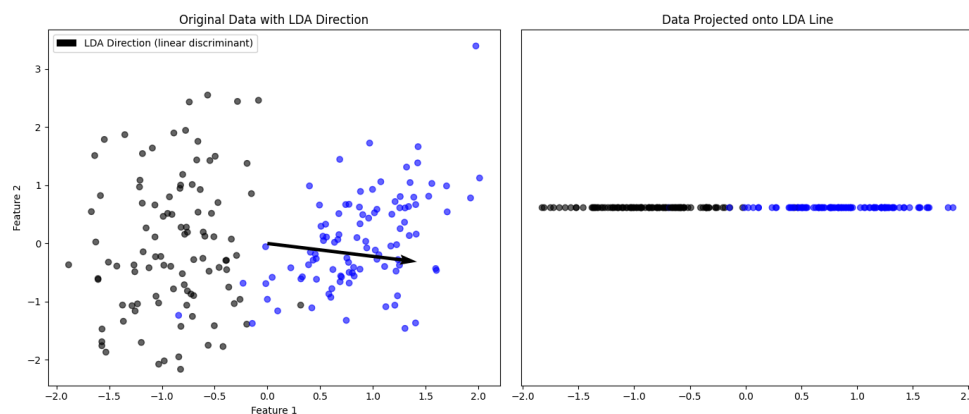


Figure 9.7: LDA process visualization

Mutation of methods to improve model performance

In this section, we will combine feature selection, elimination and hyperparameter tuning to evaluate various model permutations and how models can be improved through a combination of methods.

Baseline model and LDA

In this sub-section, we will fit a baseline XGBoost model and then use LDA to reduce the complexity of the data. This will be achieved in the following steps:

1. Create an empty results dictionary to store the intermediate results:

```
results = {}
```

2. Fit the baseline model, using an XGBoost classifier:


```
baseline_model = XGBClassifier(use_label_encoder=False,  
                                eval_metric='logloss',  
                                random_state=42)
```

```
baseline_model.fit(X_train, y_train)
```

3. Predict on the testing set and store the accuracy and F1 score in the intermediate dictionary as a key:

```
from sklearn.metrics import accuracy_score, f1_score  
results['Baseline'] = [accuracy_score(y_test, y_pred_base),  
                        f1_score(y_test, y_pred_base)]
```

4. Scale the data ready for the LDA step:

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

5. Initialize and fit, and transform the train and test sets, retaining one component:

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
```

```
lda = LDA(n_components=1)
```

```
X_train_lda = lda.fit_transform(X_train_scaled, y_train)
```

```
X_test_lda = lda.transform(X_test_scaled)
```

6. Train and fit a XGBoost classifier on the scaled and LDA reduced dataset:

```
lda_model = XGBClassifier(use_label_encoder=False,
```

```
                           eval_metric='logloss',
```

```
                           random_state=42)
```

```
lda_model.fit(X_train_lda, y_train)
```

7. Make predictions with the fitted model in the previous step on the testing data:

```
y_pred_lda = lda_model.predict(X_test_lda)
```

8. We now append the accuracy and F1 score results from the LDA model to our intermediate dictionary:

```
results['LDA Only'] = [  
    accuracy_score(y_test, y_pred_lda),  
    f1_score(y_test, y_pred_lda)]
```

We now have our baseline model and LDA model. In the next section, we will introduce recursive feature elimination.

Recursive feature elimination

We have previously explored **recursive feature elimination (RFE)**, and now we revisit it with RFECV, which enhances RFE by automatically selecting the optimal number of features using cross-validation. RFECV helps improve model performance by systematically removing less important features, reducing overfitting, and simplifying complexity. We will apply RFECV only, and then with LDA:

1. We will initially instantiate the model to use for recursive feature elimination with cross validation:

```
rfecv_model = XGBClassifier(use_label_encoder=False,  
                             eval_metric='logloss',  
                             random_state=42)
```

2. The function initializes recursive feature elimination with 5-fold cross-validation, using the model defined in the prior step. The step parameter ensures that one feature is removed at a time, evaluating different train-test split combinations to select the best feature set:

```
from sklearn.feature_selection import RFECV  
rfecv = RFECV(  
    rfecv_model,  
    step=1,  
    cv=5,  
    scoring='f1',  
    min_features_to_select=1,  
    min_features_to_remove=1)
```

```
estimator=rfevcv_model,  
step=1, cv=5,  
scoring='f1')
```

3. We then fit and transform the train and test data, to perform recursive feature elimination on:

```
X_train_rfevcv = rfevcv.fit_transform(X_train, y_train)
```

```
X_test_rfevcv = rfevcv.transform(X_test)
```

4. Now we have the redundant features removed, we can fit a model to this reduced set:

```
rfevcv_model = XGBClassifier(use_label_encoder=False,  
                             eval_metric='logloss',  
                             random_state=42)
```

```
rfevcv_model.fit(X_train_rfevcv, y_train)
```

5. We make a prediction on the test set and append the results to the intermediate dictionary:

```
y_pred_rfevcv = rfevcv_model.predict(X_test_rfevcv)
```

```
results['RFEVCV Only'] = [  
    accuracy_score(y_test, y_pred_rfevcv),  
    f1_score(y_test, y_pred_rfevcv)]
```

In the supporting workbook, we also demonstrate the use of RFE with LDA, as well as a combination of RFEVCV, LDA, and randomized hyperparameter search. For details on these implementations, please refer to the supporting code to avoid repetition. In the next section, we will explore a filter method of feature selection using chi-square, highlighting its value for improving classification models.

Feature selection with chi-square

We have used several filter-based feature selection methods throughout this book; techniques that rely on statistical measures to evaluate feature relevance before applying any machine learning model. Common examples include removing low-variance features, filtering based on correlation, or using tests like **Analysis of Variance (ANOVA)** or mutual information to highlight informative features.

In this section, we focus on the chi-square test, a simple yet effective method for improving classification models. chi-square measures how strongly each feature is associated with the target variable by comparing observed and expected frequencies, helping to identify features that have a meaningful statistical relationship with class labels. Applied to the Breast Cancer dataset, this allows us to quickly filter out less relevant features, reducing noise, simplifying the model, and potentially improving accuracy by focusing only on the metrics most indicative of whether a tumor is benign or malignant. In the following steps, we will use this selection technique with and without recursive feature elimination:

1. We define a function (namely **feature_chi2_selection**) function that select the top k features by: shifting feature values to be non-negative (as chi-square does not work with negative values); calculates the chi-square statistic for each feature to measure its association with the target variable (benign versus malignant) and returns the top features (k) with the highest chi-square scores, indicating the strongest relationship with the target variable:

```
from sklearn.feature_selection import SelectKBest, chi2
```

```
def feature_chi2_selection(X, y, k=5):
```

```
    X_non_neg = X - X.min()
```

```
    selector = SelectKBest(score_func=chi2, k=k)
```

```
    X_new = selector.fit_transform(X_non_neg, y)
```

```
    return X_new, selector
```

2. Putting this into action, we apply it to our training set and transform our testing set (in the transform step of the testing data, we need to make sure there are non-negative values, which explains the `X_test - X_test.min()` calculation):

```
X_chi2_train, chi2_selector = feature_chi2_selection(X_train,  
                                                    y_train,  
                                                    k=5)
```

```
X_chi2_test = chi2_selector.transform(X_test - X_test.min())
```

3. In the next step, we will train the XGBoost classifier, fit the model to the train and test data sets, and then make predictions on the test set:

```
chi2_only_model = XGBClassifier(use_label_encoder=False,  
                                eval_metric='logloss', random_state=42)  
chi2_only_model.fit(X_chi2_train, y_train)  
y_pred_chi2_only = chi2_only_model.predict(X_chi2_test)
```

4. We now store the results in our intermediate dictionary.

In the next couple of steps, we will use our chi-square feature extractor with recursive feature elimination, to pop up any features the chi-square selector did not reduce:

1. In this step, we will do several code steps. We will fit the XGBoost model, instantiate the chi-square feature extractor, and transform the training and testing sets:

```
rfecv_chi2_model = XGBClassifier(use_label_encoder=False,  
                                eval_metric='logloss',  
                                random_state=42)  
rfecv_chi2 = RFECV(estimator=rfecv_chi2_model,  
                   step=1, cv=5, scoring='f1')
```

```
X_train_chi2_rfecv = rfecv_chi2.fit_transform(X_chi2_train, y_train)
```

```
X_test_chi2_rfecv = rfecv_chi2.transform(X_chi2_test)
```

2. Fit on the training data and make a prediction on the testing set, as we have done in these steps throughout:

```
rfecv_chi2_model.fit(X_train_chi2_rfecv, y_train)
```

```
y_pred_chi2_rfecv = rfecv_chi2_model.predict(X_test_chi2_rfecv)
```

3. Store the results in the intermediate dictionary, as we have done throughout:

```
results['Chi2 + RFECV'] = [accuracy_score(y_test, y_pred_chi2_rfecv),
```

```
f1_score(y_test, y_pred_chi2_rfecv)]
```

Evaluation of feature selection and elimination methods

In this section, we have explored how feature selection, elimination, and dimensionality reduction can enhance model performance, along with the use of random search for further optimization. By reducing complexity, feature selection helps models better identify underlying patterns in the data. Now, we will bring these concepts together by comparing and evaluating the results across different approaches:

1. We are going to use the function we created in the *Evaluation of hyperparameter tuning methods* section to create the comparison bar chart; however, before we do this, we need to reshape the results dictionary to match the expected format for the function to process it effectively. We will do this as follows:

```
comparison_df = pd.DataFrame(results,
```

```
index=['Accuracy', 'F1 Score']).T
```

2. The T attribute stands for transpose; what this does is switch rows to columns in the DataFrame. Without this, the comparison names

would be the columns, and **Accuracy** and **F1 score** would be the index; by adding the T, we effectively switch these around.

3. As this will store the comparison methods on the index, we actually want to reset this index to make it a column called **Method**. This is how we will achieve it:

```
comparison_df_reset = comparison_df.reset_index().rename(  
    columns={'index': 'Method'})
```

4. The final stage is to use the **plot_hp_comparison** function that is defined in the supporting GitHub repository:

```
plot_hp_comparison(comparison_df_reset,  
    ['Accuracy', 'F1 Score'],  
    figsize=(16, 8),  
    x_label_rotation=None,  
    label_fontsize=12,  
    label_position='center',  
    palette='Greys',  
    title='Feature Selection and Dimensionality Reduction  
on comparison')
```

5. Running this line, will produce a visual, as shown in *Figure 9.8*:

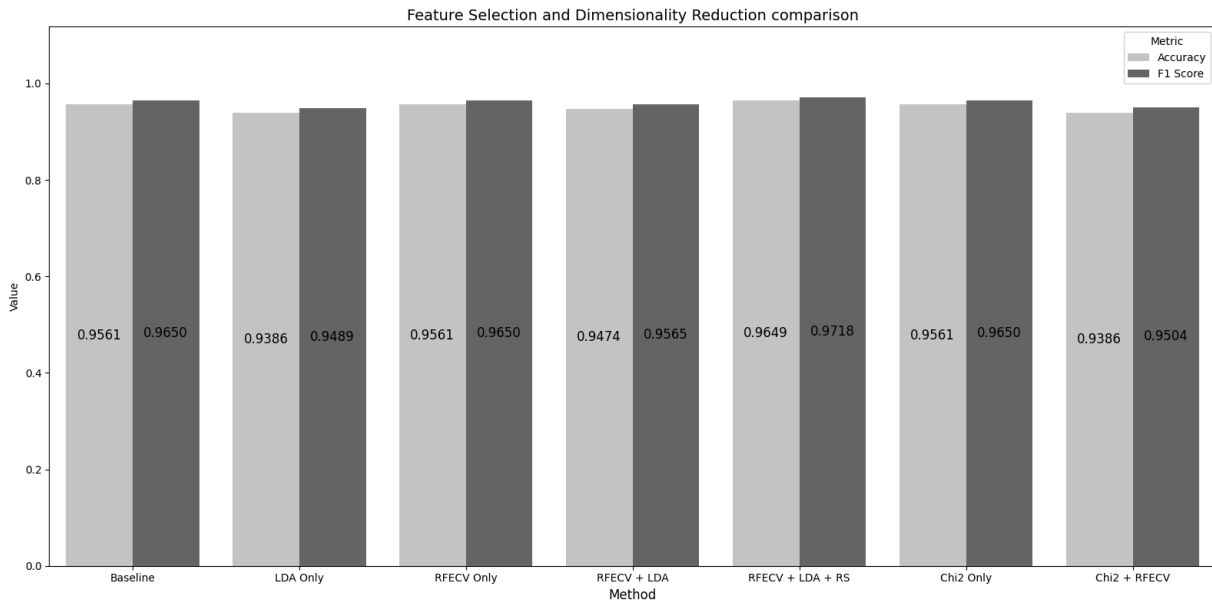


Figure 9.8: Feature selection and dimensionality reduction comparison

The clear winner, looking at accuracy and F1 score, is the combination of Linear Discriminant Analysis, recursive feature elimination, and using hyperparameter tuning, with random search, bringing together everything we have learned about in this section hitherto.

In this section, we have explored key techniques to improve model performance, including dimensionality reduction with LDA, feature elimination using RFECV, and filter-based selection methods like chi-square. While these are powerful tools, there are many other feature engineering strategies that can further enhance your models. Techniques such as creating interaction features (combining existing features to capture relationships), polynomial feature expansion (adding non-linear terms), and binarization or discretization (converting continuous variables into categorical bins) can reveal hidden patterns in the data. Additionally, methods like target encoding for categorical variables, scaling and normalization, or applying domain-specific transformations can significantly impact model effectiveness. Combined with approaches like handling missing values, outlier treatment, and feature extraction from text

or dates, these strategies offer a wide range of options to refine your dataset and help your models perform at their best.

Now we move on to model improvements with ensemble methods.

Ensemble learning

In the world of machine learning, no single model is perfect. Each algorithm has its own strengths, weaknesses, and limitations. This is where ensemble learning comes in; this is a technique that combines multiple models to produce better predictive performance than any individual model could achieve alone.

You may wonder what ensemble learning is. It is a method where we build a collection (an ensemble) of models and combine their predictions to improve accuracy, robustness, and generalization. This works, rather than relying on one model to make decisions, ensemble learning leverages the *wisdom of the crowd*. It is the same as asking several experts for advice instead of trusting only one; combining multiple models often leads to better outcomes. We have actually used boosting and bagging throughout this book and have broken down the logic of how gradient boosting and random forests work already in *Chapter 6, House Price Estimation with Regression Methods*.

Why we use ensemble learning is detailed as follows:

- **Improved accuracy:** Multiple models working together can correct each other's mistakes.
- **Reduced overfitting:** Ensembles tend to generalize better to unseen data.
- **Balanced bias and variance:** Helps in handling both underfitting (underfitting occurs when a model is too simple or not trained enough, resulting in poor performance on both the training data and new, unseen data, it fails to capture the underlying patterns. Essentially, the model misses the mark and does not learn enough

to be effective) and overfitting (Overfitting happens when a model becomes too tailored to the training data, capturing noise instead of general patterns, and struggles to perform well on new, unseen data).

- **More robust predictions:** Less sensitive to noise or peculiarities in the data.

In short, ensemble learning helps build models that are reliable, stable and accurate. Let us delve into each of them in each sub section, with examples in Python, to bring the concepts to life in scikit-learn.

Bagging

Bagging stands for **bootstrap aggregation**. The aim of the model is to:

- Create multiple versions of a model by training each on a different random subset of the data.
- Combine their predictions by voting for classification (one model predicts malignant cancer, another predicts benign for another example, many hundreds of models go through the same voting process and arrive on a vote for one patient, like having hundreds of doctors reviewing breast cancer scans to predict cancer) and averaging for regression (multiple models make a prediction on the prices of a house and then these are averaged in various ways). This helps to get a final, more accurate result.

The process of bagging is:

1. **Random sampling:** From the original dataset, create several new datasets by randomly selecting data points with replacement (the *with replacement* part means that if one patient is selected for analysis, they are placed back into the pool, so they might be selected again, either by the same model or another one. To extend a doctor analogy, imagine several doctors each receiving their own random list of patients to assess for cancer. Because the lists are

created with replacement, some patients might appear multiple times on a single doctor's list or be reviewed by multiple doctors. Once all doctors have made their diagnoses, the final decision is made by combining their opinions, like a medical board vote).

2. **Train multiple models:** Train a separate copy of the same model (like a decision tree) on each of these random datasets.
3. **Combine predictions:**
 - a. **For classification:** Let each model *vote* for a class, gather the votes and majority wins.
 - b. **For regression:** Take the average of all the model predictions.

The most famously studied example of bagging is the random forest algorithm that builds many decision trees, adds extra randomness by choosing random subsets of features for each tree (some may get different tumor dimensions from others), and the final prediction is the average of the majority votes of all trees.

We will now train a bagging model using the breast cancer dataset we have been working with throughout this chapter. To avoid repetition, we will begin from the scaling step. Please refer to the accompanying notebook for data preparation and splitting instructions:

1. The first step will be to scale our training and test data, using the **fit_transform** and transform methods, which you will be familiar with by now:

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

2. We will define a few key variables, so we do not end up double typing, these are around the number of folds to use, the number of iterations, and our reproducible seed value:

FOLDS = 5

N_ITERATIONS = 5

RANDOM_SEED = 42

3. In this step, we define our base estimator, a decision tree classifier, which will serve as the foundation for our ensemble. We will then build a **BaggingClassifier** around this base estimator. The **BaggingClassifier** will create multiple versions of the decision tree, each trained on different random subsets of the data, allowing us to combine their predictions for a more robust and accurate model:

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.ensemble import BaggingClassifier
```

```
from sklearn.model_selection import RandomizedSearchCV
```

```
base_estimator = DecisionTreeClassifier(random_state=RANDOM_SEED)
```

```
bagging = BaggingClassifier(estimator=base_estimator,  
                           random_state=RANDOM_SEED)
```

4. Next, we build a dictionary to specify the parameters of the bagging model, which are the inputs to the decision tree base estimator:

```
bagging_params = {
```

```
    'n_estimators': [10, 50, 100, 150, 200],
```

```
    'max_samples': [0.1, 0.3, 0.5, 1.0],
```

```
    'max_features': [0.5, 1.0]
```

```
}
```

5. In this step, we set up a **RandomizedSearchCV** to efficiently search through a random selection of hyperparameter combinations for our bagging model, using the parameter grid (**bagging_params**), a defined number of iterations

(**N_ITERATIONS**), cross-validation folds (**FOLDS**), and a random seed (**RANDOM_SEED**) to ensure reproducible results:

```
bagging_search = RandomizedSearchCV(bagging,  
                                     bagging_params,  
                                     n_iter=N_ITERATIONS, cv=FOLDS,  
                                     random_state=RANDOM_SEED)
```

6. The last bit is to fit the model, as we will make predictions on all the ensemble methods later, in the comparison section:

```
bagging_search.fit(X_train_scaled, y_train)
```

In the next section, we will examine boosting ensemble learning.

Boosting

Boosting is a technique to turn weak learners into strong learners. Unlike bagging, where models are trained independently in parallel, boosting trains models sequentially. Each new model tries to correct the mistakes made by previous models. The core idea is:

1. Start with a simple model (a weak learner, like a small decision tree).
2. Look at where the weak learner model performs poorly.
3. Train the next model to focus on the mistakes of the previous model.
4. Repeat this process, combining all models into a final, strong prediction.

Over time, boosting literally *boosts* the performance by learning from the errors.

In this section, we will use **XGBoost**, as we have been using this throughout this chapter, as this is one of the most powerful boosting models out there. The aim of **XGBoost** is to build trees sequentially by minimizing

errors through gradient descent techniques (an optimization algorithm used to minimize a function, like a loss function in machine learning, by iteratively adjusting parameters in the direction of the steepest decrease, based on the gradient (slope) of the function). In the following steps, we will fit a boosting model to our breast cancer dataset:

1. Load in the imports we will need for boosting and cross-validation:

```
from xgboost import XGBClassifier
from sklearn.model_selection import GridSearchCV
```

2. Create a XGBoost classifier:

```
boosting = XGBClassifier(random_state=RANDOM_SEED,
                          use_label_encoder=False,
                          eval_metric='logloss')
```

3. To help XGBoost learn effectively and generalize well, we can adjust key hyperparameters like the number of trees, how quickly the model learns, and the complexity of each tree, as defined in the **boosting_params** grid below. These settings control how XGBoost builds its ensemble:

```
boosting_params = {
    'n_estimators': [50, 100],
    'learning_rate': [0.05, 0.1, 0.2],
    'max_depth': [3, 5]}
```

4. To find the combination of hyperparameters that allows XGBoost to perform at its best, we use **GridSearchCV**, which systematically tests different values from our **boosting_params** grid using cross-validation (cv=FOLDS), ensuring the model learns in a way that balances accuracy and generalization across unseen data:

```
boosting_search = GridSearchCV(boosting,
```

```
boosting_params,  
cv=FOLDS)
```

5. Finally, we fit the model on the training and testing datasets. We will use this fit object later to make predictions for the final evaluation section:

```
boosting_search.fit(X_train_scaled, y_train)
```

In the next section, we will look at the final ensemble method, we will cover, called stacking.

Stacking

The aim of stacking is to combine predictions from multiple different models (called **base learners**) to take advantage of their individual strengths. It works by feeding the predictions of these base models into a **meta-model** (otherwise known as a **blender**), which learns how to best combine the models to improve the overall prediction performance. Instead of simply averaging or voting (like bagging or boosting), stacking learns the optimal way to blend predictions through this meta-model, often leading to higher accuracy and robustness, especially when using diverse algorithms.

Let us drill down a little more, in a stepwise way, on how stacking does this:

1. Train multiple **base models** on the training data (these are usually different model types, e.g., tree based, linear, support vector based, etc.).
2. Each of these base models makes predictions (usually on validation data or via multiple training and validation sets by cross validation to avoid overfitting).
3. The predictions from the base models become **input (features)** for a new model, the master **meta-learner**.
4. The **meta-learner** is trained to find the best way to combine these inputs to predict the final output.

In the context of the **breast cancer dataset**, stacking allows us to combine different models—like logistic regression, decision tree, and k-nearest neighbors, each bringing its own strengths to predict whether a tumor is malignant or benign. Since no single model performs best in every situation, the **meta-learner** steps in to learn how to blend their predictions intelligently, rather than simply averaging them. It observes patterns in where each model tends to be right or wrong and adjusts how much the learner *trusts* each one accordingly.

For example, if KNN consistently excels with certain types of data patterns, the meta-learner will give it more influence in those cases. If the models disagree, it may lean towards the one that has historically performed better in similar situations. The meta-learner can even capture complex, non-linear relationships between model outputs, making smarter, context-aware decisions. This approach leads to a more accurate and robust prediction, essential in tasks like cancer diagnosis, where correctly identifying subtle patterns can make all the difference. Now it is time to put this to the test, in the following steps:

1. Define all the estimators to use for modelling. We will use a combination of tree based models, simple boosters such as AdaBoost, and geometric nearest neighbor algorithms:

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.naive_bayes import GaussianNB
```

```
from sklearn.ensemble import AdaBoostClassifier
```

```
from sklearn.ensemble import StackingClassifier
```

```
from sklearn.svm import SVC
```

```
estimators = [
```

```
    ('lr', LogisticRegression(max_iter=1000)),
```

```
    ('svc', SVC(probability=True)),
```



```

('dt', DecisionTreeClassifier(random_state=RANDOM_SEED)),
('knn', KNeighborsClassifier()),
('rf', RandomForestClassifier(n_estimators=150, random_state=R
RANDOM_SEED)),
('naive_bayes', GaussianNB()),
('ada_boost', AdaBoostClassifier(n_estimators=150, random_stat
e=RANDOM_SEED))]

```

2. In this step, we define a **StackingClassifier** that combines our chosen base models (estimators), uses a logistic regression as the meta-learner (**final_estimator**), and, by setting **passthrough=True**, allows the meta-learner to access both the base models' predictions and the original features for improved decision-making:

```

stacking = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression(),
    passthrough=True)

```

3. We now fit the stacking classifier, with the logistic regression meta-learner:

```

stacking.fit(X_train_scaled, y_train)

```

There we have it, all three ensemble learners have been fit on our training data, now is the time to evaluate the results, in the next section.

Evaluation of ensemble methods

In this section, we will use our evaluation function, which we have been using throughout this chapter, to compare our ensemble methods.

To undertake the evaluation, we will use the steps indicated:

1. Gather predictions of all the fitted ensemble models:

```
bagging_preds = bagging_search.predict(X_test_scaled)
```

```
boosting_preds = boosting_search.predict(X_test_scaled)
```

```
stacking_preds = stacking.predict(X_test_scaled)
```

2. Collate all the metrics into a DataFrame. This time, we will add Precision and Recall, alongside our F1 score and accuracy metrics, into a DataFrame, called **metrics_df**:

```
from sklearn.metrics import (recall_score, precision_score,  
                             f1_score, accuracy_score)
```

```
metrics = {
```

```
    "Model": ["Bagging", "Boosting", "Stacking"],
```

```
    "Precision": [
```

```
        precision_score(y_test, bagging_preds),
```

```
        precision_score(y_test, boosting_preds),
```

```
        precision_score(y_test, stacking_preds)
```

```
    ],
```

```
    "Recall": [
```

```
        recall_score(y_test, bagging_preds),
```

```
        recall_score(y_test, boosting_preds),
```

```
        recall_score(y_test, stacking_preds)
```

```
    ],
```

```
    "F1 Score": [
```

```
        f1_score(y_test, bagging_preds),
```

```
        f1_score(y_test, boosting_preds),
```

```
        f1_score(y_test, stacking_preds)
```

```
    ],
```

```
    "Accuracy Score": [
```

```
        accuracy_score(y_test, bagging_preds),
```

```

        accuracy_score(y_test, boosting_preds),
        accuracy_score(y_test, stacking_preds)
    ]}

metrics_df = pd.DataFrame(metrics)

```

3. To plot the metric, we will use our **plot_hp_comparison** function, we defined earlier, to visualize the results:

```

plot_hp_comparison(
    results_df=metrics_df,
    metric_cols=["Precision", "Recall",
                 "F1 Score", "Accuracy Score"],
    x_col_name="Model",
    title="Comparison of Ensemble Methods Performance",
    x_label_rotation=0,
    palette='Greys')

```

4. Running the line in *Step 3* will produce the visualization, as in *Figure 9.9*:

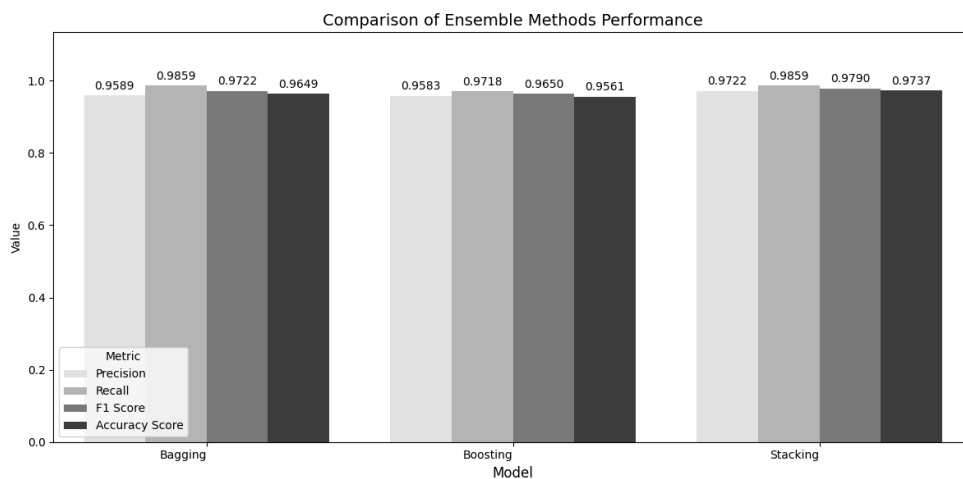


Figure 9.9: Ensemble method comparison

The comparison of ensemble methods on the breast cancer dataset shows that all three approaches, bagging, boosting, and stacking, deliver high

performance, with precision, recall, F1 score, and accuracy consistently above 95%. Among them, stacking stands out slightly, achieving the best balance across metrics, particularly in F1 score and accuracy, highlighting the strength of combining diverse models to improve predictive power. Both bagging and boosting perform well, but bagging edges out boosting, suggesting that reducing variance through methods like random forest is particularly effective on this structured dataset. Importantly, all models maintain a high recall, which is critical in a medical context where correctly identifying malignant tumors is a priority. The consistently strong recall indicates that these ensemble methods are well-suited for minimizing false negatives in cancer detection. Overall, while stacking offers a marginal performance advantage, bagging remains a robust and efficient choice, especially when simplicity and computational speed are considerations.

That brings us to the end of this section. In the next section, we will have a brief look at other factors that can affect model performance.

Other considerations for model improvement

While selecting the right machine learning algorithm and tuning hyperparameters are critical steps in model improvement strategies, there are several factors that are non-ML factors that can significantly impact model performance. Ignoring these aspects can lead to suboptimal results, even if you are using the most advanced algorithms, and this applies to all ML domains (text, tabular, vision, and audio).

In the following list, we will draw attention to these considerations for model improvement:

- **Data quality and consistency:** Poor data quality is one of the biggest obstacles to accurate predictions. These present with issues like missing values, incorrect labels and poorly defined labelling strategies (the label that is applied to the classification by human or LLM labellers), inconsistent formatting, duplicate records, and outdated information can introduce noise and bias into the model. For example, in a medical context, linked to breast cancer

diagnosis, inconsistent measurement techniques or human errors in labelling malignant versus benign cases could confuse the model.

- **Feature selection and engineering:** Even the best models will struggle if they are trained on irrelevant and poorly constructed features. Redundant, highly correlated, or noisy features can mislead the learning process of the model, while missing key variables can prevent the model from capturing important patterns. The solution to this is to use domain knowledge and statistical techniques to select meaningful features and create new ones that better represent the relationship underlying the data. Scaling, categorical encoding, and effective handling of outliers can also enhance performance.
- **Data volume and representativeness:** A small or unrepresentative dataset limits a model's ability to generalize. If the training data does not capture the full diversity of real-world cases, the model may perform well in testing but fail in production. For example, if certain age groups or tumor types are underrepresented, then the predictions could be biased to only the cases the model has been trained on. **Solution:** gather more data where possible and ensure it reflects real and wider distributions. Make sure datasets are regularly updated and models retrained, where the data is available.
- **Class imbalance:** In classification tasks, especially in the real world, datasets are often imbalanced (e.g., most of the population does not have cancer, so the sample that does have it will be imbalanced). This causes the models to be biased towards the majority class (benign), showing higher accuracy; thus, the importance of using more metrics than accuracy to evaluate models, such as F1 score, precision, and recall. We have looked at how to overcome imbalance in the classification chapters of this book, as possible solutions.

- **External factors and context:** Data in the real world is messy, due to its influence from external factors, like changes in protocols, equipment, population demographics, or even seasonal trends. A model trained on historical data might underperform if these conditions shift (known as feature drift and model drift). The way to overcome this is by consistent **Machine Learning Operations (MLOps)** processes that monitor model metrics and give an indication of when the model should be retrained.

By focusing on these foundational aspects, you create a stronger base for any machine learning model to learn from. This often leads to greater improvements than simply switching algorithms or fine-tuning hyperparameters. In fact, overcoming these challenges can transform a *good* model into a truly robust, reliable, and production-ready solution, especially in sensitive fields like healthcare, where accuracy and generalization are critical.

Conclusion

In this chapter, we explored key techniques to enhance machine learning performance, including hyperparameter tuning with grid search, random search, and Bayesian optimization. You also learned how feature elimination, dimensionality reduction with LDA, and chi-square selection can simplify models, reduce overfitting, and improve predictive power.

We further uncovered the strengths of ensemble learning through bagging, boosting, and stacking, showing how multiple models can work together for better accuracy and generalization. Alongside these technical methods, we emphasized the importance of data quality, class balance, and feature engineering, crucial factors for robust ML models that work in production.

In the next chapter, we take a significant step forward by leaving the comfort of Jupyter Notebooks to build structured, production-like workflows using scikit-learn Pipelines. You will learn how to design a complete pipeline, organizing it into clear stages: data loading, preprocessing, hyperparameter tuning, model training, and inference.

Through a practical customer churn prediction example, we will show how to automate and streamline these steps, replicating a real-world ML setup that is efficient, reusable, and ready for deployment.

Points to remember

Here are some key points summed up from this chapter:

- **Balance search space and computation:** While grid search is exhaustive, random search and Bayesian optimization offer smarter, more efficient alternatives when dealing with large hyperparameter spaces.
- **Data quality is as important as the algorithm:** Poor data, such as missing values, inconsistent labels, or outdated records, can undermine even the most advanced models
- **Dimensionality reduction with LDA:** Unlike PCA, LDA is supervised and focuses on maximizing class separability, making it ideal for classification problems with high-dimensional data.
- **Ensemble learning amplifies performance:** Bagging reduces variance, boosting reduces bias, and stacking combines diverse models for robust, accurate predictions.
- **Feature engineering:** Thoughtful creation, selection, and transformation of features often lead to greater performance gains than model tweaks alone.
- **Feature elimination reduces overfitting:** Methods like RFECV help remove irrelevant or redundant features, simplifying models and improving generalization.
- **F1 score over accuracy:** Especially in imbalanced datasets, metrics like F1 score, precision, and recall provide a clearer picture of model effectiveness than accuracy alone.
- **Hyperparameter tuning is essential:** Techniques like grid search, random search, and Bayesian optimization can significantly boost

model performance by finding the optimal configuration for learning algorithms.

- **Hyperparameter tuning with feature selection makes a powerful combo:** Combining tuning with dimensionality reduction or feature elimination often yields better results than applying them in isolation.
- **Monitor for drift:** In production, watch for feature drift and model drift; changes in data patterns over time that can degrade model performance.

Multiple choice questions

1. **What is the primary goal of hyperparameter tuning in machine learning?**
 - a. To simplify the model by reducing unnecessary features
 - b. To enhance model performance by optimizing algorithm settings
 - c. To prevent overfitting by reducing model complexity
 - d. To improve training efficiency by increasing dataset size
2. **How does random search improve hyperparameter tuning efficiency?**
 - a. By sampling random combinations within defined ranges
 - b. By testing every possible combination in a predefined grid
 - c. By selecting hyperparameters based on previous trial outcomes
 - d. By limiting the search to default parameter values
3. **What distinguishes LDA from PCA in dimensionality reduction tasks?**
 - a. LDA maximizes variance without considering class labels

- b. PCA is better suited for classification problems due to supervision
 - c. LDA uses class information to maximize class separability
 - d. PCA reduces features based on their correlation with the target variable
4. **What is the key advantage of bagging in ensemble learning?**
- a. It combines different algorithms to capture complex patterns
 - b. It reduces bias by focusing on misclassified examples
 - c. It sequentially builds models to improve weak learners
 - d. It reduces variance by averaging predictions from diverse subsets
5. **Why is feature elimination important in machine learning?**
- a. It increases model flexibility by adding interaction terms
 - b. It improves model performance by selecting features with the highest variance
 - c. It removes irrelevant data to reduce overfitting and complexity
 - d. It ensures all the original features contribute to the final model
6. **Why is stacking considered a powerful ensemble technique?**
- a. It reduces overfitting by pruning weak learners
 - b. It combines multiple model outputs using a meta-learner
 - c. It averages identical model trained on different subsets
 - d. It boosts performance by reweighting misclassified instances
7. **How does Bayesian optimization enhance hyperparameter tuning?**
- a. By predicting promising hyperparameters based on past trials
 - b. By randomly selecting hyperparameters to explore broader search spaces

- c. By exhaustively evaluating all parameter combinations
 - d. By restricting the search to predefined default values
8. **Which best describes the boosting technique in ensemble learning?**
- a. It averages independent models trained on random subsets
 - b. It combines multiple strong learners to simplify predictions
 - c. It reduces dimensionality while improving model accuracy
 - d. It sequentially corrects errors made by previous models
9. **What is model drift in a machine learning production environment?**
- a. A gradual increase in model complexity over time
 - b. A situation where hyperparameters no longer align with the training data
 - c. A mismatch between training and validation distributions
 - d. A decline in model performance due to changes in data patterns in the real world
10. **What is the main purpose of using RFECV?**
- a. To identify and retain the most important features while avoiding overfitting
 - b. To balance class distributions by removing redundant samples
 - c. To automatically generate new features from existing ones
 - d. To increase model complexity by adding interaction terms

Answers

1	b
2	a
3	c

4	d
5	c
6	b
7	a
8	d
9	d
10	a

Key terms

Some key terms used in this chapter are:

- **Bayesian optimization:** Probabilistically guides hyperparameter tuning using past results to efficiently find optimal settings.
- **Bagging:** Trains multiple models on random data subsets and combines their outputs to reduce variance.
- **Boosting:** Sequentially trains models, each correcting the previous, to reduce bias and improve prediction.
- **Chi-square test (χ^2):** Assesses the relationship between categorical features and the target for feature selection.
- **Cross-validation:** Evaluates model performance by splitting data into multiple train-test sets.
- **Dimensionality reduction:** Reduces input features while retaining key information (e.g., PCA, LDA).
- **Ensemble learning:** Combines multiple models' predictions for better accuracy and robustness.
- **Feature drift:** Changes in feature distributions over time that can harm model performance.
- **Feature elimination:** Removes irrelevant or redundant features to improve models.

- **F1 score:** Balances precision and recall, useful for imbalanced classification.
- **Grid search:** Systematically tests all parameter combinations to find the best model.
- **Hyperparameter tuning:** Optimizes model settings not learned from the data to boost performance.
- **LDA:** Projects data to maximize class separation for classification.
- **Model drift:** Model performance declines over time due to changes in data patterns.
- **Optuna:** Automated framework for efficient hyperparameter optimization.
- **Overfitting:** Model fits training data too closely, performing poorly on new data.
- **Random search:** Samples random hyperparameter combinations for tuning, more scalable than grid search.
- **Regularization:** Penalizes model complexity to prevent overfitting (e.g., L1, L2).
- **RFECV:** Recursively removes features, using cross-validation to find the optimal set.
- **Stacking:** Combines different models by feeding their predictions into a meta-model.
- **Underfitting:** Model is too simple, failing to capture data patterns.

Join our Discord space

Join our Discord workspace for latest updates, offers, tech happenings around the world, new releases, and sessions with the

authors:

<https://discord.bpbonline.com>



CHAPTER 10

Building Multi-step Pipelines

Introduction

In this chapter, we will follow a complete, production-grade approach to managing machine learning workflows using scikit-learn's pipelines and transformers. Building on your experience with tabular data and forecasting tasks, we will show you how to structure preprocessing, modeling, and evaluation steps into modular, reusable components. You will learn how to apply different transformations to numerical and categorical data, handle missing values intelligently, and even create your own custom transformers for specialized tasks like feature binning, all within a unified and reproducible framework.

We will explore how to integrate these preprocessing steps into complete model pipelines, including advanced setups like ensemble methods and class balancing strategies. By combining tools like Pipeline, ColumnTransformer, and FeatureUnion, you will be able to orchestrate complex machine learning workflows that are clean, consistent, and ready for deployment. This chapter does not just teach syntax; it demonstrates how to think in pipelines, helping you transition from ad hoc modeling scripts to scalable, maintainable systems that align with real-world engineering best practices.

Structure

This chapter covers the following topics:

- Introduction to scikit-learn pipelines and transformers

- Preprocessing steps
- Creating a union with our features
- Combining multiple models and transformers into a pipeline
- Using pipelines with hyperparameter tuning
- Boilerplate code templates for your projects

Objectives

By the end of this chapter, you will be able to build complete machine learning workflows that combine data preprocessing, feature engineering, and model training into streamlined, reproducible systems. You will learn how to apply transformations to different types of data in parallel, implement custom logic for feature creation, and integrate steps like class balancing and model ensembling, all within a unified training framework. You will also gain experience saving and reusing your preprocessing logic, organizing your code for maintainability, and configuring model tuning through external parameter files. To help you apply these concepts beyond the page, we will also provide boilerplate code and a modular project structure that you can adapt and use in the wild, whether you are prototyping new models or deploying them in production workflows. All examples throughout will be aimed at our customer churn prediction use case.

Introduction to scikit-learn pipelines and transformers

When working on machine learning projects, one of the biggest challenges is managing the series of data preprocessing, transformation, and modeling steps in a way that is both efficient and reproducible. This is where scikit-learn's pipelines and transformers come into their own. Scikit-learn is a powerful library for machine learning, providing streamlined tools that help simplify and standardize the workflow from raw data to a trained model.

Pipelines allow users to chain together multiple preprocessing steps into a single object. This not only improves code clarity, ensuring the same

sequence of transformations is consistently applied during both training and prediction steps of the pipeline. Additionally, it eliminates redundancy and minimizes the risk of errors such as data leakage or inconsistent processing.

At the very heart of these pipelines are **transformers**, which are objects that implement a fit and transform method. Transformers handle tasks such as scaling, encoding, imputing missing values, or even generating new features. Scikit-learn also provides a consistent API to create custom transformers, allowing users to tailor preprocessing to their specific needs.

In this chapter, we will explore how pipelines and transformers work together in scikit-learn to create clean, robust, and maintainable machine learning workflows. By the end, you will understand how to build your own pipeline, separate it out as a machine learning workflow, and integrate it into real-world machine learning tasks with ease and confidence.

Preprocessing steps

When working with machine learning models, preprocessing is an essential step to ensure that the input data is clean, consistent, and well-prepared for training. Throughout this book, we have been gradually introducing various preprocessing techniques. In this section, we will apply these techniques more systematically using pipelines, specifically in the context of our customer churn dataset from *Chapter 4, Supervised Multi-class Classification*.

Using pipelines, we will replicate and streamline the preprocessing steps we previously applied, which include handling missing values, encoding categorical features with **OneHotEncoder**, and scaling numerical features with appropriate scaling strategies. Pipelines not only allow us to automate this workflow but also make our process modular, reproducible, and easier to maintain, setting a solid foundation for the model training phase covered in the upcoming sections.

Loading the customer churn dataset

In this section, we will load the customer churn dataset that we previously created and use it as the foundation for building our pipeline strategies. The first steps will involve importing the data, encoding the integer-based classification labels into categorical format, and defining functions to clearly separate categorical features from numerical ones. This separation will allow us to apply tailored preprocessing techniques to each type of data within our pipeline.

Please note that several necessary imports are assumed to be in place and will not be explicitly shown in this section. You can refer to the accompanying GitHub workbook for this section for the full context. At this point, we also assume you are comfortable with the foundational concepts and syntax introduced earlier in the book.

To load and process our data, ready for preprocessing, we will follow these steps:

1. Load in the data, located in the associated GitHub projects data folder:

```
import pandas as pd
```

```
cust_df = pd.read_csv("data/synth_customer_churn.csv")
```

2. In this step, we will define a dictionary to map the categorical labels (High, Medium, and Low Risk) to their corresponding integer values for use in model training as the target variable, the class to predict:

```
y = cust_df['ChurnCategory'].map({'Low Risk': 0,  
                                  'Medium Risk': 1,  
                                  'High Risk': 2})
```

3. We will construct the feature set by dropping the target column and removing the customer ID field, as this will not be useful for this model's training:

```
X = cust_df.drop(columns=['ChurnCategory'],
```

```
'CustomerID']])
```

In the next section, we will create our numerical pipeline to process this data.

Creating the numerical pipeline

To prepare our numerical features effectively for modeling, we will build dedicated preprocessing pipelines that apply different scaling and imputation strategies tailored to the characteristics of each column. Rather than applying a one-size-fits-all transformation, we segment the features based on their distribution and sensitivity to outliers, ensuring that each is treated in the most appropriate way. This targeted approach results in a more robust and reliable pipeline.

We will go through the following steps to create this set of numerical pipelines:

1. Create lists of features that will be used by the various pipeline methods. This will be a combination of scaling and imputation:

```
minmax_median_cols = ['Age']
```

```
standard_mean_cols = ['MonthlyCharges']
```

```
standard_median_cols = ['CustomerSupportCalls']
```

```
robust_median_cols = ['ServiceUsage']
```

2. We will begin by creating the first pipeline, called **minmax_median_pipeline**, which applies **MinMax** scaling after using median imputation to handle any missing values:

```
minmax_median_pipeline = Pipeline([  
    ('imputer', SimpleImputer(strategy='median')),  
    ('scaler', MinMaxScaler())  
])
```

3. The next pipeline will be a standard pipeline, with standard scaling and mean imputation:

```
standard_mean_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())
])
```

4. Following this step, we will use median imputation as the missing value replacement strategy, holding everything else constant from *Step 3*:

```
standard_median_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])
```

5. This pipeline will be our robust pipeline, robust in the fact that it can scale outliers very effectively:

```
robust_median_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', RobustScaler())
])
```

This builds the foundation for using our pipeline downstream, but we will not use these quite yet, as they will be the discussion point in later sections.

Creating the categorical pipeline

Now that we have set up our numerical preprocessing pipelines, it is time to turn our attention to the categorical features in our dataset. Categorical data often requires different handling techniques compared to numerical data, especially when preparing it for use in machine learning models. In this section, we will create categorical pipelines that will take care of missing value imputation and apply encoding strategies, specifically, **OneHotEncoder**, to transform our categorical variables into a format that can be interpreted by most algorithms. This structured approach allows us

to process categorical features efficiently and integrate them seamlessly into the overall modeling pipeline.

In the following steps, we will define our target columns and create our pipeline strategy for handling categorical features:

1. As we did in the numerical pipeline section, we will define our target features (as a list) to specify which columns should be used for this process:

```
categorical_cols = ['Gender', 'ContractType', 'PaymentMethod']
```

2. The following pipeline fills the missing categorical values, using the most frequent category, and then applies one-hot encoding (which we have previously learned), dropping the first level to reduce multicollinearity, an issue particularly relevant for linear models:

```
categorical_pipeline = Pipeline([  
    ('imputer', SimpleImputer(strategy='most_frequent')),  
    ('encoder', OneHotEncoder(drop='first',  
                               handle_unknown='ignore'))])
```

Defining this pipeline sets up a transformation, where each categorical feature will be encoded into multiple binary columns, one for each category, with a value of 1 indicating presence and 0 indicating otherwise. One category per feature will be dropped to avoid redundancy, preparing the data for models that are sensitive to multicollinearity. In the next section, we are going to convert a numerical value to a categorical value using a concept called binning.

Creating a custom transformer

To address the issue of extreme outliers and a skewed distribution in the Tenure feature in our data, we will create a custom transformer using **TransformerMixin** from **sklearn.base**. This allows us to define a reusable and pipeline-compatible preprocessing step that bins numerical values into

discrete intervals. By converting a wide-ranging continuous variable like Tenure into a set of categorical bins, we simplify the feature, reduce the influence of outliers, and make it more suitable for models that assume categorical or evenly distributed inputs.

The custom transformer will define specific bin edges, either manually chosen or calculated, and assign each Tenure value to a bin using methods such as **pd.cut()**. Once transformed, this feature can be treated like a categorical variable and encoded accordingly. This approach is especially useful when a feature's raw numerical values add little predictive value on their own due to scale or distribution issues, but the grouped categories hold meaningful patterns.

Before we create this function, we will analyze what the distribution looks like as a histogram, by using the following matplotlib code:

```
1. import matplotlib.pyplot as plt
2. cust_df['Tenure'].hist(bins=30,
3.                        edgecolor='black',
4.                        color='grey')
5. plt.title("Distribution of Tenure")
6. plt.xlabel("Tenure (months)")
7. plt.ylabel("Frequency")
8. plt.grid(False)
9. plt.show()
```

Running these lines in Python, we get the visual, as shown in *Figure 10.1*:

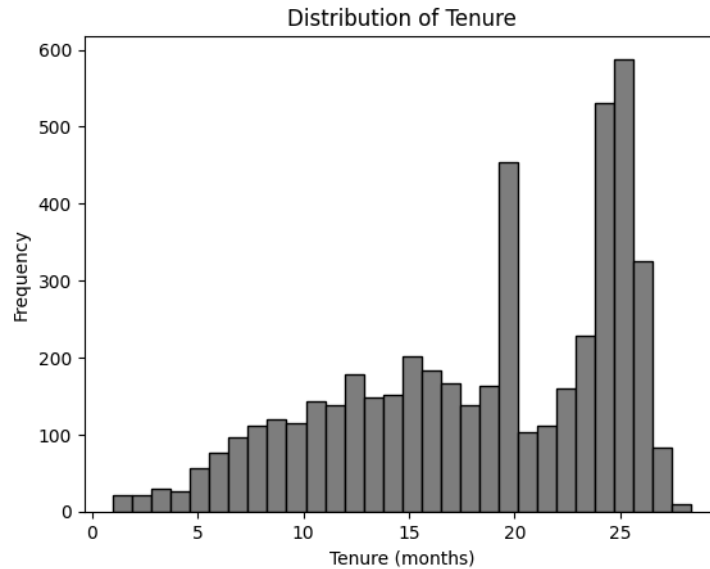


Figure 10.1: Histogram of Tenure

This distribution is heavily skewed to the right, meaning statistically it will be an ideal candidate distribution for the creation of bins, or binning the distribution. This is where our custom Transformer will come in, and this will be detailed hereunder and explained:

```

1. from sklearn.base import BaseEstimator, TransformerMixin
2. class TenureBinner(BaseEstimator, TransformerMixin):
3.     def __init__(self, column='Tenure', bins=None, labels=None):
4.         self.column = column
5.         self.bins = bins if bins is not None else [0, 6,
6.             15, 22,
7.             float('inf')]
8.
9.         self.labels = labels if labels is not None else [ 'New',
10.             'Developing',
11.             'Established',
12.             'Loyal']
13.

```

```

14.     def fit(self, X, y=None):
15.         return self
16.
17.     def transform(self, X):
18.         X_ = pd.DataFrame(X).copy()
19.         if self.column not in X_.columns:
20.             raise ValueError(f"Column '{self.column}' not found in input DataFrame.")
21.
22.         if X_[self.column].isnull().any():
23.             X_[self.column] = X_[self.column].fillna(
24.
25.                 X_[self.column].median())
26.
27.         binned = pd.cut(X_[self.column], bins=self.bins,
28.                         labels=self.labels,
29.                         include_lowest=True)
30.         return pd.DataFrame({f"{self.column}_bin": binned})
31.
32.     def get_feature_names_out(self, input_features=None):
33.         return [f"{self.column}_bin"]

```

An explanation of what this class does is detailed in the following steps:

1. We import the **BaseEstimator**, which provides basic functionality like getting and setting the hyperparameters of the transformer. The **TransformerMixin** import provides a default implementation of the **.fit_transform()** method (this automatically calls the classes **fit** method and **transform** in one step, so you do not have to create the **fit_transform** method yourself).

2. The `__init__` initializes the class, and this is where you put your class initialization variables and attributes. In our class, we define the target column, set a binning strategy (this expects a list of values to be passed), and the associated labels to be passed.
3. The class method `fit` does not do anything; it leaves this untouched, and we return the self-parameter (**self** is how a class keeps track of its own data. It returns a specific object being created or used, so each object can store and use its own values. Put simply, it is like saying 'me' inside the object, and it is how the object knows you are talking about its own variables and methods).
4. The transform class method is the real workhorse here; this copies the DataFrame, checks if the target column exists, fills missing values in the column with a median replacement strategy, applies the `pd.cut()` method to bin values using the defined edges, and labels and returns a new DataFrame with the binned columns.
5. The `get_feature_names_out` class method returns the names of the new binned features.

Now that we understand what the custom transformer does, we can go ahead and use it to create our binned columns:

```
1. tenure_binner_pipeline = Pipeline([
2.     ('tenure_binner', TenureBinner(
3.         column='Tenure',
4.         bins=[0, 6, 15, 22, float('inf')],
5.         labels=['New', 'Developing',
6.             'Established', 'Loyal']
7.     )),
8.     ('encoder', OneHotEncoder()))]
```

This pipeline creates an instance of the custom **TenureBinner** class to convert the Tenure column into categorical bins based on defined intervals

and labels. It then passes the binned output to **OneHotEncoder**, which transforms the categories into binary columns. This allows a continuous, skewed feature to be handled as a clean, model-ready categorical variable.

In the next section, we will combine the pipelines using a transformer to combine the transformers.

Using ColumnTransformer to combine pipelines

ColumnTransformer is a powerful tool that allows users to apply different preprocessing steps to different columns of a dataset within a single pipeline. This is particularly useful when working with heterogeneous data types, such as numeric, categorical, and text features, that require distinct transformations. For instance, numerical features may need scaling, while categorical features may require encoding. By using **ColumnTransformer**, these operations can be cleanly and efficiently handled in parallel, promoting modularity and reducing the risk of data leakage. The transformer also integrates seamlessly with the scikit-learn ecosystem, enabling a robust workflow design and creation. This will be achieved with the following code:

```
1. structured_preprocessor = ColumnTransformer(transformers=[
2.     ('minmax_median', minmax_median_pipeline, minmax_median_
3.     cols),
4.     ('standard_mean', standard_mean_pipeline, standard_mean_col
5.     s),
6.     ('standard_median', standard_median_pipeline, standard_median_
7.     cols),
8.     ('robust_median', robust_median_pipeline, robust_median_cols),
9.     ('categorical', categorical_pipeline, categorical_cols)])
```

This will create a pipeline **directed acyclic graph (DAG)** as in *Figure 10.2*:

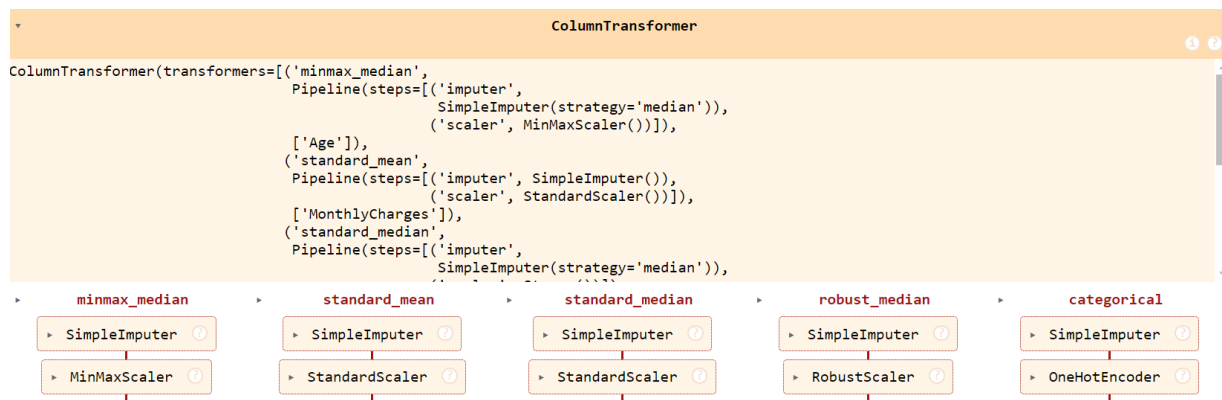


Figure 10.2: Structured pipeline DAG

These are particularly helpful for visualizing the sequence of transformation steps within each pipeline. The pipelines can be arranged horizontally or vertically, depending on the structure of your preprocessing workflow. In the next section, we will create a **FeatureUnion** between our steps.

Creating a union with our features

In scikit-learn, **FeatureUnion** is a powerful utility that enables the parallel combination of multiple feature extraction or transformation pipelines. Unlike the Pipeline object, which applies a sequence of transformations in a linear (sequential) order, **FeatureUnion** applies several independent transformations to the same input data simultaneously and then concatenates their outputs. This is especially useful when different types of features or representations need to be extracted from the same dataset. For example, one branch might scale numerical features while another extracts polynomial features or encodes categorical variables. Under the hood, **FeatureUnion** fits each transformer separately using the full input, then horizontally stacks the resulting feature matrices. This allows for more flexible and modular preprocessing architectures, particularly in cases where diverse data representations improve model performance. To create our union of features, we can import the feature union tool and use it on our pipeline, as follows:

1. Import **FeatureUnion** from the scikit-learn pipelines:

```
from sklearn.pipeline import FeatureUnion
```

2. We then use the union method to unify our structured preprocessor and our custom **Tenure** binner pipeline:

```
full_preprocessor = FeatureUnion([  
    ('structured', structured_preprocessor),  
    ('tenure_bins', tenure_binner_pipeline)])
```

3. Finally, we will print out the pipeline, to create a visual of the pipeline, as in *Figure 10.3*:

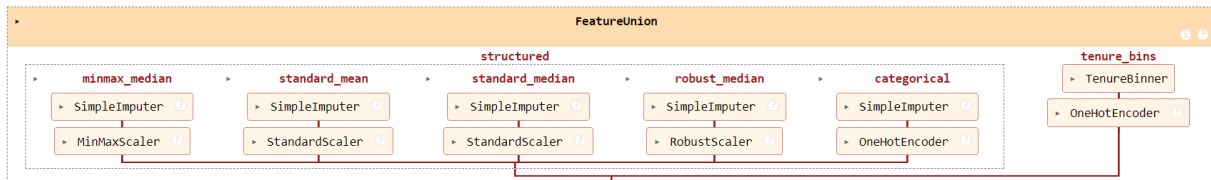


Figure 10.3: Feature union DAG

We have all our preprocessors in place at this point. In the next section, we will look at saving the processor pipeline for downstream use.

Saving our preprocessor

Now that we have built and combined our preprocessing components, including the structured data preprocessor and the custom Tenure binning pipeline, it is a good time to persist these objects to disk using joblib (<https://joblib.readthedocs.io/en/stable/>).

By saving these components, we make our machine learning pipeline more modular and maintainable. Separating preprocessing, tuning, and modeling steps allows for:

- **Reusability:** We can reuse the same preprocessing logic across experiments or projects without redefining it.
- **Consistency:** Ensures the exact same transformations are applied during both training and inference.
- **Efficiency:** Skip recomputing transformations or training custom components every time.

You may ask why we save preprocessors separately. Here is the rationale:

- **Hyperparameter tuning:** You might want to freeze preprocessing while exploring model parameters.
- **Model retraining:** Keeping preprocessing fixed ensures consistency across retraining sessions.
- **Deployment pipelines:** It becomes easier to deploy preprocessing as a separate microservice or embed it into an inference pipeline. We will create a preprocessor, tuner, and trainer Python file at the end of this chapter, so you can use these in your scikit-learn projects, as boilerplate examples.

To save the processor, we use the following steps:

1. Import the **os** package (no need to pip install, as this is shipped with Python), name a variable to store the path of the preprocessing dictionary, and use **os.makedirs** to create a new target directory for saving our **joblib** file in:

```
import os
```

```
PREPROC_DIR = 'models/preprocessing'
```

```
os.makedirs(PREPROC_DIR, exist_ok=True)
```

2. In the next step, we import **joblib** and use the dump command; to save the file to our **PREPROC_DIR** we use the following:

```
import joblib
```

```
joblib.dump(full_preprocessor,
```

```
os.path.join(PREPROC_DIR, 'preprocessor.joblib'))
```

3. Upon running this, you will see a print to the console stating that this has been saved into the desired path.

In the next section, we will explore strategies for combining models to improve performance during training. In later chapters, we will use these combined models to build prediction endpoints for deployment.

Combining multiple models and transformers into a pipeline

In machine learning, it is common to apply a series of preprocessing steps, such as scaling, encoding, or imputing, before fitting a model. Managing these steps separately can quickly become error-prone and difficult to maintain, especially when working with cross-validation or deploying models into production.

Pipelines offer a structured way to chain preprocessing and modeling steps into a single, unified workflow. By combining transformers (e.g., for feature scaling or encoding) and estimators (e.g., classifiers or regressors), pipelines ensure that each step is applied consistently and in the correct order. This not only improves code readability and modularity but also prevents common issues like data leakage during model evaluation.

In this section, we will explore how to build pipelines that seamlessly integrate preprocessing and modeling stages, making it easier to experiment, tune, and deploy machine learning models.

Loading our preprocessor

First things first, we need to load the processor for use with our modelling steps. To do this, we follow these steps:

1. Import the custom **Binner** class saved in the previous notebook.
2. This is a user-defined class stored in our project's folder structure. To use it, we import it as follows:

```
from pipeline.binner import TenureBinner
```

3. To load the preprocessor, we use the load method in **joblib**:

```
preprocessor = joblib.load("models/preprocessing/preprocessor.joblib")
```

The preprocessor will now be back in our environment to use, and we will store it in the **preprocessor** variable.

Combine multiple models with a stacking ensemble

Building on the ensemble concepts introduced in *Chapter 9, Model Improvement Strategies*, we now explore how to implement them in practice using a stacking classifier. Stacking involves training multiple models in parallel and then using their predictions as inputs to a final model, known as a meta-model, which makes the overall prediction.

In this section, we will use this approach to build a stacking classifier and integrate it into a pipeline along with preprocessing steps. This setup allows us to treat the entire machine learning workflow, which includes data transformation, model combination, and prediction, as a single, reproducible process.

Following these steps, we will prepare our data ready for the modeling process and create our stacking classifier:

1. First, we must split our data into training and test sets. We used the following command for this:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2,
                                                    random_state=42)
```

2. We will use two tree-based methods (random forests and gradient boosting classifiers), as well as a support vector machine, to build our ensemble:

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import (RandomForestClassifier,
                              GradientBoostingClassifier)

base_learners = [
    ('rf', RandomForestClassifier()),
    ('svc', SVC(probability=True)),
```

```
('gb', GradientBoostingClassifier())]
```

3. Our meta learner will be a simple logistic regression model:

```
meta_model = LogisticRegression()
```

4. To build our stacking classifier, we will import the stacking classifier class and pass in our base learners and our chosen meta model:

```
from sklearn.ensemble import StackingClassifier
```

```
stacking_clf = StackingClassifier(
```

```
    estimators=base_learners,
```

```
    final_estimator=meta_model)
```

Now that we have this built, we need to rebalance our sample, as this is a multi-class problem; there is bound to be underrepresented classes in the data that are in need of rebalancing.

Using oversampling in our pipeline

Before building our pipeline, we will introduce an oversampling step to address class imbalance. In the customer churn use case, the number of customers who churn is significantly smaller than those who do not, resulting in a heavily imbalanced dataset. To ensure our models learn effectively from the minority class (high-risk customers), we will handle this imbalance using oversampling. We will follow the steps below to achieve this:

1. Get the imports required to conduct oversampling:

```
from imblearn.pipeline import Pipeline
```

```
from collections import Counter
```

```
from imblearn.combine import SMOTEENN
```

2. It is important to note that there is no native support for SMOTE in scikit-learn Pipelines. Therefore, we need to use **imbalanced learn**

pipelines to achieve this, which is why we have imported this in *Step 1*.

3. We will use a **Counter** object from Python's **collections** module to count how many times each class label appears in the dataset. This will give us the frequency of each distinct class:

```
original_counts = Counter(y_train)
```

4. In this step, we scale the original class counts by specific factors (1.4 for class 1 and 2.5 for class 2) to create the target class distribution, oversampled:

```
target_counts = {  
    1: int(original_counts[1] * 1.4),  
    2: int(original_counts[2] * 2.5)}
```

5. Finally, we apply SMOTENN to the dataset, to give us our oversamples:

```
smotenn = SMOTEENN(sampling_strategy=target_counts,  
                    random_state=42)
```

In the next section, we will build our pipeline and visualize the associated DAG.

Building and visualizing the pipeline

In this section, we will construct and visualize our machine learning pipeline, illustrating the preprocessing steps, class rebalancing process, base learner, and meta-learners involved. We will do these in the following steps:

1. Use the `imbalanced learn Pipeline` command to create the preprocessing, rebalancing, and ensemble pipeline:

```
stacking_pipeline = Pipeline([  
    ("preprocess", preprocessor),  
    ("resample", smotenn),
```



```
("stacking_clf", stacking_clf)])
```

- To visualize this pipeline, we just call the following:

```
stacking_pipeline
```

- This created the pipeline DAG, as shown in *Figure 10.4*:

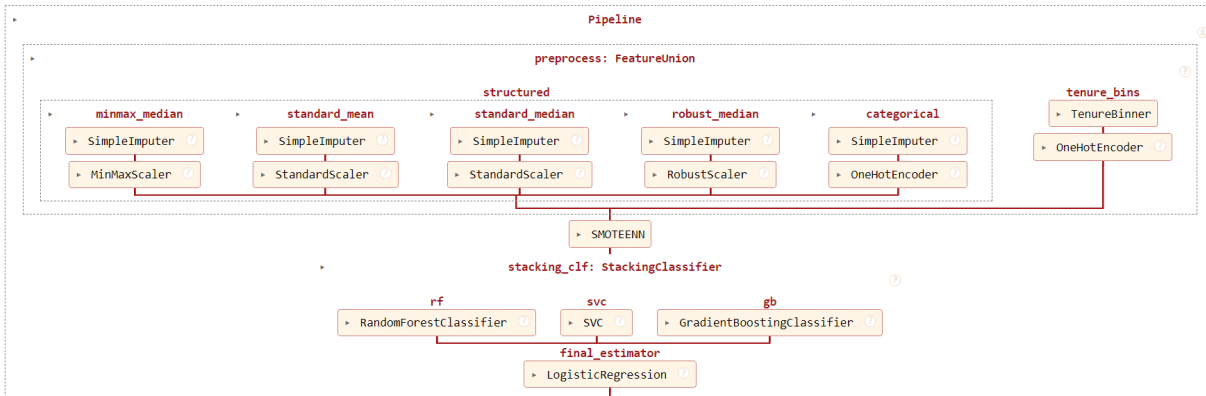


Figure 10.4: Ensemble pipeline DAG

There we have it, the final compiled ensemble pipeline.

- We will now save the pipeline in a folder, so we can use it later for hyperparameter tuning:

```
SAVE_DIR = "models/classifiers"
```

```
os.makedirs(SAVE_DIR, exist_ok=True)
```

```
joblib.dump(stacking_pipeline, os.path.join(SAVE_DIR, "stack_cla  
ss_pipe.joblib"))
```

Following these steps, we have everything we need in place to conduct hyperparameter tuning on our stacking classifier.

Creating a parameter grid and saving as JSON

To prepare for hyperparameter tuning, we will now define a parameter grid dictionary that specifies the range of values to be tested for selected model parameters. This dictionary will be used in a grid search procedure to systematically evaluate different combinations and identify the optimal configuration for our pipeline. Once the parameter grid is defined, we will

save it as a JSON file so it can be easily loaded and used during the hyperparameter tuning step. This approach ensures flexibility, reproducibility, and separation between pipeline construction and tuning. To do this, we will create the dictionary, as follows:

1. To create the grid, we pass the key value pairs as a dictionary:

```
param_grid = {  
    'stacking_clf__rf__n_estimators': [100, 200],  
    'stacking_clf__svc__C': [0.1, 1.0],  
    'stacking_clf__gb__learning_rate': [0.05, 0.1],  
    'stacking_clf__final_estimator__C': [0.1, 1.0]}
```

2. This includes the number of estimators for the random forest model, the C value (regularization strength) for the support vector classifier, and the learning rate for gradient boosting. These parameters ultimately influence the performance of the final estimator, which is a logistic regression model. Each parameter is prefixed with the name of the pipeline component, such as **stacking_clf**, as defined earlier.
3. We will then make a directory to store our hyperparameter configs in and dump the dictionary as a JSON compatible dictionary. The **json** (<https://docs.python.org/3/library/json.html>) package takes care of this for us:

```
import json  
  
os.makedirs("models/configs", exist_ok=True)  
  
with open("models/configs/stack_param_grid.json", "w") as f:  
    json.dump(param_grid, f, indent=2)
```

Now we have everything we need for hyperparameter tuning our model.

Create reusable base models in a pipeline

An alternative to using ensemble strategies like stacking or voting is to adopt a *pipeline-per-model* approach. In this method, a separate pipeline is created for each individual model, combining its specific preprocessing steps with the model itself. This modular setup allows each model to be trained, evaluated, and fine-tuned independently. It also simplifies debugging and experimentation, as components can be loaded and adjusted one at a time, offering greater flexibility and transparency in model development. To do this, we follow these steps, albeit this is a shorter process than the ensemble modeling method:

1. Specify our base models to use:

```
base_models = {  
    'rf': RandomForestClassifier(),  
    'svc': SVC(probability=True),  
    'gb': GradientBoostingClassifier()}
```

2. Apply SMOTENN again, to match the previous pipeline created:

```
from imblearn.pipeline import Pipeline  
sampler = SMOTENN(sampling_strategy=target_counts,  
                  random_state=42)
```

3. Initialize an empty dictionary to store our pipelines:

```
pipelines = {}
```

4. Create a loop that iterates over a dictionary of base models and creates a separate pipeline for each one, combining a shared preprocessing step, a resampling method, and the individual classifier, then stores each pipeline in the pipelines dictionary using the model's name as the key:

```
for name, model in base_models.items():
```

```
    pipelines[name] = Pipeline([  
        ("preprocess", preprocessor),
```

```
("resample", sampler),  
("classifier", model)])
```

5. We are going to want to store each of our pipelines in a directory to load back in later for hyperparameter tuning:

```
CLASS_PATH = "models/classifiers/"
```

```
os.makedirs(CLASS_PATH, exist_ok=True)
```

6. As a final step, we serialize and save each model pipeline to disk using **joblib**. This loop iterates through the dictionary of pipelines and writes each one to a **.joblib** file, naming the file according to the model's name. Storing pipelines in this way allows us to easily reload and reuse them later without needing to rebuild or retrain the models, ensuring efficiency and reproducibility in our workflow:

```
for name, pipe in pipelines.items():
```

```
    joblib.dump(pipe, f"{CLASS_PATH}/{name}_pipeline.joblib")
```

7. To confirm our files have been saved in the path we specified in *Step 5*, we use the list comprehension below to loop through this directory, and exclude our best model, from previous training:

```
files = [f for f in os.listdir(CLASS_PATH) if os.path.isfile(os.path.join(CLASS_PATH, f)) and not f.startswith("best_model")]
```

We have now built and configured our preprocessing steps as well as the pipelines for our models. With these foundational components in place, we are ready to move forward. In the next section, we will use the pipelines constructed here (along with the preprocessing strategies from the previous section) to perform hyperparameter tuning and evaluate model performance in a systematic way.

Using pipelines with hyperparameter tuning

With our preprocessing workflows and model pipelines fully defined, we are now equipped to put them to work. In this next section, we shift our

focus to model selection, using the pipelines we have built to systematically explore hyperparameter spaces, fine-tune model performance, and ultimately identify the best-performing models for our task. This is where the components we have constructed come together in a cohesive training and evaluation framework.

Tuning the stacked ensemble model

The first task in our model selection process is to tune our ensemble model. To ensure consistency and reproducibility, we will be using a predefined set of hyperparameters stored in a JSON file. This approach allows us to cleanly separate configuration from code, making experiments easier to manage and modify. We will load the parameters from the file and apply them to our ensemble pipeline to begin the tuning and evaluation process. This will be achieved in the following series of steps:

1. The first step is to open the hyperparameter tuning file we created in the *Creating a parameter grid and saving as JSON* section:

```
import json  
  
with open("models/configs/stack_param_grid.json", "r") as f:  
    config = json.load(f)
```

2. Convert our JSON dictionary back to a dictionary that is recognised by Python, using the **dict()** method:

```
param_grid = dict(config)
```

3. Loading the stacking ensemble pipeline we created:

```
stacking_pipeline =  
joblib.load("models/classifiers/stack_class_pipe.joblib")
```

4. This step sets up a grid search using **GridSearchCV** to perform hyperparameter tuning on the **stacking_pipeline**. It systematically tests combinations of parameters defined in **param_grid** using 5-fold cross-validation. Each combination is evaluated based on its **F1 score**, providing a balanced measure of precision and recall.

The process runs in parallel across all available CPU cores (**n_jobs=-1**) to speed up computation, and with **verbose=2**, it provides detailed progress updates. This allows us to identify the optimal hyperparameter configuration for our ensemble model in a robust and efficient manner:

```
from sklearn.model_selection import GridSearchCV  
  
grid = GridSearchCV(stacking_pipeline, param_grid,  
                    cv=5, scoring='f1',  
                    n_jobs=-1, verbose=2)
```

5. We then fit the grid search to run the training process:

```
grid.fit(X_train, y_train)
```

6. Accessing the **best_estimator_**, we can obtain the best model from the grid search, over five folds:

```
best_model = grid.best_estimator_
```

7. To obtain the best score (F1 in this instance, as this was passed to the scoring parameter), we use the following:

```
best_model.score(X_test, y_test)
```

8. Running this line shows the ensemble obtains a **F1 score** of **0.943**; we will validate this with a confusion matrix, after we have conducted the same process with the single model per pipeline approach.

9. Now that the model is fitted, we will save this model in our classifier's directory:

```
os.makedirs("models/fitted_models", exist_ok=True)  
  
joblib.dump(best_model, "models/fitted_models/stack_class_best_model.joblib")
```

In the next section, we will use random search to tune the model per the pipeline approach we implemented.

Tuning the model per pipeline strategy

With our preprocessing and modeling pipelines ready, the next step is to fine-tune each model using a randomized search. This approach allows us to efficiently explore a wide hyperparameter space by sampling a fixed number of random combinations, rather than exhaustively testing all possible options. It offers a practical balance between performance and computational cost, especially when tuning more complex models. As always, we will break this task into steps, to help us understand the process:

1. We need a way to find the pipelines we created, so we will use a list comprehension to find the models of interest:

```
CLASS_PATH = "models/classifiers/"
```

```
files = [f for f in os.listdir(CLASS_PATH)
         if f.endswith("_pipeline.joblib")
         and not f.startswith("best_")]
```

2. This step loads serialized pipeline objects from files and stores them in a dictionary called pipelines, using the prefix of each filename (before the underscore) as the key:

```
pipelines = {}
```

```
for file in files:
```

```
    name = file.split("_")[0]
```

```
    pipelines[name] = joblib.load(os.path.join(CLASS_PATH, file))
```

3. This step sets up the hyperparameter distributions for several classifiers to be used in a randomized search during model tuning. Using **randint** and **uniform** from **scipy.stats**, it defines how values should be sampled for each parameter. The **param_dists** dictionary maps each model, random forest ('rf'), support vector classifier ('svc'), and gradient boosting ('gb'), to a set of parameters and their respective search spaces. For the random forest model, it randomly selects the number of trees (**n_estimators**) between 100 and 1000

and tests a fixed set of values for **max_depth**. The support vector classifier's regularization parameter *C* is sampled from a continuous uniform distribution between 0.01 and 10, while gamma is chosen from two standard options: **scale** and **auto**. For the gradient boosting model, the number of estimators and learning rate are also sampled from specified integer and uniform ranges, respectively. This setup enables a more efficient exploration of the hyperparameter space by randomly sampling a diverse set of configurations, rather than exhaustively testing every possible combination as in grid search:

```
from scipy.stats import randint, uniform

param_dists = {
    'rf': {
        'classifier__n_estimators': randint(100, 1000),
        'classifier__max_depth': [None, 5, 10, 20, 30]
    },
    'svc': {
        'classifier__C': uniform(0.01, 10),
        'classifier__gamma': ['scale', 'auto']
    },
    'gb': {
        'classifier__n_estimators': randint(100, 1000),
        'classifier__learning_rate': uniform(0.01, 0.3)
    }
}
```

4. This step performs hyperparameter tuning using **RandomizedSearchCV** for each model pipeline. It loops through the loaded pipelines, applies randomized search with model-specific parameter distributions, evaluates each with 5-fold cross-validation using the macro F1 score, and stores the best results in

the **best_models** dictionary. It also prints the best parameters and scores for each model:

```
from sklearn.model_selection import RandomizedSearchCV

best_models = {}

for name, pipeline in pipelines.items():

    print(f"Tuning {name} with RandomizedSearchCV...")

    search = RandomizedSearchCV(

        estimator=pipeline,

        param_distributions=param_dists[name],

        n_iter=25,

        cv=5,

        scoring='f1_macro',

        n_jobs=-1,

        random_state=42)

    search.fit(X_train, y_train)

    best_models[name] = search

    print(f"Best params for {name}: {search.best_params_}")

    print(f"Best score: {search.best_score_}")
```

This step will take about 20 minutes to run.

5. The code selects the best-performing model from the **best_models** dictionary by comparing their cross-validated F1 scores. Specifically, it uses Python's **max()** function to find the entry with the **highest .best_score_** (the best F1 score found during tuning). The key argument is a lambda function that extracts the score from each **RandomizedSearchCV** result. The result, **best_overall**, is a tuple containing the model name and its corresponding **RandomizedSearchCV** object:

```
best_overall = max(best_models.items(),  
                    key=lambda x: x[1].best_score_)
```

6. In the next line, we create a print statement, with an **f_string**, to print out the best model:

```
print(f"Best overall model: {best_overall[0]} with score {best_overall[1].best_score_}")
```

7. This outputs the result of the best model: **Best overall model: svc with score 0.68.**
8. Finally, we will save the best model from that process:

```
joblib.dump(best_overall[1].best_estimator_,  
            f"models/fitted_models/best_model_random_search.joblib")
```

It is clear that relying on a single model is insufficient for tackling this challenging multi-class classification problem. While the F1 score provides a useful measure, the performance observed so far can be considered only moderate and leaves room for improvement. This highlights the limitations of individual models in capturing the full complexity of the task. As a result, the next section will shift focus to evaluating a stacked ensemble classifier, a model that leverages the strengths of multiple base learners. We will assess its performance using a broader set of metrics beyond just the F1 score, providing a more comprehensive view of its effectiveness across different aspects of the prediction task.

Evaluating the stacked classifier

With hyperparameter tuning complete and our stacked ensemble model trained, we now turn our attention to evaluation. This stage is crucial for understanding how well the model performs beyond a single metric and across all target classes. In the following section, we will assess the ensemble's performance using a range of evaluation metrics and visual tools, allowing us to gain deeper insights into its strengths, weaknesses, and

overall suitability for the task at hand. Let us walk through the steps to create the evaluation visualization:

1. Fit the model on the best classifier again, to make sure the model has been fitted and the weights updated for prediction:

```
fitted_final_model = grid.best_estimator_.fit(X_train, y_train)
```

2. Use the fitted model to make predictions on the test set:

```
preds = fitted_final_model.predict(X_test)
```

3. Create the confusion matrix object and map the associated class labels to be displayed on the visualization:

```
from sklearn.metrics import confusion_matrix
```

```
classes = ['Low Risk', 'Medium Risk', 'High Risk']
```

```
cm = confusion_matrix(y_test, preds)
```

4. Use the **modelviz** package (to install this package use **pip install modelviz** to get the latest version) to create the confusion matrix:

```
from modelviz.confusion_matrix import plot_confusion_matrix
```

```
plot_confusion_matrix(
```

```
    cm=cm,
```

```
    classes=classes,
```

```
    model_name="Stacked Classifier",
```

```
    table_fontsize=8)
```

Initiating *Step 4* will execute the package's functionality to generate the confusion matrix visualization, as illustrated in *Figure 10.5*:

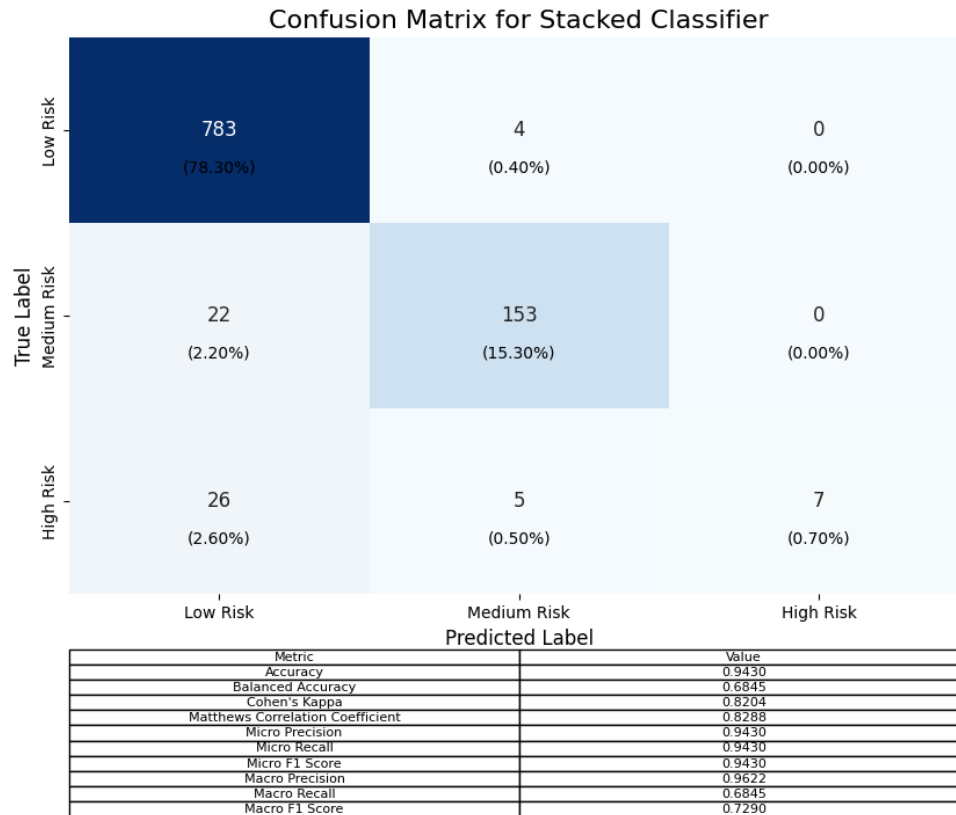


Figure 10.5: Stacked classifier confusion matrix

This confusion matrix and accompanying metrics provide a comprehensive view of how the stacked classifier performs across the three risk categories: low, medium, and high. The model shows strong performance for the low risk class, correctly identifying 783 instances (78.3%) with minimal misclassifications. It also performs reasonably well for medium risk, correctly classifying 153 cases (15.3%), though 22 were mistakenly labeled as low risk, indicating some confusion between adjacent risk levels. However, while the model struggles with accurately identifying the high risk class, correctly classifying only seven instances (0.7%), its performance across the low and medium risk categories is notably strong. This makes the model a potentially valuable tool in scenarios where the primary objective is to distinguish between lower levels of risk with high precision. In contexts where high risk cases are either handled separately, flagged through other mechanisms, or are rare enough not to significantly impact operational decisions, the model could still provide meaningful

support in streamlining assessments and reducing manual review for *many* cases. With further development or integration into a broader decision-support system, the model could be a useful component in triaging and prioritizing lower-risk categories efficiently.

Boilerplate code templates for your projects

Breaking a machine learning project into separate modules, such as **preprocessing.py**, **train_classification.py**, and **train_regression.py**, follows the core software engineering principle of separation of concerns. Each script handles a specific responsibility:

- **Preprocessing:** Takes care of data cleaning, feature engineering, transformations, and optional steps like PCA or feature selection.
- **Training:** Focuses on model fitting, hyperparameter tuning, and evaluation.

This modular structure enhances code clarity, reusability, and maintainability. For example, if you decide to change your preprocessing strategy (e.g., from standard scaling to min-max normalization), you only need to update **preprocessing.py**. Both classification and regression workflows automatically benefit from that single change. Likewise, new models or training routines can be introduced without altering the preprocessing logic.

By centralizing all data handling in one place, you ensure consistency across training and inference, especially important in production pipeline environments like **Vertex AI** (<https://cloud.google.com/vertex-ai/docs/pipelines/introduction>), **SageMaker** (<https://docs.aws.amazon.com/sagemaker/latest/dg/pipelines-overview.html>), or **Kubeflow** (<https://www.kubeflow.org/>), though these are beyond the scope of this book.

In the supporting GitHub repository, you will find a configuration-driven boilerplate for end-to-end workflows in both regression and classification. All you need to do is drop your dataset into the working directory, adapt the

appropriate YAML configuration file (for either regression or classification), and the pipeline will run automatically. No need to touch the core scripts. This setup is not only ready to use as-is but also serves as a flexible skeleton for your own projects, requiring only minimal changes to get started.

The working tree for this project is included as follows:

```
|-- boilerplate
| |-- __init__.py
| |-- directory.py
|-- config_classification.yaml
|-- config_regression.yaml
|-- data_classification.csv
|-- data_regression.csv
|-- outputs
| |-- ascii_tree.png
| |-- classification_model.joblib
| |-- regression_model.joblib
|-- preprocessing.py
|-- train_classification.py
|-- train_regression.py
|-- visualize_directory.ipynb
```

Figure 10.6: ASCII tree of boilerplate code

Taking an example config file (**config_classification.yaml**) we can see how this needs to be used, in the YAML code as follows:

1. task: classification
2. data:
3. categorical_features: []
4. numerical_features:
5. - mean radius
6. - mean texture
7. - mean perimeter
8. - mean area
9. - mean smoothness

- 10. path: data_classification.csv
- 11. random_state: 42
- 12. target: target
- 13. test_size: 0.2
- 14. model:
 - 15. model_class: sklearn.ensemble.RandomForestClassifier
 - 16. model_params:
 - 17. random_state: 42
 - 18. param_distributions:
 - 19. max_depth:
 - 20. type: choice
 - 21. values:
 - 22. - null
 - 23. - 10
 - 24. - 20
 - 25. n_estimators:
 - 26. high: 300
 - 27. low: 100
 - 28. type: int
 - 29. tune_hyperparameters: true
 - 30. type: random_forest
- 31. output:
 - 32. model_path: outputs/classification_model.joblib
- 33. preprocessing:
 - 34. cv: 5
 - 35. estimator: logistic_regression
 - 36. method: rfecv

```
37. scoring: f1_macro
38. use_feature_selection: true
39. use_pca: false
40. search:
41.   cv: 5
42.   n_iter: 10
43. scoring: f1_macro
```

This YAML configuration file defines all the necessary settings to train and evaluate a classification model using a modular, automated machine learning pipeline. It specifies the task type (**classification**), the dataset location (**data_classification.csv**), and clearly separates numerical and categorical features. It also defines the target column, test/train split, and the random seed for reproducibility. The model section outlines the estimator to be used, in this case, a **RandomForestClassifier**, along with its initial parameters and the hyperparameter search space for tuning (**n_estimators** and **max_depth**). The preprocessing section includes feature selection using RFECV with logistic regression as the selector, and the search section configures cross-validation and scoring strategy for hyperparameter optimization.

Since this configuration controls everything (from preprocessing and model selection to tuning and evaluation), the underlying Python scripts do not need to be modified. All you must do is swap out the dataset and update the YAML config with relevant columns and model settings, and the pipeline will automatically adapt. This makes the setup highly reusable and ideal for rapid experimentation across different classification problems with minimal code changes.

There is a vast world of possibilities when it comes to extending pipelines and modularizing machine learning code for production-grade systems, whether it is adding experiment tracking (MLFlow, Tensorboard, etc.), integrating with cloud services, or automating deployments. However, this

marks the end of our chapter and our exploration into the power and flexibility of pipelines. By now, you have seen how structuring your workflow with reusable, configurable components not only streamlines experimentation but also lays the foundation for scalable and maintainable machine learning projects.

Conclusion

In this chapter, we explored the full potential of scikit-learn's pipelines and transformers in structuring a complete and maintainable machine learning workflow. From basic preprocessing tasks like scaling and encoding, to advanced strategies such as building custom transformers, combining heterogeneous data pipelines with **ColumnTransformer**, and handling class imbalance through oversampling, we established a comprehensive, modular pipeline. These components were then chained together using scikit-learn's **Pipeline** and **FeatureUnion** objects, enabling a clean separation of concerns and reproducible model training steps. We also introduced best practices like saving pipeline artifacts with **joblib**, organizing code into logical modules, and using configuration files to manage experimentation efficiently.

By combining preprocessing and modeling in unified workflows, we demonstrated how pipelines reduce the risk of data leakage, streamline hyperparameter tuning, and improve the clarity of complex machine learning systems. Whether working with individual models or ensemble methods like stacking, we saw how these pipelines support flexibility and scalability, critical features for both experimentation and production deployment. As a result, you are now equipped with the tools and understanding needed to confidently build end-to-end machine learning systems that are not only robust and efficient, but also ready to adapt and evolve with new challenges.

In the next chapter, we will cover the fundamentals of neural networks and **multilayer perceptrons (MLPs)**, walk through essential preprocessing steps for deep learning, and demonstrate how to build deep neural networks

using scikit-learn wrappers. We will also guide you through transitioning to PyTorch and TensorFlow, and show how to save and load models for production use.

Points to remember

Here are some key points summed up from this chapter:

- **Boilerplate code speeds up projects:** Leverage the provided reusable pipeline templates and modular project structure to jumpstart real-world machine learning tasks quickly and cleanly.
- **ColumnTransformer handles mixed data types:** Apply different preprocessing strategies (e.g., scaling, encoding, imputation) to numerical and categorical features in parallel with a single object.
- **Custom transformers add flexibility:** Use BaseEstimator and TransformerMixin to encapsulate domain-specific preprocessing logic, such as feature binning or custom encodings.
- **FeatureUnion enables parallel processing:** Combine the output of multiple independent feature transformers ideally when extracting complementary representations from the same input.
- **Oversampling must occur inside the pipeline:** Techniques like SMOTE or SMOTEENN should be applied after preprocessing but before model fitting to avoid information leakage and must use the imblearn pipelines package imports, as the scikit-learn pipeline does not support this oversampling technique natively.
- **Pipelines prevent data leakage:** By chaining preprocessing and modeling steps, pipelines ensure consistent transformations across training and inference phases.
- **Save pipelines, not just models:** Persisting the entire pipeline (including preprocessing) ensures reproducibility and avoids retraining transformers at inference time.

- **Stacking improves generalization:** Use ensemble techniques like stacking to combine the strengths of multiple models into a more robust meta-model.
- **Use config files for maintainability:** Externalize model parameters and pipeline settings into YAML or JSON files to make experiments cleaner and easier to manage.
- **Visualizing pipelines helps debugging:** DAG-style visualizations make complex preprocessing and model workflows easier to understand, optimize, and explain.

Multiple choice questions

1. **What is one of the main advantages of using a pipeline in scikit-learn?**
 - a. It increases the complexity of your model
 - b. To enhance model performance by optimizing algorithm settings
 - c. It automatically tunes hyperparameters
 - d. It removes the need for cross-validation
2. **Why is ColumnTransformer particularly useful in tabular machine learning workflows?**
 - a. It allows different transformations to be applied to different subsets of columns
 - b. It converts all features into a single numerical format
 - c. It selects features based on importance
 - d. It automatically removes highly correlated features
3. **What risk does a pipeline help mitigate when building a machine learning model?**
 - a. Underfitting the training data

- b. Excessive model variance
 - c. Overuse of computational resources
 - d. Data leakage between preprocessing and modeling steps
4. **What role does FeatureUnion play in a machine learning pipeline?**
- a. It applies transformations sequentially
 - b. It replaces missing values
 - c. It merges outputs from multiple parallel transformers
 - d. It removes duplicate features from different pipelines
5. **What is the purpose of saving a preprocessor pipeline using joblib?**
- a. To reuse the exact same transformation logic at inference time
 - b. To compress training data
 - c. To visualize pipeline steps
 - d. To version control hyperparameter settings
6. **What is the key benefit of using a configuration file (like YAML or JSON) for pipeline settings?**
- a. It increases the runtime performance of the pipeline
 - b. It separates code from parameters for cleaner, reusable workflows
 - c. It automatically chooses the best model
 - d. It enables automatic cloud deployment
7. **In a production setting, why is modular pipeline design important?**
- a. It reduces the need for feature engineering
 - b. It prevents model overfitting

- c. It avoids the need for retraining models
 - d. It simplifies updates, testing, and integration with new data
8. **How does a custom transformer enhance a scikit-learn pipeline?**
- a. It allows you to encapsulate domain-specific logic into reusable components
 - b. It replaces the need for built-in transformers
 - c. It guarantees higher model accuracy
 - d. It automatically selects hyperparameters
9. **Why is visualizing pipeline DAGs useful in machine learning workflows?**
- a. It shortens training time
 - b. It automatically detects feature drift
 - c. It helps debug, explain, and communicate the flow of transformations clearly
 - d. It shortens training time

Answers

1	b
2	a
3	d
4	c
5	a
6	b
7	d
8	a
9	c

Key terms

Some key terms used in this chapter are:

- **ColumnTransformer:** A function that enables applying different preprocessing pipelines to different subsets of columns, especially useful for mixed-type tabular data.
- **Custom transformer:** A user-defined class based on `BaseEstimator` and `TransformerMixin` in scikit-learn, allowing encapsulation of domain-specific preprocessing logic such as binning, text normalization, or custom encoding.
- **Data leakage:** A critical issue in machine learning workflows where information from the target variable or test set is inadvertently used during training, leading to overly optimistic performance. Pipelines help mitigate this risk.
- **FeatureUnion:** A utility that allows combining the outputs of multiple transformers operating in parallel into a single feature set, useful for capturing diverse representations of the same input data.
- **Joblib:** A Python library used to efficiently serialize (save) scikit-learn pipelines, models, and transformers for reuse in deployment or inference pipelines.
- **Model per pipeline strategy:** A modular approach where each model is paired with its own pipeline, allowing independent training and tuning, improving maintainability and experimentation flexibility.
- **Pipeline (scikit-learn):** A tool that allows sequential chaining of data preprocessing and model fitting steps, ensuring that the same transformations are consistently applied during both training and prediction, thereby preventing data leakage.
- **Pipeline DAG:** A visual representation of the sequence and structure of operations within a pipeline, useful for debugging and communicating complex workflows.

- **Preprocessor pipeline:** A reusable and modular structure that encapsulates all preprocessing logic (scaling, encoding, imputing, etc.) and can be persisted independently from the model itself.
- **Transformer:** An object in scikit-learn that implements both `fit()` and `transform()` methods, used to apply preprocessing steps like scaling, encoding, and imputing.

Join our Discord space

Join our Discord workspace for latest updates, offers, tech happenings around the world, new releases, and sessions with the authors:

<https://discord.bpbonline.com>



CHAPTER 11

Getting Deep with Deep Neural Networks

Introduction

In this chapter, we will cover the foundational concepts and practical tools needed to build, train, and evaluate neural network regression models using some of the most widely used ML libraries. We will start by introducing the basics of **artificial neural networks (ANNs)** and **multi-layer perceptrons (MLPs)**, breaking down how neural networks are structured, how they learn from data, and why activation functions are essential for capturing complex patterns. Along the way, you will see how to prepare and scale your data, visualize neural network architectures, and implement MLP regression models using scikit-learn's intuitive API.

As we move to industry-standard deep learning frameworks like PyTorch and TensorFlow, you will get hands-on with the entire deep learning workflow. This includes creating efficient DataLoaders for batching and shuffling data, writing custom training loops, and leveraging checkpointing and early stopping to prevent overfitting and save your best model during training. We will also explore how to set up custom loss functions, optimizers, and callbacks for robust model training. Diagnostic plots and evaluation metrics will help you interpret and troubleshoot your models at every step. At the end of this chapter, you will have practical experience in building, tuning, checkpointing, and interpreting deep neural networks for regression tasks, and you will be well prepared for deeper explorations in modern deep learning.

Structure

This chapter covers the following topics:

- Introducing neural networks and multi-layer perceptrons
- Building deep neural networks using wrappers
- Transitioning to PyTorch and TensorFlow

Objectives

By the end of this chapter, you will have acquired hands-on experience building, training, tuning, checkpointing, and evaluating neural network regression models using scikit-learn, PyTorch, and TensorFlow. You will start by mastering the essentials of data preparation, including preprocessing techniques like feature scaling, handling missing values, and splitting your data for training and evaluation. You will also learn how to streamline your workflow by batching data with DataLoaders in PyTorch and leveraging efficient input pipelines in TensorFlow, enabling your models to scale and train effectively on larger datasets.

Moving beyond the basics, you will implement critical best practices such as early stopping to prevent overfitting and model checkpointing to save and restore your best models. To round out your skill set, you will gain the ability to interpret your results through diagnostic plots: including learning curves, residual plots, and scatter plots of predicted versus true values, helping you visualize model performance and identify opportunities for improvement. By the end of this chapter, you will be equipped with the practical tools and knowledge to confidently develop, tune, and evaluate neural network regression models in real-world scenarios.

Introducing neural networks and multi-layer perceptrons

ANNs are a class of machine learning models inspired by the structure and function of the human brain. They are composed of interconnected units called *neurons*, which work together to process data, learn patterns, and make decisions. These neurons function similarly to how the human brain

recognizes faces, understands natural language, or makes predictions on an outcome based on past experiences; ANNs try to do this computationally.

At the core of the neural network lies a fundamental principle: *learning from data*. Whether it is recognizing handwritten digits, translating languages, classifying risk, learning from imagery, or making recommendations, neural networks adjust their internal parameters through training, gradually improving their performance.

Structure of a neural network

A typical neural network has these main core components:

- **Input layer:** It receives the raw data (e.g., pixel values from an image, word vectors from a sentence, features from a numerical model, etc.).
- **Hidden layers:** These layers perform computation and feature transformation. Hidden layers replace the need for feature selection, as these layers learn latent features from the underlying data and can choose which latent variables are most important. An example of how these layers work, say the first hidden layer learning basic patterns (e.g. edges in an image, or combinations of numerical inputs); the second hidden layer may combine these layers into meaningful features and later hidden layers might detect more abstract concepts like grammar in language, faces or an object in an image or sentiment in text.
- **Output layer:** Output layer produces the final prediction or result (e.g., a class label, cancer or not, a continuous value, house price, or the likelihood of an object in an image).

Each neuron in a layer is connected to neurons in the next layer. These connections have *weights*, which determine how much influence one neuron has on another. Additionally, each neuron has a *bias*, an extra parameter that helps the model make better predictions.

Activation functions

Neural networks rely on non-linear activation functions to model complex patterns in data. Without these functions, even deep networks would behave like a single linear model, unable to capture interactions or non-linear relationships. Activation functions introduce non-linearity, enabling the network to learn features like curves (non-linearity), thresholds (decision boundaries), and combinations of inputs (interaction effects between inputs).

The most commonly used activation functions are captured as follows:

- **Rectified linear unit (ReLU):** ReLU outputs zero for inputs less than zero and returns the input unchanged for values greater than zero. This introduces non-linearity and sparsity, and helps mitigate the vanishing gradient problem (a problem that occurs when gradients become extremely small during backpropagation, causing early layers in deep networks to learn very slowly or not at all), making it well-suited for deep networks.
- **Sigmoid:** Sigmoid maps input values to a range between 0 and 1 using an S-shaped curve. It is typically used in binary classification tasks and in output layers where probability-like outputs are required.
- **Softmax:** Softmax converts a vector of arbitrary real values into a probability distribution, where the output values sum to 1. It is commonly used in the final layer of multi-class classification models to indicate the most likely class.
- **Leaky ReLU:** This is a variant of ReLU that allows a small, non-zero gradient when the input is negative. This helps avoid the *dying ReLU problem*, where neurons can become inactive and stop learning.
- **Tanh (hyperbolic tangent):** It maps inputs to the range $[-1, 1]$. Unlike sigmoid, it is zero-centered, which can lead to faster and

more stable training in some cases by ensuring gradients flow more symmetrically through the network.

These functions are applied to help capture complex patterns in data.

The foundations of a multi-layer perceptron

MLP is one of the foundational architectures of neural networks. It consists of the three layers that we have discussed in the *Structure of a neural network* section. Each node (except those in the input layer) uses a neuron that uses a non-linear activation function.

Mathematically, for a single hidden layer, the output $\hat{y}$ can be described as shown:

$$\hat{y} = f(W^{(2)} \cdot g(W^{(1)} \cdot x + b^{(1)}) + b^{(2)})$$

To describe the formula in *Figure 11.1*, these terms are the terms to be aware of:

- x is the input vector (the data vector).
- W^1 is the first set of hidden importance weights (how much each input matters).
- b^1 is the first set of bias adjustments based on the hidden layer.
- In this section, multiply each input by its importance weight, add them up, then add the bias.
- g applies the first activation function to decide which neurons get activated.
- We do the same for the output layer weights denoted with 2, and take the hidden layer results, multiply by new weights, add new bias, and apply the final activation function to either sigmoid or Softmax.
- $\hat{y}$ is the final prediction from the model.

In practice, our MLP will usually have many more layers, and the function will get much more complex as we add more hidden layers. This example just shows how the equation looks when there is only one hidden layer. To visualize an MLP, we can use the **modelviz** package and use the following steps:

1. Import the relevant module from **modelviz**:

```
from modelviz.deep_learning import draw_mlp
```

2. To draw the ML, we can use the following command, specifying the layer size as a list, with neuron radius and weight font size added as parameters:

```
draw_mlp(layer_sizes=[3,8,1],  
weight_fontsize=8,  
neuron_radius=0.35)
```

3. Upon running *Step 2*, the output will be shown, as in *Figure 11.1*:

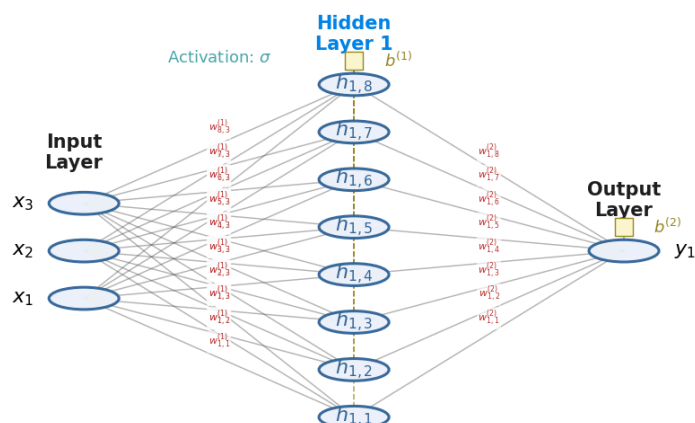


Figure 11.1: Shallow multi-layer perceptron

4. *Figure 11.2* is an example of a shallow neural network, as it only contains one hidden layer. For a neural network to be considered

deep, it must have more than one layer, normally two or three hidden layers; however, this serves as a great way to visualize the network.

5. To make a deep network, we can use our **draw_mlp** function again and pass in two hidden layers:

```
draw_mlp(layer_sizes=[2,4,8,1],
         neuron_radius=0.35,
         weight_fontsize=8)
```

6. When you run *Step 5*, you will get a perfect example of a deep MLP model:

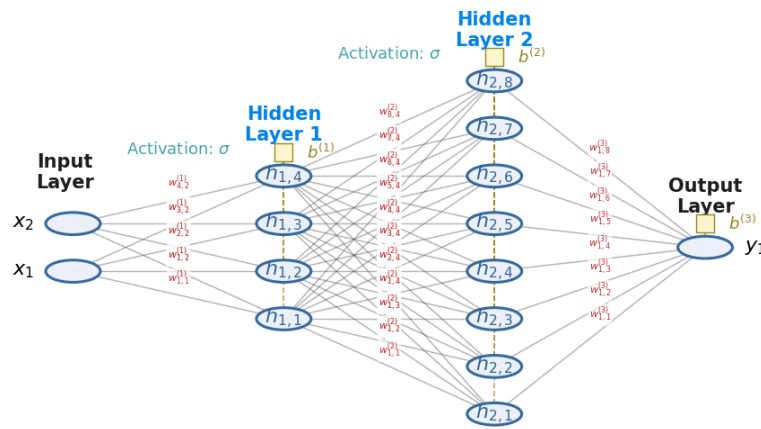


Figure 11.2: Deep multi-layer perceptron

In the following sections, we will look at training an MLP with scikit-learn's neural network packages. There are many types of neural networks designed for different tasks; while this introduction will focus on MLPs, other popular architectures include **convolutional neural networks** (CNNs) for image processing, **recurrent neural networks** (RNNs) and **long short-term memory** (LSTM) networks for sequential data,

generative adversarial networks (GANs) for data generation, and transformer-based models for natural language processing.

Building deep neural networks using wrappers

In this section, we will work with a regression dataset focused on predicting personal income based on features such as weekly working hours, experience, and other relevant factors. We will utilize various tools and modules from scikit-learn to build and train our neural network model.

Data preparation

A model cannot be built without data, and these steps will prepare our dataset in the appropriate format for a neural network. Unlike traditional tabular machine learning models, neural networks typically require less manual preprocessing. Aside from ensuring that the data is numerical and properly scaled, extensive feature selection is often unnecessary, as the network can learn latent patterns and representations directly from the data. The following steps will get the data primed and ready for modeling:

1. Read in the **.csv** file to bring the data into our Python project:

```
import pandas as pd
```

```
mlp_data = pd.read_csv('data/synthetic_income_data.csv',  
                        index_col=None)
```

2. View the top records, using the head command:

```
mlp_data.head()
```

3. Running the command in *Step 2*, the DataFrame will be output, as in *Figure 11.3*:

education_level	experience_years	hours_per_week	age	gender	marital_status	location_index	income
18	11	53	61	0	1	0	64007.910647
15	12	36	37	0	1	4	57575.419853
19	35	29	27	1	1	3	104629.078728
16	38	37	26	0	0	3	100476.778151
18	33	27	56	0	1	2	92752.119875

Figure 11.3: Income prediction data

4. Create the feature and target variables:

```
X = mlp_data.drop(columns=['income'])
```

```
y = mlp_data['income']
```

5. Perform splitting on the data to obtain a train and test dataset:

```
RANDOM_SEED = 42
```

```
SPLIT_RATIO = 0.2
```

```
(X_train, X_test,
```

```
y_train, y_test) = train_test_split(X, y,
```

```
test_size=SPLIT_RATIO,
```

```
random_state=RANDOM_SEED)
```

6. The last step is to scale the data:

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

The dataset is now ready to be used with the MLP that we will create in the next section.

Building and training the MLPRegressor

Scikit-learn provides a convenient and accessible implementation of neural networks through the **MLPRegressor** and **MLPClassifier** classes, which are part of its **neural_network** module. These models are based on the MLP architecture and allow users to apply neural networks to both regression and classification problems with minimal setup. Let us take a closer look:

- **MLPClassifier** is designed for classification tasks, where the goal is to predict a categorical label.

- **MLPRegressor** is intended for regression tasks, where the output is a continuous numerical value, as with the income dataset.

Both models support essential neural network features such as multiple hidden layers, non-linear activation functions, and backpropagation-based training.

In this chapter, we will focus on the **MLPRegressor**, using it to predict personal income based on a set of numerical input features. This will allow us to explore how neural networks can be applied to regression problems using scikit-learn's high-level API. We will explore how to do this in the following steps:

1. We need to import the regressor module:

```
from sklearn.neural_network import MLPRegressor
```

2. To inspect the **MLPRegressor** object, we can use the following:

```
help(MLPRegressor)
```

3. In this step, we are creating a neural network model using **MLPRegressor** with two hidden layers; one with **64** neurons and another with **32**. The model uses the **relu** function to help it learn complex patterns. It is trained using the **adam** optimizer, which is efficient and works well for most problems in reducing the objective function loss. We set a learning rate of 0.001 (this controls how much the model updates as it learns). A smaller value like this helps the model learn more carefully and avoid missing the best solution. We also add a small regularization term (**alpha = 0.0001**) to prevent overfitting. The model is allowed to train for up to 500 rounds (iterations), and we shuffle the data each time to help it generalize better. Finally, we fix the random seed for reproducibility and turn on verbose mode to see the training progress:

```
mlp_sklearn = MLPRegressor(  
    hidden_layer_sizes=(64, 32),
```

```
activation='relu',  
solver='adam',  
alpha=0.0001,  
learning_rate_init=0.001,  
max_iter=500,  
shuffle=True,  
random_state=RANDOM_SEED,  
verbose=True)
```

4. Now that we have initialized the model, we need to fit the model with our training data:

```
mlp_sklearn.fit(X_train_scaled, y_train)
```

5. When you hit go in *Step 4*, you will see each iteration and the current loss level. For this, we are using a regression model, so it will be the **mean squared error (MSE)** that the optimizer aims to reduce.
6. Once the model has been trained and fitted with the training data, we can make predictions against our holdout test set:

```
preds = mlp_sklearn.predict(X_test_scaled)
```

7. We will use the **compute_metrics** function we created and have been using throughout this book to statistically evaluate our fitted model:

```
compute_metrics(y_test, preds)
```

In the next section, we will tune the hyperparameters of our model to improve the model further.

Hyperparameter tuning of MLP

While our initial **MLPRegressor** model provides a solid starting point, its performance can often be improved by fine-tuning its hyperparameters.

These include settings like the number and size of hidden layers, activation functions, learning rate, regularization strength, and more.

In this section, we will use **GridSearchCV**, a tool from scikit-learn that systematically tests combinations of hyperparameters to find the set that yields the best performance based on cross-validation. This approach helps us move beyond trial-and-error and makes our model more robust and better optimized for the dataset.

By exploring different configurations, we aim to reduce prediction error, prevent overfitting, and improve generalization to unseen data. This will be achieved in the following steps:

1. To change things up, we will learn about the **make_scorer** function, which allows you to use a custom function as the objective function for the model. Here we will implement the MSE function from scratch:

```
from sklearn.metrics import make_scorer  
  
def mse(y_true, y_pred):  
    return ((y_true - y_pred) ** 2).mean()
```

2. We will now use the function, with the **make_scorer** import, to assign the objective function to be **root mean squared error (RMSE)**:

```
mse_scorer = make_scorer(mse,  
                           greater_is_better=False)
```

3. To perform grid searching, we need to tell the tuning method what parameter ranges to use:

```
param_grid = {  
    'hidden_layer_sizes': [(64, 32), (128, 64), (64, 64)],  
    'alpha': [0.0001, 0.001, 0.01],  
    'learning_rate_init': [0.001, 0.01],
```

```
'activation': ['relu', 'tanh']}]}
```

4. In this step, we use **GridSearchCV** to find the best combination of hyperparameters for our **MLPRegressor**. First, we define the base model with the ADAM optimizer, 500 training iterations, and shuffling enabled for each epoch. We then specify a parameter grid containing the hyperparameters we want to test. Using our custom MSE scoring function, the grid search evaluates each combination through three-fold cross-validation. By setting **n_jobs=-1**, we ensure all CPU cores are used to speed up the search process:

```
from sklearn.model_selection import GridSearchCV
```

```
grid = GridSearchCV(  
    estimator=MLPRegressor(  
        solver='adam',  
        max_iter=500,  
        shuffle=True,  
        random_state=RANDOM_SEED,  
        verbose=False),  
    param_grid=param_grid,  
    scoring=mse_scorer,  
    cv=3,  
    n_jobs=-1)
```

5. Once we have undertaken the initialization in *Step 4*, we are ready to fit the grid search model to choose our best model, based on the objective metric (MSE):

```
grid.fit(X_train, y_train)
```

6. To obtain the best hyperparameters, out of the cross-validation fold of models, you can use the following code to print this out to the console:

```
print("Best Params:", grid.best_params_)
```

7. To get the best score from the model:

```
print("Best Score (MSE):", grid.best_score_)
```

8. We will obtain the best model and assign it to a variable for evaluation:

```
best_model = grid.best_estimator_
```

9. As we did before, we will predict on the testing dataset, and store this in a variable:

```
preds = best_model.predict(X_test)
```

10. We have everything we need at this point, and now we can go ahead and serialize the model for utilization in later scripts:

```
import joblib
```

```
import os
```

```
model_dir = 'models'
```

```
os.makedirs(model_dir, exist_ok=True)
```

```
joblib.dump(best_model, os.path.join(model_dir,  
                                     'mlp_sklearn_best_model.joblib'))
```

In the next section, we will define new visualization functions to effectively visualize regression performance.

Visualizing regression performance

Evaluating a regression model goes beyond looking at metrics like RMSE or R^2 . Visual diagnostics help us understand how the model is making errors and whether the assumptions behind regression are reasonably met. In this section, we introduce three custom plotting functions that assist in analyzing and interpreting the residuals, the differences between actual and predicted values. In the supporting workbook are three custom functions that have been created for effective regression diagnostics:

- **actual_vs_predicted_plot:** This scatter plot shows the relationship between actual and predicted values. A well-performing model should produce points that align closely with the diagonal reference line. Deviations from this line can highlight underfitting, overfitting, or heteroscedasticity.
- **dist_of_residuals:** This function plots a histogram of the residuals, allowing us to quickly assess their distribution. Ideally, residuals should be symmetrically distributed around zero. Skewness or outliers in this plot may indicate model bias or unaccounted-for patterns in the data.
- **qq_plot:** The **quantile-quantile (Q-Q)** plot compares the distribution of residuals to a normal distribution. If residuals follow a straight line in this plot, it suggests that they are approximately normally distributed, a key assumption in many regression contexts.

Together, these visual tools provide valuable insights into the model's performance, helping identify issues like non-linearity, variance instability, or outliers that may not be obvious through numerical metrics alone.

Actual versus predicted plot

Understanding the relationship between actual and predicted values is a fundamental step in diagnosing regression model performance. This comparison provides a clear visual indication of how well the model is capturing patterns in the data. By plotting actual values on the x-axis and predicted values on the y-axis, we can assess how closely the model's outputs align with reality. Ideally, the points should cluster around the diagonal line, indicating accurate and unbiased predictions. The following steps help create this visualization:

1. Use our user-defined function (created in the supporting GitHub Jupyter Notebook) to get this function:

```
actual_vs_predicted_plot(y_test, preds)
```

2. This function generates a chart, as in *Figure 11.4*:

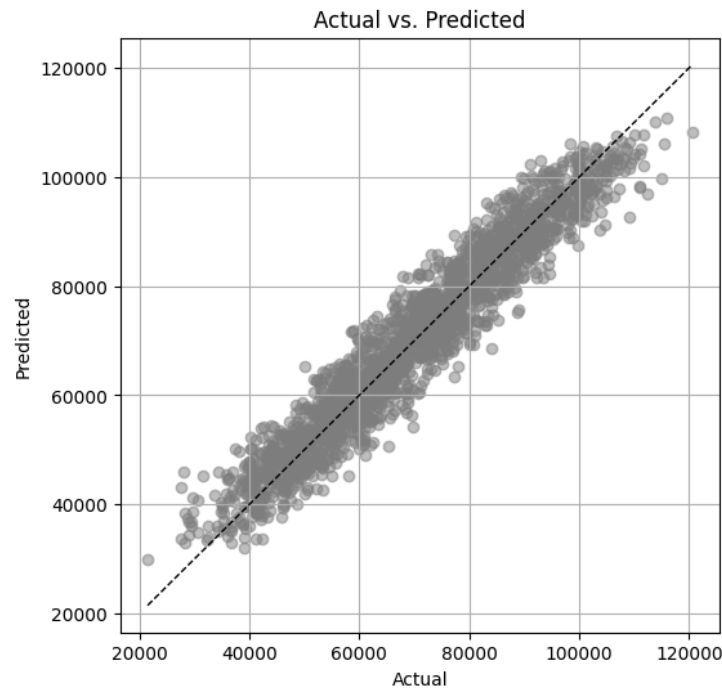


Figure 11.4: Residuals scatter

The plot in *Figure 11.5* shows a strong alignment between actual and predicted values, with most points closely following the diagonal reference line, indicating that the model is making accurate and consistent predictions across the full range of income values. We will use another method in the following subsection to visualize the performance further.

Distribution of residuals plot

Analyzing how prediction errors are spread out is an important part of evaluating a regression model's behavior. These errors, known as residuals, represent the difference between the predicted values and the actual target values. Visualizing the residuals as a histogram allows us to detect signs of model bias, such as consistently over-estimating or under-estimating outcomes, and to check whether the errors are evenly distributed. A centered, roughly symmetric shape suggests the model is performing reliably, while skewness or heavy tails may signal issues that need further

investigation. The following steps show how to generate this diagnostic plot:

1. To run the distribution of residuals user-defined function, we use the following command:

```
dist_of_residuals(y_test, preds, line_width=2)
```

2. Running the function, you see the plot as in *Figure 11.5*:

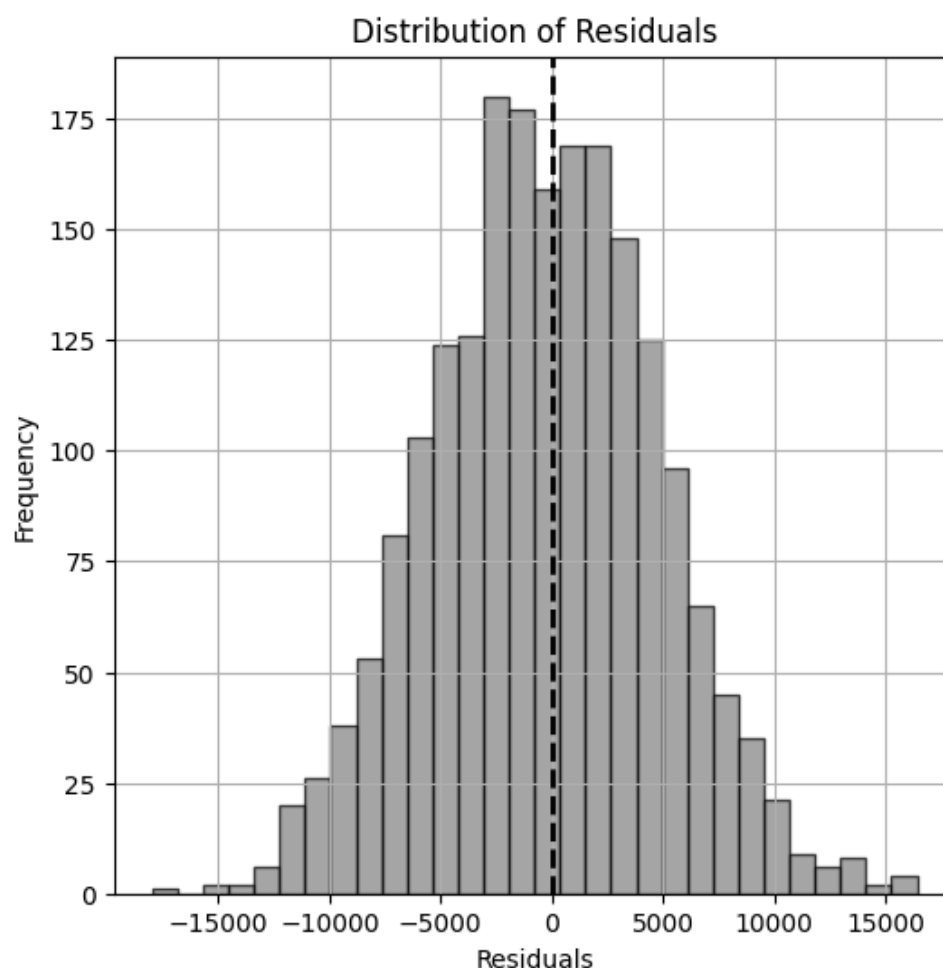


Figure 11.5: Residuals histogram

This histogram shows that the residuals are approximately normally distributed around zero, indicating that the model's errors are unbiased and mostly small.

Quantile-quantile chart

A Q-Q plot is a valuable diagnostic tool used to assess whether a dataset, such as model residuals, follows a particular theoretical distribution, most commonly, the normal distribution. In regression analysis, verifying that residuals are normally distributed is crucial because many statistical tests and confidence intervals rely on this assumption. The Q-Q plot works by plotting the quantiles of the residuals against the corresponding quantiles of a normal distribution. If the residuals are normally distributed, the points on the Q-Q plot will fall approximately along a straight diagonal line. Deviations from this line indicate departures from normality, such as skewness or the presence of outliers. To generate this, with our user-defined function, we use the following steps:

1. Use the code, shown as follows, to generate the Q-Q plot from the custom-defined functions available in the supporting workbook:

```
qq_plot(residuals, point_color='grey',  
        line_color='black',  
        line_style='--',  
        point_size=50)
```

2. This will then output the chart, as shown in *Figure 11.6*:

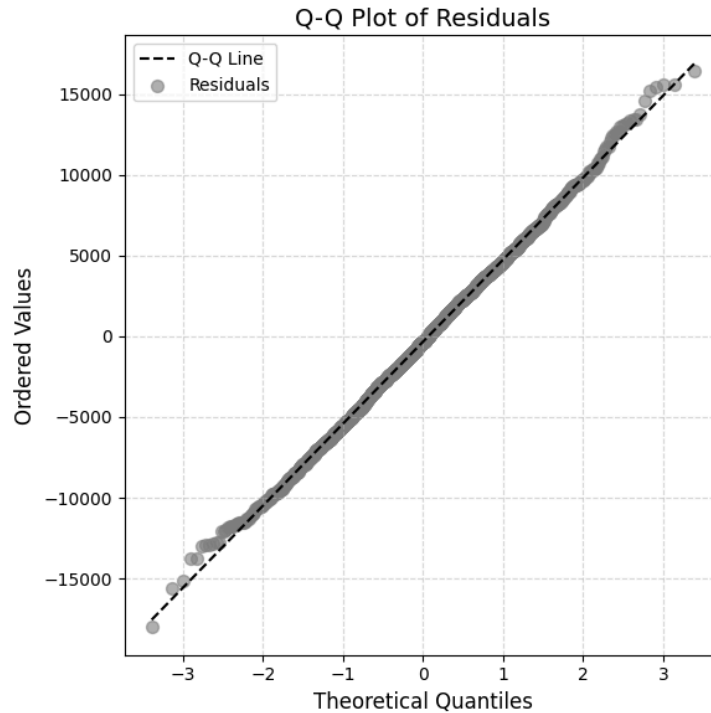


Figure 11.6: *Quantile-quantile (Q-Q) plot*

This Q-Q plot shows that the residuals closely follow the expected normal distribution, as the points lie almost perfectly along the reference line, indicating the normality assumption is well satisfied.

You now possess the foundational understanding needed to build regression neural networks with scikit-learn. In the upcoming section, we will explore the industry-standard tools for deploying deep learning models. PyTorch and TensorFlow are the most prominent frameworks in this area, and we will walk through building models with both PyTorch and the Keras API in TensorFlow.

Transitioning to PyTorch and TensorFlow

As you progress from introductory machine learning models to more advanced and large-scale deep learning applications, it becomes increasingly important to use frameworks specifically designed for building, training, and deploying neural networks. While scikit-learn is an excellent toolkit for classic machine learning algorithms and does have

support for basic neural networks, as we explored in the previous section, it is not optimized for the demands of deep learning. There are several key reasons why most practitioners and companies transition to deep learning libraries like PyTorch or TensorFlow. These are detailed in the following list:

- **Scalability and performance are crucial in deep learning:** PyTorch and TensorFlow are engineered to handle very large datasets, complex architectures (such as convolutional neural networks, transformers, and recurrent neural networks), and intensive computational tasks. Both frameworks are built to leverage modern hardware accelerators like GPUs and **tensor processing units (TPUs)**, enabling much faster training and experimentation, compared to the CPU-only approach of scikit-learn's neural network modules, as most of these are built on NumPy under the hood.
- **Flexibility of custom model deployment:** It is the core of these frameworks that allows users to build customized model architectures, loss functions, and training loops. PyTorch, in particular, is known for its *define by run* dynamic computation graph, which makes it straightforward to debug and modify models during development. It is also the backbone for most models on **HuggingFace** (<https://huggingface.co/>).
- **Community support and ecosystem maturity:** People love these frameworks, and they are widely used in industry and academia, meaning there is a vast ecosystem of pretrained models, extensions, tutorials, and community forums and support. They are continually updated to support the latest research advancements and integrate seamlessly with deployment tools, visualization libraries, and production environments.
- **Production readiness:** While scikit-learn is excellent for legacy machine learning models, these frameworks provide extensive

tooling for model serving, optimization, and cross-platform deployment, features essential for taking deep learning models from research to real-world applications.

In the following sections, we will build an MLP regression model, using the same data as we have already processed and been working with in this chapter, in PyTorch and TensorFlow's Keras API.

Building multi-layer perceptrons in PyTorch

To build an MLP regressor in **PyTorch** (<https://pytorch.org/>), you start by creating a custom neural network class that inherits from **nn.Module**. Within this class, you define the necessary layers, such as fully connected (**nn.Linear**) layers, in the constructor (In Python, the `__init__` method is a special function in a class that runs automatically when you create a new object. It initializes the object's attributes and sets up its initial state) and then implements the forward pass to specify how data moves through the network. This flexible approach allows you to easily customize the architecture, activation functions, and output layer to suit your task, providing full control over model design and training in PyTorch.

Split and scale the training data

As this is a model where we need data to guide the optimization of a loss function, we will need to split the data differently, into train, validation, and test sets. In the next set of steps, we will split the data, assuming that you have the data in feature and target formats. To get the data into the right shape, we will undertake these steps:

1. In this step, we will create a train, test, and validation set for use with our PyTorch training loop, which we will define in a later section:

```
from sklearn.model_selection import train_test_split
```

```
RANDOM_SEED = 42
```

```
X_temp, X_test, y_temp, y_test = train_test_split(
```

```
X, y, test_size=0.1, random_state=RANDOM_SEED)
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=1/9, random_state=RANDOM_SEE
D)
```

2. Now, we will scale the data, using the same scalar as we did in building the neural network in the scikit-learn section:

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
X_val_scaled = scaler.transform(X_val)
```

That prepares the data nicely for our next steps, which will involve converting the X features from each of the splits into PyTorch Tensors, analogous to NumPy arrays.

Setting a PyTorch device

Before building and training deep learning models in PyTorch, it is important to specify which device will handle the computations, either a GPU (if available) for faster performance or the default CPU. The following code automatically checks if a compatible GPU is present on your system and sets PyTorch to use it; otherwise, it falls back to using the CPU. This ensures your code is flexible and can take advantage of hardware acceleration when possible:

1. Before we specify the device, we need to import the torch library:

```
import torch
```

2. This step checks if a GPU is available and sets the device. PyTorch will use this to either perform computation on a CPU, or a GPU (if available). Furthermore, for some models, this device is used to raise exception messages if a GPU is not available, as some models are not designed to run on a CPU:

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

Now that we have our device set, we will turn to converting our raw features into tensors.

Tensor conversion and dataset creation

PyTorch requires data to be in its native tensor format, which closely resembles NumPy arrays but is specifically optimized for fast computation and GPU acceleration. Converting both the feature and target variables to PyTorch tensors ensures they can be seamlessly used with PyTorch's models and utilities. In the next steps, we will define a function for this conversion and package the tensors into a PyTorch dataset. In the following steps, we will define a function to convert these features and then create them as a dataset:

1. This codes step defines a function that converts the input data into a PyTorch tensor of type **float32**, and, if requested, reshapes it to have one column and as many rows as need (i.e., shape (N,1), where N is automatically inferred based on the number of elements) using **tensor.view(-1,1)**; otherwise it returns the tensor as is:

```
from torch.utils.data import DataLoader, TensorDataset  
def to_tensor(data, reshape=False):
```

```
    tensor = torch.tensor(data, dtype=torch.float32)
```

```
    return tensor.view(-1, 1) if reshape else tensor
```

2. The next step is to apply this to all the features and targets in our data:

```
X_train_torch = to_tensor(X_train_scaled)
```

```
y_train_torch = to_tensor(y_train.to_numpy(), reshape=True)
```

```
X_test_torch = to_tensor(X_test_scaled)
```

```
y_test_torch = to_tensor(y_test.to_numpy(), reshape=True)
```

```
X_val_torch = to_tensor(X_val_scaled)
```

```
y_val_torch = to_tensor(y_val.to_numpy(), reshape=True)
```

3. The final step, in this section, is to create a tensor dataset, which combines the data from *Step 2* into a single dataset, that is the sole purpose of this code:

```
train_dataset = TensorDataset(X_train_torch, y_train_torch)
```

```
val_dataset = TensorDataset(X_val_torch, y_val_torch)
```

In the next section, we will get into what a Tensor **DataLoader** is and why they are useful in PyTorch.

Understanding DataLoaders

A **DataLoader** in PyTorch is a powerful utility that wraps a dataset (such as a **TensorDataset**) and provides an efficient way to iterate over the data in mini-batches, optionally shuffling the data and loading it in parallel using multiple worker processes, which helps streamline the process of feeding data to a model during training or evaluation by handling batching, shuffling, and even data loading from disk automatically, making it much easier to train models on large datasets. In the following steps, we will create a loader for the train and validation sets:

1. The first loader we will create is the train **DataLoader**, which creates an iterable **train_loader** that loads data from the **train_dataset** in batches of size 32 and shuffles the data at each epoch to improve training performance and generalization.

```
train_loader = DataLoader(train_dataset, batch_size=32,  
                           shuffle=True)
```

2. We repeat the same process with the validation loader:

```
val_loader = DataLoader(val_dataset, batch_size=32,  
                        shuffle=False)
```

Following on from this, we will build the MLP class, and we will spend some time explaining the structure of the class.

Creating the base neural network in PyTorch

In PyTorch, building neural networks often starts with creating your own classes that act as the blueprint for the architecture of the network and define the forward pass of the model, which is the process of taking the input data and passing it through all layers of the network to produce an output (normally a prediction).

In the following code, we create the full class implementation of our MLP:

```
1. class MLP(nn.Module):
2.     def __init__(self, input_size, hidden_layers, output_size=1):
3.         super().__init__()
4.         layers = []
5.
6.         prev_size = input_size
7.         for hidden_size in hidden_layers:
8.             layers.append(nn.Linear(prev_size, hidden_size))
9.             layers.append(nn.ReLU())
10.            prev_size = hidden_size
11.
12.        layers.append(nn.Linear(prev_size, output_size))
13.        self.net = nn.Sequential(*layers)
14.
15.    def forward(self, x):
16.        return self.net(x)
```

Let us walk through what each part of the code accomplishes:

- The code defines a class called MLP that inherits from the `nn.Module`. You will see `nn.Module` in more or less every neural network implemented in PyTorch.
- In the constructor function (`__init__`), the class is initialized with the following parameters:
 - **input_size**: The number of input features (e.g., for images, the number of pixels, and for our use case, the number of features to pass to the neural network).
 - **hidden_layers**: A list specifying the number of neurons in each hidden layer, as we visualized in *Figure 11.2* previously.
 - **output_size**: The number of outputs (default is 1, e.g., for binary classification or regression).
- The model builds a series of layers:
 - The first linear layer connects the input to the first hidden layer.
 - For each hidden layer, it adds a fully connected (meaning that every neuron in one layer is connected to every neuron in the next layer) linear layer followed by a ReLU activation function (very ubiquitous in neural networks in every framework when you start learning deep learning frameworks).
 - Finally, it adds an output layer that maps the last hidden layer to the output size (for our use case, this will be one continuous value, so the output size will be 1).
- All these layers are combined using the **`nn.Sequential`**, which chains the layers together in a sequential fashion.
- Forward pass (`forward`) method tells PyTorch how to compute the output for a given input. In our example, it simply feeds the input through the sequence of layers.

We use a class here, as it provides modularity (meaning we can easily inherit from a subclass of the **`nn.Module`**) and provides flexibility, as the

architectures of the **init** and **forward** pass methods can be adapted, as you see fit. In the next section, we will define our optimizer and loss functions for the network.

To use our model, we will now specify the input shape, the hidden layers required, and the output size. Moreover, we will place the model on the device of the computer or virtual machine you are using:

1. `model = MLP(input_size=X.shape[1],`
2. `hidden_layers=[64, 32],`
3. `output_size=1)`
4. `model = model.to(device)`

We have now pushed our model to the device and will now look at defining the optimizer and loss functions, as we need these to help our model learn.

Define optimizer and loss functions

When training a neural network in PyTorch, we need two key ingredients to make this learning possible. These ingredients are the optimizer and a loss function. The loss function acts as a guide, telling the network how well its predictions match the real answers, measuring the difference between what the network predicts and what the correct answer should be. The goal of training is to minimize (make the difference smaller) the loss function as much as possible.

The optimizer is the tool that updates the network's internal parameters (the weights and biases) based on the information from the loss function. By repeatedly making small adjustments to these parameters, the optimizer helps the network learn to make better predictions over time. In short, the loss function tells you *how wrong* your predictions are, and the optimizer tells the network *how to get better* with each training step. In the following series of steps, we will define our optimizer and loss function:

1. This code creates the **Adaptive Moment Estimation (ADAM)** (<https://docs.pytorch.org/docs/stable/generated/torch.optim.Adam>).

html) optimizer that will update the model's parameters (here we use **model.parameters()** to grab the parameter dictionary), we then use a learning rate to control how much the parameters will change with each update, this is important as a too high learning rate may mean it misses key updates and too low would mean it is more accurate, but could run ad infinitum, dependent on the training data:

```
from torch.optim import Adam
```

```
LEARN_RATE = 0.001
```

```
optimizer = Adam(model.parameters(),  
                  lr=LEARN_RATE)
```

2. We need to build a class for our loss function, as there is no MSE function defined as a default. As with the MLP class example, we will inherit from **nn.Module**, as follows:

```
class MSELoss(nn.Module):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        self.mse = nn.MSELoss()
```

```
    def forward(self, yhat, y):
```

```
        return self.mse(yhat, y)
```

3. Now the MSELoss function class has been developed, we will use it as follows:

```
loss_fn = MSELoss()
```

At this stage, we have our model architecture set up and ready for training. Unlike some other frameworks, PyTorch gives you complete flexibility over the training process. In the next section, we will explore how to build a training loop. Though it can feel challenging when you are just starting out, it is a valuable skill to learn.

Creating the training loop

A **training loop** is at the heart of teaching your neural network how to learn. Unlike some high-level libraries that handle the training process automatically, PyTorch asks you to write your own training loop, giving you full control over every step. This is where you decide exactly how your data flows through the model, how the loss is calculated, and how the optimizer updates the network's weights.

Structuring this training loop effectively is crucial because it lets you track your model's progress, adjust learning strategies, and debug issues as you go. The training loop typically represents a series of steps, such as the forward pass, loss calculation, backward pass (PyTorch calculates how much each parameter contributed to the prediction error using gradients, so it knows what to adjust for better predictions. The optimizer then updates these parameters over several cycles, called epochs, to improve the model.). Mastering the creation of the training loops is where you spend your energy learning, as it allows you to build more powerful and reliable models in PyTorch, and it opens the door to experimenting with custom training strategies and advanced techniques, when you become more comfortable. The following code shows the training loop in its entirety, and the list after the code block explains each step:

```
1. from tqdm import tqdm
2. import torch
3.
4. EPOCHS = 100
5. patience = 2
6. best_val_loss = float('inf')
7. epochs_no_improve = 0
8.
9. train_losses = []
10. val_losses = []
```

```
11.
12. for epoch in range(EPOCHS):
13.     model.train()
14.     total_train_loss = 0
15.     train_loop = tqdm(train_loader, desc=f"Epoch {epoch+1}/{EPOCHS} [Training]», leave=False)
16.
17.     for features_batch, targets_batch in train_loop:
18.         features_batch = features_batch.to(device)
19.         targets_batch = targets_batch.to(device)
20.         predictions = model(features_batch)
21.         loss = loss_fn(predictions, targets_batch)
22.         optimizer.zero_grad()
23.         loss.backward()
24.         optimizer.step()
25.         total_train_loss += loss.item()
26.         train_loop.set_postfix(train_loss=loss.item())
27.
28.     avg_train_loss = total_train_loss / len(train_loader)
29.     train_losses.append(avg_train_loss)
30.
31.     # Validation phase
32.     model.eval()
33.     total_val_loss = 0
34.     with torch.no_grad():
35.         for val_features, val_targets in val_loader:
36.             val_features = val_features.to(device)
```

```

37.         val_targets = val_targets.to(device)
38.
39.         val_predictions = model(val_features)
40.         val_loss = loss_fn(val_predictions, val_targets)
41.         total_val_loss += val_loss.item()
42.
43.     avg_val_loss = total_val_loss / len(val_loader)
44.     val_losses.append(avg_val_loss)
45.
46.     print(f"Epoch {epoch+1} | Train Loss: {avg_train_loss:.4f} |
    Val Loss: {avg_val_loss:.4f}")
47.
48.     # Early stopping based on validation loss
49.     if avg_val_loss < best_val_loss:
50.         best_val_loss = avg_val_loss
51.         epochs_no_improve = 0
52.         torch.save(model.state_dict(), 'models/best_model.pth')
53.     else:
54.         epochs_no_improve += 1
55.
56.     if epochs_no_improve >= patience:
57.         print(f"Early stopping at epoch {epoch+1} (no improvement i
    n val loss
    for {patience} epochs)»)
58.         break

```

The training loop does the following:

- Sets up tracking variables to track the training progress. **EPOCHS** sets the total number of training cycles; **patience** controls how many times in a row validation loss is allowed to not improve before stopping early (early stopping); **best_val_loss**, **epochs_no_improve**, **train_losses**, and **val_losses** are all used for tracking performance and early stopping.
- The main training loop runs for each epoch, and we use the package **tqdm** (<https://tqdm.github.io/>) to create a progress bar for tracking the progress in the console.
- Model training mode is activated by using **model.train()** that puts the model in training mode, enabling entities like dropout (dropping neurons at random to reduce overfitting) and batch normalization (scaling and shifting the data so that each match has a mean of zero and a standard deviation of one. This helps prevent issues where some neurons produce outputs that are too small or too large, making it easier for the network to learn and often allowing the use of higher learning rates.
- Training over batches is performed using the **train_loader**, and for each batch, the training loop:
 - Moves data to the device **features_batch** and **targets_batch** are sent to the device (CPU or GPU) for faster computation.
 - Model predictions are made with **model(features_batch)**.
 - Loss is calculated as the difference between the actual and predicted values, and the loss is calculated using our custom MSE loss function.
 - Zeroing out gradients in PyTorch is done using **optimizer.zero_grad()**, which is important because, by default, PyTorch accumulates gradients each time **loss.backward()** is called. If you do not reset the gradients at the start of each training step, they will build up in memory,

leading to incorrect updates and potentially causing your model to learn incorrectly or even run out of memory.

- Backward pass using **loss.backward()** computes gradients of the loss with respect to the model parameters.
- Optimizer step updates the model parameters based on the computed gradients; we use **optimizer.step()** to action this update.
- Average training loss after all batches is calculated and stored.
- Model evaluation mode is switched on by using **model.eval()**, this turns off features like dropout that do not make sense in this context.
- Validation phase is where no learning happens; this is where the checks are done against the validation dataset to see how the model is doing on the unseen data. The components of this are:
 - No gradients. The code is wrapped with **torch.no_grad()**, telling PyTorch not to compute gradients (this saves memory and speeds up validation).
 - Validation loss is checked on batches from the **val_loader** without updating the model weights.
- Average validation loss is calculated for the epoch and stored per epoch.
- Early stopping is used if validation loss improves, the best model weights are saved, and the early stopping counter resets. However, if there is no improvement for a set number of epochs (patience), training stops early to prevent overfitting (this refers to when a model learns the training data so well that it fails to perform well on new, unseen data).
- Progress reporting is done with print statements to the console.

That is a lot to take in at once, but working through the code and referring back to these explanations will really help you understand. Do not worry if it does not click right away; feel free to revisit and run the code as many times as you need until the training loop starts to make sense.

Visualizing the training history

In the accompanying workbook, you will find a custom function that we will use to visualize the training process. Please refer to the notebook for details on how the function works. In short, it plots the training and validation losses for each epoch, helping us see how the model is progressing.

Signs that your model is learning include a steady decrease in both training and validation loss. Additionally, you might notice that the gap between these losses remains small and that the validation loss does not start increasing sharply, which would be a sign of overfitting. These trends in the training history give us valuable insight into how well our model is performing. To use the function on our train and validation loss values, we can achieve this with the following code:

```
1. plot_training_history(  
2.     train_losses, val_losses,  
3.     log_y_axis=True, figsize=(10,6),  
4.     train_args=dict(color='black',  
5.         linestyle='-',  
6.         linewidth=2),  
7.     valid_args=dict(color='grey',  
8.         linestyle='--',  
9.         linewidth=2))
```

Running this code will produce a visual like the one in *Figure 11.7* (to toggle the axes, which is set to a log scale to visualize the difference more clearly, you can toggle `log_y_axis=False`):

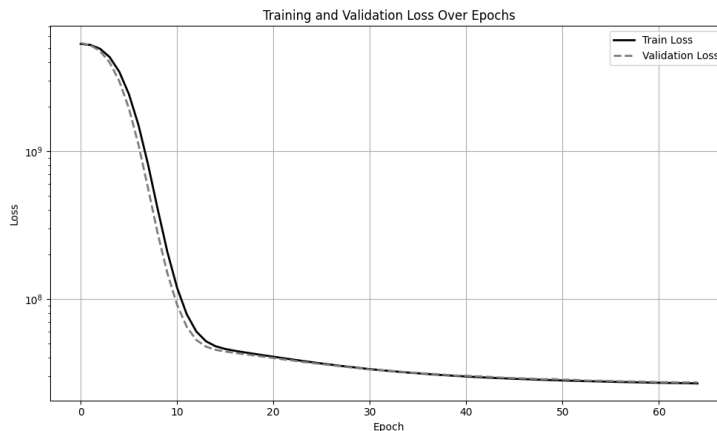


Figure 11.7: Training and validation loss over epochs

In the next section, we will show how to load your trained model. Since we saved the model's `state_dict()` during training, we will explain how to reload these saved parameters so you can evaluate your model's performance on the test dataset, providing a final check on how well it works with completely new data.

Evaluation of trained model on test dataset

In this section, we will walk through how to evaluate your trained model on new data. First, we will reinitialize the base MLP model and load the best saved weights (using `model.state_dict()`). Next, we will switch the model to evaluation mode and use RMSE as our main metric to assess its performance. Finally, we will use a diagnostic panel plot from `modelviz` to help visualize the results. The following steps will guide you through this process:

1. In this step, we will reinitialize the model:

```
model = MLP(input_size=X.shape[1],
            hidden_layers=[64, 32],
            output_size=1)
```

2. We will grab the best model weights from the training loop:

```
model.load_state_dict(torch.load('models/best_model.pth'))
```

3. In this step we will put the model back onto the device:

```
model.to(device)
```

4. As we need to perform evaluation on the model, we will turn on evaluation mode:

```
model.eval()
```

5. This step disables the gradient updating, makes predictions with the test set against the model, and calculates the RMSE (essentially the square root of MSE):

```
model.eval()
```

```
with torch.no_grad():
```

```
    preds = model(X_test_torch.to(device))
```

```
    rmse = torch.sqrt(loss_fn(preds, y_test_torch.to(device)))
```

6. To print the MSE, we use the following:

```
sprint("PyTorch MLP RMSE:", rmse.item())
```

7. To see how our predictions compare to the true values, we will use the **regression_diagnostics_panel** from **modelviz**. Before plotting, we will convert our tensors back to NumPy arrays by moving them to the CPU and flatten them. You will notice this process repeatedly in the code below to prepare the data for visualization:

```
from modelviz.regression import regression_diagnostics_panel
```

```
import numpy
```

```
regression_diagnostics_panel(
```

```
    y_test=y_test_torch.cpu().numpy().flatten(),
```

```
    y_pred=preds.cpu().detach().numpy().flatten(),
```

```
    qq_line_color='grey', qq_point_color='black')
```

8. Running the code in *Step 7*, the visualization in *Figure 11.8* will be generated:

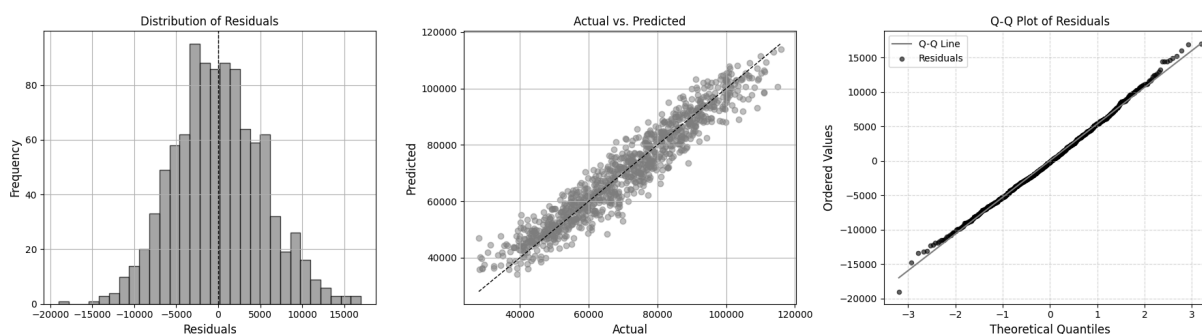


Figure 11.8: Regression diagnostic panel for PyTorch model

That wraps up our journey through building a regression model from scratch with PyTorch. You have now seen how much flexibility PyTorch offers, from designing your network architecture to customizing every step of the training process. In the next section, we will take on the same regression task using the Keras API in TensorFlow. You will notice that Keras provides a higher-level, more streamlined workflow, making the process quicker and simpler, but with less opportunity for customization compared to the hands-on approach you just completed in PyTorch.

Building a multi-layer perceptron with TensorFlow

To develop an MLP regressor using Keras (<https://keras.io/api/>), you can efficiently construct your model by employing the Sequential API. This approach allows for the straightforward stacking of layers while configuring the architecture, activation functions, and optimizer with minimal coding effort.

Keras's high-level design makes it especially easy to use and ideal for rapid prototyping, while still providing features like built-in callbacks and seamless integration with TensorFlow, though it offers less flexibility for customizing the training process compared to PyTorch.

In the next sections, we will walk through building and training a regression model in Keras. Note that the steps for scaling and splitting your data are the same as in the PyTorch example, so please review that section first to prepare your data, and then we will jump straight into creating and training the Keras model.

Define and compile the TensorFlow Keras model

To define a neural network model in TensorFlow Keras, you can use the Sequential API, which lets you build your model layer by layer in a clear and readable way. In the example ahead, we create an MLP for regression by stacking *dense (fully connected)* layers with ReLU activation functions and adding dropout layers to help prevent overfitting. The input layer matches the number of features in your training data, and the final layer outputs a single value for regression predictions. This straightforward approach makes it easy to design and experiment with different network architectures in Keras. The following code steps import the necessary packages, define the model, and compile the model with the relevant optimization functions:

1. Import all the packages we need for working with TensorFlow and the Keras API:

```
import TensorFlow as tf
```

```
from TensorFlow.keras.models import Sequential
```

```
from TensorFlow.keras.layers import Dense, Dropout
```

```
from TensorFlow.keras.callbacks import ModelCheckpoint, EarlyStopping
```

```
from TensorFlow.keras import Input
```

2. Create our model, with the relevant layers, in the Sequential API:

```
model = Sequential([
```

```
    Input(shape=(X_train_scaled.shape[1],)),
```

```
    Dense(64, activation='relu'),
```

```
    Dropout(0.3),
```

```
    Dense(32, activation='relu'),
```

```
    Dropout(0.2),
```

```
    Dense(1)])
```

3. Use MSE as the objective function and ADAM as the optimizer:

```
model.compile(  
    optimizer='adam',  
    loss='mean_squared_error',  
    metrics=['mean_squared_error'])
```

4. Early stopping has its own import in TensorFlow, and you can add a similar parameter for patience with validation loss:

```
early_stop = EarlyStopping(  
    monitor='val_loss',  
    patience=5,  
    restore_best_weights=True)
```

We have everything we need now, in terms of initialization. Now, we will add checkpointing and fit the model.

Add checkpointing and fit the model

To train deep learning models with Keras, it is essential not only to fit the model to your data but also to ensure that you can recover the best version of your model during training. This is where model checkpointing comes in, a powerful feature that allows you to save your model's weights at various stages, so you never lose progress and can always return to the optimal solution. To create a checkpoint and fit the model, we will follow these steps:

1. The **ModelCheckpoint** callback is set up with **filepath** specifying where to save the model, **monitor** indicating which validation metric to watch (**val_loss**), **save_best_only=True** ensuring only the best model is saved, **save_weights_only=False** to save the entire model (not just weights), and **verbose=1** to print messages when the model is saved. This is implemented here:

```
checkpoint_path = "models/keras_best_model.keras"
```

```
checkpoint_cb = ModelCheckpoint(  
    filepath=checkpoint_path,  
    monitor='val_loss',  
    save_best_only=True,  
    save_weights_only=False,  
    verbose=1)
```

2. In this step, the **model.fit()** function trains the model using the scaled training data (**X_train_scaled**, **y_train**), checks performance on the validation set after each epoch (**validation_data**), runs for up to 100 epochs (**epochs=100**), processes data in batches of 32 (**batch_size=32**), utilizes both early stopping and model checkpointing through callbacks (**callbacks=[early_stop, checkpoint_cb]**), and displays training progress in the output (**verbose=1**):

```
history = model.fit(  
    X_train_scaled, y_train,  
    validation_data=(X_val_scaled, y_val),  
    epochs=100,  
    batch_size=32,  
    callbacks=[early_stop, checkpoint_cb],  
    verbose=1)
```

Running *Step 2* will train the model for 100 epochs, and once finished, the model will be fitted and the best model saved to the model's directory. In the next section, we will evaluate our trained model.

Evaluation steps for Keras model

To evaluate a trained Keras regression model, we will first load the saved model from disk. Then, we will compute evaluation metrics such as MSE, RMSE, and others on the test data to quantify model performance. Finally,

we will generate a diagnostics panel plot to visualize prediction errors, residuals, and other key results, giving us insights into how well the model generalizes and where improvements can be made. These steps will be similar to those in the PyTorch section:

1. We need to load our best model checkpoint; we can achieve this as hereunder:

```
from TensorFlow.keras.models import load_model  
best_model = load_model("models/keras_best_model.keras")
```

2. In this step, we will make predictions against the model with our testing data:

```
y_pred = best_model.predict(X_test_scaled).flatten()
```

3. To obtain statistical evaluation metrics, such as RMSE and MSE, among others, we will use our **compute_metrics** function:

```
compute_metrics(y_test, y_pred)
```

4. As we did in the PyTorch section, we will also visually inspect the performance of our residuals, using the **modelviz** package:

```
from modelviz.regression import regression_diagnostics_panel  
regression_diagnostics_panel(  
    y_test=y_test,  
    y_pred=y_pred,  
    qq_line_color='grey', qq_point_color='black')
```

5. Upon running *Step 4*, the panel plot will be generated, as in *Figure 11.9*:

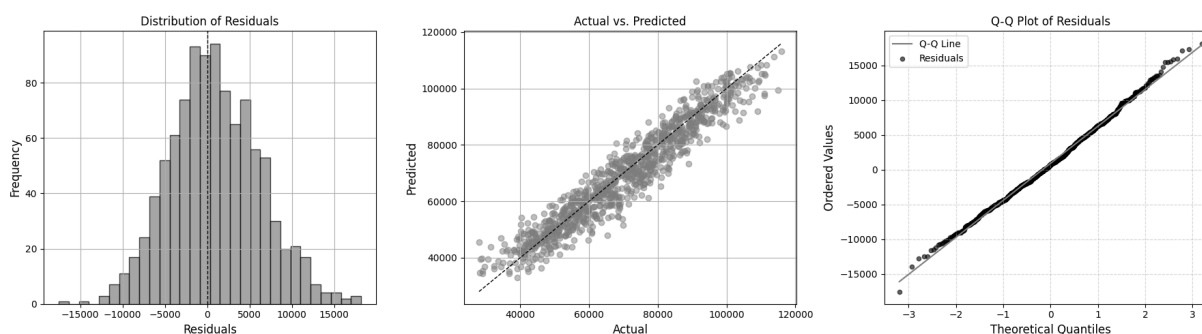


Figure 11.9: Regression diagnostic panel for TensorFlow model

The results are very similar across all three models, meaning that each model does a good job of fitting the data. The hyperparameter tuned model in scikit-learn is the best performer, but these models could easily be hyperparameter tuned to tweak the model parameters.

That brings us to the end of transitioning to other deep learning frameworks. There are many models you can create with both the PyTorch and Keras API libraries, and this is just scratching the surface of what is possible.

Conclusion

In this chapter, we have taken a hands-on journey through the foundations and practical techniques of building regression neural networks using scikit-learn, PyTorch, and TensorFlow. You have learned how to prepare and scale data, design and visualize MLP architectures, implement custom training loops, and leverage key tools like DataLoaders, checkpointing, early stopping, and model diagnostics to ensure robust model performance. By working across different frameworks, you now understand both high-level and customizable workflows, giving you a solid foundation for tackling real-world regression problems with modern neural networks.

In the next chapter, we will shift our focus from model building to deployment, introducing you to Flask and FastAPI for creating web APIs for your machine learning models. You will learn how to build RESTful endpoints, use Docker to containerize applications, and manage Docker images and containers. We will also cover how to document and test your

APIs using Swagger, culminating in an end-to-end example where you deploy a customer churn prediction model as a web service. Along the way, we will highlight best practices for deploying ML applications to production environments.

Points to remember

Here are some key points summed up from this chapter:

- **Activation functions are key:** Non-linear activation functions like ReLU, Sigmoid, and Tanh enable neural networks to model intricate relationships and make deep learning possible.
- **Checkpointing saves your best models:** Use model checkpointing and early stopping to automatically retain the best-performing weights and avoid overfitting.
- **Custom training loops offer flexibility:** Writing your own training loop in PyTorch allows granular control over loss computation, optimizer steps, and early stopping strategies.
- **Model diagnostics go beyond metrics:** Visual tools like residual plots and Q-Q plots help uncover hidden issues in your model's predictions that metrics alone can miss.
- **Neural networks learn features automatically:** MLPs can learn complex, non-linear patterns from raw data, reducing the need for manual feature engineering.
- **PyTorch DataLoaders streamline training:** DataLoaders manage batching, shuffling, and efficient data feeding, which are essential for scalable model training.
- **Scaling data matters:** Properly scaling your input features (using StandardScaler or similar) is crucial for stable and efficient neural network training.
- **TensorFlow Keras simplifies prototyping:** Keras's Sequential API enables rapid building, training, and tuning of neural networks.

with built-in callbacks for early stopping and checkpointing.

- **Transitioning frameworks broadens skills:** Practicing with both PyTorch and TensorFlow prepares you for the tools and workflows used in modern, production-ready deep learning.

Multiple choice questions

1. **What is the primary function of an activation function in a neural network?**
 - a. To initialize model weights
 - b. To introduce non-linearity into the network
 - c. To increase model interpretability
 - d. To scale input features
2. **Which of the following best describes the role of a DataLoader in PyTorch?**
 - a. To efficiently batch and shuffle data during training
 - b. To visualize model weights
 - c. To select optimal features
 - d. To tune hyperparameters
3. **What is the main advantage of using early stopping during neural network training?**
 - a. To skip the validation step
 - b. To automatically tune learning rates
 - c. To stop the model from over updating the loss function
 - d. To prevent overfitting by halting training when validation loss stops improving
4. **Why is it important to scale input features before training a neural network?**

- a. To make the model non-linear
 - b. To encode categorical variables
 - c. To ensure stable and efficient model evaluation
 - d. To ensure stable and efficient model training
5. **In an MLP, what is the function of the hidden layers?**
- a. To learn complex, non-linear representations of the input data
 - b. To reduce the number of model parameters
 - c. To merge outputs from parallel transformers
 - d. To convert all features to binary
6. **What is checkpointing used for in deep learning workflows?**
- a. To visualize loss curves
 - b. To randomly initialize weights
 - c. To scale input data
 - d. To save the model state during training for later recovery or deployment
7. **What could cause a neural network to underfit the training data?**
- a. Applying early stopping too late
 - b. Using a model that is too simple or not trained long enough
 - c. Scaling features incorrectly
 - d. Using too many hidden layers
8. **What does the loss function measure in a neural network?**
- a. How many parameters the model has lost
 - b. The speed of model convergence
 - c. How far the model's predictions are from the true values or labels

- d. Where the model has drifted
9. **What is a key reason for using dropout layers in neural networks?**
- a. To reduce overfitting by randomly dropping neurons during training
 - b. To perform feature scaling
 - c. To increase the learning rate
 - d. To convert output predictions to probabilities
10. **Which of the following is a key feature of Keras in TensorFlow for building neural networks?**
- a. It provides a low-level, manual API for defining and training neural networks
 - b. It provides a high-level, user-friendly API for defining, training, and evaluating neural networks
 - c. It provides a high-level, user-friendly API but only for unsupervised learning tasks
 - d. It provides a high-level, user-friendly API that cannot be used with TensorFlow models

Answers

1	b
2	a
3	d
4	d
5	a
6	d
7	b
8	c
9	a

Key terms

Some key terms used in this chapter are:

- **Activation function:** Adds non-linearity to neuron outputs, allowing networks to model complex patterns (e.g., ReLU, Sigmoid, Tanh).
- **Batch normalization:** Normalizes layer inputs to stabilize and speed up training.
- **Checkpointing:** Saving model and optimizer states during training to resume or recover the best model.
- **DataLoader (PyTorch):** Loads and shuffles data in batches for efficient PyTorch training.
- **Early stopping:** Stops training when validation performance stops improving, helping avoid overfitting.
- **Epoch:** One complete pass through the training dataset.
- **Fully connected (Dense) layer:** Every neuron connects to all neurons in adjacent layers.
- **Hyperparameter tuning:** Searching for the best model settings (e.g., learning rate, layers, batch size).
- **Learning rate:** Controls how much weights are updated during training.
- **Loss function:** Measures how far predictions are from true values (e.g., MSE).
- **Mini-batch:** Small subset of training data used in one update step.
- **MLP:** A basic neural network with input, hidden, and output layers.

- **Optimizer:** Algorithm (e.g., Adam, SGD) that updates network weights to reduce error.
- **Overfitting:** When a model learns training data (including noise) too well, performing poorly on new data.
- **Parameter:** Trainable values (weights, biases) updated during training.
- **ReLU:** Popular activation function; outputs zero for negatives, input for positives.
- **Tensor:** Multi-dimensional array for data in deep learning frameworks.
- **Underfitting:** Model is too simple to learn patterns, resulting in poor performance.
- **Validation set:** Data used to tune and monitor the model during training, not used for training.
- **Weight initialization:** How parameters are set before training, impacting learning efficiency.

Join our Discord space

Join our Discord workspace for latest updates, offers, tech happenings around the world, new releases, and sessions with the authors:

<https://discord.bpbonline.com>



CHAPTER 12

Deploying Your Machine Learning Application

Introduction

In this chapter, we will explore how to transform your ML models from isolated experiments into fully functional, production-ready services by deploying them as RESTful **application programming interfaces (APIs)**. As models progress beyond notebooks and scripts, the need to expose them reliably, often through **Hypertext Transfer Protocol (HTTP)** endpoints, becomes critical for real-world integration.

To achieve this, we will introduce two of the most widely used Python web frameworks for serving ML models, Flask and FastAPI. You will learn how to build and test endpoints for real-time and batch inference, validate inputs, log predictions, and generate interactive documentation.

Beyond basic deployment, we will also cover how to *containerize your API with Docker*, making it portable and scalable for use in cloud environments or microservice architectures. By the end of this chapter, you will not only understand how to expose your models as APIs, but also how to do this in the most secure and efficient way possible. This chapter will provide you with the practical skills and patterns needed to bridge the gap between model development and releasing your models as production-grade inference services.

Structure

This chapter covers the following topics:

- Introducing Flask and FastAPI for building APIs
- Creating RESTful endpoints for ML models
- Introduction to Docker
- Docker image creation, building, and running containers
- Best practices for deploying ML applications

Objectives

By the end of this chapter, you will be able to deploy ML models as RESTful APIs using both Flask and FastAPI, enabling real-time and batch inference workflows. You will understand how to validate inputs, structure prediction responses, and automatically generate interactive API documentation with Swagger UI and ReDoc. You will also gain hands-on experience containerizing your application using Docker for consistent and portable deployment. Finally, you will learn key best practices for production deployment, including monitoring, logging, model versioning, and updating models without downtime.

Introducing Flask and FastAPI for building APIs

As ML models move from experimentation (in Jupyter Notebooks or Python scripts) to production, one of the most common deployment patterns is exposing them as inference APIs. These are web endpoints that accept input data and return predictions. Python remains the dominant language in ML, and two of the most popular frameworks for serving models over HTTP are Flask and FastAPI.

Flask, the simple and reliable option

Flask (<https://flask.palletsprojects.com/en/stable/>) is a minimalist Python web framework that has long been a go-to choice for wrapping ML models in simple APIs. Its lightweight nature and low learning curve make it an attractive option for getting a model up and running quickly.

The key features, for model serving, are captured in the following list:

- Quick to set up with minimal boiler plating, which makes it ideal for prototyping.
- Easily integrates with any Python-based ML library (e.g., scikit-learn, TensorFlow, and PyTorch).
- Wide ecosystem for extensions like request parsing (extracts and formats incoming data), logging (records API activity for debugging and monitoring), and **cross-origin resource sharing (CORS)** (allows safe requests from web apps on different domains).
- Simple synchronous request handling (prevents the browser from executing additional code until the server responds), suitable for lightweight inference workloads.

The ideal case for Flask would be for wrapping traditional ML models or for internal tools, where latency and scalability are not critical concerns.

FastAPI, high-performance framework for production inference

FastAPI (<https://fastapi.tiangolo.com/>) is a modern Python web framework designed for high performance and developer productivity, especially useful for deploying ML models as APIs.

It is built on **Asynchronous Server Gateway Interface (ASGI)**, a modern alternative to **Web Server Gateway Interface (WSGI)**. While WSGI handles one request at a time, which can become a bottleneck under heavy traffic, ASGI supports asynchronous programming, allowing FastAPI to handle many requests at once without blocking. This makes it highly efficient for real-time or high-throughput inference scenarios.

FastAPI also leverages modern Python features like type annotations. Python is a dynamically typed language, meaning you do not have to specify variable types. However, with type annotations, you can add

optional *hints* about what type of data a function expects or returns (e.g., int, str, List[float]). FastAPI uses these hints, along with libraries like **Pydantic** (<https://docs.pydantic.dev/latest/>), to automatically validate inputs, serialize outputs, and generate clear, interactive documentation, all with minimal extra code.

Together, these features make FastAPI a powerful choice for creating fast, scalable, and well-documented APIs, making it especially well-suited for serving ML models in production environments.

Key features for model serving are as follows:

- **Asynchronous request handling:** Ideal for input/output bound tasks such as loading large models or calling external APIs.
- **Automatic input validation:** Using Pydantic reduces the chance of errors during inference.
- **Built-in Swagger UI and ReDoc:** Built-in Swagger UI (<https://swagger.io/>) and ReDoc (<https://redocly.com/docs/redoc>) enable auto-generating API docs that make testing and debugging easier.
- **High throughput and low latency:** Suitable for real-time (one request at a time) or batch inference (uploading data in batches) in production environments.

FastAPI is ideal when serving deep learning models, running real-time predictions, or deploying at scale behind load balancers or in containerized environments like Docker and Kubernetes.

When deploying ML models as APIs, Flask and FastAPI are both strong candidates. Flask offers a fast path to deployment for simple models and use cases, while FastAPI provides better performance, maintainability, and support for modern Python features, making it ideal for production-grade inference services. If you are experimenting or building proof-of-concept, Flask might be all you need, but if you are planning to scale, handle

concurrent requests, or serve latency-sensitive models, FastAPI is the clear winner.

Creating RESTful endpoints for ML models

A RESTful endpoint is a URL through which clients (like web apps or scripts) can interact with a server using standard HTTP methods such as GET, POST, PUT, and DELETE (these will be explained ahead). When serving an ML model, you typically expose an endpoint, like **/predict**, where a client sends input data [e.g., **Java Script Object Notation (JSON)**] and the server responds with a prediction.

REST stands for **Representational State Transfer**. In simple terms, a RESTful API organizes communication between systems around resources (like /predict) and actions (like sending a POST request). The main types of methods used in RESTful endpoints are explained hereunder:

- **GET** is used to retrieve data from the server. We will use this in our endpoint to get a health status.
- **POST** is the most used method for sending inference across HTTP. This will send the data to the server and create or process something. This is widely used on the internet for form submission when you fill in your credit card details on a website.
- **PUT** is used to update or replace an existing resource, such as updating a user's profile. This is used for managing model versioning or settings via an API.
- **DELETE** is used to delete a resource on the server, such as deleting an account, record, or file. This is seldom used for inference endpoints.

The common pattern for ML inference is to do the following:

- A POST request is made to an endpoint like **/predict**.
- The server processes the input, runs it through a trained model or scikit-learn pipeline, and returns the results as a JSON response.

In the following sections, we will create an endpoint in Flask and FastAPI, and later we will Dockerize these endpoints and make them microservices.

Creating a RESTful Flask endpoint

Now that you understand what RESTful endpoints are and why POST is typically used for model predictions, let us walk through building one in Flask. This section will show you how to structure your code, receive input data, run it through a trained model, and return the prediction.

In this section, we will use the model developed in *Chapter 10, Building Multi-step Pipelines*, along with its corresponding preprocessor and serialized (saved) model. If you have not read that chapter, do not worry. The supporting repository includes a **Model_Training** directory containing everything you need to retrain the model from scratch. In the steps that follow, we will set up all the necessary components to prepare for API integration.

Folder setup

When developing Flask and FastAPI applications, it is pivotal to separate these out into sub-directories, as these directories will be later used to build Docker images. Please follow these steps to set up the initial folder structure:

1. Create a master directory and name it **flask**.
2. Inside this master directory, you can copy the **models** and **pipeline** directories from the **trained_model** folder, as these are our serialized artefacts, which we will need for our main Flask app.
3. When you have done this, your folder structure inside the **flask** root folder should just have the models and pipeline folders.

In the next section, we will create the Flask application.

Creating the Flask application

At this point, we have the folder structure in place and are ready to create our **app.py** file inside the **flask** folder. It is not a Jupyter Notebook this time, as we need to define it as a runnable script from inside the Docker container later on. We will build the script file step by step:

1. We will define our imports (please make sure these are installed using **pip install flask flask_restx loguru**) to make our script work. Alongside **flask**, we will be using **flask_restx** (as this simplifies the Swagger documentation process massively):

```
import joblib
```

```
import numpy as np
```

```
import pandas as pd
```

```
from flask import Flask
```

```
from flask_restx import Api, Resource, fields
```

```
from loguru import logger
```

```
import os
```

2. It is good practice, when you are making **production_grade** applications, to have a logs directory to save our logs. We will be using **loguru** (<https://loguru.readthedocs.io/en/stable/>), as this is easier than the default logging inside the logging module. This makes a directory, and then we use **logger.add** to add a new config file and set the rotation to say overwrite the config when it gets 1 MB in size:

```
os.makedirs("logs", exist_ok=True)
```

```
logger.add("logs/api.log", rotation="1 MB")
```

3. Define the model path:

```
MODEL_PATH = "models/stack_class_pipe.joblib"
```

4. In the next step, we will define a try and exception block to try and load the model. If the model loads successfully, then the logger logs the success, and if an exception occurs, then it is raised:

```
try:
```

```
    model = joblib.load(MODEL_PATH)
```

```
    logger.success(f"Model loaded successfully from {MODEL_PATH}")
```

```
except Exception as e:
```

```
    logger.error(f"Failed to load model from {MODEL_PATH}: {e}")
```

```
    raise
```

5. The next step will be to define the app, which creates a Flask application instance, using the current module's name to set the app's configuration and resource paths:

```
app = Flask(__name__)
```

6. In this step, we use a **flask_restx** API object linked to the Flask app, this provides versioning, a title, and description for the API, and enables Swagger documentation at the **/docs** endpoint:

```
api = Api(app, version="1.0", title="Customer Churn Prediction API",
```

```
        description="Predict churn risk category for customer",  
        doc="/docs")
```

7. We now need a new namespace (endpoint), which will be our main **predict** method:

```
ns = api.namespace("predict",
```

```
        description="Churn prediction operations")
```

8. Here, we define what the data structure of the dataset and trained model was; these fields need to match the values and types the model expects, but we know these values are there in the data:

```
input_model = api.model("Input", {
```

```
    'CustomerID': fields.String(example='CUST00001'),
```

```

'Age': fields.Float(example=34.78),
'Gender': fields.String(enum=['Male', 'Female']),
'Tenure': fields.Float(example=22.46),
'MonthlyCharges': fields.Float(example=86.31),
'ServiceUsage': fields.Float(example=1.36),
'ContractType': fields.String(enum=['Month-to-Month',
                                     'One-Year', 'Two-Year']),
'PaymentMethod': fields.String(enum=['Credit Card', 'Bank Transfer',
                                     'Electronic Check', 'Mailed Check']),
'CustomerSupportCalls': fields.Float(example=0.0)})

```

9. We need to define the meaning of each model output, as the raw outputs, 0, 1, or 2, are just numerical labels without context. To make them understandable, we will create a dictionary that maps each integer to a meaningful category:

```
CHURN_RISK_MAP = {
```

```
    0: "Low Risk",
```

```
    1: "Medium Risk",
```

```
    2: "High Risk"}

```

10. In this step, we create a REST API endpoint ("/") using **flask_restx**, which will accept POST requests for predicting a customer's churn risk category. The **Predict** class, inheriting from **Resource**, defines this endpoint. When a request is received, it expects input data that matches the **input_model** schema. The data is extracted from the request payload and converted into a DataFrame, formatted for the model. The **CustomerID** is preserved for tracking, while the remaining features are used to generate a prediction and probability scores using a preloaded ML model.

The model's raw output, an integer representing a risk category, is then mapped to a human-readable label using the **CHURN_RISK_MAP** dictionary. The response includes the customer ID, the interpreted risk category, and the prediction probabilities for each class (low, medium, high). Logs are generated at each step for traceability, and if an error occurs during processing, a 400-error response is returned with the error message:

```
@ns.route("/")

class Predict(Resource):

    @ns.expect(input_model)

    def post(self):

        data = api.payload

        logger.info(f"Received prediction request: {data}")

        try:

            input_df = pd.DataFrame([

                'CustomerID': data['CustomerID'],

                'Age': data['Age'],

                'Gender': data['Gender'],

                'Tenure': data['Tenure'],

                'MonthlyCharges': data['MonthlyCharges'],

                'ServiceUsage': data['ServiceUsage'],

                'ContractType': data['ContractType'],

                'PaymentMethod': data['PaymentMethod'],

                'CustomerSupportCalls': data['CustomerSupportCalls'],

            ])

            input_X = input_df.drop(columns=['CustomerID'])
```

```

        logger.debug(f"Input DataFrame for prediction: {input_
X}")

        # Predict the churn risk category
        pred = model.predict(input_X)[0]
        pred_proba = model.predict_proba(input_X)[0]
        logger.info(f"Prediction result: {pred}, Probabilities: {pred_
proba}")

        #Create a response dictionary
        response = {
            "CustomerID": data['CustomerID'],
            "prediction": CHURN_RISK_MAP.get(pred, "Unknow
n"),
            "prediction_probs": {
                "Low Risk": float(pred_proba[0]),
                "Medium Risk": float(pred_proba[1]),
                "High Risk": float(pred_proba[2])}
        }

        logger.success(f"Prediction result for {data['CustomerI
D']}: {response}")

        return response, 200
    except Exception as e:
        logger.error(f"Error processing prediction request: {e}")
        return {"error": str(e)}, 400

```

11. For our API, we want to check the health of our endpoint, this is what this step does, as it sets up a **/health** GET endpoint using **flask_restx** to monitor the service's readiness and functionality. When called, it logs the request with **logger.info()** and first checks whether the ML model is loaded into memory. If the model is **None**, it logs an error and returns a 500 response indicating that the

service is unhealthy. To further ensure the model is functional, it optionally performs a dummy inference using a hardcoded DataFrame containing representative input data. This simulates a prediction to confirm the model can process inputs without errors. If the inference fails, an error is logged and a corresponding **500** error is returned. If all checks pass, the service responds with a 200 status and a message indicating the API is healthy:

```
@ns.route("/health")
```

```
class HealthCheck(Resource):
```

```
    def get(self):
```

```
        logger.info("Health check endpoint called")
```

```
        if model is None:
```

```
            logger.error("Model is not loaded!")
```

```
            return {"status": "unhealthy",  
                    "reason": "Model not loaded"}, 500
```

```
        try:
```

```
            dummy = pd.DataFrame([ {
```

```
                "Age": 30,
```

```
                "Gender": "Male",
```

```
                "Tenure": 5,
```

```
                "MonthlyCharges": 50,
```

```
                "ServiceUsage": 1,
```

```
                "ContractType": "Month-to-Month",
```

```
                "PaymentMethod": "Credit Card",
```

```
                "CustomerSupportCalls": 1,
```

```
            ]])
```

```
            _ = model.predict(dummy)
```

```
        except Exception as e:
```

```
logger.error(f"Model inference failed during health check: {e}")
```

```
return {"status": "unhealthy", "reason": "Inference failed"}, 500
```

```
logger.info("Health check passed")
```

```
return {"status": "healthy"}, 200
```

12. The final part of the script uses `if __name__ == "__main__":` to check if the file is being executed directly rather than imported as a module. If it is, the code inside this block runs. Specifically, it logs a message indicating that the server is starting, and then **`app.run(debug=True)`** launches the Flask development server. Running in debug mode enables automatic reloading when the code changes and displays detailed error messages in the browser, which is helpful during development. Once you are confident the API works correctly, it is recommended to turn off debug mode for production use.

We have everything we need now to run the application; we will cover the steps on how to do this in the next section.

Running and testing the Flask application

With all the routes, models, and configurations in place, the final step is to launch the Flask application so that it can start handling API requests. This is done using a standard Python entry-point check, which ensures the app only runs when the script is executed directly. Here is how it works:

1. Run the **`app.py`** script using either a command line interface call **`python app.py`** or in other code editors by hitting the play button.
2. After performing *Step 1*, the Flask server will spin into life, and on the console, you will see a message that says **Running on `http://127.0.0.1:5000`** (this means running on your localhost, local machine, and is running on port 5000).

3. Now we have a URL (**http://127.0.0.1:5000**) that we can use in any web browser, while the service is running locally. To test out the endpoint, we use the URL and type that into a web browser, with docs at the end, as so: **http://127.0.0.1:5000/docs**. This will bring up the Swagger endpoint, where we will see two HTTP methods, POST and GET, with the relevant endpoint routes (**/predict** and **/predict/health**).

Figure 12.1 shows our routes for the endpoint:

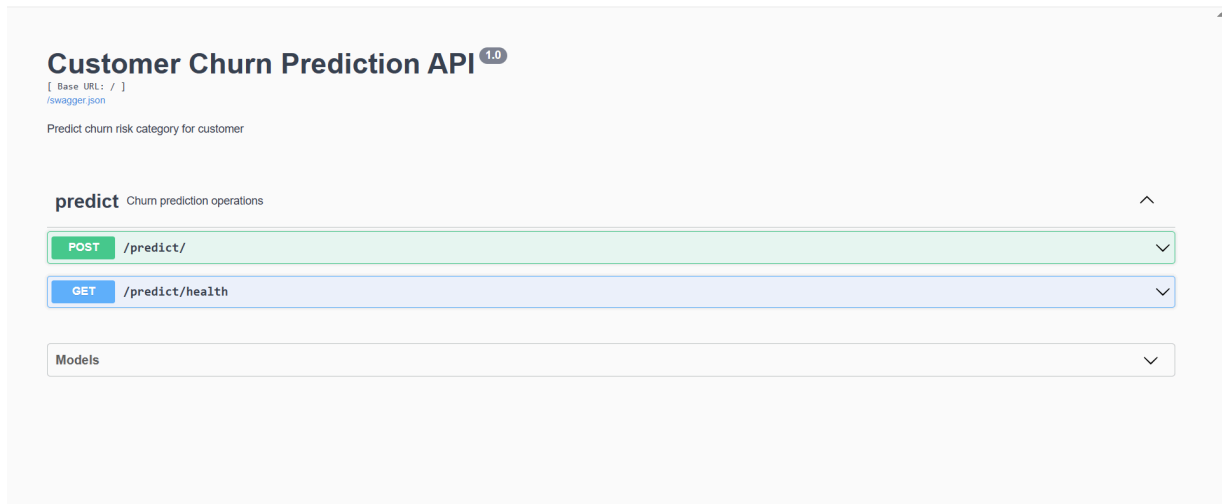


Figure 12.1: Customer Churn Swagger API

4. We will expand the POST method, and you should see an example of what a request payload looks like for the endpoint, the response content type (by default set to **application/json**), a response code, and a button entitled **Try it out**. The layout should look like *Figure 12.2* here:

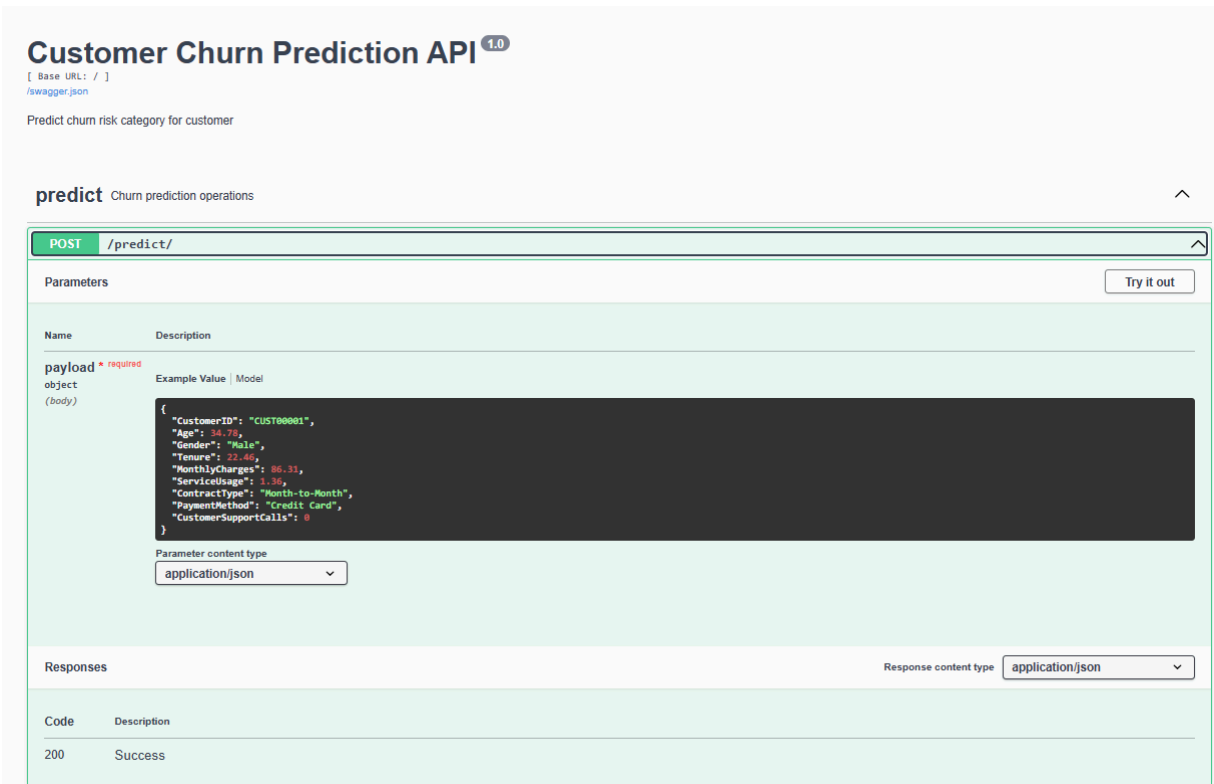


Figure 12.2: Predict route for Customer Churn endpoint

5. To try the endpoint, we use the **Try it out** button to edit the payload values in the sample JSON payload dictionary:

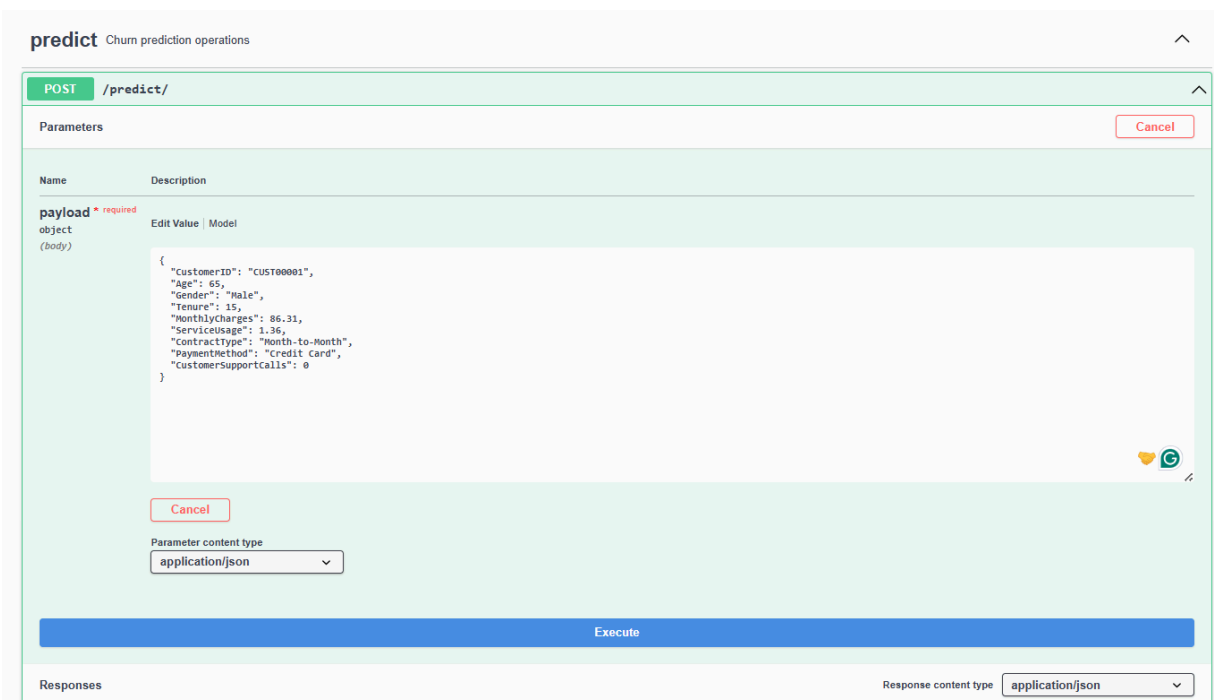


Figure 12.3: Testing the endpoint

6. Once we have edited the values, hit the **Execute** button to send a request to the server and get a response.
7. Now, in the **Responses** section, you will see the prediction from the model and the response dictionary structure we defined in the **app.py** file:

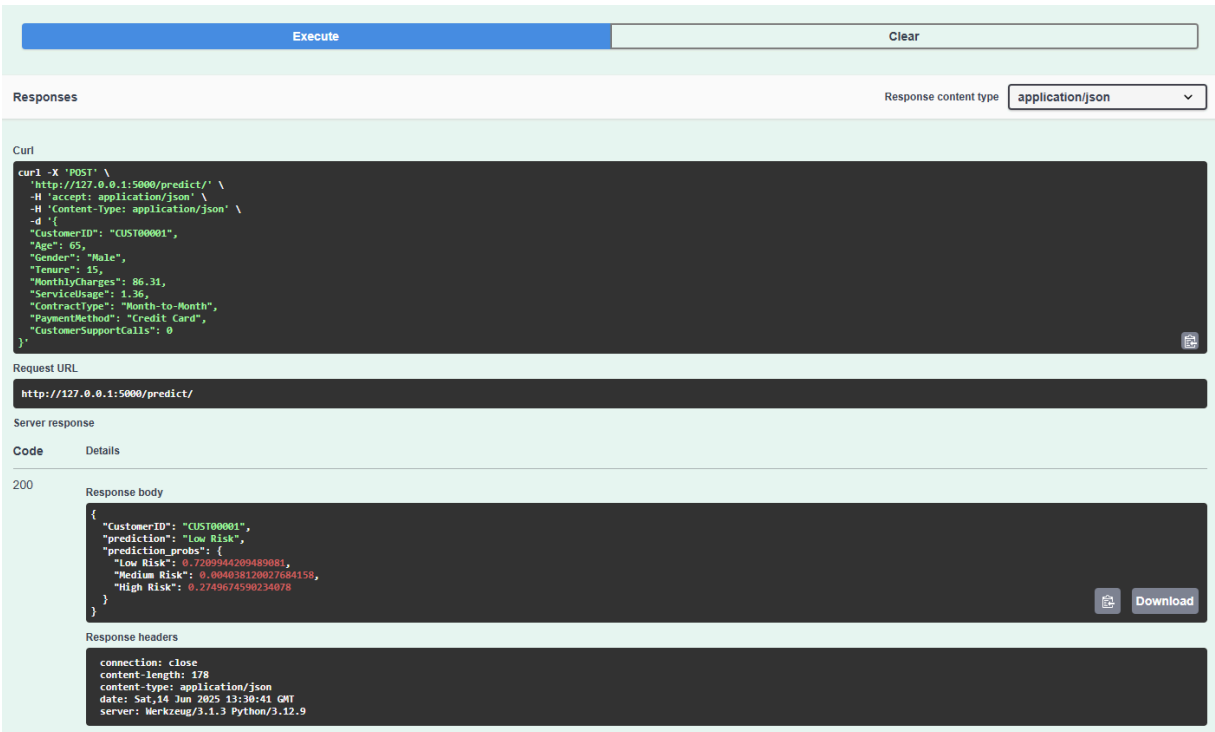


Figure 12.4: Responses from the endpoint

8. To understand the server's output, you will be shown an example using a CURL request, a command-line tool commonly used on Unix-based systems to send HTTP requests. This example will include the request URL, the server's response content, and the associated response code. However, since we are not currently using a Unix system, we will need to find an alternative method to send our prediction data to the API endpoint, such as using Python's requests library or a tool like Postman. We will cover how to do this in the next subsection to make an online prediction (one request per call).

In the next section, we will use the Python requests (<https://pypi.org/project/requests/>) library to send data to our API endpoint and receive predictions. Make sure your Flask server is still running before continuing. If it is not active, you will encounter a connection error, since everything is currently set up to run locally. Later in the chapter, we will explore how to containerize the application with Docker to make it a persistent and more portable service.

Making requests to the endpoint

This Flask-based API currently functions as an *online (real-time) prediction endpoint*, meaning that it handles one prediction request at a time. We will interact with it by sending individual prediction inputs using Python's requests package (this should be installed using **pip install requests**). Each request returns a response almost instantly, making it suitable for live use cases like dashboards or customer-facing tools.

In the FastAPI section later, we will enhance this design by structuring the endpoint to support batch predictions, allowing multiple records to be submitted in a single request. This approach is more efficient for processing larger datasets or running scheduled batch jobs.

To make a request via the requests package, we will follow the steps outlined as follows:

1. We will bring in our imports and define where the prediction route lives:

```
import requests
```

```
import json
```

```
url = http://localhost:5000/predict/
```

2. Define a request to send over to the endpoint and save this as a JSON file with the name of your choice (but ending in a **.json** file extension). In our case, we use a file named **sample_cust_input.json**:


```
{
  "CustomerID": "CUST00002",
  "Age": 65,
  "Gender": "Male",
  "Tenure": 22.46,
  "MonthlyCharges": 86.31,
  "ServiceUsage": 1.36,
  "ContractType": "Two-Year",
  "PaymentMethod": "Mailed Check",
  "CustomerSupportCalls": 0.0
}
```

3. We will open the file, or you could just send the raw JSON as the payload, using the following code:

```
with open('sample_cust_input.json', 'r') as f:
```

```
    payload = json.load(f)
```

4. To send the payload, we will use **requests.post** to make a POST request to the server:

```
response = requests.post(url,
                          json=payload)
```

5. Printing the response, you should see the response dictionary we defined in our Python app file: **{'CustomerID': 'CUST00002', 'prediction': 'Low Risk', 'prediction_probs': {'Low Risk': 0.9941382538143848, 'Medium Risk': 0.0012076553514319942, 'High Risk': 0.004654090834183183}}.**

In the next section, we will build a FastAPI endpoint that supports both real-time (online) predictions and batch requests. Unlike Flask with **flask_restx**, FastAPI automatically generates interactive Swagger

documentation without the need for separate YAML configuration files, making the setup much simpler and less time-consuming.

Creating a RESTful FastAPI endpoint

In this section, we will build a FastAPI endpoint that supports both real-time (online) predictions and batch requests. We will use the same serialized model based on the customer churn prediction challenge in *Chapter 10, Building Multi-Step Pipelines*.

Creating the FastAPI application

At this point, we have the folder structure in place and are ready to create our **app.py** file inside the **flask** folder as a Python script:

1. We will create an **app.py** file for our FastAPI service. Please look at the supporting GitHub repository for the imports and code structure. The difference between this and the Flask service is that we import the **fastapi** and **pydantic** packages.

2. The first step is to set up logging:

```
os.makedirs("logs", exist_ok=True)
logger.add("logs/fastapi_app.log",
           rotation="1 MB", retention="10 days")
```

3. Next, we specify the path to our serialized model and define a function to load the model:

```
MODEL_PATH = "models/stack_class_pipe.joblib"
def load_model(path=MODEL_PATH):
    """Load the ML model from disk."""
    try:
        model = joblib.load(path)
        logger.success(f"Model loaded successfully from {path}")
    return model
```

```
except Exception as e:
```

```
    logger.error(f"Error loading model from {path}: {e}")
```

```
    raise
```

4. We will load the model by just accessing the function we have just created in *Step 3*:

```
model = load_model()
```

5. Now we start to build the FastAPI app, by initializing the app:

```
app = FastAPI(
```

```
    title="FastAPI Churn Prediction API»,
```

```
    version="1.0.0",
```

```
    description="Predict customer churn risk. Supports batch, model versioning, and helpful info endpoints.")
```

6. In this step, we will use Pydantic objects for the first time. Pydantic allows you to define strict enumerated lists using Python's Enum, making your data models both cleaner and more reliable. This is particularly useful for fields with fixed, limited options, such as gender, contract types, or payment methods, as shown in the following step:

```
class GenderEnum(str, Enum):
```

```
    Male = 'Male'
```

```
    Female = 'Female'
```

```
class ContractTypeEnum(str, Enum):
```

```
    MonthToMonth = 'Month-to-Month'
```

```
    OneYear = 'One-Year'
```

```
    TwoYear = 'Two-Year'
```

```
class PaymentMethodEnum(str, Enum):
```

```
    CreditCard = 'Credit Card'
```

```
BankTransfer = 'Bank Transfer'
```

```
ElectronicCheck = 'Electronic Check'
```

```
MailedCheck = 'Mailed Check'
```

7. In this step, we define a **CustomerInput** schema using Pydantic's **BaseModel**, which enables automatic and reliable data validation. By specifying data types, using enumerated fields, and adding example values, we ensure that only well-structured and valid data is accepted, making the model robust, self-documenting, and safe for downstream processing:

```
class CustomerInput(BaseModel):
```

```
    CustomerID: str = Field(..., example="CUST00001")
```

```
    Age: float = Field(..., example=35)
```

```
    Gender: GenderEnum = Field(..., example='Male')
```

```
    Tenure: float = Field(..., example=22.46)
```

```
    MonthlyCharges: float = Field(..., example=86.31)
```

```
    ServiceUsage: float = Field(..., example=1.36)
```

```
    ContractType: ContractTypeEnum = Field(..., example='Month-to-Month')
```

```
    PaymentMethod: PaymentMethodEnum = Field(..., example='Credit Card')
```

```
    CustomerSupportCalls: float = Field(..., example=0.0)
```

8. For this step, we define a **BatchInput** Pydantic model that accepts a list of **CustomerInput** objects, allowing us to validate and process multiple customer records in a single request, ideal for bulk predictions or batch data handling:

```
class BatchInput(BaseModel):
```

```
    data: List[CustomerInput]
```

9. For this step, we define a target outcome mapper, as we did in the Flask example:

```
CHURN_RISK_MAP = {
```

```
    0: "Low Risk",
```

```
    1: "Medium Risk",
```

```
    2: "High Risk"}  
}
```

10. In this step, we define the **make_prediction** function, which takes a DataFrame of customer data, removes the **CustomerID** column, and passes the remaining features to a preloaded model to generate predictions. It then constructs a structured result for each customer, including their original ID, the predicted churn risk label (mapped from a class index), and the associated probability scores for each risk level. The function returns a list of these results, making it easy to interpret and display model outputs:

```
def make_prediction(input_df):
```

```
    input_X = input_df.drop(columns=['CustomerID'])
```

```
    preds = model.predict(input_X)
```

```
    pred_probs = model.predict_proba(input_X)
```

```
    results = []
```

```
    for i, row in input_df.iterrows():
```

```
        pred = preds[i]
```

```
        pred_proba = pred_probs[i]
```

```
        result = {
```

```
            "CustomerID": row['CustomerID'],
```

```
            "prediction": CHURN_RISK_MAP.get(pred, "Unknown"),
```

```
            "prediction_probs": {
```

```
                "Low Risk": float(pred_proba[0]),
```

```
                "Medium Risk": float(pred_proba[1]),
```

```

        "High Risk": float(pred_proba[2])
    }
}

results.append(result)

return results

```

11. As part of this step, we create a POST endpoint (namely **predict**) using the **@app.post("/predict", tags=["Prediction"])** decorator, which allows clients to submit a single customer's data for churn prediction (online prediction request); the function processes the input, generates a prediction with probability scores, logs the result, and returns it in a structured JSON response:

```

@app.post("/predict", tags=["Prediction"])
def predict_online(body: CustomerInput):
    """Predict churn risk for a single customer."""
    try:
        input_df = pd.DataFrame([body.dict()])
        logger.debug(f"Online prediction input: {input_df}")
        result = make_prediction(input_df)[0]
        logger.success(f"Prediction result for {body.CustomerID}: {result}")
        return result
    except Exception as e:
        logger.error(f"Error during prediction: {e}")
        raise HTTPException(status_code=400, detail=str(e)) from e

```

12. To handle multiple customer predictions at once, we define the **predict_batch** function, which accepts a **BatchInput** object containing a list of customer records. It converts the input into a DataFrame, logs the batch input, and passes it to the

make_prediction function to generate predictions for all customers. The results, including risk levels and probability scores, are returned as a list, while any errors are caught and returned as a **400 HTTP response** with detailed logging:

```
@app.post("/batch_predict", tags=["Prediction"])
```

```
def predict_batch(body: BatchInput):
```

```
    try:
```

```
        input_df = pd.DataFrame([item.dict() for item in body.data])
```

```
        logger.debug(f"Batch prediction input: {input_df}")
```

```
        results = make_prediction(input_df)
```

```
        logger.success(f"Batch prediction completed. {len(results)} results returned.»)
```

```
    return results
```

```
    except Exception as e:
```

```
        logger.error(f"Error during batch prediction: {e}")
```

```
        raise HTTPException(status_code=400, detail=str(e)) from e
```

13. To support dynamic model updates without restarting the service, we define a PUT endpoint at **/model** using the **@app.put("/model", tags=["Model Versioning"]) decorator**. This endpoint accepts an uploaded **.joblib** file containing a trained model and updates the in-memory model used for predictions. The function is marked as **async**, allowing it to handle the file upload operation asynchronously, meaning that the server can continue processing other requests while waiting for the file to be received. This improves performance and responsiveness, especially when handling large files or multiple simultaneous upload requests:

```
@app.put("/model", tags=["Model Versioning"])
```

```
async def update_model(file: UploadFile):
```

```
    global model
```

```

    logger.info(f"Received request to update model with file: {file.filename}")

    try:
        model_dir = os.path.dirname(MODEL_PATH)
        os.makedirs(model_dir, exist_ok=True)
        tmp_path = os.path.join(model_dir, "tmp_model.joblib")
        with open(tmp_path, "wb") as temp_buffer:
            shutil.copyfileobj(file.file, temp_buffer)
        logger.debug(f"Model file saved temporarily to {tmp_path}")
        ph_model = joblib.load(tmp_path)
        model = ph_model
        shutil.move(tmp_path, MODEL_PATH)
        logger.success(f"Model updated successfully to {MODEL_PATH}")

        return {"message": "Model updated successfully", "model_path": MODEL_PATH}

    except (OSError, IOError) as e:
        logger.error(f"File handling error: {e}")
        raise HTTPException(status_code=500,
                             detail=f"File handling error: {e}")

    except Exception as e:
        logger.error(f"Unexpected error updating model: {e}")
        raise HTTPException(status_code=500,
                             detail="Unexpected error during model update.")

```

This step implements the same health function we used in the Flask endpoint.

14. To provide visibility into the currently deployed model, we define a GET endpoint at **/model_info** using the

@app.get("/model_info", tags=["Model Versioning"]) decorator. This function retrieves the file modification time of the model from the filesystem and returns it along with the model's file path. This allows users or developers to check when the model was last updated, which is useful for version tracking, auditing, and ensuring the correct model is in use. If an error occurs while accessing the model file, a **500 HTTP response** is returned with a detailed error message:

```
@app.get("/model_info", tags=["Model Versioning"])
```

```
def model_info():
```

```
    try:
```

```
        mod_time = os.path.getmtime(MODEL_PATH)
```

```
        logger.info("Model info endpoint called")
```

```
    return {
```

```
        "model_path": MODEL_PATH,
```

```
        "last_updated": pd.to_datetime(mod_time, unit='s').
```

```
            isoformat() }
```

```
    except Exception as e:
```

```
        logger.error(f"Error getting model info: {e}")
```

```
    raise HTTPException(status_code=500,
```

```
                        detail=str(e))
```

15. To make the API more self-documenting and user-friendly, we define a GET endpoint at **/features** using the **@app.get("/features", tags=["Docs"])** decorator. This endpoint returns the input schema defined in the **CustomerInput** model by extracting its properties from the Pydantic schema. It provides a clear and structured list of the expected input fields and their data types for the prediction endpoint, helping developers understand

how to format their requests without needing to manually refer to external documentation:

```
@app.get("/features", tags=["Docs"])
```

```
def features():
```

```
    logger.info("Features endpoint called")
```

```
    return CustomerInput.schema()["properties"]
```

16. Monitoring is the next step, and to support basic monitoring and health checks, we define a GET endpoint at **/metrics** using the **@app.get("/metrics", tags=["Health"])** decorator. This function returns simple metadata about the currently deployed model file, including its size in bytes and the last time it was modified. It uses **os.stat()** to access the file's statistics and converts the modification time into a human-readable ISO timestamp. This information is useful for verifying that the model is loaded correctly, tracking updates, and integrating with external monitoring tools. If the file cannot be accessed, the function logs the error and returns a **500 HTTP response** with a relevant error message:

```
@app.get("/metrics", tags=["Health"])
```

```
def metrics():
```

```
    try:
```

```
        stat = os.stat(MODEL_PATH)
```

```
        logger.info("Metrics endpoint called")
```

```
        return {
```

```
            "model_file_size_bytes": stat.st_size,
```

```
            "last_updated": pd.to_datetime(stat.st_mtime, unit='s').isoformat()
```

```
        }
```

```
    except Exception as e:
```

```
        logger.error(f"Metrics endpoint failed: {e}")
```

```
raise HTTPException(status_code=500,  
                    detail="Could not retrieve metrics")
```

17. The final step is to create a message when the API is accessed via a web browser, this will give an introductory message on the root and give information about how to obtain the API docs for the endpoint:

```
@app.get("/", tags=["Docs"])  
  
def root():  
  
    return {  
        "message": "Welcome to the FastAPI Churn Prediction API!",  
        "docs": "/docs",  
        "redoc": "/redoc"}
```

In the following section, we will explore how to run the API using a serving framework called **Uvicorn**. Unlike Flask, FastAPI applications are not typically run directly from Python scripts and instead require an ASGI server like Uvicorn for proper deployment.

Running the FastAPI application with Uvicorn

To run the FastAPI application, you will use Uvicorn (<https://www.uvicorn.org/>), a lightweight and high-performance ASGI server that is ideal for serving asynchronous Python apps. Unlike Flask, FastAPI does not include a built-in server for local execution, so Uvicorn handles that role. You can start the app with a command like **uvicorn main:app --reload**, where main is the name of your Python file (without the **.py** extension) and app is the FastAPI instance.

Several optional parameters can enhance how the app runs. For example, **--reload** automatically restarts the server when code changes, great for development. The **--host 0.0.0.0** option makes the app accessible on your local network, and **--port 8000** specifies which port to use. For production, you might also include **--workers 4** to enable multiple processes for better

performance. We will use the following steps to run the FastAPI application:

1. Use the following **uvicorn** command to start our ASGI server:

```
uvicorn app:app --host 0.0.0.0 --port 8000 --reload
```

2. Upon running the app, enter the address **http://localhost:8000/** into your web browser, and you will be greeted with a message: **{"message":"Welcome to the FastAPI Churn Prediction API!","docs":"/docs","redoc":"/redoc"}**
3. To view the associated route and endpoint documentation, you can use the link: **http://localhost:8000/docs**, and this will bring up the associated Swagger documentation, as the following figure shows:

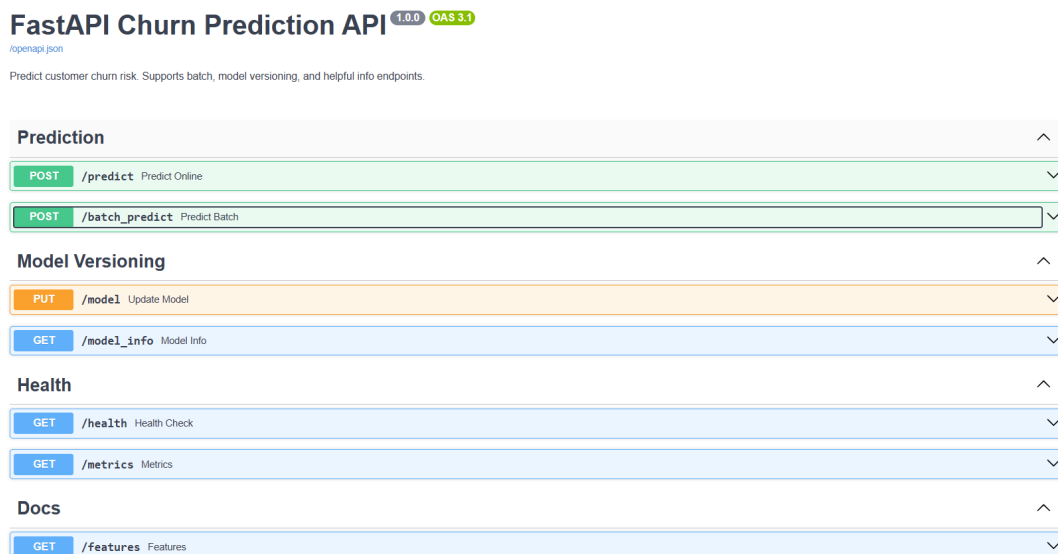


Figure 12.5: FastAPI Swagger Docs

4. One of the great features of FastAPI is that it automatically provides a full ReDoc-style documentation page, offering detailed information on how to use each endpoint and route. You can access it by visiting **http://localhost:8000/redoc**, where you will see an interactive example like the one shown as follows:

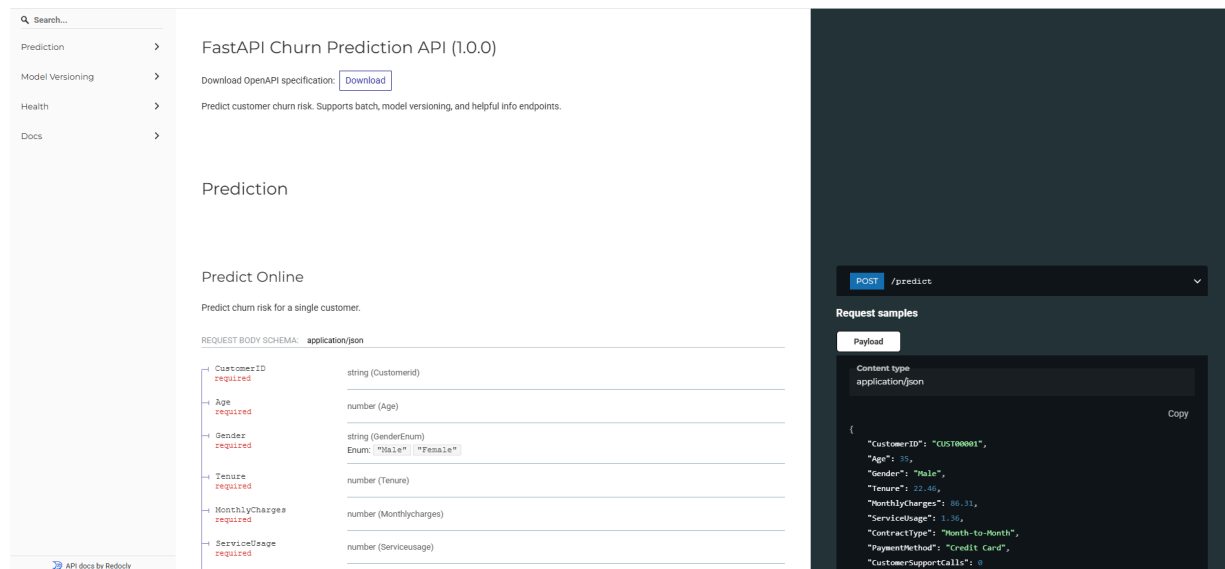


Figure 12.6: FastAPI ReDoc documentation

In the next section, we will explore the batch prediction feature of our endpoint by making multiple prediction requests at once, shifting our focus from the online (single) prediction used in the Flask example.

Using the FastAPI endpoint for batch prediction

FastAPI makes it easy to handle batch prediction requests efficiently and reliably. Instead of sending one prediction at a time, the batch prediction endpoint allows you to submit a list of customer records in a single request, streamlining large-scale inference tasks. This is especially useful when processing datasets in bulk, such as customer churn evaluations or risk assessments. By sending a well-structured JSON payload to the **/batch_predict** endpoint, you can receive predictions for multiple entries in one call, making the process faster and more scalable for real-world applications. We will do this in the following steps, using the Python requests library, as we did in the previous section:

1. To process our data in batches, we will use a sample **batch.json** file to send more than one request to our FastAPI Customer Churn endpoint:

```
{
```

```
"data": [  
  {  
    "CustomerID": "CUST00000",  
    "Age": 30,  
    "Gender": "Male",  
    "Tenure": 5.0,  
    "MonthlyCharges": 50.0,  
    "ServiceUsage": 1.0,  
    "ContractType": "Month-to-Month",  
    "PaymentMethod": "Credit Card",  
    "CustomerSupportCalls": 0.0  
  },  
  {  
    "CustomerID": "CUST00001",  
    "Age": 31,  
    "Gender": "Female",  
    "Tenure": 5.5,  
    "MonthlyCharges": 53.25,  
    "ServiceUsage": 1.1,  
    "ContractType": "One-Year",  
    "PaymentMethod": "Bank Transfer",  
    "CustomerSupportCalls": 1.0  
  }  
]
```

2. In this step, we will specify the endpoint to use and load the JSON payload to send over, with the **requests** library:

```
import requests
```

```
url = "http://localhost:8000/batch_predict"
```

```
with open("json/batch.json", "r") as f:
```

```
    data = json.load(f)
```

3. As part of this step, we will now send our bulk payload to our endpoint to return predictions:

```
response = requests.post(url, json=data)
```

4. To obtain the results as a JSON string, we will now use the following syntax:

```
print(response.json())
```

5. This will generate an output of all the responses you sent through to the model, and an example response will look like this below, encapsulated into a JSON dictionary:

```
{
```

```
    "CustomerID": "CUST00000",
```

```
    "prediction": "Low Risk",
```

```
    "prediction_probs": {
```

```
        "Low Risk": 0.8480752617672945,
```

```
        "Medium Risk": 0.006131916466936436,
```

```
        "High Risk": 0.14579282176576905
```

```
    }
```

In the next section, we will look at how to update the prediction model without needing to rebuild or redeploy the entire service. If the underlying churn prediction model has been retrained or improved, you can easily replace it by uploading the new version using a PUT request. This provides a seamless way to keep your predictions up to date while the FastAPI service remains running.

Update the model with a PUT request

In this section, we will explore how to update the ML model used in our FastAPI service through a PUT request. As models are retrained and improved over time, it is often necessary to deploy a new version without shutting down the API or redeploying the entire service. This is where the **/model** endpoint comes in.

Using PUT methods, we can upload a new **.joblib** model file directly to the running FastAPI application. When a new model file is sent to this endpoint, the server temporarily saves it, attempts to load and validate the model, and (if successful) replaces the currently loaded model in memory. This means all future predictions will immediately use the newly uploaded version, without requiring a restart.

This approach enables seamless model versioning and deployment, making it easy to keep predictions up to date with the latest training improvements, all while maintaining continuous uptime for the service. To do this with the **requests** library, we will follow these steps:

1. As we did in the previous section, import requests and specify the endpoint URL. In addition, we add the reference to the local file we want to pass to the model server:

```
import requests
```

```
url = "http://localhost:8000/model"
```

```
file_path = "models/stack_class_pipe_new.joblib"
```

2. To upload a new version of the ML model to the FastAPI service, we use a PUT request that includes the model file as part of the request body. The code opens the model file in binary read mode using **open(file_path, "rb")** as **f**, ensuring the file is properly managed and closed after use. It then constructs a files dictionary where the key **file** matches what the FastAPI endpoint expects, and the value is a tuple containing the filename, the open file object, and the **Multipurpose Internet Mail Extensions type (MIME type) application/octet-stream**. This MIME type tells the server

what kind of file is being sent, in this case, a generic binary file, which is appropriate for model files like **.joblib**. Finally, **requests.put(url, files=files)** sends the PUT request to the API endpoint, allowing the FastAPI application to receive, validate, and replace the currently loaded model with the new one. This enables dynamic model updates without restarting the server or redeploying the service:

```
with open(file_path, "rb") as f:
```

```
    files = {"file": ("stack_class_pipe_new.joblib", f,  
                    "application/octet-stream")}
```

```
    response = requests.put(url, files=files)
```

3. It is important to check the status code; we want to see a 200 code, meaning the request has been successful:

```
print("Status Code:", response.status_code)
```

4. To see the response JSON, this links to what we defined in the app.py file, as our main service file, you will see a message that says the model has been uploaded successfully. To view this yourself, use:

```
print("Response JSON:", response.json())
```

We have implemented several routes within this FastAPI application, each designed to handle specific tasks, such as single and batch predictions, model updates, health checks, and more. Now that the core functionality is in place, it is encouraged to explore the interactive ReDoc documentation at **/redoc**, where you can learn more about each endpoint and how to use them.

In the next section, we will take this service and containerize it using Docker. Running the API locally is fine for development, but the service shuts down as soon as the terminal is closed or the process is killed. By deploying it in Docker, we can ensure the service runs in a stable, consistent

environment with improved portability and persistence, allowing it to stay up and running reliably, even across restarts or system failures.

Introduction to Docker

One of the biggest challenges of deploying ML models is ensuring they run consistently across different environments, whether that be on your laptop, work machine, or on the cloud. This is especially important for **inference services** where your model is exposed via an API (like the FastAPI application we have been building) and needs to respond quickly and reliably to prediction requests.

This is where **Docker** becomes incredibly useful. Docker allows you to package your model, inference code, API server, and all dependencies (Python packages the model relies on to run) into a single, portable unit called a **container**. Once containerized, your model service can run the exact same way, locally, on a server, or inside a cloud platform, without worrying about library versions, system settings, or environment conflicts.

In the following list, we aim to break down why Docker is useful for model inference:

- **Reproducibility:** Your model will behave the same way in production as it did in development.
- **Fast deployment:** Start your API in seconds with a single command.
- **Self-contained:** Everything the model needs, Python, libraries, FastAPI, and the serialized model file, is all stored inside the container.
- **Easy updates:** Swap out the container when you retrain, improve the model, or adapt the underlying service or supporting scripts.
- **Scalability:** Containers work seamlessly with orchestration tools (like Kubernetes or Docker Compose) for scaling up.

Getting familiar with core Docker concepts

To understand Docker, we will go over the core components of Docker:

- **Images:** These act as a blueprint for your inference service. It includes the Python environment, your FastAPI app, your model file, and any necessary dependencies. You build this once and run containers from it anywhere.
- **Container:** This is a running instance of your image. It serves predictions by running the FastAPI app and loading the model into memory. If the container is stopped or restarted, a new one can be started instantly.
- **Volume:** These store data outside of the container. Since containers are temporary, a volume lets you preserve things like uploaded model files or logs even after restarting the service, e.g., you can mount a volume to store updated versions of your model when using the `/model` update endpoint.
- **Network:** Docker networks let containers communicate. This is helpful if your model server needs to talk to a database, a logging service (see MLFlow), or another microservice.

These concepts are visualized in the following figure:

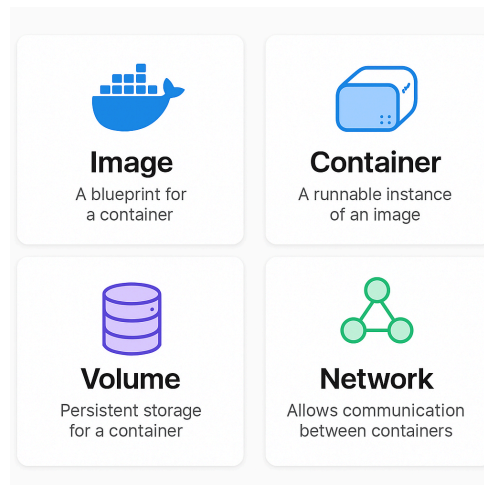


Figure 12.7: Docker core components

Docker is an ideal framework for serving ML models in production. It lets you package your model, API, and dependencies into a single, consistent unit that can be deployed anywhere with confidence. Whether you are exposing a model with FastAPI or updating it on the fly, Docker ensures your inference service is *portable, platform agnostic, repeatable, and always up*.

In the next section, we will build a Docker image and run a container for our Customer Churn Prediction API.

Docker image creation, building, and running containers

To run applications consistently across different environments, Docker offers a powerful solution through containerization. The process begins by writing a Dockerfile, which acts as a blueprint for creating a Docker image, a lightweight, standalone package that contains everything your application needs, including code, dependencies, and runtime configuration. Once the image is defined, it is built using the **docker build** command, producing a reusable artifact that can be shared or deployed anywhere. You can then run the application as a container using **docker run**, which launches an isolated instance of the image. This approach makes it easy to manage, deploy, and scale applications like APIs or ML models in a repeatable and efficient manner.

To get started, install Docker on your system by following the links provided hereunder:

- **Docker** **for** **macOS:**
<https://docs.docker.com/desktop/setup/install/mac-install/>
- **Docker** **for** **Windows:**
<https://docs.docker.com/desktop/setup/install/windows-install/>
- **Docker** **for** **Linux:**
<https://docs.docker.com/desktop/setup/install/linux/>

Once the specific version of Docker is installed on your machine, you can follow the next steps.

Dockerfile for Docker image creation

In this section, we will create a Dockerfile to containerize our Customer Churn Prediction API, allowing us to serve the model efficiently and consistently across different environments. The Dockerfile resides in the same directory as the application and contains a set of instructions that Docker uses to build the image. This image is a complete snapshot of the environment needed to run the API, including Python, dependencies, the FastAPI application, and the trained model. Alongside the Dockerfile, we will include a **.dockerignore** file to exclude unnecessary files or folders, such as logs, temporary files, or virtual environments, from being copied into the image. This helps keep the final image clean, efficient, and secure. Together, the Dockerfile and **.dockerignore** ensure that only the essential components are included in the build, resulting in a reliable and portable Docker image ready to run the model as an API. Let us get started with building this image, in the following steps:

1. Open the folder in the supporting GitHub repository for this chapter entitled **dockerized_fastapi**. This is where we are going to create our Dockerfile for creating the docker image.
2. Create a new file and call it Dockerfile with no file extension.
3. Import a base blueprint to use for the image; these blueprint images, called **base images**, can be found on Docker Hub (<https://hub.docker.com/>). For general Python applications, you might use images like **python:3.12-slim**. However, for deep learning projects using frameworks such as PyTorch, there are preconfigured base images available (e.g., **pytorch/pytorch**) that often include **Compute Unified Device Architecture (CUDA)** and **CUDA Deep Neural Network (cuDNN)** support for GPU acceleration. These images are maintained by the PyTorch team

and provide a convenient starting point for building GPU-enabled inference or training containers:

```
FROM python:3.12-slim
```

4. This line, in the Dockerfile, sets two environment variables to optimize how Python behaves inside the container. **PYTHONDONTWRITEBYTECODE=1** prevents the creation of **.pyc** bytecode files, which are normally used to speed up module loading but are not necessary in Docker since containers are often rebuilt frequently, saving disk space and avoiding clutter; **PYTHONUNBUFFERED=1** disables output buffering so that logs and print statements appear in real time, which is crucial for debugging and monitoring in containerized environments where delayed output can make troubleshooting much harder:

```
ENV PYTHONDONTWRITEBYTECODE=1 \  
    PYTHONUNBUFFERED=1
```

5. This step creates a **/app** directory and switches to that working directory inside the container:

```
WORKDIR /app
```

6. Since Docker images are typically built on a lightweight Unix-based operating system (like Debian or Alpine), we use standard Linux package management commands to maintain and configure the environment. This line updates the package list, upgrades existing system packages and **install gcc**, a system compiler sometimes required to build Python packages with C extensions. The use of **--no-install-recommends** ensures we only install what is strictly necessary, keeping the image lean. Finally, **apt-get clean** and **rm -rf /var/lib/apt/lists/*** remove temporary files to reduce the image size and keep the container clean:

```
RUN apt-get update && \  
    apt-get upgrade -y && \  
    rm -rf /var/lib/apt/lists/*
```

```
apt-get install -y --no-install-recommends gcc && \  
apt-get clean && \  
rm -rf /var/lib/apt/lists/*
```

7. This step copies all files and folders from your project's current directory on your local machine (the build context) into the **/app** directory inside the Docker container. It ensures that your FastAPI code, model files, and supporting resources are available in the container's working environment. This step is essential for making the application runnable inside the container and works in conjunction with the earlier **WORKDIR /app** command, which sets **/app** as the base directory for everything that follows:

```
COPY . /app
```

8. In this step, we install all our Python packages needed to run the application, listed in the **requirements.txt** file that has been copied over in *Step 7* to the container. The **--no-cache-dir** flag tells pip not to store downloaded package files in its cache, which helps keep the Docker image smaller. Basically, make sure we have all the packages we need to run our service:

```
RUN pip install --no-cache-dir -r requirements.txt
```

9. We are using **loguru** to manage our application logging. We will make sure a directory exists inside our container to enable the storage of these logs:

```
RUN mkdir -p /app/logs
```

10. The following instruction tells Docker that the container will listen for network traffic on port 8000, which is where the FastAPI application will be served internally:

```
EXPOSE 8000
```

11. The **CMD** instruction, in the following code, in a **Dockerfile** specifies the default command that runs when the container starts;

in this case, it tells Docker to launch the FastAPI application using **uvicorn** by running the app instance from the **app.py** file, listening to all network interfaces (0.0.0.0) at port 8000, so the API is accessible when the container is running:

```
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

12. The full Dockerfile is contained here:

```
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1
WORKDIR /app
RUN apt-get update && \
    apt-get upgrade -y && \
    apt-get install -y --no-install-recommends gcc && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*
COPY . /app
RUN pip install --no-cache-dir -r requirements.txt
RUN mkdir -p /app/logs
EXPOSE 8000
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

In the next step, we will build the Dockerfile with Docker commands in the Docker **command line interface (CLI)**.

Building the Docker container

At this point, we have created a Dockerfile that defines how to package our Customer Churn Prediction API. The next step is to build a Docker container using the Docker CLI, which transforms our project files, along with all necessary dependencies, environment variables, and model assets,

into a portable image that can run anywhere. This image creates a self-contained environment that reflects the exact setup defined in the Dockerfile, ensuring consistent behavior across development, testing, and production. Once the image is built, we can run it locally to serve the API, deploy it to cloud platforms like GCP, AWS, or Azure, or even integrate it into a Kubernetes cluster for scalable container orchestration.

To build the container, we will use the following command:

1. `docker build -t churn-predictor-api:latest .`

This command does the following:

- **docker** is the Docker command line instruction that lets you interact with the Docker engine to build, run, and manage containers.
- **build** tells Docker you want to build a new image based on the instructions of the Dockerfile.
- **-t churn-predictor-api:latest** the **-t** flag lets you tag your image with a name and optional version label, here we chose **churn-predictor-api** as the name and **latest** as the version tag.
- **.** (dot) refers to the current directory as the build context. Docker will look here for the Dockerfile and any files it needs to be copied into the image.

Once this command is run, you will see the Docker image creation progress, with progress bars, start. Docker is very efficient, as images are built in layers, so when you make changes to your application, Docker only rebuilds the layers affected by those changes, allowing faster rebuild by reusing cached steps from previous builds.

Running the Docker container

Once you have built a Docker image for your application, the next step is to run it as a container. A container is a live, isolated instance of your image, essentially, a lightweight, self-contained environment that runs your

application exactly as it was defined. Running your app inside a container ensures it behaves the same way on any machine, regardless of the operating system.

One major benefit of using containers is their persistence and stability. While the container itself can be stopped or restarted, its environment remains unchanged unless you deliberately update it. This makes it easy to keep services running reliably in production, restart them automatically if they fail, or scale them up without worrying about inconsistencies.

To run our container, we use the following command:

1. `docker run -p 8000:8000 churn-predictor-api:latest`

This command does the following:

- **docker run** starts a new container based on an image
- **-p 8000:8000** maps the port 8000 on your local machine (host) to port 8000 inside the container (where FastAPI is running) making the API accessible at `http://localhost:8000` and the docs viewable at `http://localhost:8000/docs`.
- **churn-predictor-api:latest** specifies the image to run, using the name (**churn-predictor-api**) and tag (**latest**) you assigned during the build.

Once this command is executed, your API will be accessible just like it was locally, except now it is running inside a Docker container. Docker offers a wide range of commands; you can find the full list in the official CLI reference: <https://docs.docker.com/reference/cli/docker/>.

Best practices for deploying ML applications

Deploying ML applications to production requires more than just wrapping a model in an API. To ensure reliability, scalability, and maintainability, consider the following best practices:

- **Containerization:** Use Docker to package your application and its dependencies. This ensures consistent environments across development, staging, and production. Docker versioning also allows you to keep track of changes and enable rollbacks, when and where necessary.
- **Model versioning:** Use tools like **MLFlow** (<https://mlflow.org/>) for model version tracking and metrics out of your ML models. Storing models in a central registry or model hub is also considered best practice.
- **Monitoring and logging:** To continuously monitor model performance (e.g., accuracy, latency, model drift) and log key metrics and system events for observability and debugging.
- **Continuous integration and deployment (CI/CD) for ML:** Automate the ML workflow, including testing, training, and deployment, using CI/CD pipelines for faster and safer releases.
- **Security and access control:** Secure your APIs with authentication tokens, encryption, and role-based access to protect data and model endpoints.
- **Scalability:** Use orchestration platforms like **Kubernetes** (<https://kubernetes.io/>) or cloud ML services to scale your application based on traffic and resource demands.
- **Data validation:** Validates incoming data to ensure it meets expected formats and distributions, reducing the risk of feeding bad inputs into the model.
- **Retraining strategy:** Have a plan for regular or triggered retraining of models using new data to maintain performance as real-world conditions evolve.
- **Failover and rollback:** Implement rollback mechanisms and use canary or staged deployments to minimize risk during model updates.

- **Documentation and communication:** Provide clear documentation of your model's API, expected **inputs/outputs**, and known limitations to support transparency and collaboration. FastAPI makes this easy with its auto-generated documentation routes.
- **Load testing:** Stress test your application to evaluate performance and stability under real-world and peak load conditions. This will inform your scaling strategy.

With these best practices in mind, and drawing on the insights, techniques, and real-world lessons covered throughout this book, you are now well-equipped to take your ML applications beyond the experimentation phase and confidently deploy them into production environments. Whether you are optimizing for scalability, reliability, or continuous delivery, you now have the foundational tools and understanding to bridge the gap between the lab and real-world impact.

Conclusion

In this chapter, we covered how to transition machine models from development to deployment by building RESTful APIs using Flask and FastAPI. You learned how to design real-time and batch inference endpoints, validate inputs with schema definitions, and generate interactive documentation to support testing and debugging. We demonstrated how to containerize your application using Docker for consistent, scalable deployments across environments. Finally, we explored best practices for model production readiness, including monitoring, model versioning, and seamless updating, equipping you with the foundational tools to serve your models reliably and effectively in real-world applications.

In the next and final chapter, we will move beyond deployment to explore where ML is headed and the ethical responsibilities that come with it. We will introduce key innovations such as **large language models (LLMs)**, **automated machine learning (AutoML)**, federated learning and edge computing for on-device inference. This will then move on to how to design

AI systems responsibly, looking at ethical topics including bias, privacy, and AI governance frameworks to help you navigate the growing need for transparency, fairness, and accountability in modern ML systems.

Points to remember

Here are some key points summed up from this chapter:

- **APIs enable model access:** Serving your ML model via a RESTful API allows other systems to send data and receive predictions in real time for production readiness.
- **Flask is lightweight and effective:** Flask is ideal for rapid prototyping and lightweight inference tasks, offering minimal setup and integration with any Python-based ML library.
- **FastAPI is built for scale:** FastAPI supports asynchronous processing, automatic input validation, and auto-generated documentation, making it an ideal choice for a high-performance, scalable serving framework.
- **Swagger and ReDoc increase ease of documentation:** FastAPI and Flask-RESTx can generate interactive documentation that make debugging and testing your endpoints much easier.
- **Docker ensures portability:** Containerizing your ML API using Docker guarantees that your service runs consistently across different machines and environments.
- **Logging enables observability:** Structured logging with tools like loguru helps trace requests, monitor errors and understand model behavior in production.
- **Deploying is more than wrapping:** Production deployment involves more than just exposing a model; it includes validation, monitoring, scaling, and future-proofing your system.

Multiple choice questions

1. **What is the main purpose of exposing an ML model through a RESTful API?**
 - a. To improve model accuracy
 - b. To reduce the model's training time
 - c. To allow internal SSH calls to your HTTP endpoint
 - d. To allow external systems to send data and receive predictions over HTTP
2. **Why is FastAPI considered suitable for production ML inference?**
 - a. It supports asynchronous processing and built-in data validation
 - b. It bundles models into static files for deployment
 - c. It only supports synchronous routes improving speed
 - d. It requires external tools to handle JSON serialization
3. **What is the primary benefit of using Docker for model deployment?**
 - a. It replaces the need for a model server entirely
 - b. It packages the model, code and dependencies into a consistent environment
 - c. It compiles Python scripts into optimized binaries
 - d. It boosts model accuracy during runtime
4. **Why might you choose Flask for building a model-serving API?**
 - a. It supports asynchronous inference by default
 - b. It auto-generates documentation for batch prediction
 - c. It is simple to set up and works well for lightweight or internal production use cases

- d. It enforces strict type checking using Pydantic
5. **What is the purpose of Swagger UI in deploying a ML service?**
- a. To provide interactive, auto-generated API documentation for testing endpoints
 - b. To log client-side usage for analytics
 - c. To encrypt data sent to the model endpoint
 - d. To store API logs in a visual interface
6. **Why include a health check endpoint in your model API?**
- a. To log client-side usage for analytics
 - b. To confirm the API and model are responsive and function correctly
 - c. To automatically update the model's training data
 - d. To serve static front-end assets for monitoring
7. **What is the value of model versioning in production?**
- a. It reduces latency by caching predictions
 - b. It helps encrypt the model for secure storage
 - c. It allows updates to the model while preserving rollback options
 - d. It embeds model logic into the client application
8. **Why is input validation important in a prediction API?**
- a. It replaces missing values with model predictions
 - b. It helps optimize the API routing logic
 - c. It generates synthetic data if inputs are valid
 - d. It ensures incoming requests match the expected format and data types

9. **What is the benefit of batch prediction in a model-serving context?**
- a. It allows sending multiple inputs at once
 - b. It increases memory usage to improve model accuracy
 - c. It speeds up training by grouping features
 - d. It reduces model interpretability for security
10. **What is a key feature of container orchestration tools like Kubernetes?**
- a. They compile Python code into optimized C++ binaries
 - b. They visualize model training in real time
 - c. They manage, scale, and automate containerized application deployment
 - d. They handle cross-validation during deployment

Answers

1	d
2	a
3	b
4	c
5	a
6	b
7	c
8	d
9	a
10	c

Key terms

Some key terms used in this chapter are:

- **API:** A set of rules that allow software components to communicate with each other. In ML deployment, APIs are used to expose models so other applications can send input data and receive predictions.
- **Batch inference:** The process of sending multiple prediction inputs to a model in a single API call, improving efficiency when processing large datasets.
- **Container:** A lightweight, standalone unit that packages an application, its dependencies, and environment configuration. Containers ensure consistency across development, testing, and production environments.
- **Docker:** A platform for containerizing applications, enabling consistent and portable deployment of ML models across different systems and infrastructures.
- **Dockerfile:** A script containing instructions used by Docker to build a container image, specifying the base image, dependencies, file structure, and commands to run the service.
- **Endpoint:** A specific URL path in an API where clients can send HTTP requests (e.g., **/predict**, **/health**) to interact with the model or service.
- **FastAPI:** A high-performance Python web framework for building APIs with automatic data validation, type checking, and asynchronous support, ideal for production-grade ML inference.
- **Flask:** A lightweight Python web framework commonly used for quickly wrapping ML models in simple APIs, especially during prototyping or for low-traffic applications.
- **Gunicorn (<https://gunicorn.org/>):** A Python WSGI HTTP server commonly used to serve Flask applications in production. It efficiently manages multiple worker processes to handle

concurrent requests, making it a more robust alternative to Flask's built-in development server.

- **Health check endpoint:** A special API route (often `/health`) that returns the operational status of the model and service, used to monitor the readiness and stability of a deployed system.
- **Inference:** The act of using a trained ML model to make predictions on new, unseen data.
- **Input validation:** The process of checking incoming request data to ensure it meets expected structure and types before passing it to the model, reducing the risk of errors and misuse.
- **Microservice:** A small, self-contained service that performs a specific function and can be deployed independently, common in production ML systems for modularity and scalability.
- **Model versioning:** The practice of managing different versions of a trained ML model to enable controlled updates, rollbacks, and reproducibility in production.
- **Online prediction:** A real-time inference approach where the model serves one prediction at a time, typically in response to a single API request- ideal for interactive applications and dashboards.
- **Pydantic:** A Python library used in FastAPI to enforce data types and validation rules on incoming API requests using Python type hints and schemas.
- **ReDoc:** A documentation generator provided by FastAPI that renders detailed, readable API documentation from OpenAPI specifications alongside Swagger UI.
- **RESTful API:** A type of API that follows REST principles, using standard HTTP methods (GET, POST, PUT, DELETE) to interact with server-side resources.

- **Swagger UI:** An interactive interface automatically generated by FastAPI or Flask-RESTx that allows users to test and understand API routes and request/response formats.
- **Serialization:** The process of converting a trained model or object (e.g., a scikit-learn pipeline) into a format that can be saved to disk and later reloaded for inference (e.g., using joblib).
- **Uvicorn:** A lightning-fast ASGI server used to run FastAPI applications, enabling asynchronous communication and scalable deployment of Python web apps.

Join our Discord space

Join our Discord workspace for latest updates, offers, tech happenings around the world, new releases, and sessions with the authors:

<https://discord.bpbonline.com>



CHAPTER 13

Machine Learning Future Trends and Ethical Considerations

Introduction

In this chapter, we will introduce the core concepts behind LLMs, including their architecture, evolution, and real-world applications. You will learn how to work with LLMs using APIs like OpenAI and open-source tools such as HuggingFace, as well as how AutoML and **federated learning (FL)** streamline and secure the modeling process. We will also cover practical approaches to building responsible AI systems, including bias detection, privacy preservation, and model governance.

Structure

This chapter covers the following topics:

- Introduction to large language models
- Automated machine learning
- Federated learning
- Ethical frameworks, bias, privacy, and AI governance

Objectives

By the end of this chapter, you will be able to explain how LLMs work and describe their evolution from early sequence models to modern transformer-based architectures. You will gain practical experience using both commercial APIs and open-source tools for text generation, summarization,

and translation, as well as using LLMs to program with code completion. You will also be able to apply AutoML and FL to build and optimize models efficiently and securely. Finally, you will understand how to identify and address bias, protect data privacy, and document the modeling process to support ethical and responsible AI development.

Introduction to large language models

LLMs are normally transformer models trained on enormous amounts of text data (using advanced hardware) to generate, summarize, translate, and understand natural human language. They can produce coherent and contextually relevant text across a wide range of topics, as well as be good at coding themselves.

To understand LLMs, it helps to briefly know their roots. Before 2017, **natural language processing (NLP)** was dominated by models like **recurrent neural networks (RNNs)** and **long short-term memory networks (LSTMs)**. While these models could handle sequences to some extent, they struggled with capturing the meaning across long spans or sequences of text. For example, if you were processing a paragraph or multiple sentences, an LSTM might forget important information from earlier in the input by the time it reached the end of the generation, and these models were sequential (good for time series data), which made them slower and hard to scale. Everything changed when *Vaswani et al* introduced the transformer architecture in their landmark paper *Attention Is All You Need* (2017).

The transformer uses a mechanism called self-attention, which allows it to weigh the importance of each word in a sequence (whether it is a sentence, paragraph, or entire document) regardless of its position. Multiple attention heads operate in parallel, each focusing on different parts of the input, enabling the model to capture nuanced relationships such as syntax, semantics, and long-range dependencies. For a deep dive into this framework, read the original paper: <https://arxiv.org/abs/1706.03762>

Key milestones:

- **2018 (BERT): Bidirectional Encoder Representations from Transformers (BERT)** was Google's model for understanding language; these are now known as small to medium language models.
- **2018 (GPT): Generative Pretrained Transformers (GPT)** was released by *OpenAI* for generating text. It uses a decoder-only architecture that is unidirectional, meaning it processes text from left to right. Each word is predicted based only on the words that come before it.
- **2019 to 2023 (Scaling and diversification):** The Transformer architecture rapidly scaled up and specialized:
 - **GPT2/GPT3** grew the generative approach (now known as generative AI) to hundreds of billions of model parameters (think of tuning all of those).
 - **Google's T5** model uses a format where everything, translation, summarizing, and more, is treated as a text-in, text-out task. For example, you give it English and ask for German, or feed it a long article and get a short summary back.
 - **PALM** (2022) pushed scale further with a dense 540 billion parameter model, advancing capabilities like few-shot reasoning, bridging advanced capabilities like few-shot reasoning and code generation (the age of the coding LLM).
- **March 2023 (GPT4):** GPT-4 became the flagship model for multimodal capabilities, processing text and images, and offering longer context windows (up to 32 k tokens).
- **April 2025 (Meta's LLaMA 4):** LLaMA 4 furthered this trend with **mixture of experts (MoE)** architecture (i.e., a *sparse* model design that activates only a small number of specialized subnetworks, or *experts*, for each input, enabling high capacity with efficient compute), multilingual support, and million-token context.

- **Present scenario:** As of July 2025, modern LLMs can:
 - Process and generate *text, images*, and other *modalities* (audio, video, etc.).
 - Recall and act on *thousands or millions* of tokens in a single prompt; this is like asking an LLM what the storyline behind *War and Peace* is.
 - Leverage external tools or agents (APIs, calculators, retrievers, web search tools) and memory. This is the emergence of what is called **agentic AI**.
 - Exhibit *advanced reasoning, chain-of-thought*, and *tool use* behavior.

In the next section, we will look at getting started with LLMs.

Setting up OpenAI's API

One of the most accessible and popular ways to start working with LLMs is through OpenAI's API. Rather than training a model from scratch, developers can use powerful pretrained models like **GPT4**, **DeepSeek**, **Claude**, **Mistral**, and others via simple API calls. This allows you to quickly integrate advanced language capabilities, like summarization, question answering, or code generation, into your application, with a carefully crafted prompt.

In this section, we will walk through an example of how to interact with GPT using OpenAI's **ChatCompletion API**:

1. Access to OpenAI's API is not free; however, the tokens do stretch your money. The first step is to register for the OpenAI service and log in at this link: **<https://auth.openai.com/log-in>**.
2. To generate a new API key, visit **<https://openai.com/api/>** to take you to the API keys section. To create a new one, simply name the API key, and it will generate the key for you. Please remember to take a copy of the API key.

3. In the supporting notebook for this chapter, you will bring in the relevant imports:

```
import os  
from getpass import getpass  
import openai
```

4. We will set an environment variable to store the OpenAI API key generated in *Step 2*. By using **getpass** (<https://docs.python.org/3/library/getpass.html>), the user will be prompted to enter the API key manually, ensuring that the key is not stored directly in the notebook:

```
os.environ["OPENAI_API_KEY"] = getpass("Enter your OpenAI API key: ")
```

5. The final step to set up our workbook ready for working with OpenAI is to initialize an instance of OpenAI's client and set the API key to the environment variable we have just stored:

```
client = openai.OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
```

Now that we have established communication with OpenAI, we can start using the LLM for practical tasks. This will be discussed in the next section.

Code generation with OpenAI

The AI revolution began with automating complex tasks, including **code generation**. The models at your disposal, OpenAI and others, allows for the development of high-quality code snippets, complete functions or even building entire applications with just a few prompts (a piece of text or instruction that you give to the AI model to guide its response, in this sense it will be describing a programming task).

When interacting with models, like those from OpenAI, two main types of prompts play distinct roles: the system prompt and the user prompt. The **system prompt** is used to set the overall behavior or tone of the AI,

providing context or instructions that guide how the model should respond throughout the conversation. For example, the system prompt might tell the AI to act as a helpful programming assistant or answer questions in a formal style. On the other hand, the **user prompt** is the specific input or question given by the user, such as a request to write a Python function or explain a coding concept. Together, these prompts work to shape the AI's responses, ensuring that they are relevant, accurate, or aligned with the intended purpose of the interaction.

The following steps explore how to leverage OpenAI's capabilities for code generation, streamlining the development process, and unlocking new levels of productivity for junior or experienced programmers alike:

1. Define a system prompt that defines how the LLM will behave:

```
SYSTEM_PROMPT = """
```

```
You are a helpful assistant that generates Python code  
based on the provided prompt.
```

```
"""
```

2. Next, we will tell the LLM what we want to do; this is called the user prompt, which provides the LLM with the context:

```
USER_PROMPT = """
```

```
Write a Python function that does a leet coding  
Recursive Binary Search.
```

```
The function should take a sorted list of integers  
and a target integer as input,
```

```
and return the index of the target integer in the  
list if it exists, or -1 if it does not.
```

```
The function should be implemented using recursion.
```

```
The function should be named `recursive_binary_search`  
and should have the following signature:
```

```

```python
def recursive_binary_search(arr: list[int],
 target: int) -> int:
 ...

"""

```

3. We will pick a valid model from the list of models available at <https://platform.openai.com/docs/models> on OpenAI's models page:

```
MODEL = "gpt-4o"
```

4. This step sends a request to the OpenAI API to generate a response from the language model. It specifies which model to use and provides a list of messages, including a system prompt (which sets the behavior or context for the AI) and a user prompt (which contains the user's question or instruction). The **max\_tokens** parameter controls the maximum length of the AI's response, while temperature adjusts how creative or deterministic the output will be. The result is stored in the response variable, which will contain the model's answer based on the prompts and settings provided:

```

response = client.chat.completions.create(
 model=MODEL,
 messages=[
 {"role": "system", "content": SYSTEM_PROMPT},
 {"role": "user", "content": USER_PROMPT}
],
 max_tokens = 2048,
 temperature = 0.3)

```

5. In this step, we will pull the reply from the language model (the generated code) from the response and print it to the console so we can see the result:

```
generated_code = response.choices[0].message.content
print(generated_code)
```

6. In *Step 5*, we simply printed the function to the console. However, to work with the raw Python code, we need to extract it from the response, which is the purpose of this step. First, we import the **re** module to use regular expressions for text searching. Next, we access the AI's reply and look for a code block marked with triple backticks and labeled as Python. If such a block is detected, we extract and print just the Python code within it. If not, we display a message indicating that no relevant Python code was found:

```
import re

raw_output = response.choices[0].message.content
match = re.search(r"```\python\s+(.*?)`\"",
 raw_output, re.DOTALL)

if match:
 python_code = match.group(1).strip()
 print(python_code)
else:
 print("No Python code block found.")
```

7. The function in *Step 6* has successfully extracted the generated function, and we can see it has solved a *LeetCode* (<https://leetcode.com/>) interview question, that companies like *Google* use to screen candidates, the creation of a recursive binary search or tree:

```
def recursive_binary_search(arr: list[int], target: int) -> int:
 def binary_search_helper(low: int, high: int) -> int:
 if low > high:
 return -1
```

```

Base case: target is not found
mid = (low + high) // 2
Calculate the middle index
if arr[mid] == target:
 return mid # Target found at index mid
elif arr[mid] < target:
 # Search in the right half
 return binary_search_helper(mid + 1, high)
else:
 # Search in the left half
 return binary_search_helper(low, mid - 1)
Initial call to the helper function
return binary_search_helper(0, len(arr) - 1)

```

8. To execute the code returned from the LLM, in Python directly, we can do the following, which will store this function in memory:

```
exec(python_code)
```

9. Now that the function has been stored in memory, from *Step 8*, we can use this function (completely generated by the LLM) to find a value based on an index provided:

```
sorted_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
target = 5
```

```
result = recursive_binary_search(sorted_list, target)
```

```
print(f"The TARGET specified: {target} is at INDEX: {result} due to Python's zero-based indexing.»)
```

With that, we now have our own AI-powered programming assistant at our fingertips. This tool can support us with debugging code, brainstorming new function ideas, writing unit tests, and tackling a wide variety of other programming tasks. Whether you need help solving tricky bugs or simply

want to speed up your workflow, your AI companion is ready to assist with any coding challenge you might encounter.

## Summarization and translation with OpenAI

Summarization and translation are practical techniques for making text more accessible and easier to share across languages. Summarization helps condense lengthy documents or texts into shorter versions that highlight the main ideas, saving time and effort for the reader. Once a clear and concise summary is created, translation allows this essential information to be communicated to a broader audience.

In this section, we will first demonstrate how to generate a summary from an English document. We will then take this summary and translate it into German, providing a straightforward workflow for presenting information in multiple languages.

### Summarizing the document

As part of this section, we will summarize a text, use the LLM to do the heavy lifting, and then present the summary back. This is a common use of LLMs and is one of the areas where LLMs are increasingly becoming useful. This will be achieved in the following steps:

1. The first step is to put our LLM into task mode by defining an effective system prompt:

```
SYSTEM_PROMPT = """
```

```
You are a helpful assistant that summarises long text and
produces a concise, but informative summary. Please explain your r
esponse in
```

```
a way that is easy to understand for a general audience.
```

```
"""
```

2. Next, we define the user prompt (this will relate to Machine ethics in AI), however, please refer to the supporting notebook for this

step, as the text is lengthy and for brevity it will not be included here.

3. We will define a helper function to return our text response and the response dictionary from OpenAI:

```
def get_openai_response(user_prompt, system_prompt,
 model="gpt-4o", **kwargs):
 response = client.chat.completions.create(
 model=model,
 messages=[
 {"role": "system", "content": system_prompt},
 {"role": "user", "content": user_prompt}
],
 **kwargs
)
 return response.choices[0].message.content, response
```

4. As we did before, we will pass the system and user instructions to the LLM:

```
response, _ = get_openai_response(user_prompt=USER_PROMPT,
 system_prompt=SYSTEM_PROMPT,
 model=MODEL,
 max_tokens=2048,
 temperature=0.3)
```

5. When you print the response, you will see the summarized text. To control the length of the summary, you can adjust the system prompt, for example, by requesting a summary in 500 words or characters, depending on your needs. You can also make the

response shorter by reducing the maximum number of tokens (units of text) generated by the language model.

Now that we have our summary, we will move on to translating it into a different language, such as German.

## Translating the document

LLMs are now widely used for translating documents, offering the ability to generate fluent, contextually accurate translations across many language pairs (source to target language). These models can handle idiomatic expressions and adapt tone or style to suit the intended audience. However, while LLM-based translations are often highly effective, they may struggle with specialized terminology, rare languages, or culturally specific references. For important or technical documents, it is recommended to combine the LLM-generated translations with human review to ensure accuracy and appropriateness.

In the steps that follow, we will translate the text from English to German and then assess the translation by doing a *back translation* with another tool. For this, we will use Google Translate API, as this provides free access and a convenient SDK:

1. The first thing we need to do is set the LLM in task mode for translation. We will do this by accurately describing the outcome of the task:

```
SYSTEM_PROMPT = """
```

```
Your job is to translate the provided source text into German.
```

```
Please ensure that the translation is accurate and maintains the original meaning.
```

```
"""
```

2. Now, we will pass the response we got from the summarization step, with the system prompt we defined in *Step 1*, to get the translation from English to German (to adapt this it is as easy as

changing the system prompt from German to Japanese, or any other target language, and the same applies for the source language):

```
translated_resp, _ = get_openai_response(
 user_prompt=response,
 system_prompt=SYSTEM_PROMPT,
 model=MODEL,
 max_tokens=2048,
 temperature=0.3)
```

3. To print the translated response, we will use a print statement on the object **translated\_resp**, and this will give us the raw German output from the summary. The issue at this point is knowing if the German translation is right. This is where back translation comes in, the best way would be a human reviewer who speaks German, but in absence of that, we will use this approach for the sake of demonstration.
4. To import the Google Translate API (<https://translate.google.com/>), the initial step is to utilize the PyPi package **googletrans**, which is demonstrated as follows:

```
from googletrans import Translator
```

5. Uses the **translate\_text** asynchronous function to translate text from German to English (or any specified language pair). This function leverages an async context manager with the Translator object, allowing efficient, non-blocking translation operations (an async function allows your program to start a task and then continue doing other things while it waits for the response, rather than pausing everything until the task is finished, like getting a translation from the API). The source and target languages are configurable via the **src** and **dest** parameters, and additional



operations can be passed as needed. The function returns the translated result once the translation is complete:

```
from googletrans import Translator

async def translate_text(text_to_translate,
 dest='en', src='de', **kwargs):
 async with Translator() as translator:
 result = await translator.translate(text_to_translate,
 dest=dest,
 src=src, **kwargs)

 return result
```

6. In this step, we call the asynchronous **translate\_text** function to translate the provided text, wait for the translation to finish (**await**), and then we store the result in the **google\_translate\_res** variable. Using **await** ensures the function finishes before moving on, without blocking operations in the program:

```
google_translate_res = await translate_text(text_to_translate=translated_resp)
```

7. To get the translation back into English, we then just print out the text of the translation:

```
print(google_translate_res.text)
```

To assess the accuracy of translation, consider using both, automated metrics, such as **Bilingual Evaluation Understudy (BLEU)**, **Metric for Evaluation of Translation with Explicit Ordering (METEOR)**, or **Crosslingual Optimized Metric for Evaluation of Translation (COMET)** scores, and a manual review by a bilingual speaker to evaluate meaning, fluency, and cultural appropriateness. However, it is important to recognize that automated metrics may not fully capture subtle nuances or context, especially for complex or domain-specific texts, and a hybrid approach is required.

In the following section, we will explore open-source LLMs available through HuggingFace's model hub.

## Text generation with HuggingFace

HuggingFace's model hub (<https://huggingface.co/models>) is a widely used open-source platform that hosts thousands of pre-trained (trained on data from different domains) language models covering a broad range of languages and domains. These models, built on architectures such as BERT, GPT, and T5, are freely available and can be easily integrated into custom workflows for both research and production use. Like commercial platforms, such as *OpenAI* or *Google Gemini*, HuggingFace models can perform a wide array of NLP tasks, often with comparable quality and flexibility. Some of these common tasks are highlighted as follows:

- **Text generation and completion**, which allows the LLM to generate documentation, books, and other free-form writing.
- **Language translation** as discussed in the previous section.
- **Text summarization**, as we have covered in the summarization section previously.
- **Sentiment analysis**, where a commonly used example is the sentiment of a movie, product, or service, with responses such as *good*, *bad*, or *moderate*.
- **Named entity recognition**, which includes recognizing key entities in text such as people, places, organizations, dates, etc.
- **Question answering**, which involves asking the LLM questions and getting the desired answer.
- **Text classification**, i.e., asking the LLM to classify text based on set logic and reasoning, which could be used like a binary classifier.

In the following steps, we are going to load a model from the HuggingFace hub and use this model to generate some text, but we are using it in a

question-answering fashion:

1. We will be using the transformers library and PyTorch as our imports in this section.
2. In this step, we will define an LLM to use. We will be using a model called Falcon (as it is license-free) and should be small enough to run on a CPU, albeit very slowly:

```
MODEL_NAME = "tiiuae/falcon-7b-instruct"
```

3. In this step, we load a **tokenizer** using the Hugging Face **transformers** library. A tokenizer is a tool that converts raw text into a format that a language model can understand, typically by breaking the text into smaller units called tokens and then mapping those tokens to numerical IDs. We use the **AutoTokenizer.from\_pretrained(MODEL\_NAME)** function to do this. This automatically selects the appropriate tokenizer based on the model's name provided (such as **bert-base-uncased** or **llama2**), downloads it if necessary, and prepares it for use. The resulting tokenizer is then stored in the tokenizer variable for later use in preprocessing our input text:

```
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
```

4. Now we have our tokenizer ready, we load the pre-trained language model that will be used to generate or predict text. This is done by using **AutoModelForCasualLM.from\_pretrained()** function, which automatically selects the correct model architecture for casual language modeling (e.g., GPT style models) based on the model name specified. We pass several parameters to control how the model is loaded: **torch\_dtype=torch.float32** ensures the model uses 32-bit floating point precision; **device\_map="auto"** lets the system automatically choose whether to run the model on CPU or GPU; **trust\_remote\_code=False** adds a security measure by preventing the execution of arbitrary code from the model repository, and **low\_cpu\_mem\_usage=True** helps

reduce memory usage during model loading. The model is then stored in the model variable for future use:

```
model = AutoModelForCausalLM.from_pretrained(
 MODEL_NAME,
 torch_dtype=torch.float32,
 device_map="auto",
 trust_remote_code=False,
 low_cpu_mem_usage=True)
```

5. The model and tokenizer are now loaded onto your local machine; the next step is to define a user prompt to pass to the model. We will prompt it with something like an open-ended question:

```
USER_PROMPT = "Explain how gravity affects planetary orbits."
```

6. In this step, we will define a messages dictionary containing the roles of system and user, to align with the chat completion format:

```
messages = [
 {"role": "system", "content": "You are a helpful assistant."},
 {"role": "user", "content": USER_PROMPT}]
```

7. We define a function that takes a HuggingFace model and tokenizer, along with a list of chat messages, and generates a model-based response. First, it formats the conversation using the tokenizer's chat template to create a prompt suitable for the model. This prompt is then tokenized and prepared for input to the model, ensuring it is loaded onto the appropriate device (such as a GPU if available). The function then calls the model's generate method to produce a continuation of the conversation, using parameters like **max\_tokens** to control the length of the response and allowing for further customization via additional keyword arguments (**\*\*kwargs**). After generation, the function extracts only the new tokens generated by the model (excluding the original input), and

finally decodes these tokens back into human-readable text, which is then returned as the generated response:

```
def generated_hf_response(model,
 tokenizer,
 messages,
 max_new_tokens=512,
 return_type="pt",
 skip_special_tokens=True,
 **gen_kwargs):
 text = tokenizer.apply_chat_template(
 messages,
 tokenize=False,
 add_generation_prompt=True
)
 inputs = tokenizer([text],
 return_tensors=return_type).to(model.device)
 outputs = model.generate(
 **inputs,
 max_new_tokens=max_new_tokens,
 **gen_kwargs
)
 gen_tokens = outputs[0][len(inputs.input_ids[0]):]
 return tokenizer.decode(gen_tokens,
 skip_special_tokens=skip_special_tokens)
```

8. Finally, to use our function and to generate the text required, we pass in a pre-loaded HuggingFace model, as we did in the previous step, and tokenize this, along with a list of chat messages

representing the conversation so far. The function instructs the model to generate a response of up to 100 new tokens, with the tokenization process returning PyTorch Tensors (you can also return NumPy or TensorFlow Tensors as well). By setting **skip\_special\_tokens** to **True**, the function ensures the output omits any special formatting tokens, resulting in a clean reply. Additionally, with **do\_sample** set to **False**, the model generates its response deterministically, always choosing the most likely next token, rather than introducing randomness. The result is a concise, consistent text response generated from the conversation history. This function will take approximately 10 minutes to run, as running LLMs on CPUs is not recommended due to the attention mechanisms present in the transformer model:

```
generated_hf_response(model=model, tokenizer=tokenizer,
 messages=messages,
 max_new_tokens=100,
 return_type="pt",
 skip_special_tokens=True,
 do_sample=False)
```

This concludes our section on LLMs, though we have only begun to explore their vast potential. These models offer far more capabilities than we have covered here, and their impact is significant enough to fill entire books dedicated to the subject.

## Automated machine learning

AutoML streamlines the entire ML pipeline, from data preprocessing and feature engineering to model selection, hyperparameter tuning, and ensembling, making it accessible to both beginners and experts. While it is now not considered a future trend, as it has been with the ML community and used in industry for a while, it is important to highlight it here.

Among popular open-source frameworks, **Tree-Based Pipeline Optimization Tool (TPOT)** uses genetic programming (to combine and refine models and preprocessing steps through simulated evolution) to optimize pipelines in scikit-learn, while auto-sklearn employs Bayesian optimization (a process of exploration across a probabilistic model's hyperparameters) enhanced by meta-learning and automated ensembling. H2O AutoML offers multi-threaded model training across a variety of algorithm types. While they are all variants of AutoML, each one brings something different and unique to the table.

In this section, we will use TPOT to optimize our breast cancer prediction problem.

## **AutoML with tree-based optimization tool**

In this section, we will use the power of genetic algorithms (much like Darwin's theory of natural selection) to automatically design and optimize our ML pipelines. Genetic algorithms in TPOT work in the following steps:

- **Population initialization:** The model starts by randomly generating a population of diverse ML pipelines. Each pipeline is like a *candidate solution* and can include different preprocessors, models, and parameter settings.
- **Evaluation (fitness function):** Each pipeline is trained and tested on your data. Its *fitness* is measured by its performance (the normal accuracy metrics we use, F1 score, etc.).
- **Selection:** The best-performing pipelines are selected as *parents* for the next generation. Selection favors pipelines that performed well.
- **Crossover (Recombination):** TPOT combines parts of two parent pipelines to create new *child* pipelines, mixing up different components and parameters (like combining feature selection from one pipeline with a classifier from another).

- **Mutation:** To change things, TPOT occasionally makes random changes to a pipeline, such as swapping an algorithm, tweaking a hyperparameter, or adding/removing a step.
- **Iteration:** Evaluation and mutation are repeated over multiple generations. Each time TPOT evolves the population, it gradually finds pipelines that work better for your dataset.
- **Best pipeline selection:** After a set number of generations or when performance stops improving, TPOT outputs the best performing pipeline found, ready to use in your application.

We will walk through how this can work in practice, on our breast cancer prediction problem:

1. Please refer to the supporting workbook for the data preparation, as we have done the importing and splitting of the breast cancer data several times throughout this book.
2. Once the data has been split, we need to import the TPOT package (<https://epistasislab.github.io/tpot/latest/>):

```
from tpot import TPOTClassifier
```

3. In this step, we create an instance of the **TPOTClassifier**, configuring the genetic algorithm that will be used to search for the best ML pipeline. By specifying parameters such as **generations=5** and **population\_size=50**, we control how many rounds of evolution will occur and how many pipeline candidates will be explored in each generation. The high **mutation\_rate (0.9)** encourages frequent changes to individual pipelines, promoting diversity, while the lower **crossover\_rate (0.1)** limits how often pipelines are combined. Setting **cv=5** ensures that pipeline performance is evaluated using five-fold cross-validation, and **scoring='f1'** directs TPOT to optimize for the F1 score. The **random\_state** parameter ensures reproducibility, and **n\_jobs=-1** allows TPOT to use all available CPU cores for faster computation.



The **verbosity=3** setting provides detailed progress updates during the optimization process:

```
tpot_class = TPOTClassifier(verbosity=3,
 random_state=RANDOM_SEED,
 generations=5,
 population_size=50,
 n_jobs=-1,
 mutation_rate=0.9,
 crossover_rate=0.1,
 cv=5,
 scoring='f1')
```

4. Now we fit a model on the classifier initialized in *Step 3*, and wait for the process of evolution to complete:

```
tpot_class.fit(X_train, y_train)
```

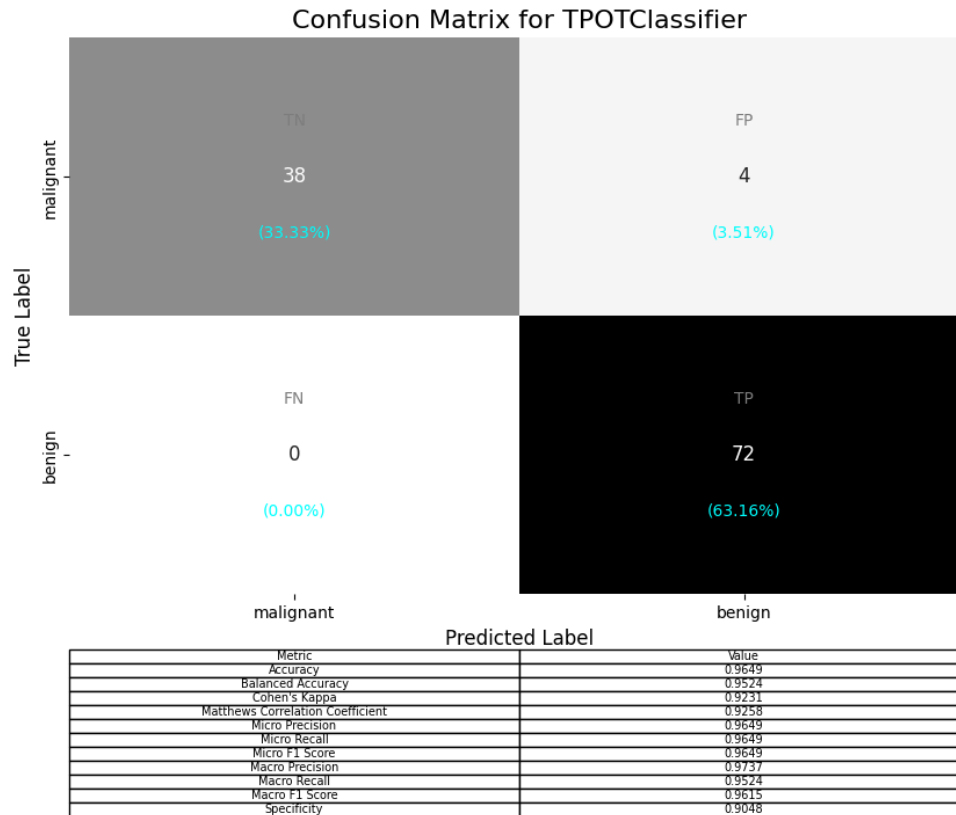
5. Once the model fitting process has been completed, we can get the score from the best performing model:

```
score = tpot_class.score(X_test, y_test)
```

6. To evaluate the performance against the test set, we will make predictions against the test set:

```
y_pred = tpot_class.predict(X_test)
```

7. To visualize the results of the model, we can use **modelviz** to plot a confusion matrix:



*Figure 13.1: Confusion matrix for TPOT*

8. The final step is to export the best pipeline to use in your estimation projects downstream:

```
import os

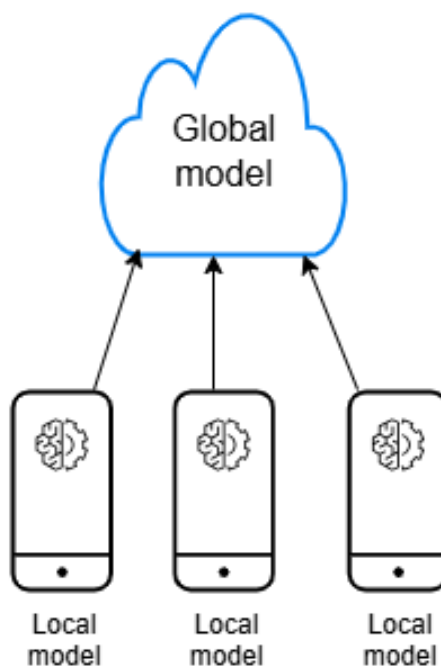
os.makedirs('tpot_exported_pipeline',
 exist_ok=True)

tpot_class.export(os.path.join("tpot_exported_pipeline",
 'tpot_best_pipeline.py'))
```

## Federated learning

FL is a decentralized ML approach where model training is distributed across multiple devices or services holding local data samples, without exchanging them. The primary goal is to build a shared global model while preserving data privacy and minimizing data transfer.

In FL setup, each client, for example, smartphone, **Internet of Things (IoT)** device, or local server, trains a local model on its own data. These local models send only the updated weights or gradients to a central server, which aggregates them, using a technique called **federated averaging (FedAvg)** to update the global model. This process repeats multiple times until convergence, a simple visual representation of this is contained in *Figure 13.2*:



*Figure 13.2: FL process*

FL offers significant advantages such as preserving data privacy, reducing latency and bandwidth usage, enabling scalable deployment across numerous edge devices, and supporting compliance with data protection regulations; however, it also faces challenges including non-identifiable data distributions, communication overhead, client heterogeneity, and susceptibility to security threats like model poisoning and inference attacks.

This approach is increasingly used in real-world systems where data is sensitive, distributed, or resource-constrained. Its decentralized approach makes it ideal for industries that prioritize privacy, compliance, and real-time learning. Common applications of FL include:

- **Mobile devices:** Used in predictive text systems where the keyboard learns from user typing behavior without uploading data to the cloud. This ensures privacy while still improving the language model.
- **Healthcare:** Enables collaborative training of diagnostic models across multiple hospitals or clinics without sharing patient data. Seen in applications such as imaging diagnostics, personalized treatment recommendations, and hospital readmission predictions.
- **Industrial IoT:** Edge devices in manufacturing or smart grids use FL to optimize maintenance schedules, detect anomalies, and enhance energy efficiency based on local sensor data.
- **Smart homes and wearables:** Devices like smart thermostats or fitness trackers use FL to personalize the user experiences while keeping behavior data local.

We will create a scenario dataset, where three hospitals obtain mobile diagnostic data from patient assessments, and then use their tablets to share the weights between hospitals. As a simple example, we will structure this in the following steps:

1. We use a **make\_classification** function to generate hospital site data:

```
def generate_site_data(seed):
```

```
 X, y = make_classification(n_samples=200,
```

```
 n_features=10,
```

```
 n_informative=6,
```

```
 n_redundant=2, n_classes=2,
```

```
 random_state=seed)
```

```
 return train_test_split(X, y,
```

```
 test_size=0.3,
```

```
 random_state=seed + 10)
```

2. Now, we will use this function to generate random seeds for the three hospital sites (this would be based on electronic observations, such as patient age, presenting complaint, etc.):

```
site_data = [generate_site_data(seed) for seed in [1, 2, 3]]
```

3. The next step creates a loop trained on a separate local model on each hospital site and stores the model alongside its test set; in FL, each site *trains independently and shares only its model weights* (not data) with a central server for global model aggregation:

```
local_models = []
```

```
for X_train, X_test, y_train, y_test in site_data:
```

```
 model = LogisticRegression(max_iter=500).fit(X_train, y_train)
```

```
 local_models.append((model, X_test, y_test))
```

4. Suppose then each hospital site evaluates its own performance score, we will use **area under the curve (AUC)** here:

```
print("AUC for individual NHS sites:")
```

```
for i, (model, X_test, y_test) in enumerate(local_models):
```

```
 auc = roc_auc_score(y_test, model.predict_proba(X_test)[:, 1])
```

```
 print(f"Hospital Site {i+1}: AUC = {auc:.3f}»)
```

5. This would already exist alongside the global model, but we will create the federated averaging function that takes the coefficients and intercepts from each model and gets the average of both the intercepts and the coefficients:

```
def federated_average(models):
```

```
 coefs = np.mean([m.coef_ for m, _, _ in models],
 axis=0)
```

```
 intercepts = np.mean([m.intercept_ for m, _, _ in models],
 axis=0)
```

```
 global_model = LogisticRegression()
```

```

global_model.coef_ = coefs
global_model.intercept_ = intercepts
global_model.classes_ = np.array([0, 1])
return global_model

```

6. The final step is to perform aggregation in the cloud. This loop simulates the progressive aggregation of local models from two and then three **National Health Service (NHS)** sites in a federated learning setup. For each step, it selects the first **n** local models (where **n** is 2 or 3), averages their weights using the federated averaging function, defined in *Step 5*, to create a new global model, and evaluates the model's performance. To do so, it combines the test sets from the selected sites and uses the global model to predict probabilities on the aggregated data. Finally, it calculates and prints the **receiver operating characteristic (ROC)** AUC score, showing how the global model's diagnostic performance improves as it incorporates more diverse data from the additional sites:

for site in [2, 3]:

```

 selected_models = local_models[:site]
 global_model = federated_average(selected_models)
 X_combined = np.vstack([X for _, X, _ in selected_models])
 y_combined = np.hstack([y for _, _, y in selected_models])
 y_pred_prob = global_model.predict_proba(X_combined)[: , 1]
 auc = roc_auc_score(y_combined, y_pred_prob)

 print(f"\nGlobal Model (aggregated over {site} site(s)) — AU
 C: {auc:.3f}")

```

FL offers a powerful framework for collaboratively training ML models across distributed data sources, such as hospitals where staff use edge devices to obtain patient information, without ever sharing raw data. By enabling each site to train locally and contribute only model updates, it

ensures strong privacy preservation, regulatory compliance, and scalability across diverse environments. Through techniques like federated averaging, a shared global model can be created that benefits from broader data representation while respecting institutional and patient boundaries. As demonstrated, performance typically improves as more sites participate, making FL especially valuable in sensitive domains like healthcare, where collaboration is essential, but data sharing is constrained.

## **Ethical frameworks, bias, privacy, and AI governance**

ML systems, no matter how technically sophisticated, are never ethically neutral. As developers and practitioners increasingly rely on tools like scikit-learn and LLMs to build decision-making systems, they shoulder growing responsibility for the real-world impacts of those systems. Whether it is predicting loan approvals, filtering job applications, or assessing health risks, such as cancer or thyroid diseases, the data and models we build can reflect and reinforce historical bias, compromise individual privacy, or lack transparency in high-stake situations.

This section provides practical tools for integrating fairness, privacy, and governance into ML workflows, using scikit-learn. Through real-world case studies and hands-on techniques, readers will learn to audit, correct, and document responsible AI practices while maintaining technical rigor.

## **Detecting and mitigating bias with scikit-learn**

Bias in ML arises when a model produces systematically unfair outcomes for certain groups, often due to imbalanced or predicted training data. Scikit-learn, while not inherently biased, makes it easy to reproduce data-driven inequalities unless we take explicit corrective steps.

## **Gender shades case study**

In this case study, researchers found that commercial facial recognition tools misclassified darker-skinned women at a much higher rate than lighter-skinned men. The underlying problem was all in the training data.

As scikit-learn is generally used to deal with tabular data, ethnicity and gender are common fields in every database, so here are some tools to help mitigate this underrepresentation in the data:

- Use **StratifiedKFold** or **StratifiedShuffleSplit** to ensure a fair distribution of demographic groups in cross-validation.
- Analyze classification report metrics (precision, recall, and F1 score) across subgroups using **sklearn.metrics.classification\_report**.
- Implement custom pipelines with **FunctionTransformer** to track subgroup performance.

In the next study, we will look at unsupervised bias detection.

## Unsupervised bias detection case study

In 2025, an audit of a Dutch public sector algorithm (<https://arxiv.org/abs/2502.01713>) used between 2012 and 2023 to assign risk scores to over 250,000 students, researchers applied an unsupervised hierarchical bias-aware clustering tool to detect disparities, without relying on known demographic labels, but based on algorithmic outcomes.

The method successfully uncovered clusters that corresponded with known inequalities: students of non-European migration background were grouped separately, and these students showed statistically significant differences in risk assignments. This highlighted that even label-free clustering could surface demographic bias embedded in decision-making systems.

To mitigate against this, the following steps could be taken:

- **Define a proxy bias variable.** For example, the misclassification rate or error margin across the prediction.
- **Cluster** individuals based on relevant features, e.g., modeled scores or outcomes.



- **Analyze clusters** to see if subgroup error or bias metrics are significantly different across clusters.
- **Manually inspect** clusters that show extreme bias values to interpret potential demographic alignment.

In the next section, we are going to look at practical steps to ensure the privacy of the data is upheld.

## Privacy preserving techniques in scikit-learn pipelines

Privacy in the context of ML and AI is about protecting individuals from being identified, targeted, or harmed by how data is collected and used. Even if a dataset lacks explicit identifiers, models can leak sensitive attributes through inference or overfitting.

To mitigate this in your pipelines, you can follow these points:

- Remove or anonymize direct identifiers using Pandas preprocessing.
- Use synthetic data generation for exploratory modeling.
- Limit feature use with **ColumnTransformer** to exclude high-risk attributes (<https://ico.org.uk/>).
- Implement noise injections during training to reduce the model's ability to memorize.
- FL is another way to ensure privacy by just sharing the weights of the model to be averaged by a global model, as we looked at in the *Federated Learning* section.
- Use a **Conditional Tabular Generative Adversarial Network (CTGAN)** model to generate the train and test data; this is a tool Kaggle (<https://www.kaggle.com/>) uses for their tabular playground series.

When preparing your ML pipelines, remember to check the following:

- If any features are directly or indirectly identifiable: name, address, zip (post) code, etc.
- Whether the model retains performance if sensitive features are excluded.
- If the model predictions can be inverted to infer private traits.

In the next section, we will look at a framework for AI governance and how this can be applied.

## AI governance

Governance in AI ensures that systems align with organizational values, legal requirements, and ethical norms. The TRUSTED framework provides a practical guide, adapted from *Leipzig (2023) Trust: Responsible AI, Innovation, Privacy and Data Leadership*:

- **Triage risk:** Identify high-stakes cases (e.g., hiring, lending, healthcare). Classify models based on potential harm. Use the model inspection tool (<https://scikit-learn.org/stable/inspection.html>) in scikit-learn.
- **Right data:** Ensure representative and high-quality data inputs. Use `train_test_split` with stratification and Pipeline objects for repeatable preprocessing.
- **Uninterrupted monitoring:** Track model performance over time. Set up data drift detection by logging metrics and model retraining triggers. Store metrics in **MLFlow** (<https://mlflow.org/>) (or similar) and set up error logging tools for monitoring errors.
- **Supervision:** Embed human-in-the-loop review for critical decisions. For example, flag edge cases for manual approval.
- **Technical documentation:** Maintain documentation of pipeline steps using scikit-learn's `set_params` and `get_params`. Save models with **joblib** and include audit metadata.

- **Explainability:** Users, auditors, and stakeholders need to understand how an AI system makes decisions, especially when the stakes are high. Explainability builds transparency and supports accountability. The following are the methods to achieve this:
  - Use interpretability tools like **SHapley Additive exPlanations (SHAP)** (covered in *Chapter 6, House Price Estimation with Regression Methods*).
  - Train models with inherently interpretable architectures (e.g., decision trees or linear models, where possible).
  - Provide post-hoc explanations for predictions, particularly for flagged or borderline cases.
- **Domain ethics oversight:** Decisions in domains like healthcare, employment, finance, or criminal justice carry unique ethical concerns. Domain-specific oversight ensures governance is sensitive to sector norms, fairness expectations, and regulatory responsibilities. Steps to apply this would be to create domain-specific ethics boards or review committees; create ethical checklists tailored to the risk in that domain, and engage external ethics experts, audit third-party fairness reports, and ensure remedy pathways for affected users.

In the next section, we will apply the principles we have learned so far to the processing steps in a pipeline.

## Building a responsible pipeline

In this series of steps, we will consider everything we have learned in this section and build a responsible model pipeline, factoring in tips we have learned throughout this book. This will be explained in the following sections.

## Loading and preparing the data

In this section, we will be using the *UCI Machine Learning Repositories Adult* dataset (<https://archive.ics.uci.edu/dataset/2/adult>). This contains Census data used to predict income, and there are a few sensitive fields included. In the following steps, we will read in the data and separate out those *sensitive* data points that relate to the individual:

1. Read in the CSV file from the data directory into a Pandas DataFrame, use `?` to treat any cells containing a question mark as missing values and ignore extra spaces that appear immediately after the commas in the CSV file:

```
df = pd.read_csv('data/adult.csv',
 na_values='?',
 skipinitialspace=True)
```

2. Remove any trailing spaces from the target (**income**) feature:

```
df['income'] = df['income'].str.strip()
```

3. The next step is to **map** the target variable to a binary encoding:

```
y = df['income'].map({'>50K': 1, '<=50K': 0})
```

4. Get rid of any NA values:

```
df = df[~y.isna()]
```

5. Drop **income** from the feature set, as this is the target variable we are going to predict:

```
X = df.drop(['income'], axis=1, errors='ignore')
```

6. Separate out the sensitive variables that are subject to privacy constraints:

```
sensitive = df[['sex', 'race']]
```

7. Use the **select\_dtypes** function to get categorical and numeric features:

```
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
```

```
numerical_cols = X.select_dtypes(include=['int64', 'float64']).columns.tolist()
```

8. The final step, in this section, is to split the training and test datasets:

```
(X_train, X_test, y_train, y_test,
sens_train, sens_test) = train_test_split(
 X, y, sensitive, test_size=0.3,
 stratify=y, random_state=42
)
```

Now, we have our data preprocessed and split, we can move on to building a simple preprocessing and modeling pipeline.

## Building the pipeline

We will now construct a simple pipeline to do the preprocessing, one-hot encoding, missing value imputation, scaling, and modeling. This will be achieved in the following steps:

1. The first strategy is to build a numeric pipeline that will use a **RobustScaler**, as we learned, these are good for outliers, and we will carry out a median replacement strategy:

```
numeric_transformer = Pipeline([
 ('imputer', SimpleImputer(strategy='median')),
 ('scaler', RobustScaler())
])
```

2. Next, we will create a categorical transformer, where we will one-hot encode the categorical features, and we will use a modal imputation strategy:

```
categorical_transformer = Pipeline([
 ('imputer', SimpleImputer(strategy='most_frequent')),
```

```
 ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])
```

3. Once we have these two pipelines in place, we can use a **ColumnTransformer** to join the numerical and categorical pipelines together:

```
preprocessor = ColumnTransformer([
 ('num', numeric_transformer, numerical_cols),
 ('cat', categorical_transformer, categorical_cols)
])
```

4. Finally, in this step, we construct the pipeline and use an interpretable model (this is not always possible, but we are reinforcing the TRUSTED framework, in this example):

```
pipeline = Pipeline([
 ('preprocessing', preprocessor),
 ('clf', LogisticRegression(solver='liblinear'))
])
```

In the next section, we will undertake the cross-validation and sub-group performance step.

## Cross-validation and sub-group performance

This section of code performs cross-validation on an ML pipeline, tracking performance metrics globally and for specific subgroups (by sensitive attributes and their intersections). It does this by splitting the training data into folds, training and evaluating the model on each fold, and then auditing the results both overall and for different demographic groups:

1. `skf = StratifiedKFold(n_splits=5)`
2. `metrics = {'accuracy': [], 'recall': [], 'precision': [], 'f1': []}`
3. `subgroup_performance = []`

```
4.
5. for train_idx, valid_idx in skf.split(X_train, y_train):
6. X_tr, X_val = X_train.iloc[train_idx], X_train.iloc[valid_idx]
7. y_tr, y_val = y_train.iloc[train_idx], y_train.iloc[valid_idx]
8. sens_val = sens_train.iloc[valid_idx]
9. pipeline.fit(X_tr, y_tr)
10. y_pred = pipeline.predict(X_val)
11. metrics['accuracy'].append(accuracy_score(y_val, y_pred))
12. metrics['recall'].append(recall_score(y_val, y_pred))
13. metrics['precision'].append(precision_score(y_val, y_pred))
14. metrics['f1'].append(f1_score(y_val, y_pred))
15.
16. # -- Attribute-level audits --
17. for attr in sens_val.columns:
18. for grp in sens_val[attr].unique():
19. mask = sens_val[attr] == grp
20. if mask.sum() == 0:
21. continue
22. subgroup_performance.append({
23. 'type': 'attribute',
24. 'attribute': attr,
25. 'group': grp,
26. 'accuracy': accuracy_score(y_val[mask], y_pred[mask])
27. })
28. intersection_series = sens_val['sex'].astype(str) + "_" + sens_val
 ['race'].astype(str)
29. for grp in intersection_series.unique():
```

```
30. mask = intersection_series == grp
31. if mask.sum() == 0:
32. continue
33. subgroup_performance.append({
34. 'type': 'intersectional',
35. 'attribute': 'sex_race',
36. 'group': grp,
37. 'accuracy': accuracy_score(y_val[mask], y_pred[mask]))
```

This script does the following:

- **Set up stratified cross-validation:** Uses **StratifiedKFold** to split the data into five folds, making sure each fold has a similar balance of target classes.
- **Initialize metric tracking:** Prepares dictionaries and lists to store global performance metrics (accuracy, recall, precision, and F1 score) and subgroup results (gender by ethnicity).
- **Loops through each fold:**
  - **Split data:** Separates data into training and validation sets for each fold
  - **Extract sensitive attributes:** Pulls out columns like sex or race for subgroup analysis.
- **Train and predict:** Fits a pipeline on the training set, with a simple explainable model, and makes predictions on the validation sets.
- **Computes global metrics:** Stores the global metrics selected in the preinitialized lists and dictionaries.
- **Audit subgroup performance:** For each sensitive attribute (like sex or race), calculates accuracy for each group within that attribute (e.g., Female, Male).



- **Intersectional-level audit:** Calculates accuracy for intersectional groups (e.g., **Female\_White**), combining the sensitive attributes.
- **Stores all results:** Saves both global and subgroup metrics for later analysis.

In the next section, we will fit the pipeline and perform an evaluation on the model results.

## Fitting the model

We have explored this code multiple times throughout this book, but let us go through the steps to reinforce how to fit a model:

1. Use the **fit** method on the pipeline and fit the training data (both target and feature fields):

```
pipeline.fit(X_train, y_train)
```

2. Now that the pipeline is fitted, we can make predictions on our test set:

```
y_test_pred = pipeline.predict(X_test)
```

3. Create a dictionary to store the overall performance of the model, this does not take into account any of the subgroup variation, as we will do this as a separate process, in the next step:

```
overall = {
```

```
 'accuracy': accuracy_score(y_test, y_test_pred),
```

```
 'recall': recall_score(y_test, y_test_pred),
```

```
 'precision': precision_score(y_test, y_test_pred),
```

```
 'f1': f1_score(y_test, y_test_pred)
```

```
}
```

4. To display the overall performance, we can simply print this out to the console. This model shows relatively good accuracy (~**0.85**), but the recall and F1 score are suboptimal, indicating a class

imbalance problem, which can be left to you to rebalance, as we talked about this extensively throughout this book.

In the next section, we will get on to evaluating the sub-group performance.

## Evaluating the sub-group performance

In the following steps, we will aim to evaluate the sub-group performance by looping through the sensitive test data for sex and race and compute the relevant metrics. Everything will be covered gradually through this section to explain each part:

1. The starting point is taking the test data for the sensitive groups and concatenating the fields together. This simply takes sex, adds an underscore, and then concatenates the race field to the end, so this becomes **Male\_White**, as a string:

```
sens_test_combined = sens_test['sex'].astype(str) + "_" + sens_test
['race'].astype(str)
```

2. Then, we initialize empty lists for the groups and relative accuracy metrics we care about for this test:

```
groups = []
```

```
accuracies = []
```

```
f1s = []
```

```
precisions = []
```

```
recalls = []
```

3. We loop through the combined **sens\_test\_combined** field for each sensitive attribute (like sex or race), and for every subgroup within that attribute (such as Female or White). For each subgroup, we identify the corresponding validation samples, ensure there are samples present (skipping if there are none), then compute performance metrics, including accuracy, precision, recall, and F1 score, for that group. We then log the group's name, attribute, type,

and performance metrics in a list for later analysis of subgroup performance:

```
for grp in sens_test_combined.unique():
 mask = sens_test_combined == grp
 acc = accuracy_score(y_test[mask], y_test_pred[mask])
 f1 = f1_score(y_test[mask], y_test_pred[mask])
 precision = precision_score(y_test[mask], y_test_pred[mask])
 recall = recall_score(y_test[mask], y_test_pred[mask])
 groups.append(grp)
 accuracies.append(acc)
 f1s.append(f1)
 precisions.append(precision)
 recalls.append(recall)
```

4. To convert these into a DataFrame, we simply pass in each list as a value and define a dictionary key, then set the DataFrame index to the **Group** column:

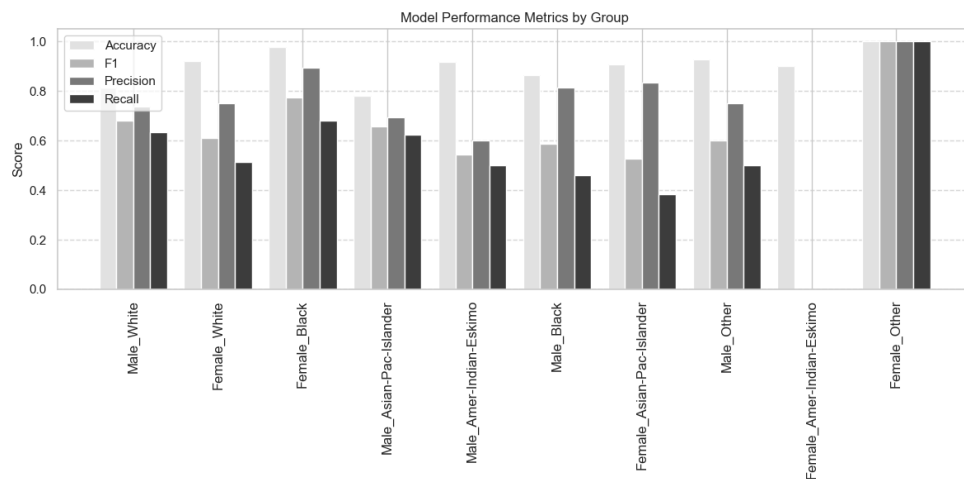
```
df_metrics = pd.DataFrame({
 'Group': groups,
 'Accuracy': accuracies,
 'F1': f1s,
 'Precision': precisions,
 'Recall': recalls
})
df_metrics = df_metrics.set_index('Group')
```

5. In the accompanying notebook, we have included a function that allows you to visualize these metrics side by side for each subgroup. Please refer to the notebook to see how the function is

implemented, as in this step, we will only be using the function, not re-implementing it:

```
plot_group_metrics_bar(
 df_metrics, ['Accuracy', 'F1',
 'Precision',
 'Recall'],
 sns_theme="whitegrid",
 show_legend=True,
 label_args={'ylabel': 'Score', 'fontsize': 12})
```

6. Upon running this function, we can see the chart as in *Figure 13.3*:



*Figure 13.3: Subgroup performance*

7. The chart in *Figure 13.3* illustrates how the model's performance varies across the different demographic subgroups. The plot reveals that the model does not perform equally well for all groups. For example, certain groups, such as **Female\_Other**, show very high scores across all metrics, while others, like **Male\_Black**, show lower recall scores, meaning it is harder to predict their income. These differences may suggest that the model may be biased or less reliable for some subgroups, which is an important consideration when assessing fairness and the overall

trustworthiness of the model's predictions. This kind of analysis helps identify where the model's weaknesses are, so that steps can be taken to improve fairness and reduce disparities in predictive performance.

8. In the supporting workbook, we also include a SHAP explainability plot for analyzing global performance.

In the next step, we will set a custom threshold up for flagging any uncertain predictions that need a human review; those where we are not confident that the model will pick them out.

## Flag uncertain predictions for human review

Continuing with the explainability component of the TRUSTED framework, we will introduce a method to identify and flag borderline cases for human review within our workflow. The following code demonstrates how this can be done:

1. We will use the model to obtain the predicted probabilities for the positive class for each sample in the test set:

```
probs = pipeline.predict_proba(X_test)[: , 1]
```

2. For the edge case threshold logic, we will identify the test samples for which the predicted probability is close to 0.5 (between 0.45 and 0.55 in this case), and record their indices as *edge cases* that may need special attention or human review:

```
edge_cases = np.where((probs > 0.45) & (probs < 0.55))[0]
```

3. Since we have the indices of these cases as integers, we can use `.iloc` to extract the corresponding rows from the test DataFrame. This subset can then be exported for further processing, integrated into internal workflows, or flagged for manual review as needed:

```
X_test.iloc[edge_cases]
```

This demonstrates how these steps can enable human quality assurance review and how insights gained from this process can inform future model retraining or data collection efforts.

## Documentation and audit trail

To ensure transparency and accountability in our modeling process, it is important to keep thorough records of the model configuration, performance, and relevant data characteristics. The following code captures essential metadata, including model parameters, performance metrics, and information about features and sensitive attributes, at the time the model is trained and evaluated. By saving both the trained model pipeline and this metadata to disk, we create a clear audit trail that documents not only the model itself, but also how it was assessed. This makes it easier to review, reproduce, and explain modeling decisions in the future, which is a key aspect of responsible AI practices:

1. Create a dictionary with the relevant features we are going to want to track and call this metadata, or similar:

```
metadata = {
 'timestamp': time.strftime("%Y-%m-%d %H:%M:%S"),
 'features': list(X.columns),
 'sensitive_features': list(sensitive.columns),
 'model_params': pipeline.named_steps['clf'].get_params(),
 'cv_metrics': metrics,
 'overall_metrics': overall,
 'subgroup_performance': subgroup_performance
}
```

2. Once the dictionary has been created, we can store all of this in a **joblib** file and we call it **responsible\_model\_audit.joblib**:

```
os.makedirs('audit', exist_ok=True)
```

```
joblib.dump({'pipeline': pipeline,
 'metadata': metadata},
 'audit/responsible_model_audit.joblib')
```

In this section, we explored how ethical frameworks and governance principles guide the responsible development and deployment of AI systems. We examined practical strategies to identify and mitigate bias, protect individual privacy, and ensure transparency and accountability throughout the modeling process. By systematically auditing our models, documenting key decisions, abiding by an overarching framework, and including human oversight, where necessary, we create a foundation for ethical AI. These practices not only help to safeguard against unintended harm but also promote fairness, compliance, and public trust in AI technologies.

## Conclusion

In this chapter, we covered the most advanced concepts in modern ML and AI, bringing together everything you have learned throughout this book. We explored the inner workings and real-world impact of LLMs, demonstrated how to apply them using both commercial APIs and open-source frameworks, and showed how tools like AutoML and FL are reshaping how models are built, deployed, and governed. We also addressed the practical and ethical challenges that come with building powerful AI systems, including how to identify and mitigate bias, protect privacy, and create transparent, auditable workflows.

As this is the final chapter, it is worth reflecting how far you have come on your journey from foundational data manipulations and classic ML workflows, spanning classification, regression, clustering, dimensionality reduction, time series modeling and deep learning, to modern deployment strategies and advanced topics such as LLMs, AutoML and FL, as well as making sure you adhere to responsible AI practices.

## Points to remember

Here are some key points summed up from this chapter:

- **APIs make LLMs accessible:** Cloud APIs from providers like OpenAI, Google Gemini, Anthropic let you leverage powerful language models for various language tasks without needing to train or host your own, but come at a cost per token processed pricing model.
- **Audit and explain your models:** Regularly assess performance across different subgroups, document your modeling decisions, and flag uncertain predictions for human review.
- **AutoML can streamline workflows:** Tools like TPOT and H2O ML automate the process of model selection, hyperparameter tuning, and pipeline optimization, making ML more efficient.
- **Ethical and responsible AI is non-negotiable:** Mitigating bias, ensuring data privacy, and building transparent governance frameworks are critical to trustworthy AI systems.
- **Federated learning preserves privacy:** FL enables collaborative model training across distributed data sources without sharing raw data, supporting privacy and compliance.
- **LLMs have transformed NLP:** Modern transformer-based LLMs underpin state-of-the-art capabilities in text and code generation, summarization, translation, and more.
- **Open-source models offer flexibility:** Open-source organization and packages by HuggingFace provide access to a range of pretrained models, allowing for customization by fine-tuning and on-premise/cloud-managed deployment.
- **Stay updated:** This field evolves rapidly; continuous learning, ethical reflection, and adaptation are key for success in the future of AI.

## Multiple choice questions



1. **What is the primary advantage of using an LLM via an API?**
  - a. It generates higher accuracy for all text tasks
  - b. It enables access to powerful language processing without the need for local model training
  - c. It reduces the token count needed for queries
  - d. It enables access to powerful language processing without local model training
2. **What best describes the main role of the transformer architecture in LLMs?**
  - a. It restricts the model to sequential computation for faster inference
  - b. It enables models to capture long-range dependencies in text using self-attention
  - c. It limits the vocabulary size to optimize performance
  - d. It is only used for multimodal architectures
3. **How does FL enhance data privacy in ML?**
  - a. It anonymizes all features before training
  - b. It encrypts the training data and sends it to the cloud
  - c. It trains models locally and shares only model updates with a central server
  - d. It replaces real data with synthetic data training
4. **Which statement accurately describes the function of AutoML tools like TPOT?**
  - a. They automate the deployment of APIs for model serving
  - b. They generate synthetic datasets for testing algorithms
  - c. They automate model selection, training and pipeline optimization

- d. They focus only on deep learning model construction
5. **Why are MoE architectures used in some large models?**
- a. They reduce model size by using only one expert at a time
  - b. They activate specialized subnetworks for each input, improving efficiency and scalability
  - c. They transform all expert outputs into the same layer
  - d. They randomize subnetworks to increase model diversity
6. **What is the main benefit of using responsible AI practices in ML development?**
- a. It improves transparency, fairness and accountability in AI systems
  - b. It guarantees the model will be 100% accurate
  - c. It increases the number of features used in modeling
  - d. It automates hyperparameter optimization for all models
7. **How does prompt engineering influence LLM output?**
- a. It limits the output to only numerical predictions
  - b. It guides the model's responses by structuring input instructions and context
  - c. It allows the model to self-train on new data
  - d. It increases the number of parameters in the LLM
8. **What is the primary purpose of the system prompt when interacting with an LLM?**
- a. To provide additional training data during inference
  - b. To set the temperature of the model effectively
  - c. To set the overall behavior, tone, or context of the model's response
  - d. To increase the length of the maximum token output

9. **In FL, what does the FedAvg step do?**

- a. It averages model parameters from local models to update the global model
- b. It splits data equally across all devices
- c. It deletes local models when training is complete
- d. It removes outliers from the combined datasets

10. **What is the purpose of bias detection in ML pipelines?**

- a. To automatically increase the accuracy score
- b. To reduce the size of the training data
- c. To select the best activation function for deep learning
- d. To identify and address unfair outcomes for specific groups in model predictions

## Answers

1.	d
2.	b
3.	c
4.	c
5.	b
6.	a
7.	b
8.	c
9.	a
10.	d

## Key terms

Some key terms used in this chapter are:

- **Agentic AI:** AI that uses external tools (APIs, web search, etc.) to complete tasks and make decisions.

- **AutoML:** Automation of ML tasks like model selection and tuning, with little human input.
- **Bias mitigation:** Techniques to reduce unfairness or errors in ML models.
- **CTGAN:** A GAN for generating realistic tabular data.
- **Ethical AI:** Designing AI to be fair, transparent, and accountable.
- **Federated Averaging (FedAvg):** Aggregates model updates from multiple sources in federated learning.
- **Federated learning:** Decentralized ML where models are trained locally, sharing only updates.
- **Few-shot reasoning:** LLMs performing tasks with just a few examples.
- **Fine-tuning:** Further training a pre-trained model on a new, specific dataset.
- **Genetic algorithm:** Optimization using evolutionary methods like selection and mutation.
- **Generative AI (GenAI):** AI that creates new content (text, images, etc.) from training data.
- **HuggingFace model hub:** Open platform for sharing pre-trained ML models.
- **LLM:** Large transformer-based models trained on vast text data.
- **MoE:** Network where different experts handle different inputs for efficiency.
- **Open-source model:** Freely available ML models or software, modifiable and shareable.
- **Privacy-preserving techniques:** Methods (like federated learning, anonymization) to protect sensitive data in ML.

- **Prompt:** Input given to an LLM to get a response.
- **Responsible AI:** Building AI with a focus on fairness, transparency, and privacy.
- **Self-attention:** Lets models weigh importance of each token in a sequence.
- **System prompt:** Instruction to set an LLM's role or behaviour.
- **TPOT:** AutoML tool that optimizes ML pipelines using genetic programming.
- **Transformer:** Neural network using self-attention, core to modern LLMs.
- **Trusted AI (TRUSTED framework):** AI governance emphasizing risk management, transparency, and ethics.
- **Zero-shot learning:** AI's ability to handle new tasks without direct training.

## Join our Discord space

Join our Discord workspace for latest updates, offers, tech happenings around the world, new releases, and sessions with the authors:

<https://discord.bpbonline.com>



# Index

## A

---

Activation Functions [300](#)

Activation Functions, terms [301](#)

Agglomerative Clustering [131](#)

Agglomerative Clustering,  
architecture [131](#)

Agglomerative Clustering,  
integrating [132](#), [133](#)

Agglomerative Clustering,  
steps [132](#)

Arrays [2](#)

Arrays Statistical, functions

Arithmetic Mean [6](#)

Median [7](#)

Min/Max [8](#)

Standard Deviation [7](#)

Variance [7](#)

Arrays, types

One-Dimensional [3](#)

Two-Dimensional [3](#)

Autocorrelation Function  
(ACF) [228](#)

AutoML [362](#)

AutoML, implementing [363](#), [364](#)

AutoML, steps [362](#)

---

## B

---

Bagging [266](#)

Bagging, configuring [266](#)

Bagging, steps [266](#), [267](#)

Bayesian Optimization [255](#)

Bayesian Optimization,  
    illustrating [256-258](#)

Bayesian Optimization, steps [255](#), [256](#)

Binary Classification [64](#)

Binary Classification, ways

    Logistic Regression [65](#)

    Logit Function [64](#)

Boilerplate Code [293](#)

Boilerplate Code,  
    illustrating [294](#), [295](#)

Boosting [267](#)

Boosting, steps [268](#)

---

## C

---

Categorical Data [280](#)

Categorical Data, steps [280](#)

Class Distribution [105](#)

Class Rebalancing [112](#)

Class Rebalancing, steps [112](#), [113](#)

Clustering Evaluation [137](#)

Clustering Evaluation, techniques

- Elbow [137](#)
- Silhouette Plot [139](#)
- Silhouette Score [139](#)
- Code Generation [355](#)
- Code Generation,
  - implementing [355-357](#)
- ColumnTransformer [109](#)
- ColumnTransformer, steps [109](#), [110](#)
- Combine Multiple Models [285](#)
- Combine Multiple Models,
  - configuring [285](#)
- Combine Multiple Models,
  - steps [285](#), [286](#)
- Condition Based Indexing [5](#)
- Confusion Matrix [83](#)
- Confusion Matrix, ensuring [83](#), [84](#)
- Customer Churn [102](#)
- Customer Churn Dataset [278](#)
- Customer Churn Dataset, steps [279](#)
- Customer Churn, use cases
  - Class Distribution [105](#)
  - Data Importing [102](#)
  - Data Partitioning [106](#), [107](#)
  - Feature Distribution
    - Visualization [103](#)
  - Miss Value Analysis [104](#)
- Customer Segmentation [141](#)
- Customer Segmentation, terms



- Clean Dataset [141](#)
- Cluster Performance
  - Visualization [144](#)
- Data Partitioning [142](#)
- Feature Scaling [143](#)
- K-Means Cluster [145](#)
- K-Means Model [145](#)
- One-Hot Encoding [142](#)
- Optimal Training Cluster [147](#)
- PCA [148](#)
- Test Dataset [149](#)

## D

---

- DataFrames [21](#)
- Data Importing [102](#)
- Data Partitioning [106](#)
- Data Preparation [303](#)
- Data Preparation, steps [303](#)
- DBSCAN [134](#)
- DBSCAN, implementing [134](#), [135](#)
- DBSCAN, integrating [135-137](#)
- Decision Boundaries [99](#)
- Decision Boundaries, configuring [99](#)
- Decision Boundaries, steps [99](#), [100](#)
- Differencing [225](#)
- Differencing, concepts
  - ADF Testing Process [227](#)
  - Log Transformation [226](#)

- Machine Learning Models [229](#)
- Partial Autocorrelation [227](#)
- Differencing, configuring [225](#), [226](#)
- Dimensionality Reduction [186](#)
- Dimensionality Reduction,  
    applications [201](#)
- Dimensionality Reduction,  
    configuring [198](#)
- Dimensionality Reduction,  
    steps [199](#), [200](#)
- Docker [343](#)
- Docker, components
  - Container [343](#)
  - Images [343](#)
  - Network [344](#)
  - Volume [343](#)
- Docker, configuring [343](#)
- Docker Container [346](#)
- Docker Container, configuring [347](#)
- Docker Image [344](#)
- Docker Image,  
    implementing [345](#), [346](#)
- Drop Columns [73](#)

## **E**

---

- EDA, terms
  - Class Distribution [71](#)
  - Feature Correlation [70](#)

Histogram [67](#), [68](#)

Missing Values [69](#)

Miss Value [72](#)

Elbow [137](#)

Elbow, ensuring [137](#), [138](#)

Ensemble Learning [265](#)

Ensemble Learning, concepts

Bagging [266](#)

Boosting [267](#)

Stacking [268](#)

Ensemble Learning, steps [270](#), [271](#)

Evaluation Loop [88](#)

Evaluation Loop, illustrating [88](#), [89](#)

Evaluation Metrics [51](#), [82](#)

Evaluation Metrics, characteristics

Accuracy [52](#)

DataFrame [52](#)

F1 Score [52](#)

Negative Predictive Value  
(NPV) [52](#)

Recall/Precision [51](#)

Evaluation Metrics,  
configuring [82](#)

Exploratory Data Analysis  
(EDA) [67](#)

---

## F

Fancy Indexing [5](#)

- FastAPI [326](#)
- FastAPI, architecture [326](#)
- FastAPI Endpoint [335](#)
- FastAPI Endpoint, implementing [340](#)
- FastAPI Endpoint, steps [335-337](#)
- FastAPI, features [326](#)
- Feature Distribution
  - Visualization [103](#)
- Feature Elimination [264](#)
- Feature Elimination,
  - configuring [264](#), [265](#)
- Feature Encoding [73](#)
- Feature Encoding, sections
  - Drop Columns [73](#)
  - Feature Selection [78](#)
  - MICE [77](#)
  - One-Hot Encoding [75](#)
  - Perform Normalization [78](#)
  - Split Data [74](#)
- Feature Scaling [143](#)
- Feature Selection [78](#), [159](#)
- Feature Selection, techniques
  - Recursive Feature Elimination (RFE) [159](#)
  - RFECV [161](#)
- FeatureUnion [283](#)
- FeatureUnion, steps [283](#), [284](#)

- Federated Learning (FL) [364](#)
- Flask [326](#)
- Flask Endpoint [327](#)
- Flask Endpoint, illustrating [331-333](#)
- Flask Endpoint, implementing [328-330](#)
- FL, configuring [365](#)
- FL, steps [365](#), [366](#)

---

## G

---

- Gradient Boosting [166](#)
- Gradient Boosting, steps [166](#)
- Grid Search [251](#)
- Grid Search, steps [252](#), [253](#)

---

## H

---

- Holts-Winters Exponential [233](#)
- Holts-Winters Exponential, components [234](#)
- Holts-Winters Exponential, implementing [234](#)
- House Price Prediction [170](#)
- House Price Prediction, concepts
  - Inspect Data [170](#)
  - Missing Values Analyzing [172](#)
  - Modeling Steps [173](#)
  - Preprocessing Function [172](#)

- Regression Model [175](#)
- Shapley Additive Explanation (SHAP) [176](#)
- Test Dataset [179](#)
- Training Loop [174](#)
- Visualize Relationships [170](#)
- HuggingFace [360](#)
- HuggingFace, steps [360](#), [361](#)
- HuggingFace, tasks [360](#)
- Hyperparameter Tuning [250](#)
- Hyperparameter Tuning, components
  - Ensemble Model [289](#)
  - Random Combinations [290](#)
  - Stacked Classifier [292](#)
- Hyperparameter Tuning, configuring [250](#), [251](#)
- Hyperparameter Tuning, evaluating [259](#)
- Hyperparameter Tuning, strategies
  - Bayesian Optimization [255](#)
  - Grid Search [251](#)
  - Random Search [253](#)

## I

---

- Indexing/Slicing Arrays [3](#)
- Indexing/Slicing Arrays, types

One-Dimensional [3](#)

Two-Dimensional [4](#)

---

## K

---

K-Fold Cross Validation [54](#)

K-Fold Cross Validation,  
evaluating [58](#)

K-Fold Cross Validation,  
implementing [55](#), [56](#)

K-Fold Cross Validation,  
steps [55](#)

K-Fold Cross Validation,  
visualizing [56](#), [57](#)

K-Means [128](#)

K-Means, architecture [128](#)

K-Means, integrating [130](#), [131](#)

K-Means, process [128](#)

---

## L

---

Lagged Features [237](#)

Lambda Function [22](#)

Large Dataset [26](#)

Large Dataset, terms

    Categorical Columns [28](#)

    Downcasting Columns [27](#)

    Faker Dataset [26](#)

    Memory Errors [26](#)

    Parallel Processing [28](#)

    Performance Operations [28](#)

PyArrow [29](#)

Sparse Data Types [28](#)

Specify Data Types [27](#)

SQL-Like Queries [29](#), [30](#)

Subsetting [27](#)

Vectorized Operations [29](#)

Large Language Models  
(LLMs) [353](#), [354](#)

LDA, architecture [260](#)

LDA, methods

Baseline Model [261](#)

Chi-Square Test [263](#)

Recursive Feature Elimination (REF) [262](#)

LDA, steps [260](#), [261](#)

Linear Discriminant Analysis  
(LDA) [260](#)

LLMs, configuring [354](#)

LLMs, section

Code Generation [355](#)

HuggingFace [360](#)

OpenAI API [355](#)

Summarization [357](#)

Log Loss [101](#)

Log Loss, classification

Binary [101](#)

Multi-Class [102](#)



## M

---

MICE [77](#)

MICE, architecture [77](#)

MICE, steps [77](#)

Missing Data [20](#)

Missing Data, architecture [20](#)

Missing Data, terms

- Dropping [21](#)

- Imputing [21](#)

- Merging [21](#)

ML, frameworks

- AI Governance [368](#)

- Bias Mitigating [367](#)

- Responsible Pipeline [369](#)

- Scikit-Learn Pipeline [368](#)

MLPRegressor [304](#)

MLPRegressor, steps [304](#)

MLP, steps [301](#), [302](#)

MNIST Dataset [201](#)

MNIST Dataset, methods

- Confusion Matrix [209](#), [210](#)

- Optimal Components [206](#)

- PCA [204](#)

- Train SVM Classifier [209](#)

- t-SNE [205](#)

- UMAP [205](#)

MNIST Dataset, terms

Data Shuffle [201](#)

Splitting [203](#)

Two-Dimensional Plotting [203](#)

Visualizing [202](#)

Model Improvement [271](#)

Model Improvement, terms

Class Imbalance [272](#)

Data Quality/Consistency [271](#)

Data Volume [272](#)

External Factors [272](#)

Feature Selection [271](#)

Model Training [119](#)

Model Training,  
    illustrating [119](#), [120](#)

Multi-Class Classification [98](#)

Multi-Class Classification,  
    configuring [98](#)

Multi-Class Classification, section

Classifier [116](#)

Macro F1 Score [115](#)

Macro Precision [114](#)

Macro Recall [115](#)

Precision Per Class [115](#)

Recall Per Class [115](#)

Multi-Class Classification,  
    steps [113](#), [114](#)

Multi-Layer Perceptron (MLP) [301](#)

Multiple Linear Regression [158](#)

Multiple Linear Regression,  
configuring [158](#), [159](#)

---

## N

---

Neural Networks [300](#)

Neural Networks, layer

Hidden [300](#)

Input [300](#)

Output [300](#)

Neural Networks, tools

Data Preparation [303](#)

Hyperparameter Tuning [305](#)

MLPRegressor [304](#)

Regression Performance [306](#)

Non-Linear Regression [164](#)

Non-Linear Regression, methods

Gradient Boosting [166](#)

Random Forests [164](#)

Numerical Imputation [107](#)

Numerical Imputation, section

Class Rebalancing [112](#)

ColumnTransformer [109](#)

Miss Value Imputation [107](#)

Modal Replacement [108](#)

One-Hot Encoding [110](#)

- Numerical Pipeline [279](#)
- Numerical Pipeline, steps [279](#)
- NumPy [2](#)
- NumPy, architecture [8](#)
- NumPy Arrays [5](#)
- NumPy Arrays, steps [6](#)
- NumPy, concepts
  - Broadcasting [9](#)
  - Concatenation [10](#)
  - Correlation Matrix [14](#)
  - Covariance [13](#)
  - Horizontal Stacking [11](#)
  - House Dataset [9](#)
  - Matrix Multiplication [11](#)
  - Normalization [12](#)
  - Reshaping Arrays [10](#)
  - Singular Value Decomposition (SVD) [12](#)
- NumPy, steps [2](#)

## O

---

- OLS, configuring [156](#), [157](#)
- One-Hot Encoding [75](#), [110](#)
- One-Hot Encoding, steps [75](#), [76](#), [111](#)
- One-vs-One [117](#)
- One-vs-Rest [116](#), [117](#)
- OpenAI API [355](#)
- Ordinary Least Squares (OLS) [156](#)
- OutputCodeClassifier [117](#)

Over/Under Sampling [85](#)

## P

---

PACF, illustrating [228](#), [229](#)

Pandas [15](#)

Pandas, concepts

- Data Inspection [16](#)

- Palmer Penguins Dataset [16](#)

Pandas, condition

- AND [19](#)

- iloc [19](#)

- IoC [18](#)

- Multiple Column [20](#)

- NOT [19](#)

- Numpy Where [19](#)

- OR [19](#)

- Single Column [20](#)

Pandas Data, types

- DataFrame [17](#)

- Multiple Columns [18](#)

- Single Column [18](#)

- Summary Statistics [17](#)

Pandas, sources

- DataFrames [15](#)

- Series [15](#)

Partial Autocorrelation

- Function (PACF) [228](#)

- PCA, configuring [186](#)
- PCA, sections
  - Covariance Metrix [187](#)
  - Dot Product [189](#)
  - Higher-Dimensional Spaces [188](#)
  - Standardize Data [187](#)
  - Vector Computing [188](#)
- PCA, steps [186](#)
- Perform Normalization [78](#)
- Principal Component
  - Analysis (PCA) [186](#)
- Prophet [237](#)
- Prophet, terms
  - Data Modeling [238](#)
  - Data Processing [238](#)
- PyTorch [309](#)
- PyTorch, architecture [310](#)
- PyTorch, reasons [309](#)
- PyTorch, terms
  - Custom Function [316](#)
  - DataLoader [311](#)
  - Loss Function [310](#)
  - Native Tensor Format [311](#)
  - Neural Networks [312](#)
  - Training Loop [314](#)

- Random Forests [164](#)
- Random Forests, integrating [165](#)
- Random Forests, steps [164](#)
- Random Search [253](#)
- Random Search, configuring [253](#)
- Random Search, steps [254](#), [255](#)
- Recursive Feature
  - Elimination (RFE) [159](#)
- Regression [163](#)
- Regression Analysis [156](#)
- Regression Evaluation [167](#)
- Regression Evaluation, metrics
  - Adjusted R-Squared [169](#)
  - Mean Absolute Error (MAE) [167](#)
  - Mean Absolute Percentage Error (MAPE) [169](#)
  - Mean Squared Error (MSE) [168](#)
  - Mean Squared Logarithmic Error (MSLE) [168](#)
- Metrics Function [169](#)
- Root MSE [168](#)
- R-Squared [168](#)
- Regression, methods [163](#)
- Regression Performance [306](#)
- Regression Performance, tools
  - Predicted Values [307](#)
  - Quantile-Quantile Plot [308](#)
  - Residuals Plot [307](#)

Regression, section

Elastic Net [164](#)

Lasso [163](#)

Ridge [163](#)

Responsible Pipeline [369](#)

Responsible Pipeline, sections

Audit Trails [375](#)

Census Data [369](#)

Cross-Validation [370](#)

Flag Uncertain Predictions [374](#)

Miss Value Imputation [370](#)

Sub-Group Performance [372](#)

RESTful endpoint [327](#)

RESTful endpoint,  
    configuring [327](#)

RESTful endpoint, section

FastAPI Endpoint [335](#)

Flask Endpoint [327](#)

RFECV [161](#)

RFECV, steps [162](#)

RFE, steps [160](#), [161](#)

ROC Curve [90](#)

ROC Curve, configuring [90](#), [91](#)

## S

---

SARIMA [235](#)

SARIMA, ensuring [235](#), [236](#)



SARIMA, steps [236](#)

Scikit-Learn [36](#)

Scikit-Learn, algorithms

- Classification [38](#)

- Regression [37](#)

- Unsupervised [38](#)

Scikit-Learn, concepts

- Estimators [40](#)

- Mean Square Error (MSE) [40](#)

- Pipelines [40](#)

- Transformers [40](#)

Scikit-Learn, features [36](#)

Scikit-Learn, options

- Cross-Validation [39](#)

- Hyperparameter Tuning [39](#)

Scikit-Learn Pipeline [278](#)

Scikit-Learn Pipeline, section

- Categorical Data [280](#)

- Combine Pipeline [283](#)

- Customer Churn Dataset [278](#)

- Custom Transformer [280](#)

- Numerical Pipeline [279](#)

Scikit-Learn, points

- Class Imbalance [44](#)

- Evaluation Metrics [51](#)

- K-Fold Cross Validation [54](#)

- Miss Values, handling [45](#)
- ML Datasets [43](#)
- Modeling [48](#)
- Model Selection [53](#)
- Numerical Scaling [46](#)
- Separate Variables [47](#)
- Test Partitions [47](#)
- Scikit-Learn, roles
  - Data Collection [37](#)
  - Modeling/Training [37](#)
- Scikit-Learn, sections
  - Classification Dataset [41](#)
  - Regression Dataset [40](#)
  - Unsupervised Dataset [42](#)
- Scikit-Learn, use cases [36](#)
- Shapley Additive Explanations (SHAP) [176](#)
- SHAP, steps
  - Global Feature [176](#)
  - Local Feature [177](#)
- Silhouette Plot [139](#), [140](#)
- Silhouette Score [139](#)
- Singular Value Decomposition (SVD) [12](#), [13](#)
- Split Data [74](#)
- Stacking [268](#)
- Stacking, configuring [268](#)

- Stacking, steps [269](#)
- Stratified K-Fold [81](#)
- Stratified K-Fold, steps [81](#)
- Summarization [357](#)
- Summarization, implementing [359](#)
- Summarization, steps [358](#)
- SVD, matrices [13](#)

---

## T

---

- TensorFlow [318](#)
- TensorFlow Keras [319](#)
- TensorFlow Keras, configuring [319](#)
- TensorFlow Keras,
  - implementing [320](#), [321](#)
- TensorFlow Keras, steps [319](#)
- Thyroid Disease Prediction [66](#)
- Time Series Data [22](#)
- Time Series Data, architecture [216](#)
- Time Series Data,
  - configuring [22](#), [23](#)
- Time Series Data, features
  - Preprocessed Dataset [222](#)
  - Stationary Series [224](#)
  - Time Series Decomposition [223](#)
- Time Series Data, methods
  - Hold-Out Split [230](#)
  - Time Series Cross-Validation (TSCV) [231](#)
- Time Series Data, models

Data Loaded [233](#)

Holts-Winter Exponential  
Smoothing [233](#)

Lagged Features [237](#)

Prophet [237](#)

SARIMA [235](#)

Time Series Data, points

Compute Metrics [239](#)

Forecast Horizon [242](#)

Mean Absolute Error [240](#)

Time Series Data, section

Backward Fill [219](#)

Forward Fill [219](#)

Generate Missing Values [219](#)

Kalman Filter [220](#)

Post Cleaning [218](#)

Resampling [221](#)

Time Series Data, steps

Date Wrangling [217](#)

Loading [216](#)

Time Series Data, term

Data Range [23](#)

Missing Data [24](#)

Resampling [23](#)

Rolling Move Average [25](#)

Shifting [24](#)

Trade-Offs, models

CatBoost Classifier [121](#)

Logistic Regression [121](#)

Random Forest Classifier [121](#)

Support Vector Machine [121](#)

Training Loop, breakdown [86](#), [87](#)

Training Loop, steps [86](#)

t-SNE [190](#)

t-SNE, section

Cost Function [194](#)

High Dimensional Space [190](#)

Kullback-Leibler Divergence [193](#)

Student t-Distribution [191](#)

---

## U

UMAP [196](#)

UMAP, levels

Dimensionality Reduction [198](#)

Graph-Based Representation [197](#)

High-Dimensional Space [196](#)

Unsupervised Learning [128](#)

---

## X

XGBoost, bullets [167](#)

**Join our Discord space**

Join our Discord workspace for latest updates, offers, tech happenings around the world, new releases, and sessions with the authors:

<https://discord.bpbonline.com>

